

Litmus Text Command Reference

Product Version 26.10

May 2026

© 2026 Cadence Design Systems, Inc.
Printed in the United States of America.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Open SystemC, Open SystemC Initiative, OSCI, SystemC, and SystemC Initiative are trademarks or registered trademarks of Open SystemC Initiative, Inc. in the United States and other countries and are used with permission.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. (Cadence) contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence's trademarks, contact the corporate legal department at the address shown above or call 800.862.4522.

All other trademarks are the property of their respective holders.

Restricted Permission: This publication is protected by copyright law and international treaties and contains trade secrets and proprietary information owned by Cadence. Unauthorized reproduction or distribution of this publication, or any portion of it, may result in civil and criminal penalties. Except as expressly permitted below, this publication may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, including automated processes, training or services involving a large language model, foundation model, deep machine learning, generative artificial intelligence, or any other similar process or technology, without prior written permission from Cadence. Unless otherwise agreed to by Cadence in writing, customers are granted permission to print one (1) hard copy of this publication subject to the following conditions:

1. The publication may be used only in accordance with a written agreement between Cadence and its customer.
2. The publication may not be modified in any way.
3. Any authorized copy of the publication or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement.
4. The information contained in this document cannot be used in the development of like products or software, whether for internal or external use, and shall not be used for the benefit of any other party, whether or not for consideration.

Disclaimer: Information in this publication is subject to change without notice and does not represent a commitment on the part of Cadence. Except as may be explicitly set forth in such agreement, Cadence does not make, and expressly disclaims, any representations or warranties as to the completeness, accuracy or usefulness of the information contained in this document. Cadence does not warrant that use of such information will not infringe any third party rights, nor does Cadence assume any liability for damages or costs of any kind that may result from use of such information. Cadence is committed to using respectful language in our code and communications. We are also active in the removal and replacement of inappropriate language from existing content. This product documentation may however contain material that is no longer considered appropriate but still reflects long-standing industry terminology. Such content will be addressed at a time when the related software can be updated without end-user impact.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227-14 and DFAR252.227-7013 et seq. or its successor.

Contents

About This Manual	4
Audience	4
Conventions Used in This Manual	4
1	7
Advanced Timing Tcl Scripting Commands	7
all_* Commands	7
Examples	8
get_* Commands	8
Examples	8
property Commands	9
Examples	9
collection Commands	9
Examples	9
add_to_collection	10
all_clocks	11
all_connected	12
all_fanin	13
all_fanout	15
all_inputs	17
all_instances	18
all_outputs	19
all_registers	20
append_to_collection	23
compare_collections	25
copy_collection	26
define_proc_arguments	27
define_property	29
filter_collection	30
foreach_in_collection	32
get_arcs	34
get_cells	35
get_clocks	37

get_designs	38
get_generated_clocks	39
get_lib_arcs	40
get_lib_cells	41
get_lib_pg_pins	43
get_lib_pins	44
get_libs	46
get_nets	47
get_object_name	48
get_path_groups	49
get_pg_nets	50
get_pg_pins	51
get_pins	52
get_ports	54
get_property	55
index_collection	150
is_collection_object	151
parse_proc_arguments	152
query_objects	154
range_collection	155
remove_from_collection	157
report_object_property_names	158
report_property	159
sizeof_collection	159
sort_collection	161
2	163
Basic Database Access Tcl Commands	163
add_thru_substrate_insts	163
convert_dbu_to_um	166
convert_um_to_dbu	168
decompress_special	170
init_hier_cell	172
uniquify	173
3	178
Constraint Promotion Commands	178
set_constraint_promotion_priority	178

set_constraint_promotion_strategy	179
promote_hierarchical_constraints	182
pushdown_hierarchical_constraints	184
4	186
Database Access and Handling Commands	186
define_attribute	187
delete_obj	192
get_db	194
is_attribute	213
set_db	215
reset_db	219
sort_db	222
5	226
Delay Calculation Commands	226
get_aging_analysis_mode	226
6	229
Design Initialization Commands	229
configure_lint_check	230
init_design	237
input_design	240
read_hdl	241
read_libs	245
read_lint_configuration	247
read_mmmc	249
read_netlist	252
read_sdc	254
report_black_box	255
report_lint_configuration	257
report_sdc	259
write_lint_configuration	261
7	264
Flowkit Commands	264
check_flow	264
create_flow	267
create_flow_step	269

delete_flow_config	271
edit_flow	272
get_flow_config	275
is_feature	276
is_flow	277
read_flow	278
schedule_flow	279
set_flow_config	281
set_flowkit_read_db_args	282
set_flowkit_write_db_args	285
write_flow	290
8	293
Functional Check Commands	293
set_sva_to_skip	294
set_sva_to_generate	295
generate_sva	296
9	299
General Commands	299
add_rule_filter	301
alias	303
check_ccr	305
check_netlist	306
convert_legacy_to_common_ui	309
create_snapshot	309
decrypt	312
define_proc	313
delete_rule_filter	320
elaborate_design	321
encrypt	323
eval_legacy	324
get_license_status	325
get_message	326
help	327
input_libs	328
input_sdc	328
is_common_ui_mode	329

litmus_setup_mmmc	330
print_message	330
read_db	333
read_metric_config	337
read_metric_html_package	338
redirect	339
rename_obj	342
rename_snapshot	345
report_dangling_ports	347
report_inactive_arcs	348
report_messages	353
report_netlist_statistics	356
report_obj	357
report_rule_check	360
report_rule_filter	364
report_rule_instance	365
report_rule_source	366
resume	367
set_compress_level	368
set_expanded_hinsts	369
set_license_check	370
set_message	372
set_rule_handling	375
Options and Arguments	375
source	375
suspend	377
time_info	378
write_db	380
write_metric_config	386
10	389
GUI Commands	389
gui_get_schematic	390
gui_hide	391
gui_schematic_add_selected	392
gui_show	393
read_power_rail_results	394

11	403
Import and Export Commands	403
read_def	403
read_physical	408
relink_db_files	412
write_mmmc	414
12	416
Miscellaneous	416
analyze_paths_by_bottleneck	416
check_timing_model	419
init_hier_design	428
13	430
Multiple-CPU Processing Commands	430
report_resource	431
set_multi_cpu_usage	435
14	438
System Commands - Basic Tool-Tcl Commands	438
man	439
15	440
Timing Analysis Commands	440
audit_timing	442
check_multi_cpu_usage	445
check_timing	448
check_timing_library	462
check_timing_library_consistency	465
create_analysis_view	466
create_constraint_mode	468
create_delay_corner	470
create_library_set	475
create_opcond	479
create_property_alias	481
create_rc_corner	482
create_timing_condition	487
delete_analysis_view	489

get_capacitance_unit	490
get_clock_network_objects	491
get_clock_to_clock_association	492
get_interactive_constraint_modes	495
get_multi_cpu_usage	496
get_propagated_clock	498
get_resistance_unit	499
get_selectively_expanded_modules	500
get_time_unit	501
list_libraries	502
read_boundary_model	504
read_ilm	507
report_analysis_coverage	508
report_analysis_summary	518
report_analysis_views	523
report_annotated_assertions	528
report_annotated_check	532
report_annotated_delay	539
report_case_analysis	543
report_cell_instance_timing	549
report_clock_gating_check	554
report_clock_propagation	557
report_clock_sense	560
report_clock_timing	561
report_clocks	579
report_constraint	589
report_design	612
report_fanin	615
report_fanout	619
report_instance_library	623
report_lib_arcs	625
report_mode	630
report_nets	632
report_path_exceptions	637
report_path_groups	641
report_ports	643
report_reference	648

report_timing	650
report_timing_derate	709
report_wire_load	715
reset_path_adjust	717
reset_path_adjust_group	718
reset_timing_derate	719
reset_wire_load_mode	721
reset_wire_load_model	722
reset_wire_load_selection_group	723
set_analysis_view	724
set_clock_exclusivity	727
set_default_view	729
set_interactive_constraint_modes	731
set_path_adjust	733
set_path_adjust_group	735
set_selectively_expanded_modules	738
set_table_style	744
set_timing_context_clock_mapping	751
set_timing_derate	753
set_wire_load_mode	764
set_wire_load_model	767
set_wire_load_selection_group	769
update_analysis_view	771
update_constraint_mode	773
update_delay_corner	775
update_library_set	780
update_opcond	782
update_rc_corner	783
update_timing	787
update_timing_condition	788
write_sdc	789
16	793
Timing Constraint Commands	793
Path Exception Priorities	793
Bidirectional Pin Defaults	794
compare_constraints	798

create_clock	799
create_generated_clock	802
current_design	807
current_instance	808
group_path	809
report_clock_mapping	811
reset_annotated_check	812
reset_annotated_delay	815
reset_annotated_transition	816
reset_case_analysis	818
reset_clock	819
reset_clock_exclusivity	821
reset_clock_gating_check	822
reset_clock_groups	823
reset_clock_latency	825
reset_clock_sense	828
reset_clock_transition	829
reset_clock_tree_latency	832
reset_clock_uncertainty	833
reset_data_check	836
reset_disable_clock_gating_check	838
reset_disable_timing	839
reset_drive	842
reset_driving_cell	843
reset_generated_clock	844
reset_ideal_latency	846
reset_ideal_network	847
reset_ideal_transition	848
reset_input_delay	849
reset_load	851
reset_max_capacitance	852
reset_max_fanout	853
reset_max_time_borrow	854
reset_max_transition	855
reset_min_capacitance	856
reset_min_fanout	857
reset_min_pulse_width	858

reset_min_transition	859
reset_mode	860
reset_output_delay	862
reset_path_exception	866
reset_path_group	874
reset_pll_timing	875
reset_propagated_clock	875
reset_property	878
reset_pulse_clock_max_transition	879
reset_pulse_clock_max_width	880
reset_pulse_clock_min_transition	881
reset_pulse_clock_min_width	882
reset_resistance	883
reset_sense	884
set_active_clocks	885
set_annotated_check	886
set_annotated_delay	889
set_annotated_transition	892
set_case_analysis	894
set_clock_gating_check	896
set_clock_groups	899
set_clock_latency	903
set_clock_sense	907
set_clock_transition	910
set_clock_uncertainty	911
set_data_check	914
set_disable_clock_gating_check	917
set_disable_timing	918
set_dont_touch	920
set_dont_touch_network	921
set_dont_use	922
set_drive	923
set_driving_cell	925
set_false_path	928
set_fanout_load	933
set_ideal_latency	934
set_ideal_network	936

set_ideal_transition	938
set_input_delay	940
set_input_transition	944
set_load	946
set_logic_one	949
set_logic_zero	950
set_max_capacitance	951
set_max_delay	953
set_max_fanout	958
set_max_time_borrow	960
set_max_transition	961
set_min_capacitance	964
set_min_delay	967
set_min_fanout	973
set_min_pulse_width	975
set_min_transition	978
set_mode	981
set_multicycle_path	983
set_output_delay	988
set_pll_timing	991
set_propagated_clock	994
set_property	996
set_pulse_clock_max_transition	997
set_pulse_clock_max_width	999
set_pulse_clock_min_transition	1001
set_pulse_clock_min_width	1003
set_resistance	1005
set_sense	1007
set_target_check_constraints	1010
17	1012
Timing Constraint Hierarchical Signoff Commands	1012
report_hierarchical_clock_mapping	1012
delete_hierarchical_clock_mapping	1014
check_target_constraints	1015
read_hierarchical_constraints	1016
18	1019

Timing Constraint Signoff Commands	1019
verify_timing_intent	1019

About This Manual

This manual provides a reference of the commands available to the user when using the Litmus software with the Stylus Common user interface.

Audience

This manual is written for experienced designers of digital integrated circuits. Such designers must be familiar with design planning, placement and routing, block implementation, chip assembly, and design verification. Designers must also have a solid understanding of UNIX and Tcl/Tk programming.

Conventions Used in This Manual

This section describes the typographic and syntax conventions used in this manual.

<code>text</code>	Indicates text that you must type exactly as shown. For example: <code>analyze_connectivity -analyze all</code>
<code>text</code>	Indicates information for which you must substitute a name or value. In the following example, you must substitute the name of a specific file for <code>configfile</code> : <code>wroute filename configfile</code>
<code>text</code>	Indicates the following: • Text found in the graphical user interface (GUI), including form names, button labels, and field names • Terms that are new to the manual, are the subject of discussion, or need special emphasis • Titles of manuals
<code>[]</code>	Indicates optional arguments. In the following example, you can specify none, one, or both of the bracketed arguments: <code>command [-arg1] [arg2 value]</code>

[]	Indicates an optional choice from a mutually exclusive list. In the following example, you can specify any of the arguments or none of the arguments, but you cannot specify more than one: <pre>command [arg1 arg2 arg3 arg4]</pre>
{ }	Indicates a required choice from a mutually exclusive list. In the following example, you must specify one, and only one, of the arguments: <pre>command {arg1 arg2 arg3}</pre>
{ [] [] }	Indicates a required choice of one or more items in a list. In the following example, you must choose one argument from the list, but you can choose more than one: <pre>command {[arg1] [arg2] [arg3]}</pre>
{ }	Indicates curly braces that must be entered with the command syntax. In the following example, you must type the curly braces: <pre>command arg1 {x y}</pre>
...	Indicates that you can repeat the previous argument.
.	Indicates an omission in an example of computer output or input.
<i>Command - Subcommand</i>	Indicates a command sequence, which shows the order in which you choose commands and subcommands from the GUI menu. In the following example, you choose <i>Power</i> from the menu, then <i>Power Planning</i> from the submenu, and then <i>Add Rings</i> from the displayed list: <i>Power - Power Planning - Add Rings</i> This sequence opens the <i>Add Rings</i> form..

Advanced Timing Tcl Scripting Commands

The following advanced timing tcl scripting commands are available in Litmus:

- [add_to_collection](#)
- [all_clocks](#)
- [all_connected](#)
- [all_fanin](#)
- [all_fanout](#)
- [all_inputs](#)
- [all_instances](#)
- [all_outputs](#)
- [all_registers](#)
- [append_to_collection](#)
- [compare_collections](#)
- [copy_collection](#)
- [define_proc_arguments](#)
- [define_property](#)
- [filter_collection](#)
- [foreach_in_collection](#)
- [get_arcs](#)
- [get_cells](#)
- [get_clocks](#)
- [get_designs](#)
- [get_generated_clocks](#)
- [get_lib_arcs](#)
- [get_lib_cells](#)
- [get_lib_pg_pins](#)
- [get_lib_pins](#)
- [get_libs](#)
- [get_nets](#)
- [get_object_name](#)
- [get_path_groups](#)
- [get_pg_nets](#)
- [get_pg_pins](#)
- [get_pins](#)
- [get_ports](#)
- [get_property](#)
- [index_collection](#)
- [is_collection_object](#)
- [parse_proc_arguments](#)
- [query_objects](#)
- [range_collection](#)
- [remove_from_collection](#)
- [report_object_property_names](#)
- [report_property](#)
- [sizeof_collection](#)
- [sort_collection](#)

The following topics provide detailed information on each category of similar commands:

- [all_* Commands](#)
- [get_* Commands](#)
- [property Commands](#)
- [collection Commands](#)

all_* Commands

The `all_*` commands return a collection of all the items of a specific type. Several of the commands provide options to filter the collection based on certain properties:

- `all_clocks`
- `all_connected`
- `all_fanin`
- `all_fanout`
- `all_inputs`
- `all_instances`
- `all_outputs`
- `all_registers`

Examples

- To set all the clocks to propagated mode, use the following command:

```
set_propagated_clock [all_clocks]
```

- To report the worst timing path that begins at a register and ends at an output port, use the following command:

```
report_timing -from [all_registers] -to [all_outputs]
```

- To store a collection of pins that fanout from a specific port, use the following command:

```
set clk1Fanout [all_fanout -from clk1 -pin_levels 1]
```

get_* Commands

The `get_*` commands return a collection of items filtered by name pattern or object type. Additionally, you can use the `-filter` option for most of the `get_*` commands to filter the collection by property value. See the [get_property](#) command for a list of all the properties.

The `filter_expression` for the `-filter` option can be created by combining several object properties using the following relational and logical operators: `>`, `<`, `==`, `!=`, `<=`, `>=`, `&&`, `||`, `==~`, `!~`, `AND`, and `OR`. You also can use parentheses to create expressions. The `get_*` commands include the following:

- `get_arcs`
- `get_cells`
- `get_clocks`
- `get_designs`
- `get_generated_clocks`
- `get_lib_arcs`
- `get_lib_cells`
- `get_lib_pins`
- `get_libs`
- `get_nets`
- `get_object_name`
- `get_path_groups`
- `get_pins`
- `get_ports`

Examples

- To return a collection of all input ports (same as running `[all_inputs]`), use the following command:

```
get_ports -filter "direction==in"
```

- To return a collection of pins that match `*/D`, use the following command:

```
get_pins -hier -filter "hierarchical_name =~ */D"
```

- To return a collection of the libraries a cell belongs to, use the following command:

```
get_lib_cells INVXL
```

property Commands

The [report_object_property_names](#) command lists the properties for the different object types. This command, without arguments, will list all the properties for all object types. Use the `-type` option to list properties for a specific object type. Legal option types

are: cell, clock, design, lib, lib_cell, lib_pin, lib_timing_arc, net, path_group, pin, port, timing_arc, timing_path, or timing_point.

Examples

- The [get_property](#) command returns the value of the specified property. To report the clock period of clk1, use the following command:

```
get_property [get_clocks clk1] period
```

- The [report_property](#) command returns the properties and their values for a specified object. To report the properties for clock clk1, use the following command:

```
report_property [get_clocks clk1]
```

- To report timing and store the top 10 paths as a collection in the variable `$paths`, use the following command:

```
set paths [report_timing -collection -max_paths 10]
```

- To filter `$paths` to include only those paths with slack between -0.806ns and 0ns, use the following command:

```
set newPaths [filter_collection $paths {(slack > -0.806 && slack < 0.0)}]
```

collection Commands

Once you have the collection defined you can manipulate, filter, cycle through, and query it using the collection related commands:

- `add_to_collection`
- `append_to_collection`
- `compare_collections`
- `copy_collection`
- `filter_collection`
- `foreach_in_collection`
- `index_collection`
- `query_objects`
- `remove_from_collection`
- `sizeof_collection`
- `sort_collection`

Examples

- To list the contents of a collection, use the following command:

```
query_objects $collectionName
```

- To cycle through the items of a collection, use the following command:

```
foreach_in_collection iCell [get_cells *] {  
  puts "Cell = [get_property $iCell hierarchical_name]"  
}
```

add_to_collection

```
add_to_collection  
base_collection  
second_collection_or_list  
[-unique]
```

Creates a new collection by adding objects from one collection to another collection. The following rules apply:

- If all of the objects in the base collection are of the same type, only the objects in the second collection that are the same type as those in the base collection are added to the new collection. If you specify an object list instead, all of the objects in the list must be the same type as those in the base collection.
- If the base collection contains different types of objects, all objects in the second collection are added. In this situation, you cannot specify an object list.

If the software finds no objects to add, an empty collection is returned.

Parameters

<i>base_collection</i>	Specifies the base collection of objects. Objects from the second collection or object list are added to this collection to create the new collection.
<i>second_collection_or_list</i>	Specifies the second object collection, or a list of objects.
<i>-unique</i>	Removes any duplicate objects from the new collection.

Examples

- The following command returns a collection which contains both the output ports as well as the input ports:

```
litmus> add_to_collection [all_outputs] [all_inputs]
```

Output:

```
CLK4 CLK3 CLK2 CLK1 IN4 IN3 IN2 IN1 OUT4 OUT3 OUT2 OUT1 OUT5
```

Here, the input and output ports are as follows:

```
litmus> all_inputs  
CLK4 CLK3 CLK2 CLK1 IN4 IN3 IN2 IN1
```

```
litmus> all_outputs  
OUT4 OUT3 OUT2 OUT1 OUT5
```

all_clocks

```
all_clocks
[-help]
```

Creates a collection of all the clocks in the current design. If no clocks were defined, an empty string "" is returned.

To filter clocks, use the `get_clocks` command to create a collection of clocks matching a specific pattern. These clocks can be assigned to a variable or passed using another command.

Parameters

<code>-help</code>	Prints out the command usage.
--------------------	-------------------------------

Examples

- The following example applies the `set_propagated_clock` command to all the clocks in the design:

```
set_propagated_clock [all_clocks]
```

Related Commands

- `create_clock`
- `get_clocks`
- `set_propagated_clock`
- `all_inputs`

all_connected

```
all_connected
[-help]
[-leaf]
obj
```

Returns a collection of all of the objects connected to the specified net, pin, or port.

Note: If you specify a pin or port object, the `all_connected` command will return the net connected to that pin or port. If you specify a net object, this command will return ports, pins, or hpins connected to that net.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-leaf</code>	Returns all instance pins connected to the specified net. When specified, only the flattened connection points (leaf pins) for the specified net are reported. <i>Default:</i> Returns hierarchical ports connected to the specified net.
<code>obj</code>	Specifies the net, pin, or port for which collection of connected objects is to be reported. You can specify an object name, or a single-object collection.

Examples

- The following command returns a collection of all of the objects connected to the pin `b1_Sin1`:

```
all_connected Sin1
```

The software returns the following information:

```
Sin1
```

- The following command returns a collection of all of the objects connected to a pin contained in the single collection object `i_block1/b1_Sin1`:

```
all_connected [get_pins {i_block1/b1_Sin1}]
```

The software returns the following information:

```
Sin1
```

- The following command returns a collection of all of the hierarchical ports connected to the net `net_1`:

```
all_connected [get_nets net_1]
```

The software returns the following information:

```
TL2/l2_in TL1/l1_out
```

- The following command returns a collection that contains only the flattened connection points for the net `net_1`:

```
all_connected -leaf [get_nets net_1]
```

The software returns the following information:

```
TL2/L1/RX/D TL1/RX/Q
```

all_fanin

```
all_fanin
[-help]
[-hpin]
[-only_cells]
[-startpoints_only]
-to {collection | object_list}
[-trace_through {case_disable | user_disable | all | clocks | loop_snipped | latches}]
[-view view_name]
[-levels value | -pin_levels value]
[ > | >> ]
```

Returns a collection of pins, ports, or cells that exist in the fanin cone of the specified objects.

Note: The `all_fanin` command does not work on netlist traversal but on timing graph traversal.

Parameters

<code>-help</code>	Prints out the command usage.
<code>> >></code>	Writes a list of the returned collection to the specified file. You can add the <code>.gz</code> extension to the file name to generate a compressed report. Note: The file name must be the last parameter specified.
<code>-hpin</code>	Returns the hierarchical pins along with the flat pins.
<code>-levels value</code>	Specifies the number of gate levels to go back from a specific pin object to create the collection.
<code>-only_cells</code>	Returns a collection of cells in the fanin zone.
<code>-pin_levels value</code>	Specifies the number of delay arcs to go back from a specific pin object to create the collection.
<code>-startpoints_only</code>	Returns a collection only of valid startpoints (sequential cells and input ports) in the fanin cone.
<code>-trace_through {case_disable user_disable all clocks loop_snipped latches}</code>	You can specify one or more of the following options: <code>case_disable</code>: Allows traversal of arcs disabled using the <code>set_case_analysis</code> command. The arcs disabled using the <code>set_disable_timing</code> command are not traversed. <code>user_disable</code>: Allows traversal of arcs disabled using the <code>set_disable_timing</code> command, but arcs disabled using <code>set_case_analysis</code> command are not traversed. <code>all</code>: Allows traversal through all the arcs. <code>clocks</code>: Allows traversal through pins for which clock constraints, such as, <code>create_clock</code>, <code>create_generated_clock</code>, are applied. The pins that are disabled using the <code>set_case_analysis</code> and <code>set_disable_timing</code> commands are not traversed. <code>loop_snipped</code>: Allows trace through the loop snipped arcs. <code>latches</code>: Allows traversal of arcs of the latch cells.
<code>-to {collection object_list}</code>	Specifies the objects for which to return the collection. You can specify a list of objects, or a collection of objects.
<code>-view view_name</code>	Specifies the view name for which fan cone traversal is to be performed.

Examples

- The following command returns the collection of cells in the fanin zone of the output port OUT1:
`all_fanin -to OUT1 -only_cells`
- The following command returns all the valid startpoints (sequential elements and/or input ports) in the fanin zone of OUT1, allowing the traversal through arcs disabled by `set_case_analysis` and `set_disable_timing`:
`all_fanin -to OUT1 -startpoints_only -trace_through all`

Related Commands

- [all_fanout](#)

all_fanout

```
all_fanout
[-help]
[-endpoints_only]
[-hpin]
[-only_cells]
[-trace_through {case_disable | user_disable | all | clocks | loop_snipped | latches}]
[-view view_name]
[-levels value | -pin_levels value]
{-from {collection | object_list} }
[ > | >> ]
```

Returns a collection of pins, ports, or cells that exist in the fanout cone of the specified objects.

Note: The `all_fanout` command does not work on netlist traversal but on timing graph traversal.

Parameters

-help	Prints out the command usage.
-endpoints_only	Returns a collection of valid endpoints (sequential elements or output ports) in the fanout cone.
-hpin	Returns hierarchical pins along with the flat pins.
> >>	Writes a list of the returned collection elements to the specified file. You can add the .gz extension to the file name to generate a compressed report. Note: The file name must be the last parameter specified.
-from {collection object_list}	Specifies the objects for which to return the collection. You can specify a list of objects, or a collection of objects.
-levels value	Specifies the number of gate levels to go forward from a specific pin object to create the collection.
-only_cells	Returns a collection only of the cells in the fanout zone.
-pin_levels value	Specifies the number delay arcs to go forward from a specific pin object to create the collection.
-trace_through {case_disable user_disable all clocks loop_snipped latches}	Allows traversal through disabled arcs. You can specify one of the following options: • case_disable: Arcs which are disabled using the <code>set_case_analysis</code> command and through physical constant are traversed. The arcs disabled using <code>set_disable_timing</code> are not traversed. • user_disable: Arcs disabled using <code>set_disable_timing</code> command are traversed but those disabled using <code>set_case_analysis</code> command are not traversed. • all: Enables trace through all the arcs. • clocks: Allows traversal through pins for which clock constraints (such as, <code>create_clock</code>, <code>create_generated_clock</code>) are applied. The pins that are disabled using the <code>set_case_analysis</code> and <code>set_disable_timing</code> commands are not traversed. • loop_snipped: Allows trace through the loop snipped arcs. • latches: Allows traversal of arcs of the latch cells.
-view view_name	Specifies the view name for which fan cone traversal is to be performed.

Examples

- The following command returns the collection of cells in the fanout zone of the input port IN1:
`all_fanout -from IN1 -only_cells`
- The following command returns all the valid endpoints (sequential elements and/or output ports) in the fanout zone of IN1, allowing the traversal through arcs disabled by `set_case_analysis` and `set_disable_timing`:
`all_fanout -from IN1 -endpoints_only -trace_through all`

Related Commands

- `all_fanin`

all_inputs

```
all_inputs
[-clock list_of_clocks]
[-no_clocks]
[-edge_triggered]
[-level_sensitive]
```

Returns a collection of input or inout ports in the current design.

To filter inputs, use the `get_ports` command to create a collection of ports matching certain criteria. You can assign these ports to a variable or pass them using another command. You can limit the contents of the collection to ports with input delay relative to a specific clock.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-clock</code> <code>list_of_clocks</code>	Considers or limits the search to ports that have input delay relative to a specified list of clocks.
<code>-edge_triggered</code>	Returns a collection of input ports (specified with the <code>set_input_delay -clock</code> constraint), that have edge-triggered input delay.
<code>-level_sensitive</code>	Returns a collection of input ports (specified with the <code>set_input_delay -level_sensitive</code> constraint), that have level-sensitive input delay.
<code>-no_clocks</code>	Returns primary input ports that do not have <code>create_clock</code> or <code>create_generated_clock</code> constraints.

Examples

- The following command returns the names all of the input ports with input delay relative to an `ideal_clk`:

```
all_inputs -clock ideal_clk
```
- The following example lists all input ports in the current design:

```
tempus > all_inputs
```
- The following example sets the drive value of all the input ports on the current design to 10:

```
tempus > set_drive 10.0 [all_inputs]
```
- The following example marks with a multicycle value of 0 to the all paths from inputs having level-sensitive input delay relative to PHI1 to level-sensitive registers clocked by CDNSCLK:

```
tempus > set_multicycle_path 0 \  
-from [all_inputs -clock CDNSCLK -level_sensitive] \  
-to [all_registers -data_pins -clock CDNSCLK]
```

Related Commands

- [all_outputs](#)
- [get_ports](#)
- [report_ports](#)
- [set_input_delay](#)

all_instances

```
all_instances
obj
[-hierarchical]
```

Returns a collection of all instances available in a design, or all instances of a specified library cell existing in the current design.

Note: You cannot use the `all_instances` command on the top level of the design (`all_instances [get_design top]`).

However, you can use the command for hierarchical modules inside the top level. For example, if `SUB` is a hierarchical module in the top level of the design, you can specify the following command to return all instances of that module:

```
all_instances [get_design SUB].
```

Parameters

<code>-help</code>	Prints out the command usage.
<code>- hierarchical</code>	Specifies that the patterns should be matched hierarchically. If the hierarchical name of an instance at a particular hierarchical level matches the patterns, it is included in the result.
<code>obj</code>	Specifies the name of a design or a library cell in the current design. You can specify a pattern for the design or library cell name, or a single collection of a design or library cell.

Examples

- The following command returns a collection of all the instances of the library cell BUF:

```
tempus> all_instances BUF
```

- The following returns a collection of all instances of the sub-module subblock:

```
tempus > all_instances [get_design subblock]
```

```
subblock_inst1 subblock_inst2
```

all_outputs

```
all_outputs
[-clock list_of_clocks]
[-edge_triggered]
[-level_sensitive]
```

Creates a collection of all output ports in the current design. You can limit the contents of a collection to ports with output delay relative to a specific clock. You can assign these ports to a variable or pass them using another command.

Parameters

-help	Prints out the command usage.
-clock <i>list_of_clocks</i>	Considers only output ports with an output delay relative to a specific collection or a list of clocks.
-edge_triggered	Returns a collection of output ports with the <code>set_output_delay -clock</code> constraint.
-level_sensitive	Returns a collection of output ports with the <code>set_output_delay -level_sensitive</code> constraint.

Examples

- The following command returns a collection of output ports with an output delay relative to `ideal_clk`:
`all_outputs -clock ideal_clk`

Related Commands

- `all_inputs`
- `get_ports`
- `report_ports`
- `set_output_delay`

all_registers

```
all_registers
[-clock {clock_list}]
[-cells]
[-rise_clock {clock_list}]
[-fall_clock {clock_list}]
[-flops | -edge_triggered]
[-no_hierarchy]
[-latches | -level_sensitive]
[-macros]
[-master_slave]
[-data_pins]
[-clock_pins]
[-output_pins]
[-async_pins]
[-slave_clock_pins]
```

Returns a collection of instances and pins of flip-flops, latches, or sequential macros in the design. The command considers a cell as sequential if the library cell defines trigger or timing check arcs.

 You can use the following options only after building the timing graph:

- -clock
- -rise_clock
- -fall_clock

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--all_registers

Parameters

-help	Prints out the command usage.
-async_pins	Returns a collection of async preset or clear pins.
-cells	Creates a collection of cells. This is the default option.
-clock {clock_list}	Specifies the name of a clock or a list of clocks. The all_registers command creates a collection of registers clocked by the specified clock_list.
-clock_pins	Returns a collection of register clock pins. You can use other parameters to further filter this collection.
-data_pins	Returns a collection of register data input pins. You can use other parameters to further filter this collection.
-fall_clock {clock_list}	Specifies the name of a clock or a list of clocks. The all_registers command creates a collection of registers opened by the falling edge of the clocks specified in the clock_list.
-flops -edge_triggered	Returns a collection of edge-triggered sequential instances. Both the -flops and -edge_triggered options return the same collection. The -flops option is provided for backward compatibility.
-latches -level_sensitive	Returns a collection of level-sensitive sequential instances. Both -latches and -level_sensitive options return the same collection. The -latches option is provided for backward compatibility.
-macros	Returns sequential instances that are neither flip-flops nor latches.
-master_slave	Returns a collection of master-slave register cells.
-no_hierarchy	Searches the current instance only; does not descend the hierarchy.
-output_pins	Returns a collection of register output pins.
-rise_clock {clock_list}	Specifies the name of a clock or a list of clocks. The all_registers command creates a collection of registers opened by the rising edge of the clocks specified in the clock_list.
-slave_clock_pins	Returns a collection of slave clock pins of the master-slave registers. You can specify the slave clock pins as clocked_on_also in the library.

Examples

- The following command reports the 10 worst paths going to registers in the design:

```
report_timing -to [all_registers] -max_paths 10
```

- The following script gets number of flops in the design by using the all_registers command along with other Litmus commands:

```
set flopName [all_registers -flops]
set flopCount [sizeof_collection $flopName]
puts "Number of flops in the design is $flopCount"
```

Related Commands

- `get_cells`
- `get_pins`

append_to_collection

```
append_to_collection
[-help]
var_name
second_collection_or_list
[-unique]
```

Appends objects from one collection to another collection. The following rules apply:

- If all of the objects in the base collection are of the same type, only the objects in the second collection that are the same type as those in the base collection are appended to the base collection.
- If the base collection contains different types of objects, all objects in the second collection are appended to it. In this situation, you cannot specify an object list.

If the software finds no objects to add, an empty collection is returned.

Parameters

-help	Prints out the command usage.
var_name	Specifies a pointer to the base collection of objects. Objects in the second collection or object list are appended to this collection. If you specify a pointer that does not refer to an existing base collection, the software creates a new collection that uses the pointer as a reference.
second_collection_or_list	Specifies the second collection of objects, or an object list.
-unique	Retains any duplicate objects that already exist in the base collection, but does not append duplicate objects from the second collection to the base collection.

Examples

- The following command appends the nets in the specified collection to the collection `newVar`:

```
append_to_collection newVar [get_nets {CLK1}]
```

- The following command appends the specified nets to the collection `newVar`:

```
append_to_collection newVar {Sin1 CLK1 Sout1}
```

- You can use the following commands to create another collection:

```
litmus 6> set base_clock [get_clocks sbw_clk]
0xe50
litmus 7> set top_clock [get_clocks aclk]
0xe51
litmus 8> set base_top_clock ""
litmus 13> append_to_collection base_top_clock $base_clock
0xe54
litmus 14> append_to_collection base_top_clock $top_clock
0xe54
litmus 15> get_clocks $base_top_clock
sbw_clk aclk
0xe55
litmus 16> get_clocks $base_clock
sbw_clk
0xe56
litmus 17> get_clocks $top_clock
aclk
0xe57
```

Note: If a new collection variable is defined with an existing collection, then both point to the same object set. Any modifications made into either collection will be visible in both cases - because they reference each other.

Related Commands

- `range_collection`
- `compare_collections`

compare_collections

`compare_collections collection1 collection2 [-order_dependent]`

Compares two collections, and returns a value of 0 if all of the objects contained in both collections are the same. If the objects contained in both collections are different, the software returns a value of 1.

Parameters

<code>collection1</code>	Specifies the first collection.
<code>collection2</code>	Specifies the second collection.
<code>-order_dependent</code>	Specifies that the order of the objects in both collections must be the same for the collections to be considered the same.

Related Commands

- `append_to_collection`
- `range_collection`
- `filter_collection`

copy_collection

```
copy_collection  
[-help]  
base_collection
```

Returns a collection that is an exact copy of the specified base collection.

Parameters

<code>-help</code>	Prints out the command usage.
<code>base_collection</code>	Specifies the base collection of objects.

Related Commands

- `append_to_collection`
- `range_collection`
- `filter_collection`

define_proc_arguments

```
define_proc_arguments
[-help]
proc_name
[-info info_text]
[-define_args arg_defs]
[-hide]
```

Defines the arguments and attributes of a Tcl procedure. The arguments and attributes are displayed when you view the help associated with the procedure.

The `define_proc_arguments` command, when used with the `parse_proc_arguments` command, enables you to validate command usage. For invalid commands, appropriate error messages are displayed.

Parameters

-help	Prints out the command usage.
proc_name	Specifies the name of an existing Tcl procedure.
-info info_text	Provides help information for the procedure.
-define_args arg_defs	Defines all possible arguments of the Tcl procedure. It can also be used to define the help text and constraints for individual arguments of the procedure and generates the help for command options. The variable <code>arg_defs</code> is a list of lists where each list element defines one argument. Each element in the list has the following format: <pre>"arg_name" "option_help" "value_help" "data_type" "attributes"</pre>
	<code>arg_name</code> : Specifies the name of the argument.
	<code>option_help</code> : Provides a short description of the argument.
	<code>value_help</code> : Specifies the help information for the associated option values.
	<code>data_type</code>: (Optional) Used for validation of options. Supported data types are <code>string</code>, <code>list</code>, <code>boolean</code>, <code>int</code>, <code>integer</code>, <code>float</code>, or <code>one_of_string</code>. Default: <code>string</code>
	<code>attributes</code>: (Optional) Used for validation of options. The attributes can be: • <code>required</code>: This argument must be specified . This is mutually exclusive with <code>optional</code> and <code>internal</code>. • <code>optional</code>: Specifying this argument is optional. This is mutually exclusive with <code>required</code>. • <code>internal</code>: Disables the display of help information for the option in the procedure help. This argument is mutually exclusive with <code>required</code>. • <code>value_help</code>: Indicates that the valid values for <code>one_of_string</code> argument should be listed whenever argument help is shown. • <code>values {allowable_values}</code>: This argument must be used with the <code>one_of_string</code> data type. • <code>merge_duplicates</code> : Ensures that when the option appears more than once in a command, its values are concatenated into a list of values. The default behavior is that the right-most value specified for the option is used. • <code>bind_option other_arg_name</code>: Binds the current option with the option specified as the <code>other_arg_name</code> variable. • <code>mutual_exclusive_group group_name</code>: Defines a group name and includes the current option in the <code>group_name</code> group. All options in the same group are treated as mutually exclusive. • Default: <code>optional</code>
-hide	Hides the procedure and its body.

Examples

The following Tcl procedure uses the `define_proc_arguments` command for defining arguments and attributes, and for validating the command usage.

```
proc cmdProc {args} {
  parse_proc_arguments -args $args results
  foreach argname [array names results] {
    puts "$argname = $results($argname)"
  }
}

define_proc_arguments cmdProc -info "Command Description" \
-define_args \
{{-string "String help" "string_val" string optional}
{-integer "Int help" "int_value" int optional}
{-list "List help" "list_value" list {optional bind_option -Bool}}
{-boolean_a "BoolA help" "" boolean {optional mutual_exclusive_group
g0}}
{-boolean_b "BoolB help" "" boolean {optional mutual_exclusive_group
g0}}
{-float_num "Float help" "float_value" float}
{-oosa "One of StringA help" "one_of_str_value" one_of_string
{optional value_help {values {v1 v2}}}}
{-oosb "One of StringB help" "one_of_str_value" one_of_string
{optional {values {v4 v5}}}}
}
```

Now, if the `cmdProc` procedure is invoked with the `-help` option at the command prompt, the following help information is displayed.

```
litmus> cmdProc -help
Usage: cmdProc # Command Description
[-string string_val] (String help)
[-integer int_value] (Int help)
[-list list_value] (List help, Bind_Option=-Bool)
[-boolean_a ] (BoolA help, Mutual_Exclusive_Group=g0)
[-boolean_b ] (BoolB help, Mutual_Exclusive_Group=g0)
[-float_num float_value] (Float help)
[-oosa one_of_str_value] (One of StringA help, Values={v1 v2 })
[-oosb one_of_str_value] (One of StringB help)
```

define_property

```
define_property
[-help]
-object_type string
-type data_type
[-lower_bound val]
[-upper_bound val]
[-string_values {values}]
property_name
```

Defines a new property.

Design objects can be associated with user-defined commands that can be used for analysis and debugging purposes.

You can use the `define_property` command if a define statement is not used for the specified property value(s). To read property values from the library, the command must be issued before the timing library is read.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-lower_bound val</code>	Specifies a lower range - integer, float, or double.
<code>-object_type string</code>	Specifies the type of object on which the property is to be defined.
<code>property_name</code>	Specifies the name of the property.
<code>-string_values values</code>	Specifies the values that can be defined for the property.
<code>-type data_type</code>	Specifies the type of property.
<code>-upper_bound val</code>	Specifies a higher range - integer, float, or double.

Examples

- The following example defines property having object type `pin` and type of property `int`:

```
define_property -object_type pin -type int slk
```
- The following example defines a property on a library arc:

```
define_property -object_type lib_arc -type int libArcProperty
```
- The following example defines `aocv_weight` property on a library arc:

```
define_property -type float -object_type lib_arc aocv_weight
```

Related Commands

- [get_property](#)
- [set_property](#)
- [reset_property](#)

filter_collection

```
filter_collection
[-help]
base_collection
{filter_expression}
[-nocase]
[-quiet]
[-regexp]
```

Returns a new collection of objects that were filtered from the specified collection of objects based on user-specified criteria (*filter_expression*). The original collection remains unmodified.

The following rules apply:

- The *filter_expression* can be created by combining several object properties using the following relational and logical operators: >, <, ==, !=, <=, >=, &&, ||, =~, !~, AND, OR. You also can use parentheses to create expressions.
- You can use the `defined` and `undefined` existence operators to determine whether an attribute is defined for an object.
- All design object properties supported through the `get_property` command can be used for filtering the base collection.

Note: You can perform filtering operations by specifying the `-filter` parameter with the `get_*` commands.

Parameters

<code>-help</code>	Prints out the command usage.
<i>base_collection</i>	Specifies the collection of objects from which to filter objects for the new collection.
<i>filter_expression</i>	Specifies the criteria with which to filter objects. The software adds any objects in the base collection that match <i>filter_expression</i> to the new collection. See " Examples " on how to create filter expressions.
<code>-nocase</code>	Specifies that pattern matching is not case sensitive. Note: You must specify <code>-regexp</code> in order to use this parameter.
<code>-quiet</code>	Suppresses all the error and warning messages that are generated when the <code>filter_collection</code> command is run.
<code>-regexp</code>	Treats the specified patterns as regular expression patterns. <i>Default:</i> Treats the specified pattern as a wildcard.

Examples

- The following command returns a single-object collection containing the pin with the hierarchical name `lat_3/Q`:

```
filter_collection [get_pins] {hierarchical_name == lat_3/Q}
```
- The following command returns a collection of pins that have a maximum slew greater than 0.1:

```
filter_collection [get_pins] {slew_max_rise > 0.1}
```
- The following commands create a new collection containing all those paths from the path collection with a slack value greater than -0.806 and less than 0.0:

```
set paths [report_timing -collection -max_paths 10]
set newPaths [filter_collection $paths {(slack > -0.806) && (slack < 0.0)}]
```
- The following command returns a collection of object names that do not match with `**clk*`:

```
filter_collection $abc {full_name !~ *clk*}
out in1 in2 in3 select TopBus[2] TopBus[1] TopBus[0] bidi
0x53
```
- The following command returns a collection of object names that are not `**clk*` only:


```
filter_collection $abc {full_name != *clk*}
out clk1 clk2 in1 in2 in3 select TopBus[2] TopBus[1] TopBus[0] bidi
0x59
```

- The following command returns a collection of object names that match with “*clk*”:

```
filter_collection $abc {full_name =~ *clk*}
clk1 clk2
0x5f
```

- The following command creates a collection of pins with user_constant_value:

```
set constant_pins [filter_collection [get_pins] {defined(user_constant_value)}]
```

Related Commands

- [append_to_collection](#)
- [copy_collection](#)
- [range_collection](#)

foreach_in_collection

```
foreach_in_collection  
[-help]  
var_name  
collection  
body
```

Iterates through all objects in the specified collection and executes the commands in the specified script body.

Parameters

<code>-help</code>	Prints out the command usage.
<code>body</code>	Specifies the script of commands to execute during each iteration.
<code>collection</code>	Specifies the collection of objects through which to iterate.
<code>var_name</code>	Specifies the iterator variable. During each iteration, <code>var_name</code> is set to a collection of exactly one object. Any command that accepts collections can accept <code>var_name</code> as a value.

Examples

- The following command prints out the hierarchical names of all the cells in a design:

```
foreach_in_collection iCell [get_cells *] {  
  puts "Cell = [get_property $iCell hierarchical_name]"  
}
```

```
Cell = ck_0  
Cell = ck_1  
Cell = ck_2  
Cell = ck_4
```

- The following example shows a more complex use of the `foreach_in_collection` command:

```
set cellCollection [get_cells temp1_reg]  
set libCellCollection [get_lib_cells -of_objects $cellCollection]  
set libArc [get_lib_arcs -of_objects $libCellCollection]  
  
foreach_in_collection arc $libArc {  
  set fpin [get_property $arc from_lib_pin]  
  
  set tpin [get_property $arc to_lib_pin]  
  set sense [get_property $arc timing_type]  
  
  set from_pin_name [get_property $fpin name]  
  set to_pin_name [get_property $tpin name]  
  
  puts [format "%25s -> %25s %-20s" $from_pin_name $to_pin_name $sense]  
}
```

```
CK -> D          setup_rising  
CK -> D          hold_rising  
CK -> CK         minimum_period  
CK -> CK         minimum_period  
CK -> CK         min_pulse_width  
  
foreach_in_collection.txt  
CK -> CK         min_pulse_width  
RN -> RN         min_pulse_width  
CK -> RN         recovery_rising  
CK -> RN         removal_rising  
CK -> Q          falling_edge  
RN -> Q          clear
```

Related Commands

- `append_to_collection`
- `copy_collection`
- `range_collection`

get_arcs

```
get_arcs
[-help]
{ [-to to_list] [-from from_list]
  [-of_objects object_list]
[-filter expr]
[-quiet]
```

Creates a collection of timing arcs. Assign these arcs to a variable, or pass them to another command. If no objects match the criteria, an empty collection is returned.

Note: The timing graph must be built before you can use this command. A warning message is issued indicating that the timing graph is not generated.

Parameters

-help	Prints out the command usage.
-filter <i>expr</i>	Filters a collection based on the specified expression. For any arcs that match the pattern, the software evaluates the expression based on the attribute of the arc. For details on how to create filter expressions, refer to the filter_collection command.
-from <i>from_list</i>	Adds all timing arcs that fan in directly to the specified pins or ports to the collection. You can specify a list of pins or a list of ports, but you cannot specify a list that contains both pins and ports. You can specify a pattern for the pin or port name, a collection of pins or ports, or a pattern and a collection. Note: You cannot specify this parameter with the -of_objects parameter.
-of_objects <i>object_list</i>	Adds all timing arcs that are associated with the specified cells or nets to the collection. You can specify a list of cells or a list of nets, but you cannot specify a list that contains both cells and nets. You can specify a pattern for the cell or net name, a collection of cells or nets, or a pattern and a collection. Note: You cannot specify this parameter with the -to or -from parameters.
-quiet	Suppresses all error and warning messages generated when the <code>get_arcs</code> command is run.
-to <i>to_list</i>	Adds all timing arcs that fan out directly from the specified list of pins or ports to the collection. You can specify a list of pins or a list of ports, but you cannot specify a list that contains both pins and ports. You can specify a pattern for the pin or port name, a collection of pins or ports, or a pattern and a collection. Note: You cannot specify this parameter with the -of_objects parameter.

Examples

- The following command returns a collection of the timing arcs fan in directly to the pin `dff1/CP`:

```
get_arcs -from dff1/CP
```
- The following command creates a collection of the timing arcs that are associated with the nets in the specified collection:

```
get_arcs -of_objects [get_nets {n1}]
```

get_cells

```
get_cells
[-help]
[-filter expr]
[-hierarchical]
[-hsc char]
[-leaf]
[-nocase]
[-quiet]
[-regexp]
[patterns | -of_objects object_list]
```

Creates a collection of instances in the current design whose name matches the supplied pattern list. Assign this collection to a variable or pass it as an argument in another command.

Parameters

-help	Prints out the command usage.
-filter <i>expr</i>	Filters the collection with the specified expression. For any cells that match the pattern, the software evaluates the expression based on the attribute of the cell. You can filter cells using any of the cell properties supported through the get_property command. For details on how to create filter expressions, refer to the filter_collection command.
-hierarchical	Specifies that the patterns should be matched hierarchically. If the flat name of an instance at a particular hierarchical level matches the patterns, it is included in the result. For example, if there is top level hierarchical instance <i>Top</i> and it contains another sub-instance called <i>FF1</i> , then the following command can be used to get the instance <i>Top/FF1</i> (since the given pattern will be matched with instance flat name <i>FF1</i>): <code>get_cells -hier FF*</code> However, the following command will display an error: <code>get_cells -hier Top/FF*</code>
-hsc <i>char</i>	Specifies the hierarchical delimiter for patterns. For example, if the hierarchical delimiter is @, the <i>sub/I1@I2</i> pattern matches instance <i>I2</i> within the hierarchy limited at <i>sub/I1</i> .
-leaf	Returns leaf cells only. When the <i>-leaf</i> parameter is not specified, the <code>get_cells</code> command returns the hierarchical instances present on a net along with the flat instances at the same hierarchical level as the specified object. Note: The <i>-leaf</i> parameter can only be specified with the <i>-of_objects</i> parameter.
-nocase	Specifies that pattern matching is not case sensitive. Note: You must specify <i>-regexp</i> in order to use this parameter.
-of_objects <i>object_list</i>	Creates a collection of all cells associated with the specified pins, nets, library cells, or design cells. You can specify a list of pins or a list of nets, but you cannot specify a list that contains both pins and nets. You can specify patterns for the pin or net names, a collection of pins or nets, or a combination of patterns and a collection. The <i>-of_objects</i> parameter supports get_designs and get_lib_cells as valid collection objects. The use model is as follows: <code>get_cells -of_objects [get_designs <i>object_list</i>]</code> <code>get_cells -of_objects [get_lib_cells <i>object_list</i>]</code>
<i>patterns</i>	Specifies patterns for instance names. Patterns include the * and ? wildcard characters and collections of instances. The <code>get_cells</code> command returns a collection of instances that match the patterns. If no such instance is found, an empty collection is returned.
-quiet	Suppresses all error and warning messages generated when the <code>get_cells</code> command is run.

<code>-regex</code>	Treats the specified patterns as a regular expression patterns. When this parameter is specified, the given patterns <code>start</code> and <code>end</code> are assumed to be matching with the start and end part of the complete object name. For example, the following can be used to retrieve the hierarchical instance <code>Top/block1/block2/FF1</code>: <pre>get_cells Top/*/*/FF1 get_cells -regex Top/.*/FF1</pre> However, the following command will display an error : <pre>get_cells Top/*/FF1</pre> If this parameter is specified with <code>-hier</code> parameter, the given pattern will be matched with the complete hierarchical name of objects instead of only the flat name and all the successful matches are included in the result. For example, if there is a top level hierarchical instance <code>Top</code> and it contains another sub-instance called <code>FF1</code>, then either of these can be used to retrieve <code>Top/FF1</code> instance: <pre>get_cells -hier FF1 get_cells -hier -regex Top/FF1</pre> However, the following commands will display an error : <pre>get_cells -hier Top/FF1 get_cells -hier -regex FF1</pre>
---------------------	---

Examples

- The following command searches for pins within the hierarchy limited at `B`:

```
get_cells -hsc @ B@*
```
- The following command searches for cells with set pattern and filter:

```
get_cells "o*" -filter "@ref_lib_cell_name != FD2 && @is_black_box==false"
```

Related Commands

- `get_libs`
- `get_lib_cells`
- `get_lib_pins`
- `get_pins`

get_clocks

```
get_clocks
[-help]
[-filter expr]
[-regexp]
[-nocase]
[-quiet]
[patterns]
```

Creates a collection of clocks whose name matches the supplied pattern list. Assign this collection to a variable or pass it as an argument to another command.

The `get_clocks` command reports all clocks, including inactive clocks.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-filter <i>expr</i></code>	Filters the collection with the specified expression. For any clocks that match the pattern, the software evaluates the expression based on the attribute of the clock. You can filter clocks using any of the clock properties supported through the <code>get_property</code> command. For details on how to create filter expressions, refer to the <code>filter_collection</code> command.
<code>-nocase</code>	Specifies that pattern matching is not case sensitive. Note: You must specify <code>-regexp</code> in order to use this parameter.
<code><i>patterns</i></code>	Specifies a pattern (or a list of patterns) for clock names. A pattern can contain <code>*</code> and <code>?</code> wildcard characters, or a collection of instances and clocks. Note: If a pattern is not specified, then <code>get_clocks</code> returns all the clocks that meet other filters.
<code>-quiet</code>	Suppresses all error and warning messages generated when the <code>get_clocks</code> command is run.
<code>-regexp</code>	Treats the specified patterns as a regular expression patterns. <i>Default:</i> Treats the specified pattern as a wild card

Exampless

- The following example returns all clocks beginning with the name `CK`:

```
get_clocks CK*
```
- The following example returns a collection of all clocks beginning with the name `CK` and `Clk`:

```
get_clocks [list CK* Clk* ]
```
- The following example returns a collection of all active clocks beginning with the name `CK`:

```
get_clocks CK* -filter {is_active == true}
```
- The following command returns all the clocks that are propagated:

```
get_clocks -filter "is_propagated_clock == true"
```

Related Commands

- `all_clocks`
- `create_clock`
- `get_pins`
- `get_ports`
- `report_clocks`

get_designs

```
get_designs
[-help]
[-quiet]
patterns
```

Creates a collection of modules and assigns this collection to a variable or pass it as an argument to an another command. The software uses this information during timing analysis.

Parameters

-help	Prints out the command usage.
patterns	Specifies patterns for module names. Patterns include the * and ? wildcard characters and collections of modules. The <code>get_designs</code> command returns a collection of modules that match the patterns. If no such module is found, an empty collection is returned.
-quiet	Suppresses all error and warning messages generated when the <code>get_design</code> command is run.

Examples

- The following command returns a complete list of modules:

```
get_designs *
```


get_generated_clocks

```
get_generated_clocks
[-help]
[-filter expr]
[-quiet]
[-regexp | -exact]
[-nocase]
[patterns]
```

Creates a collection of generated clocks whose name matches the supplied pattern list. Assign this collection to a variable or pass it as an argument to another command.

Parameters

-help	Prints out the command usage.
-exact	Performs exact pattern matching against the object name, instead of wild card or regular expression matching. Use this parameter when the object name includes special characters such as *, ?, and +. ⚠ If you specify this parameter, you cannot specify <code>-regexp</code>.
-filter <i>expr</i>	Filters the collection with the specified expression. For any clocks that match the pattern, the software evaluates the expression based on the attribute of the clock.
-nocase	Specifies that pattern matching is not case sensitive.
<i>patterns</i>	Specifies patterns for generated clock names. Patterns include the * and ? wildcard characters, and collections of generated clocks.
-quiet	Suppresses all error and warning messages generated when you run the <code>get_cells</code> command.
-regexp	Treats the specified pattern as a regular expression. <i>Default:</i> Treats the specified pattern as a wild card ⚠ If you specify this parameter, you cannot specify <code>-exact</code>.

Examples

The following command returns the collection of all the generated clocks present in the design whose name starts with "CK":

```
% get_generated_clocks CK*
CKGEN1 CKGEN2 CKGEN3
```

Related Commands

- [create_generated_clock](#)

get_lib_arcs

```
get_lib_arcs
{ [-to to_lib_pins] [-from from_lib_pins]
  [-of_objects {lib_cell_list | timing_arcs}] }
[-filter expr]
[-quiet]
```

Creates a collection of library timing arcs. Assign these library timing arcs to a variable, or pass them to another command. The command returns an empty collection if no objects match the criteria.

Parameters

-filter <i>expr</i>	Filters the collection with the specified expression. For any library arcs that match the pattern, the software evaluates the expression based on the attribute of the library arc
-from <i>from_lib_pins</i>	Creates a collection of all library arcs that originate from the specified library pins. You can specify a pattern for the pin name, a collection of pins, or a pattern and a collection. If you specify this parameter with the -to parameter, the software creates a collection of all library arcs that exist between the "to" and "from" library pins. ⚠ You cannot specify this parameter with the -of_objects parameter.
-of_objects { <i>lib_cell_list</i> <i>timing_arcs</i> }	Creates a collection of all library arcs that are associated with the specified library cells or timing arcs. You can specify a list of library cells or a list of timing arcs, but you cannot specify a list that contains both library cells and timing arcs. You can specify a pattern for the library cell or timing arc name, a collection of library cells or timing arcs, or a pattern and a collection. ⚠ You cannot specify this parameter with the -to or -from parameters.
-quiet	Suppresses all error and warning messages generated when you run the get_lib_arcs command.
-to <i>to_lib_pins</i>	Creates a collection of all library arcs that stop at the specified library pins. You can specify a pattern for the pin name, a collection of pins, or a pattern and a collection. If you specify this parameter with the -from parameter, the software creates a collection of all library arcs that exist between the "to" and "from" library pins. ⚠ You cannot specify this parameter with the -of_objects parameter.

Examples

The following command reports the timing type off all the timing arcs of the flop 'DFF':

```
% get_property [get_lib_arcs -of_objects DFF] timing_type

Sample output:
minimum_period minimum_period min_pulse_width min_pulse_width setup_rising hold_rising rising_edge rising_edge
```

Related Commands

- [get_lib_cells](#)
- [get_lib_pg_pins](#)
- [get_lib_pins](#)
- [get_libs](#)

get_lib_cells

```
get_lib_cells
[-help]
[-filter expr]
[-regexp]
[-nocase]
[-quiet]
{pattern_list | -of_objects object_list}
```

Creates a collection of library cells from the loaded libraries whose name matches the supplied pattern list. Assign these library cells to a variable or pass them to another command. If no objects match the criteria, an empty collection is returned.

Parameters

-help	Prints out the command usage.
-filter <i>expr</i>	Filters the collection with the specified expression. For any library cells that match the pattern, the software evaluates the expression based on the attribute of the library cell. You can filter library cells using any of the library cell properties supported through the get_property command. For details on how to create filter expressions, refer to the filter_collection command.
-nocase	Specifies that pattern matching is not case sensitive. Note: You must specify <code>-regexp</code> in order to use this parameter.
-of_objects <i>object_list</i>	Creates a collection of all library cells associated with the specified library pins or cells or libs. You can specify a list of library pins or a list of cells or a list of libs, but you cannot specify a list that contains mixed of library pins and cells and libs. You can specify patterns for the library pin or cell or lib names, a collection of library pins or cells or libs, or a combination of patterns and a collection.
<i>pattern_list</i>	Returns only those libraries in the collection whose name matches one or more of the patterns specified in the <i>pattern_list</i> argument. The characters <code>?</code> and <code>*</code> are wild cards. The <code>?</code> character matches any one character while the <code>*</code> character matches zero or more characters. If a library matches multiple patterns, it appears multiple times in the collection. A pattern can have only one hierarchical separator character. The left part of the pattern is the library name pattern and the right part is the cell name pattern. If there is no separator character, the pattern is compared to cell names in all the libraries. A pattern can have only one hierarchical separator character, dividing the pattern into one or two parts. If the hierarchical separator is <code>/</code> , the alternatives are: <i>cell_name_pattern</i> Selects all cells whose name matches the <i>cell_name_pattern</i> from all libraries. <i>lib_name_pattern/cell_name_pattern</i> Selects all cells whose name matches the <i>cell_name_pattern</i> from all libraries whose name matches the <i>lib_name_pattern</i>.
-quiet	Suppresses all error and warning messages generated when the <code>get_lib_cells</code> command is run.
-regexp	Treats the specified patterns as regular expression patterns. <i>Default:</i> Treats the specified pattern as wild card

Examples

- The following command prints all cells for all libraries:
`get_lib_cells *`
- The following command prints a collection with all cells in the `lca500kv` lib library:
`get_lib_cells lca500kv/*`
- The following command prints a collection with all cells whose names starts with `AN` in all libraries starting with `lca`:
`get_lib_cells lca*/AN*`
- The following command disables delay arcs on the `EF11L050032A` library cell. The `set_disable_cell_timing` command effects all instantiations of the

specified cell, and lets you disable delay arcs from and to internal pins.

```
set_disable_timing -from WACLK -to RBO1 [get_lib_cells {EF11L050032A}]
```

Related Commands

- `get_lib_arcs`
- `get_lib_pg_pins`
- `get_lib_pins`
- `get_libs`
- `get_property`

get_lib_pg_pins

```
get_lib_pg_pins
[-help]
[-filter expr]
{-of_objects object_list}
```

Returns a collection of library-level PG pins from the loaded libraries. The command returns an empty collection if no PG pin definitions are found in the loaded library.

Parameters

-help	Prints out the command usage.
-filter <i>expr</i>	Filters the collection with the specified expression. For any library pg pins that match the specified expression, the software evaluates the expression based on the attribute of the library pg pin. You can filter library pg pins along with the -of_objects parameter - using the library pg pin properties of lib_pg_pin object type (supported through the get_property command).
-of_objects <i>object_list</i>	Creates a collection of all library PG pins associated with the specified library cells. The -of_objects parameter supports get_lib_cells as valid objects. For example, <pre>get_lib_pg_pins -of_objects [get_lib_cells sc9_cmoslp/DFFQ_X1M] VDD VSS get_lib_pg_pins * **ERROR: (TCLCMD-1420): Currently only -of_objects option is supported</pre>

Examples

- The following command gets the collection of all the library-level pg pins of the specified library cell:

```
litmus> get_lib_pg_pins -of_objects [get_lib_cells sc_cmoslp/A022X]
```
- The following command filters the collection of all library-level pg pins of the library cell to return only the library pg pins containing the string name defined with the lib_pg_pin name attribute:

```
litmus> get_lib_pg_pins -of_objects [get_lib_cells sc_cmoslp/A022X] -filter "name =~ *VDD*"
```

Related Commands

- [get_lib_cells](#)
- [get_lib_pins](#)
- [get_libs](#)
- [get_property](#)

get_lib_pins

```
get_lib_pins
[-help]
[-filter expr]
[-regexp]
[-nocase]
[-quiet]
{pattern_list | -of_objects object_list}
```

Creates a collection of library cell pins from the loaded libraries whose name matches the supplied pattern list. Assign these library cell pins to a variable or pass them to another command. If no objects match the criteria, an empty collection is returned.

Parameters

-help	Prints out the command usage.
-filter <i>expr</i>	Filters the collection with the specified expression. For any library cell pins that match the pattern, the software evaluates the expression based on the attribute of the library cell pin. You can filter library pins using any of the library pin properties supported through the get_property command. For details on how to create filter expressions, refer to the filter_collection command.
-nocase	Specifies that pattern matching is not case sensitive. Note: You must specify <code>-regexp</code> in order to use this parameter.
-of_objects <i>object_list</i>	Creates a collection of all library pins associated with the specified library cells or pins. You can specify a list of cells or a list of pins, but you cannot specify a list that contains both cells and pins. You can specify patterns for the cell or pin names, a collection of cells or pins, or a combination of patterns and a collection.
<i>pattern_list</i>	Returns only those libraries in the collection whose name matches one or more of the patterns specified in the <i>pattern_list</i> argument. The characters <code>?</code> and <code>*</code> are wild cards; The <code>?</code> character matches any one character while the <code>*</code> character matches zero or more characters. If a library cell matches multiple patterns, it appears multiple times in the collection. A pattern can have only two hierarchical separator characters, dividing the pattern into one, two, or three parts. If the hierarchical separator is <code>/</code> , the alternatives are: <i>pin_name_pattern</i> Selects all pins whose name matches the <i>pin_name_pattern</i> from all cells in the libraries. <i>cell_name_pattern/pin_name_pattern</i> Selects all pins whose name matches the <i>pin_name_pattern</i> from all cells whose name matches the <i>cell_name_pattern</i> in all libraries. <i>lib_name_pattern/cell_name_pattern/pin_name_pattern</i> Select all pins whose name matches the <i>pin_name_pattern</i> from all cells whose name matches the <i>cell_name_pattern</i> from all libraries whose name matches the <i>lib_name_pattern</i>.
-quiet	Suppresses all error and warning messages generated when the <code>get_lib_pins</code> command is run.
-regexp	Treats the specified patterns as a regular expression patterns. <i>Default:</i> Treats the specified pattern as a wild card

Examples

- The following command returns all pins in all cells for all libraries:
`get_lib_pins *`
- The following command returns a collection with all pins for cell `A01` in any library:
`get_lib_pins A01/*`
- The following command returns a collection with all pins whose name starts with `Z` or `Q` for the `A01` cell in any library:
`get_lib_pins {A01/Z* A01/Q*}`
- The following command returns a collection with all pins for the `A01` cell in the `1ca500kv` lib library:

```
get_lib_pins lca500kv/A01/*
```

- The following command returns a collection with all pins starting with `z` in all cells whose names starts with `AN` in all libraries starting with `lca`:

```
get_lib_pins lca*/AN*/Z*
```

Related Commands

- `get_lib_cells`
- `get_lib_pg_pins`
- `get_libs`
- `get_property`

get_libs

```
get_libs
[-help]
[-filter expr]
[-regexp]
[-nocase]
[-quiet]
{-of_objects object_list | pattern_list}
```

Creates a collection of loaded libraries whose name matches those specified in the pattern list. Assign these libraries to a variable or pass them to another command. If no objects match the criteria, an empty collection is returned.

Parameters

-help	Prints out the command usage.
-filter <i>expr</i>	Filters the collection with the specified expression. For any libraries that match the pattern, the software evaluates the expression based on the attribute of the library. You can filter libraries using any of the library properties supported through the <code>get_property</code> command. For details on how to create filter expressions, refer to the <code>filter_collection</code> command.
-nocase	Specifies that pattern matching is not case sensitive. Note: You must specify <code>-regexp</code> in order to use this parameter.
-of_objects <i>object_list</i>	Creates a collection of libraries that belong to the library cells in a given object list.
<i>pattern_list</i>	Returns only those libraries in the collection whose name matches one or more of the patterns specified in the <i>pattern_list</i> argument. The characters <code>?</code> and <code>*</code> are wild cards; The <code>?</code> character matches any one character while the <code>*</code> character matches zero or more characters. If a library matches multiple patterns, it appears multiple times in the collection.
-regexp	Treats the specified patterns as a regular expression patterns. <i>Default:</i> Treats the specified pattern as a wild card
-quiet	Suppresses all error and warning messages generated when the <code>get_lib_cells</code> command is run.

Examples

- The following command displays a list of loaded libraries starting with a or l:

```
get_libs {a* l*}
```
- The following command queries all libraries beginning with the letter l:

```
get_libs l*
```
- The following command creates a collection of libraries that are associated with the library cells in the specified object list:

```
get_libs -of_objects [get_lib_cells *]
```

Related Commands

- `get_lib_cells`
- `get_lib_pg_pins`
- `get_lib_pins`
- `get_property`

get_nets

```
get_nets
[-help]
[-hierarchical]
[-hsc char]
[-filter expr]
[-regexp]
[-nocase]
[-quiet]
[patterns | -of_objects object_list]
```

Creates a collection of nets in the current design whose name matches the supplied pattern list. You can assign this collection to a variable or pass it as an argument to an another command. If no such net is found, an empty collection is returned.

When you use the `get_nets` command to specify hierarchical nets, the software matches the specified pattern and returns the top level net connected to it.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-filter expr</code>	Filters the collection with the specified expression. For any nets that match the pattern, the software evaluates the expression based on the attribute of the net. You can filter nets using any of the net properties supported through the <code>get_property</code> command. For details on how to create filter expressions, refer to the <code>filter_collection</code> command.
<code>-hierarchical</code>	Specifies that the patterns should match hierarchically. If the hierarchical name of a net at any hierarchical level matches the patterns, it is included in the result.
<code>-hsc char</code>	Specifies the hierarchical delimiter for patterns.
<code>-nocase</code>	Specifies that pattern matching is not case sensitive. Note: You must specify <code>-regexp</code> in order to use this parameter.
<code>-of_objects object_list</code>	Creates a collection of all nets associated with the specified cells, pins, or ports. You can specify a list of cells, a list of pins, or a list of ports, but you cannot specify a list that contains cells, pins, and ports together. You can specify patterns for the cell, pin, or port names, a collection of cells, pins, or ports, or a combination of patterns and a collection.
<code>patterns</code>	Specifies patterns for net names. Patterns include the <code>*</code> and <code>?</code> wildcard characters and collections of nets.
<code>-quiet</code>	Suppresses all error and warning messages generated when the <code>get_nets</code> command is run.
<code>-regexp</code>	Treats the specified patterns as regular expression patterns. <i>Default:</i> Treats the specified pattern as wildcard

Examples

- The following command returns all nets within the hierarchy limited at A:

```
get_nets -hsc @ A@*
```
- The following command returns all patterns that match hierarchy and identifies nets to be excluded from optimization:

```
get_nets * -hierarchical -filter "@dont_touch== true"
```

Related Commands

- [get_pins](#)

get_object_name

```
get_object_name  
[-help]  
collection
```

Returns the name of the object(s) contained in the specified collections.

By default, the command hides escape characters in the output. To enable printing of escape characters in the output, you can set `timing_enable_get_obj_escaped_name_backward_compatible` attribute to `true`.

Parameters

<code>-help</code>	Prints out the command usage.
<code>collection</code>	Specifies multiple collections that contain objects.

Examples

- The following command returns a tcl list of names, which can match the collection inputs:

```
get_object_name [get_cells and*]
```
- The following command defines a variable called `endpoint_list` to store endpoints with the ten worst slacks:

```
set endpoint_list [get_object_name [get_property \  
[report_timing -collection -max_points 10] capturing_point]]
```

Related Commands

- `query_objects`

get_path_groups

```
get_path_groups
[-help]
[-regexp]
[-nocase]
patterns
```

Creates a collection of path groups whose name matches those specified in the pattern list. Assign these path groups to a variable, or pass them to another command. If no objects match the criteria, an empty collection is returned.

Parameters

-help	Prints out the command usage.
-nocase	Specifies that pattern matching is not case sensitive.
patterns	Creates a collection of path groups whose name matches one or more of the specified patterns. Patterns include the * and ? wildcard characters and collections of path groups. If a path group matches multiple patterns, it appears multiple times in the collection.
-regexp	Treats the specified patterns a a regular expression pattern. <i>Default:</i> Treats the specified pattern as a wild card

Examples

- The following command returns a collection of all the path groups present in the design:

```
% get_path_groups *

Sample output:
reg2reg reg2gate
```

Related Commands

- [reset_path_group](#)
- [report_path_groups](#)

get_pg_nets

```
get_pg_nets
[-help]
[-filter expr]
[-of_objects object_list]
```

Creates a collection of netlist-level instance PG nets of leaf cells/pins in the current design. An empty collection is returned if no PG net definitions/connections are found in the design netlist.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-filter <i>expr</i></code>	Specifies the filters for the collection with the specified expression. For any pg nets that match the pattern, the software evaluates the expression based on the attribute of the net. You can filter pg nets along with the <code>-of_objects</code> parameter - using any of the pg net properties of object type <code>pg_net</code> (supported through the <code>get_property</code> command).
<code>-of_objects <i>object_list</i></code>	Creates a collection of all PG nets associated with the specified cells or pg pins. You can specify a list of cells or a list of pg pins, but you cannot specify a list that contains cells and pg pins together. The <code>-of_objects</code> parameter supports <code>get_cells</code> and <code>get_pg_pins</code> as valid objects. For example, <pre>get_pg_nets -of_objects [get_cells iA/out_reg] VSS VDD2 get_pg_nets -of_objects [get_pg_pins -of_objects [get_cells iA/out_reg]] VSS VDD2 get_pg_nets *</pre> **ERROR: (TCLCMD-1420): Currently only -of_objects option is supported.

Examples

- The following command filters the collection of all pg nets of the cell instance object to return only the pg nets containing the string name defined with the `pg_net hierarchical_name` attribute:

```
% get_pg_nets -of_objects [get_cells iA/out_reg] -filter "hierarchical_name =~ *VDD*"
```

- The following command gets the collection of all pg nets of the cell instance object:

```
% get_pg_nets -of_objects [get_cells iA/out_reg]
```

- The following command gets the collection of all pg nets of the `pg_pins` of the cell instance object:

```
% get_pg_nets -of_objects [get_pg_pins -of_objects [get_cells iA/out_reg]]
```

Related Commands

- [get_property](#)
- [get_nets](#)

get_pg_pins

```
get_pg_pins
[-help]
[-filter expr]
[-of_objects object_list]
```

Returns a collection of netlist-level instance PG pins in the current design. An empty collection is returned if no PG pin definitions are found in the design netlist.

Parameters

-help	Prints out the command usage.
-filter <i>expr</i>	Specifies the filters for the collection with the given expression. For any pg pins that match the pattern, the software evaluates the expression based on the attribute of the pin. You can filter pg pins along with the <code>-of_objects</code> parameter - using the pg pin properties of object type <code>pg_pin</code> (supported through the <code>get_property</code> command).
-of_objects <i>object_list</i>	Specifies the collection of all the pins associated with the specified cells or nets. The <code>-of_objects</code> parameter supports <code>get_cells</code> as valid objects. For example: <pre>get_pg_pins -of_objects [get_cells iA/out_reg] iA/out_reg/VSS iA/out_reg/VDD get_pg_pins iA/out_reg/VDD **ERROR: (TCLCMD-1420): Currently only -of_objects option is supported.</pre>

Examples

- The following command filters the collection of all the `pg_pins` of the cell instance object to return only the pg pins containing the string name defined with the `pg_pin hierarchical_name` attribute:

```
% get_pg_pins -of_objects [get_cells iA/out_reg] -filter "hierarchical_name =~ *VDD"
```

- The following command gets the collection of all pg pins of the cell instance object:

```
% get_pg_pins -of_objects [get_cells iA/out_reg]
```

Related Commands

- [get_property](#)
- [get_pg_nets](#)

get_pins

```
get_pins
[-help]
[-hierarchical]
[-hsc char]
[-filter expr]
[-leaf]
[-regexp]
[-nocase]
[-quiet]
{patterns | -of_objects object_list}
```

Creates a collection of instance pins whose name matches the supplied pattern list. Assign this collection to a variable or pass it as an argument to another command. If no such pin is found, an empty collection is returned.

Parameters

-help	Prints out the command usage.
-filter <i>expr</i>	Filters the collection with the specified expression. For any pins that match the pattern, the software evaluates the expression based on the attribute of the pin. If the expression is set to true, the pin is included in the result. You can filter pins using any of the pin properties supported through the get_property command. For details on how to create filter expressions, refer to the filter_collection command.
-hierarchical	Specifies that patterns should match hierarchically. If the hierarchical name of a pin at any hierarchical level matches the patterns, it is included in the result.
-hsc <i>char</i>	Specifies the hierarchical delimiter for patterns.
-leaf	Returns leaf pins only. When the -leaf parameter is not used, the <code>get_pins</code> command returns the hierarchical pins present on a net along with the flat pins at the same hierarchical level as the specified object.
-nocase	Specifies that pattern matching is not case sensitive. Note: You must specify the <code>-regexp</code> parameter in order to use this parameter.
-of_objects <i>object_list</i>	Creates a collection of all pins associated with the specified cells or nets. You can specify a list of cells or a list of nets, but you cannot specify a list that contains both cells and nets. You can specify patterns for the cell or net names, a collection of cells or nets, or a combination of patterns and a collection.
<i>patterns</i>	Specifies patterns for pin names. Patterns include the * and ? wildcard characters and collections of pins.
-quiet	Suppresses all error and warning messages generated when the <code>get_pins</code> command is run.
-regexp	Treats the specified patterns as regular expression patterns. <i>Default:</i> Treats the specified pattern as wildcard.

Examples

- The following command returns all instance pins within the hierarchy limited at A:

```
get_pins -hsc @ A@*
```

- The following command filters the collection for `pin_direction` pin attribute:

```
get_pins "in*" -filter "@pin_direction == in"
```

Related Commands

- `create_clock`
- `get_cells`
- `get_ports`

get_ports

```
get_ports
[-help]
[-filter expr]
[-regexp]
[-nocase]
[-quiet]
[patterns | -of_objects object_list]
```

Creates a collection of ports whose name matches the supplied pattern list. Assign this collection to a variable or pass it as an argument to another command.

Note: The `get_ports` command always creates a collection of top level ports irrespective of the current instance set using the `current_instance` command.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-filter <i>expr</i></code>	Filters the collection with the specified expression. For any ports that match the pattern, the software evaluates the expression based on the attribute of the port. If the expression is set to true, the port is included in the result. You can filter ports using any of the port properties supported through the <code>get_property</code> command. For details on how to create filter expressions, refer to the <code>filter_collection</code> command.
<code>-nocase</code>	Specifies that pattern matching is not case sensitive. Note: You must specify <code>-regexp</code> in order to use this parameter.
<code>-of_objects <i>object_list</i></code>	Creates a collection of all ports associated with the specified nets. You can specify patterns for the net names, a collection of nets, or a combination of patterns and a collection.
<code><i>patterns</i></code>	Specifies patterns for port names. Patterns include the <code>*</code> and <code>?</code> wildcard characters and collections of ports. The command returns a collection of ports that match the patterns. If no such port is found, an empty collection is returned.
<code>-quiet</code>	Suppresses all error and warning messages generated when the <code>get_ports</code> command is run.
<code>-regexp</code>	Treats the specified patterns as regular expression patterns. <i>Default:</i> Treats the specified pattern as wildcard.

Examples

- The following example returns all ports beginning with the name `inb`:

```
get_ports inb*
```
- The following command filters the collection for `port_direction` port attribute:

```
get_ports "mode*" -filter {port_direction == in}
```

Related Commands

- `all_inputs`
- `all_outputs`
- `get_clocks`
- `get_pins`

get_property

```
get_property  
[-help]  
var_name  
property  
[-clock clock_name]  
[-view view_name]  
[-quiet]
```

Retrieves the attribute value for the specified object(s) property.

You can query the following:

- [Cell Properties](#)
- [Clock Properties](#)
- [Design Properties](#)
- [Library Properties](#)
- [Library Cell Properties](#)
- [Library Pin Properties](#)
- [Library PG Pin Properties](#)
- [Library Timing Arc Properties](#)
- [Net Properties](#)
- [PG Net Properties](#)
- [Path Group Properties](#)
- [Pin Properties](#)
- [PG Pin Properties](#)
- [Port Properties](#)
- [Timing Arc Properties](#)
- [Timing Path Properties](#)
- [Timing Point Properties](#)
- [si_victim Properties](#)
- [si_attacker Properties](#)
- [User-Defined Properties](#)

The `get_property` command supports DRV constraints (`set_min_fanout`, `set_max_fanout`, `set_min_transition`, `set_max_transition`, `set_min_capacitance`, and `set_max_capacitance`) applied on library pins.

The `get_property` command does not work with partial property names. You are now required to specify the full name of the property.

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

Parameters

<code>-help</code>	Prints out the command usage.
<code>-clock</code> <code>clock_name</code>	Specifies the clock for which to query certain object properties.
<code>property</code>	Specifies the property for which to retrieve the attribute value.
<code>-quiet</code>	Suppresses all error and warning messages generated when the <code>get_property</code> command is run.
<code>var_name</code>	Specifies a pointer to a collection containing the object. Note: If the collection contains more than one object, the <code>get_property</code> command returns the property values for each object in the specified collection.
<code>-view</code> <code>view_name</code>	Specifies the analysis view in which to query the object attribute. You can query properties only for the following objects in an analysis view: library cells, library pins, ports, cells, pins, timing arcs, and nets. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode. Note: The properties (described in the tables below), which are related to slack, delay, arrival, slew, transition, capacitance, uncertainty and latency, depend on the specified view. When the <code>-view</code> parameter is not specified, the maximum (or minimum) value for all the views is returned.

Cell Properties

<code>area</code>	Returns the area of the cell. If the cell is hierarchical, also includes the net area. <i>Return type:</i> float
<code>clock_gating_integrated_cell</code>	Returns the clock gating integrated cells present in a library. When specified, the software returns a value that is used to determine the integrated cell functionality to be used by clock gating tools. For example, <code>get_property [get_cells <cell name>] clock_gating_integrated_cell</code> <i>Return type:</i> string
<code>early_cell_check_derate_factor</code>	Returns the derating factor for early paths specified through the <code>set_timing_derate</code> command with the <code>-early</code> parameter. <i>Return type:</i> float
<code>early_cell_check_mean_derate_factor</code>	Returns the derating factor for early paths on the instance – when the <code>set_timing_derate -mean</code> parameter is specified. <i>Return type:</i> float
<code>early_clock_check_incremental_derate_factor</code>	Returns the early clock path instances' incremental derating factor specified through the <code>set_timing_derate -increment</code> parameter. <i>Return type:</i> float
<code>early_data_check_incremental_derate_factor</code>	Returns the early data path instance's incremental derating factor specified through the <code>set_timing_derate -increment</code> parameter. <i>Return type:</i> float
<code>early_cell_check_sigma_derate_factor</code>	Returns the derating factor for early paths on the instance - when the <code>set_timing_derate -sigma</code> parameter is specified. <i>Return type:</i> float
<code>early_clk_cell_derate_factor</code>	Returns the derating factor for early clock paths specified through the <code>set_timing_derate</code> command with the <code>-early</code> parameter. <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

<code>early_clk_cell_mean_derate_factor</code>	Returns the derating factor for early clock paths – when the <code>set_timing_derate -mean</code> parameter is specified. <i>Return type:</i> float
<code>early_clk_cell_sigma_derate_factor</code>	Returns the derating factor for early clock paths – when the <code>set_timing_derate -sigma</code> parameter is specified. <i>Return type:</i> float
<code>early_data_cell_derate_factor</code>	Returns the derating factor for early paths specified through the <code>set_timing_derate</code> command with the <code>-early</code> parameter. <i>Return type:</i> float
<code>early_data_cell_mean_derate_factor</code>	Returns the derating factor for early paths – when the <code>set_timing_derate -mean</code> parameter is specified. <i>Return type:</i> float
<code>early_data_cell_sigma_derate_factor</code>	Returns the derating factor for early paths – when the <code>set_timing_derate -sigma</code> parameter is specified. <i>Return type:</i> float
<code>early_fall_cell_check_derate_factor</code>	Returns the early cell check derating factor specified through the <code>set_timing_derate</code> command with the <code>-fall</code> parameter. <i>Return type:</i> float
<code>early_fall_cell_check_mean_derate_factor</code>	Returns the early cell check derating factor – when the <code>set_timing_derate -cell_check</code> and <code>-mean</code> parameters are specified. <i>Return type:</i> float
<code>early_fall_cell_check_sigma_derate_factor</code>	Returns the early cell check derating factor – when the <code>set_timing_derate -cell_check</code> and <code>-sigma</code> parameters are specified. <i>Return type:</i> float
<code>early_fall_clk_cell_derate_factor</code>	Returns the early clock path derating factor specified through the <code>set_timing_derate</code> command with the <code>-fall</code> parameter. <i>Return type:</i> float
<code>early_fall_clk_cell_mean_derate_factor</code>	Returns the early clock path derating factor – when the <code>set_timing_derate -fall</code> and <code>-mean</code> parameters are specified. <i>Return type:</i> float
<code>early_fall_clk_cell_sigma_derate_factor</code>	Returns the early clock path derating factor – when the <code>set_timing_derate -fall</code> and <code>-sigma</code> parameters are specified. <i>Return type:</i> float
<code>early_fall_clock_incremental_derate_factor</code>	Returns the early clock path instances' incremental derating factor specified through the <code>set_timing_derate -increment</code> parameter - when the <code>-fall</code> parameter is specified. <i>Return type:</i> float
<code>early_fall_data_cell_derate_factor</code>	Returns the early data cell check derating factor – when the <code>set_timing_derate -fall</code> parameter is specified. <i>Return type:</i> float
<code>early_fall_data_cell_mean_derate_factor</code>	Returns the early data cell check derating factor – when the <code>set_timing_derate -fall</code> and <code>-mean</code> parameters are specified. <i>Return type:</i> float
<code>early_fall_data_cell_sigma_derate_factor</code>	Returns the early data cell check derating factor – when the <code>set_timing_derate -fall</code> and <code>-sigma</code> parameters are specified. <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

early_fall_data_incremental_derate_factor	Returns the early data path instances' incremental derating factor specified through the <code>set_timing_derate -increment</code> parameter - when the <code>-fall</code> parameter is specified. <i>Return type:</i> float
early_rise_cell_check_derate_factor	Returns the early cell check derating factor specified through the <code>set_timing_derate</code> command with the <code>-rise</code> and <code>-cell_check</code> parameter. <i>Return type:</i> float
early_rise_cell_check_mean_derate_factor	Returns the early cell check derating factor – when the <code>set_timing_derate -rise</code> and <code>-mean</code> parameters are specified. <i>Return type:</i> float
early_rise_cell_check_sigma_derate_factor	Returns the early cell check derating factor – when the <code>set_timing_derate -rise</code> and <code>-sigma</code> parameters are specified. <i>Return type:</i> float
early_rise_clk_cell_derate_factor	Returns the early clock cell check derating factor specified through the <code>set_timing_derate</code> command with the <code>-rise</code> parameter. <i>Return type:</i> float
early_rise_clk_cell_mean_derate_factor	Returns the early clock cell check derating factor – when the <code>set_timing_derate -rise</code> and <code>-mean</code> parameters are specified. <i>Return type:</i> float
early_rise_clk_cell_sigma_derate_factor	Returns the early clock cell check derating factor – when the <code>set_timing_derate -rise</code> and <code>-sigma</code> parameters are specified. <i>Return type:</i> float
early_rise_clock_incremental_derate_factor	Returns the early clock path instances' incremental derating factor specified through the <code>set_timing_derate -increment</code> parameter - when the <code>-rise</code> parameter is specified. <i>Return type:</i> float
early_rise_data_cell_derate_factor	Returns the early data cell check derating factor specified through the <code>set_timing_derate</code> command with the <code>-rise</code> parameter. <i>Return type:</i> float
early_rise_data_cell_mean_derate_factor	Returns the early data cell check derating factor – when the <code>set_timing_derate -rise</code> and <code>-mean</code> parameters are specified. <i>Return type:</i> float
early_rise_data_cell_sigma_derate_factor	Returns the early data cell check derating factor – when the <code>set_timing_derate -rise</code> and <code>-sigma</code> parameters are specified. <i>Return type:</i> float
early_rise_data_incremental_derate_factor	Returns the early data path instances' incremental derating factor specified through the <code>set_timing_derate -increment</code> parameter - when the <code>-rise</code> parameter is specified. <i>Return type:</i> float
has_sdc_assertions	Returns a value of <code>true</code>, if the specified cell has SDC assertions applied on it. For example, <pre>get_property [get_cells <cell name>] has_sdc_assertions</pre> <i>Return Type:</i> Boolean
hierarchical_level	Returns the hierarchical level of the specified cell. For example, <pre>get_property [get_cell -hierarchical *] hierarchical_level</pre> <i>Return Type:</i> Integer

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

hierarchical_name	Returns the complete hierarchical name of the cell. <i>Return type:</i> string
instance_model	Reports the model type of an instance. The Instance model type can be ecsmn, ccsm, cdb, udn, or nldm. In case a noise model does not exist, the software returns 'NA'. For example, <code>get_property [get_cells inst1/buf22] instance_model</code> <i>Return type:</i> string
is_black_box	Returns a value of <code>true</code> if the cell is a black box cell. A cell is considered as a black box if the "function" definition of the cell is missing from the library. <i>Return type:</i> boolean
is_buffer	Returns a value of <code>true</code> if the associated library cell is recognized as a buffer cell. For example, <code>get_property [get_cells mmu/IB1] is_buffer</code> <i>Return type:</i> boolean
is_clock_gating_check	Returns a value of <code>true</code> if the cell is gated. <i>Return type:</i> boolean
is_clock_network_inst	Returns a value of <code>true</code> if a cell is part of the clock network. <i>Return type:</i> boolean
is_combinational	Returns a value of <code>true</code> if the cell is a combinational cell (not a sequential cell). <i>Return type:</i> boolean
is_disable_timing	Returns a value of <code>true</code> if the cell's timing has been disabled using the <code>set_disable_timing</code> command. <i>Return type:</i> boolean
is_dont_touch	Returns a value of <code>true</code> if the cell definition includes <code>dont_touch:true</code> . <i>Return type:</i> boolean
is_fall_edge_triggered	Returns a value of <code>true</code> if the cell is triggered by the falling edge of the clock. <i>Return type:</i> boolean
is_hierarchical	Returns a value of <code>false</code> if the cell is a leaf or library cell. <i>Return type:</i> boolean
is_integrated_clock_gating_cell	Returns a value of <code>true</code> if the cell is an integrated clock gating cell. <i>Return type:</i> boolean
is_interface_timing	Returns a value of <code>true</code> if a library cell has interface timing specified for that cell. <i>Return Type:</i> boolean
is_macro_cell	Returns a value of <code>true</code> if the cell is a macro cell. For example, <code>get_property [get_cells macrol] is_macro_cell</code> <i>Return type:</i> boolean
is_memory_cell	Returns a value of <code>true</code> if the cell is a memory cell. <i>Return type:</i> boolean

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

is_msv	Returns a value of <code>true</code> if the cell has multiple supply voltages. For example, <pre>get_property [get_cells msv_cell] is_msv</pre> <i>Return type:</i> boolean
is_mux	Returns a value of <code>true</code> if the cell is mux. For example, <pre>get_property [get_cells TDSP_CORE_INST/PORT_BUS_MACH_INST/p11358A] is_mux</pre> <i>Return type:</i> boolean
is_negative_level_sensitive	Returns a value of <code>true</code> if the cell is used as a negative level-sensitive sequential element, such as a negative level-sensitive latch. <i>Return type:</i> boolean
is_pad_cell	Returns a value of <code>true</code> if the Liberty library <code>is_pad_cell</code> attribute is set to <code>true</code> for this pad cell. <i>Return type:</i> boolean
is_positive_level_sensitive	Returns a value of <code>true</code> if the cell is used as a positive level-sensitive sequential element, such as a positive level-sensitive latch. <i>Return type:</i> boolean
is_rise_edge_triggered	Returns a value of <code>true</code> if the cell is triggered by the rising edge of the clock. <i>Return type:</i> boolean
is_sequential	Returns a value of <code>true</code> if the cell is a latch or flip-flop, or if the cell has sequential timing checks. <i>Return type:</i> boolean
is_tristate	Returns a value of <code>true</code> if the cell definition includes <code>three_state:true</code>. <i>Return type:</i> boolean
is_unmapped	Returns a value of <code>true</code> if the specified cell name does not have a defined library cell. For example, <pre>get_property [get_cells <cell name>] is_unmapped</pre> <i>Return Type:</i> boolean
late_cell_check_derate_factor	Returns the derating factor for late paths specified through the <code>set_timing_derate</code> command with the <code>-late</code> parameter. <i>Return type:</i> float
late_cell_check_mean_derate_factor	Returns the derating factor for late paths – when the <code>set_timing_derate -late</code> and <code>-mean</code> parameters are specified. <i>Return type:</i> float
late_cell_check_sigma_derate_factor	Returns the derating factor for late paths – when the <code>set_timing_derate -late</code> and <code>-sigma</code> parameters are specified. <i>Return type:</i> float
late_clk_cell_derate_factor	Returns the derating factor for late clock paths specified through the <code>set_timing_derate</code> command with the <code>-late</code> parameter. <i>Return type:</i> float
late_clk_cell_mean_derate_factor	Returns the derating factor for late clock paths – when the <code>set_timing_derate -late</code> and <code>-mean</code> parameters are specified. <i>Return type:</i> float
late_clk_cell_sigma_derate_factor	Returns the derating factor for late clock paths – when the <code>set_timing_derate -late</code> and <code>-sigma</code> parameters are specified. <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

late_clock_check_incremental_derate_factor	Returns the late clock path instance's incremental derating factor specified through the <code>set_timing_derate -increment</code> parameter. <i>Return type:</i> float
late_data_cell_derate_factor	Returns the derating factor for late data paths specified through the <code>set_timing_derate</code> command with the <code>-late</code> parameter. <i>Return type:</i> float
late_data_cell_mean_derate_factor	Returns the derating factor for late data paths – when the <code>set_timing_derate -late</code> and <code>-mean</code> parameters are specified. For example, <pre>get_property [get_cells <cell name>] late_data_cell_mean_derate_factor</pre> <i>Return Type:</i> Float
late_data_cell_sigma_derate_factor	Returns the derating factor for late data paths – when the <code>set_timing_derate -late</code> and <code>-sigma</code> parameters are specified. <i>Return type:</i> float
late_data_check_incremental_derate_factor	Returns the late data path instances' incremental derating factor specified through the <code>set_timing_derate -increment</code> parameter. <i>Return type:</i> float
late_fall_cell_check_derate_factor	Returns the late cell check derating factor specified through the <code>set_timing_derate</code> command with the <code>-fall</code> parameter. <i>Return type:</i> float
late_fall_cell_check_mean_derate_factor	Returns the late cell check derating factor – when the <code>set_timing_derate -fall</code> and <code>-mean</code> parameters are specified. <i>Return type:</i> float
late_fall_cell_check_sigma_derate_factor	Returns the late cell check derating factor – when the <code>set_timing_derate -fall</code> and <code>-sigma</code> parameters are specified. <i>Return type:</i> float
late_fall_clk_cell_derate_factor	Returns the late clock cell check derating factor specified through the <code>set_timing_derate</code> command with the <code>-fall</code> parameter. <i>Return type:</i> float
late_fall_clk_cell_mean_derate_factor	Returns the late clock cell check derating factor – when the <code>set_timing_derate -fall</code> and <code>-mean</code> parameters are specified. <i>Return type:</i> float
late_fall_clk_cell_sigma_derate_factor	Returns the late clock cell check derating factor – when the <code>set_timing_derate -fall</code> and <code>-sigma</code> parameters are specified. <i>Return type:</i> float
late_fall_clock_incremental_derate_factor	Returns the late clock path instances' incremental derating factor specified through the <code>set_timing_derate -increment</code> parameter - when the <code>-fall</code> parameter is specified. <i>Return type:</i> float
late_fall_data_cell_derate_factor	Returns the late data cell check derating factor specified through the <code>set_timing_derate</code> command with the <code>-fall</code> parameter. <i>Return type:</i> float
late_fall_data_cell_mean_derate_factor	Returns the late data cell check derating factor – when the <code>set_timing_derate -fall</code> and <code>-mean</code> parameters are specified. <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

late_fall_data_cell_sigma_derate_factor	Returns the late data cell check derating factor – when the <code>set_timing_derate -fall</code> and <code>-sigma</code> parameters are specified. <i>Return type:</i> float
late_fall_data_incremental_derate_factor	Returns the late data path instances' incremental derating factor specified through the <code>set_timing_derate -increment</code> parameter - when the <code>-fall</code> parameter is specified. <i>Return type:</i> float
late_rise_cell_check_derate_factor	Returns the late cell check derating factor specified through the <code>set_timing_derate</code> command with the <code>-rise</code> parameter. <i>Return type:</i> float
late_rise_cell_check_mean_derate_factor	Returns the late cell check derating factor – when the <code>set_timing_derate -rise</code> and <code>-mean</code> parameters are specified. <i>Return type:</i> float
late_rise_cell_check_sigma_derate_factor	Returns the late cell check derating factor – when the <code>set_timing_derate -rise</code> and <code>-sigma</code> parameters are specified. <i>Return type:</i> float
late_rise_clk_cell_derate_factor	Returns the late clock cell check derating factor specified through the <code>set_timing_derate</code> command with the <code>-rise</code> parameter. <i>Return type:</i> float
late_rise_clk_cell_mean_derate_factor	Returns the late clock cell check derating factor – when the <code>set_timing_derate -rise</code> and <code>-mean</code> parameters are specified. <i>Return type:</i> float
late_rise_clk_cell_sigma_derate_factor	Returns the late clock cell check derating factor – when the <code>set_timing_derate -rise</code> and <code>-sigma</code> parameters are specified. <i>Return type:</i> float
late_rise_clock_incremental_derate_factor	Returns the late clock path instances' incremental derating factor specified through the <code>set_timing_derate -increment</code> parameter - when the <code>-rise</code> parameter is specified. <i>Return type:</i> float
late_rise_data_cell_derate_factor	Returns the late data cell check derating factor specified through the <code>set_timing_derate</code> command with the <code>-rise</code> parameter. <i>Return type:</i> float
late_rise_data_cell_mean_derate_factor	Returns the late data cell check derating factor – when the <code>set_timing_derate -rise</code> and <code>-mean</code> parameters are specified. <i>Return type:</i> float
late_rise_data_cell_sigma_derate_factor	Returns the late data cell check derating factor – when the <code>set_timing_derate -rise</code> and <code>-sigma</code> parameters are specified. <i>Return type:</i> float
late_rise_data_incremental_derate_factor	Returns the late data path instances' incremental derating factor specified through the <code>set_timing_derate -increment</code> parameter - when the <code>-rise</code> parameter is specified. <i>Return type:</i> float
lib_cell	Returns a collection of all the library cell objects associated with the specified library pins/cells. For example, <code>get_property [get_cells AAA] lib_cell -view VIEW1</code> <i>Return type:</i> collection
name	Returns the leaf-level name of the cell. <i>Return type:</i> string

Litmus Text Command Reference
Advanced Timing Tcl Scripting Commands--get_property

object_type	Returns the object type <code>cell</code> . <i>Return type:</i> string
pin_count	Returns the total number of signal pins. <i>Return type:</i> integer
power_domain	Returns the power domains of a cell. For example, <code>get_property [get_cells FE_OCPC5_psw_en2] power_domain</code> <i>Return type:</i> string
ref_lib_cell_name	Returns the name of reference module of the given cell. The reference module of the cell can be a Verilog module or a leaf library cell. <i>Return type:</i> string
retention_cell	Returns the value of the <code>retention_cell</code> attribute derived from Liberty. For example, <code>get_property [get_cells retention_cell] retention_cell</code> <i>Return type:</i> string
timing_model_type	Returns the model type for a given cell or instance. The supported values are <code>abstracted</code> , <code>extracted</code> , and <code>qtm</code> . <i>Return type:</i> string
x_coordinate_max	Returns the maximum x coordinate of the cell. <i>Return type:</i> float
x_coordinate_min	Returns the minimum x coordinate of the cell. <i>Return type:</i> float
y_coordinate_max	Returns the maximum y coordinate of the cell. <i>Return type:</i> float
y_coordinate_min	Returns the minimum y coordinate of the cell. <i>Return type:</i> float
is_always_on	Returns a value of <code>true</code> if a cell instance is an always-on cell. For example, <code>get_property [get_cells mmu/U1] is_always_on</code> <i>Return type:</i> boolean
is_inverter	Returns a value of <code>true</code> if a cell instance is an inverter cell. For example, <code>get_property [get_cells mmu/INV1] is_inverter</code> <i>Return type:</i> boolean
is_isolation_cell	Returns a value of <code>true</code> if a cell instance is an isolation cell. For example, <code>get_property [get_cells mmu/U1] is_isolation_cell</code> <i>Return type:</i> boolean
is_level_shifter	Returns a value of <code>true</code> if a cell instance is a level-shifter cell. For example, <code>get_property [get_cells mmu/U1] is_level_shifter</code> <i>Return type:</i> boolean

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

is_power_switch	Returns a value of <code>true</code> if a cell instance is a power switch cell. For example, <code>get_property [get_cells mmu/U1] is_power_switch</code> Return type: boolean
is_retention	Returns a value of <code>true</code> if a cell instance is a retention cell. For example, <code>get_property [get_cells mmu/U1] is_retention</code> Return type: boolean
level_shifter_type	Returns the type of level-shifter (e.g., LH, HL, or HL_LH) that a cell instance is. For example, <code>get_property [get_cells mmu/U1] level_shifter_type</code> Return type: string
pg_pins	Returns a collection of the PG pins that are present in a cell instance. For example, <code>get_property [get_cells mmu/U1] pg_pins</code> Return type: collection
power_switch_type	Returns the type of power switch (e.g., coarse-grain, fine-grain) that a cell instance is. For example, <code>get_property [get_cells mmu/U1] power_switch_type</code> Return type: string
std_cell_main_rail_name	Returns the name of the main power rail driving the power pins of a cell instance that is the <code>pg_pin</code>, which has the <code>std_cell_main_rail</code> library attribute set as <code>true</code> for the instance cell. For example, <code>get_property [get_cells mmu/U1] std_cell_main_rail_name</code> Return type: string

Clock Properties

child_clocks	Returns the first level of generated clocks. This property also works for nested generated clocks. Return type: collection For example, <code>get_property [get_clocks CK] child_clocks</code>
clock_network_pins	Returns a collection of all pins on the clock network for the clock. Return type: collection
comment	Returns the comment set for the specified clock. Return Type: String For example, <code>get_property [get_clocks CK] comment</code>
delay_max_fall	Returns the maximum falling delay value for the clock. Return type: float
delay_max_rise	Returns the maximum rising delay value for the clock. Return type: float
delay_min_fall	Returns the minimum falling delay value for the clock. Return type: float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

delay_min_rise	Returns the minimum rising delay value for the clock. <i>Return type:</i> float
divide_by	Returns dfactor set for the specified clock. For example, <code>get_property [get_clocks ck] divide_by</code> <i>Return Type:</i> Integer
driving_network_pins	Returns a collection of the network pins of the driving clock. For example, <code>get_property [get_clocks clk] driving_network_pins</code> <i>Return Type:</i> Collection
duty_cycle	Returns the duty cycle for the specified clock. For example, <code>get_property [get_clocks CLK] duty_cycle</code> <i>Return Type:</i> Float
edges	Return the edges that are set for a specified clock. For example, <code>get_property [get_clocks ck] edges</code> <i>Return Type:</i> String
edge_shift	Return the edge shift that is set for a specified clock. For example, <code>get_property [get_clocks ck] edge_shift</code> <i>Return Type:</i> String
generated_clocks_extended	Returns generated clocks derived from the regular clock. <i>Return type:</i> collection
hierarchical_name	Returns the complete hierarchical name of the clock. <i>Return type:</i> string
hold_uncertainty	Returns the most conservative uncertainty of all the possible uncertainty assertions associated with a pin, when the <code>set_clock_uncertainty -hold</code> parameter is specified. For example, <code>get_property [get_clocks <clock name>] hold_uncertainty</code> <i>Return Type:</i> Float
is_context_mapped	Returns a value of <code>true</code> if a clock is mapped to the top in a context. Additionally, this is also reported with the <code>report_clocks</code> command. For example, <code>get_property [get_clocks <clock name>] is_context_mapped</code> <i>Return Type:</i> Boolean
ideal_transition_max_fall	Returns the ideal clock transition asserted on clock waveforms specified with the <code>set_clock_transition</code> command using the <code>-fall</code> and <code>-max</code> parameters. <i>Return type:</i> float
ideal_transition_max_rise	Returns the ideal clock transition asserted on clock waveforms specified with the <code>set_clock_transition</code> command using the <code>-rise</code> and <code>-max</code> parameters. <i>Return type:</i> float
ideal_transition_min_fall	Returns the ideal clock transition asserted on clock waveforms specified with <code>set_clock_transition</code> command using the <code>-fall</code> and <code>-min</code> parameters. <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

ideal_transition_min_rise	Returns the ideal clock transition asserted on clock waveforms specified with <code>set_clock_transition</code> command using the <code>-rise</code> and <code>-min</code> parameters. <i>Return type:</i> float
is_active	Returns a value of <code>true</code> if the clock specified is active in the current view. <i>Return type:</i> boolean
is_combinational_source_path	Returns a value of <code>true</code> if <code>create_generated_clock -combinational</code> is specified for the creation of the generated clock. For example, <pre>get_property [get_clocks GCLK] is_combinational_source_path</pre> <i>Return type:</i> boolean
is_generated	Returns a value of <code>true</code> if the clock is a generated clock. <i>Return type:</i> boolean
is_inverted	Returns a value of <code>true</code>, if a specified clock is inverted. For example, <pre>get_property [get_clocks ck] is_inverted</pre> <i>Return Type:</i> boolean
is_library_created	Returns a value of <code>true</code> if <code>generated_clock</code> is specified in a Liberty <code>generated_clock</code> group and not created by a <code>create_generated_clock</code> constraint. For example, <pre>get_property [get_clocks UMC/generated_clock_cell/clkint] is_library_created</pre> <i>Return type:</i> boolean
is_source_inverted	Returns a value of <code>true</code> if a specified clock source is inverted. For example, <pre>get_property [get_clocks clk] is_source_inverted</pre> <i>Return Type:</i> Boolean
is_propagated_clock	Returns a value of <code>true</code> if the clock is in propagated mode. <i>Return type:</i> boolean
is_virtual	Returns a value of <code>true</code> if the specified clock is virtual. For example, <pre>get_property [get_clocks v_clk] is_virtual</pre> <i>Return Type:</i> boolean
latency_fall_max	Returns the maximum fall latency for a clock, when the <code>set_clock_latency -max -fall</code> parameters are specified. <i>Return type:</i> float
latency_fall_min	Returns the minimum fall latency for a clock, when the <code>set_clock_latency -min -fall</code> parameters are specified. <i>Return type:</i> float
latency_rise_max	Returns the maximum rise latency for a clock, when the <code>set_clock_latency -max -rise</code> parameters are specified. For example, <pre>get_property [get_clocks <clock name>] latency_rise_max</pre> <i>Return Type:</i> float
latency_rise_min	Returns the minimum rise latency for a clock, when the <code>set_clock_latency -min -rise</code> parameters are specified. For example, <pre>get_property [get_clocks <clock name>] latency_rise_min</pre> <i>Return Type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

lead_edge_type	Returns an edge type (rise or fall) corresponding to the leading edge for a specified clock. For example, <pre>get_property [get_clocks <clock name>] lead_edge_type</pre> <i>Return Type:</i> string
master_clock	Returns a collection of the master clocks for a specified clock. For example, <pre>get_property [get_clocks clk] master_clock</pre> <i>Return Type:</i> collection
master_source	Returns a collection of the master sources for a specified clock. For example, <pre>get_property [get_clocks ck] master_source</pre> <i>Return Type:</i> collection
max_capacitance_clock_path_fall	Returns fall max_capacitance value on clock paths. <i>Return type:</i> float
max_capacitance_clock_path_rise	Returns rise max_capacitance value on clock paths. <i>Return type:</i> float
max_capacitance_data_path_fall	Returns fall max_capacitance value on data paths. <i>Return type:</i> float
max_capacitance_data_path_rise	Returns rise max_capacitance value on data paths. <i>Return type:</i> float
max_transition_clock_path_fall	Returns fall max_transition value on clock paths. <i>Return type:</i> float
max_transition_clock_path_rise	Returns rise max_transition value on clock paths. <i>Return type:</i> float
max_transition_data_path_fall	Returns fall max_transition value on data paths. <i>Return type:</i> float
max_transition_data_path_rise	Returns rise max_transition value on data paths. <i>Return type:</i> float
min_pulse_width_high	Returns the minimum pulse width for high signals for the specified clock, when the <code>set_min_pulse_width - high</code> parameter is specified. For example, <pre>get_property [get_clocks <clock name>] min_pulse_width_high</pre> <i>Return Type:</i> float
min_pulse_width_low	Returns the minimum pulse width for low signals for the specified clock, when the <code>set_min_pulse_width - low</code> parameter is specified. For example, <pre>get_property [get_clocks <clock name>] min_pulse_width_low</pre> <i>Return Type:</i> float
multiply_by	Returns mfactor set for the specified clock. For example, <pre>get_property [get_clocks clk] multiply_by</pre> <i>Return Type:</i> integer
name	Returns the name of the clock. <i>Return type:</i> string

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

object_type	Returns the object type <code>clock</code> . <i>Return type:</i> string
period	Returns the clock period. <i>Return type:</i> float
setup_uncertainty	Returns the most conservative uncertainty of all the possible uncertainty assertions associated with pins, when the <code>set_clock_uncertainty -setup</code> parameter is specified for a clock. For example, <code>get_property [get_clocks <clock name>] setup_uncertainty</code> <i>Return Type:</i> float
source_jitter_early_fall_max	Returns the amount of the maximum early fall source insertion delay that is due to cycle-to-cycle variation. <i>Return Type:</i> float
source_jitter_early_fall_min	Returns the amount of the minimum early fall source insertion delay that is due to cycle-to-cycle variation. <i>Return Type:</i> float
source_jitter_early_rise_max	Returns the amount of the maximum early rise source insertion delay that is due to cycle-to-cycle variation. <i>Return Type:</i> float
source_jitter_early_rise_min	Returns the amount of the minimum early rise source insertion delay that is due to cycle-to-cycle variation. <i>Return Type:</i> float
source_jitter_full_cycle_hold	Returns the early full cycle path jitter that is specified on the clock source. <i>Return Type:</i> float
source_jitter_full_cycle_setup	Returns the late full cycle path jitter that is specified on the clock source. <i>Return Type:</i> float
source_jitter_half_cycle_hold	Returns the early half cycle path jitter that is specified on the clock source. <i>Return Type:</i> float
source_jitter_half_cycle_setup	Returns the late half cycle path jitter that is specified on the clock source. <i>Return Type:</i> float
source_jitter_late_fall_max	Returns the amount of the maximum late fall source insertion delay that is due to cycle-to-cycle variation. <i>Return Type:</i> float
source_jitter_late_fall_min	Returns the amount of the minimum late fall source insertion delay that is due to cycle-to-cycle variation. <i>Return Type:</i> float
source_jitter_late_rise_max	Returns the amount of the maximum late rise source insertion delay that is due to cycle-to-cycle variation. <i>Return Type:</i> float
source_jitter_late_rise_min	Returns the amount of the minimum late rise source insertion delay that is due to cycle-to-cycle variation. <i>Return Type:</i> float
source_latency_early_fall_max	Returns the early source latency for the falling transition at the maximum corner, when the <code>set_clock_latency -source -early -fall -max</code> parameters are specified for the clock. For example, <code>get_property [get_clocks <clock name>] source_latency_early_fall_max</code> <i>Return Type:</i> float
source_latency_early_fall_min	Returns the early source latency for the falling transition at the min corner, when the <code>set_clock_latency -source -early -fall -min</code> parameters are specified for the clock. For example, <code>get_property [get_clocks <clock name>] source_latency_early_fall_min</code> <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

source_latency_early_rise_max	Returns the early source latency for the rising transition at the max corner, when the <code>set_clock_latency -source -early -rise -max</code> parameters are specified for the clock. For example, <pre>get_property [get_clocks <clock name>] source_latency_early_rise_max</pre> <i>Return Type:</i> float
source_latency_early_rise_min	Returns the early source latency for the rising transition at the min corner, when the <code>set_clock_latency -source -early -rise -min</code> parameters are specified for the clock. For example, <pre>get_property [get_clocks <clock name>] source_latency_early_rise_min</pre> <i>Return type:</i> float
source_latency_late_fall_max	Returns the late source latency for the falling transition at the max corner, when the <code>set_clock_latency -source -late -fall -max</code> parameters are specified for the clock. For example, <pre>get_property [get_clocks <clock name>] source_latency_late_fall_max</pre> <i>Return type:</i> float
source_latency_late_fall_min	Returns the late source latency for the falling transition at the min corner, when the <code>set_clock_latency -source -late -fall -min</code> parameters are specified for the clock. For example, <pre>get_property [get_clocks <clock name>] source_latency_late_fall_min</pre> <i>Return type:</i> float
source_latency_late_rise_max	Returns the late source latency for the rising transition at the max corner, when the <code>set_clock_latency -source -late -rise -max</code> parameters are specified for the clock. For example, <pre>get_property [get_clocks <clock name>] source_latency_late_rise_max</pre> <i>Return type:</i> float
source_latency_late_rise_min	Returns the late source latency for the rising transition at the min corner, when the <code>set_clock_latency -source -late -rise -min</code> parameters are specified for the clock. For example, <pre>get_property [get_clocks <clock name>] source_latency_late_rise_min</pre> <i>Return type:</i> float
source_latency_pins	Returns a list of pin or port object(s), which are present between the target pins of a generated clock and master clock. For example, <pre>get_property [get_clocks GCLK] source_latency_pins</pre> <i>Return type:</i> collection
sources	Returns a collection of the clock's source pins or ports. <i>Return type:</i> collection
view_name	In multi-mode multi-corner mode, returns the analysis view name associated with a given clock. <i>Return type:</i> string
waveform	Returns the rising and falling edge times of the waveform. <i>Return type:</i> string

Design Properties

analysis_type	Returns the analysis mode: <code>single</code>, <code>bc_wc</code>, or <code>on_chip_variation</code>. <i>Return type:</i> string
---------------	---

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

boundary_model_analysis	Returns a value of <code>true</code> if the design is a boundary model. For example, <pre>get_property [get_designs MPTC] boundary_model_analysis</pre> <i>Return type:</i> boolean
capacitance_unit_in_farad	Returns the capacitance unit specified in the library. <i>Return type:</i> float
chip_size	Returns the chip size. The chip size can be specified with the <code>timing_aocv_chip_size</code> attribute. If this attribute is not set, then the software will calculate the chip size from the design node locations. The attribute setting takes precedence over the software setting. <i>Return type:</i> float
current_unit_in_amp	Returns the current unit specified in the library. <i>Return type:</i> float
hierarchical_name	Returns the complete hierarchical name of the design. <i>Return type:</i> string
is_dont_touch	Returns <code>true</code> if the design is specified through the <code>set_dont_touch</code> command. <i>Return type:</i> boolean
is_max_transition_asserted	Returns a value of <code>true</code> if a maximum transition value (set using the <code>set_max_transition</code> command) is defined in the design. <i>Return type:</i> boolean
is_min_transition_asserted	Returns a value of <code>true</code> if a minimum transition value (set using the <code>set_min_transition</code> command) is defined in the design. <i>Return type:</i> boolean
max_capacitance	Returns the maximum capacitance value for the design. <i>Return type:</i> float
max_fanout	Returns the maximum fanout value for the design. <i>Return type:</i> float
min_fanout	Returns the minimum fanout value for the design. <i>Return type:</i> float
max_transition	Returns the maximum transition design rule limit for the design. <i>Return type:</i> float
min_transition	Returns the minimum transition design rule limit for the design. <i>Return type:</i> float
min_capacitance	Returns the minimum capacitance value for the design. <i>Return type:</i> float
object_type	Returns the object type <code>design</code>. <i>Return type:</i> string
operating_condition_max	Returns the name of the maximum operating condition specified for the design. <i>Return type:</i> string
operating_condition_min	Returns the name of the minimum operating condition specified for the design. <i>Return type:</i> string

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

process_max	Returns the process value of the maximum operating condition specified for the design. <i>Return type:</i> float
process_min	Returns the process value of the minimum operating condition specified for the design. <i>Return type:</i> float
resistance_unit_in_ohm	Returns the resistance value from the library. <i>Return type:</i> float
slew_threshold_percent_fall_high	Returns the upper slew trip point for falling waveforms, specified as a percent. <i>Return type:</i> float
slew_threshold_percent_fall_low	Returns the lower slew trip point for falling waveforms, specified as a percent. <i>Return type:</i> float
slew_threshold_percent_rise_high	Returns the upper slew trip point for rising waveforms, specified as a percent. <i>Return type:</i> float
slew_threshold_percent_rise_low	Returns the lower slew trip point for rising waveforms, specified as a percent. <i>Return type:</i> float
temperature_max	Returns the temperature value of the maximum operating condition for the design. <i>Return type:</i> float
temperature_min	Returns the temperature value of the minimum operating condition for the design. <i>Return type:</i> float
timing_context_analysis	Returns a value of <code>true</code> if timing context analysis is enabled on a design. <i>Return type:</i> boolean
time_unit_in_second	Returns the time units of the design, in seconds. that is, time unit of the design is 1.000000ns, the property returns 1.000000e-09. <i>Return type:</i> float
tree_type_max	Returns the value of the <code>tree_type</code> attribute for the specified maximum operating condition. <i>Return type:</i> string
tree_type_min	Returns the value of the <code>tree_type</code> attribute for the specified minimum operating condition. <i>Return type:</i> string
voltage_max	Returns the voltage value of the maximum operating condition for the design. <i>Return type:</i> float
voltage_min	Returns the voltage value of the minimum operating condition for the design. <i>Return type:</i> float
voltage_unit_in_volt	Returns the voltage units specified in the library for the design. <i>Return type:</i> float

Library Properties

default_max_transition	Returns the default maximum transition time limit specified for a given library. <i>Return type:</i> string
default_operating_conditions	Returns the default operating conditions specified for a given library. For example, <code>get_property [get_libs FreePDK45_lib_v1.0] default_operating_conditions</code> <i>Return type:</i> string

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

hierarchical_name	Returns the hierarchical path name for the library. <i>Return type:</i> string
input_threshold_pct_fall	Returns the value of threshold point on an input pin signal falling from 1 to 0, which is used in modelling the delay of a signal transmitting from an input pin to an output pin. <i>Return type:</i> float
input_threshold_pct_rise	Returns the value of threshold point on an input pin signal rising from 0 to 1, which is used in modelling the delay of a signal transmitting from an input pin to an output pin. <i>Return type:</i> float
lib_file_path	Reports full on-disk path of a library. In case of a library database (LDB) file, the property will return the full on-disk path of the original library from which LDB was compiled. <i>Return type:</i> string
lib_version	Returns the software version in which the library file (LDB) was generated. <i>Return type:</i> string
nominal_process	Returns a value of the process either modeled as nominal process or as the process in default operating condition in the library. For example, <code>get_property [get_libs sc_cmoslp] nominal_process</code> <i>Return Type:</i> float
nominal_temperature	Returns a value of the temperature either modeled as nominal temperature or as the temperature in default operating condition in the library. For example, <code>get_property [get_libs sc_cmoslp] nominal_temperature</code> <i>Return Type:</i> float
nominal_voltage	Returns a value of the voltage either modeled as nominal voltage or as the voltage in default operating condition in the library. For example, <code>get_property [get_libs sc_cmoslp] nominal_voltage</code> <i>Return Type:</i> float
object_type	Returns the object type library. <i>Return type:</i> string
output_threshold_pct_fall	Returns the value of threshold point on an output signal falling from 1 to 0, which is used in modelling the delay of a signal transmitting from an input pin to an output pin. <i>Return type:</i> float
output_threshold_pct_rise	Returns the value of threshold point on an output signal rising from 0 to 1, which is used in the modelling of a signal transmitting from an input pin to an output pin. <i>Return type:</i> float
power_rails	Returns the name of the power rails along with their corresponding voltages as defined in the <code>voltage_map</code> attribute in the library. For example, <code>get_property [get_libs sc_cmoslp] power_rails</code> Output: Is a list of lists, each list showing the power rail name along with its associated voltage. { {VDDG 1.045} {VDD 1.045} {VSS 0} } <i>Return Type:</i> string
slew_derate_from_library	Specifies how the transition times found in the library need to be derated to match the transition times between the characterization trip points. <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

slew_lower_threshold_pct_fall	Returns the value of lower threshold point used in modeling the delay of a pin falling from 1 to 0. <i>Return type:</i> float
slew_lower_threshold_pct_rise	Returns the value of lower threshold point used to model the delay of a pin rising from 0 to 1. <i>Return type:</i> float
slew_upper_threshold_pct_fall	Returns the value of upper threshold point used to model the delay of a pin falling from 1 to 0. <i>Return type:</i> float
slew_upper_threshold_pct_rise	Returns the value of upper threshold point used to model the delay of a pin rising from 0 to 1. <i>Return type:</i> float
source_file_name	Returns the full path of all library files that contain the specified library name. The use model is given below: <code>get_property [get_libs libname] source_file_name [-view viewName]</code> If the <code>-view</code> parameter is specified, the property returns the path of the library files in the given analysis view. If this parameter is not specified, the command returns the path of all library files in all the analysis views. <i>Return Type:</i> string
threshold_voltage_group	Returns the threshold voltage group of a library. For example, <code>get_property [get_lib FreePDK45_lib_v1.0] threshold_voltage_group</code> <i>Return Type:</i> string

Library Cell Properties

area	Returns the area of the library cell. <i>Return Type:</i> float
cell_footprint	Returns the cell footprint against a library cell, which is used to assign the same footprint class to all cells that share the same layout boundary. For example, <code>get_property [get_lib_cells 1.08_CTS_HT.db/CTB_F18_HT] cell_footprint</code> <i>Return Type:</i> String
clock_gating_integrated_cell	Returns the clock gating integrated cells present in a library cell. For example, <code>get_property [get_lib_cells CTS_HT.db/ICG_F4_HVT] clock_gating_integrated_cell</code> <i>Return Type:</i> String
late_clock_check_incremental_derate_factor	Returns the late clock path library cell check incremental derating factor specified through the <code>set_timing_increment</code> parameter. <i>Return Type:</i> float
late_data_check_incremental_derate_factor	Returns the late data path library cell check incremental derating factor specified through the <code>set_timing_increment</code> parameter. <i>Return Type:</i> float
late_fall_clock_incremental_derate_factor	Returns the late clock path library cell check incremental derating factor specified through the <code>set_timing_increment</code> parameter - when the <code>-fall</code> parameter is specified. <i>Return Type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

late_fall_data_incremental_derate_factor	Returns the late data path library cell check incremental derating factor specified through the <code>set_timing_increment</code> parameter - when the <code>-fall</code> parameter is specified. Return Type: float
early_clock_check_incremental_derate_factor	Returns the early clock path library cell check incremental derating factor specified through the <code>set_timing_increment</code> parameter. Return Type: float
early_data_check_incremental_derate_factor	Returns the early data path library cell check incremental derating factor specified through the <code>set_timing_increment</code> parameter. Return Type: float
early_fall_clock_incremental_derate_factor	Returns the early clock path library cell check incremental derating factor specified through the <code>set_timing_increment</code> parameter - when the <code>-fall</code> parameter is specified. Return Type: float
early_fall_data_incremental_derate_factor	Returns the early data path library cell check incremental derating factor specified through the <code>set_timing_increment</code> parameter - when the <code>-fall</code> parameter is specified. Return Type: float
early_fall_mean_slew_derate_factor	Returns the early fall mean transition derate factor on the library cell. For example, <pre>get_property [get_lib_cells FreePDK45_lib_v1.0/AND2_X1] early_fall_mean_slew_derate_factor</pre> Return Type: float
early_fall_sigma_slew_derate_factor	Returns the early fall sigma transition derate factor on the library cell. For example, <pre>get_property [get_lib_cells FreePDK45_lib_v1.0/AND2_X1] early_fall_sigma_slew_derate_factor</pre> Return Type: float
early_rise_clock_incremental_derate_factor	Returns the early clock path library cell check incremental derating factor specified through the <code>set_timing_increment</code> parameter - when the <code>-rise</code> parameter is specified. Return Type: float
early_rise_data_incremental_derate_factor	Returns the early data path library cell check incremental derating factor specified through the <code>set_timing_increment</code> parameter - when the <code>-rise</code> parameter is specified. Return Type: float
early_rise_mean_slew_derate_factor	Returns the early rise mean transition derate factor on the library cell. For example, <pre>get_property [get_lib_cells FreePDK45_lib_v1.0/AND2_X1] early_rise_mean_slew_derate_factor</pre> Return Type: float
early_rise_sigma_slew_derate_factor	Returns the early rise sigma transition derate factor on the library cell. For example, <pre>get_property [get_lib_cells FreePDK45_lib_v1.0/AND2_X1] early_rise_sigma_slew_derate_factor</pre> Return Type: float
early_fall_slew_derate_factor	Returns the derate factor applied on a library cell when the <code>set_timing_derate -transition</code> and <code>-fall</code> parameters are specified. Return Type: float
early_rise_slew_derate_factor	Returns the derate factor applied on a library cell when the <code>set_timing_derate -transition</code> and <code>-rise</code> parameters are specified. For example, <pre>get_property [get_lib_cells CTS_HT.db/ICG_F4_HT] early_rise_slew_derate_factor</pre> Return Type: float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

fall_input_switching_derate_factor	Returns the derate factor applied on a library cell when the <code>set_timing_derate -input_switching</code> parameter is specified. For example, <pre>get_property [get_lib_cells PDK45_v1.0/IV_X32] fall_input_switching_derate_factor</pre> <i>Return Type:</i> float
has_ccs_noise	Returns a value of <code>true</code> if a library cell has a CCS noise look-up table. For example, <pre>get_property [get_lib_cells tst/tst_nm6] has_ccs_noise</pre> <i>Return type:</i> boolean
has_socv	Returns a value of <code>true</code> if a library cell has a spatial derate table (SOCV). For example, <pre>get_property [get_lib_cells PK4_lib_v1.0/A4_X2] has_socv</pre> <i>Return type:</i> boolean
has_spatial	Returns a value of <code>true</code> if a spatial derate exists for a library cell. <i>Return type:</i> boolean
hierarchical_name	Returns the complete hierarchical name of the library cell (the library name followed by the library cell name). For example, <pre>get_property [get_lib_cells -of [get_cell AB_IT/E_0146_t1]] hierarchical_name</pre> <i>Return Type:</i> string
is_black_box	Returns a value of <code>true</code> if the cell is a black box cell. A cell is considered as a black box if the "function" definition of the cell is missing from the library. <i>Return type:</i> boolean
is_buffer	Returns a value of <code>true</code> if the associated library cell is recognized as a buffer cell. For example, <pre>get_property [get_lib_cells -of [get_cell AB_IT/E_0146_t1]] is_buffer</pre> <i>Return type:</i> boolean
is_combinational	Returns a value of <code>true</code> if the library cell is a combinational cell (not a sequential cell). <i>Return type:</i> boolean
is_dont_touch	Returns a value of <code>true</code> if the library cell definition includes <code>dont_touch:true</code>. <i>Return Type:</i> boolean
is_dont_use	Returns a value of <code>true</code> if the library cell description includes <code>dont_use:true</code>. <i>Return Type:</i> boolean
is_fall_edge_triggered	Returns a value of <code>true</code> if the library cell is triggered by the falling edge of the clock. <i>Return Type:</i> boolean
is_instantiated	Returns a value of <code>true</code> if library cells are instantiated in a design. For example, <pre>get_property [get_lib_cells P4_lib_v1.0/N4_X2] is_instantiated</pre> <i>Return type:</i> boolean
is_interface_timing	Returns a value of <code>true</code> if a library cell has interface timing specified for that cell. <i>Return type:</i> boolean
is_level_shifter	Returns a value of <code>true</code> if a cell instance is a level-shifter cell. <i>Return type:</i> boolean

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

is_integrated_clock_gating_cell	Returns a value of <code>true</code> if a cell is an integrated clock gating cell. For example, <pre>get_property [get_lib_cells CTS_HT.db/ICG_F7_HT] is_integrated_clock_gating_cell</pre> <i>Return type:</i> boolean
is_inverter	Returns a value of <code>true</code> if a cell instance is an inverter cell. For example, <pre>get_property [get_lib_cells -of [get_cells inv]] is_inverter</pre> <i>Return type:</i> boolean
is_level_shifter	Returns a value of <code>true</code> if a cell instance is a level-shifter cell. For example, <pre>get_property [get_lib_cells -of [get_libs mk_lpv_125c]] is_level_shifter</pre> <i>Return type:</i> boolean
is_macro_cell	Returns a value of <code>true</code> if a macro cell exists in a library. For example, <pre>get_property [get_lib_cells L_mro2/mro2y] is_macro_cell</pre> <i>Return type:</i> boolean
is_memory_cell	Returns a value of <code>true</code> if the cell is a memory cell. <i>Return type:</i> boolean Note: The library cells that have a memory group defined in the dotlib are considered as memory cells. For <pre>cell () { : memory() { type : ram ; address_width : 10 ; word_width : 7 ; } }</pre>
is_msv	Returns a value of <code>true</code> if a cell is an MSV cell. For example, <pre>get_property [get_lib_cells mk_lpv_125c/L8MTR] is_msv</pre> <i>Return type:</i> boolean
is_negative_level_sensitive	Returns a value of <code>true</code> if the library cell is used as a negative level-sensitive latch. <i>Return type:</i> boolean
is_pad_cell	Returns a value of <code>true</code> if the Liberty library <code>is_pad_cell</code> attribute is set to <code>true</code> for this pad cell. <i>Return type:</i> boolean
is_pll_cell	Returns a value of <code>true</code> if the Liberty library <code>is_pll_cell</code> attribute is set to <code>true</code> for this library cell. <i>Return type:</i> boolean
is_positive_level_sensitive	Returns a value of <code>true</code> if the library cell is used as a positive level-sensitive latch. <i>Return type:</i> boolean
is_rise_edge_triggered	Returns a value of <code>true</code> if the library cell is triggered by the rising edge of the clock. <i>Return Type:</i> boolean
is_sequential	Returns a value of <code>true</code> if the library cell is a latch or flip-flop, or if the cell has sequential timing checks. <i>Return type:</i> boolean

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

is_tristate	Returns a value of <code>true</code> if the library cell definition includes <code>three_state:true</code> . <i>Return type:</i> boolean
late_fall_clock_period_derate_factor	Returns the derate factor for the clock period when the <code>set_timing_derate -late</code> and <code>-fall</code> parameters are specified. <i>Return type:</i> float
late_fall_clock_period_mean_derate_factor	Returns the late/fall derate factor for the clock period when the <code>set_timing_derate -mean</code> parameter is specified. <i>Return type:</i> float
late_fall_clock_period_sigma_derate_factor	Returns the late/fall derate factor for the clock period when the <code>set_timing_derate -sigma</code> parameter is specified. <i>Return type:</i> float
late_fall_mean_slew_derate_factor	Returns the late/fall mean transition derate factor on the library cell. For example, <code>get_property [get_lib_cells FreePDK45_lib_v1.0/AND2_X1] late_fall_mean_slew_derate_factor</code> <i>Return type:</i> float
late_fall_pulse_width_derate_factor	Returns the late/fall derate factor for the pulse width when the <code>set_timing_derate -late</code> and <code>-fall</code> parameters are specified. <i>Return type:</i> float
late_fall_pulse_width_mean_derate_factor	Returns the late/fall derate factor for the pulse width when the <code>set_timing_derate -mean</code> parameter is specified. <i>Return type:</i> float
late_fall_pulse_width_sigma_derate_factor	Returns the late/fall derate factor for the pulse width when the <code>set_timing_derate -sigma</code> parameter is specified. <i>Return type:</i> float
late_fall_sigma_slew_derate_factor	Returns the late/fall sigma transition derate factor on the library cell. For example, <code>get_property [get_lib_cells FreePDK45_lib_v1.0/AND2_X1] late_fall_sigma_slew_derate_factor</code> <i>Return type:</i> float
late_fall_slew_derate_factor	Returns the derate factor applied on a library cell, when the <code>set_timing_derate -transition</code> and <code>-fall</code> parameters are specified. <i>Return type:</i> float
late_rise_clock_incremental_derate_factor	Returns the late clock path library cell check incremental derating factor specified through the <code>set_timing_derate -increment</code> parameter - when the <code>-rise</code> parameter is specified. <i>Return Type:</i> float
late_rise_data_incremental_derate_factor	Returns the late data path library cell check incremental derating factor specified through the <code>set_timing_derate -increment</code> parameter - when the <code>-rise</code> parameter is specified. <i>Return Type:</i> float
late_rise_clock_period_derate_factor	Returns the derate factor for the clock period when the <code>set_timing_derate -late</code> and <code>-rise</code> parameters are specified. <i>Return type:</i> float
late_rise_clock_period_mean_derate_factor	Returns the late/rise derate factor for the clock period when the <code>set_timing_derate -mean</code> parameter is specified. <i>Return type:</i> float
late_rise_clock_period_sigma_derate_factor	Returns the late/rise derate factor for the clock period when the <code>set_timing_derate -sigma</code> parameter is specified. <i>Return type:</i> float
late_rise_mean_slew_derate_factor	Returns the late/rise mean transition derate factor on the library cell. For example, <code>get_property [get_lib_cells FreePDK45_lib_v1.0/AND2_X1] late_rise_mean_slew_derate_factor</code> <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

late_rise_pulse_width_derate_factor	Returns the derate factor for the pulse width check when the <code>set_timing_derate -late</code> and <code>-rise</code> parameters are specified. <i>Return type:</i> float
late_rise_pulse_width_mean_derate_factor	Returns the late/rise derate factor for the pulse width when the <code>set_timing_derate -mean</code> parameter is specified. <i>Return type:</i> float
late_rise_pulse_width_sigma_derate_factor	Returns the late/rise derate factor for the pulse width when the <code>set_timing_derate -sigma</code> parameter is specified. <i>Return type:</i> float
late_rise_sigma_slew_derate_factor	Returns the late/rise sigma transition derate factor on the library cell. For example, <code>get_property [get_lib_cells FreePDK45_lib_v1.0/AND2_X1] late_rise_sigma_slew_derate_factor</code> <i>Return type:</i> float
late_rise_slew_derate_factor	Returns the derate factor applied on a library cell, when the <code>set_timing_derate -transition</code> and <code>-rise</code> parameters are specified. For example, <code>get_property [get_lib_cells PK4_v1.0/IV_32] late_rise_slew_derate_factor</code> <i>Return type:</i> float
mis_alignments_factor_min_fall	Returns the minimum fall tuning factor applied on a library cell, when the <code>set_multi_input_switching_mode</code> and <code>switching_alignment_factor</code> parameters are specified. <i>Return type:</i> float
mis_alignments_factor_min_rise	Returns the minimum rise tuning factor applied on a library cell, when the <code>set_multi_input_switching_mode</code> and <code>switching_alignment_factor</code> parameters are specified. <i>Return type:</i> float
master_physical_variant_cell	Returns the master library cell of the specified physical variant cell. For example, <code>get_property [get_lib_cell cell_w/INV2] master_physical_variant_cell</code> <i>Return type:</i> string
name	Returns the leaf-level name of the library cell. <i>Return Type:</i> string
object_type	Returns the object type <code>lib_cell</code> . <i>Return Type:</i> string
physical_variant_cells	Returns the physical variant cells that are available in a library cell. For example, <code>get_property [get_lib_cells lca_md/SR1_md] physical_variant_cells</code> <i>Return Type:</i> string
pin_count	Returns the total number of signal pins. For example, <code>get_property [get_lib_cells PK45_v1.0/IN_X32] pin_count</code> <i>Return Type:</i> integer
receiver_capacitance_type	Returns the value of input capacitance group being used for delay calculations – C1C2 (2-piece model) or C1 (1-piece model). <i>Return Type:</i> string
retention_cell	Returns the value of the <code>retention_cell</code> attribute specified in the liberty. <i>Return type:</i> string

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

rise_input_switching_derate_factor	Returns the derate factor applied on a library cell, when the <code>set_timing_derate -input_switching</code> parameter is specified. For example, <pre>get_property [get_lib_cells PK45_v1.0/DL_X2] rise_input_switching_derate_factor</pre> <i>Return type:</i> float
signal_pins	Returns the signal pins of a cell, along with the library name, library cell name, and pin name. For example, <pre>get_property [get_lib_cells PK45_v1.0/IN_X32] signal_pins</pre> <i>Return Type:</i> collection
sync_clear	Returns the collection of synchronous clear pins. Note: For a synchronous clear pin: - The flip-flop's function definition should have the clear signal in the <code>next_state</code> attribute's logic expression. - The pin corresponding to the synchronous clear input must specify: <code>nextstate_type: clear</code> For example, <pre>get_property [get_lib_cells tcbn05_bwph21016p51cnod_base_svtffgnp_0p825v_125c_cbest_CCbest_T_em_ccs/SDFKSRPQD1BWP210H6P51C] sync_clear</pre> <i>Return type:</i> collection
sync_enable	Returns the collection of synchronous enable pins. For example, <pre>get_property [get_lib_cells tcbn05_bwph21016p51cnod_base_ulvtl1ffgnp_0p825v_125c_cbest_CCbest_T_em_ccs/EDFQD1BWP210H6P51C] sync_enable</pre> <i>Return type:</i> collection
sync_preset	Returns the collection of synchronous preset pins. For example, <pre>get_property [get_lib_cells tcbn05_bwph21016p51cnod_base_svtffgnp_0p825v_125c_cbest_CCbest_T_em_ccs/SDFKSRPQD1BWP210H6P51C] sync_preset</pre> <i>Return type:</i> collection
test_scan_enable	Returns the collection of the <code>test_scan_enable</code> pins. For example, <pre>get_property [get_lib_cell SR80_W_125_0.855_0.855_CORE_SVT.db/sdffbqs_f4_12t_svt] test_scan_enable</pre> <i>Return type:</i> collection
test_scan_enable_inverted	Returns the collection of the <code>test_scan_enable_inverted</code> pins. For example, <pre>get_property [get_lib_cells SR80_W_125_0.855_0.855_CORE_SVT.db/sdffbqs_f4_12t_svt] test_scan_enable_inverted</pre> <i>Return type:</i> collection
test_scan_in	Returns the collection of the <code>test_scan_in</code> pins. For example, <pre>get_property [get_lib_cells SR80_W_125_0.855_0.855_CORE_SVT.db/sdffbqs_f4_12t_svt] test_scan_in</pre> <i>Return type:</i> collection
test_scan_in_inverted	Returns the collection of the <code>test_scan_in_inverted</code> pins. For example, <pre>get_property [get_lib_cell SR80_W_125_0.855_0.855_CORE_SVT.db/sdffbqs_f4_12t_svt] test_scan_in_inverted</pre> <i>Return type:</i> collection

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

test_scan_out	Returns the collection of the test_scan_out pins. For example, <pre>get_property [get_lib_cells SR80_W_125_0.855_0.855_CORE_SVT.db/sdffbqs_f4_12t_svt] test_scan_out</pre> <i>Return type:</i> collection
test_scan_out_inverted	Returns the collection of the test_scan_out_inverted pins. For example, <pre>get_property [get_lib_cell SR80_W_125_0.855_0.855_CORE_SVT.db/sdffbqs_f4_12t_svt] test_scan_out_inverted</pre> <i>Return type:</i> collection
output_signal_level_high	Returns the maximum voltage value of an output pin after a rising transition. For example, <pre>get_property [get_lib_arcs -of [get_lib_cells px_ipid_3/px_ipid_3]] output_signal_level_high</pre> <i>Return type:</i> float
output_signal_level_low	Returns the minimum voltage value of an output pin after a falling transition. For example, <pre>get_property [get_lib_arcs -of [get_lib_cells px_ipid_3/px_ipid_3]] output_signal_level_low</pre> <i>Return type:</i> float
threshold_voltage_group	Returns the voltage group defined at the library level or a library cell specific group definition (specified using the set_voltage_threshold_group command). For example, <pre>get_property [get_lib_cells L40_HT.db/CG_F8_HT] threshold_voltage_group</pre> <i>Return type:</i> string
timing_model_type	Returns the model type for a given cell or instance. The supported values are abstracted, extracted, and extracted_and_abstracted. <i>Return type:</i> string
ground_pins	Returns a collection of the ground pins of a library cell. For example, <pre>get_property [get_lib_cells sc_cmoslp/A022X] ground_pins</pre> <i>Return type:</i> collection
is_always_on	Returns a value of true if the always_on attribute of a library cell is set to true indicating that the library cell is always-on cell. For example, <pre>get_property [get_lib_cells sc_cmoslp/A0N22_X4M] is_always_on</pre> <i>Return type:</i> boolean
is_isolation_cell	Returns a value of true if the library cell is an isolation cell. For example, <pre>get_property [get_lib_cells sc_cmoslp/ISO22_X4M] is_isolation_cell</pre> <i>Return type:</i> boolean
is_power_switch	Returns a value of true when the switch_cell_type attribute is defined for a library cell indicating that the library cell is a power-switch cell. For example, <pre>get_property [get_lib_cells sc_cmoslp/A022X] is_power_switch</pre> <i>Return type:</i> boolean
is_retention	Returns a value of true when the retention_cell attribute is defined for a library cell indicating that the library cell is a retention cell. For example, <pre>get_property [get_lib_cells sc_cmoslp/RET22_X4M] is_retention</pre> <i>Return type:</i> boolean

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

level_shifter_type	Returns the type of level-shifter (e.g., LH, HL, or HL_LH) as defined by the <code>level_shifter_type</code> attribute of cell. For example, <pre>get_property [get_lib_cells sc_cmoslp/A2LVLD_X4M] level_shifter_type</pre> <i>Return type:</i> string
lib_pg_pins	Returns a collection of all PG pins (both power and ground) defined in the library of a library cell. For example, <pre>get_property [get_lib_cells sc_cmoslp/A022X] lib_pg_pins</pre> <i>Return type:</i> collection
power_pins	Returns a collection of the power pins of a library cell. For example, <pre>get_property [get_lib_cells sc_cmoslp/A022X] power_pins</pre> <i>Return type:</i> collection
power_switch_type	Returns the type of power-switch (e.g., coarse-grain, fine-grain) as defined by the <code>switch_cell_type</code> attribute of library cells. For example, <pre>get_property [get_lib_cells sc_cmoslp/A022X] power_switch_type</pre> <i>Return type:</i> string
std_cell_main_rail_name	Returns the name of a power rail that has the <code>std_cell_main_rail</code> library attribute defined as <code>true</code> for a library cell. <i>Return type:</i> string For example, <pre>get_property [get_lib_cells sc_cmoslp/A022X] std_cell_main_rail_name</pre>

Library Pin Properties

all_q_pin_of_d_pin	Returns a collection of <code>all_q_pin_of_d_pin</code> pins. For example, <pre>get_property [get_lib_pin srpg/SDRFFSRPQ_X1/D] all_q_pin_of_d_pin</pre> <i>Return type:</i> collection
capacitance	Returns the capacitance for the library pin. <i>Return type:</i> float
capacitance_max_fall	Returns the maximum value of the falling capacitance range. <i>Return type:</i> float
capacitance_max_rise	Returns the maximum value of the rising capacitance range. <i>Return type:</i> float
capacitance_min_fall	Returns the minimum value of the falling capacitance range. <i>Return type:</i> float
capacitance_min_rise	Returns the minimum value of the rising capacitance range. <i>Return type:</i> float
cell_name	Returns the library cell name for the corresponding library pins. For example, <pre>get_property [get_lib_pins PDK45_lib_v1.0/A0I211_X2/B] cell_name</pre> <i>Return type:</i> string
direction	Returns the direction of the library pin: <code>in</code>, <code>out</code>, or <code>inout</code>. <i>Return type:</i> string

Litmus Text Command Reference
Advanced Timing Tcl Scripting Commands--get_property

drive_resistance_fall	Specifies the driving resistance of the cell for falling transitions. For example, <pre>get_property [get_lib_pins FreePDK45_lib_v1.0/BUF_X1/Z] drive_resistance_fall</pre> <i>Return type:</i> float
drive_resistance_rise	Specifies the driving resistance of the cell for rising transitions. For example, <pre>get_property [get_lib_pins FreePDK45_lib_v1.0/BUF_X1/Z] drive_resistance_rise</pre> <i>Return type:</i> float
fanout_load	Returns the fanout load value of the library pin. <i>Return type:</i> float
function	Returns the function for the corresponding library pins. For example, <pre>get_property [get_lib_pins PK4_lib_v1.0/AND2_X1/ZN] function</pre> <i>Return type:</i> string
hierarchical_name	Returns the hierarchical name of the library pin. <i>Return type:</i> string
input_signal_level	Returns the power rail name/signal that is defined for a library cell input pin. For example, <pre>get_property [get_lib_pins PK4_lib_v1.0/BUF_X32/A] input_signal_level</pre> <i>Return type:</i> string
input_signal_level_high	Returns the upper limit of partial voltage swing value that is defined for a library cell input pin. For example, <pre>get_property [get_lib_pins rvt_ss_0p9v/LVLL/A] input_signal_level_high</pre> <i>Return type:</i> float
input_signal_level_low	Returns the lower limit of partial voltage swing value that is defined for a library cell input pin. For example, <pre>get_property [get_lib_pins rvt_ss_0p9v/LVLL/A] input_signal_level_low</pre> <i>Return type:</i> float
is_analog	Returns a value of <code>true</code> if the library pin is an analog pin. For example, <pre>get_property [get_lib_pins PK4_lib_v1.0/AOI222_X2/ZN] is_analog</pre> <i>Return type:</i> boolean
is_async	Returns a value of <code>true</code> if the library pin is an asynchronous preset pin, or an asynchronous clear pin. <i>Return type:</i> boolean
is_clear	Returns a value of <code>true</code> if the pin is an asynchronous clear pin. <i>Return type:</i> boolean
is_clock	Returns a value of <code>true</code> if the library pin definition includes <code>clock:true</code>. <i>Return type:</i> boolean
is_clock_gating_clock	Returns a value of <code>true</code> if the library pin is defined as a clock pin of a clock gating cell. For example, <pre>get_property [get_lib_pins P40_CTS_HVT.db/ICG_HVT/CLKIN] is_clock_gating_clock</pre> <i>Return type:</i> boolean

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

is_clock_gating_enable	Returns a value of <code>true</code> if the library pin corresponds to the enable pin of a clock gating cell. For example, <pre>get_property [get_lib_pins P40_CTS_HVT.db/ICG_HVT/CLKIN] is_clock_gating_enable</pre> <i>Return type:</i> boolean
is_clock_gating_test	Returns a value of <code>true</code> if the pin corresponds to the test pin of a clock gating cell. For example, <pre>get_property [get_lib_pins */NAND2_X2/A2] is_clock_gating_test</pre> <i>Return type:</i> boolean
is_clock_pin	Returns a value of <code>true</code> if the library pin is a clock pin. For example, <pre>get_property [get_lib_pins srpg/SDRFFSRPQ_X1/CK] is_clock_pin</pre> <i>Return Type:</i> boolean
is_data	Returns a value of <code>true</code> if the library pin with defined sequential arcs is the data pin of a flip-flop. <i>Return type:</i> boolean
is_fall_edge_triggered_clock	Returns a value of <code>true</code> if the library pin is the clock pin of a falling edge triggered device. <i>Return type:</i> boolean
is_fall_edge_triggered_data	Returns a value of <code>true</code> if the library pin is the data pin of a falling edge triggered device. <i>Return type:</i> boolean
is_iq_function	Returns a value of <code>true</code> if the output is an IQ pin. For example, <pre>get_property [get_lib_pins */SDFFRS_X1/Q] is_iq_function</pre> <i>Return Type:</i> boolean
is_iqn_function	Returns a value of <code>true</code> if the output is an IQN pin. For example, <pre>get_property [get_lib_pins */SDFFRS_X1/QN] is_iqn_function</pre> <i>Return Type:</i> boolean
is_isolation_cell_clock	Returns a value of <code>true</code> if the clock pin is a pin of isolation cell clock. For example, <pre>get_property [get_lib_pins cell_w/INV/CLK] is_isolation_cell_clock</pre> <i>Return Type:</i> boolean
is_negative_level_sensitive_clock	Returns a value of <code>true</code> if the library pin is an enable pin of an active low level-sensitive device. <i>Return type:</i> boolean
is_negative_level_sensitive_data	Returns a value of <code>true</code> if the pin is a data pin of an active low level-sensitive device. <i>Return type:</i> boolean
is_pad	Returns a value of <code>true</code> if the library pin is a pad pin. For example, <pre>get_property [get_lib_pins *] is_pad</pre> <i>Return type:</i> boolean
is_pll_feedback_pin	Returns a value of <code>true</code> if the Liberty library <code>is_pll_feedback_pin</code> attribute is set to <code>true</code> for this library pin. <i>Return type:</i> boolean
is_pll_output_pin	Returns a value of <code>true</code> if the Liberty library <code>is_pll_output_pin</code> attribute is set to <code>true</code> for this library pin. <i>Return type:</i> boolean

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

<code>is_pll_reference_pin</code>	Returns a value of <code>true</code> if the Liberty library <code>is_pll_reference_pin</code> attribute is set to <code>true</code> for this library pin. <i>Return type:</i> boolean
<code>is_positive_level_sensitive_clock</code>	Returns a value of <code>true</code> if the library pin is an enable pin of an active high level-sensitive device. <i>Return type:</i> boolean
<code>is_positive_level_sensitive_data</code>	Returns a value of <code>true</code> if the pin is a data pin of an active high level-sensitive device. <i>Return type:</i> boolean
<code>is_preset</code>	Returns a value of <code>true</code> if the pin is a preset pin. <i>Return type:</i> boolean
<code>is_ref_clock_pin</code>	Returns a value of <code>true</code> : • If the library pin is a reference clock pin. • Valid instance pin object and on an active clock pin where a clock signal reaches, and sequential cell is not disabled by disabled timing arcs or case analysis. Any pin with clock check-arcs or trigger-arcs will also be reported, if an active clock phase is present on that pin. • Clock pin on an inferred clock gating cell. • Valid <code>lib_pin</code> object, if there is at least one instance of this <code>lib_pin</code> in the design, and if <code>is_clock_pin</code> property is set to <code>true</code>. Returns a value of <code>false</code> , if clock propagation is disabled with the <code>set_disable_timing</code> or <code>set_sense - stop_propagation</code> command. For example, <pre>get_property [get_lib_pins srpg/SDRFFSRPQ_X1/CK] is_ref_clock_pin</pre> <i>Return Type:</i> boolean
<code>is_rise_edge_triggered_clock</code>	Returns a value of <code>true</code> if the library pin is the clock pin of a rising edge triggered device. <i>Return type:</i> boolean
<code>is_rise_edge_triggered_data</code>	Returns a value of <code>true</code> if the library pin is the data pin of a rising edge triggered device. <i>Return type:</i> boolean
<code>is_tristate</code>	Returns a value of <code>true</code> if the library pin definition includes <code>three_state:true</code> . <i>Return type:</i> boolean
<code>max_capacitance</code>	Returns the maximum capacitance limit for the library pin. <i>Return type:</i> float
<code>max_fanout</code>	Returns the maximum fanout value for the library pin. <i>Return type:</i> float
<code>max_transition</code>	Returns the maximum transition time limit specified for a given library pin. If the limit is not specified for a library pin, then the limit specified at the corresponding library level will be used. <i>Return type:</i> float
<code>min_transition</code>	Returns the minimum transition time limit specified for a given library pin. If the limit is not specified for a library pin, then the limit specified at the corresponding library level will be used. <i>Return type:</i> float
<code>min_capacitance</code>	Returns the minimum capacitance limit for the library pin. <i>Return type:</i> float
<code>min_fanout</code>	Returns the minimum fanout value for the library pin. <i>Return type:</i> float
<code>name</code>	Returns the leaf-level name of the library cell pin. <i>Return type:</i> string

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

object_type	Returns the object type; in this case, lib_pin. <i>Return type:</i> string
related_ground_pin_name	Returns related ground pin names for the corresponding library pins. <i>Return type:</i> string For example, <pre>get_property [get_lib_pins sc_cmoslp/A022X/I1] related_ground_pin_name</pre>
related_power_pin_name	Returns related power pin names for the corresponding library pins. <i>Return type:</i> string
slew_threshold_percent_fall_high	Returns the upper slew trip point for falling waveforms, specified as a percent. <i>Return type:</i> float
slew_threshold_percent_fall_low	Returns the lower slew trip point for falling waveforms, specified as a percent. <i>Return type:</i> float
slew_threshold_percent_rise_high	Returns the upper slew trip point for rising waveforms, specified as a percent. <i>Return type:</i> float
slew_threshold_percent_rise_low	Returns the lower slew trip point for rising waveforms, specified as a percent. <i>Return type:</i> float
input_signal_level_voltage	Returns the voltage value corresponding to the power rail name/signal which is defined for the library-cell input pin. This voltage value is the full rail-to-rail voltage value that is associated with the relative power rail name through the voltage_map attribute defined in the library. For example, <pre>get_property [get_lib_pins sc_cmoslp/A022X/I1] input_signal_level_voltage</pre> <i>Return type:</i> float
is_always_on	Returns a value of true if the always_on attribute of a library signal pin is set to true indicating that the library cell signal pin is an always-on pin. For example, <pre>get_property [get_lib_pins sc_cmoslp/A022X/I1] is_always_on</pre> <i>Return type:</i> boolean
is_isolated	Returns a value of true if the is_isolated attribute of a library signal pin is set to true indicating that the library cell signal pin is an internally isolated pin. For example, <pre>get_property [get_lib_pins sc_cmoslp/A022X/I1] is_isolated</pre> <i>Return type:</i> boolean
is_isolation_cell_data	Returns a value of true if the isolation_cell_data_pin attribute of a library signal pin is set to true indicating that the library cell signal pin is a data pin of an isolation library cell. For example, <pre>get_property [get_lib_pins sc_cmoslp/A022X/I1] is_isolation_cell_data</pre> <i>Return type:</i> boolean
is_isolation_cell_enable	Returns a value of true if the isolation_cell_enable_pin attribute of a library signal pin is set to true indicating that the library cell signal pin is an enable or control pin of an isolation library cell. For example, <pre>get_property [get_lib_pins sc_cmoslp/A022X/I1] is_isolation_cell_enable</pre> <i>Return type:</i> boolean

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

is_level_shifter_data	Returns a value of <code>true</code> if the <code>level_shifter_data_pin</code> attribute of a library signal pin is set to <code>true</code>, indicating that the library cell signal pin is a data pin of a library level-shifter cell. For example, <code>get_property [get_lib_pins sc_cmoslp/A022X/I1] is_level_shifter_data</code> <i>Return type:</i> boolean
is_level_shifter_enable	Returns a value of <code>true</code> if the <code>level_shifter_enable_pin</code> attribute of a library signal pin is set to be <code>true</code> indicating that the library-cell signal pin is an enable pin of a library level shifter cell. For example, <code>get_property [get_lib_pins sc_cmoslp/A022X/I1] is_isolation_cell_enable</code> <i>Return type:</i> boolean
is_power_switch_enable	Returns a value of <code>true</code> if the <code>switch_cell_type</code> attribute of the corresponding library cell is defined and the <code>switch_pin</code> attribute of the given library-cell signal pin is defined to be <code>true</code>. For example, <code>get_property [get_lib_pins sc_cmoslp/A022X/I1] is_power_switch_enable</code> <i>Return type:</i> boolean
is_retention_cell_enable	Returns a value of <code>true</code> if the <code>retention_pin</code> attribute of a signal pin is defined. For example, <code>get_property [get_lib_pins sc_cmoslp/A022X/I1] is_retention_cell_enable</code> <i>Return type:</i> boolean
output_signal_level	Returns the power rail name/signal that is defined for a library cell output and inout pins. For example, <code>get_property [get_lib_pins sc_cmoslp/A022X/I1] output_signal_level</code> <i>Return type:</i> string
output_signal_level_high	Returns the upper/higher limit of partial voltage swing value that is defined for a library cell output and inout pins. For example, <code>get_property [get_lib_pins sc_cmoslp/A022X/I1] output_signal_level_high</code> <i>Return type:</i> float
output_signal_level_low	Returns the lower limit of partial voltage swing value that is defined for a library cell output and inout pins. For example, <code>get_property [get_lib_pins sc_cmoslp/A022X/I1] output_signal_level_low</code> <i>Return type:</i> float
pin_capacitance	Returns the pin capacitance. For example, <code>get_property [get_lib_pins FreePDK45_lib_v1.0/BUF_X1/A] pin_capacitance</code> <i>Return type:</i> float
pin_capacitance_max_fall	Returns the maximum value of the falling capacitance range of the corresponding library pin. For example, <code>get_property [get_lib_pins FreePDK45_lib_v1.0/BUF_X1/A] pin_capacitance_max_fall</code> <i>Return type:</i> float
pin_capacitance_max_rise	Returns the maximum value of the rising capacitance range of the corresponding library pin. For example, <code>get_property [get_lib_pins FreePDK45_lib_v1.0/BUF_X1/A] pin_capacitance_max_rise</code> <i>Return type:</i> float
pin_capacitance_min_fall	Returns the minimum value of the falling capacitance range of the corresponding library pin. For example, <code>get_property [get_lib_pins FreePDK45_lib_v1.0/BUF_X1/A] pin_capacitance_min_fall</code> <i>Return type:</i> float

Litmus Text Command Reference
Advanced Timing Tcl Scripting Commands--get_property

pin_capacitance_min_rise	Returns the minimum value of the rising capacitance range of the corresponding library pin. For example, get_property [get_lib_pins FreePDK45_lib_v1.0/BUF_X1/A] pin_capacitance_min_rise <i>Return type:</i> float
related_ground_pin	Returns the related ground pins as a collection for the corresponding library cell pins. For example, get_property [get_lib_pins sc_cmoslp/A022X/I1] related_ground_pin <i>Return type:</i> collection
related_ground_pin_name	Returns the related ground pin names for the corresponding library cell pins. For example, get_property [get_lib_pins sc_cmoslp/A022X/I1] related_ground_pin_name <i>Return type:</i> string
related_ground_pin_rail_voltage	Returns the voltage value corresponding to the ground rail name defined for the related_ground_pin attribute of the library cell pin. This voltage value is the full rail-to-rail value that is associated with the relative ground rail name through the voltage_map attribute defined in the library. For example, get_property [get_lib_pins sc_cmoslp/A022X/I1] related_ground_pin_rail_voltage <i>Return type:</i> float
related_power_pin	Returns the related power pins as a collection for the corresponding library cell pins. For example, get_property [get_lib_pins sc_cmoslp/A022X/I1] related_power_pin <i>Return type:</i> collection
related_power_pin_name	Returns the related power pin names for the corresponding library cell pins. For example, get_property [get_lib_pins sc_cmoslp/A022X/I1] related_power_pin_name <i>Return type:</i> string
related_power_pin_rail_voltage	Returns the voltage value corresponding to the power rail name defined for the related_power_pin attribute of a library cell pin. This voltage value is the full rail-to-rail value that is associated with the relative power rail name through the voltage_map attribute defined in the library. For example, get_property [get_lib_pins sc_cmoslp/A022X/I1] related_power_pin_rail_voltage <i>Return type:</i> float

Litmus Text Command Reference
Advanced Timing Tcl Scripting Commands--get_property

Library PG Pin Properties

is_std_cell_main_rail	Returns a value of <code>true</code> if the library power PG pin is the main rail of the library cell. This is determined by the <code>std_cell_main_rail</code> library attribute of the library <code>pg_pin</code> . <i>Return type:</i> boolean For example, <code>get_property [get_lib_pg_pins -of_objects [get_lib_cells sc_cmoslp/A022X]] is_std_cell_main_rail</code>
name	Returns the name of the library PG pin as defined in the library and associated with the <code>pg_pin</code> library attribute. <i>Return type:</i> string For example, <code>get_property [get_lib_pg_pins -of_objects [get_lib_cells sc_cmoslp/A022X]] name</code>
object_type	Returns <code>lib_pg_pin</code> as the object type for a library PG pin. <i>Return type:</i> string For example, <code>get_property [get_lib_pg_pins -of_objects [get_lib_cells sc_cmoslp/A022X]] object_type</code>
pg_type	Returns the type of the library PG pin as associated with the <code>pg_type</code> attribute of the corresponding library PG pin. <i>Return type:</i> string For example, <code>get_property [get_lib_pg_pins -of_objects [get_lib_cells sc_cmoslp/A022X]] pg_type</code> <code>get_property [get_lib_pg_pins -of_objects [get_lib_cells sc_cmoslp/A022X] -filter " name =~ *VDD*"] pg_type</code>
rail_name	Returns the name of the rail to which the PG pin is connected and is associated with the <code>voltage_name</code> attribute of the library PG pin. <i>Return type:</i> string For example, <code>get_property [get_lib_pg_pins -of_objects [get_lib_cells sc_cmoslp/A022X]] rail_name</code>
rail_voltage	Returns the voltage from the <code>voltage_map</code> association of the rail to which the library PG pin is connected. <i>Return type:</i> float For example, <code>get_property [get_lib_pg_pins -of_objects [get_lib_cells sc_cmoslp/A022X]] rail_voltage</code>
tied_to	Returns the library PG pin which is connected to substrate-bias PG pin. <i>Return type:</i> string For example, <code>get_property [get_lib_pg_pins -of_objects [get_lib_cells sc_cmoslp/A022X]] tied_to</code>

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

Library Timing Arc Properties

from_lib_pin	Returns a collection of the source library pins of the library timing arc. <i>Return type:</i> collection
has_ccs_noise	Returns a value of <code>true</code> if a cell has CCS noise data. For example, <code>get_property [get_lib_arc -from ccs_timing_si/AN2D1BWP20P90/A1 -to ccs_timing_si/AN2D1BWP20P90/Z] has_ccs_noise</code> <i>Return type:</i> boolean
has_mis_fall	Returns a value of <code>true</code> , if MIS (multi-input switching) derate fall value is specified for library timing arcs. For example, <code>get_property [get_lib_arc -from FreePDK45_lib_v1.0/AND2_X1/A1 -to FreePDK45_lib_v1.0/AND2_X1/ZN] has_mis_fall</code> <i>Return type:</i> boolean
has_mis_rise	Returns a value of <code>true</code> , if MIS (multi-input switching) derate rise value is specified for library timing arcs. For example, <code>get_property [get_lib_arcs -from FreePDK45_lib_v1.0/OR2_X1/A1 -to FreePDK45_lib_v1.0/OR2_X1/ZN] has_mis_rise</code> <i>Return type:</i> boolean
has_socv	Returns a value of <code>true</code> if a timing library arc in the library has a spatial derate table (SOCV). <i>Return type:</i> boolean
is_disabled	Return a value of <code>true</code> if a <code>set_disable_timing</code> constraint exists for an arc. <i>Return type:</i> boolean
mode	Returns the name of the group if the timing arc is defined as part of a Liberty mode group. For example, <code>get_property [get_lib_arc -from c065lesram76x2x1x96b_PttV1200T025_lib/c065lesram76x2x1x96b/CL -to c065lesram76x2x1x96b_PttV1200T025_lib/c065lesram76x2x1x96b/RV] mode</code> <i>Return type:</i> string
object_type	Returns the object type <code>lib_timing_arc</code> . <i>Return type:</i> string
sdf_cond	Returns the <code>sdf_cond</code> expression specified for the timing library arc in the library. <i>Return type:</i> string
sense	Returns the <code>timing_sense</code> specified for the timing library arc in the library. <i>Return type:</i> string
to_lib_pin	Returns a collection of the sink library pins of the library timing arc. <i>Return type:</i> collection
timing_type	Returns the type specified for the timing library arc in the library. <i>Return type:</i> string
when	Returns the <code>when</code> condition specified for the library timing arc in the library. <i>Return type:</i> string
when_library_string	Returns the exact value of the Liberty 'when' attribute for the specified arc. For example, <code>get_property [get_lib_arc -from FreePDK45_lib_v1.0/BUF_X1/A -to FreePDK45_lib_v1.0/BUF_X1/Z] when_library_string</code> <i>Return type:</i> string

Net Properties

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

<code>annotated_capacitance_max</code>	Returns the maximum annotated capacitance of the net for late path analysis. For example, <code>get_property [get_nets w2] annotated_capacitance_max</code> <i>Return type:</i> float
<code>annotated_capacitance_min</code>	Returns the minimum annotated capacitance of the net for late path analysis. For example, <code>get_property [get_nets w2] annotated_capacitance_min</code> <i>Return type:</i> float
<code>attackers</code>	Reports the parasitic data on a net - capacitance, resistance, or cross-coupling. <i>Return type:</i> collection
<code>capacitance_max</code>	Returns the total capacitance of the net for maximum conditions. <i>Return type:</i> float
<code>coupling_capacitance_max</code>	Returns the maximum value of coupling capacitance of a net over all the views. <i>Return type:</i> float
<code>capacitance_min</code>	Returns the total capacitance of the net for minimum conditions. <i>Return type:</i> float
<code>coupling_capacitance_min</code>	Returns the minimum value of coupling capacitance of a net over all the views. <i>Return type:</i> float
<code>delta_delay_max</code>	Returns the delta delay for the maximum condition. <i>Return type:</i> float
<code>delta_delay_max_fall</code>	Returns the maximum falling delta delay value. <i>Return type:</i> float
<code>delta_delay_max_rise</code>	Returns the maximum rising delta delay value. <i>Return type:</i> float
<code>delta_delay_min</code>	Returns the delta delay for the minimum condition. <i>Return type:</i> float
<code>delta_delay_min_fall</code>	Returns the minimum falling delta delay value. <i>Return type:</i> float
<code>delta_delay_min_rise</code>	Returns the minimum rising delta delay value. <i>Return type:</i> float
<code>driver_pins</code>	Returns a collection of the driver pins of the net. <i>Return type:</i> collection
<code>early_fall_clk_net_derate_factor</code>	Returns the early fall clock net derating factor specified through <code>set_timing_derate</code> command. <i>Return type:</i> float
<code>early_fall_clk_net_delta_derate_factor</code>	Returns the early fall clock delta delay net derating factor specified through the <code>set_timing_derate</code> command. <i>Return type:</i> float
<code>early_fall_data_net_delta_derate_factor</code>	Returns the early fall data delta net derating factor specified through the <code>set_timing_derate</code> command. <i>Return type:</i> float
<code>early_fall_data_net_derate_factor</code>	Returns the early fall data net derating factor specified through <code>set_timing_derate</code> command. <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

<code>early_rise_clk_net_delta_derate_factor</code>	Returns the early rise clock delta net derating factor specified through the <code>set_timing_derate</code> command. <i>Return type:</i> float
<code>early_rise_clk_net_derate_factor</code>	Returns the early rise clock net derating factor specified through the <code>set_timing_derate</code> command. <i>Return type:</i> float
<code>early_rise_data_net_delta_derate_factor</code>	Returns the early rise data delta net derating factor specified through the <code>set_timing_derate</code> command. <i>Return type:</i> float
<code>early_rise_data_net_derate_factor</code>	Returns the early rise data net derating factor specified through the <code>set_timing_derate</code> command. <i>Return type:</i> float
<code>has_detailed_parasitics</code>	Returns a value of <code>true</code> if the net, or one of its parts, has detailed parasitics associated with it from either RC extraction or SPEF annotation. <i>Return type:</i> boolean
<code>hierarchical_name</code>	Returns the complete hierarchical name of the net. <i>Return type:</i> string
<code>is_clock_source_latency</code>	Returns a value of <code>true</code> if the net is a part of clock source latency network. The clock source latency network consists of the clock network cells, its pins, and the nets connecting the master clock source pin and the generated clock source pin. For example, <code>get_property [get_nets A] is_clock_source_latency</code> <i>Return type:</i> boolean
<code>is_dont_touch</code>	Returns a value of <code>true</code> if a <code>set_dont_touch</code> constraint exists for the net. <i>Return type:</i> boolean
<code>is_hierarchical</code>	Returns a value of <code>true</code> if the net is a part of hierarchical module boundary. For example, <code>get_property [get_nets W1] is_hierarchical</code> <i>Return type:</i> boolean
<code>is_ideal</code>	Returns a value of <code>true</code> if <code>set_ideal_latency</code> , <code>set_ideal_transition</code> , or <code>set_ideal_network</code> constraints exist for the net. <i>Return type:</i> Boolean
<code>late_fall_clk_net_delta_derate_factor</code>	Returns the late fall clock delta net derating factor specified through the <code>set_timing_derate</code> command. <i>Return type:</i> float
<code>late_fall_clk_net_derate_factor</code>	Returns the late fall clock net derating factor specified through the <code>set_timing_derate</code> command. <i>Return type:</i> float
<code>late_fall_data_net_delta_derate_factor</code>	Returns the late fall data delta net derating factor specified through the <code>set_timing_derate</code> command. <i>Return type:</i> float
<code>late_fall_data_net_derate_factor</code>	Returns the late fall data net derating factor specified through the <code>set_timing_derate</code> command. <i>Return type:</i> float
<code>late_rise_clk_net_delta_derate_factor</code>	Returns the late rise clock delta net derating factor specified through the <code>set_timing_derate</code> command. <i>Return type:</i> float
<code>late_rise_clk_net_derate_factor</code>	Returns the late rise clock net derating factor specified through the <code>set_timing_derate</code> command. <i>Return type:</i> float
<code>late_rise_data_net_delta_derate_factor</code>	Returns the late rise data delta net derating factor specified through the <code>set_timing_derate</code> command. <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

late_rise_data_net_derate_factor	Returns the late rise data net derating factor specified through the <code>set_timing_derate</code> command. <i>Return type:</i> float
load_pins	Returns a collection of the load pins of the net. <i>Return type:</i> collection
lumped_capacitance_max	Returns the maximum value of capacitance of a net based on the input library and parasitic model. <i>Return type:</i> float
lumped_capacitance_min	Returns the minimum value of capacitance of a net based on the input library and parasitic model. <i>Return type:</i> float
lumped_resistance_max	Returns the maximum resistance of the net for late path analysis. You can use <code>-index</code> to return the value for a specific view. For example, <code>get_property [get_nets FE_SIG_N671_FE_OFCN486_ds_data_6_] lumped_resistance_max</code> <i>Return type:</i> float
lumped_resistance_min	Returns the minimum resistance of the net for late path analysis. You can use <code>-index</code> to return the value for a specific view. For example, <code>get_property [get_nets FE_SIG_N671_FE_OFCN486_ds_data_6_] lumped_resistance_min</code> <i>Return type:</i> float
name	Returns the name of the net. <i>Return type:</i> string
num_driver_pins	Returns the number of driver pins on a net. <i>Return type:</i> integer
num_load_pins	Returns the number of load pins on the net. <i>Return type:</i> integer
object_type	Returns the object type <code>net</code> . <i>Return type:</i> string
pin_capacitance_max	Returns the sum of all pin capacitances of the net for maximum conditions. <i>Return type:</i> float
pin_capacitance_max_fall	Returns the sum of all pin capacitances of the net for the maximum value of the falling capacitance range. <i>Return type:</i> float
pin_capacitance_max_rise	Returns the sum of all pin capacitances of the net for the maximum value of the rising capacitance range. <i>Return type:</i> float
pin_capacitance_min	Returns the sum of all pin capacitances of the net for minimum conditions. <i>Return type:</i> float
Note: For all the <code>pin_capacitance_*</code> properties (described above), the pin capacitance matches with the capacitance displayed under the <code>pin_load</code> column of the <code>report_timing</code> output.	
pin_capacitance_min_fall	Returns the minimum fall capacitance present at the pin of a net. For example, <code>get_property [get_nets UMC/w3] pin_capacitance_min_fall</code> <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

pin_capacitance_min_rise	Returns the minimum rise capacitance present at the pin of a net. For example, <code>get_property [get_nets UMC/w3] pin_capacitance_min_rise</code> <i>Return type:</i> float
resistance_max	Returns the maximum value of resistance of a net. <i>Return type:</i> float
resistance_min	Returns the minimum value of resistance of a net. <i>Return type:</i> float
timing_constant	Reports a constant value on the net. For example, <code>get_property [get_nets UMC/constant_net] timing_constant</code> <i>Return type:</i> string
timing_constant_source	Reports the source through which the timing constant is applied. For example, <code>get_property [get_nets UMC/constant_net] timing_constant_source</code> <i>Return type:</i> string
total_capacitance_max_fall	Returns the total falling capacitance of the net used for late path analysis. For example, <code>get_property [get_nets FE_SIG_N643_ds_data_7_] total_capacitance_max_fall -view func_slow_RCMAX</code> <i>Return type:</i> float
total_capacitance_max_rise	Returns the total rising capacitance of the net used for late path analysis. For example, <code>get_property [get_nets FE_SIG_N643_ds_data_7_] total_capacitance_max_rise -view func_slow_RCMAX</code> <i>Return type:</i> float
total_capacitance_min_fall	Returns the total falling capacitance of the net used for early path analysis. For example, <code>get_property [get_nets FE_SIG_N643_ds_data_7_] total_capacitance_min_fall -view func_fast_RCMIN</code> <i>Return type:</i> float
total_capacitance_min_rise	Returns the total rising capacitance of the net used for early path analysis. For example, <code>get_property [get_nets FE_SIG_N643_ds_data_7_] total_capacitance_min_rise -view func_fast_RCMIN</code> <i>Return type:</i> float
total_lumped_capacitance_max_fall	Returns the maximum value of falling capacitance of a net based on the input library and parasitic model. <i>Return type:</i> float
total_lumped_capacitance_max_rise	Returns the maximum value of rising capacitance of a net based on the input library and parasitic model. <i>Return type:</i> float
total_lumped_capacitance_min_fall	Returns the minimum value of falling capacitance of a net based on the input library and parasitic model. <i>Return type:</i> float
total_lumped_capacitance_min_rise	Returns the minimum value of rising capacitance of a net based on the input library and parasitic model. <i>Return type:</i> float
vl_vh_analysis_status	Reports the glitch analysis (VL/VH) status for a net. For example, <code>get_property [get_nets -of [get_cells FE_SIG_C622_FE_OF517_digit_2_]] vl_vh_analysis_status</code> <i>Return type:</i> string

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

vlu_vho_analysis_status	Reports the glitch analysis status for undershoot (VLU) and overshoot (VHO) glitch for a net. For example, <pre>get_property [get_nets FE_SIG_N642_ds_data_5_] vlu_vho_analysis_status</pre> <i>Return type:</i> string
wire_capacitance_max	Returns the wire capacitance of the net for maximum conditions. <i>Return type:</i> float
wire_capacitance_max_fall	Returns the wire capacitance of the net for the maximum value of the falling capacitance range. For example, <pre>get_property [get_nets FE_SIG_N643_ds_data_7_] wire_capacitance_max_fall -view func_slow_RCMAX</pre> <i>Return type:</i> float
wire_capacitance_max_rise	Returns the wire capacitance of the net for the maximum value of the rising capacitance range. For example, <pre>get_property [get_nets FE_SIG_N643_ds_data_7_] wire_capacitance_max_rise -view func_slow_RCMAX</pre> <i>Return type:</i> float
wire_capacitance_min	Returns the wire capacitance of the net for minimum conditions. <i>Return type:</i> float
wire_capacitance_min_fall	Returns the wire capacitance of the net for the minimum value of the falling capacitance range. For example, <pre>get_property [get_nets FE_SIG_N643_ds_data_7_] wire_capacitance_min_fall -view func_fast_RCMIN</pre> <i>Return type:</i> float
wire_capacitance_min_rise	Returns the wire capacitance of the net for the minimum value of the rising capacitance range. For example, <pre>get_property [get_nets FE_SIG_N643_ds_data_7_] wire_capacitance_min_rise -view func_fast_RCMIN</pre> <i>Return type:</i> float
wire_lumped_capacitance_max	Returns the portion of the net's total lumped capacitance which comes extracted or annotated wire lumped capacitance values. For example, <pre>get_property [get_nets FE_SIG_N643_ds_data_7_] wire_lumped_capacitance_max -view func_slow_RCMAX</pre> <i>Return type:</i> float
wire_lumped_capacitance_max_fall	Returns the lumped wire capacitance of the net for the maximum value of the falling capacitance range. For example, <pre>get_property [get_nets FE_SIG_N643_ds_data_7_] wire_lumped_capacitance_max_fall -view func_slow_RCMAX</pre> <i>Return type:</i> float
wire_lumped_capacitance_max_rise	Returns the lumped wire capacitance of the net for the maximum value of the rising capacitance range. For example, <pre>get_property [get_nets FE_SIG_N643_ds_data_7_] wire_lumped_capacitance_max_rise -view func_slow_RCMAX</pre> <i>Return type:</i> float
wire_lumped_capacitance_min	Returns the portion of the net's total lumped capacitance which comes extracted or annotated wire lumped capacitance values. For example, <pre>get_property [get_nets FE_SIG_N643_ds_data_7_] wire_lumped_capacitance_min -view func_fast_RCMIN</pre> <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

wire_lumped_capacitance_min_fall	Returns the lumped wire capacitance of the net for the minimum value of the falling capacitance range. For example, <pre>get_property [get_nets FE_SIG_N643_ds_data_7_] wire_lumped_capacitance_min_fall -view func_fast_RCMIN</pre> <i>Return type:</i> float
wire_lumped_capacitance_min_rise	Returns the lumped wire capacitance of the net for the minimum value of the rising capacitance range. For example, <pre>get_property [get_nets FE_SIG_N643_ds_data_7_] wire_lumped_capacitance_min_rise -view func_fast_RCMIN</pre> <i>Return type:</i> float
x_coordinate_max	Returns the maximum value of the x coordinate of the net. For example, <pre>get_property [get_nets FE_SIG_N671_FE_OFCN486_ds_data_6_] x_coordinate_max</pre> <i>Return type:</i> float
x_coordinate_min	Returns the minimum value of the x coordinate of the net. For example, <pre>get_property [get_nets FE_SIG_N671_FE_OFCN486_ds_data_6_] x_coordinate_min</pre> <i>Return type:</i> float
y_coordinate_max	Returns the maximum value of the y coordinate of the net. For example, <pre>get_property [get_nets FE_SIG_N671_FE_OFCN486_ds_data_6_] y_coordinate_max</pre> <i>Return type:</i> float
y_coordinate_min	Returns the minimum value of the y coordinate of the net. For example, <pre>get_property [get_nets FE_SIG_N671_FE_OFCN486_ds_data_6_] y_coordinate_min</pre> <i>Return type:</i> float

PG Net Properties

asserted_voltage_max	Returns the maximum asserted voltage on the power ground net. <i>Return type:</i> string
asserted_voltage_min	Returns the minimum asserted voltage on the power ground net. <i>Return type:</i> string
hierarchical_name	Returns the name of the PG net connected to the corresponding PG pin. <i>Return type:</i> string For example, <pre>get_property [get_pg_nets -of_objects [get_cells iA/out_reg]] hierarchical_name get_property [get_pg_nets -of_objects [get_cells iA/out_reg] -filter "is_power == true"] hierarchical_name</pre>
is_ground	Returns a value of <code>true</code> if the PG net is a ground net. <i>Return type:</i> boolean For example, <pre>get_property [get_pg_nets -of_objects [get_cells iA/out_reg]] is_ground get_property [get_pg_nets -of_objects [get_pg_pins -of_objects [get_cells iA/out_reg]]] is_ground</pre>
is_power	Returns a value of <code>true</code> if the PG net is a power net. <i>Return type:</i> boolean For example, <pre>get_property [get_pg_nets -of_objects [get_cells iA/out_reg]] is_power get_property [get_pg_nets -of_objects [get_cells iA/out_reg] -filter "name =~ *VDD*"] is_power</pre>

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

is_primary	Returns a value of <code>true</code> if the PG net is primary. <pre>get_property [get_pg_nets VDD1] is_primary</pre> <i>Return type:</i> boolean
voltage_max	Returns the maximum voltage of a given PG net. This could be voltage associated with a driver term of a net, power intent or CPF, <code>related_pg_pins</code> attribute in Liberty, or the operating voltage of a power domain or system default voltage. <i>Return type:</i> float For example, <pre>get_property [get_pg_nets -of_objects [get_cells iA/out_reg]] voltage_max</pre>
voltage_min	Returns the minimum voltage of a given PG net. This could be voltage associated with a driver term of a net, power intent or CPF, <code>related_pg_pins</code> attribute in Liberty or the operating voltage of a power domain or system default voltage. <i>Return type:</i> float For example, <pre>get_property [get_pg_nets -of_objects [get_cells iA/out_reg]] voltage_min</pre>
object_type	Returns <code>pg_net</code> as the object type for a library PG pin. <i>Return type:</i> string For example, <pre>get_property [get_pg_nets -of_objects [get_cells iA/out_reg]] object_type</pre>
pg_net_group	Returns the name of the PG net group for the <code>pg_net</code> object. <pre>get_property [get_pg_nets VDDR] pg_net_group</pre> <i>Return type:</i> string
parent_pg_net	Returns a collection of a parent power net that is the net powering the power switch that drives the PG net. In case of designs that do not have any power switches powering the instances, this property will always return the net directly powering those instances. It therefore be noted that the usage of this property is more relevant for designs in which the leaf cell instances are powered through power switch cells. <i>Return type:</i> Collection For example, <pre>get_property [get_pg_nets mmu/U1/vdd] parent_pg_net get_property [get_pg_nets -of_objects [get_cells iA/pwr_swch]] parent_pg_net</pre>
top_pg_net	Returns a collection of the top most un-switched power net of the current PG net. Even in the case of a cascading of power switches present in a design, the software traces through all power switches back to the un-switched top-level primary power net. The top-most un-switched power net will be returned - switched output power net will not be returned. <i>Return type:</i> Collection For example, <pre>get_property [get_pg_nets -of_objects [get_cells iA/pwr_swch]] top_pg_net get_property [get_pg_nets -of_objects [get_pg_pins -of_objects [get_cells iA/pwr_swch]]] top_pg_net</pre>

Path Group Properties

name	Returns the name of the path group. <i>Return type:</i> string
hierarchical_name	Returns the hierarchical name of the path group. <i>Return type:</i> string
group_type	Returns the type of group, when group_path and -name, -from, -to parameters are specified for the path. For example, get_property [get_path_groups] group_type <i>Return Type:</i> string
object_type	Returns the object type path_group. <i>Return type:</i> string
view_name	Returns the view name for the specified internal clock group. For example, get_property [get_path_groups m_clk -include_internal_groups] view_name <i>Return type:</i> string

Pin Properties

actual_latency_early_fall_max	When specified with the set_clock_latency -clock parameter, returns the worst actual early fall insertion delay in setup mode for the specified clock at the given pin. The latency reported is ideal or propagated, depending on the clock phase. When specified without the -clock parameter, returns the worst actual early fall insertion delay on the pin. If multiple clocks reach the pin, the reported latency is the worst among all the clocks. When specified with the -view parameter, returns the worst latency for the given view. <i>Return type:</i> float
actual_latency_early_fall_min	When specified with the set_clock_latency -clock parameter, returns the worst actual early fall insertion delay in hold mode for the specified clock at the given pin. The latency reported is ideal or propagated, depending on the clock phase. When specified without the -clock parameter, returns the worst actual early fall insertion delay on the pin. If multiple clocks reach the pin, the reported latency is the worst among all the clocks. When specified with the -view parameter, returns the worst latency for the given view. <i>Return type:</i> float
actual_latency_early_rise_max	When specified with the set_clock_latency -clock parameter, returns the worst actual early rise source insertion delay in setup mode for the specified clock at the given pin. The latency reported is ideal or propagated, depending on the clock phase. When specified without the -clock parameter, returns the worst actual early rise source insertion delay on the pin. If multiple clocks reach the pin, the reported latency is the worst among all the clocks. When specified with the -view parameter, returns the worst latency for the given view. <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

<code>actual_latency_early_rise_min</code>	When specified with the <code>set_clock_latency -clock</code> parameter, returns the worst actual early rise source insertion delay in hold mode for the specified clock at the given pin. The latency reported is ideal or propagated, depending on the clock phase. When specified without the <code>-clock</code> parameter, returns the worst actual early rise source insertion delay on the pin. If multiple clocks reach the pin, the reported latency is the worst among all the clocks. When specified with the <code>-view</code> parameter, returns the worst latency for the given view. <i>Return type:</i> float
<code>actual_latency_late_fall_max</code>	When specified with the <code>set_clock_latency -clock</code> parameter, returns the worst actual late fall source insertion delay in setup mode for the specified clock at the given pin. The latency reported is ideal or propagated, depending on the clock phase. When specified without the <code>-clock</code> parameter, returns the worst actual late fall source insertion delay on the pin. If multiple clocks reach the pin, the reported latency is the worst among all the clocks. When specified with the <code>-view</code> parameter, returns the worst latency for the given view. <i>Return type:</i> float
<code>actual_latency_late_fall_min</code>	When specified with the <code>set_clock_latency -clock</code> parameter, returns the worst actual late fall source insertion delay in hold mode for the specified clock at the given pin. The latency reported is ideal or propagated, depending on the clock phase. When specified without the <code>-clock</code> parameter, returns the worst actual late fall source insertion delay on the pin. If multiple clocks reach the pin, the reported latency is the worst among all the clocks. When specified with the <code>-view</code> parameter, returns the worst latency for the given view. <i>Return type:</i> float
<code>actual_latency_late_rise_max</code>	When specified with the <code>set_clock_latency -clock</code> parameter, returns the worst actual late rise source insertion delay in setup mode for the specified clock at the given pin. The latency reported is ideal or propagated, depending on the clock phase. When specified without the <code>-clock</code> parameter, returns the worst actual late rise source insertion delay on the pin. If multiple clocks reach the pin, the reported latency is the worst among all the clocks. When specified with the <code>-view</code> parameter, returns the worst latency for the given view. <i>Return type:</i> float
<code>actual_latency_late_rise_min</code>	When specified with the <code>set_clock_latency -clock</code> parameter, returns the worst actual late rise source insertion delay in hold mode for the specified clock at the given pin. The latency reported is ideal or propagated, depending on the clock phase. When specified without the <code>-clock</code> parameter, returns the worst actual late rise source insertion delay on the pin. If multiple clocks reach the pin, the reported latency is the worst among all the clocks. When specified with the <code>-view</code> parameter, returns the worst latency for the given view. <i>Return type:</i> float
<code>arrival_clocks</code>	Returns the data phases, clock source data phases, and latency phases. <i>Return type:</i> collection The following attributes support these phases: • <code>timing_property_arrival_clocks_consider_clock_source_data_phase</code>: Considers the clock source data phases. • <code>timing_property_arrival_clocks_consider_data_phase</code>: Considers the pure data phase. • <code>timing_property_arrival_clocks_consider_latency_phase</code>: Considers the latency phase. By default, these attributes are set to <code>true</code>.
<code>arrival_max</code>	Returns the value of the arrival max at specified pin. You can verify through <code>report_timing</code>. For example, <pre>get_property [get_pins <pin name>] arrival_max</pre> <i>Return Type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

arrival_max_fall	Returns the arrival time for the falling transition at the max corner. <i>Return type:</i> float
arrival_max_rise	Returns the arrival time for the rising transition at the max corner. <i>Return type:</i> float
arrival_mean_max_fall	Returns the mean component of the arrival time for the falling transition at the max corner. <i>Return type:</i> float
arrival_mean_max_rise	Returns the mean component of the arrival time for the rising transition at the max corner. <i>Return type:</i> float
arrival_mean_min_fall	Returns the mean component of the arrival time for the falling transition at the min corner. <i>Return type:</i> float
arrival_mean_min_rise	Returns the mean component of the arrival time for the rising transition at the min corner. <i>Return type:</i> float
arrival_min	Returns the value of the arrival min at specified pin. You can verify through <code>report_timing</code> . For example, <pre>get_property [get_pins <pin name>] arrival_min</pre> <i>Return Type:</i> Float
arrival_min_fall	Returns the arrival time for the falling transition at the min corner. <i>Return type:</i> float
arrival_min_rise	Returns the arrival time for the rising transition at the min corner. <i>Return type:</i> float
arrival_sigma_max_fall	Returns the sigma component of the arrival time for the falling transition at the max corner. <i>Return type:</i> float
arrival_sigma_max_rise	Returns the mean component of the arrival time for the rising transition at the max corner. <i>Return type:</i> float
arrival_sigma_min_fall	Returns the sigma component of the arrival time for the falling transition at the min corner. <i>Return type:</i> float
arrival_sigma_min_rise	Returns the sigma component of the arrival time for the rising transition at the min corner. <i>Return type:</i> float
arrival_skewness_max_fall	In SOCV analysis mode, this returns the skewness value of the latest falling arrival time to the specified pin across all the concurrent MMMC views. <pre>get_property [get_pins cell1/ZN] arrival_skewness_max_fall</pre> <i>Return type:</i> float
arrival_skewness_max_rise	In SOCV analysis mode, this returns the skewness value of the latest rising arrival time to the specified pin across all the concurrent MMMC views. <pre>get_property [get_pins cell1/ZN] arrival_skewness_max_rise</pre> <i>Return type:</i> float
arrival_skewness_min_fall	In SOCV analysis mode, this returns the skewness value of the earliest falling arrival time to the specified pin across all the concurrent MMMC views. <pre>get_property [get_pins cell1/ZN] arrival_skewness_min_fall</pre> <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

arrival_skewness_min_rise	In SOCV analysis mode, this returns the skewness value of the earliest rising arrival time to the specified pin across all the concurrent MMMC views. <pre>get_property [get_pins cell1/ZN] arrival_skewness_min_rise</pre> <i>Return type:</i> float
arrival_window	Returns the arrival times for the earliest and latest rising and falling transitions in timing window format. <i>Return type:</i> string
attacker_fall_slew	Returns the minimum falling slew of an attacker driver pin. For example: <pre>get_property [get_pins inst1/y] attacker_fall_slew</pre> <i>Return type:</i> float
attacker_rise_slew	Returns the minimum rising slew of an attacker driver pin. For example: <pre>get_property [get_pins inst1/y] attacker_rise_slew</pre> <i>Return type:</i> float
capacitance_max	Returns the value of the maximum pin capacitance specified for the capacitance attribute in the library for the corresponding pin. For example, <pre>get_property [get_pins <pin_name>] capacitance_max</pre> <i>Return Type:</i> float
capacitance_max_fall	Returns the maximum value of the falling capacitance range of the corresponding library pin. <i>Return type:</i> float
capacitance_max_rise	Returns the maximum value of the rising capacitance range of the corresponding library pin. <i>Return type:</i> float
capacitance_min	Returns the value of the minimum pin capacitance specified for the capacitance attribute in the library for the corresponding pin. For example, <pre>get_property [get_pins <pin_name>] capacitance_min</pre> <i>Return Type:</i> float
capacitance_min_fall	Returns the minimum value of the falling capacitance range of the corresponding library pin. <i>Return type:</i> float
capacitance_min_rise	Returns the minimum value of the rising capacitance range of the corresponding library pin. Note: For the <code>capacitance_max_*</code> and <code>capacitance_min_*</code> properties (described above), the capacitance value does not match with the <code>pin_load</code> (<code>report_timing -format</code>) in the timing report because the pin load is a sum of all pin capacitances of a net. <i>Return type:</i> float
capture_clocks	Returns the capture clock of the pin. <pre>get_property [get_pins UC/ret_cell/Q] capture_clocks</pre> <i>Return type:</i> collection
cell_name	Returns the cell name for the corresponding pins. For example, <pre>get_property [get_pins addr4_/A] cell_name</pre> <i>Return type:</i> string

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

clock_pulse_width_high	Returns the actual high pulse width for the specified pin. <pre>create_clock -name CK -period 30 -waveform {0 10} [get_ports {CK}]</pre> You can verify through <code>report_min_pulse_width -clock <clock name></code> For example, <pre>get_property [get_pins <pin_name>] clock_pulse_width_high</pre> <i>Return Type:</i> float
clock_pulse_width_low	Returns the actual low pulse width for the specified pin. <pre>create_clock -name CK -period 30 -waveform {0 10} [get_ports {CK}]</pre> You can verify through <code>report_min_pulse_width -clock <clock name></code> For example, <pre>get_property [get_pins <pin_name>] clock_pulse_width_low</pre> <i>Return Type:</i> float
clock_sources	Returns a collection of the source pins of all the clocks arriving at the pin. <i>Return type:</i> collection
clocks	Returns a collection of all of the clocks arriving at the pin. This property will report clocks on hierarchical pins even before a full timing update is performed (with the <code>update_timing -full</code> command). <i>Return type:</i> collection
constrained_capture_clocks	Returns the constrained capture clocks at the specified pin. <pre>get_property [get_pins UC/ret_cell/Q] constrained_capture_clocks</pre> <i>Return type:</i> collection
constant_value	Returns a value of 0 or 1 if a logic state has been asserted or propagated to the pin. <pre>get_property [get_pins RM_l28x16_INST/oe] constant_value</pre> <i>Return type:</i> string
delay_max_fall	Returns the maximum falling delay - specified using the <code>set_max_delay -fall</code> command. <i>Return type:</i> float
delay_max_rise	Returns the maximum rising delay - specified using the <code>set_max_delay -rise</code> command. <i>Return type:</i> float
delay_min_fall	Returns the minimum falling delay - specified using the <code>set_min_delay -fall</code> command. <i>Return type:</i> float
delay_min_rise	Returns the minimum rising delay - specified using the <code>set_min_delay -rise</code> command. <i>Return type:</i> float
direction	Returns the direction of the pin: in, out, or inout. <i>Return type:</i> string
driver_cell	Returns the driver cell name for the corresponding pins. For example, <pre>get_property [get_pins ON28_n_17/A] driver_cell</pre> <i>Return type:</i> string
driver_cell_name	Returns the driving cell instance for the specified pin. For example, <pre>get_property [get_pins <pin name>] driver_cell_name</pre> <i>Return Type:</i> string

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

driver_power_pin_voltage_max	Reports the maximum power pin voltage of a driver cell. <pre>get_property [get_pins UC/mcr_cell/a] driver_power_pin_voltage_max</pre> <i>Return type:</i> float
driver_power_pin_voltage_min	Reports the minimum power pin voltage of a driver cell. <pre>get_property [get_pins UC/mcr_cell/a] driver_power_pin_voltage_min</pre> <i>Return type:</i> float
driver_ref_lib_cell_name	Returns the library defined driving cell for the specified pin. For example, <pre>get_property [get_pins <pin name>] driver_ref_lib_cell_name</pre> <i>Return Type:</i> string
escaped_name	Returns the full hierarchical name, including escaped characters (if any). <i>Return Type:</i> string
fanin	Returns the number of sources connected with net of the specified pin. <i>Return type:</i> integer
fanout	Returns the number of sinks connected with the net of the specified pin. <i>Return type:</i> integer
fanout_load	Returns the fanout load for the pin. <i>Return type:</i> float
fastest_clock	Returns the collection of the clock name defined for the specified pin. <pre>create_generated_clock -name ck -source CK -divide_by 2 <pin name></pre> For example, <pre>get_property [get_pins <pin_name>] fastest_clock</pre> <i>Return Type:</i> collection
fastest_clock_name	Returns the clock name defined for the specified pin. <pre>create_generated_clock -name clk -source CLK -multiply_by 2 <pin name></pre> For example, <pre>get_property [get_pins <pin name>] fastest_clock_name</pre> <i>Return Type:</i> string
fastest_clock_period	Returns the clock period of the clock defined for the specified pin. <pre>create_clock -name CLK -period 10 -waveform {0 5} [get_ports {CK2}]</pre> <pre>create_generated_clock -name clk -source CLK</pre> <pre>-multiply_by 3 <pin name></pre> You can verify through <code>report_clocks</code>. For example, <pre>get_property [get_pins <pin_name>] fastest_clock_period</pre> <i>Return Type:</i> float
hierarchical_level	Returns the hierarchical level of the pin. For example, <pre>get_property [get_pins <pin name>] hierarchical_level</pre> <i>Return Type:</i> integer
hierarchical_name	Returns the complete hierarchical name of the pin. <i>Return type:</i> string

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

hold_uncertainty	Returns the most conservative uncertainty of all possible uncertainty assertions associated with the pin. <i>Return type:</i> float
holding_resistance_high	Returns the highest holding resistance of a pin. For example, <pre>foreach pin \$pinList { set vicArray [get_property [get_pins \$pin] noise_victim] foreach_in_collection vic \$vicArray { get_property \$vic holding_resistance_high } }</pre> <i>Return type:</i> float
holding_resistance_low	Returns the lowest holding resistance of a pin. <i>Return type:</i> float
interp_info_max	Returns information for late analysis. When specified, the report output will show the data for interpolation, extrapolation, and/or matched information for late analysis, along with the library details. <i>Return type:</i> string
interp_info_min	Returns information for early analysis. When specified, the report output will show the data for interpolation, extrapolation, and/or matched information for early analysis, along with the library details. <i>Return type:</i> string
is_always_on	Returns a value of <code>true</code> if an instance pin is an always-on pin. <i>Return type:</i> boolean For example, <pre>get_property [get_pins mmu/U1/AON] is_always_on</pre>
is_async	Returns a value of <code>true</code> if the pin is an asynchronous clear or preset pin. <i>Return type:</i> boolean
is_clear	Returns a value of <code>true</code> if the pin is an asynchronous clear pin. <i>Return type:</i> boolean
is_clock	Returns a value of <code>true</code> if the pin has the Liberty pin attribute: <code>clock</code> . <i>Return type:</i> boolean
is_clock_gating	Returns a value of <code>true</code> if the pin is defined as a pin of a clock gating cell. <i>Return type:</i> boolean
is_clock_gating_clock	Returns a value of <code>true</code> if the pin is defined as a clock pin of a clock gating cell. <i>Return type:</i> boolean
is_clock_gating_enable	Returns a value of <code>true</code> if the pin corresponds to the enable pin of a clock gating cell. <i>Return type:</i> boolean
is_clock_root	Returns <code>true</code> for a pin or port if real clock or generated clock is defined on the pin/port. <pre>create_generated_clock -name clk -source CLK -multiply_by 2 <pin name></pre> For example, <pre>get_property [get_pins <pin_name>] is_clock_root</pre> <i>Return Type:</i> boolean
is_clock_source_pin	Returns a value of <code>true</code> for a valid instance pin or port object, if a clock is declared with that clock source at pin/port. <i>Return Type:</i> Boolean

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

is_clock_source_latency	Returns a value of <code>true</code> if a pin, port or any of the leaf pins of a net is part of a clock source latency network. <i>Return Type:</i> Boolean
is_clock_used_as_clock	Returns a value of <code>true</code> if the pin lies in the clock network and at least one of the clocks arriving at the pin is used as a clock in the downstream network of the pin. When the <code>-clock</code> parameter is specified, this property returns <code>true</code> if the above condition is satisfied for the specified clock. <i>Return type:</i> boolean
is_clock_used_as_data	Returns a value of <code>true</code> if the pin lies in the clock source path and at least one of the clocks arriving on the pin is used as data in the downstream network of the pin. When the <code>-clock</code> parameter is specified, this property returns <code>true</code> if the above condition is satisfied for the specified clock. <i>Return type:</i> boolean
is_constrained	Returns a value of <code>true</code> if the specified pin is constrained. <code>get_property [get_pins UM/A] is_constrained</code> <i>Return type:</i> boolean
is_data	Returns a value of <code>true</code> if the pin is a data pin of a flip-flop (that is, is not a clock pin). <i>Return type:</i> boolean
is_disable_timing	Returns a value of <code>true</code> if the pin's timing has been disabled. <i>Return type:</i> boolean Note: This property checks if the associated arc for the pin is active for propagation and analysis. For accurate results, re-run the <code>update_timing</code> command.
is_fall_edge_triggered_clock	Returns a value of <code>true</code> if the pin is a clock pin of a flop, and is triggered by the falling edge of a clock. <i>Return type:</i> boolean
is_fall_edge_triggered_data	Returns a value of <code>true</code> if the pin corresponds to the data pin of a fall edge triggered device. <i>Return value:</i> boolean
is_hierarchical	Returns a value of <code>true</code> if the pin is a pin of a hierarchical module. <i>Return type:</i> boolean
is_isolation_cell_data	Returns <code>true</code> if the instance signal pin corresponding to the <code>isolation_cell_data_pin</code> attribute of a library signal pin is set to <code>true</code>, indicating that the library cell signal pin is a data pin of an isolation library cell. Note : The <code>is_isolation_cell</code> attribute must be set to <code>true</code> in the library. For example, <code>get_property [get_pins <pin_name>] is_isolation_cell_data</code> <i>Return Type:</i> boolean
is_isolation_cell_clock	Returns <code>true</code> if the instance signal pin corresponding to the <code>isolation_cell_clock_pin</code> attribute of a library signal pin is set to <code>true</code>, indicating that the library cell signal pin is a data pin of an isolation library cell. Note : The <code>is_isolation_cell</code> attribute must be set to <code>true</code> in the library. <i>Return Type:</i> boolean
is_isolation_cell_enable	Returns <code>true</code> if the instance signal pin corresponding to the <code>isolation_cell_enable_pin</code> attribute of a library signal pin is set to <code>true</code>, indicating that the library cell signal pin is an enable or control pin of an isolation library cell. Note : The <code>is_isolation_cell</code> attribute must be set to <code>true</code> in the library. For example, <code>get_property [get_pins <pin_name>] is_isolation_cell_enable</code> <i>Return Type:</i> boolean

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

is_latency_network_pin	Returns <code>true</code> if the specified pin is having generated or created clock. <pre>create_generated_clock -name ck -source CK -divide_by 2 <pin name></pre> For example, <pre>get_property [get_pins <pin name>] is_latency_network_pin</pre> <i>Return Type:</i> boolean
is_level_shifter_enable	Returns <code>true</code> if the instance signal pin corresponding to the <code>level_shifter_enable_pin</code> attribute of a library signal pin is set to <code>true</code>, indicating that the library cell signal pin is a enable pin of an isolation library cell. <i>Return Type:</i> boolean
is_level_shifter_data	Returns <code>true</code> if the instance signal pin corresponding to the <code>level_shifter_data_pin</code> attribute of a library signal pin is set to <code>true</code>, indicating that the library cell signal pin is a data pin of an isolation library cell. Note: The <code>is_level_shifter</code> attribute must be set to <code>true</code> in the library. For example, <pre>get_property [get_pins <pin_name>] is_level_shifter_data</pre> <i>Return Type:</i> boolean
input_signal_level	Returns the power rail name associated with the input signal pin, only if it has been explicitly specified in the Liberty description of the cell, using the <code>input_signal_level</code> attribute. Typically, this attribute can only be defined in overdrive situations where the expected input voltage does not match the <code>related_power_pin</code> of the input. In other cases, a value of NA will be returned. <i>Return type:</i> string For example, <pre>get_property [get_pins mmu/U1/I] input_signal_level</pre>
input_signal_level_voltage	If the <code>input_signal_level</code> has an explicit value, then this attribute will return the voltage of that power rail. This is the expected incoming signal voltage for the input pin. A value of NA will be returned if <code>input_signal_level</code> is undefined. <i>Return type:</i> float For example, <pre>get_property [get_pins mmu/U1/I] input_signal_level_voltage</pre>
is_multiple_clock_fanin_point	Returns a value of <code>true</code> if multiple clock phases converge at the pin. <i>Return type:</i> boolean
is_mux_select_pin	Returns <code>true</code> if the specified pin is the mux select pin. For example, <pre>get_property [get_pins m1/S] is_mux_select_pin</pre> <i>Return Type:</i> boolean
is_negative_level_sensitive_clock	Returns a value of <code>true</code> if the library pin is an enable pin of an active low level-sensitive device. <i>Return type:</i> boolean
is_negative_level_sensitive_data	Returns a value of <code>true</code> if the pin is a data pin of an active low level-sensitive device. <i>Return type:</i> boolean
is_pin	Returns a value of <code>true</code>. The <code>is_pin</code> and <code>is_port</code> queries are provided for both pin and port objects to provide a quick way of determining the object type of an identifier. <code>is_port</code> always returns <code>false</code> for pins. <code>is_pins</code> always returns <code>false</code> for ports. <i>Return type:</i> boolean

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

is_port	Returns a value of <code>false</code>. The <code>is_pin</code> and <code>is_port</code> queries are provided for both pin and port objects to provide a quick way of determining the object type of an identifier. <code>is_port</code> always returns <code>false</code> for pins. <code>is_pins</code> always returns <code>false</code> for ports. <i>Return type:</i> boolean
is_positive_level_sensitive_clock	Returns a value of <code>true</code> if the library pin is an enable pin of an active high level-sensitive device. <i>Return type:</i> boolean
is_positive_level_sensitive_data	Returns a value of <code>true</code> if the pin is a data pin of an active high level-sensitive device. <i>Return type:</i> boolean
is_power_mode_disabled	Returns a value of <code>true</code> if power mode of the specified pin is disabled. <pre>get_property [get_pins MD_C/C_IS] is_power_mode_disabled -view view_func_8V_OFF</pre> <i>Return type:</i> boolean
is_preset	Returns a value of <code>true</code> if the pin is a preset pin. <i>Return type:</i> boolean
is_propagated_clock	Returns a value of <code>true</code> if there is an explicit <code>set_propagated_clock</code> assertion at the pin. <i>Return type:</i> boolean
is_ref_clock_pin	Returns a value of <code>true</code> for: - Valid instance pin object and on an active clock pin where a clock signal reaches, and sequential cell is not disabled by disabled timing arcs or case analysis. Any pin with clock check-arcs or trigger-arcs will also be reported, if an active clock phase is present on that pin. - Clock pin on an inferred clock gating cell. Returns a value of <code>false</code>, if clock propagation disabled with the <code>set_disable_timing</code> or <code>set_sense - stop_propagation</code> command. <i>Return Type:</i> Boolean
is_rise_edge_triggered_clock	Returns a value of <code>true</code> if the pin is a clock pin of a flop, and is triggered by the rising edge of a clock. <i>Return type:</i> boolean
is_rise_edge_triggered_data	Returns a value of <code>true</code> if the pin corresponds to the data pin of a rise edge-triggered device. <i>Return type:</i> boolean
is_tristate	Returns a value of <code>true</code> if the pin has the <code>three_state</code> attribute in the Liberty timing library. <i>Return type:</i> boolean
is_tristate_enable	Returns a value of <code>true</code> if the pin is the source pin of timing arcs with either the <code>three_state_enable</code> or <code>three_state_disable</code> attribute in the Liberty timing library. <i>Return type:</i> boolean
is_tristate_output	Returns a value of <code>true</code> if the pin corresponds to a three-state output pin. <i>Return type:</i> boolean
latency_fall_max	Returns the maximum fall insertion delay specified by an explicit <code>set_clock_latency</code> at the pin. <i>Return type:</i> float
latency_fall_min	Returns the minimum fall insertion delay specified by an explicit <code>set_clock_latency</code> at the pin. <i>Return type:</i> float
latency_rise_max	Returns the maximum rise insertion delay specified by an explicit <code>set_clock_latency</code> at the pin. <i>Return type:</i> float
latency_rise_min	Returns the minimum rise insertion delay specified by an explicit <code>set_clock_latency</code> at the pin. <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

lib_pins	Returns the collection of the library pin for the specified pin. For example, <pre>get_property [get_pins <pin name>] lib_pins</pre> <i>Return Type:</i> collection
max_capacitance	Returns the maximum capacitance limit for the pin. <i>Return type:</i> float
max_fanout	Returns the maximum fanout load that the pin can drive. This value is set using <code>set_max_fanout</code> or the <code>default_max_fanout</code> library attribute. <i>Return type:</i> float
max_time_borrow	Returns the value of the maximum time borrowed for the specified pin when the <code>set_max_time_borrow</code> parameter is specified. For example, <pre>get_property [get_pins <pin name>] max_time_borrow</pre> <i>Return Type:</i> float
max_time_borrow_io	Returns the maximum time that a path arriving at a latch input can borrow from the next stage during latch-based analysis. <pre>get_property [get_pins mro_dc/LP/d] max_time_borrow_io</pre> <i>Return Type:</i> float
max_transition	Returns the maximum transition time limit specified for the pin. <i>Return type:</i> float
min_capacitance	Returns the value of the minimum capacitance for the specified pin when the <code>set_min_capacitance</code> parameter is specified. For example, <pre>get_property [get_pins <pin name>] min_capacitance</pre> <i>Return Type:</i> float
min_fanout	Returns the minimum fanout design rule limit of the corresponding library pin. <i>Return type:</i> float
min_transition	Returns the value of the minimum transition for the specified pin. For example, <pre>get_property [get_pins <pin name>] min_transition</pre> <i>Return Type:</i> float
name	Returns the leaf-level name of the pin. For example, <pre>get_property [get_pins <pin name>] name</pre> <i>Return Type:</i> string
netlist_constant_value	Returns a constant logic value of 1'b0 or 1'b1 in the Verilog. <pre>get_property [get_pins UC/flp/pez] netlist_constant_value</pre>
net_name	Returns the name of the net connected to the specified pin. <i>Return type:</i> string
noise_victim	Returns a collection of <code>si_victim</code> objects that can be used to query glitch attributes. <i>Return type:</i> collection
object_type	Returns the object type <code>pin</code>. <i>Return type:</i> string

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

output_signal_level	Returns the rail name of the <code>related_power_pin</code> attribute of the output/inout signal pin. <i>Return type:</i> string For example, <code>get_property [get_pins mmu/U1/Y] output_signal_level</code>
pin_capacitance_max	Reports the maximum capacitance of a pin. <code>get_property [get_pins UMC/buff/A] pin_capacitance_max</code> <i>Return type:</i> float
pin_capacitance_max_fall	Returns the maximum falling capacitance of a pin. <i>Return type:</i> float
pin_capacitance_max_rise	Returns the maximum rise capacitance of pin. <i>Return type:</i> float
pin_capacitance_min	Returns the minimum capacitance of pin. <i>Return type:</i> float
pin_capacitance_min_fall	Returns the minimum fall capacitance of Pin. <i>Return type:</i> float
pin_capacitance_min_rise	Returns the minimum rise capacitance of Pin. <i>Return type:</i> float
pin_direction	Reports the direction of a pin. <code>get_property [get_pins UC/buf/B] pin_direction</code> <i>Return type:</i> string
power_rail_voltage_bidir_input_max	Reports power rail voltage for input part of bidi pins/ports for maximum operating condition. <i>Return type:</i> float
power_rail_voltage_bidir_input_min	Reports power rail voltage for input part of bidi pins/ports for minimum operating condition. <i>Return type:</i> float
power_rail_voltage_max	Reports power rail voltage for maximum operating condition for bidirectional, input and output pins. For bidirectional pins, the power rail voltage for output signal is reported. <i>Return type:</i> float
power_rail_voltage_min	Reports power rail voltage for minimum operating condition for bidirectional, input and output pins. For bidirectional pins, the power rail voltage for output signal is reported. <i>Return type:</i> float
receiver_ccc_type	Returns the type of receiver cell model. The return values can be <code>udn</code>, <code>single_ccc</code>, <code>dual_ccc</code>, <code>multi_ccc</code>, or <code>black_box</code>. This attribute applies to receiver pins only. For example, <code>get_property [get_pins inst1/a] receiver_ccc_type</code> <i>Return type:</i> string
ref_lib_pin_name	Returns the name of the library pin associated with the instance pin. <i>Return type:</i> string
related_ground_pin	Returns the related ground pins as a collection for the corresponding instance pins. <i>Return type:</i> collection For example, <code>get_property [get_pins mmu/U1/I] related_ground_pin</code>

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

related_power_pin	Returns the related power pins as a collection for the corresponding instance pins. <i>Return type:</i> collection For example, get_property [get_pins mmu/U1/I] related_power_pin
setup_uncertainty	Returns the most conservative uncertainty of all possible uncertainty assertions associated with the pin. <i>Return type:</i> float
si_slew_max	Returns the maximum slew from all the incoming signals after performing SI analysis. This value is a maximum of si_slew_max_rise and si_slew_max_fall. <i>Return type:</i> float
si_slew_max_fall	Reports the maximum falling slew considering the incremental slew from signal integrity analysis. get_property [get_pins UC/buf/A] si_slew_max_fall <i>Return type:</i> float
si_slew_max_rise	Reports the maximum rising slew considering the incremental slew from signal integrity analysis. get_property [get_pins UC/buf/A] si_slew_max_rise <i>Return type:</i> float
si_slew_min_fall	Reports the minimum falling slew considering the incremental slew from signal integrity analysis. get_property [get_pins UC/buf/A] si_slew_min_fall <i>Return type:</i> float
si_slew_min_rise	Reports the minimum rising slew considering the incremental slew from signal integrity analysis. get_property [get_pins UC/buf/A] si_slew_min_rise <i>Return type:</i> float
si_slew_mean_max_fall	Returns the mean component of maximum falling slew from the incoming signals after SI analysis is performed. <i>Return type:</i> float
si_slew_mean_max_rise	Returns the mean component of maximum rising slew from the incoming signals after SI analysis is performed. <i>Return type:</i> float
si_slew_mean_min_fall	Returns the mean component of minimum falling slew from the incoming signals after SI analysis is performed. <i>Return type:</i> float
si_slew_mean_min_rise	Returns the mean component of minimum rising slew from the incoming signals after SI analysis is performed. <i>Return type:</i> float
si_slew_min	Returns the minimum slew from all the incoming signals after performing SI analysis. This value is a minimum of si_slew_min_rise and si_slew_min_fall. <i>Return type:</i> float
si_victim	Returns a collection of the si_victim objects that can be used to query graph-based SI delay attributes. <i>Return type:</i> collection
slack_max	Returns the slack maximum property for a timing path. <i>Return type:</i> float
slack_max_edge	Returns the edge (rise or fall) of the worst slack-causing path at the specified pin in late mode. <i>Return type:</i> string
slack_max_fall	Returns the slack time for the falling transition at the max corner. <i>Return type:</i> float
slack_max_rise	Returns the slack time for the rising transition at the max corner. When the -view parameter is not specified, the worst slack for all the views is returned. <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

slack_mean_max	Returns the maximum mean component of the slack time for a timing path. <i>Return type:</i> float
slack_mean_min	Returns the minimum mean component of the slack time for a timing path. <i>Return type:</i> float
slack_mean_max_fall	Returns the mean component of the slack time for the falling transition at the max corner. <i>Return type:</i> float
slack_mean_max_rise	Returns the mean component of the slack time for the rising transition at the max corner. <i>Return type:</i> float
slack_mean_min_fall	Returns the mean component of the slack time for the falling transition at the min corner. <i>Return type:</i> float
slack_mean_min_rise	Returns the mean component of the slack time for the rising transition at the min corner. <i>Return type:</i> float
slack_min	Returns the slack time for a timing path at the min corner. <i>Return type:</i> float
slack_min_edge	Returns the edge (rise or fall) of the worst slack-causing path at the specified pin in early mode. <i>Return type:</i> string
slack_min_fall	Returns the slack time for the falling transition at the min corner. <i>Return type:</i> float
slack_sigma_max	Returns the variation component of the worst slack across all the concurrent MMMC views for Setup-style late data path checks. get_property [get_pins df1/Q] slack_sigma_max <i>Return type:</i> float
slack_sigma_min	Returns the variation component of the worst slack across all the concurrent MMMC views for Hold-style early data path checks. get_property [get_pins df1/Q] slack_sigma_min <i>Return type:</i> float
slack_min_rise	Returns the slack time for the rising transition at the min corner. <i>Return type:</i> float
slack_skewness_max_fall	Returns the skewness component of the worst falling slack across all the concurrent MMMC views for Setup-style late data path checks. get_property [get_pins df1/Q] slack_skewness_max_fall <i>Return type:</i> float
slack_skewness_max_rise	Returns the skewness component of the worst rising slack across all the concurrent MMMC views for Setup-style late data path checks. get_property [get_pins df1/Q] slack_skewness_max_rise <i>Return type:</i> float
slack_skewness_min_fall	Returns the skewness component of the worst falling slack across all the concurrent MMMC views for Hold-style early data path checks. get_property [get_pins df1/Q] slack_skewness_min_fall <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

slack_skewness_min_rise	Returns the skewness component of the worst rising slack across all the concurrent MMMC views for Hold-style early data path checks. <pre>get_property [get_pins df1/Q] slack_skewness_min_rise</pre> <i>Return type:</i> float
slack_sigma_max_fall	Returns the sigma component of the slack time for the falling transition at the max corner. <i>Return type:</i> float
slack_sigma_max_rise	Returns the sigma component of the slack time for the rising transition at the max corner. <i>Return type:</i> float
slack_sigma_min_fall	Returns the sigma component of the slack time for the falling transition at the min corner. <i>Return type:</i> float
slack_sigma_min_rise	Returns the sigma component of the slack time for the rising transition at the min corner. <i>Return type:</i> float
slew_max	Returns the value of the maximum slew for the specified pin. You can verify through <code>report_timing</code>. For example, <pre>get_property [get_pins <pin name>] slew_max</pre> <i>Return Type:</i> float
slew_max_fall	Returns the slew time for the falling transition at the max corner. <i>Return type:</i> float
slew_max_rise	Returns the slew time for the rising transition at the max corner. <i>Return type:</i> float
slew_mean_max_fall	Returns the mean component of the slew time for the falling transition at the max corner. <i>Return type:</i> float
slew_mean_max_rise	Returns the mean component of the slew time for the rising transition at the max corner. <i>Return type:</i> float
slew_mean_min_fall	Returns the mean component of the slew time for the falling transition at the min corner. <i>Return type:</i> float
slew_mean_min_rise	Returns the mean component of the slew time for the rising transition at the min corner. <i>Return type:</i> float
slew_min	Returns the value of the minimum slew for the specified pin. You can verify through <code>report_timing</code>. For example, <pre>get_property [get_pins <pin name>] slew_min</pre> <i>Return Type:</i> float
slew_min_fall	Returns the slew time for the falling transition at the min corner. <i>Return type:</i> float
slew_min_rise	Returns the slew time for the rising transition at the min corner. <i>Return type:</i> float
slew_sigma_max_fall	Returns the sigma component of the slew time for the falling transition at the max corner. <i>Return type:</i> float
slew_sigma_max_rise	Returns the sigma component of the slew time for the rising transition at the max corner. <i>Return type:</i> float
slew_sigma_min_fall	Returns the sigma component of the slew time for the falling transition at the min corner. <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

slew_sigma_min_rise	Returns the sigma component of the slew time for the rising transition at the min corner. <i>Return type:</i> float
slew_skewness_max_fall	Returns the skewness component of the maximum slew time for falling transitions across all the concurrent MMMC views. <code>get_property [get_pins UMC/buff/A] slew_skewness_max_fall</code> <i>Return type:</i> float
slew_skewness_max_rise	Returns the skewness component of the maximum slew time for rising transitions across all the concurrent MMMC views. <code>get_property [get_pins UMC/buff/A] slew_skewness_max_rise</code> <i>Return type:</i> float
slew_skewness_min_fall	Returns the variation component of the minimum slew time for falling transitions across all the concurrent MMMC views. <code>get_property [get_pins UMC/buff/A] slew_skewness_min_fall</code> <i>Return type:</i> float
slew_skewness_min_rise	Returns the variation component of the minimum slew time for rising transitions across all the concurrent MMMC views. <code>get_property [get_pins UMC/buff/A] slew_skewness_min_rise</code> <i>Return type:</i> float
source_latency_early_fall_max	Returns the maximum early fall source insertion delay specified by an explicit <code>set_clock_latency</code> at the pin. <i>Return type:</i> float
source_latency_early_fall_min	Returns the minimum early fall source insertion delay specified by an explicit <code>set_clock_latency</code> at the pin. <i>Return type:</i> float
source_latency_early_rise_max	Returns the maximum early rise source insertion delay specified by an explicit <code>set_clock_latency</code> at the pin. <i>Return type:</i> float
source_latency_early_rise_min	Returns the minimum early rise source insertion delay specified by an explicit <code>set_clock_latency</code> at the pin. <i>Return type:</i> float
source_latency_late_fall_max	Returns the maximum late fall source insertion delay specified by an explicit <code>set_clock_latency</code> at the pin. <i>Return type:</i> float
source_latency_late_fall_min	Returns the minimum late fall source insertion delay specified by an explicit <code>set_clock_latency</code> at the pin. <i>Return type:</i> float
source_latency_late_rise_max	Returns the maximum late rise source insertion delay specified by an explicit <code>set_clock_latency</code> at the pin. <i>Return type:</i> float
source_latency_late_rise_min	Returns the minimum late rise source insertion delay specified by an explicit <code>set_clock_latency</code> at the pin. <i>Return type:</i> float
stolen_slack	Returns the slack stolen at the specified pin. This property can be queried only for pins in late mode. Note: The specified pin must be the endpoint of a transparent latch flush arc. <i>Return type:</i> float
timing_constant	Reports the constant value specified on a pin. <code>get_property [get_pins UC/retent_cell/SZ] timing_constant</code> <i>Return type:</i> string

Litmus Text Command Reference
Advanced Timing Tcl Scripting Commands--get_property

timing_constant_source	Reports the source from which the timing constant was applied. <code>get_property [get_pins UC/retent_cell/SZ] timing_constant_source</code> <i>Return type:</i> string
user_constant_value	Returns the constant values from constraints for the specified pin. <code>get_property [get_pins UC/retent_cell/SZ] user_constant_value</code> <i>Return type:</i> integer
waveform_max_fall	Returns the fall waveform for the specified input pins during late analysis. <i>Return type:</i> string
waveform_max_rise	Returns the rise waveform for the specified input pins during late analysis. <i>Return type:</i> string
waveform_min_fall	Returns the fall waveform for the specified input pins during early analysis. <i>Return type:</i> string
waveform_min_rise	Returns the rise waveform for the specified input pins during early analysis. <i>Return type:</i> string
x_coordinate	Returns the x coordinates of the placed pin. <i>Return type:</i> float
y_coordinate	Returns the y coordinates of the placed pin. <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

PG Pin Properties

hierarchical_name	Returns the complete hierarchical name of the PG pin. <i>Return type:</i> string For example, <pre>get_property [get_pg_pins -of_objects [get_cells iA/out_reg]] hierarchical_name</pre> <pre>get_property [get_pg_pins -of_objects [get_cells iA/out_reg] -filter "pg_type == primary_power"] hierarchical_name</pre>
net	Returns the immediate PG net(s) connected to the corresponding PG pin objects, as a collection. <i>Return type:</i> Collection For example, <pre>get_property [get_pg_pins -of_objects [get_cells iA/out_reg]] net</pre>
object_type	Returns pg_pin as the object type for a PG pin. <i>Return type:</i> string
pg_type	Returns the type of PG pin. <i>Return type:</i> string For example, <pre>get_property [get_pg_pins -of_objects [get_cells iA/out_reg]] pg_type</pre> <pre>get_property [get_pg_pins -of_objects [get_cells iA/out_reg] -filter "hierarchical_name =~ *VDD*"] pg_type</pre>
ref_lib_pin_name	Returns the name of the library PG pin associated with an instance PG pin. <i>Return type:</i> string For example, <pre>get_property [get_pg_pins -of_objects [get_cells iA/out_reg]] ref_lib_pin_name</pre>
voltage_max	Returns the maximum voltage of a given PG pin. This could be the voltage associated with a pin, power intent or CPF, related_pg_pins attribute in Liberty, or the operating voltage of a power domain or system default voltage. <i>Return type:</i> float For example, <pre>get_property [get_pg_pins -of_objects [get_cells iA/out_reg]] voltage_max</pre>
voltage_min	Returns the minimum voltage of a given PG pin. This voltage may be associated with a pin, power intent or CPF, related_pg_pins attribute in Liberty, or the operating voltage of a power domain or system default voltage. <i>Return type:</i> float For example, <pre>get_property [get_pg_pins -of_objects [get_cells iA/out_reg]] voltage_min</pre>

Port Properties

actual_latency_early_fall_max	Returns the maximum fall latency on the specified port caused due to source and network latency asserted using the set_clock_latency command for early analysis. <pre>get_property [get_ports CLK] actual_latency_early_fall_max</pre> <i>Return type:</i> float
actual_latency_early_fall_min	Returns the minimum fall latency on the specified port caused due to source and network latency asserted using the set_clock_latency command for early analysis. <pre>get_property [get_ports CLK] actual_latency_early_fall_min</pre> <i>Return type:</i> float
actual_latency_early_rise_max	Returns the maximum rise latency on the specified port caused due to source and network latency asserted using the set_clock_latency command for early analysis. <pre>get_property [get_ports CLK] actual_latency_early_rise_max</pre> <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

actual_latency_early_rise_min	Returns the minimum rise latency on the specified port caused due to source and network latency asserted using the <code>set_clock_latency</code> command for early analysis. <pre>get_property [get_ports CLK] actual_latency_early_rise_min</pre> <i>Return type:</i> float
actual_latency_late_fall_max	Returns the maximum fall latency on the specified port caused due to source and network latency asserted using the <code>set_clock_latency</code> command for late analysis. <pre>get_property [get_ports CLK] actual_latency_late_fall_max</pre> <i>Return type:</i> float
actual_latency_late_fall_min	Returns the minimum fall latency on the specified port caused due to source and network latency asserted using the <code>set_clock_latency</code> command for late analysis. <pre>get_property [get_ports CLK] actual_latency_late_fall_min</pre> <i>Return type:</i> float
actual_latency_late_rise_max	Returns the maximum rise latency on the specified port caused due to source and network latency asserted using the <code>set_clock_latency</code> command for late analysis. <pre>get_property [get_ports CLK] actual_latency_late_rise_max</pre> <i>Return type:</i> float
actual_latency_late_rise_min	Returns the minimum rise latency on the specified port caused due to source and network latency asserted using the <code>set_clock_latency</code> command for late analysis. <pre>get_property [get_ports CLK] actual_latency_late_rise_min</pre> <i>Return type:</i> float
aocv_early_input_stage_weight	Returns the input stage weight of the specified port for AOCV mode early analysis. <pre>get_property [get_ports int] aocv_early_input_stage_weight</pre> <i>Return type:</i> float
aocv_early_output_stage_weight	Returns the output stage weight of the specified port for AOCV mode late analysis. <pre>get_property [get_ports vcop] aocv_early_output_stage_weight</pre> <i>Return type:</i> float
aocv_late_input_stage_weight	Returns the input stage weight of the specified for AOCV mode late analysis. <pre>get_property [get_ports int] aocv_late_input_stage_weight</pre> <i>Return type:</i> float
aocv_late_output_stage_weight	Returns the output stage weight of the specified port for AOCV mode late analysis. <pre>get_property [get_ports vcop] aocv_late_output_stage_weight</pre> <i>Return type:</i> float
arrival_clocks	Reports clocks that correspond to the arrival times at the port. <pre>get_property [get_ports out] arrival_clocks</pre> <i>Return type:</i> collection
arrival_max	Reports the maximum arrival time at a port. <pre>get_property [get_ports out] arrival_max</pre> <i>Return type:</i> float
arrival_max_fall	Returns the worst arrival on the port for a falling signal. <i>Return type:</i> float
arrival_max_rise	Returns the worst arrival on the port for a rising signal. <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

arrival_mean_max_fall	In SOCV analysis mode, this returns the mean value of the latest falling arrival time to the specified port across all concurrent MMMC views. <pre>get_property [get_ports out] arrival_mean_max_fall</pre> <i>Return type:</i> float
arrival_mean_max_rise	In SOCV analysis mode, this returns the mean value of the latest rising arrival time to the specified port across all concurrent MMMC views. <pre>get_property [get_ports out] arrival_mean_max_rise</pre> <i>Return type:</i> float
arrival_mean_min_fall	In SOCV analysis mode, this returns the mean value of the earliest falling arrival time to the specified port across all concurrent MMMC views. <pre>get_property [get_ports out] arrival_mean_min_fall</pre> <i>Return type:</i> float
arrival_mean_min_rise	In SOCV analysis mode, this returns the mean value of the earliest rising arrival time to the specified port across all concurrent MMMC views. <pre>get_property [get_ports out] arrival_mean_min_rise</pre> <i>Return type:</i> float
arrival_min	Reports minimum arrival time at the port. <pre>get_property [get_ports out] arrival_min</pre> <i>Return type:</i> float
arrival_min_fall	Returns the best arrival on the port for a falling signal. <i>Return type:</i> float
arrival_min_rise	Returns the best arrival on the port for a rising signal. <i>Return type:</i> float
arrival_sigma_max_fall	In SOCV analysis mode, this returns the variation component of the latest falling arrival time to the specified port across all the concurrent MMMC views. <pre>get_property [get_ports out] arrival_sigma_max_fall</pre> <i>Return type:</i> float
arrival_sigma_max_rise	In SOCV analysis mode, this returns the variation component of the latest rising arrival time to the specified port across all the concurrent MMMC views. <pre>get_property [get_ports out] arrival_sigma_max_rise</pre> <i>Return type:</i> float
arrival_sigma_min_fall	In SOCV analysis mode, this returns the variation component of the earliest falling arrival time to the specified port across all the concurrent MMMC views. <pre>get_property [get_ports out] arrival_sigma_min_fall</pre> <i>Return type:</i> float
arrival_sigma_min_rise	In SOCV analysis mode, this returns the variation component of the earliest rising arrival time to the specified port across all the concurrent MMMC views. <pre>get_property [get_ports out] arrival_sigma_min_rise</pre> <i>Return type:</i> float
attacker_fall_slew	Reports fall slew of attackers seen for nets connected to the port. <pre>get_property [get_ports CLK] attacker_fall_slew</pre> <i>Return type:</i> float
attacker_rise_slew	Reports the rise slew of attackers seen for nets connected to the port. <pre>get_property [get_ports CLK] attacker_rise_slew</pre> <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

arrival_window	Returns the arrival times for the earliest and latest rising and falling transitions in timing window format. <i>Return type:</i> string
capture_clocks	Reports capture clocks at the port. <i>Return type:</i> collection
clock_sources	Returns a collection of the source pins of all the clocks arriving at the port. <i>Return type:</i> collection
clocks	Returns a collection of all of the clocks arriving at the port. <i>Return type:</i> collection
constant_value	Returns a constant value of 0 or 1, if a logic state has been asserted or propagated to the port. <code>get_property [get_ports D] constant_value</code> <i>Return type:</i> integer
constrained_capture_clocks	Returns the capture clocks, which are constrained, at the port. <code>get_property [get_ports C] constrained_capture_clocks</code> <i>Return type:</i> collection
capacitance_max	Returns the maximum pin capacitance for the port. <i>Return type:</i> float
capacitance_min	Returns the minimum pin capacitance for the port. <i>Return type:</i> float
delay_max_fall	Returns the maximum falling delay value for the port. <i>Return type:</i> float
delay_max_rise	Returns the maximum rising delay value for the port. <i>Return type:</i> float
delay_min_fall	Returns the minimum falling delay value for the port. <i>Return type:</i> float
delay_min_rise	Returns the minimum rising delay value for the port. <i>Return type:</i> float
direction	Returns the direction of the port: in, out, or inout. <i>Return type:</i> string
driving_cell_no_design_rule	Returns a value of true if there is no design rule transfer from the driving cell set on a port. For example, if a driving cell is defined with the following command, then this property will return true: <code>set_driving_cell -lib_cell <lib_cell_name> -no_design_rule <port></code>
drive_resistance_fall_max	Returns the falling linear drive resistance at the port for the max condition. <i>Return type:</i> float
drive_resistance_fall_min	Returns the falling linear drive resistance at the port for the min condition. <i>Return type:</i> float
drive_resistance_rise_max	Returns the rising linear drive resistance at the port for the max condition. <i>Return type:</i> float
drive_resistance_rise_min	Returns the rising linear drive resistance at the port for the min condition. <i>Return type:</i> float
driving_cell_fall_max	Returns the name of the library cell used to compute max falling delays and transitions at the port. <i>Return type:</i> string

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

driving_cell_fall_min	Returns the name of the library cell used to compute min falling delays and transitions at the port. <i>Return type:</i> string
driving_cell_from_pin_fall_max	Returns the input pin of the driving cell used in the max fall delay calculation at the port. <i>Return type:</i> string
driving_cell_from_pin_fall_min	Returns the input pin of the driving cell used in the min fall delay calculation at the port. <i>Return type:</i> string
driving_cell_from_pin_rise_max	Returns the input pin of the driving cell used in max rise delay calculation at the port. <i>Return type:</i> string
driving_cell_from_pin_rise_min	Returns the input pin of the driving cell used in min rise delay calculation at the port. <i>Return type:</i> string
driving_cell_library_fall_max	Returns the name of the library used to compute max falling delays and transitions at the port. <i>Return type:</i> string
driving_cell_library_fall_min	Returns the name of the library used to compute min falling delays and transitions at the port. <i>Return type:</i> string
driving_cell_library_rise_max	Returns the name of the library used to compute max rising delays and transitions at the port. <i>Return type:</i> string
driving_cell_library_rise_min	Returns the name of the library used to compute min rising delays and transitions at the port. <i>Return type:</i> string
driving_cell_max_fall_itrans_fall	Returns the falling input transition of the <code>from_pin</code> of the driving cell, which is used for the max falling delay calculation at the port. <i>Return type:</i> float
driving_cell_max_fall_itrans_rise	Returns the falling input transition of the <code>from_pin</code> of the driving cell, which is used for the max rising delay calculation at the port. <i>Return type:</i> float
driving_cell_max_rise_itrans_fall	Returns the rising input transition of the <code>from_pin</code> of the driving cell, which is used for the max falling delay calculation at the port. <i>Return type:</i> float
driving_cell_max_rise_itrans_rise	Returns the rising input transition of the <code>from_pin</code> of the driving cell, which is used for the max rising delay calculation at the port. <i>Return type:</i> float
driving_cell_min_fall_itrans_fall	Returns the falling input transition of the <code>from_pin</code> of the driving cell, which is used for the min falling delay calculation at the port. <i>Return type:</i> float
driving_cell_min_fall_itrans_rise	Returns the falling input transition of the <code>from_pin</code> of the driving cell, which is used for the min rising delay calculation at the port. <i>Return type:</i> float
driving_cell_min_rise_itrans_fall	Returns the rising input transition of the <code>from_pin</code> of the driving cell, which is used for the min falling delay calculation at the port. <i>Return type:</i> float
driving_cell_min_rise_itrans_rise	Returns the rising input transition of the <code>from_pin</code> of the driving cell, which is used for the min rising delay calculation at the port. <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

driving_cell_pin_fall_max	Returns the name of the output pin used for max fall delay calculation at the port. <i>Return type:</i> string
driving_cell_pin_fall_min	Returns the name of the output pin used for min fall delay calculation at the port. <i>Return type:</i> string
driving_cell_pin_rise_max	Returns the name of the output pin used for max rise delay calculation at the port. <i>Return type:</i> string
driving_cell_pin_rise_min	Returns the name of the output pin used for min rise delay calculation at the port. <i>Return type:</i> string
driving_cell_rise_max	Returns the name of the library cell used to compute max rising delays and transitions at the port. <i>Return type:</i> string
driving_cell_rise_min	Returns the name of the library cell used to compute min rising delays and transitions at the port. <i>Return type:</i> string
fanin	Returns the number of sources connected with net of the specified port. <i>Return type:</i> integer
fanout	Returns the number of sinks connected with the net of the specified port. <i>Return type:</i> integer
fanout_load	Returns the fanout load for ports. (For input ports, the value is the sum of all fanout loads of all connected pins. For output ports, the value is set using <code>set_fanout_load</code> .) <i>Return type:</i> float
fastest_clock	Reports the clock with maximum frequency associated with the port. <code>get_property [get_ports CLK] fastest_clock</code> <i>Return type:</i> collection
fastest_clock_name	Reports the name of the clock with maximum frequency associated with the port. <code>get_property [get_ports CLK] fastest_clock_name</code> <i>Return type:</i> string
fastest_clock_period	Reports the period of the clock with maximum frequency associated with the port. <code>get_property [get_ports CLK] fastest_clock_period</code> <i>Return type:</i> float
hierarchical_level	Returns the hierarchical level for an instance port. <code>get_property [get_ports C] hierarchical_level</code> <i>Return type:</i> integer
hierarchical_name	Returns the complete hierarchical name of the port. <i>Return type:</i> string
hold_uncertainty	Returns the most conservative uncertainty of all possible uncertainty assertions associated with the port. <i>Return type:</i> float
input_slew_max_fall	Returns the max falling slew for the input or inout ports. (This value is set with the <code>set_input_transition</code> command.) <i>Return type:</i> float
input_slew_max_rise	Returns the max rising slew for the input or inout ports. (This value is set with the <code>set_input_transition</code> command.) <i>Return type:</i> float
input_slew_min_fall	Returns the min falling slew for the input or inout ports. (This value is set with the <code>set_input_transition</code> command.) <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

input_slew_min_rise	Returns the min rising slew for the input or inout ports. (This value is set with the <code>set_input_transition</code> command.) <i>Return type:</i> float
is_clock_root	Returns a value of <code>true</code> if a real or generated clock is defined on a port. <code>get_property [get_ports CLK] is_clock_root</code> <i>Return type:</i> boolean
is_clock_source_latency	Returns a value of <code>true</code> if the object is a part of the latency network. <code>get_property [get_ports C] is_clock_source_latency</code> <i>Return type:</i>
is_clock_source_pin	Returns a value of <code>true</code> if the object is a clock source pin. <code>get_property [get_ports C] is_clock_source_pin</code> <i>Return type:</i> boolean
is_clock_used_as_clock	Returns a value of <code>true</code> if the port lies in the clock network and at least one of the clocks arriving at the pin is used as a clock in the downstream network of the port. <code>get_property [get_ports C] is_clock_used_as_clock</code> <i>Return type:</i> boolean
is_clock_used_as_data	Returns a value of <code>true</code> if the port lies in the clock source path and at least one of the clocks arriving on the port is used as data in the downstream network of port. <code>get_property [get_ports C] is_clock_used_as_data</code> <i>Return type:</i> boolean
is_clock_network_port	Returns a value of <code>true</code> if a port is part of the clock network. <i>Return type:</i> boolean
is_constrained	Returns a value of <code>true</code> if the specified port is constrained. <code>get_property [get_ports Ip] is_constrained</code> <i>Return type:</i> boolean
is_disable_timing	Returns a value of <code>true</code> if the timing of the port has been disabled using the <code>set_disable_timing</code> command. <i>Return type:</i> boolean
is_hierarchical	Returns a value of <code>true</code> if the port is a port of a hierarchical module. <i>Return type:</i> boolean
is_latency_network_pin	Returns a value of <code>true</code> if the specified port has a generated clock. <code>get_property [get_ports CLK] is_latency_network_pin</code> <i>Return type:</i> boolean
is_pin	Returns a value of <code>true</code> for pin objects and <code>false</code> for port objects. <i>Return type:</i> boolean
is_port	Returns a value of <code>true</code> for port objects and <code>false</code> for pin objects. <i>Return type:</i> boolean
is_power_mode_disabled	Returns a value of <code>true</code> if power mode of the specified port is disabled. <code>get_property [get_ports B_OFF] is_power_mode_disabled -view view_func_18V_OFF</code> <i>Return type:</i> boolean
is_propagated_clock	Returns a value of <code>true</code> if there is an explicit <code>set_propagated_clock</code> assertion at the port. <i>Return type:</i> boolean

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

latency_fall_max	Returns the maximum fall insertion delay specified by an explicit <code>set_clock_latency</code> at the port. <i>Return type:</i> float
latency_fall_min	Returns the minimum fall insertion delay specified by an explicit <code>set_clock_latency</code> at the port. <i>Return type:</i> float
latency_rise_max	Returns the maximum rise insertion delay specified by an explicit <code>set_clock_latency</code> at the port. <i>Return type:</i> float
latency_rise_min	Returns the minimum rise insertion delay specified by an explicit <code>set_clock_latency</code> at the port. <i>Return type:</i> float
max_capacitance	Returns the maximum capacitance limit asserted on the specified port. <code>get_property [get_ports Zf] max_capacitance</code> <i>Return type:</i> float
max_fanout	Returns the maximum fanout load that the port can drive. (This value is set using <code>set_max_fanout</code> , or the <code>default_max_fanout</code> library attribute.) <i>Return type:</i> float
max_transition	Returns the maximum transition time limit specified for the port. <i>Return type:</i> float
netlist_constant_value	Returns a value if the port has a constant logic value of 1'b0 or 1'b1 in the Verilog. <code>get_property [get_ports constant] netlist_constant_value</code> <i>Return type:</i> string
net_name	Returns the name of the net connected to the specified port. <i>Return type:</i> string
noise_victim	Returns a collection of <code>si_victim</code> objects that can be used to query glitch attributes. <i>Return type:</i> collection
object_type	Returns the object type <code>port</code> . <i>Return type:</i> string
power_rail_voltage_max	Reports power rail voltage for maximum operating condition for bidirectional, input and output pins. <code>get_property [get_ports in] power_rail_voltage_max -view Setup_m3C</code> <i>Return type:</i> float
power_rail_voltage_min	Reports power rail voltage for minimum operating condition for bidirectional, input and output pins. <code>get_property [get_ports in] power_rail_voltage_min -view Hold_m3C</code> <i>Return type:</i> float
setup_uncertainty	Returns the most conservative uncertainty of all possible uncertainty assertions associated with the port. <i>Return type:</i> float
si_victim	Returns a collection of the <code>si_victim</code> objects that can be used to query graph-based SI delay attributes. <i>Return type:</i> collection
slack_max	Returns the worst slack across all the concurrent MMMC views for setup style late data path checks. For example, <code>get_property [get_ports clk_en] slack_max</code> <i>Return type:</i> float
slack_max_edge	Returns the edge (rise or fall) of the worst slack-causing path at the specified port in late mode. <i>Return type:</i> string

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

slack_max_fall	Returns the worst setup slack for a falling signal at the port endpoint. <i>Return type:</i> float
slack_max_rise	Returns the worst setup slack for a rising signal at the port endpoint. When the <code>-view</code> parameter is not specified, the worst slack for all the views is returned. <i>Return type:</i> float
slack_mean_max	In SOCV analysis, it will return the mean component of the worst slack across all concurrent MMMC views for setup-style late data path checks. For example, <code>get_property [get_ports clk_en] slack_mean_max</code> <i>Return type:</i> float
slack_mean_max_fall	Returns the mean component of the worst setup slack for a falling signal at the port endpoint. For example, <code>get_property [get_ports scan_clk] slack_mean_max_fall</code> <i>Return type:</i> float
slack_mean_max_rise	Returns the mean component of the worst setup slack for a rising signal at the port endpoint. For example, <code>get_property [get_ports scan_clk] slack_mean_max_rise</code> <i>Return type:</i> float
slack_mean_min	Returns the mean component of the worst hold slack at the port endpoint. For example, <code>get_property [get_ports clk_en] slack_mean_min</code> <i>Return type:</i> float
slack_mean_min_fall	Returns the mean component of the worst hold slack for a falling signal at the port endpoint. For example, <code>get_property [get_ports clk_en] slack_mean_min_fall</code> <i>Return type:</i> float
slack_mean_min_rise	Returns the mean component of the worst hold slack for a rising signal at the port endpoint. For example, <code>get_property [get_ports clk_en] slack_mean_min_rise</code> <i>Return type:</i> float
slack_min	Returns the worst hold slack at the port endpoint. For example, <code>get_property [get_ports clk_en] slack_min</code> <i>Return type:</i> float
slack_min_edge	Returns the edge (rise or fall) of the worst slack-causing path at the specified port in early mode. <i>Return type:</i> string
slack_min_fall	Returns the worst hold slack for a falling signal at the port endpoint. <i>Return type:</i> float
slack_min_rise	Returns the worst hold slack for a rising signal at the port endpoint. <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

slew_max	Returns the maximum slew from all of the incoming signals on the port. For example, <pre>get_property [get_ports ahbsi_out[402]] slew_max</pre> <i>Return type:</i> float
slew_max_fall	Returns the maximum falling slew from all of the incoming signals on the port. <i>Return type:</i> float
slew_max_rise	Returns the maximum rising slew from all of the incoming signals on the port. <i>Return type:</i> float
slew_mean_max_fall	Returns the mean component of the maximum falling slew from all of the incoming signals on the port. For example, <pre>get_property [get_ports ahbsi_out[402]] slew_mean_max_fall</pre> <i>Return type:</i> float
slew_mean_max_rise	Returns the mean component of the maximum rising slew from all of the incoming signals on the port. For example, <pre>get_property [get_ports ahbsi_out[407]] slew_mean_max_rise</pre> <i>Return type:</i> float
slew_mean_min_fall	Returns the mean component of the minimum falling slew from all of the incoming signals on the port. For example, <pre>get_property [get_ports BG_scan_out_3] slew_mean_min_fall</pre> <i>Return type:</i> float
slew_mean_min_rise	Returns the mean component of the minimum rising slew from all of the incoming signals on the port. For example, <pre>get_property [get_ports BG_scan_out_3] slew_mean_min_rise</pre> <i>Return type:</i> float
slew_min	Returns the minimum slew from all of the incoming signals on the port. For example, <pre>get_property [get_ports BG_scan_out_3] slew_min</pre> <i>Return type:</i> float
slew_min_fall	Returns the minimum falling slew from all of the incoming signals on the port. <i>Return type:</i> float
slew_min_rise	Returns the minimum rising slew from all of the incoming signals on the port. <i>Return type:</i> float
source_latency_early_fall_max	Returns the maximum early fall source insertion delay specified by an explicit <code>set_clock_latency</code> at the port. <i>Return type:</i> float
source_latency_early_fall_min	Returns the minimum early fall source insertion delay specified by an explicit <code>set_clock_latency</code> at the port. <i>Return type:</i> float
source_latency_early_rise_max	Returns the maximum early rise source insertion delay specified by an explicit <code>set_clock_latency</code> at the port. <i>Return type:</i> float
source_latency_early_rise_min	Returns the minimum early rise source insertion delay specified by an explicit <code>set_clock_latency</code> at the port. <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

source_latency_late_fall_max	Returns the maximum late fall source insertion delay specified by an explicit <code>set_clock_latency</code> at the port. <i>Return type:</i> float
source_latency_late_fall_min	Returns the minimum late fall source insertion delay specified by an explicit <code>set_clock_latency</code> at the port. <i>Return type:</i> float
source_latency_late_rise_max	Returns the maximum late rise source insertion delay specified by an explicit <code>set_clock_latency</code> at the port. <i>Return type:</i> float
source_latency_late_rise_min	Returns the minimum late rise source insertion delay specified by an explicit <code>set_clock_latency</code> at the port. <i>Return type:</i> float
timing_constant	Returns the constant value on a port. <code>get_property [get_ports D] timing_constant</code> <i>Return type:</i> string
timing_constant_source	Returns the source by which the constant value is applied on a port. <code>get_property [get_ports constant] timing_constant_source</code> <i>Return type:</i> string
total_capacitance_fall	Returns the total capacitance of ports for falling transition. <code>get_property [get_ports out] total_capacitance_fall</code> <i>Return type:</i> float
total_capacitance_rise	Returns the total capacitance of ports for rising transition. <code>get_property [get_ports out] total_capacitance</code> <i>Return type:</i> float
transition_max	Returns the maximum transition limit asserted on the specified port. <code>get_property [get_ports Ip] transition_max</code> <i>Return type:</i> float
user_constant_value	Returns the constant values from constraints for the specified ports. <code>get_property [get_ports D] user_constant_value</code> <i>Return type:</i> integer
wire_capacitance_max	Returns the wire capacitance of the port for the maximum value of the capacitance range. For example, <code>get_property [get_ports port_pad_data_out[9]] wire_capacitance_max</code> <i>Return type:</i> float
wire_capacitance_max_fall	Returns the wire capacitance of a port for the maximum value of the falling capacitance range For example, <code>get_property [get_ports port_pad_data_out[9]] wire_capacitance_max_fall</code> <i>Return type:</i> float
wire_capacitance_max_rise	Returns the wire capacitance of a port for the maximum value of the rising capacitance range. For example, <code>get_property [get_ports port_pad_data_out[0]] wire_capacitance_max_rise</code> <i>Return type:</i> float
wire_capacitance_min	Returns the wire capacitance of a port for the minimum value of the capacitance range. For example, <code>get_property [get_ports tdigit[2]] wire_capacitance_min</code> <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

wire_capacitance_min_fall	Returns the wire capacitance of a port for the minimum value of the falling capacitance range. For example, <pre>get_property [get_ports tdigit[2]] wire_capacitance_min_fall</pre> <i>Return type:</i> float
wire_capacitance_min_rise	Returns the wire capacitance of a port for the minimum value of the rising capacitance range. For example, <pre>get_property [get_ports tdigit[2]] wire_capacitance_min_rise</pre> <i>Return type:</i> float
wire_lumped_capacitance_max	Returns the wire lumped capacitance of the net associated to port for maximum conditions. Returns the portion of a port's total lumped capacitance, which may be extracted or annotated wire lumped capacitance values. For example, <pre>get_property [get_ports tdigit[7]] wire_lumped_capacitance_max</pre> <i>Return type:</i> float
wire_lumped_capacitance_max_fall	Returns the lumped wire capacitance of a port for the maximum value of the falling capacitance range. For example, <pre>get_property [get_ports port_pad_data_out[9]] wire_lumped_capacitance_max_fall</pre> <i>Return type:</i> float
wire_lumped_capacitance_max_rise	Returns the lumped wire capacitance of a port for the maximum value of the rising capacitance range. For example, <pre>get_property [get_ports port_pad_data_out[0]] wire_lumped_capacitance_max_rise</pre> <i>Return type:</i> float
wire_lumped_capacitance_min	Returns the portion of a port's total lumped capacitance, which may be extracted or annotated wire lumped capacitance values. For example, <pre>get_property [get_ports tdigit[3]] wire_lumped_capacitance_min</pre> <i>Return type:</i> float
wire_lumped_capacitance_min_fall	Returns the lumped wire capacitance of a port for the minimum value of the falling capacitance range. For example, <pre>get_property [get_ports tdigit[3]] wire_lumped_capacitance_min_fall</pre> <i>Return type:</i> float
wire_lumped_capacitance_min_rise	Returns the lumped wire capacitance of a port for the minimum value of the rising capacitance range. For example, <pre>get_property [get_ports tdigit[3]] wire_lumped_capacitance_min_rise</pre> <i>Return type:</i> float
x_coordinate	Returns the x coordinates of the placed port. <i>Return type:</i> float
y_coordinate	Returns the y coordinates of the placed port. <i>Return type:</i> float

Timing Arc Properties

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

<code>aocv_derate_capture_clock_early_fall</code>	Returns graph based AOCV derate factors in AOCV mode for a given timing arc that is a part of early capture clock paths with fall sink pin transitions. <i>Return type:</i> float
<code>aocv_derate_capture_clock_early_rise</code>	Returns graph based AOCV derate factors in AOCV mode for a given timing arc that is a part of early capture clock paths with rise sink pin transitions. <i>Return type:</i> float
<code>aocv_derate_capture_clock_late_fall</code>	Returns graph based AOCV derate factors in AOCV mode for a given timing arc that is a part of late capture clock paths with fall sink pin transitions. <i>Return type:</i> float
<code>aocv_derate_capture_clock_late_rise</code>	Returns graph based AOCV derate factors in AOCV mode for a given timing arc that is a part of late capture clock paths with rise sink pin transitions. <i>Return type:</i> float
<code>aocv_derate_data_early_fall</code>	Returns AOCV derate values for a timing arc on an early data path with fall transitions. <i>Return type:</i> float
<code>aocv_derate_data_early_rise</code>	Returns AOCV derate values for a timing arc on an early data path with rise transitions. <i>Return type:</i> float
<code>aocv_derate_data_late_fall</code>	Returns AOCV derate values for a timing arc on a late data path with fall transitions. <i>Return type:</i> float
<code>aocv_derate_data_late_rise</code>	Returns AOCV derate values for a timing arc on a late data path with rise transitions. <i>Return type:</i> float
<code>aocv_derate_launch_clock_early_fall</code>	Returns graph based AOCV derate factors in AOCV mode for a given timing arc that is a part of early launch clock paths with fall sink pin transitions. <i>Return type:</i> float
<code>aocv_derate_launch_clock_early_rise</code>	Returns graph based AOCV derate factors in AOCV mode for a given timing arc that is a part of early launch clock paths with rise sink pin transitions. <i>Return type:</i> float
<code>aocv_derate_launch_clock_late_fall</code>	Returns graph based AOCV derate factors in AOCV mode for a given timing arc that is a part of late launch clock paths with fall sink pin transitions. <i>Return type:</i> float
<code>aocv_derate_launch_clock_late_rise</code>	Returns graph based AOCV derate factors in AOCV mode for a given timing arc that is a part of late launch clock paths with rise sink pin transitions. <i>Return type:</i> float
<code>aocv_stage_count_capture_clock_early</code>	Returns graph based AOCV stage count values in AOCV mode for a given timing arc that is a part of early capture clock paths. <i>Return type:</i> float
<code>aocv_stage_count_capture_clock_late</code>	Returns graph based AOCV stage count values in AOCV mode for a given timing arc that is a part of late capture clock paths. <i>Return type:</i> float
<code>aocv_stage_count_data_early</code>	Returns AOCV stage count values for a timing arc on an early data path. <i>Return type:</i> float
<code>aocv_stage_count_data_late</code>	Returns AOCV stage count values for a timing arc on a late data path. <i>Return type:</i> float
<code>aocv_stage_count_launch_clock_early</code>	Returns AOCV stage count values in AOCV mode for a given timing arc that is a part of early launch clock paths. <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

aocv_stage_count_launch_clock_late	Returns AOCV stage count values in AOCV mode for a given timing arc that is a part of late launch clock paths. <i>Return type:</i> float
arc_type	Returns the Liberty <code>timing_type</code> attribute of the arc. <i>Return type:</i> string
delay_max	Returns the delay for the maximum corner. For example, <code>get_property [get_arcs -of_objects [get_cells regbuf]] delay_max</code> <i>Return type:</i> float
delay_max_fall	Returns the fall delay for the max corner. <i>Return type:</i> float
delay_max_rise	Returns the rise delay for the max corner. <i>Return type:</i> float
delay_mean_max_fall	Returns the mean component of the maximum falling delay value for the timing arc. <i>Return type:</i> float
delay_mean_max_rise	Returns the mean component of the maximum rising delay value for the timing arc. <i>Return type:</i> float
delay_mean_min_fall	Returns the mean component of the minimum falling delay value for the timing arc. <i>Return type:</i> float
delay_mean_min_rise	Returns the mean component of the minimum rising delay value for the timing arc. <i>Return type:</i> float
delay_min	Returns the delay for the minimum corner. For example, <code>get_property [get_arcs -of_objects [get_cells clkbuf]] delay_min</code> <i>Return type:</i> float
delay_min_fall	Returns the fall delay for the minimum corner. <i>Return type:</i> float
delay_min_rise	Returns the rise delay for the minimum corner. <i>Return type:</i> float
delay_sigma_max	Returns the 1-sigma variation component of the maximum delay for the specified timing arc. <i>Return type:</i> float
delay_sigma_max_fall	Returns the 1-sigma variation component of the maximum falling delay for the specified timing arc. <i>Return type:</i> float
delay_sigma_max_rise	Returns the 1-sigma variation component of the maximum rising delay for the specified timing arc. <i>Return type:</i> float
delay_sigma_min	Returns the 1-sigma variation component of the minimum delay for the specified timing arc. <i>Return type:</i> float
delay_sigma_min_fall	Returns the 1-sigma variation component of the minimum falling delay for the specified timing arc. <i>Return type:</i> float
delay_sigma_min_rise	Returns the 1-sigma variation component of the minimum rising delay for the specified timing arc. <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

delta_delay_max	Returns the delta delay for the maximum corner. For example, <pre>get_property [get_arcs -of_objects [get_cells clkbuf]] delta_delay_max</pre> <i>Return type:</i> float
delta_delay_max_fall	Returns the delta fall delay for the max corner. <i>Return type:</i> float
delta_delay_max_rise	Returns the delta rise delay for the max corner. <i>Return type:</i> float
delta_delay_min	Returns the delta delay for the minimum corner. For example, <pre>get_property [get_arcs -of_objects [get_cells clkbuf]] delta_delay_min</pre> <i>Return type:</i> float
delta_delay_min_fall	Returns the delta fall delay for the min corner. <i>Return type:</i> float
delta_delay_min_rise	Returns the delta rise delay for the min corner. <i>Return type:</i> float
dynamic_delta_delay_max_fall	Returns the dynamic delay component of the maximum falling delay through the arc. For example, <pre>get_property -view func_worst_cmax_w_dynv [get_arcs -from i_0/A -to i_0/Y] dynamic_delta_delay_max_fall</pre> <i>Return type:</i> float
dynamic_delta_delay_max_rise	Returns the dynamic delay component of the maximum rising delay through the arc. For example, <pre>get_property -view func_worst_cmax_w_dynv [get_arcs -from i_0/A -to i_0/Y] dynamic_delta_delay_max_rise</pre>
dynamic_delta_delay_min_fall	Returns the dynamic delay component of the minimum falling delay through the arc. For example, <pre>get_property -view func_worst_cmax_w_dynv [get_arcs -from i_0/A -to i_0/Y] dynamic_delta_delay_min_fall</pre>
dynamic_delta_delay_min_rise	Returns the dynamic delay component of the minimum rising delay through the arc. For example, <pre>get_property -view func_worst_cmax_w_dynv [get_arcs -from i_0/A -to i_0/Y] dynamic_delta_delay_min_rise</pre>
has_mis_fall	Returns a value of <code>true</code>, if MIS (multi-input switching) derate falling value is specified for timing arcs in the library. <pre>get_property [get_arc -from UC/and1/A1 -to UC/and1/ZN] has_mis_fall</pre> <i>Return type:</i> boolean
has_mis_rise	Returns a value of <code>true</code>, if MIS (multi-input switching) derate rising value is specified for timing arcs in the library. <pre>get_property [get_arcs -from UC/or1/A1 -to UC/or1/ZN] has_mis_rise</pre> <i>Return type:</i> boolean
is_cellarc	Returns a value of <code>true</code> if the arc belongs to a cell. <i>Return type:</i> boolean
is_disabled	Returns a value of <code>true</code> if a <code>set_disable_timing</code> has been applied on the supplied library timing arc. <i>Return type:</i> boolean

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

mode	Returns the mode specified for the timing arc in the library. <i>Return type:</i> string
object_type	Returns the object type <code>timing_arc</code> . <i>Return type:</i> string
sdf_cond	Returns the <code>sdf_cond</code> expression specified for the timing arc in the library. <i>Return type:</i> string
sdf_cond_end	Returns the <code>sdf_cond_end</code> expression specified for the timing arc in the library. <i>Return type:</i> string
sdf_cond_start	Returns the <code>sdf_cond_start</code> expression specified for the timing arc in the library. <i>Return type:</i> string
sense	Returns the sense of the timing arc. <i>Return type:</i> string
sink_pin	Returns the sink pin of the delay arc, or the reference pin of the check arc. <i>Return type:</i> collection
source_pin	Returns the source pin of the delay arc, or the signal pin of the check arc. <i>Return type:</i> collection
ssi_derate_fall	Returns the SSI (simultaneously switching input) derate falling value specified on library cells. For example, <pre>set_timing_derate 0.5 -fall -data -early -input_switching [get_lib_cells Frlib_v1.0/AND2_X1], get_property [get_arcs -from UC/and1/A1 -to UC/and1/ZN] ssi_derate_fall</pre> <i>Return type:</i> float
ssi_derate_rise	Returns the SSI (simultaneously switching input) derate rising value specified on library cells. For example, <pre>set_timing_derate 0.6 -rise -data -early -input_switching [get_lib_cells Frlib_v1.0/AND2_X1] get_property [get_arcs -from UC/and1/A1 -to UC/and1/ZN] ssi_derate_rise</pre> <i>Return type:</i> float
when	Returns the <code>when</code> condition specified for the timing arc in the library. <i>Return type:</i> string
when_library_string	Returns the exact value of the Liberty 'when' attribute if specified for an arc. <pre>get_property [get_arc -from UC/buff/A -to UC/buff/Z] when_library_string</pre> <i>Return type:</i> string

Timing Path Properties

applied_exception_type	Returns the type of applied exceptions for a timing path. This property can be used to query dominant timing exceptions affecting a timing path. You can use this property to query on constrained/unconstrained timing paths. The possible return values are - <code>multi_cycle</code>, <code>path_delay</code> and <code>false_path</code> - indicating multicycle path, minimum/maximum delay path, and timing false path, respectively. The software will return one of these values for pin-based timing exceptions. However, the software returns "NA" when false paths are defined for clock objects, or if the paths do not have any related timing exception. <i>Return Type:</i> String
------------------------	---

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

arrival	Returns the arrival time at the timing path. <i>Return type:</i> float
arrival_mean	Returns the mean component of the arrival time for the timing path. <i>Return type:</i> float
arrival_sigma	Returns the sigma component of the arrival time for the timing path. <i>Return type:</i> float
arrival_skewness	Returns the skewness in the arrival time for a timing path. <i>Return type:</i> float
arrival_stddev	Returns the standard deviation of the arrival time for a timing path. <i>Return type:</i> float
arrival_wire_stddev	Returns the standard deviation of arrival time due to wire variation for the specified timing paths. get_property [report_timing -retime path_slew_propagation -split_delay -path_type full_clock -collection] arrival_wire_stddev <i>Return type:</i> float
capturing_arc	Returns the capturing arc for the specified timing path. get_property [report_timing -clock_to clk2 -collection] capturing_arc <i>Return type:</i> collection
capture_clock_path	Returns the capturing clock path of the timing path. This property can be used only for path_type full_clock. <i>Return type:</i> collection
capturing_clock	Returns the capturing clock of the timing path. <i>Return type:</i> collection
capturing_clock_close_edge_time	Returns the closing edge time of the capturing clock. <i>Return type:</i> float
capturing_clock_close_edge_type	Returns the capturing clock edge direction: rise or fall. <i>Return type:</i> string
capturing_clock_is_inverted	Returns a value of true if the phase of the clock at the end point of the capture clock path is opposite to the phase of the capturing clock at the source. <i>Return type:</i> boolean
capturing_clock_is_propagated	Returns a value of true if the capturing clock is in the propagated mode, at the clock end point. <i>Return type:</i> boolean
capturing_clock_latency	Returns the ideal or propagated latency of the capturing clock. <i>Return type:</i> float
capturing_clock_latency_mean	Returns the capturing clock mean latency. get_property [get_timing_paths] capturing_clock_latency_mean <i>Return type:</i> float
capturing_clock_latency_sigma	Returns the sigma variation for the capturing clock latency. get_property [report_timing -collection] capturing_clock_latency_sigma <i>Return type:</i> float
capturing_clock_latency_skewness	Returns the skewness variation for the capturing clock latency. get_property [report_timing -collection] capturing_clock_latency_skewness <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

capturing_clock_latency_stddev	Returns the stddev variation for the capturing clock latency. <pre>get_property [report_timing -collection] capturing_clock_latency_stddev</pre> <i>Return type:</i> float
capturing_clock_open_edge_type	Returns the opening edge time of the capturing clock. <i>Return type:</i> string
capturing_clock_pin	Returns the name of the capturing clock pin. <i>Return type:</i> collection
capturing_point_name	Returns the name of the capturing point. <pre>get_property [report_timing -collection] capturing_point_name</pre> <i>Return type:</i> string
capturing_clock_source_arrival_time	Returns the capturing clock arrival time at the clock root pin/port of the capturing clock path. This value is a sum of the clock edge time and the source latency constraints specified at the clock root. Note: You must specify <code>report_timing -path_type full_clock</code> in order to return capturing clock arrival time information. <i>Return type:</i> float
capturing_clock_source_arrival_time_mean	Returns the mean component of the capturing clock arrival time at the clock root pin/port of the capturing clock path. <i>Return type:</i> float
capturing_clock_source_arrival_time_sigma	Returns the sigma component of the capturing clock arrival time at the clock root pin/port of the capturing clock path. <i>Return type:</i> float
capturing_point	Returns a pointer to the end point of the timing path. <i>Return type:</i> collection
capturing_point_is_level_sensitive	Returns a value of <code>true</code> if the end point of the data path is level sensitive. <i>Return type:</i> boolean
check_delay	Returns the check delay value for the timing path. <i>Return type:</i> float
check_delay_mean	Returns the mean component of the check delay value for the timing path. <i>Return type:</i> float
check_delay_sigma	Returns the sigma component of the check delay value for the timing path. <i>Return type:</i> float
check_delay_sigma_detail	Returns separate delay sigma variation due to correlated variation of current stage, the correlated variation of previous stage, and uncorrelated variation of the current stage on the timing paths, when the <code>delaycal_socv_accuracy_mode</code> attribute is set to <code>medium</code>, <code>high</code>, or <code>ultra</code>. <pre>get_property [report_timing -collection] check_delay_sigma_detail</pre> <i>Return type:</i> string
check_delay_skewness_detail	Returns separate delay skewness variation due to correlated variation of current stage, the correlated variation of previous stage, and uncorrelated variation of the current stage on the timing path when the <code>delaycal_socv_accuracy_mode</code> attribute is set to <code>high</code> or <code>ultra</code>. <pre>get_property [report_timing -collection] check_delay_skewness_detail</pre> <i>Return type:</i> string
check_delay_skewness	Returns the skewness component of the check delay value for a timing path. <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

check_delay_stddev	Returns the check delay standard deviation component of the check delay value for a timing path. <i>Return type:</i> float
check_type	Returns the check type for the timing path. <i>Return type:</i> float
clock_path_end_point	Returns the last point in the capture clock path of the timing path. <i>Return type:</i> collection
clock_source_jitter	Returns the clock source jitter value applied on a specific clock. <i>Return type:</i> float
clock_uncertainty	Returns the clock uncertainty of the timing path. <i>Return type:</i> float
cprr_adjustment	Returns the CPPR adjustment of the timing path. <i>Return type:</i> float
cprr_adjustment_mean	Returns the mean component of CPPR adjustment of the timing path. <i>Return type:</i> float
cprr_adjustment_sigma	Returns the sigma component of CPPR adjustment of the timing path. <i>Return type:</i> float
cprr_adjustment_skewness	Returns the skewness component of CPPR adjustment of a timing path. <i>Return type:</i> float
cprr_adjustment_stddev	Returns the standard deviation component of CPPR adjustment of a timing path. <i>Return type:</i> float
cprr_branch_point	Returns the CPPR branch point of the given timing path. <i>Return Type:</i> collection
cumulative_manhattan_length	Returns the cumulative manhattan length for wires of timing paths. <code>get_property [report_timing -collection] cumulative_manhattan_length</code> <i>Return type:</i> float
cumulative_manhattan_length_x	Returns the cumulative horizontal length for wires of timing paths. <code>get_property [report_timing -collection] cumulative_manhattan_length_x</code> <i>Return type:</i> float
cumulative_manhattan_length_y	Returns the cumulative vertical length for wires of timing paths. <code>get_property [report_timing -collection] cumulative_manhattan_length_y</code> <i>Return type:</i> float
cycle_adjustment	Returns the cycle adjustment value for the timing paths. <code>get_property [report_timing -early -collection] cycle_adjustment</code> <i>Return type:</i> float
drive_adjustment	Returns the drive adjustment value for the timing path. <i>Return type:</i> float
drive_adjustment_mean	Returns the mean component of the drive adjustment value for the timing path. <i>Return type:</i> float
drive_adjustment_sigma	Returns the sigma component of the drive adjustment value for the timing path. <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

external_delay	Returns the value of the <code>set_output_delay</code> constraint at the end point of the path. <i>Return type:</i> float
gba_slack	Returns the original slack time for the timing path – before PBA retiming is performed. <i>Return type:</i> float
gba_slack_mean	Returns the mean component of the slack time for the timing path – before PBA retiming is performed. <i>Return type:</i> float
gba_slack_sigma	Returns the sigma component of the slack time for the timing path – before PBA retiming is performed. <i>Return type:</i> float
gba_slack_skewness	Returns the skewness variation of slack for GBA analysis. <code>get_property [report_timing -skip_io_paths -not_through [get_ports *] -collection -early] gba_slack_skewness</code> <i>Return type:</i> float
gba_slack_stddev	Returns the standard deviation of slack for GBA analysis. <code>get_property [report_timing -collection] gba_slack_stddev</code> <i>Return type:</i> float
hold	Returns the hold time at the end point of the path. <i>Return type:</i> float
is_borrow_edge_adjust	Returns a value of <code>true</code> if borrow edge adjustment is set on the timing paths. <code>get_property [report_timing -through micro_dec/LOCKUP/d -collection] is_borrow_edge_adjust</code> <i>Return type:</i> boolean
is_clock_gating_hold	Returns a value of <code>true</code> if the path is a clock gating hold check. <i>Return type:</i> boolean
is_clock_gating_setup	Returns a value of <code>true</code> if the path is a clock gating setup check. <i>Return type:</i> boolean
is_coverage_violating_path	Returns a value of <code>true</code> if a timing path contains violations from the timing check coverage. <code>get_property [report_timing -collection] is_coverage_violating_path</code> <i>Return type:</i> boolean
is_path_borrowing	Returns a value of <code>true</code> if the specified path is a time borrowing path. <code>get_property [report_timing -collection -to micro_dec/LOCKUP/d] is_path_borrowing</code> <i>Return type:</i> boolean
is_retimed	Returns a value of <code>true</code> if the timing path is recalculated using path-based analysis. <code>get_property [report_timing -retime path_slew_propagation -collection] is_retimed</code> <i>Return type:</i> boolean
is_time_given	Returns a value of <code>true</code> if for the specified timing path time is applied to the starting point. <code>get_property [report_timing -to lat/Q -collection] is_time_given</code> <i>Return type:</i> boolean
is_transparent_latch	Returns a value of <code>true</code> if the path is a transparent latch path. <i>Return type:</i> boolean
launching_clock_latency_mean	Returns mean component of latency of the launching clock. <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

launching_clock_latency_sigma	Returns sigma component of latency of the launching clock. <i>Return type:</i> float
launch_clock_path	Returns the launching clock path of the timing path. This property can be used only for path_type full_clock. <i>Return type:</i> collection
launching_clock	Returns the launching clock of the timing path. <i>Return type:</i> collection
launching_clock_is_inverted	Returns a value of true if the phase of the clock at the start point of the launch clock path is opposite to the phase of the launch clock at the source. <i>Return type:</i> boolean
launching_clock_is_propagated	Returns a value of true if the launching clock is in the propagated mode, at the clock end point. <i>Return type:</i> boolean
launching_clock_latency	Returns the ideal or propagated latency of the launching clock. <i>Return type:</i> float
launching_clock_latency_skewness	Returns the skewness variation for the launching clock latency. get_property [report_timing -clock_from clk2 -path_type full_clock -collection] launching_clock_latency_skewness <i>Return type:</i> float
launching_clock_latency_stddev	Returns the standard deviation for the launching clock latency. get_property [report_timing -clock_from clk2 -path_type full_clock -collection] launching_clock_latency_stddev <i>Return type:</i> float
launching_clock_root_open_edge_type	Returns the open edge type for the root of the launching clock for the specified timing path. get_property [report_timing -collection] launching_clock_open_edge_type <i>Return type:</i> string
launching_clock_source_arrival_time_skewness	Returns the skewness variation of the launching clock arrival time at the source. get_property [report_timing -clock_from glck -path_type full_clock -collection] launching_clock_source_arrival_time_skewness <i>Return type:</i> float
launching_clock_source_arrival_time_stddev	Returns the standard deviation of the launching clock arrival time at the source. get_property [report_timing -clock_from glck -path_type full_clock -collection] launching_clock_source_arrival_time_stddev <i>Return type:</i> float
launching_clock_open_edge_time	Returns the opening edge time of the launching clock. <i>Return type:</i> float
launching_clock_open_edge_type	Returns the launching clock's open edge type at the launching point of the timing path. In case of input delay path with respect to a clock it will return the clock edge type, which is triggering the path. <i>Return type:</i> string
launching_clock_source_arrival_time	Returns the launching clock arrival time at the clock route pin/port of the launching clock path. This value is the sum of the clock edge time plus any source latency constraint specified at the clock root. Note: You must specify <code>report_timing -path_type full_clock</code> in order to return launching clock arrival time information. <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

launching_clock_source_arrival_time_mean	Returns the mean component of the launching clock arrival time at the clock route pin/port of the launching clock path. <i>Return type:</i> float
launching_clock_source_arrival_time_sigma	Returns the sigma component of the launching clock arrival time at the clock route pin/port of the launching clock path. <i>Return type:</i> float
launching_input_delay	Returns the input delay if the timing path starts from an input or output port. This value is set using <code>set_input_delay</code> . <i>Return type:</i> float
launching_point	Returns a pointer to the start point of the timing path. <i>Return type:</i> collection
launching_point_is_level_sensitive	Returns a value of <code>true</code> if the begin point of the data path is level sensitive. <i>Return type:</i> boolean
launching_point_name	Returns the name of the launching point. <code>get_property [report_timing -collection] launching_point_name</code> <i>Return type:</i> string
lending_path_d_pin	Returns the D pin for the specified lending timing path. <code>get_property [report_timing -to lat/Q -collection -path_type full_clock] lending_path_d_pin</code> <i>Return type:</i> string
nets	Returns the nets for the specified timing paths. For example, <code>get_property [get_timing_paths] nets</code> <i>Return type:</i> string
num_cell_arcs	Returns the number of cell arcs in the given timing path. For example, <code>set path [report_timing -from <start_point> -to <end_point> -collection]</code> <code>get_property \$path num_cell_arcs</code> <i>Return Type:</i> float
num_net_arcs	Returns the number of net arcs in the given timing path. For example, <code>set path [report_timing -from <start_point> -to <end_point> -collection]</code> <code>get_property \$path num_net_arcs</code> <i>Return Type:</i> float
object_type	Returns the object type <code>timing_path</code> . <i>Return type:</i> string
other_end_arrival	Returns the capture clock path's begin time, In case of SOCV analysis, mean+N*sigma quantile value is reported. <i>Return type:</i> float
other_end_arrival_mean	Returns the mean component of capture clock path's begin time. <i>Return type:</i> float
other_end_arrival_sigma	Returns the sigma component of capture clock path's begin time. <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

other_end_arrival_sigma_detail	Returns separate other end arrival time variation due to correlated and uncorrelated effects on timing paths, when <code>set_delay_cal_mode -socv_accuracy_mode</code> is set to medium, high, or ultra. <pre>get_property [report_timing -collection] other_end_arrival_sigma_detail</pre> <i>Return type:</i> string
other_end_arrival_skewness	Returns the skewness of other end arrival time for the timing path. <pre>get_property [report_timing -collection] other_end_arrival_skewness</pre> <i>Return type:</i> float
other_end_arrival_skewness_detail	Returns separate other end arrival time skewness due to correlated and uncorrelated effects on timing path, when <code>set_delay_cal_mode -socv_accuracy_mode</code> is set to high or ultra. <pre>get_property [report_timing -collection] other_end_arrival_skewness_detail</pre> <i>Return type:</i> string
other_end_arrival_stddev	Returns the standard deviation of other end arrival time for a timing path. <pre>get_property [report_timing -collection] other_end_arrival_stddev</pre> <i>Return type:</i> float
path_adjust_value	Returns the path adjust value asserted on timing paths using the <code>set_path_adjust</code> command. <pre>get_property [report_timing -through UMC/and1 -collection] path_adjust_value</pre> <i>Return type:</i> float
path_cell_delay	Returns the cell delay for the timing path. <pre>get_property [report_timing -collection] path_cell_delay</pre> <i>Return type:</i> float
path_cell_delay_mean	Returns the mean cell delay for the timing path. <pre>get_property [report_timing -collection] path_cell_delay_mean</pre> <i>Return type:</i> float
path_cell_delay_sigma	Returns the sigma component of cell delay for the timing path. <pre>get_property [report_timing -collection] path_cell_delay_sigma</pre> <i>Return type:</i> float
path_delay_adjustment	Returns the delay adjustment asserted on paths using the <code>set_max_delay/set_min_delay</code> command. <pre>get_property [report_timing -through UMC/and1 -collection] path_delay_adjustment</pre> <i>Return type:</i> float
path_delay_wire_stddev	Returns the standard deviation of path delay due to wire variation for the specified timing path. <pre>get_property [report_timing -retime path_slew_propagation -split_delay -path_type full_clock -collection] path_delay_wire_stddev</pre> <i>Return type:</i> float
path_net_delay	Returns the net delay for the timing paths. <pre>get_property [report_timing -collection] path_net_delay</pre> <i>Return type:</i> float
path_net_delay_mean	Returns the mean net delay for the timing paths. <pre>get_property [report_timing -collection] path_net_delay_mean</pre> <i>Return type:</i> float
path_net_delay_sigma	Returns the sigma variation for path net delay for the specified timing path. <pre>get_property [report_timing -retime path_slew_propagation -split_delay -path_type full_clock -collection] path_net_delay_sigma</pre> <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

path_delay	Returns the delay of data path component of a timing path. In case of SOCV analysis, mean+N*sigma quantile delay is reported. <i>Return type:</i> float
path_delay_mean	Returns the mean delay component of data path of a timing path. <i>Return type:</i> float
path_delay_sigma	Returns the sigma delay component of data path of a timing path. <i>Return type:</i> float
path_group	Returns a collection of path groups associated with the timing path. <i>Return type:</i> collection
path_group_name	Returns the name of the path group for the timing path. <i>Return type:</i> string
path_type	Returns a value of <code>max</code> if the path is for setup, or <code>min</code> if the path is for hold. <i>Return type:</i> string
period	Returns the time duration between the launching clock edge and capturing clock edge of the path. The launching clock edge indicates the launching clock time when the signal at the start point of the path is launched, and the capturing clock edge indicates the capturing clock time when the signal at the path end point is captured. For both of these edges, their ideal clock cycle times are considered for calculating the path period. For those paths, where either the launching or capturing clock is absent or asynchronous, the property prints NA. The property works for both setup (late) paths, and hold (early) paths. Usually the path period is decided by a simple subtraction of effective launching edge from the effective capturing edge of the path. Under certain scenario, these edges are moved back or ahead in time, and accordingly the path period is also affected. The following factors impact period: a) Multicycle paths: The launch or capture edge will move by the cycle factor depending on the type of multi cycle exception, and this cycle factor is taken into account while computing the period. b) Max delay paths: Any <code>max_delay</code> specification overrides the relationship between capturing and launching edges and sets it to the <code>max_delay</code> value. The period of the path is thus returned as equal to the specified max delay value for sequential max delay paths. Note that a combinational max delay path has at least one of the launch or capture clocks either absent or asynchronous, and hence the effective period is not calculated for them. c) Clock uncertainty: The uncertainties applicable to the interaction of launch and capture clocks for this particular path also affect the effective edges, but this factor is not taken into consideration while computing the period. d) Unconstrained paths: The unconstrained paths have no capture clock and hence period can not be calculated for them. The property will return NA. <i>Return value:</i> float For more information, see "Examples".
phase_shift	Returns the phase shift for the timing paths. <code>get_property [report_timing -collection] phase_shift</code> <i>Return type:</i> float
recovery	Returns the recovery time at the end point of the path. <i>Return type:</i> float
removal	Returns the removal time at the end point of the path. <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

required_time	Returns the required time for the timing path. <i>Return type:</i> float
required_time_mean	Returns the mean component of the required time for the timing path. <i>Return type:</i> float
required_time_sigma	Returns the sigma component of the required time for the timing path. <i>Return type:</i> float
required_time_sigma_detail	Returns separate required time variation due to correlated and uncorrelated effects on timing paths, when set_delay_cal_mode -socv_accuracy_mode is set to medium, high, or ultra. get_property [report_timing -collection] required_time_sigma_detail <i>Return type:</i> string
required_time_skewness	Returns the skewness in required time of a timing path. get_property [report_timing -collection] required_time_skewness <i>Return type:</i> float
required_time_skewness_detail	Returns separate required time skewness due to correlated and uncorrelated effects on timing paths, when set_delay_cal_mode -socv_accuracy_mode is set to high or ultra. get_property [report_timing -collection] required_time_skewness_detail <i>Return type:</i> string
required_time_stddev	Returns the standard deviation of the required time for a timing path. get_property [report_timing -collection] required_time_stddev <i>Return type:</i> float
setup	Returns the setup time at the end point of the path. <i>Return type:</i> float
skew	Returns the latency difference between the launching clock latency and capturing clock latency of the path (after considering the effect of CPPR factor). <i>Return type:</i> float
slack	Returns the slack time for the timing path. <i>Return type:</i> float
slack_mean	Returns the mean component of the slack time for the timing path. <i>Return type:</i> float
slack_sigma	Returns the sigma component of the slack time for the timing path. <i>Return type:</i> float
slack_sigma_detail	Returns separate slack sigma variation due to correlated and uncorrelated effects on timing paths, when set_delay_cal_mode -socv_accuracy_mode is set to medium, high, or ultra. get_property [report_timing -collection] slack_sigma_detail <i>Return type:</i> string
slack_skewness	Returns slack skewness for the timing paths. get_property [report_timing -collection] slack_skewness <i>Return type:</i> float
slack_skewness_detail	Returns separate slack skewness variation due to correlated and uncorrelated effects on timing paths, when set_delay_cal_mode -socv_accuracy_mode is set to high or ultra. get_property [report_timing -collection] slack_skewness_detail <i>Return type:</i> string

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

slack_wire_stddev	Returns the standard deviation of slack due to wire variation for the specified timing paths. <pre>get_property [report_timing -retime path_slew_propagation -split_delay -path_type full_clock -collection] slack_wire_stddev</pre> <i>Return type:</i> float
slack_stddev	Returns standard deviation of the slack. <i>Return type:</i> float
spatial_distance	Reports the bounding box distance for spatial derating applied on a timing path. <i>Return type:</i> float
spatial_distance_cell	Reports the bounding box distance for cells. <i>Return type:</i> float
spatial_distance_net	Reports the bounding box distance for nets. <i>Return type:</i> float
time_borrowed	Returns the amount of time borrowed from the timing end point. <i>Return type:</i> float
time_lent	Returns the amount of time lent to the timing start point. <i>Return type:</i> float
timing_points	Returns a collection of the timing points that make up the path. <i>Return type:</i> collection
total_cell_delta_delay	Returns the total cell incremental delay present in the timing paths. <pre>get_property [report_timing -through clk_I1/ZN -unconstrained -collection] total_cell_delta_delay</pre> <i>Return type:</i> float
total_delta_delay	Returns the total incremental delay present in the timing paths. <pre>get_property [report_timing -through clk_L1_I1/ZN -unconstrained -collection] total_delta_delay</pre> <i>Return type:</i> float
total_net_delta_delay	Returns the total net incremental delay present in the timing paths. <pre>get_property [report_timing -through clk_L1_I1/ZN -unconstrained -collection] total_net_delta_delay</pre> <i>Return type:</i> float
underated_slack	Returns the underated slack time for the timing path. The underated slack value is available with timing path object only if it is generated using the <code>-derate</code> parameter. <i>Return type:</i> float
view_name	Returns the analysis view name associated with the timing path. <i>Return type:</i> string
wire_variation_capacitance_slack_delta	Returns a list of lists with slack shift due to capacitance of each layer. <i>Return type:</i> string
wire_variation_layer_info	Returns layer information for wire variation analysis. <i>Return type:</i> string
wire_variation_layer_slack_delta	Returns a list of lists with total slack shift per layer. <i>Return type:</i> string
wire_variation_resistance_slack_delta	Returns a list of lists with slack shift due to resistance of each layer. <i>Return type:</i> string

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

wire_variation_slack_shift	Returns a list of lists with slack shift. <i>Return type:</i> string
wire_variation_slack_shift_total	Returns a list of lists with total slack shift. <i>Return type:</i> string
worst_cell_delay	Returns the worst cell delay of the timing paths. get_property [report_timing -collection] worst_cell_delay <i>Return type:</i> float
worst_cell_delay_mean	Returns the worst mean cell delay of the timing paths. get_property [report_timing -collection] worst_cell_delay_mean <i>Return type:</i> float
worst_cell_delay_sigma	Returns the sigma component for worst cell delay in the timing paths. get_property [report_timing -collection] worst_cell_delay_sigma <i>Return type:</i> float
worst_cell_delay_skewness	Returns the skewness for worst cell delay in the timing paths. get_property [report_timing -collection] worst_cell_delay_skewness <i>Return type:</i> float
worst_cell_delay_stddev	Returns the standard deviation for worst cell delay in the timing paths. get_property [report_timing -collection] worst_cell_delay_stddev <i>Return type:</i> float
worst_cell_delta_delay	Returns the worst cell delta delay of the timing paths. get_property [report_timing -through clk_L16/ZN -unconstrained -collection] worst_cell_delta_delay <i>Return type:</i> float
worst_delay	Returns the worst delay for the timing paths. get_property [report_timing -collection] worst_delay <i>Return type:</i> float
worst_delay_mean	Returns the mean worst delay for the timing paths. get_property [report_timing -collection] worst_delay_mean <i>Return type:</i> float
worst_delay_sigma	Returns the sigma component for worst delay in the timing paths. get_property [report_timing -collection] worst_delay_sigma <i>Return type:</i> float
worst_delay_skewness	Returns the skewness for worst cell delay in the timing paths. get_property [report_timing -collection] worst_delay_skewness <i>Return type:</i> float
worst_delay_stddev	Returns the standard deviation for worst cell delay in the timing paths. get_property [report_timing -collection] worst_delay_stddev <i>Return type:</i> float
worst_delta_delay	Returns the worst delta delay for the timing paths. get_property [report_timing -through clk_L16/ZN -unconstrained -collection] worst_delta_delay <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

worst_manhattan_length	Returns the worst manhattan length of the timing paths. <pre>get_property [report_timing -collection] worst_manhattan_length</pre> <i>Return type:</i> float
worst_manhattan_length_net_name	Returns the name of the net with worst manhattan length in the timing paths. <pre>get_property [report_timing -collection] worst_manhattan_length_net_name</pre> <i>Return type:</i> string
worst_net_delay	Returns the worst net delay for the timing paths. <pre>get_property [report_timing -collection] worst_net_delay</pre> <i>Return type:</i> float
worst_net_delay_mean	Returns the mean worst net delay for the timing paths. <pre>get_property [report_timing -collection] worst_net_delay_mean</pre> <i>Return type:</i> float
worst_net_delay_sigma	Returns the sigma variation for worst net delay for the specified timing paths. <pre>get_property [report_timing -retime path_slew_propagation -split_delay -path_type full_clock -collection] worst_net_delay_sigma</pre> <i>Return type:</i> float
worst_net_delay_skewness	Returns the skewness for the worst net delay for the specified timing paths. <pre>get_property [report_timing -retime path_slew_propagation -split_delay -path_type full_clock -collection] worst_net_delay_skewness</pre> <i>Return type:</i> float
worst_net_delay_stddev	Returns the standard deviation for the worst net delay for the specified timing paths. <pre>get_property [report_timing -retime path_slew_propagation -split_delay -path_type full_clock -collection] worst_net_delay_stddev</pre> <i>Return type:</i> float
worst_net_delta_delay	Returns the worst net incremental delay for the timing paths. <pre>get_property [report_timing -through clk_L16/ZN -unconstrained -collection] worst_net_delta_delay</pre> <i>Return type:</i> float

Timing Point Properties

adjustment_in_delay	Returns the adjustment in delay for the specified timing point of a timing path. <pre>get_property [get_property [get_property [report_timing -from dff0/Q -to dff2/D -path_type full_clock -collection] capture_clock_path] timing_points] adjustment_in_delay</pre> <i>Return type:</i> float
arrival	Returns the arrival time at the timing point. <i>Return type:</i> float
arrival_mean	Returns the mean component of the arrival time at the timing point. <i>Return type:</i> float
arrival_sigma	Returns the sigma component of arrival time at the timing point. <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

arrival_skewness	Returns the skewness in arrival for a timing point. <pre>get_property [get_property [report_timing -collection] timing_points] arrival_skewness</pre> <i>Return type:</i> float
arrival_stddev	Returns the standard deviation of arrival time for a timing point. <pre>get_property [get_property [report_timing -collection] timing_points] arrival_stddev</pre> <i>Return type:</i> float
arrival_wire_stddev	Returns the standard deviation of arrival time due to wire variation for the specified timing point. <pre>get_property [get_property [report_timing -retime path_slew_propagation -split_delay -path_type full_clock -collection] timing_points] arrival_wire_stddev</pre> <i>Return type:</i> float
delay	Returns the delay value for the timing point. <i>Return type:</i> float
delay_mean	Returns the mean component of the delay value for the timing point. <i>Return type:</i> float
delay_sigma	Returns the sigma component of the delay value for the timing point. <i>Return type:</i> float
delay_skewness	Returns the standard deviation of slew due to wire variation for the specified timing points. <pre>get_property [get_property [report_timing -collection] timing_points] delay_skewness</pre> <i>Return type:</i> float
delay_stddev	Returns the standard deviation of delay for the timing points. <pre>get_property [get_property [report_timing -collection] timing_points] delay_stddev</pre> <i>Return type:</i> float
delay_wire_stddev	Returns the standard deviation of delay due to wire variation for the specified timing points. <pre>get_property [get_property [report_timing -retime path_slew_propagation -split_delay -path_type full_clock -collection] timing_points] delay_wire_stddev</pre> <i>Return type:</i> float
direction	Returns the direction of the timing points. <pre>get_property [get_property [report_timing -collection] timing_points] direction</pre> <i>Return type:</i> string
delta_delay	Returns the incremental delay on a timing point after SI analysis is performed. <i>Return type:</i> float
fanin_arc	Returns the delay arc for the corresponding timing point. <i>Return type:</i> collection
hierarchical_name	Returns the hierarchical name of the timing points. <pre>get_property [get_property [report_timing -collection] timing_points] hierarchical_name</pre> <i>Return type:</i> string
is_fanin_arc_cellarc	Returns a value of <code>true</code> if the fan in arc of the timing points is a cell arc. <pre>get_property [get_property [report_timing -through UMC/and1 -collection] timing_points] is_fanin_arc_cellarc</pre> <i>Return type:</i> boolean
is_hierarchical	Returns a value of <code>true</code> if the timing point is hierarchical. <pre>get_property [get_property [report_timing -through UMC/latch/D -unconstrained -collection] timing_points] is_hierarchical</pre> <i>Return type:</i> boolean

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

net	Returns the net for the timing points. <pre>get_property [get_property [report_timing -collection] timing_points] net</pre> <i>Return type:</i> collection
object_type	Returns the <code>timing_point</code> object type. <i>Return type:</i> string
pin	Returns a collection of pins or ports of the timing point. <i>Return type:</i> collection
retime_mis_delta	Returns the retime MIS (multi-input switching) delta delay of PBA paths. <i>Return type:</i> float
si_object	Returns the <code>si_victim</code> object type that can be used to query SI delay attributes. Note: To enable SI objects, you can set the <code>timing_report_enable_si_debug</code> global variable to <code>true</code>.
slew	Returns the path-specific slew of the timing point. <i>Return type:</i> float
slew_mean	Returns the mean component of the path-specific slew of the timing point. <i>Return type:</i> float
slew_sigma	Returns the sigma component of the path-specific slew of the timing point. <i>Return type:</i> float
slew_skewness	Returns the skewness of slew for the specified timing points. <pre>get_property [get_property [report_timing -collection] timing_points] slew_skewness</pre> <i>Return type:</i> float
slew_stddev	Returns the standard deviation of slew for the specified timing points. <pre>get_property [get_property [report_timing -collection] timing_points] slew_stddev</pre> <i>Return type:</i> float
slew_wire_stddev	Returns the standard deviation of slew due to wire variation for the specified timing points. <pre>get_property [get_property [report_timing -retime path_slew_propagation -split_delay -path_type full_clock -collection] timing_points] slew_wire_stddev</pre> <i>Return type:</i> float
spatial_derate	Returns the spatial derate on the timing points. <pre>get_property [get_property [report_timing -collection] timing_points] spatial_derate</pre> <i>Return type:</i> float
spatial_derate_sigma	Returns the sigma component of spatial derate on the timing points. <pre>get_property [get_property [report_timing -collection] timing_points] spatial_derate_sigma</pre> <i>Return type:</i> float
ssi_derate	Returns the path-specific SSI derate of the timing points. <pre>get_property [index_collection [get_property [report_timing -collection] timing_points] 4] ssi_derate</pre> <i>Return type:</i> float
total_derate	Returns the total derate on the timing points. <pre>get_property [get_property [report_timing -collection] timing_points] total_derate</pre> <i>Return type:</i> float

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

total_derate_sigma	Returns the sigma component of total derate on the timing points. <pre>get_property [get_property [report_timing -collection] timing_points] total_derate_sigma</pre> <i>Return type:</i> float
transition_type	Returns the transition type of the timing point - rise or fall. <i>Return type:</i> string
user_derate	Returns the derate set using the <code>set_timing_derate</code> command on the timing points. <pre>get_property [get_property [report_timing -collection] timing_points] user_derate</pre> <i>Return type:</i> float
user_derate_sigma	Returns the sigma variation derate set using the <code>set_timing_derate</code> command on the timing points. <pre>get_property [get_property [report_timing -collection] timing_points] user_derate_sigma</pre> <i>Return type:</i> float
voltage	Returns the voltage at a timing points. <pre>get_property [get_property [report_timing -collection] timing_points] voltage</pre> <i>Return type:</i> float
vt skew_mean_derate	Reports the mean derating factors of voltage threshold (VT) skew for a timing point. <i>Return type:</i> float
vt skew_sigma_derate	Reports the sigma derating factors of voltage threshold (VT) skew for a timing point. <i>Return type:</i> float

Signal Integrity Attributes

The signal integrity attributes can be queried using the following object types:

- `si_victim`
- `si_attacker`
- `noise_victim`: Returns a collection of `si_victim` objects that can be used to query glitch attributes.
Return type: collection

These are explained below.

si_victim Properties

The following information is saved under the `si_victim` object type:

Property Name	Description	Valid for Properties		
		si_victim	si_objec t	noise_vi ctim
annotated_glitch	Reports the annotated glitch. <i>Return type:</i> float	N	N	Y
attackers	Returns a collection of <code>si_attacker</code> objects that can be used to query attacker attributes. <i>Return type:</i> collection	Y	Y	Y
capacitance	Returns the total capacitance of victim net. <i>Return type:</i> float	Y	Y	Y

Litmus Text Command Reference
Advanced Timing Tcl Scripting Commands--get_property

coupling_capacitance	Returns the total coupling capacitance of a victim net. <i>Return type:</i> float	Y	Y	Y
glitch_area	Returns the glitch area. <i>Return type:</i> float	N	N	Y
glitch_width	Returns the glitch width. <i>Return type:</i> float	N	N	Y
hierarchical_name	Returns the hierarchical name of a victim net. <i>Return type:</i> string	Y	Y	Y
input_peak	Returns the Receiver Input Peak (RIP) value. <i>Return type:</i> float	N	N	Y
input_peak_slack	Returns the Receiver Input Peak (RIP) slack value. <i>Return type:</i> float	Y	N	N
input_failure_threshold	Returns the failure threshold at RIP. This will always be reported irrespective of whether failure happens at RIP or ROP. <i>Return type:</i> float	N	N	Y
level	Returns the noise level - VL or VH. <i>Return type:</i> string	N	N	Y
num_active_attackers	Returns the number of active attackers. <i>Return type:</i> float	Y	Y	Y
num_attackers	Returns the number of total attackers (distinct + accumulated small attacker - as 1). The attackers that are dropped from the accumulated small attackers will not be included with the total attackers. <i>Return type:</i> float	Y	Y	Y
object_type	Returns the object type si_victim. <i>Return type:</i> string	Y	Y	Y
prop_glitch	Reports the propagated glitch seen at the victim driver output. <i>Return type:</i> float	N	N	Y
receiver_peak	Returns the Receiver Output Peak (ROP) value. <i>Return type:</i> float	N	N	Y
receiver_peak_slack	Returns the Receiver Output Peak (ROP) slack value. <i>Return type:</i> string	Y	N	N

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

receiver_peak_threshold	Returns the failure threshold at ROP. This will always be reported irrespective of whether failure happens at RIP or ROP. <i>Return type:</i> float	N	N	Y
resistance	Returns the total resistance of a victim net. <i>Return type:</i> string	Y	Y	Y
transition	Returns the victim transition direction (rise or fall). <i>Return type:</i> string	Y	Y	N
view_type	Returns the type of analysis (early or late). <i>Return type:</i> string	Y	Y	N

si_attacker Properties

The following Information is saved under the `si_attacker` object type for each attacker of the victim net in the path:

Property Name (for each attacker retrieved from a victim)	Description
coupling_capacitance	Returns the coupling of a victim net. <i>Return type:</i> float
hierarchical_name	Returns the hierarchical name of an attacker. <i>Return type:</i> string
is_active	Returns whether an attacker is active or not. <i>Return type:</i> string
noise_peak	Returns the Xtalk delay bump induced by an attacker. <i>Return type:</i> float
object_type	Returns the <code>si_attacker</code> object type. <i>Return type:</i> string
slew	Returns the slew of attackers. <i>Return type:</i> float
state	Returns the status of attackers. The status descriptions are given below: • SY: Synchronous • INF: Attacker with infinite timing window • PE: Filtered due to physical exclusion • UF: Filtered by user • CN: Filtered because it is a constant net • SB: Filtered due to small bump • TA: Filtered because of timing window does not overlap • LC: Filtered due to logical correlation • NA: Attacker status is not known <i>Return type:</i> string

Examples

The following `get_property` example shows the usage of SI attributes to query noise information:

```
source pinlist.tcl
set chan [open attribute.log w]
foreach pin $pinList {
    set vicArray [get_property [get_pins $pin] noise_victim]
    foreach_in_collection vic $vicArray {
        set input_peak [ get_property $vic input_peak]
        #if { $input_peak > 250 } {}

        set vic_name [ get_property $vic hierarchical_name ]
        set x [ get_property [get_net -of_objects $vic_name ] name ]
        set level [ get_property $vic level]
        set ccap [ get_property $vic capacitance]
        set vdd [get_property [get_pins $vic_name] power_rail_voltage_max]
        set libs [get_libs -of_objects [get_lib_cells -of_objects [get_cells -of_objects [get_pins $vic_name]]]]
        foreach_in_collection libName $libs {set library [get_property $libName hierarchical_name]}
        set prop_glitch [ get_property $vic prop_glitch]
        set cap [format %.2f $ccap]
        set annotated_glitch [ get_property $vic annotated_glitch]
        set receiver_peak [ get_property $vic receiver_peak]
        set input_peak_threshold [ get_property $vic input_failure_threshold ]
        set receiver_peak_threshold [ get_property $vic receiver_peak_threshold]
        puts $chan "-----"
        puts $chan "Peak(mV) Level TotalCap(fF) Vdd(V) Library VictimNet"
        puts $chan "$input_peak $level $cap $vdd $library $vic_name {$x}"

        if { $receiver_peak > 0 } {
            puts $chan "Receiver output peak:"
            puts $chan "Value ReceiverNet"
            puts $chan "$receiver_peak ( $receiver_peak_threshold) $vic_name (CELL)"
        }

        if { $input_peak > 0 } {
            puts $chan "Receiver Input peak:"
            puts $chan "Value ReceiverNet"
            puts $chan "$input_peak ( $input_peak_threshold) $vic_name (CELL)"
        }

        puts $chan "\nConstituents:"
        puts $chan "Source Peak(mV) Offset(ps) Slew(ps) Xcap(fF) Vdd(V) Edge Status Net"
        set aggs [ get_property $vic attackers ]
        foreach_in_collection current_agg $aggs {

            set hierarchical_name [ get_property $current_agg hierarchical_name ]
            set noise_peak [ get_property $current_agg noise_peak]
            set state [ get_property $current_agg state]
            set slew [ get_property $current_agg slew]
            set edge [ get_property $current_agg transition]
            if {[string match ~* $hierarchical_name]} {
                set vdd [get_property [get_pins $vic_name] power_rail_voltage_max]
            } else {
                set vdd [get_property [get_pins -of_objects [get_nets $hierarchical_name]] power_rail_voltage_max]
            }
            set coupling_capacitance [ get_property $current_agg coupling_capacitance]

            puts $chan "Cpl: $noise_peak $slew $edge $vdd $coupling_capacitance $state $hierarchical_name"
        }

        if { $prop_glitch > 0 } {
            puts $chan "Prp: $prop_glitch"
        }
    }
}
close $chan
```

For more information on querying SI attributes, refer to "Signal Integrity Delay and Glitch Analysis" chapter in the *Tempus User Guide*.

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

User-Defined Properties

The user-defined properties allow you to define certain properties which can be interpreted by the software in a predefined way. When these properties are not specified, the software will revert to the default behavior, as applicable.

Property Name	Type	Description
capacitive_load_unit	lib	Provides the load unit mentioned in the library.
time_unit	lib	Provides the time unit mentioned in the library.
aocv_weight	lib_cell	Defines AOCV weight for a given cell.
k_input_switching_window_fall*	lib	Defines the switching window threshold for fall signal during the application of SSI (simultaneous switching input) derating feature. <i>Default: 1</i>
k_input_switching_window_rise*	lib	Defines the switching window threshold for rise signal during the application of SSI (simultaneous switching input) derating feature. <i>Default: 1</i>

* When the above properties are defined in SSI derating environment, the delay of the respective cell multiplied by the above factor will be compared with the difference in (rise/fall) signal arrivals at two inputs to determine if switching derate needs to be applied to cell delay.

Examples

- The following command returns the hierarchical name for the pin contained in the collection referenced by \$pin:

```
get_property $pin hierarchical_name
```

- The following example shows how you can query a user-defined property from stage_weighting library at the cell level:

```
set_property [get_lib_arcs -of fast/BUF2] stage_weighting 3 // sets the value
get_property [get_lib_arcs -of fast/BUF2] stage_weighting
```

- The following example shows power and ground pins for the corresponding library pins:

```
get_property [get_lib_pins CELLA/A] related_power_pin
get_property [get_lib_pins CELLA/A] related_ground_pin
```

- The following example shows active clocks:

```
get_property [get_clocks Iclk1] is_active true
```

- The following commands define the switching window threshold for rise signal:

```
define_property -type float -object_type lib k_input_switching_window_rise
set_property [get_libs slow] k_input_switching_window_rise 1.1
```

- The following example shows that the phase shift returned by the period property:

```
report_timing -from [get_clocks refclk] -to [get_clocks m_spi_clk]
Path 1: VIOLATED (-1.869 ns) Setup Check with Pin SP_IN/sp_reg[0]/CK->D
Group: m_spi_clk
Startpoint: (F) spi_data Clock: (R) refclk
Endpoint: (F) SP_IN/sp_reg[0]/D
Clock: (R) m_spi_clk
```

	Capture	Launch
Clock Edge:+	4.000	0.000
Src Latency:+	0.000	0.000
Net Latency:+	0.780 (P)	0.000 (I)
Arrival:=	4.780	0.000
Setup:-	1.292	
Uncertainty:-	0.400	
Copr Adjust:+	0.000	
Required Time:=	3.088	
Launch Clock:=	0.000	
Input Delay:+	4.000	
Data Path:+	0.958	
Slack:=	-1.869	

#-----

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--get_property

```

Timing Point      Flags Arc      Edge Cell      Fanout Trans Delay Arrival
                    (ns)  (ns)  (ns)
#-----
SP_IN/p132A/ZN    -      B2->ZN  R      AOI22_X1  1      0.049  0.506  4.534
SP_IN/p135A/ZN    -      A->ZN   F      INV_X1    1      0.845  0.424  4.958
SP_IN/sp_reg[0]/D -      D       F      SDFF_X2   1      0.399  0.000  4.958
#-----
.....
litmus > set path [report_timing -from [get_clocks refclk] -to [get_clocks m_spi_clk] -collection]
0x14
litmus > get_property $path period

```

- In this following example, there is a clock reference for launch clock (Null phase- @), so Litmus does not show any clock period for this path:

```

litmus > report_timing -to [get_clocks m_spi_clk] -early
Path 1: MET (0.406 ns) Hold Check with Pin SP_IN/sp_reg[4]/CK->SI
Group: m_spi_clk
Startpoint: (R) SP_IN/sp_reg[3]/CK Clock: (R) m_spi_clk
Endpoint: (R) SP_IN/sp_reg[4]/SI
Clock: (R) m_spi_clk

      Capture      Launch
      Clock Edge:+  0.000      0.000
      Src Latency:+  0.000      0.000

      Net Latency:+  0.780 (P)  0.767 (P)
      Arrival:=     0.780      0.767

      Hold:+        -0.056
      Uncertainty:+  0.600
      Cppr Adjust:-  0.000
      Required Time:= 1.324
      Launch Clock:=  0.767
      Data Path:+    0.963
      Slack:=        0.406
#-----
Timing Point      Flags Arc      Edge Cell      Fanout Trans Delay Arrival
                    (ns)  (ns)  (ns)
#-----
SP_IN/sp_reg[3]/CK -      CK      R      (arrival) 3      0.113  -      0.767
SP_IN/sp_reg[3]/QN -      CK->QN  R      SDFF_X2   1      0.113  0.963  1.730
SP_IN/sp_reg[4]/SI -      SI       R      SDFF_X2   1      0.255  0.000  1.730
#-----
litmus > set path [report_timing -to [get_clocks m_spi_clk] -early]
0x16
litmus > get_property $path period
NA

```

- The following example shows if a real or generated clock is defined on the pins or ports:

```

litmus > create_clock [get_ports {CLK_t}] -name clk -period 100 -waveform {0 50}
litmus > create_generated_clock -divide_by 2 -source CLK -name d_2 CLK_b
litmus > get_property [get_ports CLK_t] is_clock_root
true

litmus > get_property [get_pins CLK_b] is_clock_root
true

```

Related Commands

- [report_property](#)
- [set_property](#)
- [reset_property](#)
- [define_property](#)
- [report_object_property_names](#)

index_collection

index_collection
[-help]
base_collection index

Returns a collection containing the object that exists at the specified index of the specified object collection.

Parameters

-help	Prints out the command usage.
base_collection	Specifies the base collection.
index	Specifies the index number. You can specify a number from 0 to collection_size-1.

Examples

- The following command returns the object that exists at the index number 2 in the object collection referenced by \$nets:

```
% index_collection $nets 2
```

Related Commands

- range_collection

is_collection_object

```
is_collection_object  
[-help]  
collection
```

Returns a value of `true` if the specified object is a collection, otherwise `false` is returned.

Parameters

<code>-help</code>	Prints out the command usage.
<code>collection</code>	Specifies the collection to be checked.

Examples

- The following example shows how the `is_collection_object` command can be used to check for collection objects:

```
set cells [get_cells]  
set pins [get_pins]  
set nets [get_nets]  
set paths [report_timing -max_paths 5 -coll]  
set heterogenColl ""  
append_to_collection heterogenColl $cells  
append_to_collection heterogenColl $pins  
append_to_collection heterogenColl $nets  
append_to_collection heterogenColl $paths  
set name "123"  
  
'1' is returned for all of the following 'is_collection_object' commands  
is_collection_object $cells  
is_collection_object $pins  
is_collection_object $nets  
is_collection_object $paths  
is_collection_object $heterogenColl  
  
'0' is returned for these 'all is_collection_object' commands  
is_collection_object $name  
is_collection_object [get_cells xyz* -quiet]
```

Related Commands

- [sizeof_collection](#)

parse_proc_arguments

```
parse_proc_arguments
-help
-args string
result_array
[forced_proc]
```

Supports argument validation within a Tcl procedure and enables the use of the `-help` option to view the help information associated with the procedure.

Procedures using the `parse_proc_arguments` command will validate the arguments. If the required arguments are missing or have invalid values, the `parse_proc_arguments` command generates a Tcl error and the calling procedure terminates.

The `parse_proc_arguments` command must be the first command called in the procedure. This command cannot be used outside of a procedure.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-args string</code>	Specifies the list of arguments passed to the Tcl procedure.
<code>result_array</code>	Specifies the result array into which the parsed arguments are to be stored.
<code>forced_proc</code>	Specifies the name of the invoking procedure.

Examples

The following Tcl procedure uses the `parse_proc_arguments` command for argument validation.

```
proc cmdProc {args} {
    parse_proc_arguments -args $args results
    foreach argname [array names results] {
        puts "$argname = $results($argname)"
    }
}

define_proc_arguments cmdProc -info "Command Description" \
-define_args \
{{-string "String help" "string_val" string optional}
{-integer "Int help" "int_value" int optional}
{-list "List help" "list_value" list {optional bind_option -Bool}}
{-boolean_a "BoolA help" "" boolean {optional mutual_exclusive_group g0}}
{-boolean_b "BoolB help" "" boolean {optional mutual_exclusive_group g0}}
{-float_num "Float help" "float_value" float}
{-oosa "One of StringA help" "one_of_str_value" one_of_string {optional value_help {values {v1 v2}}}}
{-oosb "One of StringB help" "one_of_str_value" one_of_string {optional {values {v4 v5}}}}
}
```

Now, if the `cmdProc` procedure is invoked using the following command at the command prompt, argument validation takes place and the following errors are reported.

```
litmus> cmdProc -boolean_a -boolean_b

**ERROR: (Litmus-39): Options '-boolean_a -boolean_b' are mutually exclusive for 'cmdProc'.

Usage: cmdProc [-string string_val] [-integer int_value] [-list list_value] [-boolean_a ]
[-boolean_b ] [-float_num float_value] [-oosa one_of_str_value {v1 v2 } ] [-oosb one_of_str_value]

**ERROR: (Litmus-27): Errors in 'cmdProc'.
```

The following command to invoke the `cmdProc` procedure will also return the errors shown below.

```
litmus> cmdProc -oosa v5
```

Litmus Text Command Reference

Advanced Timing Tcl Scripting Commands--parse_proc_arguments

****ERROR:** (Litmus-42): The arguments specified (v5) in cmdProc -oosa are not valid. Refer to the documentation for valid arguments.

Usage: cmdProc [-string string_val] [-integer int_value] [-list list_value] [-boolean_a]
[-boolean_b] [-float_num float_value] [-oosa one_of_str_value {v1 v2 }] [-oosb one_of_str_value]]

****ERROR:** (Litmus-27): Errors in 'cmdProc'.

query_objects

```
[-help]  
query_objects collection [-limit value]
```

Displays the names of the objects contained inside the specified collection.

 The `query_objects` command displays object names only. To return a tcl list of object names, the `get_object_name` command may be used instead.

Parameters

<code>-help</code>	Prints out the command usage.
<code>collection</code>	Specifies the object collection for which to print names.
<code>-limit value</code>	Specifies the maximum number of object names to print. Specify a value of 0 to query the names of all of the objects in the collection. <i>Default:</i> Prints the first 100 object names in the collection.

Examples

- The following commands create collections of pins and nets, add them together to create a new collection, and prints the names of the objects in the collection:

```
set pins [get_pins {i_block1/b1_Sin1 i_block1/b1_CLK1 i_block1/b1_Sout1}]  
set nets [get_nets {Sin1 CLK1 Sout1}]  
add_to_collection newVar $pins  
add_to_collection newVar $cells  
query_objects $newVar
```

The software returns the following information:

```
i_block1/b1_Sin1 i_block1/b1_CLK1 i_block1/b1_Sout1 Sin1 CLK1 Sout1
```

Related Commands

- `get_object_name`

range_collection

```
range_collection  
[-help]  
base_collection  
from  
to
```

Returns a sub-collection of the base collection. The sub-collection range lies between the 'from' and 'to' (including 'from' and 'to') index numbers specification of this command.

Parameters

-help	Prints out the command usage.
base_collection	Specifies the base collection.
from	Specifies the 'from' index number.
to	Specifies the 'to' index number.

The index number can be an integer or an expression, such as, "end OR end-integer" (with no spaces within the expression).

If the specified to/from value is negative (less than 0), then the software will change this value to 0. If the value is greater than the "collection_size - 1", then it will be changed to "collection_size - 1".

Examples

- The following example sets `$points` as the base collection:

```
% set points [get_cells *]
0x13

% sizeof_collection $points
22
```

- The following example returns the sub-collection of the base collection `$points` between the index number range from 2 to 4:

```
% range_collection $points 2 4
clk_buf2 clk_buf3 clk_buf4
0x16

% range_collection $points 4 2
clk_buf2 clk_buf3 clk_buf4
0x17
```

- The following example returns a sub-collection starting from index number 0 till the end of the base collection:

```
% range_collection $points 0 end
clk_buf1 pad1 clk_buf2 clk_buf3 clk_buf4 buf1 buf2 buf3 buf4 mem dff1 dff2 clkand2 pu_cell_1 pu_lat_1 u1 cg4 genLoop comboLoop pad2
dff3 latch1
0x18
```

- The following example returns a sub-collection starting from end-integers 4 to 2:

```
% range_collection $points end-4 end-2
genLoop comboLoop pad2
0x19
```

- The following example shows a range starting from a negative value to a value greater than the collection size. The following warning messages are displayed:

```
% range_collection $points -5 100
**WARN: (TCLCMD-1419): 'from' index with value -5 is less than start index, so it is changed to 0 .
**WARN: (TCLCMD-1419): 'to' index with value 100 is greater than end index, so it is changed to 21 .
clk_buf1 pad1 clk_buf2 clk_buf3 clk_buf4 buf1 buf2 buf3 buf4 mem dff1 dff2 clkand2 pu_cell_1 pu_lat_1 u1 cg4 genLoop comboLoop pad2
dff3 latch1
0x1a
```

- The following example shows the warning message displayed for a negative start index number:

```
% range_collection $points end-25 end-20
**WARN: (TCLCMD-1419): 'from' index with value -4 is less than start index, so it is changed to 0 .
clk_buf1 pad1
0x1b
```

- The following command shows an error if the specified index number is not an integer:

```
% range_collection $points s end
**ERROR: (TCLCMD-1418): Bad index 's': must be integer? or end?[-]integer?
```

Related Commands

- [index_collection](#)

remove_from_collection

```
remove_from_collection
[-help]
base_collection
[-intersect]
collection_or_obj_list
```

Creates a new collection by removing the objects specified in one collection from another collection.

Parameters

-help	Prints out the command usage.
base_collection	Specifies the base collection of objects. Objects from the second collection or object list are removed from this collection to create the new collection.
-intersect	Provides intersection of base collection elements with second collection of objects or lists.
collection_or_obj_list	Specifies the collection or list of objects that should be removed from the base collection to create a new collection. When specified, this parameter returns those elements that intersect and exist in both the <i>base_collection</i> and the second collection. Also, all the elements of <i>base_collection</i> that do not intersect with second collection will be removed.

Examples

- The following commands create a collection of pins that does not include clock pins:

```
set allP [get_pins *]
set all_pin_no_clock [remove_from_collection $allP [get_pins -filter "is_clock == true"]]
```

You can use the `sizeof_collection` command to see that the `all_pin_no_clock` collection contains fewer objects (that is, no clock pins) than the original `allP` collection:

```
sizeof_collection $allP
50

sizeof_collection $all_pin_no_clock
47
```

Related Commands

- [range_collection](#)
- [index_collection](#)

report_object_property_names

```
report_object_property_names [-help]
[-obj_type {cell clock design lib lib_cell lib_pin lib_timing_arc net path_group pin port timing_arc timing_path timing_point}]
```

Lists all of the properties associated with the specified object type.

If you do not specify any object type, this command lists all the supported object types, and all the properties associated with those object types.

Parameters

-help	Prints out the command usage.
-obj_type	Lists the properties associated with the specified object type. The following object types are supported: cell, clock, design, lib, lib_cell, lib_pg_pin, lib_pin, lib_timing_arc, net, path_group, pg_net, pg_pin, pin, port, si_attacker, si_victim, timing_arc, timing_path, or timing_point Additional SI properties can be included with the timing_point object. To enable SI properties, you can set timing_report_enable_si_debug attribute to true.

Related Commands

- `define_property`
- `get_property`
- `report_property`
- `reset_property`
- `set_property`

report_property

```
report_property
[-help]
{collection | list_of_collections}
[-property_list list of properties]
[{> | >>} filename]
```

Reports the properties associated with each object in the specified collection.

For properties that return a populated collection, the command output displays {...}. For any empty collections,{} will be displayed.

Parameters

-help	Prints out the command usage.
collection list_of_collections	Specifies a collection, or a list of collections to report. The collections can contain any of the following object types: cell, clock, design, lib, lib_cell, lib_pin, lib_pg_pin, pg_net, pg_pin, lib_timing_arc, net, path_group, pin, port, timing_arc, timing_path, or timing_point
-property_list list of properties	Specifies a list of valid property names to report. Note: If the -property_list parameter is not specified, then all the properties associated with a collection of objects will be reported.
{> >>} filename	When specified with >, the software writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with >>, the software writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. You can add the .gz extension to the file name to generate a compressed report. Note: The file name must be the last argument in the list.

Examples

- The following command shows the values of the specified properties:

```
report_property [get_pins TU11/A] -property_list {name direction clocks}

property | value
name | TU11/A
direction | in
clocks | {...}
```

Related Commands

- [get_property](#)
- [report_object_property_names](#)
- [set_property](#)
- [define_property](#)
- [reset_property](#)

sizeof_collection

```
sizeof_collection
[-help]
collection
```

Returns the total number of objects contained in the specified collection. If the collection contains different types of objects, sizeof_collection returns the sum of all the objects of different types in the collection.

Parameters

<code>-help</code>	Prints out the command usage.
<code>collection</code>	Specifies the collection.

sort_collection

```
sort_collection  
[-help]  
collection  
[-dictionary]  
{property_or_list_of_properties}  
[-descending]
```

Returns a sorted collection of objects based on the property.

Parameters

-help	Prints out the command usage.
collection	Specifies the collection to sort. The collection must be a homogenous collection of any one of the following object types: cell, clock, design, lib, lib_cell, lib_pin, lib_timing_arc, net, path_group, pin, port, timing_arc, timing_path, or timing_point.
-descending	Sorts the objects in the collection by descending order. <i>Default:</i> Sorts the collection in ascending order.
-dictionary	Sorts the objects in a collection by dictionary order.
property list_of_properties	Sorts the collection by the specified property. You can specify any property associated with the object type. You can specify one or more properties. If you specify more than one property, the software sorts the collection in the order by which you specified the properties.

Examples

- The following command returns a collection of all the input ports in the design sorted on the basis of their hierarchical names:

```
% sort_collection [all_inputs] hierarchical_name  
  
CLK1 CLK2 CLK3 CLK4 IN1 IN2 IN3 IN4
```

Basic Database Access Tcl Commands

- [add_thru_substrate_insts](#)
- [convert_dbu_to_um](#)
- [convert_um_to_dbu](#)
- [decompress_special](#)
- [init_hier_cell](#)
- [uniquify](#)

add_thru_substrate_insts

```
[-help]  
[-driver_thru_substrate_cell cell1]  
-load_thru_substrate_cell cell2  
-net net  
[-redraw]  
[-selected]  
[-shared_load_thru_substrate_cell]
```

Adds thru-substrate instances to a net to specify two-sided routing for a critical net (backside routing has lower RC than frontside routing). All pins or ports of the net must be on the front side, and as an exception, one backside port is allowed. A thru-substrate instance is a device that tunnels through the substrate to connect frontside routing with backside routing.

The driver thru-substrate instance is placed on top of the driver and a load thru-substrate instance is placed on top of its load. If a load thru-substrate instance drives multiple loads, it is placed in the center of the group of loads. It means that all instances with a pin on the net must be placed, and if the net has ports, they must also be placed before when the thru-substrate instances can be added to the net. Hence, this command creates unallowed placement that needs to be resolved with the `place_detail` command.

When a net has a backside port with everything else on the front side, this command creates one thru-substrate instance placed on top of the backside port, so that the net's frontside routing can be

brought to the backside, typically to connect to a backside bump. For this usage, specify the same value for the `-load_thru_substrate_cell` and `-driver_thru_substrate_cell` parameters.

Use this command to create one FTV for a net with multiple FS leaf pin, multiple FS port(s), and/or a single BS port.

FTV is placed on the top of the BS port.

Parameters

<code>-help</code>	Prints a brief description that includes the type and default information for each <code>add_thru_substrate_insts</code> parameter. For a detailed description of the command and all its parameters, use the man command: <pre>man add_thru_substrate_insts</pre>
<code>-driver_thru_substrate_cell</code> <i>cell1</i>	Specifies the thru-substrate macro/cell to instantiate a thru-substrate instance, whose front pin is to be routed to the driver.
<code>-load_thru_substrate_cell</code> <i>cell2</i>	Specifies the thru-substrate macro/cell to instantiate a thru-substrate instance(s), whose front pin(s) is to be routed to load(s).
<code>-net</code> <i>net</i>	Specifies the net to add thru-substrate instances on both sides, front and back.
<code>-redraw</code>	Redraws the graphics window.

<code>-selected</code>	Groups the loads where each group will have its own load thru-substrate instance. • When the <code>-selected</code> and <code>-load_thru_substrate_cell</code> parameters are specified, the only driver thru-substrate instance is first created and the selected loads either share one load thru-substrate instance or get their own load thru-substrate instances. Additionally, the unselected loads are driven directly by the net driver. • In case the <code>-selected</code> parameter is specified and the <code>-load_thru_substrate_cell</code> parameter is not specified, each selected load driven by the net driver is reconnected to a new load thru-substrate instance. This may be shared through the <code>-shared_load_thru_substrate_cell</code> parameter, which is driven by the driver thru-substrate instance. This process repeats until the loads are grouped as desired.
<code>-shared_load_thru_substrate_cell</code>	Loads share one thru-substrate instance (one thru-substrate instance for each load, if not specified).

Related Topics

[delete_thru_substrate_insts](#)

convert_dbu_to_um

```
convert_dbu_to_um  
[-unit number]  
{int | {int_list}}
```

Takes an arbitrary list of ints and returns it in the same form, with each database integer int converted to its user unit floating point equivalent.

Parameters

<code>-unit <i>number</i></code>	Specifies the convert factor by which to divide the database integer int. <i>Value:</i> One of 100, 200, 1000, 2000, 10000, or 20000 <i>Default:</i> Uses the LEF <code>UNITS DATABASE MICRONS</code> int from the Tempus database technology file, or the OpenAccess equivalent.
<code>{<i>int</i> {<i>int_list</i>}}</code>	
	Specifies the int or int list to convert. You can specify a single int, or an arbitrary list of ints, including points, rectangles and coordinates. <i>Type:</i> Integer

Examples

- The following command converts the database int 100 into its equivalent user unit int, based on a convert factor of 2000:

```
convert_dbu_to_um -unit 2000 100
```

The software returns the following information:

```
0.050
```

- The following command converts the database ints for the list of numbers {1000 1000 2000 2000} into their equivalent user unit ints, based on a convert factor of 1000:

```
convert_dbu_to_um -unit 1000 {1000 1000 2000 2000}
```

The software returns the following information for the list of numbers. The list is implied, but not shown.

```
1.000 1.000 2.000 2.000
```


Related Commands

`convert_um_to_dbu`

convert_um_to_dbu

```
convert_um_to_dbu  
[-unit number]  
{double | {double_list}}
```

Takes an arbitrary list of doubles and returns it in the same form, with each user unit floating point double converted to its database integer double equivalent.

Parameters

<code>-unit <i>number</i></code>	Specifies the convert factor by which to multiply the user unit double. <i>Value:</i> One of 100, 200, 1000, 2000, 10000, or 20000 Default: Uses the LEF <code>UNITS DATABASE MICRONS</code> double from the Tempus database technology file, or the OpenAccess equivalent.
<code>{<i>double</i> {<i>double_list</i>}}</code>	
	Specifies the double or double list to convert. You can specify a single double, or an arbitrary list of doubles, including points, rectangles and coordinates. <i>Type:</i> Float

Examples

- The following command converts the user unit double 30 into its equivalent database integer double, based on a convert factor of 2000:

```
convert_um_to_dbu -unit 2000 30
```

The software returns the following information:

```
60000
```

- The following command converts the user unit doubles for the list of numbers {30 40} into their equivalent database unit doubles, based on a convert factor of 200:

```
convert_um_to_dbu -unit 200 {30 40}
```

The software returns the following information for the list of numbers. The list is implied, but not shown.

```
6000 8000
```

Related Commands

`convert_dbu_to_um`

decompress_special

```
decompress_special  
[-help]  
[-area {x1 y1 x2 y2}]  
[-layers layer+]  
[-nets net+]
```

Decompresses Power-Grid (PG) objects within the specified area, [layer](#), or [net](#). This command decompresses only the overlapped part in case of partial overlapping. The rest of the part is regrouped.

Parameters

<code>-help</code>	Prints a brief description that includes the type and default information for each <code>decompress_special</code> parameter. For a detailed description of the command and all its parameters, use the man command: <code>man decompress_special</code>
<code>-area</code> <code>{x1 y1</code> <code>x2 y2}</code>	Specifies the coordinates for the area from which PG needs to be decompressed. The <code>decompress_special</code> command decompresses only those special wires and special vias, which exist within the area specified by this parameter. <i>Default:</i> All PG structures are decompressed.
<code>-</code> <code>layers</code> <code>layer+</code>	Specifies the layers on which PG decomposition is needed. This parameter requires either LEF layer names or a routing layer index value (for example, <code>1</code> for the first routing layer, <code>2</code> for the second routing layer, and so on).
<code>-nets</code> <code>net+</code>	Specifies the nets from which PG decomposition is needed.

Example

The following command decompresses %s special_wires and %s special_vias:

```
decompress_special -area {100 100 120 120} -layer Via56
```

Related Topics

[compress_special](#)

init_hier_cell

```
init_hier_cell  
[-help]  
cell_names  
[-dual_view]
```

Initializes one or more hierarchical cells based on the applicable order.

Parameters

-help	Prints a brief description that includes the type and default information for each <code>init_hier_cell</code> parameter. For a detailed description of the command and all its parameters, use the <code>man</code> command: <pre>man init_hier_cell</pre>
cell_names	Specifies the names of the hierarchical cells to be initialized.
-dual_view	Binds the top and global views, and initializes the hierarchical cells inside ILMs.

uniquify

```
uniquify  
[-help]  
{[design | module] | -new_master hinsts} [-exclude modules]  
[-verbose]
```

Supports regrouping of master-clone partitions inside the non-unique modules. This command uniquifies or regroups the master/clones (one master module used by two or more `hinst` clones) by creating unique modules for each `hinst` clone, as needed. In all the cases, the new module names are generated by appending the '`_number`' suffix (such as `_1` and `_2`) sequentially to the original module name for creating a unique module name.

Parameters

<code>-help</code>	Prints a brief description that includes the type and default information for each <code>uniquify</code> parameter. For a detailed description of the command and all its parameters, use the <code>man</code> command: <pre>man uniquify</pre>
<code>design module</code>	Regroups the master/clones within the <code>design</code> or <code>module</code> and recursively below it. This assures that all the <code>hinsts</code> inside those would have their own unique module, except for the modules specified using the <code>-exclude</code> parameter. In case multiple <code>hinsts</code> are referenced to the specified <code>module</code>, those will remain as clones. <i>Type: Mandatory</i>
<code>-new_master hinsts</code>	Splits the <code>hinst</code> clones of a master module into the clones of a new master. The new master is also regrouped, except for the modules specified using the <code>-exclude</code> parameter. It may also happen that one of such <code>hinst</code> clones is forced to have a new master (refer to the Examples section). In case a <code>hinst</code> is not a clone of the same module, an error message is displayed. <i>Type: Mandatory</i>
<code>-exclude modules</code>	Excludes the specified modules from unification or regrouping. Use this parameter if a specific sub-module is a partition, which will be implemented and shared only once. <i>Type: Optional</i>
<code>-verbose</code>	Reports the <code>hinst</code> clones (if any), which are regrouped. <i>Type: Optional</i>

Examples

- The following command uniquifies the whole design, where the `h1`, `h2`, and `h3 hinsts` are clones of the `mod1` module, and the `h4` and `h5 hinsts` (clones of the `mod2` module) are inside `mod1`:

```
uniquify top -verbose
```



```
#hinst name module name
h1 mod1 (unchanged)
h2 mod1_1
h3 mod1_2
h1/h4 mod2 (unchanged)
h1/h5 mod2_1
h2/h4 mod2_2
h2/h5 mod2_3
h3/h4 mod2_4
h3/h5 mod2_5
```

Note that the first clone of each module is not modified. So, `h1` still uses `mod1`, and `h1/h4` still use `mod2`. However, these are still reported to see the `hinst` clone, which forces a new module for the other `hinst` clones.

- Using the same scenario as in the previous example, the following command uniquifies the design below `mod1`:

```
uniquify mod1 -verbose
#hinst name module name
h1/h4 mod2 (unchanged)
h1/h5 mod2_1
h2/h4 mod2 (unchanged)
h2/h5 mod2_1
h3/h4 mod2 (unchanged)
h3/h5 mod2_1
```

Note that only the design inside `mod1` is uniquified. So, `h1`, `h2`, and `h3` are still the clones of `mod1`, and there are still master/clones grouping between `h1`, `h2`, and `h3`. For instance, all the `h4s` use `mod2`, and all the `h5s` use `mod2_1`.

- Using the same scenario as in the first example, the following command splits `h1` and `h2` into a new master. This is normally done, if `h1`, `h2`, and `h3` are partitions and need a different implementation than `h3`. It is recommended not to uniquify `mod2`, if it is a lower-level partition, which will be implemented once and shared by all `h1`, `h2`, and `h3`.

```
uniquify -new_master {h1 h2} -exclude mod2 -verbose
#hinst name module name
h1 mod1_1
h2 mod1_1
```

Note that `h3` is still using `mod1`, and the `mod2` `hinsts` inside `mod1` and `mod1_1` (such as `h4` and `h5`) are still the clones of `mod2`. If there is also a `mod3` inside `mod1`, it will also be uniquified, but nothing inside `mod2` is uniquified.

- Using the same scenario as in the first example, the following command splits `h1` and `h4` into a new master. This implicitly forces `h1` to be uniquified from `h2` and `h3`.

```
uniquify -new_master {h1/h4} -verbose
```

```
#hinst name module name
```

```
h1 mod1_1 (forced by lower level change)
```

```
h1/h4 mod2_1
```

If there is a `mod4` inside `mod2_1`, it will also be uniquified, but it will not be shared with `mod2`.

Constraint Promotion Commands

- [set_constraint_promotion_priority](#)
- [set_constraint_promotion_strategy](#)
- [promote_hierarchical_constraints](#)
- [pushdown_hierarchical_constraints](#)

set_constraint_promotion_priority

```
set_constraint_promotion_priority  
[-help]  
[<obj_list>]
```

Determines the sequence in which block/instance-level constraints are analyzed. By default, constraints are processed in alphabetical order of instance names, but you can override this sequence to align with integration priorities or known design dependencies.

Parameters

-help	Prints out the command usage.
<obj_list>	Specifies an ordered list of module names and/or instance names.

Related Commands

- [init_design](#)

set_constraint_promotion_strategy

```
set_constraint_promotion_strategy
[-help]
-method <enum>
-name <string>
[-types <type_list>]
[-object_type <obj_type>]
```

Determines the sequence in which block/instance-level constraints are analyzed. By default, constraints are processed in alphabetical order of instance names, but you can override this sequence to align with integration priorities or known design dependencies.

Parameters

-help	Prints out the command usage.
-method <string>	Selects the promotion strategy. (enum, required) • Available Methods: ◦ <code>block_worst_case</code> (<i>default</i>) — Retains the most conservative constraint. ◦ <code>block_priority</code> — Prioritizes block-level constraints over top-level definitions. ◦ <code>block_ignore</code> — Ignores the specified constraint class during promotion.
-name <string>	Specifies the strategy name (string, optional)

<pre>-types <type_list></pre>	Applies the strategy to one or more specific constraint types. (string, optional) • Valid Constraint Types: ◦ case_analysis ◦ clock (covers create_clock, create_generated_clock) ◦ input_delay ◦ output_delay ◦ clock_latency ◦ clock_transition ◦ clock_uncertainty ◦ clock_gating ◦ disable_clock_gating ◦ load ◦ min_fanout, max_fanout ◦ min_trans, max_trans ◦ ideal_network ◦ false_path ◦ multicycle_path ◦ min_delay ◦ max_delay ◦ clock_group ◦ slew (covers set_input_transition, set_drive, set_driving_cell)
---	---

<code>-object_type</code> <code><obj_type></code>	Limits the strategy to specific object types. (enum, optional) • Valid Object Types: ◦ pin ◦ clock ◦ net ◦ lib_pin ◦ promoted_clock ◦ mapped_clock
--	--

Examples

Override default behavior and use block priority globally:

```
set_constraint_promotion_strategy -method block_priority
```

Ignore all slew-related constraints from blocks:

```
set_constraint_promotion_strategy -method block_ignore -types slew
```

Ignore min/max delay constraints specifically on library pins:

```
set_constraint_promotion_strategy -method block_ignore -types "min_delay max_delay" -  
object_type lib_pin
```

Related Commands

- `init_design`

promote_hierarchical_constraints

```
promote_hierarchical_constraints  
[-help]  
[-blocks <string>]  
[-instances <string>]  
[-types <string>]  
[> <file>]  
[>> <file>]
```

Promotes hierarchical timing constraints to a higher design level with optional filtering by blocks, instances, or constraint types.

Parameters

-help	Prints out the command usage
-blocks <string>	Promotes constraints for specific module(s).
-instances <string>	Promotes constraints only for listed instances.
-types <string>	Promotes only selected constraint classes.
> <file>	Writes the promoted SDC to a specific output file.
>> <file>	Appends to an existing file.

Examples

Promote only selected instances:

```
promote_hierarchical_constraints -instances {U1 U2}
```

Promote only clock creation constraints:

```
promote_hierarchical_constraints -types "create_clock create_generated_clock"
```


Promote only case analysis settings:

```
promote_hierarchical_constraints -types set_case_analysis
```

Write output directly to a file:

```
promote_hierarchical_constraints > promote.sdc
```

Related Commands

- `init_design`

pushdown_hierarchical_constraints

```
pushdown_hierarchical_constraints  
[-help]  
-dir <string>  
[-types <string>]  
[-instances <string> | -blocks <string>]
```

Pushes down timing constraints from a higher design level down to selected blocks or instances, with optional filtering by constraint type.

Parameters

-help	Prints out the command usage
-dir <string>	Specifies directory name where output to be redirected (string, required)
-blocks <string>	Pushdowns constraints for specific module(s).
-instances <string>	Pushdowns constraints only for listed instances.
-types <string>	Pushdowns only selected constraint classes.

Related Commands

- `init_design`

Database Access and Handling Commands

- [define_attribute](#)
- [delete_obj](#)
- [get_db](#)
- [is_attribute](#)
- [set_db](#)
- [reset_db](#)
- [sort_db](#)

define_attribute

```
define_attribute
[-help]
attribute_name
-obj_type object_type
[-obsolete]
{-alias existing_attribute | {-category category [-data_type {bool | int | double | area
| coord | string | point | rect | line | delay | resistance | capacitance | voltage |
in_dir | out_dir | in_file | out_file | object | object*} [-enum_values values]]
[-check_function check_function]
[-compute_function compute_function]
[-default default]
[-help_string description]
[-hidden]
[-set_function set_function]
[-skip_in_db]]}
```

Creates a new, user-defined attribute with the specified characteristics.

The attribute value can be reset only if a default value is specified. For example, the following command defines a new instance attribute of integer type with a default value equal to 1:

```
define_attribute myattr -default 1 -obj_type inst -data_type int -help_string "my
attribute" -category mycat
```

Parameters

-help	Outputs a brief description that includes type and default information for each <code>define_attribute</code> parameter. For a detailed description of the command and all of its parameters, use the man command: <code>man define_attribute.</code>
-alias existing_attribute	Makes <code>attribute_name</code> a hidden alias for <code>existing_attribute</code>, which must have been defined. This parameter is used along with the <code>-obj_type</code> parameter. It could also be used with the <code>-obsolete</code> parameter to generate an obsolete warning.

<code>attribute_name</code>	Specifies the name of the attribute to be defined.
<code>-category category</code>	Specifies the category of the attribute. Categories group attributes that perform similar functions, whereas object types describe where in the design an attribute is valid. You can specify any category name, both new and existing category names are allowed.
<code>-check_function check_function</code>	Specifies a previously defined Tcl procedure's name to ensure that the newly defined attribute is valid. The Tcl procedure should be of the form <code>proc {object value}</code>.
<code>-compute_function compute_function</code>	Specifies a previously defined Tcl procedure's name to get the newly defined attribute's value later (with the <code>get_attribute</code> command). The Tcl procedure must be of the form <code>proc {object}</code>. When you use this option, the attribute becomes a read-only attribute because its value is always computed.
<code>-data_type {bool int double area coord string point rect line polygon delay resistance capacitance voltage in_dir out_dir in_file out_file}</code>	
	Defines the data type of the attribute. Possible data types are basic only, no objects.
<code>-default default</code>	Specifies a default value for the attribute. In case this parameter is not specified while creating an attribute, the default value will be: <code>no_value</code> for simple numeric types such as <code>int</code>, <code>double</code>, <code>delay</code>, <code>capacitance</code>, <code>coord</code>, and <code>boolean</code> <code>""</code> (empty string) for all other types, including <code>enum</code>, <code>string</code>, <code>in_file</code>, <code>out_file</code>, <code>in_dir</code>, <code>out_dir</code>, <code>point</code>, <code>rect</code>, <code>polygon</code>, <code>line</code>, and <code>obj_type</code>
<code>-enum_values values</code>	Specifies a list of all the possible values of the enumerated type. For example, <code>-enum_values {a b c}</code> .

-help_string <i>description</i>	Specifies the help text for the attribute.
-hidden	Specifies whether the defined attribute is a hidden attribute.
-obj_type <i>object_type</i>	Specifies the object type of the attribute. All valid object types can be found by typing <code>get_db obj_types .name</code> at the prompt. The attribute is assigned to the specified type.
-obsolete	Specifies if the defined attribute is obsolete.
-set_function <i>set_function</i>	Specifies a previously defined Tcl procedure's name. This option allows you to override user-defined values provided it conforms to the parameters in the Tcl procedure you created. The Tcl procedure should be of the form <code>proc {object new_value current_value}</code> . Returns new value assigned to the attribute.
-skip_in_db	Prevents the <code>write_db</code> command to write out the defined attribute.

Examples

- The following command creates a Tcl procedure, `check_fxn`, and then creates an attribute named `test_check`. The `-check_function` parameter is specified so that the `test_check` attribute can be tested for validity when it is specified later:

```
proc check_fxn {obj val} {  
+ if {$val >0} { return 0}  
+ return 1  
+ }  
  
define_attribute test_check1 -obj_type inst -data_type int -category test -  
help_string "test_check_function" -check_function check_fxn
```

Based on the `check_function` parameter, the checking criteria attribute value can be set only to a negative value. As a result the following command sets the attribute value to -1:

```
set_db insts .test_check1 -1
```

but the following command displays an error and keeps the previous value.

```
set_db insts .test_check1 2
```

- The following command creates a Tcl procedure, `set_fxn`, and then creates an attribute named `test_set`. The `-set_function` parameter is specified so that you can change the value of the `test_set` attribute (provided it is valid):

```
{proc set_fxn {obj new_val cur_val} {  
+ if {$new_val > $cur_val} {  
+ return $new_val  
+ }  
+ return $cur_val  
+ }  
+ }  
  
define_attribute test_set -obj_type root -data_type int -category test -  
help_string "test set function" -set_function set_fxn
```

Now, the following command sets the attribute value to 1 because the previous value was undefined

```
set_db test_set 1
```

The following command changes the Attribute value to 4 as the new value is greater than the current value


```
set_db test_set 4
```

But with the following command the value is not changed as the new value is less than the current value (4)

```
set_db test_set 2
```

- The following command uses the `compute_function` to set attribute values. Note that the attribute becomes read-only if `compute_function` is applied.

```
proc comp_func {obj} { return 5}
```

```
define_attribute test_compute -obj_type root -data_type int -category test -
```

```
help_string "test set function" -compute_function comp_func
```

```
get_db test_compute 5
```

Now, the following command will issue an error as attribute value cannot be changed.

```
set_db test_compute 3
```

Related Commands

- `delete_obj`
- `get_db`
- `reset_db`
- `set_db`

delete_obj

```
delete_obj any_object+  
[-help]
```

Deletes one or more existing objects from the database. The command can delete `netlist` objects and physical data like `routing`, `floorplan`, and DFT objects, but not the `timing` objects (like `rc_corner`).

The following `obj_types` can be deleted through this command: `special_wire` | `special_via` | `place_blockage` | `route_blockage` | `pin_blockage` | `resize_blockage` | `text` | `gui_text` | `gui_line` | `gui_rect` | `gui_polygon` | `marker` | `row` | `bump` | `bus_guide` | `pin_guide` | `pin_group` | `net_group` | `group` | `flow` | `flow_step` | `wire` | `via` | `patch_wire` | `virtual_wire` | `port_shape` | `attribute` | `route_type` | `what_if_wire` | `what_if_via` | `hinst` | `hpin` | `hport` | `port` | `module` | `inst` | `net` | `hnet` | `sdp` | `formal_verification_constraint` | `test_signal` | `test_clock` | `test_clock_domain` | `port_bus` | `hport_bus`

Parameters

<code>-help</code>	Provides a brief description of the command that includes the type and default information for parameters. For a detailed description of the command and all its parameters, use the <code>man</code> command: <code>man delete_obj</code>
<code>any_object+</code>	Specifies the object or the list of objects to be deleted.

Examples

- The following command deletes all the place_blockages:
`delete_obj [get_db place_blockages]`
- The following command deletes the specified place_blockage object:
`delete_obj place_blockage:0x2b77dfbf0048`
- The following command deletes all the special routes from the VDD net:
`set my_net [get_db pg_nets VDD]`
`delete_obj [get_db $my_net .special_wires]`
`delete_obj [get_db $my_net .special_vias]`
- The following command deletes a group of SDPs:
`delete_obj sdp:DTMF_CHIP/wCardInstGrp_1`

Related Commands

- `define_attribute`
- `get_db`
- `reset_db`
- `set_db`

get_db

```
get_db
[-help]
{[start [chain] [pattern1 [pattern2 ..] [[-if expression] | [-expr expression]] [-dbu]
[-foreach tcl_body] [-index {object | obj_type1 name1 [object | obj_type2 name2 ...]] [-
invert] [-regex] [-match_hier] [-unique] [-computed]]
[-category category]
[-depth {[min] max}]
[-quiet | -skip_fail]}
```

Returns objects and attributes in the database with different types of filtering.

The `get_db` command takes a dual-ported object, dual-ported object list or a collection as input, then uses a period (.) to traverse to other related objects or to access attributes. You can use additional periods to traverse the data base schema.

Note: A Dual-Ported Object (DPO) is a `Tcl_Obj` in the C++ programming interface. The pointer is kept as a C++ pointer inside Tcl unless Tcl forces it to be evaluated as a string. In normal usage it never gets converted to a string which is more efficient, but if you do a “puts \$var” or return the value to the shell, it will be converted to its “dual” string form.

- The string form is normally a useful name preceded by the `obj_type`, so a `base_cell` object string form might look like `base_cell:and2`.
- A netlist name will have the current design name included, so an instance named `i1/i2` in design top, would look like `inst:top/i1/i2`.
- An object with no name like a wire, will just show the pointer hex-value like `wire:0x22222`.

The `get_db` command accepts collections as an input but it only returns objects in dual-ported object list form.

For descriptions on all objects and attributes, see [Common UI Database Object Information](#).

The root object has the current design data under the `current_design` attribute, while technology and library data are normally directly accessible as root attributes (e.g. layers and `base_cells`). The `root` object is the default starting point if there is no input list or collection given.

You can query a root attribute in any of the following ways:

- `get_db / .layers`
where, `'/'` is the notation for the root object
- `get_db layers`
This implicitly starts at the root. In this usage, no `"."` is used before the "layers" attribute name.

For example, you can use the following command to get all the driver-pins of all nets in `current_design`:

```
>get_db current_design .nets.drivers
```

Note: The `current_design` is implicitly created by the `init_design` or the `read_db` commands.

Frequently used `current_design` attributes like `insts` or `hinsts` (hierarchical `insts`) have root attribute aliases (or shortcuts). These attributes can be accessed by just typing the root attribute alias name. For example, the following commands gets all instances of the current design with names that start with `i1/i2*`:

- `get_db current_design .insts i1/i2*`
- `get_db insts i1/i2*`

Where, `insts` is a root attribute alias for `current_design .insts`

Some of the more commonly used aliases from the `current_design` attributes include: `nets`, `pins`, `insts`, `hnets`, `hpins`, and `hinsts`.

The `get_db` command supports auto-complete when you press the <Tab> key. It will give a list of all matching attributes starting with the letters entered so far, or it will complete the name if it is unique.

For example:

```
get_db insts .par<Tab>
parent partition
```

If you press the <Shift+Tab> keys together, the matching attributes are displayed along with their help string, like this:

```
>get_db insts .par<Shift+Tab>
```

```
parent # The parent of the inst, which is either a hinst for an instance within
hierarchy or the design object for an instance at the toplevel.
```

```
partition # If this inst is a physical black_box, this partition will carry the pin
constraints and related data for the blackbox. Otherwise it is empty.
```

There are two database definition objects: `obj_type` and `attribute`. If you want to query all the details for an attribute definition on an `obj_type`, rather than an attribute value, use the following command:

```
>get_db attribute:<obj_type>/<attribute_name> .*
```

You can use `get_db` to query user-defined properties from LEF. Following is the mapping between the LEF `objectType` and database object:

```
LAYER -> layer
LIBRARY -> design
MACRO -> base_cell
```

```
NONDEFAULTRULE -> route_rule  
PIN -> base_pin  
VIA -> via_def  
VIARULE -> via_def_rule
```

Note: If `PROPERTYDEFINITION` is defined for `LIBRARY`, the `PROPERTY` will be mapped as a design attribute with a `tech_lef_property_*` prefix.

Note: The Litmus software does not support copy-pasting of text containing pointers. The following should not be copied within the same or different terminal:

- part of a pointer
- pointer from one session to another session
- dynamic object pointer released after a command

Note: For better performance, it is recommended to use the `get_property` and `get_<object>` commands for object search in timing constraints and custom reports.

Parameters

<code>-help</code>	Outputs a brief description that includes the type and default information for each <code>get_db</code> parameter.
--------------------	--

`-category category`

Specifies a category of root attributes to return. This is intended to return all the root attributes associated with one application category. You cannot use a *start* argument with the `-category` option, so this option cannot be used for anything but root attributes. The return value is list of root attribute names.

Example:

The following command can be used to see all the root attributes associated with the category place:

```
get_db -category place
```

The output is as follows:

```
place_design_floorplan_mode place_design_refine_place  
place_detail_allow_border_pin_abut ...
```

You can view the complete list of categories with the `get_db categories` command as follows:

```
get_db categories
```

```
active_logic_view add_endcaps add_fillers add_rings ...
```

<i>chain</i>	Specifies the <i>chain</i> in the form <code>[.attribute_name1 [.attribute_name2] ...]</code>. A chain always starts with a <code>"."</code>. The specified attribute names are valid for each level of the traversal in the <i>chain</i>. Example: The following command returns the names of pins of <code>\$my_insts</code> <pre>get_db \$my_insts .pins.name</pre> If the chain has a <code>*</code> or <code>?</code> in the attribute name, it will be expanded to all matching attribute names, and the display will show the attribute name and current value for every object. This is intended for interactive usage, and no values are returned to Tcl. Values that may require significant time to compute (like timing data that requires the timing-graph) will only be displayed if <code>-computed</code> is given. For example: <pre>get_db \$my_insts .b* Object: inst:top/i1 base_cell: base_cell:BUFX2 bbox: {34.32 10.08 38.94 15.12} Object: inst:top/i2 ...</pre>
--------------	--

<code>-computed</code>	Forces <code>get_db</code> to output the values of attributes that may require significant time to compute (like timing data that requires the timing-graph) when browsing many attributes at the same time. This is intended for interactive usage when using a pattern match with <code>*</code> or <code>?</code> for an attribute name. If only a single attribute name is given, then the value is always returned. Example: <pre>>get_db \$my_pins .slack_max* Object: pin:top/inst\$odd/Y slack_max: NC slack_max_edge: NC ... >get_db \$my_pins .slack_max* -computed #if timing-graph not available yet, it will be computed Object: pin:top/inst\$odd/Y slack_max: 550.12 slack_max_edge: rise ... >get_db \$my_pins .slack_max #single attribute names always return value and force computation if needed 550.12 ...</pre>
------------------------	---

<code>-dbu</code>	Indicates that attribute values (coord, pt, rect, polygon, area, ...) should be returned as an int in database units. <i>Default:</i> If not given, the value returned is a double in microns (or microns**2). Examples: • In the following command the return type is a double in microns. <pre>get_db \$my_inst .location {1.0 1.0}</pre> • In the following command the return type is an int in database units. <pre>get_db -dbu \$my_inst .location {1000 1000}</pre>
<code>-depth {[min] max}</code>	Specifies the hierarchy depth range. It expands each <code>design</code> or <code>hinst</code> object in the input list into a list of <code>design</code> or <code>hinst</code> objects by descending the hinst hierarchy from <i>min</i> to <i>max</i> depth. The default value of <i>min</i> is 0, if not specified. 0 here signifies the level of the starting object. The <code>-depth</code> expansion happens first, and the resulting expanded list is then processed as the input list of objects by <code>get_db</code>. For example: <pre>get_db hinst:top/h1 -depth {0 1}</pre>Returns <code>hinst:top/h1</code>, and the first level of hinsts below it.

`-expr expression`

Returns the object if the *expression* is true. The *expression* argument is a strict Tcl expression that can be passed as is to the Tcl expression command. It has a pre-defined Tcl array named `$obj` that is populated by `get_db` with any attribute or chain value for each object as they are visited, and `$obj(.)` returns the object itself. For example,

```
get_db insts -expr {$obj(.base_cell.name) eq "AND2"}    #all
insts that are AND2
```

Normally the `-if expression` parameter is faster for simple expressions, but it is limited to a few operators, and simple `$var` substitution. If you need a full Tcl expression semantics including the proc calls inside `[]`, special char escaping, or complex `$var` substitution, you must use `-expr`.

This parameter can be used with a *chain* and *pattern* like this:

```
get_db $my_insts .pins i1/* -expr {$obj(.base_name) eq "A"}
```

Here, the `i1/*` pattern is checked first for pin names that match `i1/*`, then the pin `.base_name` is checked if it is equals "A".

`-expr` cannot be used at the same time with `-if`.

<code>-foreach tcl_body</code>	Specifies a list of Tcl commands to process for each object. This parameter avoids the overhead of creating long lists of objects. The Tcl array <code>\$obj()</code> is populated with the attributes of each object when it is visited, and <code>\$obj(.)</code> is the object itself. The Tcl commands <code>break</code> and <code>continue</code> also work as expected in a <code>foreach</code> loop (<code>break</code> stops the traversal of any more objects and <code>get_db</code> returns). Unlike inside <code>-if</code>, the <code>tcl_body</code> is strict Tcl. The <code>.attribute</code> names are not substituted, you must use <code>\$obj()</code> to access the attribute values instead. If you have a <code>pattern</code> or <code>-if</code>, or <code>-expr</code> usage, then <code>-foreach</code> only processes each object that matches the <code>pattern</code>, <code>-if</code>, or <code>-expr</code> expression. The following command prints the names of every inst that starts with "abc" and is an AND2 gate: <pre>get_db insts abc* -if {\$.base_cell.name == AND2} -foreach {puts \$obj(.name)}</pre> It returns the total number of <code>insts</code> that are printed.
<code>-if expression</code>	Specifies a boolean expression, with extensions to make pattern matching simpler. • <code>==</code> and <code>!=</code> do not require a quoted string, and they support pattern matching with <code>*</code> and <code>?</code>. • Other operators like <code>></code>, <code><</code>, <code><=</code>, <code>>=</code>, <code>&&</code>, <code> </code>, and the unary negate operator <code>!</code> are supported. • Parentheses <code>"()"</code> can be used to group complex expressions. Note: Unlike the <code><pattern></code> argument, inside the <code>-if</code> expression, you cannot use a string pattern to implicitly match the <code>.name</code> attribute of an object. You must use the <code>.name</code> field directly. For example, the following command will not match anything because <code>.base_cell</code> will become something like <code>base_cell:AND2</code>. <pre>get_db insts -if {\$.base_cell == buf*} #matches nothing</pre> use either of these instead: <pre>get_db insts -if {\$.base_cell.name == buf*} #match the .name attribute</pre>

```
get_db insts -if {.base_cell == base_cell:buf*} #match the
DPO name directly
```

Note: If you use an attribute that is an object list (like .pins), every object in the list is tested in an “implicit OR”, so if any of them pass, the expression is matched.

Handling of indexed attributes inside of the `-if` statement:

- The `-index` option is kept outside of the `-if` statement
- The `-index` option MUST be valid for ALL attributes inside the `-if` statement otherwise an error occurs

Examples:

- The following command finds all instances with `base_cell` names that matches “buf*”.

```
get_db insts -if {.base_cell.name == buf*}
```

- The following command returns all insts that have one or more pins that match `CLK*`. Only one copy of the matching inst object is returned, even if multiple pin names match `CLK*`.

```
get_db insts -if {.pins.base_name == CLK*}
```

- The following command gets all pins where `delay_max_rise` is greater than 5 or `slew_max_fall` is smaller than 8. This is a valid command as both `delay*` and `slew*` are attributes indexed by `view` (`view` is a legal abbreviation of `analysis_view`).

```
get_db pins -if {.delay_max_rise > 5 || .slew_max_fall <
8 } -index {view v1}
```

Note: The `-if` parameter does not support the usage of embedded Tcl command as operands. Use the `-expr` for this requirement.

```
-index {object |
obj_type1 name1
[object | obj_type2
name2 ...]}
```

Some attributes represent an array of values. For example, a `pin.slack_max_rise` attribute has a different value for each `analysis_view` and each `clock` that can affect that pin. With no index, a default method defined for that attribute is used (for example, the maximum, or minimum of all values in the array). For `slack_max_rise` with no `-index` value, the value returned is the worst-case slack across all `analysis_views` and `clocks`.

If you want to return a specific value, you must include the index value. The index values are object pointers, or a pair of `obj_type` name and object name (for example `clock clk1`). The word `view` is accepted as an abbreviation for `analysis_view`.

For example:

```
get_db $my_pins .slack_max_rise -index "view func_wc_cworst"
```

Will return the worst case `.slack_max_rise` for the `analysis_view` `func_wc_cworst` across all clocks. This attribute also accepts a clock index, so you can further refine it like this:

```
get_db $my_pins .slack_max_rise -index "view func_wc_cworst
clock clk1"
```

To get the value for that specific `analysis_view` and `clock`.

In the special case of a clock and `analysis_view` as shown above, a clock object is sufficient for both, because a clock object is based on both the clock name and the view name. So you could also do this:

```
set my_clk_obj [get_db clocks -if {.view_name ==
func_wc_worst && .base_name == clk1}]

get_db $my_pins .slack_max_rise -index $my_clk_obj
```

The `-index` parameter also accepts a pattern of attribute name to be matched. While specifying an attribute pattern string with `-index`, attributes that do not support `-index`, will return their normal value. But the attributes that support `-index` will return the values specified by the index. For example:

```
get_db net:dtmf_recvr_core/scan_out2 .* -index {view func-
wc_cworst}
```

Will return all the attributes and values for the `net` object. Any attribute that supports a view index will return the value for that view, while the other attributes will return the value as if `-index` was not specified.

`-invert`

Negates the `pattern`, `-if expression`, and `-expr expression` filters to return the objects/attributes that do not match instead of the ones that do.

<code>-match_hier</code>	Specifies that the <code>*</code> in the <i>pattern</i> and <i>expression</i> expands to the next hierarchical divider character <code>/</code> instead of a complete string match. Example: The following command matches <code>i1</code>, <code>i2</code>, but not <code>i1/i2</code>. <pre>get_db insts i* -match_hier</pre>
--------------------------	--

<i>pattern1</i> [<i>pattern2</i> ..]	Specifies string pattern(s), where: * matches 0 or more of any characters ? matches exactly one character More complex patterns require <code>-regex</code> or <code>-if</code> expression. Refer to the following examples: - The following command finds all instance names starting with A: <pre>get_db insts .name A*</pre> A0 A1/A2 ... - The pattern matching can also work directly on a dual-ported object. In that case, it will match the <code>.name</code> attribute of the object, but return the object rather than the <code>.name</code> string. This matching works only for objects that have a <code>.name</code> attribute defined. For example, the following command finds all instance objects (not names) with a name starting with A: <pre>get_db insts A*</pre> inst:top/A0 inst:top/A1/A2 ... - The pattern matching can also work to match multiple lists of the name patterns. Refer to the following example: <pre>get_db insts */BC* *TC*</pre> CG/BC1 CG/BC2 TC1 TC2 TC3 TC4 Note: For the special case where a pattern string starts with <code>.</code> you need to use an extra <code>.</code> (an empty attribute name) so that <code>.text</code> is treated as a pattern and not as an attribute name. - The following command finds all nets whose names equal to <code>.odd_name</code> <pre>get_db nets . .odd_name</pre> - The following command finds all instance names starting with A or B: <pre>get_db insts A* B*</pre>
<code>-quiet</code>	Prevents displaying warning and error messages while using the <code>get_db</code> command.

-regexp	Specifies that <i>pattern</i> use <i>regexp</i> syntax instead of the simple * and ? style. It does not change the -if pattern matching. See the example below to understand the usage of the -regexp parameter: <pre>get_db insts -regexp {u_warmreset_cntr/reset_cntr_reg_(\d+)_}</pre>
-skip_fail	Skips the fail error, when the object does not have such an attribute, or the object/attribute itself cannot be recognized.
start	The starting objects or root attributes. Can be an object list, collection, root (/), or root attribute. Example: • The following command returns the root attribute <code>insts</code> that is an object list of all <code>insts</code> in <code>current_design</code>. <pre>>get_db insts</pre> • The following command starts from a dual-ported object list or collection of objects in the Tcl variable <code>\$my_list</code>, and returns a dual-ported object list. <pre>>get_db \$my_list</pre> • The following command returns a root attribute that is a simple value. <pre>>get_db timing_cap_unit</pre> • If the name has a * or ?, it will be expanded to all matching root attribute names, and the display will show the attribute names and current value. This is intended for interactive usage, and no values are returned to Tcl. For example, <pre>>get_db timing* Object: root:/ timing_allow_input_delay_on_clock_source: false timing_analysis_aocv: false ...</pre>

-unique	Removes duplicate entries from the return list. If the specified attribute is a list, this parameter returns a unique set of lists., and does not uniquifies the elements within the list. In such a case, use <code>lsort</code> .
---------	---

Built-In Sub-attributes

The types `rect`, `line`, `point`, and `polygon` have built-in sub-attributes. For a full list of all built-in sub-attributes, see [Common UI Database Object Information](#).

Example:

The `rect` type has built-in sub-attributes like:

area	Area of rect
dx	Delta x
dy	Delta y
ll	Lower-left point
ur	Upper-right point
width	Shorter of dx and dy
length	Longer of dx and dy
perimeter	Length of the rect perimeter

A `point` has sub-attributes like:

x	X coordinate
y	Y coordinate

Like other attributes you can chain them together.

For example, an `inst` object has a `.bbox` (bounding box) that is type “`rect`”. Here are some operations allowed on the `.bbox` attribute of an `inst`.

```
get_db $my_inst .bbox
{1.0 2.0 3.0 4.0}
get_db $my_inst .bbox.ll
```

```
{1.0 2.0}
get_db $my_inst .bbox.area
4.0
get_db $my_inst .bbox.ll.x
1.0
set box [get_db $my_inst .bbox]
{1.0 2.0 3.0 4.0}
get_db $box .ll
{1.0 2.0}
```

Examples

- The following command finds insts with names starting with PTN* that have at least one pin capacitance > 0.1. The pattern is checked first, and those that pass will be tested with the `-if` expression. Inside the `-if` expression, there are many `.pins` for one inst, so each pin is tested. If any of the pins have a capacitance > 0.1, the single inst will be matched.

```
get_db insts PTN2* -if {.pins.capacitance_max_rise > 0.1}
```

- The following command returns insts that are not physical-only cells whose name matches `i1/i2*`.

```
get_db insts i1/i2* -if {!.is_physical}
```

- The two commands below generate the same list of pins from `$my_insts` that have a `base_name` `clk`. The first uses a `*` pattern to match all pins, while the second has no pattern and defaults to all pins, so both cases check the `-if` expression for all pins of `$my_insts`.

```
get_db $my_insts .pins * -if {.base_name == clk}
get_db $my_insts .pins -if {.base_name == clk}
```

- The following command finds `buf1` cell instances connected to nets named `clk` and changes their placement status to `fixed`.

Note: The examples below is inefficient and used to illustrate different methods of traversal done in different styles. A more efficient method would be to start from the nets and looking at their pin connections directly.

Example 1:

```
set cell_name buf1
set net_name clk
foreach inst_obj [get_db insts] {
```

```
set cell_obj [get_db $inst_obj .base_cell]
if { [get_db $cell_obj .name] == $cell_name } {
    foreach pin_obj [get_db $inst_obj .pins] {
        if { [get_db $pin_obj .net.name] == $net_name } {
            set_db $inst_obj .place_status fixed
        }
    }
}
}
```

Example 2: Same output using full traversal style with intermediate variables. The second `-if` returns true if any of the `.pins.net.name` match `$net_name` (implicit OR).

```
set buf1_list [get_db insts -if {.base_cell.name == $cell_name}]
set buf1_clk1_list [get_db $buf1_list -if {.pins.net.name == $net_name}]
set_db $buf1_clk1_list .place_status fixed
```

Example 3: Same output using full traversal style (fully chained)

```
set_db [get_db insts -if {.base_cell.name == $cell_name && .pins.net.name ==
$net_name}] .place_status fixed
```

Example 4: Or use the `-foreach` method so no list is returned, but the number of buffers processed is returned.

```
get_db insts -if {.base_cell.name == $cell_name && .pins.net.name == $net_name} -
foreach {set_db $obj(.) .place_status fixed}
```

- The following command finds all the insts named `i1/i2` that end with `_buf`:

```
get_db insts i1/i2/*_buf
```

Or you could start inside the `hinst` named `i1/i2`:

```
get_db [get_db hinsts i1/i2] .insts *_buf
```

Or use the `hinst` dual-ported object name directly. Note that netlist dual-ported object names have the design name included, so if `top` is the design name you could do this:

```
get_db hinst:top/i1/i2 .insts *_buf
```

- The following command gets `arrival_max_rise` for the `analysis_view setup_test` for all pins. Here the abbreviation "view" is allowed for `analysis_view`.

```
get_db pins .arrival_max_rise -index "view setup_test"
```

- The following command finds insts that have a `cell_port` named `CK` and that pin is connected to a net named `clk*`:

```
get_db insts -if {.lib_cells.lib_pins.name == */CK && .pins.net.name == clk*}
```

- The following command uses `-foreach` to find the first inst named `u1/*` with `.place_status` "fixed" and then break. It uses strict Tcl inside the `-foreach`.

```
get_db insts u1/* -foreach { if {[string match fixed $obj(.place_status)]} {  
puts $obj(.name) ; break } }
```

- The following `get_db` command uses `-foreach` to count the number of each `base_cell` used in the netlist inside the `cell_count` Tcl array. You can then use `array get cell_count` to see a list of all the `base_cell` names and how many times they are used in the netlist.

```
get_db insts .base_cell -foreach {incr cell_count($obj(.name))}
```

- The following `get_db` command displays all the details of the `flow_template_type` attribute of the `root` `obj_type`:

```
get_db attribute:root/flow_template_type .*  
Object: attribute:root/flow_template_type  
base_name: flow_template_type  
category: flow  
check_function: {}  
compute_function: {}  
data_type: string  
default_value: {}  
help: {Type of template being run.}  
indices:  
is_computed: false  
is_obsolete: false  
is_settable: true  
is_user_defined: true  
name: root/flow_template_type  
obj_type: obj_type:root  
set_function: {}  
skip_in_db: false
```

- The following example depicts the use of the `-if` parameter to find all HVT insts:

```
get_db insts -if {.base_cell.name == *_HVT}
```

- The following command returns all hinsts 1 or 2 levels inside the design:

```
get_db design:top -depth {1 2}  
hinst:top/h1 hinst:top/h1/h2
```

- The following command finds all `local_insts` of each `hinst` at depth 2 that have `.dont_touch == true`:

```
get_db design:top .local_insts -depth {2 2} -if { .dont_touch == true }  
inst:top/h1/h2/i1 inst:top/h1/h2/i2
```

- All the following commands return inst with `.name i1/i2`:

```
set a i1/i2  
get_db insts i1/i2  
get_db insts -expr {$obj(.name) eq $a}  
get_db insts -expr {[get_db $obj(.) .name] eq "i1/i2"}
```

- The following command returns all nets with more than 5 loads:

```
get_db nets -expr { [llength $obj(.loads)] > 5 }
```

- The following command returns all insts with `base_cell.name AND2`:

```
get_db insts -expr {$obj(.base_cell.name) eq "AND2"}
```

Related Commands

- [define_attribute](#)
- [delete_obj](#)
- [reset_db](#)
- [set_db](#)
- For details on LEF property definitions, see the 'Property Definitions' section on the [LEF Syntax](#) chapter in the *LEF/DEF Language Reference*.
- For a list of objects attributes that can be queried for information or changed, see [Common UI Database Object Information](#).

is_attribute

```
is_attribute
[-help]
attribute_name
-obj_type obj_type
```

The `is_attribute` command checks whether attribute exists or not.

Parameters

<code>-help</code>	Outputs a brief description that includes type and default information for each <code>is_attribute</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man is_attribute.</code>
<code>attribute_name</code>	Specifies the name of the attribute to be checked.
<code>-obj_type obj_type</code>	Specifies the object type of the attribute. All valid object types can be found by typing <code>get_db object_types .name</code> at the prompt. The attribute is assigned to the specified type.

Examples

- The following `is_attribute` command applies check to existing attribute:

```
define_attribute test_delete -obj_type inst -data_type int -category test -
help_string "test delete function" -default 1
is_attribute -obj_type inst test_delete
1
```
- The following `is_attribute` command applies check to non-existent attribute:

```
is_attribute -obj_type pins test_delete
0
```

Related Commands

- For information on how to query DB attributes, see the [get_db](#) command.
- For a list of objects attributes that can be queried for information or changed, see the [Common UI Database Object Information](#) document.

set_db

```
set_db
[-help]
start
[chain]
value
[-dbu]
[-index {index_type1 name1 [index_type2 name2 ...]}]
[-quiet]
[-verbose]
```

Changes the specified attribute value for a database object. The `set_db` command takes a dual-ported object, dual-ported object list, or a collection as input, and then uses a period (.) to traverse to other related objects or to access attributes. You can use additional periods to traverse the data base schema.

This command returns the number of attributes set and the value. See the example below:

```
set_db hinsts .ungroup_ok false
6 false
```

In the above scenario usage, `values == value` as a single command can set only a single value (for multiple objects).

Note: A Dual-Ported Object (DPO) is a `Tcl_Obj` in the C++ programming interface. The pointer is kept as a C++ pointer inside Tcl unless Tcl forces it to be evaluated as a string. In normal usage it never gets converted to a string which is more efficient, but if you do a `puts $var` or return the value to the shell, it will be converted to its dual string form.

- The string form is normally a useful name preceded by the `obj_type`, so a `base_cell` object string form might look like `base_cell:and2`.
- A `netlist` name will have the current design name included, so an instance named `i1/i2` in design top, would look like `inst:top/i1/i2`.
- An object with no name like a wire, will just show the pointer hex-value like `wire:0x22222`.

The `set_db` command does not create or delete objects.

For information on what object attributes are settable, see the [Common UI Database Object Information](#) document. It contains a description of all database objects and attributes.

The root object has the current design data under the `current_design` attribute, while technology and library data are normally directly accessible as root attributes (For example, layers and libraries). The root object is the default starting point if there is no input list or collection given.

You can set any root attribute in any of the following ways:

- `set_db / .flow_mail_to name`
where, `'/'` is the notation for the root object , or,
- `set_db flow_mail_to name`
This implicitly starts at the root.

For example, you can use the following command to change the design text font:

```
set_db current_design .texts.font_number 4
```

Note: The `current_design` is implicitly created by the `init_design` or the `read_db` commands.

Frequently used `current_design` attributes like `insts` or `hinsts` (hierarchical insts) have root attribute aliases (or shortcuts). These attributes can be accessed by just typing the root attribute alias name.

For example, the following two commands set all insts `.place_status` to fixed.

```
set_db current_design .insts.place_status fixed
set_db insts .place_status fixed
```

Some of the more commonly used aliases from the `current_design` attributes include: `insts`, `hinsts`, `nets`, `hnets`, `ports`, `hpins`, `pins`.

The `set_db` command supports auto-complete when you press the `<Tab>` key. It will give a list of all matching attributes starting with the letters entered so far, or it will complete the name if it is unique.

For example:

```
set_db plan_design_use<Tab>
```

returns `plan_design_use_guide_boundary plan_design_use_sdp_group`

For information on how to access DB objects, see the `get_db` command.

Parameters

<code>-help</code>	Outputs a brief description that includes type and default information for each <code>set_db</code> parameter.
<code>start</code>	The starting objects or attributes. Can be an object list, collection, root (<code>/</code>), or root attribute.

<i>chain</i>	Specifies a <i>chain</i> in the form [.attribute1[.attribute2]...]. The attribute names must be valid for each level of the traversal in the <i>chain</i>, where the first name is valid for the start object. For example, <pre>set_db \$my_pins .inst.place_status fixed</pre> where .inst is an attribute of an pin object and .place_status is an attribute of an inst object.
<i>value</i>	Specifies new value for the specified object attribute elements. The value type depends on which object attribute type is being modified.
-dbu	Indicates that attribute values (coord, pt, rect, polygon, area, ...) should be in database units. <i>Default:</i> The default is microns (or microns**2).
<pre>-index {index_type1 name1 [index_type2 name2 ...]}</pre>	
	Some attributes represent an array of values. For example, a pins .slack_max_rise attribute has a different value for each analysis_view. With no index, all the values defined for that attribute are modified by the set_db command. If you want to change a specific value, you must include the index value. The index values are pairs of obj_type names (for example, analysis_view, clock, skew_group) followed by the name. The word “view” is accepted as an abbreviation for “analysis_view”. Example: Some ccopt attributes can take both a view and clock index like this: <pre>-index "view func_wc_cworst clock clk1"</pre>
-quiet	By default set_db is quiet. If the set_db_verbose root attribute is set to true, then the name of each object that is modified is echoed to the terminal (see the -verbose parameter's description for an example). -quiet forces set_db to be quiet even if set_db_verbose is true. This is intended for internal scripts that want to be certain that the verbose output is off.

<code>-verbose</code>	Turns on verbose output that echoes the name of each object that is modified. This can be useful during interactive debugging to ensure that the correct objects were affected. For example: <pre>set_db -verbose [get_db hinsts D*] .dont_touch false Setting attribute of hinst 'DATA_BUS_MACH_INST': 'dont_touch' = false Setting attribute of hinst 'DECODE_INST': 'dont_touch' = false 2 false</pre> Note: The <code>set_db_verbose</code> root attribute can also be set to true, to force verbose output for all <code>set_db</code> and <code>reset_db</code> usage without requiring <code>-verbose</code>.
-----------------------	---

Examples

- The following command sets a value of root attribute.

```
set_db place_global_solver_effort high
```

- The following commands define an instance attribute and change its value to 0.

```
define_attribute myattr -default 1 -obj_type inst -data_type int -help_string "my
attribute" -category my_cat
set_db insts .myattr 0
```

- The following command sets an instance attribute value to 5 only if its current value is 0.

```
set_db [get_db insts -if {.myattr == 0}] .myattr 5
```

Related Commands

- For information on how to query DB attributes, see the [get_db](#) command.
- For information on how to reset attribute values, see the [reset_db](#) command.
- For a list of objects attributes that can be queried for information or changed, see the [Common UI Database Object Information](#) document.
- [set_db Category Attributes](#)

reset_db

```
reset_db  
[-help]  
[start[chain]] |  
[-category category_name]  
[-quiet]  
[-verbose]
```

Resets attribute value to a pre-defined default value. It returns a single value when resetting a single attribute, a list of values when resetting same attribute on different object instances, or a human readable table for multiple attributes in the following format:

```
Attribute_name    value
```

For example:

```
Place_xxx true  
Add_filler  true
```



- The `-category` parameter of the `reset_db` command can be used to reset root attributes only. Both user-defined and system root attributes can be reset using the `-category` parameter.
- The attribute value can be reset if the `default_value` attribute is specified.

When the `-category` parameter is used, the command resets the values of both the hidden and unmapped attributes. In such a case, the command does not return the count of the reset attributes. The output in such cases will be like the following:

```
Some hidden attributes were also reset.
```

Parameters

<code>-help</code>	Outputs a brief description that includes type and default information for each <code>reset_db</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <pre>man reset_db.</pre>
<code>start</code>	Can be any object list (or collection) or <i>shorthand</i>. For example <code>nets</code>, <code>insts...</code>
<code>chain</code>	Specifies the <i>chain</i> in the form <code>[.attribute_name1 [.attribute_name2]...]</code>. The specified attribute names are valid for each level of the traversal in the <i>chain</i>.
<code>-category category_name</code>	Specified root attribute category based on predefined list of categories. For example <code>timing</code>. Note: Applies only to root level attributes and ignored if applied to lower levels.
<code>-quiet</code>	By default <code>set_db</code> is quiet. If the <code>set_db_verbose</code> root attribute is set to true, then the name of each object that is modified is echoed to the terminal (see the <code>-verbose</code> parameter's description for an example). <code>-quiet</code> forces <code>set_db</code> to be quiet even if <code>set_db_verbose</code> is true. This is intended for internal scripts that want to be certain that the verbose output is off.
<code>-verbose</code>	Turns on verbose output that echoes the name of each object that is modified. This can be useful during interactive debugging to ensure that the correct objects were affected. For example: <pre>set_db -verbose [get_db hinsts D*] .dont_touch false Setting attribute of hinst 'DATA_BUS_MACH_INST': 'dont_touch' = false Setting attribute of hinst 'DECODE_INST': 'dont_touch' = false 2 false</pre> Note: The <code>set_db_verbose</code> root attribute can also be set to true, to force verbose output for all <code>set_db</code> and <code>reset_db</code> usage without requiring <code>-verbose</code>.

Examples

- The following reset_db command resets the value of instance attribute:

```
define_attribute test_reset_inst -obj_type inst -data_type int -category reset -
help_string "test reset function" -default 1
1
set_db insts .test_reset_inst 10
10 10 10 10 10          #Four instances in the design
reset_db insts .test_reset_inst
1 1 1 1 1              #All 4 attributes are reset to default values
```

- The following command resets same set of attributes using the -category parameter.

```
reset_db -category reset
```

The attributes are reset to their default values and a human readable table with new values is displayed.

```
test_reset1 1
test_reset2 1
test_reset3 1
```

Related Topics

- For information on how to query DB attributes, see the [get_db](#) command.
- For information on how to set DB attributes, see the [set_db](#) command.
- For a list of objects attributes that can be queried for information or changed, see the [Common UI Database Object Information](#) document.
- [set_db Category Attributes](#)

sort_db

```
sort_db  
[-help]  
obj_list  
[attribute_list]  
[-descending]  
[-dictionary]
```

Returns a sorted DPO (dual-ported object) list based on the contents of one or more attributes. It is based on `lsort` using `get_db` to access the various attributes.

This command sorts objects only based on the attributes that have a "simple" data type of string, float, or integer, or object. If it is an object, it is implicitly sorted by the `.name` field and treated as a string. By default, the `sort_db` command sorts any input list as string. But when the `-dictionary` parameter is specified, the numeric values are sorted numerically, like `lsort`.

Parameters

<code>-help</code>	Outputs a brief description that includes type and default information for each <code>sort_db</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <pre>man sort_db</pre>
<code>obj_list</code>	A DPO list to be sorted. If the first element of the specified list is not a DPO, then it is treated as a root attribute name and passed directly to <code>get_db</code> to return a DPO list. So, the following two commands have identical results: <pre>sort_db insts</pre> <pre>sort_db [get_db insts]</pre>
<code>attribute_list</code>	Sorts the DPO list by the specified attribute values. The first attribute in the <code>attribute_list</code> has the highest sort precedence. Each attribute can be a chain identical to what <code>get_db</code> supports. A leading "." is optional, so all the following commands presenting the use of <code>attribute_list</code> specify legal arguments for an inst list: <pre>sort_db insts name</pre> <pre>sort_db insts .name</pre> <pre>sort_db insts {location.x location.y}</pre> <pre>sort_db insts {.location.x .location.y}</pre>
<code>-descending</code>	Sorts the list of objects by the descending order.
<code>-dictionary</code>	Sorts the objects using string comparison in all cases, except when the strings contain numbers. The numbers in strings are compared as integers/real.

Examples

- The following command sorts all the insts by name:

```
sort_db insts
```
- The following command sorts the pins based on their slack. The pins with the same slack are further sorted by the x-location:

```
sort_db [get_db pins] {.slack .location.x}
```

Related Topics

- [get_db](#)
- [reset_db](#)
- [set_db](#)

Delay Calculation Commands

- [get_aging_analysis_mode](#)

get_aging_analysis_mode

```
get_aging_analysis_mode
[-help]
[-activity_propagation_mode]
[-constraints]
[-disable_aging]
[-max_duty_cycle]
[-max_toggle_rate]
[-max_toggle_rate_limit <value>]
[-min_duty_cycle]
[-min_toggle_rate]
[-param_priority_sets]
[-recovery]
[-sequential_activity_propagation]
[-sequential_max_duty_cycle]
[-sequential_max_toggle_rate]
[-sequential_min_duty_cycle]
[-sequential_min_toggle_rate]
[-type]
[-use_fixed_duty_cycle]
[-views]
```

Returns the aging analysis related global variable settings and defaults - that have been made using the `set_aging_analysis_mode` command.

Parameters

Displays the following information about the specified [set_aging_analysis_mode](#) parameters in the Tempus log file and the Tempus console:

- Parameter name

- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

If you do not specify any parameters, the software displays information for all of the parameters.

Parameters

<code>-help</code>	Prints out the command usage.
<code>parameter_names</code>	Displays information for the specified parameters. You can specify one or more parameters. See set_aging_analysis_mode for descriptions of the parameters you can specify.

Related Commands

- [set_aging_analysis_mode](#)
- `set_db timing_analysis_aging`
- [set_aging_activity](#)
- [create_aging_stress_profile](#)
- [write_spice_deck](#)

Related Information

- "[Aging-Aware STA Analysis](#)" chapter in the *Tempus Stylus User Guide*.

Design Initialization Commands

- [configure_lint_check](#)
- [init_design](#)
- [input_design](#)
- [read_hdl](#)
- [read_libs](#)
- [read_lint_configuration](#)
- [read_mmmc](#)
- [read_netlist](#)
- [read_sdc](#)
- [report_black_box](#)
- [report_lint_configuration](#)
- [report_sdc](#)
- [write_lint_configuration](#)

configure_lint_check

```
configure_lint_check <cmd_name> {  
    command {  
        usage { supported | unsupported | forbidden | unhandled}  
        is_sdc_compliant { yes | no }  
        warn_if_overwritten { yes | no }  
        mandatory_options { <option> | <option> [ | <option> ]* }  
        recommended_options { <option> | <option> [ | <option> ]* }  
        mutex_options { <option> | <option> [ | <option> ]* }  
        append_msg { lint_rule_name "msg" }  
    }  
    <option> {  
        usage { mandatory | recommended | dontcare | forbidden | unsupported}  
        is_sdc_compliant { yes | no }  
        required_options { <option> [ | <option> ]* }  
        valid_types { [<type>]+ }  
        valid_values < [<pattern>]+  
    }  
    valid_range { <Tcl expression> }  
    valid_object_types { [<object_type> [-if <expression>] ]+ }  
    flag_on_object_types { [<object_type> [-if <expression>] ]* }  
    append_msg { <lint_rule_name> "msg" }  
}
```

Configures the parameters that affect SDC lint checks.

Parameters

Parameters for `command`

usage

	supported: Command is supported in the SDC file. No checks are flagged. unsupported: Command is not supported in the SDC file. <code>CONST_LINT_CMD2</code> rule check is flagged every time the command is used. forbidden: User-defined forbidden command in the SDC file. Specifies that the command, although valid (in Litmus CD and the target tool), cannot be used in the SDC file. <code>CONST_LINT_CMD3</code> is flagged every time the command is used. unhandled: User-defined unhandled command in the SDC file. Specifies that the command is valid in Litmus, but is not valid in the target tool. <code>CONST_LINT_CMD8</code> is flagged every time the command is used.
<code>is_sdc_compliant</code>	
	yes: Specifies that this command is SDC compliant. No checks are flagged. no: Specifies that the command is not SDC compliant. <code>CONST_LINT_CMD4</code> is flagged every time the command is used.
<code>warn_if_overwritten</code>	
	yes: Issues a command each time this command is overwritten. <code>CONST_LINT_CMD6</code> is flagged each time this command is overwritten. no: Do not issue a warning if this command is overwritten.
<code>mandatory_options</code>	Species that at least one of the listed options must be specified for the given command. <code>CONST_LINT_OPT6</code> is flagged each time the given command is used without any of the specified options. You can specify more than one set of mandatory options.
<code>recommended_options</code>	Specifies that at least one of the listed options is recommended for the given command. <code>CONST_LINT_OPT7</code> is flagged each time the given command is used without any of the listed options. You can specify more than one set of recommended options.

mutex_options	Specifies that the listed options are mutually exclusive for the given command. <code>CONST_LINT_OPT8</code> is flagged each time the given command is used with more than one of the listed options. You can specify more than one set of mutually exclusive options.
append_msg	Append the specified message to each occurrence of the specified lint rule check.

Parameters for `option`

usage	
	mandatory: Option is mandatory in the SDC command. <code>CONST_LINT_OPT6</code> rule check is flagged each time the given option is not used. recommended: Option is recommended in the SDC command. <code>CONST_LINT_OPT7</code> is flagged each time this option is not used. dontcare: No checks are flagged. forbidden: Option is forbidden in the SDC command. <code>CONST_LINT_OPT4</code> is flagged each time this option is used. unsupported: Option is not supported in the SDC command. <code>CONST_LINT_OPT2</code> rule check is flagged every time the option is used. When an option is specified as unsupported, you cannot modify it; if Litmus encounters an SDC command with an unsupported option, it will ignore the option.
is_sdc_compliant	
	yes: Specifies that this option is SDC compliant. No checks are flagged. no: Specifies that the option is not SDC compliant. <code>CONST_LINT_OPT4</code> is flagged every time the command is used.
required_options	Specifies that at least one of the listed options must be specified if the given option is used. <code>CONST_LINT_OPT5</code> is flagged each time the given command is used without any of the specified options. You can specify more than one set of required options.

valid_types	Specifies a list of valid types for the value(s) of the given option. <code>CONST_LINT_VAL2</code> is flagged each time a specified value does not match one of the listed values. Accepts wildcards. The following are the valid types: <code>enum</code>, <code>string</code>, <code>float</code>, <code>boolean</code>, <code>integer</code>, <code>explicit_or_implicit_obj</code>, and <code>explicit_obj</code>. See Examples section for more information.
valid_values	Specifies a list of valid values for the given option. <code>CONST_LINT_VAL3</code> is flagged each time a specified value does not match one of the listed values. Accepts wildcards. To specify an AND condition between lists of valid values, each list must be on a separate line. For example: <pre>valid_values *b valid_values {a* c*}</pre> specifies that the tool allows values that start with a or c AND end with b.
valid_range	Specifies a Tcl expression that determines if a specified value has a valid range or not. The expression should evaluate to 1 (true) if the value has a valid range. <code>CONST_LINT_VAL4</code> is flagged each time a specified value is out of range. Two predefined variables can be used in the expression: <code>\$value</code> is the value being checked and <code>\$length</code> is the number of elements if the value is a list.
valid_object_types	Specifies a list of valid object types for the given option. <code>CONST_LINT_REF4</code> is flagged for every object of an invalid type. In addition, you can define an expression (in the format accepted by the <code>find</code> command) to determine if an object of a certain type is valid. If the expression evaluates to true, the object is valid. See Supported Object Types.
flag_on_object_types	Specifies a list of object types that are not recommended for the given option. <code>CONST_LINT_REF7</code> is flagged each time one of the specified object types is used. See the Examples section for an example that uses an expression to specify the object type.
append_msg	Append the specified message to each occurrence of the specified lint rule check.

Supported Object Types

The following lists the supported object types:

port	Any type of port: <code>input_port</code> , <code>output_port</code> , or <code>inout_port</code> .
net	Type of net. <code>leaf_net</code> specifies a net with a leaf-level driver pin.
hier_net	Hierarchical net.
pin	Any type of pin: <code>leaf_cell_pin</code> (pin must be a leaf cell pin), <code>timing_exc_start_point</code> (clock pin of a DFF/DLAT, a data pin of a DLAT, or a pin with input or output delay specified), or <code>timing_exc_end_point</code> (clock pin of a DFF/DLAT or a data pin of a DLAT, or a pin with output delay specified).
hier_cell_pin	Hierarchical pin.
cell	Any type of cell: <code>hier_cell</code> (hierarchical cell), <code>leaf_cell</code> (non-root, nonhierarchical; includes black boxes, library cells and RTL primitives), <code>seq_cell</code> (sequential element), or <code>library_cell</code> (library cell).
clock	Any type of clock: <code>virtual_clock</code> , <code>real_clock</code> , or <code>generated_clock</code> .
design	Any type of design.
lib_cell	Library cell.
lib_pin	Library pin.
lib	Library.

Examples

The following command changes the valid values of the `-analysis` type option and makes the `-max` option mandatory in the `set_operating_conditions` command:

```
configure_lint_check set_operating_conditions {  
-analysis_type { valid_values { single bc_wc } }  
-max { usage mandatory }  
}
```

The following command specifies that input ports are the only valid objects for the object list of the `set_clock_uncertainty` command:

```
configure_lint_check set_clock_uncertainty {
  object_list {
    valid_object_types {
      port -if "(direction==in)"
    }
  }
}
```

The following command specifies that any clock name specified for the `create_clock` command must start with `clk` and the period specified must be between 0 and 10:

```
configure_lint_check create_clock {
  -name { valid_values clk* }
  -period { valid_range { (($value>=0) && ($value <= 10)) } }
}
```

The following command specifies that the `-min` option is required when using the `-max` option in the `set_clock_transition` command:

```
configure_lint_check set_clock_transition {
  -max { required_options { -min } }
}
```

The following command specifies that only objects (not strings) can be specified for the `set_clock_transition` command:

```
configure_lint_check set_clock_transition {
  clock_list { valid_types object }
}
```

The following command specifies that the `set_level_shifter_strategy` is not supported:

```
configure_lint_check set_level_shifter_strategy {
  command { usage unsupported }
}
```

The following command specifies that the message "Unhandled for RC" be appended each time `CONST_LINT_OPT10` is triggered for the `-reset_path` option of the `set_false_path` command:

```
configure_lint_check set_false_path {
  -reset_path { append_msg { CONST_LINT_OPT10 "Unhandled for RC" } }
}
```

The following command specifies that objects for the `create_clock` command can be specified with or without a corresponding `get*` command that specifies the object type:

```
configure_lint_check create_clock {  
  source_objects { valid_types explicit_or_implicit_obj }  
}
```

With the following command, objects for the `create_clock` command that are specified by name, but do not have a corresponding `get*` command that specifies their object type, will trigger the `CONST_LINT_REF5` rule check since this option has more than one available object type (`port`, `pin`, and `net`):

```
configure_lint_check create_clock {  
  source_objects { valid_types explicit_obj }  
}
```

The following example specifies a pin whose `is_start_point` attribute is false is not recommended:

```
configure_lint_check create_clock {  
  flag_on_object_types { pin -if {!is_start_point}}  
}
```

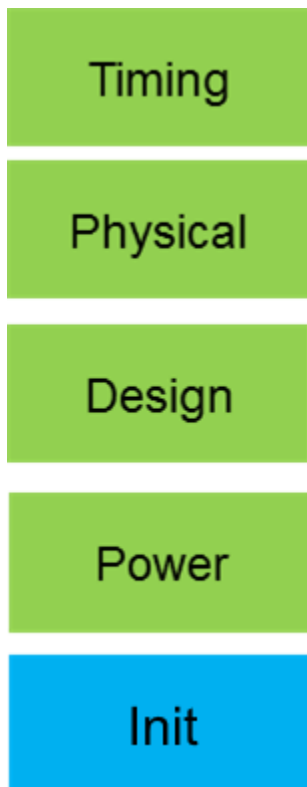
init_design

```
init_design [-help]
```

Initializes the database and ensures that the tool is ready for full execution.

During `init_design`, the tool steps through the defined MMMC objects, the design object, and the power requirements and builds up the full representation of the design. After initialization is complete, the design is in a consistent state and is ready for manipulation (floorplanning, synthesis, and so on) or timing.

The `init_design` command corresponds to the *Init* step in the Initialization flow. In this step, the full database is prepared for execution. The constraints are loaded into the correct mode, the power data is loaded and applied to the appropriate instances, and the operating conditions are applied to the power domains. Extensive error checking is performed, including checking of objects, and checking for completeness of data. After the *Init* step is complete, the database is completely consistent and ready for execution.



The `init_design` command can be broken into the following sub-steps, which are executed by the tool without requiring any user intervention:

1. Power Domain Creation

First, the power domains are populated by the design data. The design instances associated with a particular domain are defined in the Common Power Format (CPF) or UPF file. This instance data is used to populate the power domain attributes. At the end of this sub-step, the different power domains are resolved, and every instance in the system is associated with its power domain.

2. Power Domain Binding to Domain Corners

After the power domains are fully resolved, they can be bound to the library sets in the various `create_domain_corner` commands. In this sub-step, the active domain corners are processed with each instance getting assigned to the appropriate library set.

3. Power Domain Object Processing

After the power domains are associated with library sets, the power domain objects (state retention, iso cells, and level shifters) can be processed. The power object rules are processed against the defined library sets for each domain to identify the power cells to be used.

4. Constraint Loading

During constraint loading, the SDC files required by the active view are loaded and error checking is done. In addition, the RC corner defined by the required `domain_corner` is processed and loaded. In both cases, syntax and error checking is performed.

5. Active View Setting

The final step is to set the active view based on the `set_active_view` command, and to perform final checks to ensure the view is fully and completely defined. At the conclusion of this command, the system is ready for timing.

After a design has been initialized with `init_design`, the design is ready for execution. Execution can include incremental updates to timing.

Timing Incremental Updates

After `init_design`, you can enter MMMC commands at the command line. These commands populate the mmmc database objects, and perform error checking to report missing objects. For example, if you create a timing condition involving libsets that do not exist, an error will be issued. No data load takes place during command entry. Instead, the loading of new libraries, constraints, and so on, is deferred until an additional `init_design` is entered.

After you have completed entering the MMMC commands, issue the `init_design` command to load the new timing intent. After `init_design` is complete, the database is ready for timing again.

The exception to requiring `init_design` is `set_analysis_view`. When this command is issued and

datasets are not present in memory, the tool will effectively issue an `init_design` to load in the required data. If the data is already in memory, the view information is changed without any additional load.

Note: Design data (netlist) and physical data (LEF/OA) incremental update is not allowed. After the first `init_design` is completed, these steps are effectively disabled. If you issue a `read_netlist` or `read_physical` command after the first `init_design`, the tool returns an error.

Parameters

<code>-help</code>	Prints out the command usage. For a detailed description of the command, use the <code>man</code> command: <pre>man init_design</pre>
--------------------	---

Related Commands

- `read_netlist`

input_design

```
input_design  
[-help]  
[-hdl_script_1 <string> | [-netlist_1 <string> -top_1 <string>]]  
[-hdl_script_2 <string> | [-netlist_2 <string> -top_2 <string>]]
```

Loads a design using either an HDL script or a netlist with a specified top module to establish the design context.

Parameters

-help	Prints out the command usage.
-hdl_script_1 <string>	Specifies Design1's HDL Script (string, required)
-hdl_script_2 <string>	Specifies Design2's HDL Script (string, required)
-netlist_1 <string>	Specifies Design1's Netlist (string, required)
-netlist_2 <string>	Specifies Design2's Netlist (string, required)
-top_1 <string>	Specifies Design1's top module (to be used when -netlist_1 is specified) (string, required)
-top_2 <string>	Specifies Design2's top module (to be used when -netlist_2 is specified) (string, required)

read_hdl

```
read_hdl file_list
[-language {v2001 | v1995 | sv | vhdl }]
[-library library_name[=library_name2]...]
[-netlist] [-f filename]
[-define macro=value]... file_list.....
```

Loads one or more HDL files in the order given into memory. Files containing macro definitions should be loaded before the macros are used. Otherwise, there are no ordering of constraints.

If you do not specify either the v1995, v2001, sv or thevhdl option, the default language format is that specified by the hdl_language attribute. The default value for the hdl_language attribute is v2001.

The HDL files can contain structural code for combining lower level modules, behavioral design specifications, or RTL implementations. You can automatically read in or write out a compressed HDL file in gzip format.

When you load a parameterized Verilog module or VHDL architecture, each parameter in the module or architecture will be identified as an hdl_parameter object and located under ../architecture_name/parameters. The default hdl_parameter attribute value for these parameters will be true.

Parameters

- define macro=value	Defines a Verilog macro with the specified value, which is equivalent to the 'define macro value.  You can also define a macro definition list.
-f filename	Specifies the name of the list file for reading files from the simulation environment.  This option is not supported for VHDL designs.

<i>file_list</i>	Specifies the name of the HDL files to load. If several files must be loaded, specify them in a string.  The files can be encrypted. The host directory where the HDL files are looked for is specified via the <code>hdl_search_path</code> root attribute.
<code>-language</code>	Specifies the language of the HDL files. It can be any one of the following: <i>sv</i>, <i>v1995</i>, <i>v2001</i> (default) and <i>vhdl</i> . The <code>hdl_vhdl_read_version</code> root attribute value specifies the standard to which the VHDL files conform. Default: VHDL-1993
<code>-library library_name[=library_name2]...</code>	
	Specifies the name of the Verilog or VHDL library in which the definitions will be stored. A virtual directory with this name will be created in the <code>hdl_libraries</code> directory of the design hierarchy if it does not already exist. If you specify multiple libraries, they become multiple names (aliases) of one library. In this case, separate the names with the equal sign (=). Only one of the library names in the list becomes a virtual directory in the <code>hdl_libraries</code> directory. The library definitions remain in effect until elaboration, after which all library definitions are deleted. By specifying Verilog and VHDL library names, you can read in multiple Verilog modules and VHDL entities (and VHDL packages) with the same name without overwriting each other.

<code>-netlist</code>	Reads structural input files when parts of the input design is in the form of a structural netlist. You can read partially structural files provided the structural part of the input design is in the form of structural Verilog-1995 constructs and is contained in separate files from the non-structural (RTL) input.  If this option is specified, <code>-language</code> option is ignored.
-----------------------	---

Examples

- The following example first loads the `example1.v` file, then the `example2.v` file.

```
read_hdl {example1.v example2.v}
```

- The following commands with macro definitions are equivalent.

```
read_hdl -define "A B=4 C" example.v  
  
read_hdl -define A -define B=4 -define C ...
```

- The following command loads a single VHDL file and specifies a single VHDL library.

```
read_hdl -language vhd1 -library lib1 test1.vhd1
```

- The following commands read structural Verilog files when the design includes RTL (VHDL or Verilog) files.

```
read_hdl -language vhd1 file1_bh1.vhd1  
  
read_hdl -language sv file2_bh1.v  
  
read_hdl -netlist file3_str.v  
  
elaborate
```

- In the following command, the `-v1995` option is ignored. Both `rtl.v` and `struct.v` are parsed in the structural mode.

```
read_hdl -language v1995 rtl.v -netlist struct.v
```

- The following command defines VHDL libraries `lib1`, `lib2` as aliases for `lib3`.

```
read_hdl -language vhdl -library lib1=lib2=lib3 test1.vhdl
```

- The following commands read in two Verilog files that each contain a Verilog module with the same name (`compute`) but with different functionality. To store both definitions, the `-lib` option indicates in which library to store the definition.

```
read_hdl -language v2001 -library lib1 test_01_1.v  
read_hdl -language v2001 -library lib2 test_01_2.v
```

Inspection of the design hierarchy shows:

```
litmus> vls hdl_libraries  
/hdl_libraries:  
./          DP/          DW04/          GB/          STD/          lib2/  
AMBIT/      DW/          DW05/          GTECH/         SYNERGY/       synthetic/  
CADENCE/    DW01/         DW06/          IEEE/          SYNOPSIS/        
CW/         DW02/         DWARE/         IEEE_SYNERGY/  lib1/            
CWTECH      DW03/            
litmus> vls /hdl_libraries/lib1/architectures/  
hdl_lib:lib1.architectures:  
./          compute/  
litmus> vls /hdl_libraries/lib2/architectures/  
hdl_lib:lib2.architectures:  
./          compute/
```

read_libs

```
read_libs  
[library_files]  
[-aocv_libs aocvLibs]  
[-max_aocv <string>]  
[-max_libs maxLibs]  
[-max_noise_libs maxNoiseLibs]  
[-min_aocv <string>]  
[-min_libs minLibs]  
[-min_noise_libs minNoiseLibs]  
[-noise_libs noiseLibs]  
[-oa_ref_libs oaLibs]  
[-power_ground_files powerGridLibFiles]  
[-socv_libs soceLibs]  
[-typ <string>]
```

Schedules reading of one or more technology library files. These files either contain timing-related or noise-related information for the primitive cells used in the design.

Timing libraries can be in text format or Cadence binary library format (LDB). The LDB (library database) files generated from other tools, like Innovus, Voltus, or Genus are read in to Litmus with an error message (TECHLIB-1249). The LDB files are not forward compatible, that is, a LDB generated from 15.2 version cannot be used with 15.1 or prior versions of the software.

Note: You can use wildcards with the `read_libs` command as follows:

```
read_libs [glob *.lib]
```

Parameters

<code>-help</code>	Prints a brief description that includes the type and default information for each <code>read_libs</code> parameter.
<code>library_files</code>	Specifies to read the library files. Supported compressed file formats: <code>gzip</code> , <code>bz2</code> , <code>ldb</code>
<code>-aocv_libs aocvLibs</code>	Specifies to read the default AOCV libraries.
<code>-max_aocv string</code>	Specifies to read the max AOCV libraries.
<code>-max_libs maxLibs</code>	Specifies to read the max libraries.
<code>-max_noise_libs maxNoiseLibs</code>	Specifies to read the CDBs for max noise analysis.
<code>-min_aocv string</code>	Specifies to read the minimum AOCV libraries.
<code>-min_libs minLibs</code>	Specifies to read the min LIBs.
<code>-min_noise_libs minNoiseLibs</code>	Specifies to read the CDBs for min noise analysis.
<code>-noise_libs noiseLibs</code>	Specifies to read the default CDB files for noise analysis.
<code>-oa_ref_libs oaLibs</code>	Specifies to read the OA reference library.
<code>- power_ground_files powerGridLibFiles</code>	Specifies to read the power grid library files.
<code>-socv_libs socvLibs</code>	Specifies to read the default SOCV libraries.
<code>-typ string</code>	Specifies to read the common libraries.

read_lint_configuration

```
read_lint_configuration  
< <filename> | -default >  
[-version <version>]  
[-summary | -verbose]
```

Reads a configuration file for SDC lint checks. Only the `configure_lint_check` command should be used in this file.

To specify for which SDC version should the linting be configured, use the "-version" option of this command. The Tcl variable `::sdc::sdc_version_list` contains all the SDC version numbers supported by the tool. To apply the same configuration change file `updated_config.txt` to all supported versions, write:

```
tclmode  
foreach v $::sdc::sdc_version_list {  
    read_lint_configuration updated_config.txt -version $v  
}
```

Parameters

<filename>	Specifies the name of the configuration file to read.
-default	Specifies that tool's default configuration will be loaded.
-version <version>	Specifies the SDC version to which the configuration will be applied. If this is not specified, the configuration will be applied for version 2.0.
-summary	Reports only a summary of the configuration changes. This is the default.
-verbose	Reports details of all changes made by the specified configuration.

Examples

- The following command reads the lint configuration from file `config.txt` and applies it to version 2.0:

```
% read_lint_configuration config.txt
```

- The following command reads the lint configuration from file `config.txt`, applies it to version 1.6, and reports any changes:

```
% read_lint_configuration config.txt -version 1.6 -verbose
```

Related Commands

- `report_lint_configuration`
- `write_lint_configuration`

read_mmmc

```
read_mmmc  
[-help]  
file_name
```

Reads in the Multi-Mode, Multi-Corner (MMMC) file. MMMC refers to the full set of timing libraries, constraints, and the associated modes.

When the `read_mmmc` command is run, the MMMC file is read and processed. The majority of elements in the MMMC control file is saved in the MMMC caching attributes. Except for the library sets defined by the first `set_analysis_view` command, no data is loaded into the database. This command does basic syntax checking on the MMMC commands, rejecting commands that have incorrect options and commands that are not MMMC or tcl commands. However, no consistency checking is performed. For example, if a constraint mode is used in an analysis view, but is not defined, it will not be flagged.

The MMMC file contents are cached in attributes in the database. These attributes allow you to write out a full and complete MMMC file later in the flow. In addition, the names of the MMMC files are cached for querying.

The exceptions to data set load are the libraries pointed to by the first `set_analysis_view` command in the MMMC file. These library sets are loaded into the database. This load uses the standard library parser, and includes standard library syntax checking.

The `read_mmmc` command corresponds to the *Timing* step of the Initialization flow:



In the *Timing* step, the MMMC objects are set up by populating the basic MMMC information attributes. As Synthesis needs library information before the *Design* step, the *Timing* step also loads some library information. The library data to be loaded is pointed to by the first `set_analysis_view` command in the MMMC file.

See **Example** section for more details.

At a minimum, the MMMC file must contain one each of:

```
create_library_set
create_rc_corner
create_timing_condition
create_delay_corner
create_constraint_mode
create_analysis_view
set_analysis_view
```

If the MMMC file does not contain this minimum set, an error will be issued.

Note: The library sets can be specified with absolute paths in the MMMC, or relative to the local directory or to library search path attributes. The tool first attempts to load based on the name in the MMMC. If the library is not found, the tool sweeps through the search directory list in the specified order. Only the first hit is loaded.

Parameters

<code>-help</code>	Prints a brief description that includes type and default information for each <code>read_mmmc</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <pre>man read_mmmc</pre>
<code>file_name</code>	Specifies the name of the MMMC file.

Examples

When reading an MMMC file with `read_mmmc`, the tool sets the MMMC attributes with the data specified in the file. When the first `set_analysis_view` command is processed, the tool loads in the libraries specified by this analysis view. No other data is loaded.

For example, consider an MMMC file with the following commands:

```
create_library_set -name LS1 -timing [list lib1.lib lib2.lib lib3.lib]
create_library_set -name LS2 -timing [list lib4.lib lib5.lib lib6.lib]
create_library_set -name LS3 -timing [list lib7.lib lib8.lib lib9.lib]
create_rc_corner -name QX -qrc_tech file.qrc
create_timing_condition -name TC1 -library_set LS1
create_timing_condition -name TC2 -library_set LS2
create_timing_condition -name TC3 -library_set LS3
create_delay_corner -name DC1 -timing_condition {TC1 PD2@TC2} -rc_corner QX
create_delay_corner -name DC2 -timing_condition {TC1 PD2@TC3} -rc_corner QX
create_constraint_mode -name CM1 -sdc_files cond1.sdc
create_constraint_mode -name CM2 -sdc_files cond2.sdc
create_analysis_view -name AV1 -constraint_mode CM1 -delay_corner DC1
create_analysis_view -name AV2 -constraint_mode CM2 -delay_corner DC2
set_analysis_view -setup AV1 -hold AV2
```

When the `set_analysis_view` command is processed, the tool traces backwards from that command to determine the library set. In this case, the analysis view specified by `set_analysis_view` is AV1. The AV1 analysis view points to domain corner DC1, which in turn points to two library sets, LS1 and LS2. As a result, lib1.lib and lib2.lib are loaded into the database.

In this case, lib3.lib is not loaded into the database during `read_mmmc` because it is not in the library sets required by the analysis view.

Related Commands

- `init_design`
- `read_netlist`

read_netlist

```
read_netlist  
[-help]  
{{<in_file> | -oa_cell_view {lib cell view}} [-top top_module_name]}
```

Reads in a Verilog structural netlist. A structural Verilog file contains only structural Verilog-1995 constructs, such as module and gate instances, concurrent assignment statements, references to nets, bit-selects, part-selects, concatenations, and the unary ~ operator.

`read_netlist` also supports OpenAccess through the `-oa_cell_view` option.

The `read_netlist` command corresponds to the *Design* step of the Initialization flow:



During the *Design* step, the design is loaded into the database with `read_netlist`. After issuing this command, the database objects are populated and can be queried and manipulated (ungrouped, renamed, and so on). Full error checking is performed on the design details. This includes, but is not limited to, unresolved references, port mismatches, and empty modules.

Parameters

<code>-help</code>	Prints a brief description that includes the type and default information for each <code>read_netlist</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man read_netlist</code>.
<code><in_file></code>	Specifies the list of netlist files.
<code>-oa_cell_view {lib cell view}</code>	Specifies the name of the design object to be loaded in <code>{lib cell view}</code> format. No other options can be used if this option is specified.
<code>-top top_module_name</code>	Specified the top module name. By default, <code>read_netlist</code> identifies the highest module and links to that. If there are multiple top modules, the command returns an error.

Related Topics

- [init_design](#)
- [read_physical](#)

read_sdc

read_sdc
[-help]
[-reset]
sdc_files

Loads a timing constraints file. By default, the constraints files are loaded incrementally.

The constraints can be in the design constraints (SDC) format, or they can be more complicated Tcl constraints embedded in a Tcl script using commands such as [get_property](#) and [filter_collection](#).

A list of constraints files can also be specified in the configuration file.

Parameters

-help	Prints out the command usage.
-reset	Replaces all the previously loaded constraints information with the details specified in the specified SDC file.
sdc_files	Specifies the timing constraints file. Supported compressed file format: gzip .

Command Order

Use this command after importing the design and loading the timing library.

Examples

- The following command reads a SDC file called `dma_mac_m.sdc`:

```
% read_sdc dma_mac_m.sdc
```

Related Commands

- [write_sdc](#)

report_black_box

```
report_black_box
[-class <full | user | system | undefined | unsupported | empty | notranslate>]
[-detail]
[-module | -instance]
[-timing_arcs_missing]
```

Displays blackboxes from the design. The blackboxes might already exist in the design or might have been added previously with the `add_black_box` command.

Parameters

<code>-class</code>	Displays one of the following classes of blackboxes, where: • <code>full</code>: Blackboxes from both the user and system classes. This is the default. • <code>user</code>: Blackboxes added previously with the <code>add_black_box</code> command. • <code>system</code>: Blackboxes that are contained in the original design. • <code>undefined</code>: Blackboxes for undefined modules. • <code>unsupported</code>: Blackboxes for unsupported modules. • <code>empty</code>: Blackboxes for empty modules. • <code>notranslate</code>: Blackboxes for notranslate modules.
---------------------	---

-detail	Displays details about blackboxes, where: • <code>user</code>: Indicates that the blackbox was added by the <code>add_black_box</code> command. • <code>system(empty)</code>: Indicates that the module contains no logic. • <code>system(graybox)</code>: Indicates that a Liberty cell has an incomplete functional definition. The cell has undriven outputs and a tied output, but no well-functioning output pin. • <code>system(lef)</code>: Indicates that the macro is defined in the LEF file, but not defined in library/design modules. • <code>system(nottranslate)</code>: Indicates that the blackbox was added by the <code>add_nottranslate_modules</code> command. • <code>system(undefined)</code>: Indicates that the blackbox was added by the <code>set_undefined_cell_blackbox</code> command. • <code>system(unsupported)</code>: Indicates that the module contains unsupported statements.
-module	Reports only the blackbox modules. This is the default.
-instance	Reports the blackbox instances.
- timing_arcs_missing	Specifies that only blackboxed Liberty cells that have no timing arcs should be reported.

report_lint_configuration

```
report_lint_configuration  
[<command_name*>] ...  
[-all | changed | -unchanged]  
[-default]  
[-lint_rules]  
[-version <version>]
```

Reports the current lint configuration used by Litmus to perform SDC lint checks. This command can also be used to report all SDC lint checks that will be performed based on the current settings if an SDC file is read.

Parameters

<command_name*>	Reports only on the configuration of these selected SDC commands. This accepts wildcards.
-all	Report all attributes. This is the default.
-changed	Reports only attributes that have been changed.
-unchanged	Reports only attributes that have not been changed.
-default	Reports tool's default configuration for the specified version.
-lint_rules	Reports all SDC lint checks performed based on the selected configuration.
version <version>	Specifies the SDC version the lint configuration to be reported. If this is not specified, the configuration for version 2.0 is reported.

Examples

- The following command reports the default lint configuration for the `create_clock` SDC command in version 1.6:

```
% report_lint_configuration -default -version 1.6 create_clock
```

- The following command reports all lint rules checked based on the lint configuration for the `create_clock` SDC command in version 2.0:

```
% report_lint_configuration create_clock -lint_rules
```

Related Commands

- `read_lint_configuration`
- `write_lint_configuration`

report_sdc

```
report_sdc
[-help]
[-view <string>]
[-scope <string>]
[-verbose [-status <string>]]
[-sdc_compare]
```

Reports SDC parsing results and constraint lint rule results for the specified SDC commands. By default all the SDC commands are reported, and reporting consists of two sections : SDC parsing status and constraint lint rule violations.

Parameters

-help	Prints out the command usage.
-view <i>string</i>	Specifies the name of view.
-status <i>string</i>	Specifies rule status as pass or fail.
-scope <i>string</i>	Specifies the SDC commands that are associated with the scope to report.
-verbose	Provides additional information.
-sdc_compare	Provides SDC Compare results (bool, optional)

Command Order

Use this command after design initialization ([init_design](#)).

Examples

- The following command generates SDC command report for create_clock and create_generated_clock commands.

```
% report_sdc create*clock
```

- The following command generates SDC command report for the SDC commands that failed SDC parsing.

```
% report_sdc -status fail
```

Related Commands

- [read_sdc](#)
- [init_design](#)

write_lint_configuration

```
write_lint_configuration  
<filename>  
[<command_name* ...>]  
[-all | changed| -unchanged]  
[-default]  
[-replace]  
[-version <version>]
```

Writes out the current lint configuration used by Litmus to perform SDC lint checks.

Parameters

<filename>	Specifies the path of the file where the selected configuration should be written.
<command_name* ...>	Writes only the configuration of these selected SDC commands.
-all	Writes out all attributes. This is the default.
-changed	Writes out only attributes that have been changed.
-unchanged	Writes out only attributes that have not been changed.
-default	Writes out tool's default configuration for the specified version.
-replace	If the specified file already exists, this option deletes it and creates a new one.
-version <version>	Specifies the SDC version the lint configuration to be written. If this is not specified, the configuration for version 2.0 is written out.

Examples

- The following command writes the lint configuration for version 2.0 in file `config.txt`:

```
% write_lint_configuration config.txt
```

- The following command writes the default lint configuration for version 1.6 in file `config.txt`:

```
% write_lint_configuration -default -version 1.6 config.txt
```

Related Commands

- `read_lint_configuration`
- `report_lint_configuration`

Flowkit Commands

- [check_flow](#)
- [create_flow](#)
- [create_flow_step](#)
- [delete_flow_config](#)
- [edit_flow](#)
- [get_flow_config](#)
- [is_feature](#)
- [is_flow](#)
- [read_flow](#)
- [schedule_flow](#)
- [set_flow_config](#)
- [set_flowkit_read_db_args](#)
- [set_flowkit_write_db_args](#)
- [write_flow](#)

check_flow

```
check_flow  
[-help]  
[-fail string]  
[-flow string]  
[-from string]  
[-proc string]  
[-step string]  
[-to string]
```

Creates a single flow_step object and fills-in the required, and some optional attributes. Further

modifications of the step can be performed using the `set_db` command. If you try to define a step with the same name as an existing step, it results in error. To redefine a flow step, you must use the `set_db` command to modify the attributes, or delete and recreate it.

Note: Unlike Tcl procedures, flow steps do not take arguments and refer to attributes instead. The use of attributes requires that the data on which a step relies, is stored in the database and may be examined later.

Parameters

<code>-help</code>	Outputs a brief description that includes type and default information for each <code>check_flow</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man check_flow</code>.
<code>-fail string</code>	Reports a problem. this is not used for analysis. It can only be used from Tcl called as part of static checking. Calling from outside static checking results in error.
<code>-flow string</code>	Runs the specified flow. the <code>from/to</code> values are based on the flow steps. <i>Default:</i> <code>flow_step_order</code>
<code>-from string</code>	Specifies the name of the step to start the flow from. By default, the flow starts from the first step that has not been run, or the first step if all steps have been run previously. Note: This must be a step in the flow.
<code>-proc string</code>	Checks the specified procedure instead of analysing a flow step.
<code>-step string</code>	Runs just the specified step. This takes priority over <code>from/to</code> options.
<code>-to string</code>	Specifies the name of the step to run to (inclusive; in other words, the named step is executed). By default, runs to the end of the last step in the flow. Must be a step in the flow, later than the <code>-from</code> step (if any).

See also:

[Flow](#) chapter of the *Tempus Stylus Common UI Migration Guide* document.

create_flow

```
create_flow
[-help]
[steps]
-name name
[-owner string]
[-skip_metric]
[-tool tool]
[-tool_options options]
```

Creates a new, named flow object. `run_flow` then allows the execution of a named flow (in addition to its existing functionality).

Parameters

<code>-help</code>	Outputs a brief description that includes type and default information for each <code>create_flow</code> parameter. For a detailed description of the command and all of its parameters, use the man command: <code>man create_flow.</code>
<code>[steps]</code>	Specifies a list of flow step names in the order they must be executed. Note: This is an optional parameter. You can either provide <code>[steps]</code> or use the <code>-tool</code> parameter.
<code>-name <i>name</i></code>	The name of the flow object to create.
<code>-owner <i>string</i></code>	Specifies the owner of the flow.
<code>-skip_metric</code>	Restricts the tool from capturing the flow's metric snapshot.
<code>-tool <i>tool</i></code>	Specifies the name of the tool with which to run this flow. This option allows you to create flows for other tools than the one currently running. If not specified, default is the current tool.
<code>- tool_options <i>options</i></code>	Specifies options for use when launching this flow.

See also:

[Flow](#) chapter of the *Tempus Stylus Common UI Migration Guide* document.

create_flow_step

```
create_flow_step
[-help]
body
[-categories string]
[-check checkTcl]
[-description string]
[-exclude_time_metric]
-name name
[-owner string]
[-write_db]
[-skip_metric]
```

Creates a single `flow_step` object and fills-in the required, and some optional attributes. Further modifications of the step can be performed using the `set_db` command. If you try to define a step with the same name as an existing step, it results in error. To redefine a flow step, you must use the `set_db` command to modify the attributes, or delete and recreate it.

Note: Unlike Tcl procedures, flow steps do not take arguments and refer to attributes instead. The use of attributes requires that the data on which a step relies, is stored in the database and may be examined later.

Parameters

<code>-help</code>	Outputs a brief description that includes type and default information for each <code>create_flow_step</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <pre>man create_flow_step.</pre>
<code>body</code>	Specifies a block of Tcl that is run as the main action of the step.
<code>-categories string</code>	Defines the metric categories to calculate the flow step.
<code>-check checkTcl</code>	Specifies a block of Tcl to run step-specific static checks.
<code>-description string</code>	Describes the purpose for the current flow step.
<code>-exclude_time_metric</code>	Ensures that the CPU and wall time of the corresponding step is not included in any parent steps.
<code>-name name</code>	Specifies the name of the Tcl of the step. This can consist of alphanumeric characters and underscore only, and may not start with a digit. The following names have special meanings and may not be used for steps: • <code>next</code> • <code>end</code>
<code>-owner string</code>	Specifies the owner of the flow.
<code>-write_db</code>	Automatically saves a database when the flow step is complete.
<code>-skip_metric</code>	Skips generating metrics when the step is complete. By default, every step saves the design and runs metrics when completed.

See also:

[Flow](#) chapter of the *Tempus Stylus Common UI Migration Guide* document.

delete_flow_config

```
delete_flow_config  
[-help]  
key ...
```

Deletes the configuration value at the specified key, allowing the user to change the type of a `setup.yaml` key by first deleting it.

Parameters

<code>- help</code>	Outputs a brief description that includes type and default information for each <code>delete_flow_config</code> parameter. For a detailed description of the command and all of its parameters, use the man command: <code>man delete_flow_config</code>
<code>key ...</code>	Specifies the key to delete the configuration value.

See also:

[set_flow_config](#)

edit_flow

```
edit_flow  
[-help]  
[-after string]  
[-append string]  
[-before string]  
[-enabled_feature string]  
[-owner string]  
[-prepend string]  
[-reset]  
[-unique]
```

Allows you to insert flow steps into an existing flow without redefining the flow itself. This command is very useful for flow customization.

Parameters

<code>-help</code>	Outputs a brief description that includes type and default information for each <code>edit_flow</code> parameter.
<code>-after string</code>	Adds the specified <code>flow</code> and/or <code>flow_step</code> objects after the flow supporting a full canonical path to a flow step object.
<code>-append string</code>	Appends the specified <code>flow</code> and/or <code>flow_step</code> objects before or after the flow.
<code>-before string</code>	Adds the specified <code>flow</code> and/or <code>flow_step</code> objects before the flow supporting a full canonical path to a flow step object.
<code>-enabled_feature string</code>	Executes the <code>edit_flow</code> command only if the specified user-defined features are enabled.
<code>-owner string</code>	Specifies the owner of the flow being edited.
<code>-prepend string</code>	Adds the specified <code>flow</code> and/or <code>flow_step</code> objects before or after the flow.
<code>-reset</code>	Resets the customizations for the specified <code>flow</code> and/or <code>flow_step</code> objects before or after the flow.
<code>-unique</code>	Prevents the addition of the flow and/or <code>flow_step</code> objects, if those are already provided. This avoids duplication of the flow defined in the YAML file and the <code>edit_flow</code> statements scripted in the <code>tech_flow_steps</code> file. Note: This parameter prevents the specified <code>flow_step</code> from getting added to the same flow twice, irrespective of the values provided to the <code>-after</code> or <code>-before</code> parameters. For example, if <code>edit_flow -append bob -unique</code> is called twice to insert different <code>flow_steps</code> in the floorplan flow, it will add <code>bob</code> only once.

See also:

[Flow](#) chapter of the *Tempus Stylus Common UI Migration Guide* document.

Examples

- Following is an example of customizing the reference flow process using the `flow_config.template` file:

```
create_flow_step -name write_sdf {  
    write_sdf [get_db flow_report_directory]/[get_db local_flow].sdf  
}  
edit_flow -after flow_step:report_late_timing -append flow_step:write_sdf
```

- Following is an example of inserting an additional `step_2` step after the `step_1` step:

```
set file_dir [file dirname [ dict get [ info frame [ info frame ] ] file ] ]  
source ${file_dir}/scripts/run_flow.tcl  
create_flow_step -name step_2 {  
}  
edit_flow -append step_2 -after step_1
```

- Following is an example of adding flow customization to the `flow_config.tcl` file:

```
create_flow_step -name output_files {  
    set design_name [get_db current_design .name]  
    write_netlist [get_db flow_report_directory]/${design_name}.v  
    write_def [get_db flow_report_directory]/${design_name}.def  
}  
edit_flow -append flow_step:output_files -after flow_step:block_finish
```

get_flow_config

```
get_flow_config
[-help]
key ...
[-quiet]
```

Returns the feature configuration for a flow.

Parameters

-help	Outputs a brief description that includes type and default information for each <code>get_flow_config</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man get_flow_config</code>
key ...	Specifies the key to get configuration value.
-quiet	Suppresses errors in case the key does not exist.

See also:

[Flow](#) chapter of the *Tempus Stylus Common UI Migration Guide* document.

is_feature

```
is_feature  
[-help]  
[feature_name]  
[-obj string]
```

Tests if the specified flow feature is defined. The format of this command is similar to that of the `get_feature` command.

Parameters

<code>-help</code>	Outputs a brief description that includes type and default information for each <code>is_feature</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man is_feature</code>
<code>feature_name</code>	Specifies the name of the feature to be tested.
<code>-obj string</code>	Specifies if the <code>flow</code> or <code>flow_step</code> will be returned.

See also:

- `get_feature`
- [Flow](#) chapter of the *Tempus Stylus Common UI Migration Guide* document.

is_flow

```
is_flow
[-help]
[-after string]
[-before string]
[-hierarchical]
[-inside string]
[-quiet]
```

Allows you to query during the execution of a flow for further action.

Parameters

-help	Outputs a brief description that includes type and default information for each <code>is_flow</code> parameter.
-after <i>string</i>	Specifies the current execution after the <code>flow</code> or <code>flow_step</code> object.
-before <i>string</i>	Specifies the current execution before the <code>flow</code> or <code>flow_step</code> object.
-hierarchical	Displays all parent flows.
-inside <i>string</i>	Specifies the current execution within the <code>flow</code> object.
-quiet	Ensures that errors are not reported for missing <code>flow</code> or <code>flow_step</code> objects.

See also:

[Flow](#) chapter of the *Tempus Stylus Common UI Migration Guide* document.

read_flow

```
read_flow
[-help]
filename
[-no_check]
```

Loads the content for the current flow and debugs the flow in the tool. The command accepts single argument as the name of the files that describe the `flow` and `flow_step` objects, as shown below:

```
read_flow scripts/run_flow.tcl

read_flow scripts/flow.yaml
```

This command should not be used for YAML or TCL files, in general. For example, the following command should not be used, in which the `setup.yaml` file is used by the flow, but does not contain flow information:

```
read_flow scripts/setup.yaml
```

Parameters

-help	Outputs a brief description that includes type and default information for each <code>read_flow</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man read_flow.</code>
filename	Specifies the flow script to read.
-no_check	Skips running the <code>yamllint</code> and <code>check_tcl</code> flow checks.

See also:

Flow chapter of the *Tempus Stylus Common UI Migration Guide* document.

schedule_flow

```
schedule_flow  
[-help]  
[-branch string]  
[-db string]  
[-defer]  
[-directory string]  
[-flow string]  
[-include_in_metrics]  
[-no_db]  
[-sync | -no_sync]
```

Schedules a flow to be run after the current flow. The details of scheduled flows are stored in the `flow_schedule` attribute.

Parameters

<code>-help</code>	Outputs a brief description that includes type and default information for each <code>schedule_flow</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <pre>man schedule_flow.</pre>
<code>-branch string</code>	Specifies the branch for the new flow.
<code>-db string</code>	Specifies the starting database of the flow. If not supplied, the final database of the preceding flow (the one currently running) is used.
<code>-defer</code>	Prevents the scheduled flow from running, until requested.
<code>-directory string</code>	Specifies the working directory to use for the flow. By default, the current directory is used.
<code>-flow string</code>	Specifies the flow to be scheduled.
<code>-include_in_metrics</code>	Includes the flow in the current metric snapshot.
<code>-no_db</code>	When specified, flowtool does not use the database saved at the end of the step as the starting database for this flow.
<code>-no_sync</code>	When specified, flowtool does not wait for this flow to complete before continuing.
<code>-sync</code>	When specified, flowtool waits for all flows in the current branch to complete before starting.

See also:

[Flow](#) chapter of the *Tempus Stylus Common UI Migration Guide* document.

set_flow_config

```
set_flow_config
[-help]
key
value ...
```

Saves the feature configuration for a flow.

Parameters

-help	Outputs a brief description that includes type and default information for each <code>set_flow_config</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man set_flow_config</code>
key	Specifies the key to set configuration value.
value ...	Specifies the value to set configuration value.

See also:

[Flow](#) chapter of the *Tempus Stylus Common UI Migration Guide* document.

set_flowkit_read_db_args

```
set_flowkit_read_db_args
[-help]
[-dynamic_view analysis_view]
[-hold_views hold_views]
[-leakage_view analysis_view]
[-lef_files list_of_files]
[-setup_views setup_views]
[{-oa_lib_cell_view {libname cellname viewname}}] [{-no_timing}] [{-no_timing_graph}]
[{-mmmc_file fileName}]
```

Controls the run-time options used by `read_db` when a flow script is loading `flow_starting_db` specified by `flowtool` or used by `schedule_db`.

Note: This command supports all the parameters supported by `read_db`.

Parameters

<code>-help</code>	Prints a brief description that includes the type and default information for each <code>set_flowkit_read_db_args</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man set_flowkit_read_db_args</code>
<code>-dynamic_view <i>analysis_view</i></code>	Specifies the analysis view for the dynamic power analysis.
<code>-hold_views <i>hold_views</i></code>	Specifies the hold views for loading rather than the active hold views.
<code>-leakage_view <i>analysis_view</i></code>	Specifies the analysis view for the leakage power analysis.

<code>-lef_files list_of_files</code>	Specifies the list of LEF files for loading. Use this option if you want to override the existing LEF files in the design being restored and load different LEF files instead. • This option is intended to allow upgrading to new versions of LEF files that are compatible with the old versions. The list of all LEF files used by this design must be provided. • If any pin locations, cell boundaries, or OBS shapes are changed for used cells, new DRC violations may occur. • If pins are added or removed from used cells, or used cells are removed, fatal errors may occur while loading the netlist. • If the removed pins and cells are unused in the design, warnings or errors may still occur, if any library-specific data was stored for those cells or pins during the design flow (such as <code>dont_touch</code> data).
<code>-mmm_file fileName</code>	Specifies the path name to a Multi-Mode/Multi-Corner (MMMC) configuration file (for example, the same file format used by <code>read_mmmc</code>) to replace the MMMC file saved by <code>write_db</code> (and all the corresponding .lib files, .sdc files, etc. referenced by the MMMC file).

<code>-no_timing</code>	Skips reading of the MMMC file, timing libraries and timing constraints. Note: • Most designs that use LEF rely on the .lib information to define the bus order for MACROs that have bus bit terminals. Therefore, using the <code>-no_timing</code> parameter can result in the connectivity to bus ports of leaf level instances being reversed. When .lib information is not available, the default is for bus ports to be treated as descending (for example, [7:0]). While using Verilog modules or OpenAccess abstract views to define the bus order, the connectivity will still be correct even when <code>-no_timing</code> is specified. • Timing libraries cannot be loaded in the same session after a design is read using the <code>-no_timing</code> option. You must use the <code>set_flowkit_read_db_args</code> command without the <code>-no_timing</code> parameter in a new session to load the design with timing libraries and constraints. • If you save a design after <code>set_flowkit_read_db_args -no_timing</code>, it will not save timing information unless you use <code>set_flowkit_write_db_args -add_ignored_timing</code>.
<code>-no_timing_graph</code>	Skips reading the timing graph even if it was saved to the disk by <code>set_flowkit_write_db_args</code>.
<code>-oa_lib_cell_view {libname cellname viewname}</code>	Specifies the library name, cell name, and view name for reading an OpenAccess DB design.
<code>-setup_views setup_views</code>	Specifies the setup views for loading rather than the setup views active during <code>set_flowkit_write_db_args</code>.

Related Topics

- [read_db](#)
- [set_flowkit_write_db_args](#)
- [Flow](#) chapter of the *Tempus Stylus Common UI Migration Guide* document.

set_flowkit_write_db_args

```
set_flowkit_write_db_args
[-help]
[-add_ignored_timing]
[-def]
[-lib]
[-no_constraint]
[-no_fill]
[-no_pvs_fill]
[-no_wait string]
[-sdc]
[-user_path]
[-verilog]
[{-rc_extract}] [{-timing_graph}] [{-oa_lib_cell_view string}] [{-oa_view string}] [{-lib_to_ldb}]
```

Allows you to save databases through flowkit in the Open Access format and specify the library/cell/view information. The `set_flowkit_write_db_args` command can be used to extend the options used to save a design on flow-steps, which are marked with `skip_db false`. This command needs to be placed inside the `flowstep body_tcl` of interest since it only affects that `flowstep` and resets back to default state when that step completes. The format of this command is similar to what would be used for `write_design`:

```
create_flow_step -name save {set_flowkit_write_db_args -rc -lib2ldb}
```

This saves a database named `save`, which uses the `-rc` and `-lib2ldb` options for `write_db`. Previous and subsequent flowsteps use default `write_design` options unless overridden using the `set_flowkit_write_db_args` command.

Note: This command supports all the parameters supported by `write_db`.

Parameters

-help	Outputs a brief description that includes type and default information for each <code>set_flowkit_write_db_args</code> parameter.
-------	---

-add_ignored_timing	Copies the old timing data when loaded in physical-only mode. If you use <code>set_flowkit_read_db_args -no_timing</code> to read the DB, the timing data is not loaded. This option allows you to copy the ignored timing data from the original <code>read_db</code> location and add it back during <code>set_flowkit_write_db_args</code> like this: <pre>set_flowkit_read_db_args -no_timing test1 <physical editing commands ...> set_flowkit_write_db_args -add_ignored_timing test2</pre> This usage model is intended to allow faster physical-only editing, like DRC fixing, where the netlist is not modified. If you change the netlist, unpredictable results may occur when reading the <code>test2</code> DB using timing data copied from <code>test1</code>.
-def	Saves a DEF file, in addition to the normal DB files in the DB directory. This parameter is normally not used for flat or block flows and is not used by <code>set_flowkit_read_db_args</code>. It can be useful in a hierarchical flow, where you want to use <code>assemble_design</code> to read only the netlist and the DEF files without opening up each design DB and writing out the Verilog and DEF files separately (see <code>-verilog</code>).
-lib	Resolves symbolic links of the timing lib files and side files. Normally external file references, such as to the LEF, .lib, SDC, files are saved using full path-name, symbolic links to the original files inside a libs sub-directory inside the DB directory. This parameter forces copies of the files rather than symbolic links in the libs sub-directory so there are no external file references. That makes the DB directory much larger, and makes it suitable for sending to a different network where the library and external data files are not available. An OA DB will still have OA library references using the normal <code>cds.lib</code> methodology, but any other external file references will be copied inside the cellview directory.

<code>-lib_to_ldb</code>	Converts the <code>.lib</code> timing libraries referenced in the MMMC file to the <code>.ldb</code> format and writes them in a <code>libs</code> sub-directory inside the output design directory. References to the original <code>.lib</code> files in the MMMC file are changed to the corresponding <code>.ldb</code> files inside the <code>libs</code> sub-directory. This makes the design directory much larger because it has all the timing library data inside it and is not recommended for normal flows. Normally, you should setup <code>.ldb</code> files in an external library directory and have the MMMC files use them directly to avoid this overhead. Using <code>.ldb</code> files will make <code>set_flowkit_read_db_args</code> faster when it reads the library data, so this option is useful for doing “throw-away” debugging experiments that require repeated sessions at the cost of a one-time conversion, and carrying the library data with the design.
<code>-no_constraint</code>	Saves the timing graph data without the SDC constraints data. This parameter requires the <code>-timing_graph</code> parameter and is useful in reducing the disk size, and speeding up <code>set_flowkit_read_db_args</code> and <code>set_flowkit_write_db_args</code>. However, the timing-graph data cannot be loaded by any other release version. Additionally, the timing constraints would be lost without the SDC files, if loaded by a different release version. Note: You can still load the timing-graph with the current release version, and then use <code>set_flowkit_write_db_args</code> without the <code>-no_constraint</code> parameter to regenerate the SDC files from the timing-graph and then use <code>set_flowkit_read_db_args</code> with a later release.
<code>-no_fill</code>	Prevents writing of any metal or via fill shapes inside the database. This is useful during the ECO routing, since the fill data is regenerated anyway. This way, it is not worth saving the current fill data. All the fill data that would go to DEF FILLS section (floating fill data), or the DEF SPECIALNETS section with <code>+ SHAPE FILLWIRE</code> or <code>FILLWIREOPC</code> will be ignored and not written.

<code>-no_pvs_fill</code>	Prevents saving the PVS fill data file. In some PVS flows, a stream (GDS) file from PVS is saved inside the software DB directory. During <code>set_flowkit_read_db_args</code> , this file is not read, but can be optionally loaded or used later in the flow. By default, if the PVS fill file exists, <code>set_flowkit_write_db_args</code> will copy it from the <code>set_flowkit_read_db_args</code> location to the new DB directory. If ECO changes to the current design have made the PVS fill file invalid, use this parameter to stop it from being copied.
<code>-no_wait string</code>	Writes the design in a separate process. You can continue executing new commands simultaneously while the design is being saved. This is useful when you want to save intermediate results without waiting for the save to complete.
<code>-oa_lib_cell_view string</code>	Saves the database to an Open Access library cell view.
<code>-oa_view</code>	Specifies the Open Access view name.
<code>-rc_extract</code>	Writes RC extraction data, including data for incremental extraction. By default, the software does not automatically save RC extraction data to save disk space and run-time. The <code>-rc_extract</code> parameter does not save RC extraction data if the design is not extracted or if <code>extract_rc_engine = pre_route</code> .
<code>-sdc</code>	Saves the timing constraint file even if it is not modified.
<code>-timing_graph</code>	Saves the timing graph data.
<code>-user_path</code>	Saves external file references exactly same as the user entered it.
<code>-verilog</code>	Saves a Verilog file inside the DB directory, in addition to the normal DB files. This parameter is not normally used for flat or block flows. It can be used in a hierarchical flow, where you want to use <code>assemble_design</code> to read only the Verilog and the DEF files without opening up each design DB and writing out the Verilog and DEF files separately.

References

- `write_db`
- `set_flowkit_read_db_args`
- [Flow](#) chapter of the *Tempus Stylus Common UI Migration Guide* document.

write_flow

```
write_flow
[-help]
[-directory string]
[-out_file string]
[-flow_tcl]
[-flow_yaml]
[-include_state]
[-setup_yaml]
```

Generates a Tcl script that may be sourced to reproduce the current flow. The script includes commands to:

- Set the value of all top-level attributes with non-default values to their current values
- Recreate all flow step objects with their current attributes, aside from those relating to run history (specifically, `run_count` and `status`).

Sourcing this script reproduces (but not runs) the current flow in a fresh session, and forms a user readable and editable record of the flow (compared to the flow data saved in the DB, which should not be read or edited directly).

Parameters

-help	Outputs a brief description that includes type and default information for each <code>write_flow</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man write_flow</code> .
- directory <i>string</i>	Specifies the directory in which all the files need to be written.
-out_file <i>string</i>	Specifies the name of the output file to generate.
-flow_tcl	Writes only the <code>flow.tcl</code> file in the directory.
-flow_yaml	Writes only the <code>flow.yaml</code> file in the directory.
-include_state	Writes out the current flow state. This can be used to restore the flow.
-setup_yaml	Writes only the <code>setup.yaml</code> file in the directory.

Related Topics

- [write_flow_template](#)
- [Flow](#) chapter of the *Tempus Stylus Common UI Migration Guide* document.

Functional Check Commands

- [set_sva_to_skip](#)
- [set_sva_to_generate](#)
- [generate_sva](#)

set_sva_to_skip

```
set_sva_to_skip  
[-help]  
[-file <string>]  
[-start_line <integer>]  
[-end_line <integer>]  
[-reset ]
```

This command is used to specify the file name and line number of the constraint that the SVA isn't to be generated. By default no lines are skipped.

Parameters

-help	Prints out the command usage.
-end_line <integer>	Ending line number of timing constraints to skip assertions (int, optional)
-file <string>	SDC file specification: <file_name> (with -start_line/-end_line) or <file_name>:<line_no> (single constraint) (string, optional)
-reset	Reset the previous set_sva_to_generate settings (bool, optional)
-start_line <integer>	Starting line number of timing constraints to skip assertions (int, optional)

set_sva_to_generate

```
set_sva_to_generate  
[-help]  
[-file <string>]  
[-start_line <integer>]  
[-end_line <integer>]  
[-reset ]
```

This command is used to specify the file name and line number of the constraint that the SVA is to be generated. By default SVAs are generated for all constraints.

Parameters

-help	Prints out the command usage.
-end_line <integer>	Ending line number of timing constraints to generate assertions (int, optional)
-file <string>	SDC file specification: <file_name> (with -start_line/-end_line) or <file_name>:<line_no> (single constraint) (string, optional)
-reset	Reset previous set_sva_to_skip settings (bool, optional)
-start_line <integer>	Starting line number of timing constraints to generate assertions (int, optional)

generate_sva

```
generate_sva  
[-category_comment_keyword string]  
[-category_file string]  
[-cgc]  
[-mcp]  
[-output_path output_path]  
[-replace]  
[-sca]  
[-sdt]  
[-sfp]  
[-use_category]  
[-include_clock_to_clock]  
[-help]
```

Generates sfp.sv, mcp.sv, cgc.sv, macro_lib.sv, "top_degisn_name"_defines.h, and a "top_design_name".summary log file.

Parameters

- category_comment_keyword	Specify the keyword to identify category tag in SDC comment. For example, If SDC has set_false_path -comment {foo**SVA FP_01 **SVA bar}, keyword is **SVA (string, optional)
-category_file	Specify the file containing categorization of SFPs.
-cgc	Write verilog testbench to verify create_generated_clock
-mcp	Writes verilog testbench to verify set_multicycle_path.
-output_path output_path	Specify the output path.
-replace	Replaces the original SVA (.sv) files if they exist
-sca	Writes verilog testbench to verify set_case_analysis.
-sdt	Writes verilog testbench to verify set_disable_timing.
-sfp	Writes verilog testbench to verify set_false_path.
-use_category	Generates custom assertions based on category of SFP define in file specified by -category_file or -comment in SDC.
-include_clock_to_clock	Include MCPs & FPs defined with -from clock -to clock (bool, optional)
-help	Prints out the command usage.

General Commands

- [add_rule_filter](#)
- [alias](#)
- [check_ccr](#)
- [check_netlist](#)
- [convert_legacy_to_common_ui](#)
- [create_snapshot](#)
- [decrypt](#)
- [define_proc](#)
- [delete_rule_filter](#)
- [elaborate_design](#)
- [encrypt](#)
- [eval_legacy](#)
- [get_license_status](#)
- [get_message](#)
- [help](#)
- [input_libs](#)
- [input_sdc](#)
- [is_common_ui_mode](#)
- [litmus_setup_mmmc](#)
- [print_message](#)
- [read_db](#)
- [read_metric_config](#)
- [read_metric_html_package](#)

- [redirect](#)
- [rename_obj](#)
- [rename_snapshot](#)
- [report_dangling_ports](#)
- [report_inactive_arcs](#)
- [report_messages](#)
- [report_netlist_statistics](#)
- [report_obj](#)
- [report_rule_check](#)
- [report_rule_filter](#)
- [report_rule_instance](#)
- [report_rule_source](#)
- [resume](#)
- [set_compress_level](#)
- [set_expanded_hinsts](#)
- [set_license_check](#)
- [set_message](#)
- [set_rule_handling](#)
- [source](#)
- [suspend](#)
- [time_info](#)
- [write_db](#)
- [write_metric_config](#)

add_rule_filter

```
add_rule_filter
[-help]
filter_name
[-description string]
[-filter string [-regex]]
[-object_filter object_names]
[-rule string]
[-replace]
```

Adds a new rule filter that can be applied to any number of rule objects. Rule filters are used to select which occurrences should be ignored by commands such as [report_rule_check](#).

Parameters

-help	Prints out the command usage.
filter_name	Specify the name of the rule filter to add or apply.
-description string	Specify the description for the added rule filter.
-filter filter_expression	The rule filter will apply to the rule occurrences for which the specified <i>filter_expression</i> evaluates to true. The syntax of the <i>filter_expression</i> must follow the format "<attribute_name> <operator> <value>". The operator can be ==, !=, >=, >, <=, <, =~, and !~. Multiple conditions can be combined with the logical operators !, &&, , and parentheses ().
-regex	Specifies that the pattern matching operators used inside <i>filter_expression</i> (such as =~) should use regular expressions as patterns, rather than glob-style patterns.
-object_filter object_names	Specifies the objects which will be filtered out from the rule occurrences. Supported rules: CONST_CLK_DEF1, CONST_CLK_DEF6.
-rule string	Specify the description for the added rule filter.
-replace	Deletes and creates a new file if the specified file already exists.

Examples

- This command adds the rule filter `occurrence_filter_1` that filters out a rule occurrence if its witness contains `my_reg`. The rule filter is applied to `CONST_CLK_DEF1` rule.

```
% add_rule_filter occurrence_filter_1 -filter "witness =~ *my_reg*" -rule  
CONST_CLK_DEF1
```

- This command adds the rule filter `object_filter_1` that filters out `my_reg` from a rule occurrence. Then the rule filter is applied to `CONST_CLK_DEF1` and `CONST_CLK_DEF6` rules.

```
% add_rule_filter object_filter_1 -object_filter my_reg  
% add_rule_filter object_filter_1 -rule CONST_CLK_DEF1  
% add_rule_filter object_filter_1 -rule CONST_CLK_DEF6
```

Related Commands

- `delete_rule_filter`
- `report_rule_filter`

alias

```
alias
[-help]
new_cmd
old_cmd
[-args {{new old} ...}]
```

Creates command name aliases for Tcl commands, except built-in Tcl commands. This command supports argument name aliases for application commands and procs that use `define_proc` and `define_variables` commands for parsing the arguments.

The argument aliases are private, so these do not display with `-help`, and also do not support partial option completion or matching. However, the command aliases are public, so `<tab>` complete works.

You can use this command to migrate an existing Tcl script when some of the commands or argument names have changed.

Parameters

<code>-help</code>	Prints a brief description that includes type and default information for each <code>alias</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man alias</code>
<code>new_cmd</code>	Specifies the new command name that is an alias for the existing <code>old_cmd</code> name. It can be the same name as the <code>old_cmd</code> if you only want to create new <code>-arg</code> aliases, and do not need a new command alias.
<code>old_cmd</code>	Specifies the existing old command name.
<code>-args</code> <code>{{new</code> <code>old} ...}</code>	Specifies a list of new and old argument names to create aliases. You can give multiple <code>new</code> names for the same <code>old</code> name. The new aliases are private, and do not show up with <code>-help</code> . You can create more than one alias for the same <code>old</code> name.

Example

The following command adds a new command name alias `new_read_sdf` and some argument name aliases for `read_sdf`:

```
>alias new_read_sdf read_sdf -args {  
    {-ignore_error -continue_on_error}  
    {-summary -report_summary}  
    {-sum -report_summary}  
}
```

Then, the following commands

```
>new_read_sdf in.sdf -ignore_error -summary  
>new_read_sdf in.sdf -ignore_error -sum  
>new_read_s in.sdf -ignore_error -sum
```

would behave the same as

```
>read_sdf in.sdf -continue_on_error -report_summary
```

Note: Tcl partial command matching works if the partial command is unique as shown in the above example, but the alias argument names are private, and will not support partial option matching, or appear in the `-help` output.

Related Commands

- `define_proc`
- `define_variables`

check_ccr

```
check_ccr  
[-help]  
ccr_number
```

Returns 1 if the specified CCR is integrated into this build and 0 if it is not.

- It includes all integrated CCRs that have the `Checked_In` or `Validated` status at the time of the build.
- It only includes master CCRs. Duplicate CCRs are not included.
- It currently lists all CCRs fixed in the current release stream or previous release. For example, checking in release version 16.22, will find all CCRs integrated into 16.10 through 16.22 that are in the current 16.22 release.

Parameters

<code>-help</code>	Prints a brief description that includes type and default information for each <code>check_ccr</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <pre>man check_ccr</pre>
<code>ccr_number</code>	Specifies the CCR number to check.

Examples

- In the example below, the return value "1" indicates that the CCR 1234567 is integrated into this build.

```
litmus> check_ccr 1234567  
  
1
```

check_netlist

```
check_netlist
-out_file filename
[-sub_module_ports]
```

Performs several checks on the design netlist and creates a report, named `checknetlist.rpt`. This report shows the following information:

- Design Summary
- Design Statistics
- I/O Pad Summary
- Design Rule Checking
- Submodule Port Definition Checking (optional)

The relevant information can be found in the related checklist directory.

You can use this command after the design netlist is imported.

Parameters

<code>-out_file</code> <i>filename</i>	Specifies the name of the file where the report will be written. If a name is not specified, the default filename, <code>checknetlist.rpt</code> , is used.
<code>-</code> <code>sub_module_ports</code>	If this option is included, <code>check_netlist</code> checks the port definitions for any submodules.

Example

- The error conditions are indicated by an asterisk (*) preceding the reported statistic, such as:

```
Number of Ports Connect to Core Cells *: 57
Number of Ports Connect to multiple Pads *: 0
Number of Floating Ports *: 0
```

- The warnings, or possible errors are indicated by a question mark in front of the statistic, such as:

Number of undefined Floating Pins ?: 0

- The following example shows the information contained in the generated by `check_netlist` report:

```
Design: TOPModule
```

```
----- Design Summary:
```

```
Total Standard Cell Number (cells) : 5908
```

```
Total Block Cell Number (cells) : 4
```

```
Total I/O Pad Cell Number (cells) : 0
```

```
Total Standard Cell Area ( um^2) : 133864.32
```

```
Total Block Cell Area ( um^2) : 366346.56
```

```
Total I/O Pad Cell Area ( um^2) : 0.00
```

```
----- Design Statistics:
```

```
Number of Instatnces : 5912
```

```
Number of Nets : 6217
```

```
Average number of Pins per Net : 3.69
```

```
Maximum number of Pins in Net : 541
```

```
----- I/O Pad summary
```

```
Number of Primary I/O Ports : 57
```

```
Number of Input Ports : 28
```

```
Number of Output Ports : 29
```

```
Number of Bidirectional Ports : 0
```

```
Number of Power/Ground Ports : 0
```

```
Number of Ports Connect to Core Cells *: 57
```

```
Number of Ports Connect to multiple Pads *: 0
```

```
Number of Floating Ports *: 0
```

```
----- Design Rule Checking:
```

Number of Output Pins connect to Power/Ground *: 0

Number of Input Floating Pins *: 1

Number of Output Floating Pins : 245

Number of InOut Floating Pins : 0

Number of UniPort Floating Pins : 0

Number of undefined Floating Pins ?: 0

Number of Multiple Driving Nets *: 1

Number of Parallel Driving Nets : 1

Number of Tristate Driving Nets : 0

Number of Input Floating Nets(No FanIn) *: 0

Number of Output Floating Nets(No FanOut) : 1

Number of Hign Fanout Nets (>50) : 3

----- Sub Module Port Definition Checking

* module arb has 3 port define error

* module accum_stat has 1 port define error

* module tdsp_core_glue has 128 port define error

convert_legacy_to_common_ui

```
convert_legacy_to_common_ui  
[-help]  
in_dir_or_file  
out_dir_or_file
```

Allows usage of legacy scripts to be used in Stylus Common UI.

Parameters

<code>-help</code>	Prints out the command usage.
<code><i>in_dir_or_file</i></code>	Specifies the name of the legacy script or script directory name.
<code><i>out_dir_or_file</i></code>	Specifies the Stylus Common UI script or the script directory name.

create_snapshot

```
create_snapshot  
-help  
[pattern]  
[-auto {min default}]  
[-categories [* flow design setup power check clock hold route]]  
[-exclude_time_metric]  
-name metric_name
```

Creates snapshots of the current metric list at various points in the flow.

All super commands that save metrics call `create_snapshot` to save all metrics in a consistent manner. When `create_snapshot` is called, it creates an internal record designated by a label. The label for super commands are the command names.

Metrics that do not require extraction and/or timing/power analysis are calculated when the metric snapshot is saved. Timing/clock/power metrics are reported only if they are up-to-date(i.e. if `time_design/report_ccopt*/report_power` have been run). Metrics are saved as part of db and can be accessed using [get_db](#).

Note: Metric values should be saved such that the unit is easily known/accessible.

Parameters

<code>-auto {min default}</code>	Specifies the set of auto metrics to run.
<code>-categories [* flow design setup power check clock hold route]</code>	Specifies the metric categories to compute.
<code>-exclude_time_metric</code>	Excludes CPU and wall time from parent snapshot calculations.
<code>-name <i>metric_name</i></code>	Specifies the name of the metric.
<code>-help</code>	Outputs a brief description that includes type and default information for each <code>set_metric</code> parameter. For a detailed description of the command and all of its parameters, use the man command: <code>man set_metric</code>
<code><i>pattern</i></code>	Specifies specific metrics to save.

Example

The following code shows the manual usage of `create_snapshot`:

```
# -----
# Manual Usage:
# -----

opt_design -preCTS
report_power ...
create_snapshot -label prects
csopt_design
report_power
create_snapshot -label cts
add_filler
route_design
create_snapshot -label route ; # Timing and power are not up-to-date
opt_design -postRoute
report_power
create_snapshot -label postroute
```

The following code shows the automatic/mixed usage of `create_snapshot`:

```
# -----  
# Automatic / Mixed Usage:  
# -----  
  
setDesignMode -enableMetrics true  
opt_design -preCTS ; # Saves snapshot "opt_design_<count>"  
report_power  
create_snapshot -name "prects"  
ccopt_design ; # Saves snapshot "ccopt_design"  
report_power  
create_snapshot -name "postcts" ; # Includes up-to-date power metrics  
add_filler  
route_design ; # Saves snapshot "route_design"  
opt_design -postRoute ; # Saves snapshot "opt_design_<count>"  
report_power  
create_snapshot -name "postroute" ; # Includes up-to-date power metrics
```

<http://rdwiki.cadence.com/display/ssvpubsmgmt/Parameters+not+Documented+-+16.20>

decrypt

```
decrypt  
[-help]  
file_name
```

Decrypts and evaluates a Tcl file that was encrypted with the encrypt command.

Parameters

<code>-help</code>	Prints out the command usage.
<code><i>file_name</i></code>	Specifies the name of the Tcl file to be decrypted and evaluated. <i>Data_type</i> : string, required

Related Topics

- [encrypt](#)

define_proc

```
define_proc
[-help]
proc_name
arg_list
proc_body
[-description proc_help]
[-hidden]
```

This command defines a Tcl proc that uses an extended argument syntax to check for legal argument names and legal values. It is a super-set of the functionality supported by `define_proc_arguments`, and `parse_proc_arguments` and easier to use.

The arguments are parsed, and if any errors occur, an error message is written, and a `TCL_ERROR` is returned, so the `proc_body` is not executed. If the arguments are parsed successfully the Tcl vars are set accordingly, and the `proc_body` is executed.

It supports simple types like int, string, etc., built-in types like point, rect, and objects like inst, net, pin.

Commands defined with `define_proc` also have the same support as other Litmus commands, including:

- help support (e.g. "`cmd -help`" to see the options, and "`help c*`" would find the command named `cmd`, etc.)
- tab-completion of partial command or option names (e.g. "`cmd<tab>`" or "`cmd -opt<tab>`")
- partial argument name matching is allowed (e.g. "`cmd -opt`" will work if `-opt` only matches one option name prefix like `-opt_test`)
- redirect output to files or Tcl vars using `>` or `>>` (e.g. "`cmd > my_file`", or "`cmd > &my_tcl_var`")

Parameters

<code>arg_list</code>	Defines the argument list of the Tcl procedure in the following format for each argument: <pre>{var default arg_name data_type flags help}</pre> For example <pre>{report_file report.out {-out_file file_name} string optional {The output file for the report} }</pre> In this format:
-----------------------	---

- *var* is the local Tcl variable to assign, such as `report_file` in the example above. The value for the argument is assigned to this variable before the body of `define_proc` is invoked.
- *default* is the default value, such as `report.out` in the example above. If the argument is not given, then *var* will be assigned the *default* value. Note, unlike an argument value passed to the command, the type of the default value is not checked.
For example, the following command would set `int_var` to `"no_value"` if `-my_int` is not given. Normally the default value should be a legal int value, but you can use this to check if the argument was given for an optional argument when there is no good default value.

```
{int_var no_value {-my_int int} int optional {int value help  
description}}
```

The default value is ignored if the argument is a required argument (see *flags* below).

- *arg_name* is the argument name with optional value help. Both named `{-opt value}` and unnamed `{value}` arguments are supported.

If the first word starts with a dash, it is a named argument with optional help string. For example, `{-out_file file_name}` will appear in the help output as:
`-out_file <file_name>.`

If no help string is given like this:
`{-out_file}`

The help output will include the *data_type* like: `-out_file <string>.`

If the first word does not start with a dash, it is an unnamed argument and the first word is used as the help string. For example, `{out_file}` will appear in the help output as: `<out_file>.`

- *data_type* is the argument data type. Many types are supported including simple types, bool type, enum type, built-in types, and object types.
 - These simple types are supported: **int**, **float**, **double**, **string**. They all require an argument value that matches the type (e.g. `-my_int 23`, or `-my_string "my test string"`). **float** and **double** are both allowed, and mean the same thing. For example

```
{int_var 0 {-my_int int} int optional {int value help description}}
```

Note that a integer passed to a float or double type will get a .0 added to the end so it can be passed to expr and forced to use floating point arithmetic.

- The **bool** type supports an argument with or without a value. If the *arg_name* definition has a value help string, then a bool value is required, otherwise a value is not allowed. So, a switch that takes no value would look like:

```
{switch_var false -my_switch bool optional "-my_switch help  
description"}
```

and would be used like this with no value

```
-my_switch
```

The *switch_var* will be set to true if *-my_switch* is given, and set to the default value of false if not given.

A bool with a value required, would look like this

```
{bool_var false {-my_bool bool} bool optional "bool with value help  
description"}
```

and would be used like this

```
-my_bool true
```

Any Tcl allowed bool value can be used for the input value (e.g. true, false, 1, 0, on, off, etc.) but the *bool_var* will always have either true, false or the *default* value if the argument is not given.

- These built-in types are supported: **point**, **rect**.
 - **point** is exactly two doubles. The input can be either a single list or list of lists style, so both {1.0 2.0} or {{1.0 2.0}} are allowed as input. The *point_var* will always be assigned as a simple list of two values for either type of input. An integer value gets a .0 added, so it can be used in a Tcl expr and forced to be treated as a double. So an input like {1 2} will be assigned to the *point_var* as {1.0 2.0}.
 - **rect** is exactly four doubles. It can be in a single list or list of lists style, so {0 1 2 3}, {{0 1} {2 3}} or {{0 1 2 3}} are allowed. The *rect_var* will always be assigned as a simple list of four values.
- An **enum** type must include the list of legal enum values like this {**enum** val1 val2 ...}. It supports an optional *list_type* of * (a list of 0 or more) or + (a list of 1 or more) as the last enum value. It allows an optional {**hidden** val1 val2 ...} list for hidden enum values. The enum value inputs are case-insensitive, but the return value assigned to *var* is always forced to be all lower-case

characters.

For example,

```
{sort_var name {-sort} {enum name age {hidden city state} +}  
optional {help description}}
```

would allow `-sort age`, or `-sort "name CITY"`, but would give errors for `-sort ""`, or `-sort abc`.

The help string will show the enum values but not the hidden enum values like this:

```
-sort {name age} # help description (enum+, optional)
```

- An **object** type allows object names and pointers to be passed in as values like this `{object obj_type1 [obj_type2 ...]/[list_type]}`. It supports an optional `list_type` of `*` (a list of 0 or more) or `+` (a list of 1 or more) as the last value. If `list_type` is not given, exactly one value is required for input. For example,

```
{driver_var {} {-drivers} {object pin port *} optional {help  
description}}
```

would allow a list of 0 or more pin or port objects for input. The input could be a list of pin or port pointers, or a name-pattern like `"i1/p"` that allows `*` and `?` pattern matching. `driver_var` will be assigned a list of pointers of the correct type (or types). So both of these uses would have the same result:

```
-drivers [concat [get_db pins i1/p*] [get_db ports i1/p*]]
```

```
-drivers i1/p*
```

In both cases, all the pins and ports that match `i1/p*` would be returned as a list of objects and assigned to `driver_var`.

If any objects are passed in that are not pin or port objects, an error will occur during parsing.

If at least one pointer is required (e.g. `list_type +` or no `list_type`), and none are matched by the name pattern, an error will occur during parsing.

Design objects like **hinst**, **inst**, **hnet**, **net**, **hpin**, **pin**, **port**, etc and library objects like **base_cell**, **base_pin**, **lib_cell**, **lib_pin**, etc, and technology objects like **layer**, **route_rule**, etc. are supported, but only objects that have a `.name` attribute can use a name-pattern in addition to pointers (e.g. a wire object has no `.name` so it cannot be looked up with a name-pattern). See the Stylus Common UI Database Reference for a list of all the `obj_types`.

- *flags* is list of flags that may be an empty list. The supported flags are
 - **optional** means the argument is optional. If the argument is not given, *var* will be set to the *default* value. Arguments are **optional** by default, so this is mostly to make it easy to see it in the definition. It cannot be combined with **required**.
 - **required** means the argument is required. It is an error to not have the argument.
 - **multiple** means the argument can appear multiple times and *var* will be set to a list of all the values.
 - **hidden** means the argument is not shown in the help output, partial argument name matching will not match it, and <tab> complete will not show it. **hidden** must be optional, so it cannot be combined with **required**, **required_group**, or **exclusive_required_group** flags.
 - **obsolete** means the argument still works but is considered obsolete. It will give a warning message if it is used that the user should stop using this argument to be compatible with future releases. An **obsolete** argument will automatically be **hidden** and cannot be a required argument. See **alias** if you just want to change the *arg_name* to a new *arg_name*.
 - **{exclusive_group group_name}** means only one of the arguments can be given from the same *group_name*. For example, only one of `-my_int` or `-my_int2` are allowed:

```
{int_var 0 {-my_int int} int {{exclusive_group group1}} {help for -my_int}}
{int_var2 0 {-my_int2 int} int {{exclusive_group group1}} {help for -my_int2}}
```
 - **{exclusive_required_group group_name}** means exactly one of the arguments must be given from the same *group_name*.
 - **{required_group group_name}** means one or more of the arguments must be given from the same *group_name*.
 - **{requires arg_name1 [arg_name2 ...]}** means this argument requires all the other arguments in the list. For example both `-arg1` and `-arg2` are required if `-my_int` is given.

```
{int_var 0 {-my_int int} int {optional {requires -arg1 -arg2}}
```


	<pre>{help for -my_int}}</pre> ◦ {alias obsolete_name [warn]} means an alias of <i>obsolete_name</i> to the <i>arg_name</i> will be added. If warn is given, a warning message will be given if the <i>obsolete_name</i> is used telling the user to change to <i>arg_name</i>, otherwise <i>obsolete_name</i> will behave just like <i>arg_name</i>. The <i>obsolete_name</i> is not shown in help output. This can be used to migrate users to new argument names, but keep the old name for backward compatibility. For example, to migrate from <i>-old_my_int</i> to <i>-new_my_int</i> you would use this: <pre>{int_var 0 {-new_my_int int} int {{alias -old_my_int}} {help for -new_my_int}}</pre> • <i>help</i> is a help string for the argument, such as “<i>a file name to read</i>”.
- description <i>proc_help</i>	Specifies the help description for the defined procedure, shown by "help cmd" or "cmd -help".
-help	Outputs the command usage and a brief description about the command parameters.
-hidden	Makes the procedure hidden and not public. That means <tab> complete will not return this proc name, and "info commands *" will not return this proc name. But if you type the full name like "info commands <i>full_cmd_name</i> " it will return the command, or " <i>full_cmd_name</i> -help" will return help for it.
<i>proc_body</i>	Defines the Tcl procedure body that uses var names from <i>arg_list</i> .
<i>proc_name</i>	Specifies the name of the Tcl procedure.

Examples

Here is an example of some common usage:

```
define_proc test -description "An example of the extended argument syntax" {
    {out_file "default.out" out_file string optional "The output file name"}
    {my_count 0 {-count num_inputs} int required "The number of inputs"}
    {verbose false -verbose bool optional "Verbose output"}
    {check_err not_set {-check_err bool} bool optional "Override global check setting"}
    {effort low -effort {enum low medium high} optional "How much effort to use"}
    {my_insts {} {-insts} {object inst +} optional "One or more insts"}
} {
# Proc body goes here
    ...
}
```

The above procedure generates the following help output:

```
> test -help
```

An example of the extended argument syntax:

```
Usage: test [-help] [<out_file>] [-check_err <bool>] -count <num_inputs> [-effort {low
medium high}] [-verbose]

-help                                # Prints out the command usage

<out_file>                           # The output file name (string, optional)

-check_err <bool>                     # Override global check setting (bool, optional)

-count <num_inputs>                   # The number of inputs (int, required)

-effort {low medium high}             # How much effort to use (enum, optional)

-insts <inst>+                        # One or more insts (Iinst)+, optional)

-verbose                             # Verbose output (bool, optional)
```

delete_rule_filter

```
delete_rule_filter  
<filter_name* ...>  
[-rule <rule_name*>] ...
```

Deletes rule filters from the current session or stops applying rule filters to certain rule objects (rule sets, rule groups, or rule instances).

Parameters

<code><filter_name* ...></code>	Specifies the name(s) of the rule filter(s) to delete. If the <code>-rule</code> option is not specified, the specified rule filters will be deleted and removed from all rule objects to which they are applied.
<code>-rule <rule_name*></code>	Removes only the selected rule filters from the specified rule objects. At least one filter name is required when using this option.

Examples

- The following command deletes filterf1 and all references to it:

```
delete rule filter f1
```

- The following command removes filter f1 from the list of filters that are applied to rule set s1:

```
delete rule filter f1 -rule s1
```

Related Commands

- `add_rule_filter`
- `report_rule_filter`

elaborate_design

```
elaborate_design  
[-help]  
-script hdl_script
```

Converts RTL to a generic netlist based on the specified script, which should contain commands for reading HDL files into memory and creating a design hierarchy consisting of a top-level design and its referenced sub-designs from Verilog or SystemVerilog modules or VHDL entity/architectures. The command transforms each module/architecture to be represented as a design in the form of a structural netlist. It also performs semantic checking, sequential register (flops/latches) inferencing, and high-level HDL optimizations. Instances of undefined modules or entities are marked as unresolved and the corresponding modules/entities reported as blackboxes.

A sample script that can be passed to `elaborate_design` is shown below:

⚠ Sample hdl_script.tcl

```
read_hdl -language sv "cpu_if.sv cpu_alu.sv cpu_rf.sv"  
  
read_hdl -language v2001 "cpu_top.v"  
  
elaborate cpu_top
```

⚠ Note that the commands specified above are only valid within the script being passed to `elaborate_design`. You cannot use them directly at the Innovus prompt.

Parameters

-help	Outputs a brief description that includes type and default information for each <code>elaborate_design</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <pre>man elaborate_design.</pre>
-script hdl_script	Specifies the script for HDL manipulation within the Synthesis engine. The software looks for the specified script file in each search path directory set using the <code>script_search_path</code> attribute. Alternatively, you can specify the complete path to the script. <i>Data_type</i>: string, required

Examples

- The following command elaborates the top-level design and all of its references based on the specified script:

```
elaborate_design -script Xm_hifi3_ss_spfpu_7Xtmem.hdl.tcl
```

Related Information

- [hdl](#) Category Attributes
- [script_search_path](#) attribute in [system](#) Category Attributes
- The Innovus Synthesis Flow
- Design Initialization and Elaboration

encrypt

```
encrypt  
[-help]  
input_file_name  
[-pragma]  
[> file]  
[-tcl | -verilog ]
```

Encrypts a file in an nc-protect compatible format.

Parameters

-help	Prints out the command usage.
<i>input_file_name</i>	Specifies the file to be encrypted. <i>Data_type</i> : string, required
> <i>file</i>	Specifies the name of the encrypted file. By default, the encrypted file is printed to standard out (stdout). <i>Data_type</i> : redirect_operator, optional
-pragma	Specifies to only encrypt the text between the protect begin and protect end NC Protect pragmas. Encryption block must start with pragma protect comment. <i>Data_type</i> : bool, optional
-tcl	Specifies that the file to be encrypted is a Tcl file. This is the default option. <i>Data_type</i> : bool, optional
-verilog	Uses Verilog style comments for NC Protect pragmas. <i>Data_type</i> : bool, optional

Related Topics

- [decrypt](#)

eval_legacy

```
eval_legacy  
[-help]  
script
```

Runs a legacy command in the legacy Tcl interpreter. This command is not required in normal usage. However, it may be needed in cases where some legacy usage is not yet available in the Common UI. This command is intended as a temporary workaround as users transition to the Common UI and will be removed in a future release.

You should first check if there is a Common UI equivalent command before using this command by using the [get_common_ui_map](#) command:

```
>get_common_ui_map <legacyCommand>
```

Most legacy commands are available, but legacy commands that have behavior changes in the Common UI will not work properly (`saveDesign`, `restoreDesign`, `init_design`, `report_timing` etc.).

The legacy commands run in a separate Tcl interpreter, so Tcl variables in the current interpreter are not available. That means the *script* should normally be inside " " rather than { } so Tcl variable substitution occurs in the current interpreter before passing the script to the legacy Tcl interpreter.

Parameters

<code>-help</code>	Outputs a brief description that includes type and default information for each <code>eval_legacy</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man eval_legacy</code>
<code>script</code>	Specifies the name of the legacy script.

Example

The following command runs `setFillerMode` and `addFiller` commands in the legacy Tcl interpreter.

```
set my_prefix fill_  
  
eval_legacy "setFillerMode -corePrefix $my_prefix"  
  
eval_legacy "addFiller ..."
```

get_license_status

get_license_status [-help]

Displays a summary of all currently checked-out licenses.

Parameters

- help	Prints the usage of the <code>get_license_status</code> command. For a detailed description of the command, use the <code>man</code> command: <code>man get_license_status</code> .
-----------	--

Example

- The following command lists all licenses checked out currently:

```
get_license_status
```

Sample output:

Summary of all checked out licenses:

```
'ai' (Innovus_AI_Impl_System; INVS200; 26.1)
```

Related Information

- [Product and Licensing Information](#)

get_message

```
get_message  
-help  
-id msgID  
[-severity | -limit | -suppress | -count | -short | -long ]
```

Gets the limit, severity or suppression for a message.

Returns the following information about `set_message` parameters in the Litmus log file and in the Litmus console:

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

If you do not specify a parameter, the software displays values for all of the `set_message` mode parameters.

help

help

```
[<command_name> | <message_name> | -message]  
[-verbose]
```

Note: Although the `help` command is still available, it is recommended that you use the `man` command.

Displays all the Litmus commands and their command syntax. To display a group set of commands, use a keyword such as `add`, `delete`, `report`, or `set`. To display the usage for all commands, use `help -help`.

While in the Tcl mode, the `help` command displays a list of all available Tcl mode commands.

Parameters

<command_name>	Displays the command syntax for a given command name. If you do not specify a command name, the Litmus displays all the commands.
<message_name>	Displays help for the corresponding rule check message.
-message	Displays all rule check messages.
-verbose	Expands information about the command, including examples and related commands.

Examples

```
% help  
% help read_hdl  
% help read_hdl -verbose  
% help -help  
% help -message
```

input_libs

`input_libs library_files`

Loads timing library files specified in the RTL flow.

Litmus proceeds with the libraries that you define for timing analysis. If not specified, the tool proceeds with default library.

Parameters

<code>[-help]</code>	Prints out the command usage.
<code>library_files</code>	Specify the library files. This is a mandatory argument.

input_sdc

`input_sdc`

`[-help]`

`[ [-sdc_1 <string> -sdc_2 <string>] | <sdc_files>]`

Loads timing constraint files in the RTL flow. The constraints can be in the design constraints (SDC) format.

Parameters

<code>[-help]</code>	Prints out the command usage.
<code>sdc_files</code>	Specifies the timing constraint files (string, optional)
<code>-sdc_1 <string></code>	Specifies Design1's SDC (string, required)
<code>-sdc_2 <string></code>	Specifies Design2's SDC (string, required)

is_common_ui_mode

is_common_ui_mode

[-help]

Checks if Stylus Common UI mode is enabled.

Parameters

-help	Prints out the command usage.
-------	-------------------------------

Examples

The following command checks if Stylus Common UI mode is enabled on the Litmus prompt:

```
% is_common_ui_mode
```

```
1
```

Return value 1 indicates that Stylus Common UI mode is enabled.

litmus_setup_mmmc

```
litmus_setup_mmmc  
[-help]  
[mmmc_file]  
[lef_file]
```

Creates Multi-Mode, Multi-Corner (MMMC) file and LEF file. MMMC refers to the full set of timing libraries, constraints, and the associated modes.

During `litmus_setup_mmmc`, the tool creates and reads MMMC file using timing libraries and timing constraint files specified by the user. It also creates and reads the LEF file required for the RTL flow. This command should be used after `input_libs` and `input_sdc`

Parameters

<code>[-help]</code>	Prints out the command usage.
<code>[mmmc_file]</code>	Creates and reads MMMC files.
<code>[lef_file]</code>	Creates and reads LEF files.

print_message

```
print_message  
-help  
[ -id msgID | formatString]  
[args]  
[-error]  
[-warning]
```

Generates a message that is included in the `report_messages` output if the message is designated as an error or a warning.

Parameters

<i>args</i>	Specifies the format argument value to print.
<code>-error</code>	Specifies the message is an error. A prefix will be added.
<i>formatString</i>	Specifies the format specifier like <code>sprintf</code> or Tcl's <code>format</code> command.
<code>-help</code>	Prints a brief description that includes the type and default information for each <code>print_message</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man print_message</code>.
<code>-id msgID</code>	Specifies the message id.
<code>-warning</code>	Specifies that the message is a warning. A prefix will be added.

Examples

The following commands generate an error and a message, which are included in the `report_messages` output:

```
> print_message -error "A test %s." "from the user"
ERROR: (user): A test from the user.
```

```
> print_message -warning "A test %s." "from the user"
WARN: (user): A test from the user.
```

```
> print_message "A test %s." "from the user"
A test from the user.
```

```
> report_messages
*** Summary of all messages that are not suppressed in this session:
Severity      ID          Count      Summary
ERROR        no_id         1        A test %s.
ERROR        user          1        A test %s.
WARN         user          1        A test %s.
```

*** Message Summary: 1 warning(s), 2 error(s)

Related Command

- [report_messages](#)

read_db

```
read_db
[-help]
[-debug]
[-module_dirs list_of_dirs]
[-no_timing_graph]
[-read_globals tcl_file_name]
[-physical_data]
{[<in_dir> [-mmmc_file <string>]] | [-oa_lib_cell_view {<libname> <cellname> <viewname>}
[-mmmc_file <string>][-lef_files list_of_files]]}
[-setup_views <string> -hold_views <string>]
[-leakage_view <string> -dynamic_view <string>]
```

Loads the saved design files of a previous design session. The files included depend on the work completed in the design session when it was saved. The directory can include floorplan files, special route files, placement files, early global route files, and power switch state files.

To restore designs saved in Litmus format, usage is as follows:

```
read_db {sessionName.dat}
```

Note: You must rebuild the timing graph after you restore the design. Run extraction, then timing analysis.

You must specify all pin types in the LEF file. Without these definitions, the Litmus software attempts to determine the pin type from the pin name when restoring a design, and does not always do this correctly. If this is the case, Litmus generates the following error message:

```
**ERROR: Current net list does not match wires file.
```

 Modifying the contents of the `sessionName.dat` directory created by `saveDesign` can cause major unexpected results. Never modify the `design.dat` directory contents.

To restore OpenAccess designs, usage is as follows:

```
read_db
{-oa_lib_cell_view string}
```

You can use the `read_db` command after starting the Litmus software.

Note: Litmus supports only the following begin/end extension values for DEF `NETS` style wiring segments (0, width/2 or LEF `LAYER WIREEXTENSION`). If any of the wire extensions has unsupported extension value, then Litmus converts it to a special wire (DEF `SPECIALNETS`). While writing back

OpenAccess, these special wires will again be converted back to DEF `NETS` style wiring.

From the 14.1 release, the `read_db` command accepts a tar file, when a tar ball is given. For instance, if you specify the following command:

```
read_db myDesign.inn.dat.tar.gz
```

`read_db` will untar the tar ball under the current working directory, restore the design, and then remove the temporary data directory. This enables a single DB file operation all the time.

Parameters

<code>-help</code>	Prints a brief description that includes the type and default information for each <code>read_db</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man read_db</code>
<code>-dynamic_view string</code>	Specifies the analysis view for the dynamic power analysis.
<code>-debug</code>	Enables debug data.
<code>-hold_views string</code>	Specifies the hold views for loading, rather than the hold views active during <code>write_db</code> .
<code>in_dir</code>	Specifies the <code>.dat</code> directory from which to load the files. If the session name of the saved design includes the extension <code>.enc</code> , make sure that you include this extension in the <code>in_dir</code> name: for example, <code>session1.enc.dat</code> .
<code>-leakage_view string</code>	Specifies the analysis view for leakage power analysis.

<code>-lef_files list_of_files</code>	Specifies the LEF files name for loading. Use this option if you want to override the existing LEF files in the design being restored and load different LEF files instead. • This option is intended to allow upgrading to new versions of LEF files that are compatible with the old versions. The list of all LEF files used by this design must be given. • If any pin locations or cell boundaries or OBS shapes are changed for used cells, new DRC violations may occur. • If pins are added or removed from used cells, or used cells are removed, fatal errors may occur when loading the netlist. • If the removed pins and cells are unused in the design, warnings or errors may still occur if any library-specific data was stored for those cells or pins during the design flow (such as dont_touch data) Examples Existing LEFs used by <code>write_db: tech.lef, A.lef</code> You can change <code>A.lef</code> to <code>A_v2.lef</code> as given below: <pre>read_db place.dat -lef_files {tech.lef A_v2.lef}</pre>
<code>-mmmc_file</code>	Specifies the path name to a Multi-Mode/Multi-Corner (MMMC) configuration file (<code>viewDefinition.tcl</code>). Use this option if you want to override the settings in the existing MMMC configuration file in the design being restored and instead load a different MMMC file.
<code>-module_dirs list_of_dirs</code>	Specifies module-level sessions which should override the corresponding modules in the top-level hierarchical session.
<code>-no_timing_graph</code>	Restores the design data without a timing graph.

<code>-oa_lib_cell_view string</code>	Specifies the library name, cell name, and view name for restoring design information from the previous design session in the OpenAccess database format.
<code>-physical_data</code>	Enables physical data to be read.
<code>-read_globals tcl_file_name</code>	Specifies the UDM variables to be honored when restoring the design. You need to provide a tcl file stating the pre-defined UDM variable names to be honored.
<code>-setup_views string</code>	Specifies the setup views for loading rather than the setup views active during <code>write_db</code> .

Examples

- The following command loads the saved design files from the `test.inn.dat` directory:
`read_db test.inn.dat`
- The following command restores the `my.enc.dat` design without loading the timing libraries and MMMC configuration information:
`read_db -no_timing_graph my.inn.dat`
- The following command restores the `placed.enc.dat` design but overrides the existing MMMC settings in the design with the settings in the `new_mmmc.tcl` file:
`read_db placed.inn.dat -mmmc_file new_mmmc.tcl`
- The following command restores the "designLib top placed" OpenAccess cellview but overrides the existing MMMC settings in the design with the settings in the `new_mmmc.tcl` file:
`read_db -oa_lib_cell_view "designLib top placed" -mmmc_file new_mmmc.tcl`

Related Topics

- [write_db](#)

read_metric_config

```
read_metric_config  
[-help]  
  fileName  
[-format {html excel text}]
```

Reads the YAML file written by the `write_metric_config` command.

Parameters

<code>fileName</code>	Specifies the name to read metric configuration from.
<code>-format {html excel text}</code>	Specifies the configuration for specified <code>report_metric</code> format.
<code>-help</code>	Outputs a brief description that includes type and default information for each <code>read_metric_config</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man read_metric_config</code>

Example

The following example writes the YAML structure in the `text` format in `small.text`, reads the YAML file, and generates the report in the text format.

```
> write_metric -format text small.text ;  
> read_metric_config -format text small.yaml  
> report_metric -format text
```

Related Topics

- [report_metric](#)
- [reset_metric_config](#)
- [write_metric_config](#)

read_metric_html_package

```
read_metric_html_package  
[-help]  
filename
```

Overwrites the HTML for `report_metric -format vivid` and the vivid page configuration (accessible via `read_metric_config/write_metric_config`) with the contents from the specified *filename*.

Parameters

<i>filename</i>	Specifies the name of the file from where the tool will read the metric package. If the specified <i>filename</i> is not a supported format, the tool will give an error message that the file format is invalid.
<code>-help</code>	Outputs a brief description that includes the type and default information for each <code>read_metric_html_package</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <pre>man read_metric_html_package</pre>

Related Topics

- [read_metric_config](#)
- [write_metric_config](#)

redirect

```
redirect  
[-help]  
[file_or_var_name]  
[command]  
[-bg | -tee]  
[-stdin | -append]  
[-stdin | -variable]  
[-variable]  
[-stderr | -stdin]
```

Redirects the output to a file or variable. You can also directly use redirect characters without explicit use of the redirect command for Litmus commands.

Any Tcl return value from *command* is returned by redirect. If redirect is used at the prompt, any echoing of the Tcl return at the prompt is not redirected.

You can also use the `redirect` command to write the contents directly to a gzip compressed file by ending the file name with `.gz`.

Parameters

<code>-help</code>	Prints a brief description that includes type and default information for each <code>redirect</code> parameter.
<code>file_or_var_name</code>	File or variable name for output to go.
<code>-bg</code>	Runs the commands in a forked process in background. Note: Forking processes have significant impact on memory. Ensure that you have enough memory and do not use this option in a loop. <i>Data_type:</i> bool, optional
<code>command</code>	Command to be executed.
<code>-append</code>	Appends stdout/stderr to file or variable.
<code>-stderr</code>	Redirects stderr.
<code>-stdin</code>	Redirects stdin.
<code>-tee</code>	Redirects output to both screen and file or variable.
<code>-variable</code>	Redirects to a Tcl variable.

It is also possible to use redirect characters similar to Unix usage, for Litmus commands, but not for built-in Tcl commands. This usage works for any Litmus command that does not explicitly register `>` or `>>` as options (if `>` or `>>` are visible in the man page or `-help` output for the command, it is explicitly registered). Note that the "source" command is overlaid by Litmus to add various verbose options, so it also supports these characters.

The table below shows the usage of redirect characters, and how to use `&` to indicate a Tcl var name.

Redirect Characters	Meaning
<code>> <name></code>	redirect stdout to the file <code><name></code>
<code>>> <name></code>	redirect stdout and append to the file <code><name></code>
<code>> &<name></code>	redirect stdout to the Tcl var <code><name></code>
<code>>> &<name></code>	redirect stdout and append to the Tcl var <code><name></code>

Examples

- `redirect timing.txt "time_design -postRoute"`

Redirects the output to a new file `timing.txt`.

- `redirect test.out.gz "source test.tcl" -append`

Appends the output to the file `test.out.gz` in compressed gzip format.

- `redirect timing.txt "time_design -postRoute" -append -tee`

Redirects the output to both screen and a file.

- `source test.tcl > a.log.gz` #redirect to the file `a.log.gz` in compressed gzip format
- `report_area >> a.log` #redirect and append to the file `a.log`
- `report_area > &a` #redirect to the Tcl var `a`
- `report_area >> &a` #redirect and append to the Tcl var `a`

rename_obj

```
rename_obj  
[-help]  
[-escape_ok]  
[-flexible]  
object  
new_name  
[-quiet]
```

Renames the `base_name` of a single design object, which could only be a `inst`, `hinst`, `design`, `module`, `port`, `hport`, or `hnet`. This command changes only the `base_name` of the specified objects, and cannot move those across the netlist hierarchy. This command also returns the new name of the renamed object.

The names used for SDC timing constraints can only be modified using this command, if they have been loaded into the memory. So, ensure that all the timing views are loaded using the `set_analysis_view` command before running the `rename_obj` command so that the SDC constraints are updated properly. The modified SDC files can be retrieved after renaming the objects.

The `rename_obj` command cannot update the names used inside a CPF `power_intent` file. This is because the CPF usage assumes that the CPF file is golden and should not be modified. To change the names used inside a CPF file, first update the CPF file manually, and then use the updated CPF file to run the `rename_obj` command.

An IEEE1801 `power_intent` file is allowed to be changed during the design flow. So, changing names using the `rename_obj` command will automatically update the IEEE1801 object names. You can use the `rename_obj` command to write out the new IEEE1801 file with the updated names.

Parameters

-help	Provides a brief description of the command that includes the type and default information for its parameters. For a detailed description of the command and all its parameters, use the man command: <code>man rename_obj</code>
-------	--

- <i>escape_ok</i>	Allows <code>new_name</code> to have either the hierarchy chars '/' or the bus-bit chars '[' in it, which will automatically be escaped. If this parameter is not specified, any set of '/' or '[' chars in <code>new_name</code> will cause an error and the object will not be renamed. <i>Data_type</i>: bool, optional
-flexible	If <code>new_name</code> coincides with an existing name, this parameter appends a suffix (such as _1 and _2) to differentiate it. If this parameter is not specified, a name collision in such a scenario causes an error, and the specified object is not renamed. <i>Data_type</i>: bool, optional
<i>object</i>	Specifies the object to be renamed. The object here must be an <code>inst</code>, <code>hinst</code>, <code>design</code>, <code>module</code>, <code>port</code>, <code>hport</code>, or <code>hnet</code>. If <i>object</i> is a <code>port</code> or <code>hport</code>, any <code>hnet</code> connected to it with the same <code>base_name</code> is also renamed to keep it matched (for example, renaming <code>hport:top/il/clkin</code> also renames <code>hnet:top/il/clkin</code>). If <i>object</i> is an <code>hnet</code> that is connected to a <code>port</code> or <code>hport</code> with the same <code>base_name</code>, an error occurs. In such a case, you must change the <code>port</code> or <code>hport</code> name to proceed. If an <code>hnet</code> name matches with its associated <code>net</code> name (for example, <code>get_db \$my_hnet .net</code>), then the <code>net</code> name is also renamed on using the <code>rename_obj</code> command. For example, renaming <code>hnet:top/net1</code> also renames <code>net:top/net1</code>. Therefore, to rename a <code>net</code>, you need to rename the <code>hnet</code> with the same name. This command does not allow renaming a bus-bit object. So any <code>port</code>, <code>hport</code>, or <code>hnet</code> that has a <code>.base_name</code> like <code>busA[0]</code> (a bus-bit object), cannot be renamed, While a <code>.base_name</code> with a scalar name (like <code>busA\[0\]</code>) can be changed. <i>Data_type</i>: (<code>inst</code> <code>hInst</code> <code>vCell</code> <code>topCell</code> <code>term</code> <code>hTerm</code> <code>hNet</code>), required

<i>new_name</i>	Specifies the new <code>base_name</code> for the specified object. <i>Data_type</i> : string, optional
<code>-quiet</code>	Suppress messages about renamed objects. <i>Data_type</i> : string, optional

Examples

- The following command renames an `inst` `base_name` to `i2_new`:

```
rename_obj inst:top/i1/i2 i2_new
```

```
inst:top/i1/i2_new          #returned object with new name
```
- The following command renames an `inst` `base_name` `i2\i3` with escaped hierarchy:

```
rename_obj {inst:top/i1/i2\i3} i2/i3_new -escape_ok
```

```
inst:top/i1/i2\i3_new      #returned inst object, no hierarchy change
```
- The following command renames a scalar `hport` name to remove escaped bus-bit chars:

```
rename_obj {hport:top/i1/busA\[0\]} busA_0
```

```
hport:top/i1/busA_0        #returned hport object
```

Note: `hnet:top/i1/busA\[0\]` is also renamed to `hnet:top/i1/busA_0`.
- The following command renames a `port` name:

```
rename_obj port:top/clkin clkin_new
```

```
port:top/i1/clkin_new      #returned port object
```

Note: Both `hnet:top/i1/clkin` and `net:top/i1/clkin` are renamed.

rename_snapshot

```
rename_snapshot  
[-help]  
[-branch newBranch]  
[-id metricID]  
[-name newName]  
-uuid snapshotuuid
```

Allows you to rename existing snapshots.

`rename_snapshot` does not support canonical paths and cannot change the location of a snapshot within the hierarchy.

Parameters

<code>- branch newBranch</code>	Specifies the new branch for the snapshot. <i>Data_type</i> : string, optional
<code>-help</code>	Outputs a brief description that includes type and default information for each <code>rename_snapshot</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man rename_snapshot</code>
<code>-id metricID</code>	Specifies the metric database ID to change. If you do not specify any id, the tool will take current database as default. <i>Data_type</i> : string, optional Note: You can find the uuid of a snapshot by using <code>get_metric</code> for the uuid metric.
<code>-name newName</code>	Specifies the new name for the snapshot. The tool will error out if you set the name to empty string. <i>Data_type</i> : string, optional
<code>-uuid snapshotuuid</code>	Specifies the snapshot UUID. The tool will error out if there is no snapshot with the provided uuid. <i>Data_type</i> : string, required

Related Information

- `get_metric`

report_dangling_ports

```
report_dangling_ports
-help
-partition
-out_file fileName
```

Reports ports that are disconnected from both the inside and outside of a module, and ports that are connected within the module but disconnected outside the module.

Note: If the design has some dangling or unconnected ports, the tool will add `FE_UNCONNECTED_*` nets for these dangling or unconnected ports in the design during optimization. You can report these dangling ports using the `report_dangling_ports` command.

Parameters

-help	Prints a brief description that includes type and default information for each <code>report_dangling_ports</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man report_dangling_ports</code>
-out_file	Specifies the name of the report generated by this command.
-partition	The option a partition only flag of the type boolean. This is an optional.

Examples

- The following command writes information about the dangling ports to the `test.rpt` file:

```
report_dangling_ports -out_file test.rpt

Total number of dangling ports: 125
```

- The following command writes information about the dangling ports to the `sample.rpt` file:

```
report_dangling_ports -partition -out_file sample.rpt

Total number of dangling ports: 0
```

report_inactive_arcs

```
report_inactive_arcs
[-help]
[instance_port_obj]
[-exclude_type {const snipped disable library_disable global_disable conditional_disable
library_mode power_mode_disable}]
[-include_net_arcs]
[-include_when_cond]
[-type {const snipped disable library_disable global_disable conditional_disable
library_mode power_mode_disable}]
[-view view_name]
[> <filename[.gz]>]
[>> <filename[.gz]>]
[-delay_arcs_only | -check_arcs_only]
```

Reports information about disabled timing and timing check arcs in the design. Reports all arcs disabled due to user-specified exceptions such as [set_disable_timing](#) or [set_case_analysis](#), as well as information about arcs disabled by constant propagation during timing analysis, snipped loop arcs, and arcs disabled in the timing library.

Parameters

-help	Prints out the command usage.
instance_port_obj	Reports disabled timing arcs for the specified list of instances or ports. If you specify a list of instances, the software reports all disabled arcs of the specified cells. If you specify a list of ports, the software reports all disabled arcs directly connected to the ports. <i>Default:</i> Reports all disabled net and cell arcs.
-check_arcs_only	Lists only disabled check arcs. The <code>check_arcs</code> are disabled by using the <code>set_disable_timing</code> command. <i>Default:</i> Both disabled timing delay arcs and check arcs are listed.
-delay_arcs_only	Lists only disabled timing delay arcs.

<pre>-exclude_type {const snipped disable library_disable global_disable conditional_disable library_mode power_mode_disable}</pre>	Excludes the disabled arcs that belong to the specified category of disabled arcs. The valid options are: const, snipped, disable, library_disable, global_disable, conditional_disable, library_mode, and power_mode_disable.
<pre>-include_net_arcs</pre>	Includes disabled net arcs in the report.
<pre>-include_when_cond</pre>	Shows the when_cond of arcs in the output report.
<pre>-type {const snipped disable library_disable global_disable conditional_disable library_mode power_mode_disable}</pre>	Lists only the disabled arcs that belong to the specified category of disabled arcs. The <code>-type</code> options restrict the report to only those arcs that are disabled due to a particular timing analysis step, such as constant propagation, specified disables, or loop arc disables. The valid options are: • <code>const</code>: Propagated constant -- arcs disabled due to a constant value. The constant value can apply to one of the arc nets/pins, to side inputs which disable a condition for the arc, to nets that disable an active mode for the arc, or disable a default conditional arc. • <code>snipped</code>: Loop snipped arcs -- arcs automatically disabled by the Litmus software as part of a feedback loop. • <code>disable</code>: Defined by using the <code>set_disable_timing</code> command. • <code>library_disable</code>: Delay arc disabled due to library disables. These disables may be caused by the following conditions: ◦ Disables specified using the <code>set_disable_timing</code> command. ◦ The arc is an asynchronous arc and the attribute <code>timing_enable_preset_clear_arcs</code>

	is set to <code>false</code>. • <code>global_disable</code>: Lists the arcs that are disabled through user-specified global variable settings. • <code>conditional_disable</code>: Arcs disabled if a <code>when</code> condition is specified. There is no constant on sink or source pin and arc is disabled due to the <code>when</code> condition. • <code>library_mode</code>: Displays library mode of disabled arcs. • <code>power_mode_disable</code>: Allows reporting the arcs for which power mode is disabled.
<code>-view view_name</code>	Generates the report for the specified analysis view only. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode.
<code>{> >>} filename</code>	When specified with <code>></code>, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with <code>>></code>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The <code>filename</code> parameter must be the last argument in the list. Note: To write a compressed report, add the <code>.gz</code> extension to the file name. <i>Default:</i> Report is displayed on standard output without being saved.

Examples

- The following command lists disabled check arcs for the specified view:

```
litmus > report_inactive_arcs -check_arcs_only -view sn_fast_RMAX
```

```
-----
```

From	To	ArcType	Sense	Reason	View

--					
BT/se_0/CK	BT/se_0/D	hold_rising	non_unate	constant disabled	sn_fast_RMAX
BT/se_0/CK	BT/se_0/D	setup_rising	non_unate	constant disabled	sn_fast_RMAX
BT/se_0/RN	BT/se_0/RN	min_pulse_width	non_unate	constant disabled	sn_fast_RMAX
BT/se_0/CK	BT/se_0/RN	removal_rising	non_unate	constant disabled	sn_fast_RMAX

--					

- The following command reports all disabled arcs in the design:

```
litmus> report_inactive_arcs > Report
```

The following example shows the report generated:

Flags :				
const		propagated constant		
snipped		loop snipped arcs		
disable		set_disable_timing		
disable_clock_gating		set_disable_clock_gating		
library_disable		set_disable_cell_timing		
Missing_Phase		No Arc Phase Data		
From	To	DisableType	ArcType	Reason

u8/CK ^	u8/D ^	const	hold_rising_clk_rise	u8/D = 1
u8/CK ^	u8/D v	const	hold_falling_clk_rise	u8/D = 1
seg3/u12/A ^	seg3/u12/Y v	disable	combinational	User Disable

- When the software is in multi-mode multi-corner analysis mode, the report generated lists the

disabled arcs by active analysis view, as shown in the following example:

```
-----
Flags : const propagated constant
snipped loop snipped arcs
disable set_disable_timing
disable_clock_gating set_disable_clock_gating
library_disable set_disable_cell_timing
Missing_Phase No Arc Phase Data
From          To          DisableType ArcType  Reason          View
-----
gscanmode ^ FE_0305_gmode/I ^  const          net_arc  gscanmode = 0  setup_view2
gscanmode v FE_0305_gmode/I v  const          net_arc  gscanmode = 0  setup_view2
gscanmode ^ FE_0305_gmode/I ^  const          net_arc  gscanmode = 0  hold_view1
gscanmode v FE_0305_gmode/I v  const          net_arc  gscanmode = 0  hold_view1
gscanmode ^ FE_0305_gnmode/I ^  const          net_arc  gscanmode = 0  hold_view2
-----
```

Related Commands

- [set_disable_timing](#)
- [set_case_analysis](#)

report_messages

report_messages

[-help]

[[{-errors | -warnings | -all } [-count]] | -suppressed] [[-prefix <string> [-detail] [-summary]]]

[-detail | -count | -suppressed] [-summary | -count | -suppressed]

Reports the messages that have been issued since the software was launched.

When you run the `report_messages` command without any parameters, it prints a summary table and does not return any Tcl value. However, when you specify a parameter(s), it returns a list a of message IDs.

Parameters

-help	Prints a brief description that includes type and default information for each <code>report_messages</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <pre>man report_messages</pre>
-all	Returns a list of all issued message IDs as a Tcl result. The result does not include suppressed message IDs.
-count	Returns a count of every message when the <code>-errors</code> , <code>-warnings</code> , or <code>-all</code> parameter is specified.
-detail	Returns a list of detailed message of the specified prefix.
-errors	Returns a list of issued error message IDs as a Tcl result.
-prefix	Returns a list of issued error message IDs matched with the specified prefix as a Tcl result.
-summary	Returns a message summary of the specified prefixes.
-suppressed	Returns a list of suppressed message IDs as a Tcl result.
-warnings	Returns a list of issued warning message IDs as a Tcl result.

Examples

- The following command reports the count of all issued message IDs:

```
report_messages -all -count
```

```
{IMPMSMV-1810 855} {IMPEXT-3570 6} {IMPESI-3140 1} {IMPMF-5054 1} {CHKTIM-4 16}  
{IMPTCM-6 1} {IMPTCM-46 2} {IMPTCM-48 1} {SPEF-1152 4} {TCLCMD-981 2} {TCLCMD-1110 1}  
{TCLCMD-1247 1} {TCLCMD-1470 1}
```

- The following command reports the count of all issued warning message IDs:

```
report_messages -count -warnings
```

```
{IMPMSMV-1810 858} {IMPEXT-3570 6} {IMPESI-3140 1} {IMPMF-5054 1} {CHKTIM-4 16}  
{SPEF-1152 4}
```

- The following command reports a list of suppressed message IDs:

```
report_messages -suppressed
```

```
IMPESI-3086 IMPESI-3188 IMPESI-3194 IMPESI-3199 IMPESI-3198 IMPESI-3197 IMPESI-3196
```

report_netlist_statistics

report_netlist_statistics

Reports statistical data of the design netlist such as number of nets, number of IOs, number of pins in the design and number of terms per net(s).

Example

*** Statistics for net list DTMF_CHIP ***

Number of cells = 4263
Number of nets = 4551
Number of tri-nets = 0
Number of degen nets = 21
Number of pins = 16490
Number of i/os = 57

Number of nets with 1 terms = 21 (0.5%)
Number of nets with 2 terms = 2510 (55.2%)
Number of nets with 3 terms = 771 (16.9%)
Number of nets with 4 terms = 510 (11.2%)
Number of nets with 5 terms = 223 (4.9%)
Number of nets with 6 terms = 108 (2.4%)
Number of nets with 7 terms = 76 (1.7%)
Number of nets with 8 terms = 46 (1.0%)
Number of nets with 9 terms = 30 (0.7%)
Number of nets with >=10 terms = 256 (5.6%)

*** 72 Primitives used:

Primitive ADDFHX1 (195 insts)
Primitive ADDHX1 (15 insts)
Primitive AND2X1 (279 insts)
Primitive AND3X1 (1 insts)
Primitive AND4X1 (8 insts)
Primitive AOI211X1 (65 insts)
Primitive AOI21X1 (282 insts)

report_obj

```
report_obj
[<start> [<pattern>]]
[-all | [-computed] | [-selected]]
[-help]
[-categories list_of_categories]
[-file fileName]
[-index index_name index_value ...]
[-tabular]
[-set_by_user | -read_only ]
[-tcl | -verbose]
```

Creates a report about the attributes for a given pattern/category/object. This command is mainly intended for root attributes but can be used for any object.

The default output reports those attributes that are set to non-default values and are not computed. This makes it easy to see any changes that were made in the run scripts.

You can also add more attributes or filter out some attributes using various parameters below.

Parameters

-help	Prints a brief description that includes the type and default information for each system root attribute.
-all	Reports all non-computed attributes and their current value, even if the current value matches the default value.
-categories <i>list_of_categories</i>	Reports only those attributes that are in the given categories. Wildcards * and ? are allowed. Attributes that do not have a category are assigned the reserved category name <code>NULL</code> . The report puts all the attributes in a category together.
-computed	Includes attributes that are computed (for example <code>is_computed == true</code>). By default, the computed attributes are not included. If the data is not computed, this parameter triggers building the timing graph, and so on, that may take significant run time. This parameter can be used with <code>-all</code> .
-file <i>fileName</i>	Specifies the output file name.

- index index_name index_value ...	Specifies the list of indices in a form of object pointers or pairs of <i>obj_type object_name</i> , for example. '-index {view view1}' or '-index \$obj'.
<pattern>	Returns all the attribute names that match the specified <pattern> using the * and ? wildcards. The <start> value is required before the <pattern> value. <i>Default:</i> *
-read_only	Reports attributes with .is_settable = false.
-set_by_user	Reports the options set by the user.
<start>	Returns attributes for the objects in the <start> list. <i>Default:</i> root
-selected	Reports attributes for only selected objects. This parameter cannot be used with -all.
-tabular	Reports attributes details in a table.
-tcl	Returns the objects and their values in a TCL style list, which is a list of keyword-value pairs.
-verbose	Returns the object's data type and default value. This parameter cannot be used with the -tcl parameter.

Examples

- The following command displays the root attributes for the `timing` category that are not computed, and do not match the default value with their current values:

```
report_obj -categories timing -all
```

```
*****
```

```
** Object:
```

```
root:*****
```

```
*****
```

```
===== Category:
```

```
timing=====
```

```
Attribute Name
```

```
Current Value
```

```
-----
-

budget_timing_compare_with_new_opt_data
0budget_timing_push_down_virtual_clock
0budget_timing_write_clock_to_virtual_clock_map
0timing_all_registers_include_icg_cells
truetiming_allow_input_delay_on_clock_source          falsetiming_analysis_aocv
false

...
```

- The following command displays all the root attributes that start with place*.

```
• report_obj / place* -
all*****
***** Object:
/*****
*****

Attribute Name                                     Current Value -----
-----
-place_blockages
{}place_cell_edge_spacing
{}place_design_floorplan_mode
falseplace_design_refine_place                    true...
```

- The following command displays the attributes that do not match the default value for the net objects in the list:

```
report_obj [get_db nets *clk]
*****
***** Object:
net:top/tclk*****
*****

Attribute Name                                     Current Value -----
-----
arc:0x2b1c79348088 arc:0x2b1c793485b0base_name
tclkbbbox                                           {1073741.8235
1073741.8235 -1073741.824 -1073741.824}...

*****
```

```
***** Object:
net:top/tclk2*****
*****

Attribute Name                                Current Value -----
-----arcs
arc:0x2b1c79348088base_name
tclk2bbox                                     {1073741.8235
1073741.8235 -1073741.824 -1073741.824}...
```

report_rule_check

```
report_rule_check
[<rule_name* ...>]
[-category category_name] ...
[-filter filter_expression [-regexp]]
[-filtered_by filter_name*]
[-limit [natural_number]]
[-nofiltered_out | -filtered_out | -complete | -object_filtered_out]
[-occurrence_count]
[-rule_separation num]
[-sdc_design design_name]
[-sdc_lint command_name*]
[-sdc_mode mode_name]
[-severity severity_name] ...
[-status name]
[-summary | -verbose]
[-spreadsheet file_name.xlsx | -csv file_name.csv]
[-replace]
[-no_header]
```

Reports the rule occurrences encountered when running the specified rule instances.

Parameters

<rule_name* ...>	Specifies the name(s) of the rule set(s), group(s), or instance(s) to report. This accepts wildcards.
------------------	---

-category <category_name>	Reports only rule instances in the given category.
-filter <filter_expression>	Report only occurrences for which the specified <code>filter_expression</code> evaluates to true. The syntax for <code>filter_expression</code> is the same as for the Tcl <code>find</code> command.
-regexp	Specifies that the pattern matching operators used inside the <code>filter_expression</code> (such as <code>=~</code>) should use regular expressions as patterns, rather than glob-style patterns.
-filtered_by <filter_name*>	Reports only the occurrences filtered by the specified filter(s). Wildcards are accepted.
-limit [<number>]	Specifies the maximum number of occurrences of each rule instance that be reported if the <code>-verbose</code> option is specified. Zero (0) means no limit, which is the default if <code><number></code> is not specified.
-nofiltered_out	Reports only occurrences that have not been filtered. This is the default.
-filtered_out	Reports only occurrences that have been filtered.
-complete	Reports all rule occurrences.
-object_filtered_out	Reports only occurrences that are related to the filtered-out objects. Object filter is specified by <code>add_rule_filter -object_filter</code> .
-occurrence_count	Reports the number of occurrences even if it is zero.
-rule_separation <num>	<code><num></code> specifies the number of blank lines to print before each rule is reported. Without this option, no blank lines are printed before each reported rule.
-sdc_design <design_name>	Reports rule instances in hierarchical flow for design with name ' <code>design_name</code> '.
-sdc_lint <command_name*>	Reports only SDC lint occurrences related to the specified SDC command names. If this option is given, only SDC lint rules will be reported. This accepts wildcards.
-sdc_mode <mode_name>	Reports rule instances in multimode flow for mode with name ' <code>mode_name</code> '.

<code>-severity</code> <code><severity_name></code>	Reports only rule instances of the specified severity level. <code><name></code> can be Error, Warning, Note, Ignore, or the name of any severity level added by the <code>add_severity</code> command.
<code>-status</code>	When used with the <code>-sdc_lint</code> option, <code>-status</code> refers to the status of the <code>sdcstmt</code> . Otherwise, <code>-status</code> refers to status of rule instances. When used with <code>-sdc_lint</code> , valid values are <code>pass</code> and <code>fail</code> . Otherwise, valid values are <code>pass</code> , <code>fail</code> , <code>not-run</code> , and <code>inconclusive</code> . See the Examples section below, for more information.
<code>-summary</code>	Reports a summary of each rule instance that has been run and has rule occurrences. This is the default.
<code>-verbose</code>	Reports extended information for all rule occurrences of each specified rule instance.
<code>-spreadsheet</code> <code><file_name>.xlsx</code>	Reports rule instances in spreadsheet format. File name must end with <code>'.xlsx'</code> . It generates <code>'<file_name>.xlsx'</code> for all of specified rule instances.
<code>-csv <file_name>.csv</code>	Reports rule instances in csv format. File name must end with <code>'.csv'</code> . It generates <code>'<file_name>_<rule_name>.csv'</code> for each of specified rule instances.
<code>-replace</code>	Overwrites the output file.
<code>-no_header</code>	Hides the header in the spreadsheet or CSV file.

Examples

The following command reports a summary of the check results of the rule instances of the Error severity in the CONSTRAINT_LINT category:

```
report_rule_check -severity error -category CONSTRAINT_LINT
```

CONST_LINT_* rules with a severity of Warning or Note do not cause an SDC command to fail. Therefore, when you use the `"report_rule_check -sdc_lint * -status fail"` command, it does not report occurrences of CONST_LINT_* rules that have a severity of Warning or Note; it only reports occurrences of severity Error.

While for all other rule types, rule occurrences of Warning or Note are considered fail and would be included when you issue the following command:

```
report_rule_check -status fail
```

Related Commands

- `write_lint_configuration`
- `verify_timing_intent`

report_rule_filter

```
report_rule_filter  
<filter_name* ...>  
[-rule <rule_name*>] ...
```

Reports rule filters in the current session.

Parameters

<code><filter_name* ...></code>	Specifies the name(s) of the rule filter(s) to report. By default, the command reports all rule filters. This accepts wildcards.
<code>-rule <rule_name*></code>	Reports rule filters applied to the specified rule objects. If filters names are also specified, only selected filters that apply to the given rule objects will be reported.

Examples

- The following command reports all rule filters:

```
report rule filter
```

- The following command reports all rule filters applied to rule set s1:

```
report rule filter -rule s1
```

Related Commands

- [add_rule_filter](#)
- [delete_rule_filter](#)

report_rule_instance

```
report_rule_instance
[<full_name* ...>]
[-summary | -verbose]
```

Reports one or more rule instances in the current session. This command applies to rules that are not related to SDC integration.

Parameters

<full_name* ...>	Specifies the fully qualified name(s) of the rule instance(s) to report. By default, the command reports all rule instances. This accepts wildcards.
-summary	Reports the name, description, and severity level of the specified rule instances.
-verbose	Reports extended information for the specified rule instances.

report_rule_source

```
report_rule_source  
[<name* ...>]  
[-summary | -verbose]
```

Reports one or more rule sources in the current session.

Parameters

<name* ...>	Specifies the name(s) of the rule source(s) to report. By default, the command reports all rule sources. This accepts wildcards.
- summary	Reports attributes of the specified rule sources.
- verbose	Reports extended information for the specified rule sources.

Examples

The following command reports rule sourceCHECK_CLK:

```
report rule source CHECK_CLK
```

resume

`resume [-help]`

Resumes a suspended script from the point where it had stopped.

Use the `suspend` and `resume` commands to debug your scripts interactively. The `suspend` command stops a script and restores access to the Litmus prompt. You can then type any command required for debugging. Whenever you want to resume the suspended script, type `resume` at the Litmus prompt.

Parameters

<code>-help</code>	Prints the usage of the <code>resume</code> command. For a detailed description of the command, use the <code>man</code> command: <code>man resume</code> .
--------------------	--

Related Commands

- `suspend`

set_compress_level

`set_compress_level compression_level`

Specifies the compression speed to use when compressing large data files. A faster compression speed creates a less compressed file. A slower compression speed creates a more compressed file. By default, the software uses the `gzip` default speed `compression_level`.

Parameters

<code>-help</code>	Prints a brief description that includes type and default information for each <code>set_compress_level</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man set_compress_level</code>
<code>compression_level</code>	Specifies the speed level to use when compressing files. You can specify a <code>compression_level</code> between 1 and 9, where: 1 Performs the quickest compression, and least compressed files. 9 Performs the slowest compression, and most compressed files. Note: To return to the <code>gzip</code> default <code>compression_level</code> (<code>Z_DEFAULT_COMPRESSION</code>), specify <code>-1</code>.

set_expanded_hinsts

```
set_expanded_hinsts  
[-help]  
<hinst_names>
```

Parameters

-help	Prints out the command usage
<hinst_names>	Expanded hinsts string (string, required)

set_license_check

```
set_license_check  
[-help]  
[-checkout lic]  
[-default]  
[-option_list "lic1 lic2 ..."]  
[-server_license_report]  
[-status_report]  
[-wait_time t]
```

Manages licenses for product options. Specifies licenses to check out, license wait time, and the status of all product option licenses.

Note: This command does not manage multi-CPU licenses. For information on the commands that manage multi-CPU licenses, see the "*Multiple-CPU Processing Commands*" chapter.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-checkout lic</code>	Checks out the licenses for the specified product options when this command runs.
<code>-default</code>	Restores the default. values for this command.
<code>-option_list "lic1 lic2 ..."</code>	Specifies licenses to check out dynamically. Dynamic licenses are not checked out until they are needed. If you specify a license that is not allowed with your base license, or a license that is not available, the software does not check out that license and instead issues a warning message. If you specify more than one license, begin and end the option list with double quotation marks or braces.
<code>- server_license_report</code>	Prints out all the licenses that exist on the license server.
<code>-status_report</code>	Reports the current status of all licenses and the specified wait time. Outputs the following information: • The name of the base license that is checked out • The current wait time • A list of the allowed license options • A list of the available license options • A list of the currently checked-out license options.
<code>-wait_time t</code>	Allows the software to wait for up to <i>t</i> minutes when checking-out an optional license. If the license is available in less than the specified wait time, the system checks out the next needed license without waiting. <i>Default:</i> 0 (no wait time) <i>Value range:</i> 0 to 10,000

Examples

- The following command specifies that licenses for Litmus-L and Litmus-XL may be checked out. Litmus-L licenses are checked out before Litmus-XL licenses.

```
set_license_check -option_list {tpsl tpsxl}
```

- The following command sets the wait time to 5 minutes:

```
set_license_check -wait 5
```

set_message

```
set_message
-help
-id list_of_msgIDs
[-reset_count | -severity {warn error info} | -limit number | -no_limit | -suppress | -
unsuppress | -on_error {msg_only exit stop_script}| -stop_script_limit limitNumber]
```

Specifies to change the message severity levels, `INFO`, `WARNING` or `ERROR`, at the beginning of the message. This command allows you to control the messages that are displayed for a particular design. For example, if there are some warnings that are critical for the design, you can change the message severity to `ERROR`. Similarly, if there are messages that you want to ignore, you can change the message severity to `INFO`.

Parameters

<code>-help</code>	Outputs a brief description that includes type and default information for each <code>set_message</code> parameter. For a detailed description of the command and all of its parameters, use the man command: <code>man set_message</code> .
<code>-id <i>list_of_msgIDs</i></code>	Specifies the list of message IDs that require a change in the message severity level. If you do not specify a message ID, the change in severity level is applied to all message IDs.
<code>-limit <i>number</i></code>	Sets the message limit to <i>number</i> per every public command. The default limit is 20, so any specific message will be written out 20 times for one command and then further output of that message will be disabled (along with an additional message saying the message limit has been exceeded). If the number is set to 0, the message will be completely suppressed for the next public command that triggers the message. You can unsuppress the message by setting the number greater than 0.
<code>-no_limit</code>	Removes any existing limit on the selected messages. Suppressed messages are not unsuppressed.

<code>-on_error {msg_only exit stop_script}</code>	Specifies an extra action for error messages (not for warn or info messages). If no message is specified, the action applies to all error messages. • <code>msg_only</code>: Prints error message only, and the command proceeds as it normally would. This is the default. • <code>exit</code>: Exits immediately and returns to Linux. • <code>stop_script</code>: Waits for the current Tcl command to finish, then stops executing any Tcl script, and returns to the interactive Tcl prompt. If the message severity is changed from <code>error</code> to <code>warn</code> or <code>info</code>, the software will no longer trigger extra action.
<code>-reset_count</code>	Resets the count of the specified message. You can use this option when you want to clear previously generated warnings or errors before beginning the next major step in the design flow.
<code>-severity {warn error info reset}</code>	Specifies the new severity level for the message ID being modified. The <code>reset</code> argument sets the message severity back to its original state.
<code>-stop_script_limit limitNumber</code>	Sets the message limit for a public command. If you set <code>-stop_script_limit</code> for a command as <code>10</code> and message for the command is triggered for the 10th time, the tool will wait for the current Tcl command to finish, then stop executing any Tcl script, and return to the interactive Tcl prompt. The default value of <code>-stop_script_limit</code> is <code>-1</code>, which is unlimited.
<code>-suppress</code>	Suppresses the message.
<code>-unsuppress</code>	Unsuppresses the message.

Examples

- The following command changes the message IMPLF-81 from `ERROR` to `WARN`:

```
set_message -id IMPLF-81 -severity warn
```

- The following command returns all messages to their original severity:

```
set_message -severity reset
```

- The following command resets the specified message to its original severity:

```
set_message -id ENCLF-18 -severity reset
```

Related Commands

- `get_message`

set_rule_handling

```
set_rule_handling  
full_name  
[-limit num]  
[-severity name]  
[-suppress | -unsuppress]
```

Sets the severity level of rule instances or limits the number of occurrences that each rule instance can have.

Options and Arguments

<i>full_name</i>	Specifies the name(s) of the rule instance(s) to configure. This accepts wildcards.
[-limit]	Sets the maximum number of occurrences that a rule instance can have. Zero (0) means no limit, which is the default if not specified.
[-severity]	Sets severity level of the rule instance(s). The specified severity can be Error, Warning, Note, Ignore. If this option is not used, Warning is the default severity level.
[-suppress -unsuppress]	Suppresses or unsuppresses the rule instance(s).

source

```
source  
[-help]  
filename  
[-quiet | verbose]  
[-quiet | -echo ]
```

Reads a file and executes the commands in the file (scripts).

Parameters

-help	Describes the command usage.
-echo	Enables verbose source. If -echo is specified, the tool will ignore it and continue with the source instead of giving an error. Note: This is optional.
<i>filename</i>	Specifies the name of the file to source.
-quiet	Forces source to be normal Tcl source command. Note: This is optional.
-verbose	Forces source to be in verbose mode. Note: This is optional.

suspend

`suspend [-help]`

Suspends your script and returns to the Litmus prompt.

The `suspend` command gives you more flexibility in managing and debugging your scripts. You can insert multiple `suspend` commands in a script. The `suspend` command can also be used in nested scripts. When the script reaches a point where you have inserted the `suspend` command, it stops and restores access to the Litmus prompt. You can then type any command required for debugging.

Whenever you want to resume your script, just type `resume` at the Litmus prompt.

Parameters

<code>-help</code>	Prints the usage of the <code>suspend</code> command. For a detailed description of the command, use the <code>man</code> command: <code>man suspend</code> .
--------------------	--

Related Commands

- `resume`

time_info

```
time_info
[-help]
[stage_name]
[-current]
[-report]
[-stamp]
[-table metric_table_id]
```

Reports and tracks runtime and memory performance.

When the `time_info` command is used without any name, an entry for the stage is created in the table named as "undefined". When the command is used multiple times without any stage name, the stage name is incremented, for example undefined1, undefined2, undefined3. This will ensure that each stage name in the table is unique.

Parameters

<code>-help</code>	Outputs a brief description that includes type and default information for each <code>time_info</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man time_info</code> .
<code>stage_name</code>	Specifies the name of the stage.
<code>-current</code>	Reports the current stage.
<code>-report</code>	Reports all stages up to this point
<code>-stamp</code>	Creates a time stamp.
<code>- table metric_table_id</code>	Specifies metrics table ID to support multiple tables. <i>Default:</i> 'default'

Examples

- The following command shows the usage of the `-current` parameter:

```
litmus > time_info -current

=====

CPU Runtime: 0:00:52.5
Wall Runtime: 0:03:41
Memory Usage: 1387.4 MBytes
Peak Memory Usage: 1505.3 MBytes
Current Time: 12:17:13 (Jul07)

=====
```

- The following command shows the suage of `-report`:

```
litmus > time_info -report

Total Wall | Stage Wall | Wall % | Date - Time      | Memory  | Stage
-----+-----+-----+-----+-----+-----+-----
0:00:00    | 0:00:00    | 0.0%   | 8:27:08 (Jul07)  | 824.3 MB | init
-----+-----+-----+-----+-----+-----+-----

Number of threads: 1 * 1 (id: default)
```

write_db

```
write_db
[-help]
[-keep_partial]
[-no_timing_graph]
[-overwrite]
{{out_dir [[-def ] [-common ]]} | {-oa_lib_cell_view {libname cellname viewname} }}
[-no_rc]
[-lib]
[-user_path]]
```

Saves the files for design import, floorplan, placement, routing, and power switch state information in the specified directory. You can save a design to the same location from which you restored the design. The new data, including the netlist and constraints, overwrite the previous data.

You can use the `write_db` command at any time after importing a design.

❗ Modifying the contents of the `filename.dat` directory created by `write_db` can cause major unexpected results. Never modify the directory contents.

To save a design in OpenAccess database format, use `write_db -oa_lib_cell_view {libname cellname viewname}`.

The `write_db` command saves a tool design in the portable mode by default. In this mode, the design and library data as well as Litmus control files do not have any path dependency. As a result, the saved database can be restored from any directory within the same network. The saved database can also be moved to any directory within the same network without any change. You can set the `write_db_portable` variable to `false` if you do not wish to save the design in portable mode.

The portable `write_db` feature is currently not supported for OpenAccess databases.

Note: The `write_db` command will issue an error message if you try to save a multiple-supply voltage (MSV) design in native format. Instead, create a Common Power Format (CPF) file containing CPF-based MSV commands and load the file, or use the following commands in this specific order to save your native MSV design in a CPF database:

1. `saveCPF`
2. `loadCPF`
3. `commitCPF`
4. `write_db`

For backward compatibility, if you must save the MSV design in a native database, set the following variable before saving the design:

```
setVar dbgSaveOldMsvDBOnly
```

Cadence does not recommend that you do this: use the `saveCPF`, `commitCPF`, `loadCPF`, `write_db` flow instead.

Note: The `write_db` command does not save the state of the `timing_enable_simultaneous_setup_hold_mode` timing attribute. When the attribute is set to true, various physical design functions in the system are disabled. When the design is restored (using the `read_db` command), the physical design commands are required to be active so that the system can be properly initialized.

Parameters

<code>-help</code>	Prints a brief description that includes the type and default information for each <code>write_db</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man write_db</code>.
<code>-common</code>	Saves the common-db design data to the specified directory.
<code>-def</code>	Saves a DEF file, in addition to saving place and route files when you save the design.
<code>-keep_partial</code>	Keeps the partially saved database for debugging purposes only, because restoring this database is not recommended. If the <code>write_db</code> command does not complete successfully, the session data will be removed by default.

<code>-oa_lib_cell_view {libname cellname viewname}</code>	Saves the design in OpenAccess database format using the specified library, cell and view names. The design is saved in the <i>libname/cellname/viewname</i> directory. Note: Usually, the cell name is the same as the design top cell name. If the cell name specified is different from the top cell name, it means that the top module created in OpenAccess would have the new cell name instead of the original name.
<code>out_dir</code>	Represents the user-defined name of the current Litmus session. The recommended file extension is <i>.enc</i>. The associated data directory is <i>dir_name.enc.dat</i>.
<code>-lib</code>	Resolves symlinks of timing library files and side files. When you specify <code>-lib</code> with <code>write_db</code> while saving a design in OpenAccess format, the links under <i>libname/libs</i> directory are replaced with real files. When you specify <code>-lib</code> with <code>write_db</code> while saving a design in Litmus format, the links under <i>filename.enc.dat/libs</i> directory are replaced with real files.
<code>-no_rc</code>	Does not write RC extraction data.
<code>-no_timing_graph</code>	Does not save timing graph information.
<code>-overwrite</code>	Overwrites the information.
<code>-user_path</code>	By default all external file references (such as. library files) are stored as absolute paths without any symbolic-links. So, the DB directory is portable anywhere on the same network. This option stores external file references exactly as you entered it For example, full paths like <i>/x/y</i> or relative paths like <i>../x/y</i>, <i>./x/y</i>), and keeps any symlinks used inside the path names. <i>Data_type</i>: bool, optional Example:

Consider that there are two library files a.lib and b.lib under:

```
/global/project1/libs/mmmc
```

And two lef files c.lef and d.lef under:

```
/global/project1/libs/lef
```

There are two symlinks to the libs:

```
/usa/db1/libs1.0/mmmc/a.lib
```

```
→ /global/project1/libs/mmmc/a.lib
```

```
/usa/libs/mmmc/b.lib           → /global/project1/libs/mmmc/b.lib
```

And two symlinks to the lefs:

```
/usa/db1/libs1.0/lef/c.lef
```

```
→ /global/project1/libs/lef/c.lef
```

```
/usa/libs/lef/d.lef           → /global/project1/libs/lef/d.lef
```

Your current work dir is /usa/db1.1/. You have an init script like:

```
read_mmmc ./mmmc.tcl
```

```
read_physical -lefs
```

```
{/usa/db1/libs1.0/lef/c.lef
```

```
../libs/lef/d.lef}
```

```
read_netlist design.v
```

```
init_design
```

Where mmmc.tcl has:

```
create_library_set -name test -timing
```

```
{/usa/db1/libs1.0/mmmc/a.lib
```

```
../libs/mmmc/b.lib }
```

Then after write_db, without -user_path, will save the libs and lefs to link to the real files.

```
> write_db db1.2
```

The ./mmmc.tcl file is always stored inside the DB dir, so no external reference is used.

By default, external file references are resolved to their fullpath name without any symlinks.

The .lib files inside the mmmc.tcl file will be referenced

```
as: {/global/project1/libs/mmmc/a.lib  
/global/project1/libs/mmmc/b.lib}
```

The .lef files will be referenced

```
as: {/global/project1/libs/lef/c.lef  
/global/project1/libs/lef/d.lef}
```

This is portable and can be loaded by read_db anywhere on the same network.

```
/usa/db1.2/libs1.2/mmmc/a.lib    →  
/global/project1/libs/mmmc/a.lib  
  
/usa/db1.2/libs1.2/mmmc/b.lib    →  
/global/project1/libs/mmmc/b.lib  
  
/usa/db1.2/libs1.2/lef/c.lef     →  
/global/project1/libs/lef/c.lef  
  
/usa/db1.2/libs1.2/lef/d.lef     → /global/pro  
ject1/libs/lef/d.lef
```

With -user_path will save the libs and lefs to keep the symlinks of the current db.

```
> write_db db1.2 -user_path
```

The .lib files will be referenced as entered by the

```
user: {/usa/db1/libs1.0/mmmc/a.lib  
../libs/mmmc/b.lib}
```

The .lef files will referenced as entered by the

```
user: {/usa/db1/libs1.0/lef/c.lef  
../libs/lef/d.lef}
```

So read_db must be run in the same directory where write_db was run, or in a directory with the

same ./y and /x directory structure.

/usa/db1.2/libs1.2/mmmc/a.lib	→	/usa/db1/libs1.0/mmmc/a.lib
/usa/db1.2/libs1.2/mmmc/b.lib	→	../../../../libs/mmmc/b.lib
/usa/db1.2/libs1.2/lef/c.lef	→	/usa/db1/libs1.0/lef/c.lef
/usa/db1.2/libs1.2/lef/d.lef	→	../../../../libs/lef/d.lef

Example

- The following command copies the current design import configuration, floorplan, placement, and route files into directory `test.enc`:

```
write_db test.enc
```

Related Topics

- [read_db](#)
- [write_db](#) Category Attributes

write_metric_config

```
write_metric_config  
[-help]  
  fileName  
[-format {html excel text}]
```

Writes the YAML structure for the requested format (text, html, or excel) to a file. As the configuration is written in YAML format, you can customize the page layout and table content you need in the report such as adding pages and tables, or adding new metrics to existing tables, removing metrics from existing tables, and rearranging the metric ordering.

Parameters

<code>fileName</code>	Specifies the name of the file in which metric config is written.
<code>-format {html excel text}</code>	Specifies the config for specified <code>report_metric</code> format.
<code>-help</code>	Outputs a brief description that includes type and default information for each <code>write_metric_config</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <pre>man write_metric_config</pre>

Example

generates the report in the text format.

```
> write_metric_config -format text small.text ; # Edit the summary table  
> read_metric_config -format text small.yaml  
> report_metric -format text
```

Related Topics

- [read_metric_config](#)
- [report_metric](#)

GUI Commands

- [gui_get_schematic](#)
- [gui_hide](#)
- [gui_schematic_add_selected](#)
- [gui_show](#)
- [read_power_rail_results](#)

gui_get_schematic

```
gui_get_schematic
[-help]
[-current | -all ]
```

Gets the name of the current (active) schematic view. If `-all` is specified, returns the name of all open schematic views.

Parameters

-help	Prints a brief description of the <code>gui_get_schematic</code> command. For a detailed description of the command, use the <code>man</code> command: <code>man gui_get_schematic</code> .
-all	Gets the names of all open schematic views. <i>Data_type:</i> bool, optional
-current	Gets the name of the current active schematic view. <i>Data_type:</i> bool, optional

Related Information

- `gui_schematic_add_selected`

gui_hide

gui_hide

Closes the graphical user interface (GUI) for .

Parameters

-help	Prints out the command usage.
-------	-------------------------------

Related Commands

- `gui_show`

gui_schematic_add_selected

```
gui_schematic_add_selected  
[-help]  
[-name window_name]
```

Adds the selected object to the current or specified schematic view.

Parameters

<code>-help</code>	Prints a brief description of the <code>gui_schematic_add_selected</code> command. For a detailed description of the command, use the <code>man</code> command: <code>man gui_schematic_add_selected</code>.
<code>-name</code> <i>window_name</i>	Specifies the unique window name for an open schematic view to which the selected object needs to be added. <i>Data_type</i>: string, optional

Related Information

- `gui_get_schematic`

gui_show

```
gui_show  
[-help]  
[on | off]
```

Opens the Litmus main window. This is usually done after executing the command `Litmus -init tclCommandFile`.

If you run the `gui_show` command when the Litmus main window is already displayed, the form is refreshed and the display is fitted to the design area.

Note: If you started the Litmus software with the `litmus -gui_show off` command, you cannot then use the `gui_show` command to display the Litmus main window.

Parameters

<code>-help</code>	Prints a brief description that includes type and default information for each <code>win</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man gui_show</code>
<code>on off</code>	Specifies to on or off the window.

Related Commands

- `gui_hide`

read_power_rail_results

```
read_power_rail_results
[-help]
[-detail_delta_temperature_file file]
[-effective_resistance_file file]
[-force]
[-instance_delta_temperature_file file]
[-power_db file ]
[-rail_directory dir]
[-tile_delta_temperature_file file]
[-cell_library { library1.cl}]
[-reset]
[-custom_value_inst_file_1 file]
[-nets net_names]
[-instance_voltage_method { best | worst | avg | worstavg }]
[-instance_voltage_window { timing | whole }]
[-instances instancenames]
[-enable_plots plot_names]
[-signal_em_db file]
[-short_file file]
[-esd_directory dir]
[-thermal_file filename.txt]
[-esd_em dir]
[-die_instance_name dieinstname]
[-data_type {rail | all}]
[-profiling_power_file file]
[-tile_thermal_config_file filename]
[-physical_db dir_name]
[-top_cell_name topcell_name]
[-power_directory power_output_directory]
```

Loading power/rail analysis results into GUI.

Parameters

<code>-help</code>	Outputs a brief description that includes type and default information for each <code>read_power_rail_results</code> parameter. For a detailed description of the command and all of its parameters, use the man command: <code>man read_power_rail_results</code>.
<code>-cell_library { library1.cl }</code>	
	Specifies the single cell library that is required to load the state directory. The specified cell library must contain the technology layer information that is required to load the state directory. Therefore, you do not need to load all cell libraries used during analysis.
<code>-custom_value_inst_file_1 file</code>	
	Specifies the name of the custom value instance file (CVIF). This file can be used to display a user-specified instance voltage file in GUI. The format of the CVIF file is: <pre>value1 instancename1 value2 instancename2</pre> where, <i>value1</i> and <i>value2</i> are voltage values mapping to the specified instance. You can use the <code>cvil</code> argument of the <code>gui_set_power_rail_display -plot</code> command parameter to view the EIV map in GUI.
<code>-data_type {rail all}</code>	
	Controls the type of data to be loaded in the GUI. The possible arguments of this parameter are: • <code>rail</code> - Loads only the rail analysis results. • <code>all</code> - Loads both the power and rail analysis results. This is the default value.

<code>-detail_delta_temperature_file file</code>	
	Specifies the name of the detailed wire-based delta temperature file from self-heating analysis.
<code>-die_instance_name dieinstname</code>	
	Specifies the name of the die whose results you need to view. This parameter is only applicable to the multi-die analysis flow (<code>set_rail_analysis_config -die_mode multi-die</code>). The <code>-die_instance_name</code> parameter must be specified with the <code>-rail_directory</code> parameter (multi-die rail directory).
<code>-effective_resistance_file file</code>	
	Specifies the result file from effective resistance analysis (only instance-based result is supported). When the rail analysis plot type <code>effr</code> is selected and a result file is specified, the layout shows effective resistance analysis result for each instance. Note: Currently, this feature only supports loading one net at a time.
<code>-enable_plots plot_names</code>	
	Specifies the list of power or rail plots names that are to be loaded.
<code>-esd_directory dir</code>	
	Specifies the ESD analysis directory.
<code>-esd_em dir</code>	

	Specifies the name of the directory where the current density/EM results generated by a previous ESD analysis run reside. This parameter allows you to selectively load the required EM result directory if there are multiple ESD EM result directories under the <code>ESD</code> directory. You must either load the rail directory (<code>-rail_directory</code>) before running this command, or specify the rail directory with the <code>-esd_em</code> parameter.
<code>-force</code>	
	Specifies to block the pop-up window that appears when the <code>read_power_rail_results -reset</code> parameter is specified. The pop-up window displays a confirmation message for resetting the power and rail directories. Example: <pre>read_power_rail_results -reset -force</pre>
<code>-instances <i>instancenames</i></code>	
	Specifies a list of instances to load in the GUI. This parameter loads only those partitions that include the specified instances. The <code>-instances</code> parameter allows you to analyze parts of the designs, thereby reducing the loading time and memory footprint. The <code>-instances</code> parameter is applicable to the Voltus-XP flow only.
<code>-instance_delta_temperature_file <i>file</i></code>	
	Specifies the name of the instance-based delta temperature file from self-heating analysis. This is the device instance temperature.
<code>-instance_voltage_method { best worst avg worstavg }</code>	
	Loads the instance voltage data for the specified EIV computing method. If this parameter is not specified (default), the instance voltage results for all the four methods are loaded.

<code>-instance_voltage_window { timing whole }</code>	
	Loads the instance voltage data for the selected EIV window. If this parameter is not specified (default), the instance voltage results for both the timing and switching windows, and the entire rail simulation are loaded.
<code>-nets <i>net_names</i></code>	
	Loads only the specified nets in the domain-based state directory to the GUI. If this parameter is not specified with the <code>-rail_directory</code> parameter, all the nets will be loaded.
<code>-power_db <i>file</i></code>	
	Specifies the power database file. This file was created by using the <code>power_write_db</code> attribute.
<code>-power_directory <i>power_output_directory</i></code>	
	Specifies to load the power analysis results and display the power plots directly without loading the rail analysis results.
<code>-profiling_power_file <i>file</i></code>	
	Specifies to display the tile-based power density results generated from the vector profiling flow.
<code>-physical_db <i>dir_name</i></code>	Specifies the name of the hierarchical design directory to be loaded to the GUI along with the rail analysis results.
<code>-reset</code>	
	Frees the loaded analysis state directory or power database from the memory.
<code>-rail_directory <i>dir</i></code>	
	Specifies the rail analysis directory. You can load all the domain results by selecting the automatically generated analysis directory under the output directory (<code><domain>_<temp>_<dynamic/avg>_#</code>), or load individual nets by going one level further and selecting the net directory.

<code>-short_file file</code>	
	Specifies the path to the short violations report (<code>voltus_short.report</code>) . This file is generated using the <code>set_rail_analysis_config -report_shorts true</code> command parameter, and is located in the work directory.
<code>-signal_em_db file</code>	
	Specifies to read the signal electromigration data from the specified database file generated by the <code>check_ac_limits -report_db</code> command parameter.
<code>-thermal_file filename.txt</code>	
	Specifies to load the tile-based temperature map file generated by Sigrity PowerDC to be used for both power calculation and IR drop analysis. The following is an example of reading the thermal map file: <pre>read_power_rail_results -rail_directory output/static_rail/core_25C_avg_1/ -thermal_file output/power_map_by_layer.txt gui_set_power_rail_display -plot thermal</pre>
<code>-tile_delta_temperature_file file</code>	
	Specifies the name of the tile-based delta temperature file of self-heating analysis. This is the worst instance temperature among all wire segments in a tile. This is also layer-based. The FEOL or cell layer is used to show the device instance temperature.
<code>-tile_thermal_config_file filename</code>	

	Specifies to load the configuration file that contains the mapping between the hierarchical levels with corresponding layer-based temperature map files generated by Celsius (system-level thermal analysis tool). When specified, you can view tile-based temperature distribution for each layer of the chip. The following is the format and example of the hierarchical configuration file: <pre><hierarchy> <block name> <X_tile_no,Y_tile_no> <LEF/DEF Flip/Rotation> <temperature map> TOP DIE 0,0 N /thermal_data/DIE_TemperatureMap TOP/IP1 IP1 25,50 N /thermal_data/IP1_TemperatureMap TOP/IP1/B1 B1 50,125 N /thermal_data/B1_TemperatureMap TOP/IP1/B2 B2 200,125 S /thermal_data/B2_TemperatureMap</pre> This is a mandatory parameter for the <code>tile_thermal</code> plot when loading hierarchical temperature maps.
<pre>-top_cell_name topcell_name</pre>	
	Specifies the name of the top module or top cell name to be loaded. This parameter must be specified with the <code>-physical_db</code> parameter.

Examples

- The following command specifies the appropriate files needed for viewing the instance voltage and power data:

```
read_power_rail_results\  
-rail_directory analysis/domain_25C_avg_1 \  
-power_db powermeter.db \
```

- The following command specifies to load only the rail directory:

```
read_power_rail_results -rail_directory dynamic_rail/core_25C_dynamic_1 -data_type  
rail
```

- The following command specifies to display the tile-based power density results in the GUI:

```
read_power_rail_results -profiling_power_file  
./vectorProfileResults/profile.rpt.density
```

- The following command specifies to load the power analysis results and display the power plots directly by running the power analysis without loading the rail analysis results:

```
read_power_rail_results -power_directory ./dynamic_power
```

Related Topics

- "[Static Power, IRdrop and EM Analysis](#)" in the *Voltus User Guide*

Import and Export Commands

- [read_def](#)
- [read_physical](#)
- [relink_db_files](#)
- [write_mmmc](#)

read_def

```
read_def
[-help]
[def_file]
[-design]
[-design_offset (x y)]
[-design_orient orientation]
[-force_create_phys_inst]
[-keep_pin_geometry]
[-preserve_shape]
[-rdl_def def_file]
[-rdl_offset (x y)]
[-rdl_orient orientation]
[-read_flat_dir directoryName]
[-skip_pg]
[-skip_signal_nets]
[-top_scope | -top_scope_ignore_block_internal_nets_on_boundary_path ]]
[-write_flat_dir directoryName]
```

Reads a single DEF file. This command can be used to supply single DEF to generate a physical database.

This command can be used at any step after importing a design. The typical usage is:

- Load a DEF file containing the floorplan saved by Tempus or other tool.
- Load a DEF file containing the scan chain information so that placement can do scan chain reorder.

- Load a DEF file containing the cell's placement and routing saved by Tempus or other tool.

The `read_def` command cannot read multiple or hierarchical DEF files. To read multiple or hierarchical DEF files and create a flattened database, use the `read_hierarchical_def` command.

When you load a DEF file, you must be at the same level in the hierarchy where the DEF file was saved.

The Tempus software loads the DEF file information into the database in several ways, depending on the object:

- Tracks, gcells, rows, die area
The software deletes all of the existing objects in the database and reads in the new information.
Note: The `read_def` command can read a rectilinear die area, but converts it to a rectangular die area with blockages at the cut off area.
- Blockages, fills, special nets
The software compares the physical data (such as shape and layer) to the existing data in the database. If it finds the same data in the database, it ignores the data in the DEF file. If the data does not already exist, the software adds it to the database. For special nets, the software merges wire segments in the DEF file with any existing ones in the database that overlap and have the same attributes, such as layer, width, shape, and direction. The software does not delete any existing data.
- Nondefault rules, vias
The software adds the objects to the database if they do not already exist there. If a nondefault rules definition with the same name already exists, the software ignores the one in the DEF file. If a via definition with the same name already exists, and the via is a fixed via, the software ignores the one in the DEF file. If a generated via definition with the same name already exists, the geometries are read, but the name might be changed. The software does not delete any existing objects of these types.
- Nets
For each net in the DEF file, the software removes the existing regular routing (not special routing), then adds the routing from the DEF file for this net.
- Pins
For each pin in the DEF file, the software removes the existing physical information for the pin, then adds the physical information from the DEF file for this pin. If a pin does not already exist,

and it is a power or ground pin, the software adds it to the database. If the pin is not a power or ground pin, the software generates an error message.

- **Groups, regions**

If a group name in the DEF file matches an existing hierarchical instance name in the database, the region's boundary constraint is attached to the hierarchical instance, overriding any existing boundary constraint. The software checks whether the group members belong to the hierarchical instance, and generates a warning message if any do not belong. If a group name does not match any hierarchical instance name, the software creates the instance group with an attached region boundary constraint, if one exists. Regions that are not used by groups in the same DEF file are ignored. The software does not remove existing groups.

Note: You can import DEF files containing wildcards (*) in the `GROUPS` statement. The `read_def` command does not support multiple groups per region.

- **Components**

You cannot add new logical cells. You can add new physical cells, such as:

- Cells with `CLASS PAD`
- Cells with `CLASS COVER` and no signal pin
- Instance with `+ SOURCE DIST`

You can use the `read_def` command after importing a design.

❗ After loading the DEF netlist using the `read_def -verilog_from_def_netlist_flow` command, you can perform floorplanning, non-timing driven placement and routing, wire Innovusting, and verification. You cannot use the DEF netlist flow for parasitic extraction, delay calculation, and timing-driven placement and routing effectively because the DEF names do not properly match the Verilog names used in timing constraints and timing analysis.

Parameters

-help	Prints a brief description that includes the type and default information for each <code>read_def</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man read_def</code>
<i>def_file</i>	Specifies the names of the DEF files to load.
-design	Specifies to read def files with the same top cell name
-design_offset (x y)	Specifies the placement for the top cell.
-design_orient orientation	Specifies the orientation for the top cell.
-force_create_phys_inst	Specifies to force create physical cell instances.
-keep_pin_geometry	Keeps pins shapes equivalent to the block pin shapes, in the form of special route.
-preserve_shape	Prevents creation of symbolic center-line connected routing data and preserves the routing data as-is.
-rdl_def def_file	Specifies the RDL def file.
-rdl_offset (x y)	Specifies the offset of RDL def.
-rdl_orient orientation	Specifies the orientation for RDL def.
-read_flat_dir directoryName	Reads flat data from the specified directory.
-skip_pg	Skips merging routing on pg nets.
-skip_signal_nets	Specifies to skip reading signal nets.
-top_scope	Specifies to merge routing on top scope nets only.
-top_scope_ignore_block_internal_nets_on_boundary_path	Specifies merging routing on top scope nets which are not block internal nets on boundary path.
-write_flat_dir <i>directoryName</i>	Writes flat data to the specified directory.

Examples

- The following command reads the compressed DEF file `dma_mac_filled.def.gz`:

```
tempus > read_def dma_mac_filled.def.gz
```

Related Commands

- [read_hierarchical_def](#)
- [write_def](#)

read_physical

```
read_physical  
[-help]  
{[-oa_ref_libs lib1 [lib2] ... [-oa_search_libs lib1 [lib2] ...] [-oa_tech_lib lib1 [lib2]  
...]] | -lefs lib1 [lib2] ... /[-add_oa_ref_libs lib1 [lib2]... [-oa_lib_path libPath]] | -  
add_lefs lib1 [lib2] ...}
```

Reads the physical library information. When this command is run, the physical library information is loaded into the database. Both LEF and OA are supported, but not simultaneously.

The `read_physical` command corresponds to the *Physical* step of the Initialization flow:



During the *Physical* step, the physical libraries (OA or LEF) are loaded into the tool's database using `read_physical`. When the command is issued, the database is loaded with the specified objects, and is available for querying of attributes associated with the physical data, such as size, layer details, and so on.

The tool uses the standard OA parser, and performs full error checking on the physical data. When the command is complete, a full timing library set and a physical library set would have been loaded into the database.

If `-lef` is used, the technology LEF must be first specified.

This command is optional for Synthesis and STA, but is required for Implementation.

The tool can support a physical only mode when the attribute `init_physical_only` is set to `true`. This attribute must be set prior to `read_physical`, otherwise an error will be issued.

Parameter

<code>-help</code>	Prints a brief description that includes type and default information for each <code>read_physical</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man read_physical</code>
<code>-add_lefs lib1 [lib2]</code>	Specifies a list of LEF files to read and to add to the original list of LEF files that were added before <code>init_design</code>. This option can only be used after <code>init_design</code>, and cannot be combined with any other option. You can add new macros, non-default rules, sites etc. using this option, but you cannot use it to modify any existing LEF data that is already loaded. The LEF files added using this option will be used when the saved design is loaded again using <code>read_db</code>.
<code>-add_oa_ref_libs lib1 [lib2]</code>	Specifies the list of OpenAccess library names to be loaded incrementally. These library names are added in the <code>cds.lib</code> file, if not present. Use this parameter to specify standard cell libraries or I/O driver libraries that should all be loaded after the design is read.

<code>-lefs lib1 [lib2] ...</code>	Specifies the list of LEF files to be loaded. The first LEF file must have the technology section present. The file names are saved to the <code>init_lef_files</code> root attribute. This option cannot be combined with <code>-oa_ref_libs</code>. Note: This option can only be used before <code>init_design</code> or <code>read_db</code>. If you specify this option after <code>init_design</code>, the tool will give an error.
<code>-oa_ref_libs lib1 [lib2] ...</code>	Specifies the list of OpenAccess library names to be loaded. These names must also be listed in the <code>cds.lib</code> file. All the abstract views in the specified libraries are loaded. Use this parameter to specify standard cell libraries or I/O driver libraries that should all be loaded, even if some of the cells are not currently used in the netlist. The first OpenAccess library that is specified is also used to load the OpenAccess technology data. The specified library names are saved to the <code>init_oa_ref_libs</code> root attribute.
<code>-oa_lib_path libPath</code>	Specifies the path of oa reference libraries. This option can only be used with the <code>-add_oa_ref_libs</code> option.
<code>-oa_search_libs lib1 [lib2] ...</code>	Specifies the list of OpenAccess libraries to be searched. This parameter can be specified only with <code>-oa_ref_libs</code>. As the netlist is read, these libraries are searched for cell abstracts to be load on demand for any Verilog module that does not have a <code>lib_cell</code> already defined. This parameter is typically used for libraries of block IP, such as RAMs, where only the abstracts that are actually used in the netlist need to be loaded.
<code>-oa_tech_lib lib1 [lib2] ...</code>	Specifies the OA tech file to use if the netlist is coming from Verilog rather than an OA cellview.

Related Topics

- [init_design](#)
- [read_db](#)

relink_db_files

```
relink_db_files  
[-help]  
-db_dir db_directory  
-new_lib_dirs {new_lib_directory1 new_lib_directory2 ...}
```

Relinks symbolic links to library files inside a DB directory.

By default, `write_db` saves the location of library files as symbolic links under `<db_dir>/libs/...`. When a DB directory is copied to a new network where all the libraries are in a different location, or a new version of the library files are needed, you can use `relink_db_files` to change the symbolic links to new locations .

The `relink_db_files` command will search each `-new_lib_dirs` directory and the sub-directory tree below it for each library file name inside `<db_dir>`, and create a symbolic link to the new location. Only the base file name is used for the search (except for `qrcTechFile`). Therefore, two path names inside the new directory, such as `/timing/fast/std.lib` and `/timing/slow/std.lib`, will have duplicate names of “`std.lib`”. The duplicate names are not supported and will cause an error. You can use the `read_db` command to update all the LEF full-path names or use a new MMMC files with all the new .lib full-path names if duplicate file names need to be updated. If there are missing library file names, the tool will show an error.

The `relink_db_files` command supports `.lib`, `.lef`, and `qrcTechFile` library file types.

Parameters

<code>-help</code>	Prints a brief description that includes the type and default information for each <code>relink_db_files</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man relink_db_files</code>
<code>-db_dir <i>db_directory</i></code>	The current db directory which contains incorrect library symbolic links.
<code>-new_lib_dirs</code> <code>{<i>new_lib_directory1 new_lib_directory2 ...</i>}</code>	The new directories to search for library files.

Example

If the current LEF files are all under `/libs/lefs_v1.0/` (or sub-directories) and the current `.lib` and `qrcTechFiles` are all under `/libs/timing_v1.0/` (or sub-directories), you use the following command to update the LEF files to use `/libs/lefs_v2.0/`:

```
>relink_db_files -db_dir my_db -new_lib_dirs {/libs/lefs_v2.0 /libs/timing_v1.0}
```

Note: The command must find every library file. Therefore, you need to list the top of the old timing directory tree along with the top of the new LEF files directory tree.

Related Information

- `write_db`
- `read_db`

write_mmmc

write_mmmc

[-help]

[-library]

Writes out the MMMC file.

Parameters

-help	Prints out the command usage.
-library	Writes out only the library part of the MMMC file.

Related Topic

- [read_mmmc](#)

Miscellaneous

- [analyze_paths_by_bottleneck](#)
- [check_timing_model](#)
- [init_hier_design](#)

analyze_paths_by_bottleneck

```
analyze_paths_by_bottleneck  
[-help]  
{-category n | -delta_delay {worst|total}}  
[-path_num m]  
[-max_slack float]  
[-min_slack float]  
[-master master_category_name]
```

Creates categories according to the bottleneck analysis of the critical paths and fixes the violation according to the bottleneck information. Bottleneck Analysis detects the instances that are often involved in the critical paths in the design. Any change in timing for these instances impacts multiple critical paths in the design. Therefore you can use this information to change your design (floorplan, placement, constraints etc.) around these instances. The bottleneck analysis computes the number of occurrences that an instance appears in the paths that you display in the Timing Debug window, and provides the following output:

- Colors most used instance based on number of occurrences.
- Creates *n* number of path categories for the instances with the highest number of occurrences. *n* is the number you specify using the `-category` parameter.

Users can also perform bottleneck analysis of the critical paths based on incremental delay. Categories will be created based on the worst *m* paths with:

- The biggest worst `incr_delay` contribution
- The biggest total `incr_delay` contribution

The command can be applied wherever the user loads the machine readable timing report.

Parameters

<code>-help</code>	Outputs a brief description that includes type and default information for each <code>analyze_paths_by_bottleneck</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <pre>man analyze_paths_by_bottleneck</pre>
<code>-category <i>n</i></code>	Specifies the number of categories to be created for most occurring instances.
<code>-delta_delay {worst total}</code>	Specifies that the categories are created based on worst <code>incr_delay</code> contribution or total <code>incr_delay</code> contribution.
<code>-master <i>master_category_name</i></code>	Specifies that the analysis should be performed only for the specified <code>master</code> category.
<code>-max_slack <i>float</i></code>	Specifies the max slack value of the critical paths for bottleneck analysis based on <code>incr_delay</code>.
<code>-min_slack <i>float</i></code>	Specifies the min slack value of the critical paths for bottleneck analysis based on <code>incr_delay</code>.
<code>-path_num <i>m</i></code>	Specifies the number of critical paths to be analyzed when performing bottleneck analysis based on <code>incr_delay</code>.

Examples

- The following command creates path categories based on bottleneck analysis of the critical paths on the instances:

```
analyze_paths_by_bottleneck -category 2
```

The following categories are created: `BOT_instance1` and `BOT_instance2`. You can change the number of categories.
- The following command creates path categories based on bottleneck analysis of the critical 20 paths on the worst `incr_delay`:

```
analyze_paths_by_bottleneck -path_num 20 -delta_delay worst
```

The following category is created: `BOT_SI_WORST`.

Related Commands

- [create_path_category](#)
- [delete_path_category](#)

check_timing_model

```
check_timing_model
[-absolute_cap_tolerance float]
[-absolute_tolerance float]
[-absolute_tolerance_optimistic float]
[-absolute_tolerance_pessimistic float]
[-absolute_transition_tolerance float]
-compare_file string
[-ignore_view]
-out_file string
[-percentage_cap_tolerance float]
[-percentage_tolerance float]
[-percentage_tolerance_optimistic float]
[-percentage_tolerance_pessimistic float]
[-percentage_transition_tolerance float]
-reference_file string
```

Compares two reports that contain interface timing characteristics generated by the `report_timing_model` command.

This command compares the interface timing report generated by using the `report_timing_model` command on the original netlist with the interface timing report generated by using the `report_timing_model` command on the model extracted for the same design.

Parameters

-help	Outputs a brief description that includes type and default information for each <code>check_timing_model</code> parameter. For a detailed description of the command and all of its parameters, use the man command: <code>man check_timing_model</code> .
-------	--

<code>-absolute_cap_tolerance float</code>	Specifies the tolerance value for capacitance (as an absolute value) while comparing two report files. This parameter specifies the permissible difference, as an absolute value, between the capacitance values of two report files. If the difference in the value of capacitance exceeds the absolute tolerance value specified, the comparison status is set to <code>FAIL</code>. Note: When this parameter is not specified, the software uses the absolute value specified with the <code>-absolute_tolerance</code> parameter. If the <code>-absolute_tolerance</code> parameter is not specified, the value defaults to 0.  When the <code>-percentage_cap_tolerance</code> and <code>-absolute_cap_tolerance</code> parameters are used together, the comparison status is set to <code>FAIL</code> only when the difference in the value of capacitance exceeds the specified absolute as well as percentage tolerance values.
<code>-absolute_tolerance float</code>	Specifies the absolute tolerance value. The tolerance value is used when there is a difference between the timing characteristics of the reference file (<code>-reference_file</code>) and the file that is being compared (<code>-compare_file</code>) to determine the <code>PASS</code> or <code>FAIL</code> status of the comparison. If the difference in the value of a timing characteristic exceeds the absolute tolerance value, the comparison status is set to <code>FAIL</code>. Note: When used with the <code>-percentage_tolerance</code> option, the difference in the value of a timing characteristic should exceed both absolute and percentage tolerance values for the comparison result status of an arc to be <code>FAIL</code>. <i>Default:</i> 0

<code>-absolute_tolerance_optimistic</code> <i>float</i>	Specifies the absolute tolerance limit for flagging an optimistic failure. An optimistic path violating absolute as well as percentage tolerance limit will be flagged as FAIL in the report output. <i>Default:</i> 15ps
<code>-absolute_tolerance_pessimistic</code> <i>float</i>	Specifies the absolute tolerance limit for flagging a pessimistic failure. A pessimistic path violating absolute as well as percentage tolerance limit will be flagged as FAIL in report output. <i>Default:</i> 15ps
<code>-absolute_transition_tolerance</code> <i>float</i>	Specifies the tolerance value for transition time at ports (as an absolute value) while comparing two report files. This parameter specifies the permissible difference, as an absolute value, between the transition time at ports of two report files. If the difference in the transition time exceeds the absolute tolerance value specified, the comparison status is set to <code>FAIL</code>. Note: When this parameter is not specified, the software uses the absolute value specified with the <code>-absolute_tolerance</code> parameter. If the <code>-absolute_tolerance</code> parameter is not specified, the value defaults to 0. When the <code>-percentage_transition_tolerance</code> and <code>-absolute_transition_tolerance</code> parameters are used together, the comparison status is set to <code>FAIL</code> only when the difference in the value of transition exceeds the specified absolute as well as percentage tolerance values.

<code>-compare_file <i>string</i></code>	Specifies the name of the report file that contains the interface timing characteristics extracted from the model created using the <code>write_timing_model</code> command. The timing characteristics of this file are compared with the timing characteristics of the reference file that you specify with the <code>-reference_file</code> option.
<code>-ignore_view</code>	Compares non-MMMC design <code>report_timing_model</code> output report with the MMMC design output report (of <code>report_timing_model</code> command). When you specify this parameter, the software ignores the view information while comparing the reports. So you do not require to create dummy views for timing model validation flow.
<code>-out_file <i>string</i></code>	Specifies the name of the output file in which the comparison results will be generated. This file contains the <code>PASS</code> or <code>FAIL</code> status for the timing data comparison.

<code>-percentage_cap_tolerance value</code>	Specifies the tolerance value for capacitance (as a percentage) while comparing two report files. This parameter specifies the permissible difference, in percentage, between the capacitance values of two report files. If the percentage difference in the value of capacitance exceeds the percentage tolerance value specified, the comparison status is set to <code>FAIL</code>. Note: When this parameter is not specified, the software uses the percentage value specified with the <code>-percentage_tolerance</code> parameter. If the <code>-percentage_tolerance</code> parameter is not specified, the value defaults to 0. When the <code>-percentage_cap_tolerance</code> and <code>-absolute_cap_tolerance</code> parameters are used together, the comparison status is set to <code>FAIL</code> only when the difference in the value of capacitance exceeds the specified absolute as well as percentage tolerance values.
<code>-percentage_tolerance float</code>	Specifies the percentage tolerance value for comparing the timing data of two report files. The percentage tolerance value is used when there is a difference between the timing characteristics of the reference file (<code>-reference_file</code>) and the file that is being compared (<code>-compare_file</code>) to determine the <code>PASS</code> or <code>FAIL</code> status of the comparison. If the percentage difference in the value of a timing characteristic exceeds the percentage tolerance value, the comparison status is set to <code>FAIL</code>. Note: When used with the <code>-absolute_tolerance</code> option, the difference in the value of a timing characteristic should exceed both percentage and absolute tolerance values for the comparison result status of an arc to be <code>FAIL</code>. <i>Default:</i> 0

<code>-percentage_tolerance_optimistic</code> <i>float</i>	Specifies the percentage tolerance limit for flagging an optimistic failure. An optimistic path violating percentage as well as absolute tolerance limit will be flagged as FAIL in the report output. Default: 3%
<code>-percentage_tolerance_pessimistic</code> <i>float</i>	Specifies the percentage tolerance limit for flagging a pessimistic failure. A pessimistic path violating percentage as well as absolute tolerance limit will be flagged as FAIL in the report output. Default: 3%
<code>-percentage_transition_tolerance</code> <i>float</i>	Specifies the tolerance value for transition time at ports (as a percentage) while comparing two report files. This parameter specifies the permissible difference, in percentage, between the transition time at ports of two report files. If the percentage difference in the transition time exceeds the percentage tolerance value specified, the comparison status is set to <code>FAIL</code>. Note: When this parameter is not specified, the software uses the percentage value specified with the <code>-percentage_tolerance</code> parameter. If the <code>-percentage_tolerance</code> parameter is not specified, the value defaults to 0. When the <code>-percentage_transition_tolerance</code> and <code>-absolute_transition_tolerance</code> parameters are used together, the comparison status is set to <code>FAIL</code> only when the difference in the value of transition exceeds the specified absolute as well as percentage tolerance values.

<code>-reference_file string</code>	Specifies the name of the reference file that is used for comparison. This is the report file that was generated by using the <code>write_timing_model</code> command on the original design. The file specified with the <code>-compare_file</code> option is compared with this file.
-------------------------------------	---

Examples

- The following command compares the timing characteristics of the `sample_compare.rpt` file with the `sample_reference.rpt` file and generates the comparison results in the

`sample_output.rpt` file:

```
tempus > check_timing_model -reference_file sample_reference.rpt -compare_file
sample_compare.rpt
```

```
-out_file sample_output.rpt
```

```
#####
```

```
# check_timing_model report
```

```
# Design : netlist = topcell =
```

```
# Version : 07.10-dev.indfe_cm
```

```
# Analysis Mode : setup
```

```
# Date : Wed Mar 14 15:56:59 IST 2007
```

```
#####
```

```
#Section 1: Worst-case Arc-Delay
```

```
#Tolerance Percent: 0.0%
```

```
#Tolerance Absolute: 0.0
```

```
#NOTE: The various columns list values in the form 'X[Y]', where:
```

```
# X = value that is compared.
```

```
# Y = reference value.
```

```
#NOTE2: If a comparison fails (i.e. status = FAIL), then the clauses
```

```
# (i.e. 'X[Y]' pairs) that caused the failure are identified with
```

```
# a '*' in front of them.
```

```
#From    To      Arc-Type    Transition    Arc-Delay      abdDiff    %Diff    Status
```

```
#####
```

```
in      CLK3      setup      fall/rise    0.134[0.134]    0.000    0.000%    PASS
in      CLK3      setup      rise/rise    -0.005[-0.005]   0.000   -0.000%    PASS
```

```
#####
```

```
#Section 2: Pin-capacitances
```

```
#Tolerance: 0.0%
```

```
#NOTE: The various columns list values in the form 'X[Y]', where:
```

```
# X = value that is compared.
```

```
# Y = reference value.
```

```
#NOTE2: If a comparison fails (i.e. status = FAIL), then the clauses
```

```
# (i.e. 'X[Y]' pairs) that caused the failure are identified with
```

```
# a '*' in front of them.
```

```
#
```

```
#Pin C(total)[rise]  C(total)[fall]  Status
```

```
#####
```

```
clk    0.002[0.002]  0.002[0.002]    PASS
```

```
in     0.002[0.002]  0.002[0.002]    PASS
```

```
out    0.000[0.000]  0.000[0.000]    PASS
```

```
#####
```

```
#Section 3: Transition-times
```

```
#Tolerance: 0.0%
```

```
#NOTE: The various columns list values in the form 'X[Y]', where:
```

```
# X = value that is compared.
```

```
# Y = reference value.
```

```
#
```

```
#NOTE2: If a comparison fails (i.e. status = FAIL), then the clauses
```

```
# (i.e. 'X[Y]' pairs) that caused the failure are identified with
```

```
# a '*' in front of them.
```

```
#
```

```
#Pin  Rise      Fall      Status
```

```
#####
```

```
clk    0.000[0.000] 0.000[0.000] PASS
```

```
in     0.000[0.000] 0.000[0.000] PASS
```

```
out    0.056[0.056] 0.051[0.051] PASS
```

```
#####
#Section 4: Design Rules
#Tolerance: 0.0%
#NOTE: The various columns list values in the form 'X[Y]', where:
# X = value that is compared.
# Y = reference value.

#NOTE2: If a comparison fails (i.e. status = FAIL), then the clauses
# (i.e. 'X[Y]' pairs) that caused the failure are identified with
# a '*' in front of them.
#
#Pin      MaxCap    MinCap    MaxTrans    MaxFanout    FanoutLoad    Status
#####
clk       NA[NA]    NA[NA]    NA[NA]      NA[NA]       NA[NA]        PASS
in        NA[NA]    NA[NA]    NA[NA]      NA[NA]       NA[NA]        PASS
out       NA[NA]    NA[NA]    NA[NA]      NA[NA]       NA[NA]        PASS

#####
# SUMMARY
#
# Check-Type Passed Failed N/A Total
#####
Worst-Slacks 2 0 0 2
Capacitances 3 0 0 3
Trans-Times  3 0 0 3
Design-Rules 3 0 0 3
```

Related Commands

- [report_timing_model](#)
- [write_timing_model](#)

init_hier_design

```
init_hier_design
[-help]
[-add_on_shape_def string]
-defs def_files
[-design_offset {x y}]
[-design_orient orient]
[-keep_blockages]
[-rdl_def def_file]
[-rdl_offset {x y}]
[-rdl_orient orient]
[-skip_signal_nets]
[-top_cell name]
```

<description>

Parameters

-add_on_shape_def <i>string</i>	Specify add-on-shape DEF files to be loaded Example: {{def_1 x_loc y_loc orient} {def_2 x_loc y_loc orient} ... }
-defs <i>def_files</i>	Specify DEF files to be loaded.
-design_offset { <i>x y</i> }	Specify placement for <i>top_cell</i> .
-design_orient <i>orient</i>	Specify orientation for <i>top_cell</i> .
-keep_blockages	Loads blockages.
-rdl_def <i>def_file</i>	Specify RDL def file.
-rdl_offset { <i>x y</i> }	Specify offset of RDL def.
-rdl_orient <i>orient</i>	Specify orientation for RDL def.
-skip_signal_nets	Skips loading signal nets.
-top_cell <i>name</i>	Specify name of <i>top_cell</i> .

Examples

<examples>

Related Commands

<input>

Multiple-CPU Processing Commands

- [report_resource](#)
- [set_multi_cpu_usage](#)

report_resource

```
report_resource  
[-help]  
[-peak]  
[-verbose]  
[-start string | [-end string [-session_cpu] [-diff_memory]]]  
[-check_MemCpu {true | false} [-period integer]  
[-memory_threshold float]]
```

Displays the peak memory used during a software session. This command is used to measure the peak memory at any given point in the flow and and make the script better for automation flows.

Parameters

-help	Outputs the command usage and a brief description about the command parameters.
- check_memory_cpu {true false}	Checks the machine's physical memory and CPU resources. When specified, the software enables memory/CPU checking and reports a warning if the free memory is below 5 percent of the total memory or if the average CPU load is large.
-diff_memory	Outputs different peak memory and current memory between the step start and step end.
-end <i>string</i>	Specifies the end point to measure the peak memory attained during a specific command.
- memory_threshold <i>float</i>	Specifies the threshold as ratio of total machine/LSF memory. A warning is issued if the available memory goes below this threshold. <i>Min: 0</i> <i>Max: 1</i>
-peak	Reports current peak memory till a given point in a flow. This parameter also works with the -start and -end parameters of the report_resource command so that peak memory observed by an individual command can be reported.

<code>-period integer</code>	Specifies the period for checking the physical memory and CPU usage. <i>Default:</i> 1800s <i>Min:</i> 1 <i>Max:</i> INT_MAX
<code>-session_cpu</code>	Reports the current CPU and real-time after the reports' last step status.
<code>-start string</code>	Specifies the start point to measure the peak memory attained during a specific command. Note: The <code>-start</code> and <code>-end</code> parameters support any user-specified string, such as <code>fullFlow</code>, <code>report_timing</code>, and <code>readDesign</code>. You must specify the same string for both the <code>-start</code> and <code>-end</code> parameters to measure the peak memory attained during a specific command.
<code>-verbose</code>	Gives detailed memory usage information. It displays how much memory is being used at any time and of what form (physical vs. virtual). When specified, the following information is displayed: • <code>Cpu(s)</code> is the number of available processors in the machine. • <code>Load average</code> is the system load averages for the last 1 minute, 5 minutes, and 15 minutes. • <code>Mem</code> and <code>Swap</code> are the current memory information of the machine. • <code>Data Resident Set (DRS)</code> is the amount of physical memory devoted to other than executable code. "<code>current mem</code>" shows this value (Total current DRS) . • <code>Private Dirty (DRT)</code> is the memory which must be written to disk before the corresponding physical memory location can be used for some other virtual page. "<code>peak res</code>" shows this value (Total peak DRT). This is the minimum number that you must reserve to run the program. • <code>Virtual Size (VIRT)</code> is the total amount of virtual memory used by the task. • <code>Resident Size (RES)</code> is the non-swapped physical memory a task has used. The number of "Total Peak RES" is the recommended physical memory to reserve.

Examples

- The following command specifies to display the memory used during a software session:

```
litmus> report_resource
```

The following message is displayed:

```
Current (total cpu=0:04:19, real=18:28:46, peak res=3540.1M, current mem=3403.7M)
```

- The following command specifies to display the peak memory up to a specified point in a flow:

```
litmus> report_resource -peak
```

The following message is displayed:

```
Current peak res=3540.1M
```

- The following command specifies to display the detailed memory information:

```
litmus> report_resource -verbose
```

The following output is displayed:

```
Current (total cpu=0:04:19, real=18:30:07, peak res=3540.1M, current mem=3403.7M)
```

```
Cpu(s) 40, load average (past 1 minute): 4.25, load average (past 5 minutes):  
5.66, load average (past 15 minutes): 10.70
```

```
Mem: 790948060k total, 741280860k used, 49667200k free, 7512k buffers  
Swap: 134217724k total, 18325308k used, 115892416k free, 244555880k cached
```

Memory Detailed Usage :

	Data Resident Set (DRS)	Private Dirty (DRT)	Virtual
Size (VIRT)	Resident Size (RES)		
Total current :	5575.0M	2338.3M	9711.3M
3403.7M			
peak :	5761.5M	2364.6M	9897.9M
3540.1M			

- The following command specifies to display the end point to measure the peak memory attained during the `report_timing` command.

```
litmus> report_resource -end report_timing
```

The following message is displayed:

```
Ending "report_timing" (total cpu=0:00:00.7, real=0:03:57, peak res=3430.0M,  
current mem=3403.7M)
```

- The following command script specifies to display the peak memory attained during the `report_timing` command:

```
litmus > report_resource -start report_timing
```

```
litmus > report_timing -machine_readable -max_paths 100
litmus > report_resource -end report_timing
```

The following message is displayed:

```
Ending "report_timing" (total cpu=0:00:55.9, real=0:00:56.0, peak res=1032.5M,
current mem=1213.2M)
```

- The following command script specifies to display the peak memory attained till the `report_timing` command:

```
litmus > report_timing -machine_readable -max_paths 100
litmus > report_resource -peak
```

The following message is displayed:

```
Current peak res=1032.4M
```

- The following command script specifies to display CPU time/memory information during `Test`:

```
litmus > report_resource -start Test
litmus > report_resource -start read_design
litmus > report_resource -end read_design
litmus > report_resource -start report_timing
litmus > report_timing -machine_readable -max_paths 100
litmus > report_resource -end report_timing
litmus > report_resource -end Test
```

The following message is displayed:

```
Ending "report_timing" (total cpu=0:00:55.8, real=0:00:56.0, peak res=1032.4M,
current mem=1213.1M)
Ending "Test" (total cpu=0:01:37, real=0:01:38, peak res=1032.4M, current
mem=1213.1M)
```

Related Commands

- [report_timing](#)

set_multi_cpu_usage

```
set_multi_cpu_usage  
[-help]  
[-local_cpu integer]  
[-keep_license {true|false}]  
[-release_license]
```

Specifies the number of threads to use for multi-threading.

This command is required for multi-threading.

The `set_multi_cpu_usage` command enables multi-threaded timing reporting under the following conditions:

- `timing_report_group_based_mode` attribute is set to `true`. If there are multiple paths with the same slack then the selection can change on multiple runs. This may result in some changes in the `report_timing` output.
- `report_analysis_coverage` command is in non-verbose mode.
- `check_timing -include clock_crossing` parameter is specified.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-local_cpu</code> {integer}	Specifies the number of CPUs on the local machine . This parameter is required for multi-threading. <i>Default:</i> 4
<code>-keep_license</code> {true false}	Specifies whether to keep the acquired multiple CPU-licenses until the current session ends. Specify this parameter before running any commands that require multiple-CPU applications. To release all multiple-CPU licenses immediately, use -release_license option. <i>Default:</i> true
<code>-release_license</code>	Releases all multiple-CPU license(s) immediately. By default, the software holds multiple-CPU licenses until the end of the current session. To specify that the software release multiple-CPU licenses after every multiple-CPU command runs, use -keep_license option.

Examples

Here are some examples.

- The following command specifies that 8 CPUs can be used on a local machine for multi-threading. This is based on the assumption that 8 CPUs are available on the local machine; otherwise the software will utilize the available CPUs:

```
litmus> set_multi_cpu_usage -local_cpu 8
```

- The following command releases all the multi-CPU licenses immediately:

```
litmus> set_multi_cpu_usage -release_license
```

System Commands - Basic Tool-Tcl Commands

- [man](#)

man

man
command_name | *msg_id*

Displays information for a command or variable from the *Litmus Stylus Text Command Reference*, or prints detailed information for a specified message ID. If no manual page is located, man prints an error message.

The man pages for Litmus commands and variables are located in the following directory:

<install_dir>/<platform>/share/litmus/man

Parameter

<i>command_name</i>	Indicates the name of the text command (or variable) for which the manual information is required.
<i>msg_id</i>	Specifies the unique message ID for which you want to display detailed information. The output is categorized in three sections: • NAME: Displays the message ID and its severity level. • SUMMARY: Displays the summary of the message text. • DESCRIPTION: Displays detailed information about the message. The detailed information can include the background of the message, likely causes, and alternative steps to troubleshoot the problem.

Example

- The following command returns the manual description of the command `report_timing`:

```
man report_timing
```
- The following command returns the manual description of the `CONST_DGN_CMB2` message ID:

```
man CONST_DGN_CMB2
```

Timing Analysis Commands

- [audit_timing](#)
- [check_multi_cpu_usage](#)
- [check_timing](#)
- [check_timing_library](#)
- [check_timing_library_consistency](#)
- [create_analysis_view](#)
- [create_constraint_mode](#)
- [create_delay_corner](#)
- [create_library_set](#)
- [create_opcond](#)
- [create_property_alias](#)
- [create_rc_corner](#)
- [create_timing_condition](#)
- [delete_analysis_view](#)
- [get_capacitance_unit](#)
- [get_clock_network_objects](#)
- [get_clock_to_clock_association](#)
- [get_interactive_constraint_modes](#)
- [get_multi_cpu_usage](#)
- [get_propagated_clock](#)
- [get_resistance_unit](#)
- [get_selectively_expanded_modules](#)
- [get_time_unit](#)
- [list_libraries](#)
- [read_boundary_model](#)
- [read_ilm](#)

- [report_analysis_coverage](#)
- [report_analysis_summary](#)
- [report_analysis_views](#)
- [report_annotated_assertions](#)
- [report_annotated_check](#)
- [report_annotated_delay](#)
- [report_case_analysis](#)
- [report_cell_instance_timing](#)
- [report_clock_gating_check](#)
- [report_clock_propagation](#)
- [report_clock_sense](#)
- [report_clock_timing](#)
- [report_clocks](#)
- [report_constraint](#)
- [report_design](#)
- [report_fanin](#)
- [report_fanout](#)
- [report_instance_library](#)
- [report_lib_arcs](#)
- [report_mode](#)
- [report_nets](#)
- [report_path_exceptions](#)
- [report_path_groups](#)
- [report_ports](#)
- [report_reference](#)
- [report_timing](#)
- [report_timing_derate](#)
- [report_wire_load](#)
- [reset_path_adjust](#)
- [reset_path_adjust_group](#)
- [reset_timing_derate](#)

- [reset_wire_load_mode](#)
- [reset_wire_load_model](#)
- [reset_wire_load_selection_group](#)
- [set_analysis_view](#)
- [set_clock_exclusivity](#)
- [set_default_view](#)
- [set_interactive_constraint_modes](#)
- [set_path_adjust](#)
- [set_path_adjust_group](#)
- [set_selectively_expanded_modules](#)
- [set_table_style](#)
- [set_timing_context_clock_mapping](#)
- [set_timing_derate](#)
- [set_wire_load_mode](#)
- [set_wire_load_model](#)
- [set_wire_load_selection_group](#)
- [update_analysis_view](#)
- [update_constraint_mode](#)
- [update_delay_corner](#)
- [update_library_set](#)
- [update_opcond](#)
- [update_rc_corner](#)
- [update_timing](#)
- [update_timing_condition](#)
- [write_sdc](#)

audit_timing

`audit_timing`

Performs a focused set of timing constraint coverage and sanity checks. It is a Litmus pre-STA audit command, that helps identify unconstrained paths, missing clock definitions, and common timing modeling issues before final STA signoff. Following checks are performed by this command:

Constraint Coverage Checks: These checks identify timing checks (`report_analysis_coverage - RAC`) that were not exercised due to missing, blocked, or incomplete constraints.

Rule	Description
CONST_COV_SH	Setup and/or hold checks not performed on pins
CONST_COV_RR	Recovery and/or removal checks not performed on pins
CONST_COV_CG	Clock-gating setup/hold/pulse-width checks not performed
CONST_COV_DATA	Data setup and/or hold checks not performed
CONST_COV_CP	Clock period checks not performed
CONST_COV_CS	Clock separation checks not performed
CONST_COV_SKEW	Skew checks not performed
CONST_COV_PW	Pulse-width checks not performed

Clock Gating and Clock Quality Checks:

Rule	Description
CONST_CG_CHK5	Clock-gating enable pin not reached by any constrained data (<i>check_timing: clock_gating_no_data</i>)
CONST_CG_CHK6	Clock clipping due to incorrect gate type or trigger (<i>check_timing: clock_clipping_gate</i>)
CONST_CG_CHK7	Clock clipping due to incompatible clock and data frequencies (<i>check_timing: clock_clipping_freq</i>)

Data Path and Structural Timing Checks:

Rule	Description
CONST_DAT_CHK5	Internal endpoints unconstrained by clocks but affected by max-delay exceptions (<i>check_timing: max_delay_constrained_internal_endpoint</i>)
CONST_DGN_TIM2	Latch and its fanout operating on the same clock (<i>check_timing: latch_fanout_clock_connectivity</i>)

Clock Interaction and Exception Checks:

Rule	Description
CONST_EXC_CLK2	Interactions between multiple clock domains, including true, false, or partial crossings (<i>check_timing: clock_crossing</i>)

Missing Generated Clock Checks:

Rule	Description
CONST_CLK_DEF1	Detects missing generated clocks on clock generator circuits (flop/ICG driving another clock pin through logic without a generated clock)
CONST_CLK_DEF46	Detects missing <code>create_generated_clock</code> definitions

- To enable `check_timing` checks, add following to your Litmus runscript:
 - `set_rule_handling -category constraint_enhanced_signoff -severity warning`
- To enable `report_analysis_coverage` checks, add following to your Litmus runscript:
 - `set_rule_handling -category constraint_coverage_signoff -severity warning`

`audit_timing` provides a compact, signoff-oriented audit to ensure that:

- All critical timing checks are exercised
- Clock gating and generated clocks are correctly modeled
- Cross-clock interactions and structural issues are visible early.

Parameters

None

Related Commands

- `check_timing`
- `report_analysis_coverage`

check_multi_cpu_usage

check_multi_cpu_usage

Checks if distributed processing can work in the software environment. This command also ensures that the working directory is accessible from the master machine as well as the Tempus clients. The `check_multi_cpu_usage` command is specified after configuring the multi-CPU settings (`set_distribute_host` and `set_multi_cpu_usage`) to check if all the specified CPUs can be accessed. This command should be invoked before running any commands that use multiple CPUs.

Parameters

None

Examples

Here are some examples.

- The following example command script checks the distributed computing environment when `set_distributed_hosts -local` is specified:

```
tempus > check_multi_cpu_usage

/it/sjclapa06p6_scr02/clre/exp_2017_05_30/0223/dy/2014/2017/0413
/it/sjclapa06p6_scr02/clre/exp_2017_05_30/0223/dy/2014/2017/0413
/it/sjclapa06p6_scr02/clre/exp_2017_05_30/0223/dy/2014/2017/0413
/it/sjclapa06p6_scr02/clre/exp_2017_05_30/0223/dy/2014/2017/0413
Connected to sjfnl624 35779 0 ( PID=13350 )
Connected to sjfnl624 54715 1 ( PID=13356 )
Connected to sjfnl624 40469 3 ( PID=13397 )
Connected to sjfnl624 50442 2 ( PID=13378 )
Task 0 successfully accessed the current working directory.
Task 3 successfully accessed the current working directory.
Task 1 successfully accessed the current working directory.
Task 2 successfully accessed the current working directory.
Task 3: CPU(workload/CPU): 7.84/12 mem(free/total): 94118680k/98813572k.
Task 1: CPU(workload/CPU): 7.84/12 mem(free/total): 94118432k/98813572k.
Task 2: CPU(workload/CPU): 7.84/12 mem(free/total): 94117068k/98813572k.
Task 0: CPU(workload/CPU): 7.84/12 mem(free/total): 94119084k/98813572k.
Test heartbeat for task 0...
The distributed processing environment has been set up correctly.
```

- The following example command script checks the distributed computing environment when when

set_distributed_hosts -lsf -queue lnx64 is specified:

tempus > check_multi_cpu_usage

```
Job <5685326> is submitted to queue <lnx64>. The associated lsf task ID is '0'
Executing /it/sjclapa06p6_scr02/clre/exp_2017_05_30/0223/dy/2014/2017/0413/.clre_sub_22662_0
Job <5685328> is submitted to queue <lnx64>. The associated lsf task ID is '1'
Executing /it/sjclapa06p6_scr02/clre/exp_2017_05_30/0223/dy/2014/2017/0413/.clre_sub_22662_1
Job <5685329> is submitted to queue <lnx64>. The associated lsf task ID is '2'
Executing /it/sjclapa06p6_scr02/clre/exp_2017_05_30/0223/dy/2014/2017/0413/.clre_sub_22662_2
Job <5685330> is submitted to queue <lnx64>. The associated lsf task ID is '3'
Executing /it/sjclapa06p6_scr02/clre/exp_2017_05_30/0223/dy/2014/2017/0413/.clre_sub_22662_3
Connected to sjfnl625 43039 1 ( PID=30833 )
Connected to sjfnl625 34830 2 ( PID=30832 )
Connected to sjfnl625 33887 3 ( PID=30831 )
Connected to sjfib046 60125 0 ( PID=32727 )
Task 1 successfully accessed the current working directory.
Task 3 successfully accessed the current working directory.
Task 2 successfully accessed the current working directory.
Task 0 successfully accessed the current working directory.
Task 3: CPU(workload/CPU): 1.43/12 mem(free/total): 74177208k/82277684k.
Task 2: CPU(workload/CPU): 1.43/12 mem(free/total): 74177208k/82277684k.
Task 1: CPU(workload/CPU): 1.43/12 mem(free/total): 74177208k/82277684k.
Task 0: CPU(workload/CPU): 19.79/20 mem(free/total): 352917068k/396858936k.
Test heartbeat for task 0...
The distributed processing environment has been set up correctly.
```

Related Topics

- "Appendix - Multi-CPU Usage" in the *Tempus User Guide*
 - "Checking the Distributed Computing Environment"

Related Commands

- set_multi_cpu_usage
- get_multi_cpu_usage

check_timing

```
check_timing
[-help]
[-check_only warning_list]
[-clock_crossing_output_format {summary | csv}]
[-exclude_warnings warning_list]
[-include_warnings warning_list]
[-tcl_list]
[-type type_list]
[-verbose]
[-view view_name]
[> <file_name[.gz]>]
[>> <file_name[.gz]>]
[port_or_pin_list]
```

Performs a variety of consistency and completeness checks on the timing constraints specified for a design. The checks include arrival time and external delay (or required time) for each clock in a multiple clock system. In addition, clock connectivity and data connectivity are checked to make sure the clock or data is propagated as expected. Clock gating points are also reported.

Use this command on generic or mapped netlists. The `check_timing` command considers constants applied to a pin while reporting warnings.

Use the `check_timing` command before using any commands for timing analysis. Warnings are displayed when problems occur. The [Table](#) below lists the `check_timing` warnings. For best results, re-run the `check_timing` command after resolving each warning. Redirecting the output to a file is the quickest run method.

Parameters

-help	Prints out the command usage.
-------	-------------------------------

<code>-check_only warning_list</code>	Allows you to specify warning types. For example, the following command lists the specified warnings only: <pre>check_timing -check_only {uncons_endpoint clock_crossing}</pre> This list can also include warnings that are not listed by default. Valid entries in <i>warning_list</i> are: clock_clipping_freq, clock_clipping_gate, clock_crossing, clock_expected, clock_gating_controlling_edge_unknown, clock_gating_no_data, clock_missing_at_sequential_output, clock_not_propagated, clocks_masked_by_another_clock, const_collision, data_check_multiple_reference_signal, data_check_no_reference_signal, ideal_clock_waveform, invalid_pll_configuration, loop, master_clk_edge_not_reaching, missing_clock_groups, missing_drive, multiple_clock, no_drive, no_gen_clock_source, no_input_delay, partial_input_delay, pulse_non-pulse_clock_merge, signal_level, uncons_endpoint, uncons_endpoints_due_to_case_analysis
<code>-clock_crossing_output_format {summary csv}</code>	Specifies the format of the clock crossing report to be generated - as a summary or in csv format.
<code>{> >>} file_name</code>	When specified with <code>></code>, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with <code>>></code>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The <i>filename</i> parameter must be the last argument in the list. The <i>filename</i> and <code>-tcl_list</code> parameters are mutually exclusive; you cannot specify them together. Note: To write a compressed report, add the <code>.gz</code> extension to the file name. Default: Report is displayed on standard output without being saved.

<code>-exclude_warnings warning_list</code>	Excludes the specified warning types when generating the report. Use this parameter to exclude any of the warnings included in the report by default. By default, the software includes the following warnings in the report: <code>clock_expected</code>, <code>loop</code>, <code>ideal_clock_waveform</code>, <code>master_clk_edge_not_reaching</code>, <code>partial_input_delay</code>, <code>missing_drive</code>, <code>no_input_delay</code>, <code>no_drive</code>, <code>no_gen_clock_source</code>, <code>uncons_endpoint</code>, <code>uncons_endpoints_due_to_case_analysis</code>. For a complete list of warnings and their descriptions, refer to Table
<code>-include_warnings warning_list</code>	Includes the specified warning types when generating the report. Use this parameter to add any of the warnings not included in the report by default. By default, the software reports all warnings except the following: <code>clock_clipping_gate</code>, <code>const_collision</code>, <code>clock_crossing</code>, <code>clock_not_propagated</code>, <code>missing_drive</code>, <code>multiple_clock</code>, <code>data_check_multiple_reference_signal</code>, <code>data_check_no_reference_signal</code>, <code>clock_gating_no_data</code>, <code>clock_gating_controlling_edge_unknown</code>, <code>clock_missing_at_sequential_output</code>, <code>signal_level</code>, and <code>clocks_masked_by_another_clock</code>. For a complete list of warnings and their descriptions, refer to Table.
<code>port_or_pin_list</code>	Lists pins or ports. For example: <pre>check_timing -verbose [get_ports in*]</pre>
<code>-tcl_list</code>	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. The <code>-tcl_list</code> and <code>filename</code> parameters are mutually exclusive; you cannot specify them together.
<code>-type type_list</code>	Limits the checks to the specified type. <i>Default:</i> All check types are reported.

	clock_clipping: Warns when there is a problem with clock gating that can result in clock clipping. Reports the following warnings: • clock_clipping_freq • clock_clipping_gate
	clocks: Reports the following clock warnings: • clock_crossing • clock_expected • clock_not_propagated • ideal_clock_waveform • master_clk_edge_not_reaching • no_gen_clock_source
	clock_gating_inferred: Warns when inferred clock gating checks are not analyzed by the software, even if a clock signal reaches and is used downstream the clock tree. Reports the following warnings. • clock_gating_controlling_edge_unknown • clock_gating_no_data
	constant_collision: Warns when there is a constant collision on a net connected to the pin or if there is a contradiction on a pin. Constant collision occurs when a net is driven simultaneously by conflicting values and a constant contradiction occurs when a constant constraint on a pin conflicts with the driven value.
	endpoints: For output ports, warns if no output delay constraint is applied to the port or if a specific (min/max rise/fall) output delay is missing. It also warns if an output delay constraint is applied with no clock information. For all endpoints (output ports and register data pins), it warns if any incoming signal is unconstrained. Also warns if a clock arrives where clock is not expected, or if multiple signals arrive at an endpoint.

	<code>inputs:</code> Reports the following input delay, arrival time assertion, and drive assertion warnings: • <code>partial_input_delay</code> • <code>missing_drive</code> • <code>no_input_delay</code> • <code>no_drive</code>	
	<code>loops:</code> Warns when a combinational loop is detected. Reports the loop and the arc used to break the loop. The <code>-pin</code> option is ignored in this case. Also reports loops caused by generated clocks.	
<code>-verbose</code>	Gives detailed information about the warnings. When specified, warnings are grouped and sorted by warning type. <i>Default:</i> Gives a summary of how many warnings of each type exists.	
<code>-view</code>	Reports warning(s) that belong to the specified view.	

Command Order

Use this command after setting all the constraints but before optimizing your design or generating any timing reports.

Warning Messages

The following table shows the `check_timing` warning messages.

check_timing Warning Messages

Warning	Description
clock_clipping_gate	Warning: Clock clipping possible due to wrong gate type or wrong trigger for data input.
clock_clipping_freq	Warning: Clock clipping possible due to incompatible clock signal and data signal frequencies.
clock_crossing	Warning: Clock domains interact. Comments: Checks clock interactions between multiple clock domains in the design. When you specify this parameter, the Timing Check Crossing Clocks report shows the interaction of clocks in the design and indicates whether the paths between the clocks are <code>False</code> or <code>Partial False</code>. If a clock launches one or more paths, which are captured by other clocks, the report lists all pairs of crossing clocks. • This includes inferred clock gating with different reference and signal clock. • If a path has been disabled using the <code>set_disable_timing</code> command, crossing clocks on that path are not reported. If all the paths between two clocks are false paths or they are exclusive/asynchronous clocks, the path is marked as <code>False</code>. The false path can be defined on clocks, for example, <code>set_false_path -clock_to CLK</code> or on pins or ports, such as, <code>-through buf1/Y</code>. If only part of paths are set as false paths or exclusive/asynchronous clocks, the path is marked as <code>Partial False</code>.

	• If a false path is defined only for one check type (say, setup), the crossing clocks are marked as <code>Partial False</code> if the simultaneous setup hold mode is set to <code>on</code>. The crossing clocks are marked <code>False</code> if the analysis type is setup and the simultaneous setup hold mode is set to <code>off</code>. • If only one edge (rise/fall) of a particular clock pair has false paths (say, using <code>-clock_fall</code>), then the path is marked as a <code>Partial False</code>.
<code>clock_expected</code>	Warning: Clock not found where clock is expected. Comments: Indicates that no clock signal is defined at the clock pin. See <code>create_clock</code>.
<code>clock_gating_controlling_edge_unknown</code>	Warning: Unknown controlling edge to perform clock gating check. Comments: Reports library cells with complex functions (MUX or AOI) where controlling edge of signal to perform clock gating check cannot be determined automatically. You need to specify correct edge by <code>-high/-low</code> option using the <code>set_clock_gating_check</code> command to resolve such warning. Note: The software does not report this warning when either: • <code>set_disable_clock_gating_check</code> command is applied on such pins, or • global settings to disable clock gating checks are on.

clock_gating_no_data	Warning: No data signal reaching clock gating enable pin Comments: Reports clock gating enable pins where no valid data signal arrives. You need to do either of the following: • specify valid data signal to reach enable pin of clock gating cells, or • remove any path exceptions or disabled checks that blocks valid data signal to reach enable pin Note: The software does not report this warning when either • <code>set_disable_clock_gating_check</code> is applied on such pins, or • global settings to disable clock gating checks are on.
clocks_masked_by_another_clock	Warning: Clock(s) in fanin cone are dropped due to another clock Comments: Reported timing pins on which creation of a regular or a generated clock causes clock(s) in fanins of such pins have been blocked or masked from propagating further. However, for pins having only master clock waveform on the fanin, this warning is not reported. You can use the detailed mode (<code>-verbose</code> option) to report names of all the incoming clock waveforms that have been blocked.
clock_missing_at_sequential_output	Warning: Found latency phase at sequential output on clock tree instead of clock phase Comments: Reports output pins of sequential elements that lie on source clock tree for the generated clocks created downstream and do not have any valid clock phase. This indicates possible missing clock related checks (e.g., clock gating) on downstream clock tree. You can add clocks on this pin or downstream clock tree only if no valid clock exists until downstream generated clock creation point.

clock_not_propagated	Warning: Clock not propagated Comments: Reports ideal clocks reaching the reference pin of a timing check.
const_collision	Warning: Constant collision Comments: See <code>set_case_analysis</code>.
data_check_multiple_reference_signal	Warning: Multiple clocked signals are reaching data check reference pin Comments: Reports on reference pins for non sequential timing check (data check) where more than one reference data arrives.
data_check_no_reference_signal	Warning: No clocked signal is reaching data check reference pin Comments: Reports reference pins for non-sequential timing check (data check) where no valid reference signal (data phase) arrives. You can do one of the following to resolve such warnings: • add valid reference signals to reach at such reference pins • remove path exceptions or disabled arcs that are blocking such reference signals
ideal_clock_waveform	Warning: Clock waveform is ideal. Comments: Reports clocks that are ideal and are not propagated. This warning type is reported by default (unless <code>-exclude_warning</code> or <code>-check_only</code> parameter is specified). You can use the <code>set_propagated_clock</code> command to avoid this warning.
invalid_pll_configuration	Warning: No valid path exists from output pin to feedback pin of PLL. Comments: Shows PLL (phase locked loop) check warnings. These checks allow you to identify and fix issues in the PLL flow.

loop	Warning: Timing loop found in the design Comments: Indicates that there are timing loops in the design. It also includes loops caused by generated clocks.
master_clk_edge_not_reaching	Warning: Master clock edge does not reach the generated clock target. Comments: This warning means that rise/fall latency edge is not reaching to create generated clock root from the master clock. You can verify if the path that is seeing this warning is actually broken, or edge is somewhere disabled earlier on this path. There is a global <code>timing_enable_genclk_edge_based_source_latency</code> that controls how the software chooses generated clock source latency paths. Details of this global can be found at Litmus text command reference.
missing_clock_groups	Warning: Set clock group related checking Comments: Reports any potential issues with the defined clock grouping data.
partial_input_delay	Warning: Indicates that a rise or fall input delay assertion is not applied.
missing_drive	Warning: Missing specific (rise/fall) drive constraint Comments: Either a rise or fall drive constraint is applied and the other one is missing.
multiple_clock	Warning: Multiple clocks are reaching clock reference pin. Comments: Reports clock pins of sequential or clock gating cells where more than one clock waveform arrives. This warning is for information only and the software performs timing analysis with respect to all such clocks.
no_input_delay	Warning: No input delay assertion with or without <code>-clock</code> option specified. Comments: See <code>set_input_delay</code>.

no_drive	Warning: No drive constraint Comments: See <code>set_driving_cell</code> and <code>set_drive</code>.
no_gen_clock_source	Warning: No clock source found for generated clock. Comments: Indicates that the source pin of the clock generated by the <code>create_clock</code> command is not driven by a clock source.
pulse_non-pulse_clock_merge	Warning: Merge of pulse and non-pulse clock paths. Comments: Reports points where a pulse clock and a non-pulse clock merges.
signal_level	Warning: Driver pin voltage value do not match with sink pin voltage level Comments: Reports net driver/receiver pin pairs for which the pin voltage does not match.
uncons_endpoint	Warning: No constrained signal reaching pin. Comments: For output ports: Primary output port has no implicit timing check. Use <code>set_output_delay</code>, <code>set_max_delay</code>, or <code>set_min_delay</code> to create a valid constraint. For register data pins: There is no valid clock reaching the CK pin of the register data pin. Check for missing <code>create_clock</code> for the clock network. The endpoints are reported as unconstrained in both in early and late timing. This does not cover all the cases and scenarios because of which endpoint can become unconstrained. To cover all possible cases, you can use the <code>report_analysis_coverage</code> command.
uncons_endpoints_due_to_case_analysis	Warning: Unconstrained signal arriving at end point due to case analysis Comments: Reports unconstrained endpoints due to case analysis. To enable this warning, you must set the <code>timing_report_check_timing_unconstrained_endpoints_due_to_constants</code> attribute to <code>true</code>.

Examples

Here are some examples.

- Consider the following example:

```
in-----|>-----|>-----out

buf1 buf2
```

The Timing Check Crossing Clocks reports False or Partial False in some of the scenarios given below:

- a) `set_input_delay` with respect to CLK1 on in and `set_output_delay` with respect to CLK2 on out:

From Clock	Crossing Clocks
-----	-----
CLK1	CLK2

- b) `set_input_delay` with respect to CLK1 on in, `set_output_delay` with respect to CLK2 on out, and `set_false_path -to CLK2` (clock-based false path):

From Clock	Crossing Clocks
-----	-----
CLK1	CLK2 (False)

- c) If the `set_false_path` becomes `set_false_path -rise_to CLK2`, the following is reported:

From Clock	Crossing Clocks
-----	-----
CLK1	CLK2 (Partial False)

The following command displays the default summary report:

```
litmus> check_timing

-----
                                TIMING CHECK SUMMARY
-----
Warning                        Warning Description                        Number of
                                Warning Description                        Warnings
-----
ideal_clock_waveform           Clock waveform is ideal                        1
```

Litmus Text Command Reference

Timing Analysis Commands--check_timing

clock_expected	Clock not found where clock is expected	3
master_clk_edge_not_reaching	Master clock edge does not reach the generated clock target	2
no_drive	No drive assertion	8
no_input_delay	No input delay assertion with respect to clock	4
uncons_endpoint	Unconstrained signal arriving at end point	5

- The following command displays a specified type of warning using the `-type` option if a timing loop is found in the design:

```
litmus> check_timing -type loops
```

```
-----
```

TIMING CHECK SUMMARY		
Warning	Warning Description	Number of Warnings
loop	Timing loop found in the design	0

```
-----
```

- The following command provides more information about problems and prints a summary using the `-verbose` option:

```
litmus> check_timing -verbose
```

```
-----
```

TIMING CHECK SUMMARY		
Warning	Warning Description	Number of Warnings
ideal_clock_waveform	Clock waveform is ideal	1
clock_expected	Clock not found where clock is expected	3
master_clk_edge_not_reaching	Master clock edge does not reach the generated clock target	2
no_drive	No drive assertion	8
no_input_delay	No input delay assertion with respect to clock	4
uncons_endpoint	Unconstrained signal arriving at end point	5

```
-----
```

TIMING CHECK DETAIL

```
-----
```

Pin	Warning
u99/CK	Clock not found where clock is expected

```
-----
```



```

u5/CK      Clock not found where clock is expected
u8/CK      Clock not found where clock is expected
u8/Q       Master clock edge does not reach the target for generated clock GCLK4
CLK1       No drive assertion
in1        No drive assertion
in1        No input delay assertion with respect to clock
out1       Unconstrained signal arriving at end point
-----

```

TIMING CHECK IDEAL CLOCKS

```

Clock  Waveform
-----
CLK1
-----

```

- When `timing_report_check_timing_unconstrained_endpoints_due_to_constants` global variable is enabled, the case analysis warnings are reported as shown below:

```
litmus 11> check_timing
```

TIMING CHECK SUMMARY

Warning	Warning Description	Number of Warnings
clock_expected	Clock not found where clock is expected	4202
no_drive	No drive assertion	241
no_input_delay	No input delay assertion with respect to clock	160
uncons_endpoint	Unconstrained signal arriving at end point	145259
uncons_endpoints_due_to_case_analysis		
	>>>>Unconstrained signal arriving at end point due to case analysis	5969437

Related Commands

- [report_timing](#)

check_timing_library

```
check_timing_library  
[-help]  
[-check_power]  
[-lib_cell string]  
[-library string]  
[-out_file string]  
[-report_missing_power_only]
```

Checks the contents of the timing library and reports any inconsistencies in the log file.

You can use the `check_timing_library` command after importing the design and timing libraries.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-check_power</code>	Checks if power constructs, such as, <code>internal_power</code>, <code>leakage_power</code>, or <code>ecsm_power</code>, are defined for the cells. Note: The <code>-check_power</code> parameter is optional. However, if you specify <code>-report_missing_power_only</code>, then you must specify <code>-check_power</code>. If you use the <code>-library</code> parameter, you do not have to specify the <code>-check_power</code> parameter, because when a library-specific cell is given as input, the software automatically checks for power constructs. <i>Default:</i> Only checks if timing is defined for the cells.
<code>-lib_cell string</code>	Specifies the name of the library cell. Note: The <code>-lib_cell</code> parameter is optional. However, if you specify the <code>-library</code> parameter, then you <i>must</i> specify <code>-lib_cell</code>.

<code>-library string</code>	Specifies the timing library in which to check a particular cell. You must specify the <code>-lib_cell</code> parameter with <code>-library</code>, to specify the particular library cell to check. Note: If you use the <code>-library</code> parameter, you do not have to specify the <code>-check_power</code> parameter, because when a library-specific cell is given as input, the software automatically checks for power constructs.
<code>-out_file string</code>	Saves the report output to the specified file.
<code>-report_missing_power_only</code>	Reports only those cells that do not have any defined <code>internal_power</code> (at the pin level), or <code>leakage_power</code>. You must specify the <code>-check_power</code> parameter with the <code>-report_missing_power_only</code> parameter.

Examples

- The following command checks if timing is defined for all cells that have been read into the design, and writes a report to the log file:
`check_timing_library`
- The following command checks if timing is defined for all cells that have been read into the design, and writes a report to a file named `CheckTiming1`:
`check_timing_library -out_file CheckTiming1`
- The following command checks if timing and power constructs are defined for all cells that have been read into the design, and writes the report to the log file:
`check_timing_library -check_power`
- The following command checks if timing and power constructs are defined for all cell that have been read into the design, and writes the report to a file named `CheckTiming2`:
`check_timing_library -out_file CheckTiming2 -check_power`
- The following command checks if timing is defined for all cells with the name `A01` in all timing libraries, and writes the report to a file named `CheckTiming3`:
`check_timing_library -out_file CheckTiming3 -lib_cell A01`
- The following command checks if timing and power constructs are defined for all cells with the name `A01`, in all timing libraries, and writes the report to a file named `AllA01Check`:
`check_timing_library -out_file AllA01Check -lib_cell A01 -check_power`

- The following command checks if timing and power constructs are defined for the cell with the name `A01` in the timing library `minFile1.lib`, and writes the report to a file named `A01Check`:

```
check_timing_library -out_file A01Check -lib_cell A01 -library minFile1.lib
```

Note: When you use the `-library` parameter, you do not have to specify the `-check_power` parameter, because when a library-specific cell is given as input, the software automatically checks for power constructs.

- The following command checks if the `internal_power` and/or `leakage_power` constructs are defined for all cells that have been read into the design, and reports only those cells that do not have the constructs defined. The software writes the report to a file named `MissingPower`:

```
check_timing_library -out_file MissingPower -check_power -report_missing_power_only
```

check_timing_library_consistency

check_timing_library_consistency
[-help]

Checks for missing libraries for instances in the active view.

This command checks whether the library for the cell associated with the instance is present in the associated power domain library set of the instance in the active views.

Parameters

-help	Prints out the command usage.
-------	-------------------------------

Examples

- The following command displays a report showing missing libraries:

```
check_timing_library_consistency
```

The software displays the following information:

```
Checking the Library binding for instance in active view 'vA'
Pass. All instances have library definition in view 'vA'
Checking the Library binding for instance in active view 'vB'
No library found for instance 'U1', cell 'AND2X1', in powerdomain 'PD_BAD', of view 'vB'
Fail. Missing Library for some instance found in view 'vB'
Checking the Library binding for instance in active view 'vAB'
Pass. All instances have library definition in view 'vAB'
```

create_analysis_view

```
create_analysis_view  
[-help]  
-constraint_mode modeName  
-delay_corner dcCornerObj  
-name viewName
```

Creates an analysis view object that associates a delay calculation corner with a constraint mode. An analysis view object provides all of the information necessary to control a single multi-mode multi-corner analysis.

After creating analysis views, use the `set_analysis_view` command to specify which views to use for setup and hold optimization or timing analysis.

Use this command after creating constraint modes (`create_constraint_mode`) and delay calculation corners (`create_delay_corner`).

Parameters

<code>-help</code>	Prints out the command usage.
<code>-name <i>viewName</i></code>	Specifies the name of the analysis view being defined.
<code>-constraint_mode <i>modeName</i></code>	Specifies the name of the constraint mode to associate with this analysis view. A constraint mode groups a set of constraints to be used with any given analysis. To create the constraint mode, use the <code>create_constraint_mode</code> command.
<code>-delay_corner <i>dcCornerObj</i></code>	Specifies the name of the delay calculation corner object to associate with this analysis view. A delay calculation corner object contains all of the information needed to control delay calculation for a specific analysis view. To create a delay calculation corner, use the <code>create_delay_corner</code> command.

Examples

- The following command creates an analysis view called `missionSlow` using the constraint mode `missionSetup` and the delay calculation corner `dcWCCOM`:

```
litmus> create_analysis_view  
-name missionSlow  
-constraint_mode missionSetup  
-delay_corner dcWCCOM
```
- The following command creates an analysis view called `missionFast` using the constraint mode `missionHold` and the delay calculation corner `dcBCCOM`:

```
litmus> create_analysis_view  
-name missionFast  
-constraint_mode missionHold  
-delay_corner dcBCCOM
```

Related Commands

- `set_analysis_view`
- `create_constraint_mode`
- `create_delay_corner`

create_constraint_mode

```
create_constraint_mode
[-help]
-name modeName
-sdc_files {file1.sdc file2.sdc ...}
[-ilm_sdc_files {file1.sdc file2.sdc ...}]
[-tcl_variables {{var_name1 value1} {var_name2 value2} {var_name3 value3} ...}]
```

Associates a list of SDC constraint files with a specified constraint mode name, for multi-mode multi-corner analysis. This constraint mode name can be referred to later when creating analysis views. A constraint mode defines one of possibly many different functional, test, or Dynamic Voltage and Frequency Scaling (DVFS) modes of a design. SDC files can be shared by many different constraint modes, and the same constraint mode can be associated with multiple analysis views.

SDC files typically contain timing analysis information, such as the clock specifications, case analysis constraints, I/O timings, and path exceptions that make each mode unique. If the ILM is being used, the timing system utilizes the files specified with `-ilm_sdc_files` during the flatten ILM mode (`flatten_ilm`); otherwise, the system uses the files specified with `-sdc_files`.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-ilm_sdc_files {file1.sdc file2.sdc ...}</code>	Specifies a Tcl list of ILM SDC files to be included in the constraint mode.
<code>-name modeName</code>	Specifies the name of the constraint mode to be created.
<code>-sdc_files {file1.sdc file2.sdc ...}</code>	Specifies a Tcl list of SDC files to be included in the constraint mode.
<code>-tcl_variables {{var_name1 value1} {var_name2 value2} {var_name3 value3} ...}</code>	
	Specifies a list of tcl variables and values to be used in the constraint mode.

Examples

- The following command groups the SDC files `io.sdc`, `mission1-clks.sdc`, and `mission1-except.sdc` to create a mode object named `missionSetup`:

```
litmus> create_constraint_mode -name missionSetup
      -sdc_files [list io.sdc mission1-clks.sdc mission1-except.sdc]
```


- The following command groups the SDC files `io.sdc`, `mission1-clks.sdc`, `mission1-except.sdc`, and `dont_use.sdc` to create a mode object named `missionHold`:

```
litmus> create_constraint_mode -name missionHold  
      -sdc_files [list io.sdc mission1-clks.sdc mission1-except.sdc dont_use.sdc]
```

- The following command groups the SDC files `test-io.sdc`, `test-clks.sdc`, and `test-except.sdc` to create a mode object named `testHold`:

```
litmus> create_constraint_mode -name testHold  
      -sdc_files [list test-io.sdc test-clks.sdc test-except.sdc]
```

Related Commands

- [update_constraint_mode](#)
- [create_analysis_view](#)

create_delay_corner

```
create_delay_corner
[-help]
[-early_estimated_worst_irdrop_factor <percent>]
-name delayCornerName
[-early_irdrop_files list_of_files]
[-early_rc_corner rcCornerObj]
[-early_temperature_files list_of_files]
[-early_timing_condition conditional_pairs]
[-late_estimated_worst_irDrop_factor percent]
[-late_irdrop_data files_or_directories]
[-late_irdrop_files list_of_files]
[-late_rc_corner rcCornerObj]
[-late_temperature_files list_of_files]
[-late_timing_condition conditional_pairs]
[-irdrop_data files_or_directories]
[-irdrop_files list_of_files | -irdrop_data files_or_directories]
[-pg_net_voltages voltage_pairs]
[-rc_corner rcCornerObj]
[-si_enabled <true | false>]
[-temperature_files list_of_files]
[-timing_condition conditional_pairs]
```

Creates a named delay calculation corner object that can be referenced later when creating an analysis view. A delay calculation corner provides all of the information necessary to control delay calculation for a specific view. Each corner contains information on the libraries to use, the operating conditions with which the libraries should be accessed, and the RC extraction parameters to use for calculating parasitic data. Delay corner objects can be shared by multiple top-level analysis views.

The `-early_*` and `-late_*` parameters should not be used to represent best and worst case corners. Instead use separate delay corners to define Best-Case and Worst-Case differences. Use the `-early_*` and `-late_*` parameters within a single delay calculation corner to control on-chip variation. In this latter situation different conditions or libraries will be used for the launch and capture paths of individual timing paths.

Note: A single delay calculation corner object specifies the delay calculation rules for the entire design. If a design includes power domains, the delay calculation corner can contain domain-specific subsections that specify the required operating condition information, and any necessary timing library rebinding for the power domain. Use the `update_delay_corner` command to add a power domain definition to a delay calculation corner.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-early_estimated_worst_irDrop_factor percent</code>	Specifies the early voltage derating factor. When this parameter is specified, the software performs delay calculations using the voltage = $\text{nomVolt} * (1 - \text{value})$. <i>Default:</i> 0.0
<code>-early_irdrop_data files_or_directories</code>	Specifies the IR drop files or directories to apply when calculating the early arrival times at a single delay corner.
<code>-early_irdrop_files list_of_files</code>	Specifies the list of IR drop files to apply when calculating early arrival times at a single power domain. This parameter is used primarily when configuring the multi-mode multi-corner environment for on-chip-variation (OCV) analysis. Note: This parameter is obsolete and will be removed in a future release. To ensure compatibility with future releases, you should update your scripts.
<code>-early_rc_corner rcCornerObj</code>	Specifies the RC corner object to associate with the early corner object.
<code>-early_temperature_files list_of_files</code>	Specifies the name of the temperature file for temperature aware delay calculation for early corner.
<code>-early_timing_condition conditional_pairs</code>	Specifies pair list of domains/supply sets and timing conditions to associate with this delay corner object.
<code>-irdrop_data files_or_directories</code>	Specifies the IR drop files or directories to apply to both the early and late delay calculation for the specified delay corner object.

<code>-irdrop_files list_of_files</code>	Specifies the list of IR drop files to apply to both early and late delay calculation for this power domain object. This parameter is used primarily to configure single-corner or Best-Case Worst-Case (BC-WC) analysis modes. Note: This parameter is obsolete and will be removed in a future release. To ensure compatibility with future releases, you should update your scripts.
<code>-late_estimated_worst_irDrop_factor percent</code>	Specifies the late voltage derating factor. When this parameter is specified, the software performs delay calculations using the voltage = $\text{nomVolt} * (1 - \text{value})$. <i>Default:</i> 0.0
<code>-late_irdrop_data files_or_directories</code>	Specifies the IR drop files or directories to apply when calculating the late arrival times at a single delay corner.
<code>-late_irdrop_files list_of_files</code>	Specifies the list of IR drop files to apply when calculating late arrival times at a single power domain. This parameter is used primarily when configuring the multi-mode multi-corner environment for on-chip-variation (OCV) analysis. Note: This parameter is obsolete and will be removed in a future release. To ensure compatibility with future releases, you should update your scripts.
<code>-late_temperature_files list_of_files</code>	Specifies the name of the temperature file for temperature aware delay calculation for late corner.
<code>-late_rc_corner rcCornerObj</code>	Specifies the RC corner object to associate with the late corner object.

<code>-late_timing_condition conditional_pairs</code>	Specifies pair list of domains/supply sets and timing conditions to associate with this delay corner object. Note: The <code>-early_timing_condition</code> and <code>-late_timing_condition</code> parameters should be specified together.
<code>-name delayCornerName</code>	Specifies the delay calculation corner to which to add the power domain definition, or to which to update the delay corner or power domain attribute values.
<code>-pg_net_voltages voltage_pairs</code>	Specifies the power/ground net voltages to be used for both early/late timing analysis.
<code>-rc_corner rcCornerObj</code>	Specifies the new RC corner object to associate with this delay corner.
<code>-si_enabled {true false}</code>	Controls signal integrity (SI) analysis. When set to <code>true</code>, the analysis of all MMMC views will include SI analysis by default. You can use this parameter to further determine which analysis views receive SI analysis and which do not. When this parameter is set to <code>false</code> on a delay corner, the software will disable SI analysis for all the views that reference that corner. By default (or when this parameter is set to <code>true</code>), SI analysis is computed for all the views related to that corner - whenever SI is globally enabled. (The global control of SI analysis is determined by the <code>delaycal_enable_si true</code> attribute setting.)
<code>-temperature_files list_of_files</code>	Specifies the name of the temperature file for temperature aware delay calculation for single corner.
<code>-timing_condition conditional_pairs</code>	Specifies pair list of domains/supply sets and timing conditions to associate with this delay corner object.

Examples

- The following example specifies a list of conditional pairs of domains/supply sets and the timing conditions associated with the specified delay corner object:

```
create_delay_corner -name ccworst_setup\  
-timing_condition {tc_1 PD1@tc_2 PD2@tc_2 PD3@tc_2 PD4@tc_2}
```

Related Commands

- `update_delay_corner`

create_library_set

```
create_library_set
[-help]
[-library_side_file string]
-name libSetName
[-timing {lib1.lib lib2.lib lib3.lib ...}]
[-aocv string]
[-si {lib1.cdb lib2.cdb lib3.udn lib4.udn...}]
[-socv string]
```

Associates a TCL list of timing and cdb/UDN libraries with a specified library set name. Library sets allow a group of library files to be treated as a single entity so that higher-level descriptions (delay calculation corners) can simply refer to the library configuration by name. The same library set can be referenced multiple times by different delay calculation corners.

Note: You must have a set of timing libraries specified in the configuration file for the software to initialize properly. When configuring your multi-mode multi-corner environment, you can choose to specify these libraries in your own custom library set, or refer to them through the two library set definitions that are automatically initialized from the configuration file: `default_libs_min` and `default_libs_max`.

Note: The `create_library_set` command determines the priority of ECSM vs CCS data (if both are present). The software first checks for ECSM construct, if it is missing then CCS construct is loaded.

The order in which you define timing libraries is important. The software considers the first library you specify in the list as the master library, with each successive library having a lower priority.

Parameters

-help	Prints out the command usage.
-aocv <i>string</i>	Specifies the aocv libraries to include in the library set.
-library_side_file <i>string</i>	Specifies the library side file.
-name <i>libSetName</i>	Specifies the name for the library set being created. This name is used to associate the library list with a specific delay calculation corner.

<pre>-si {lib1.cdb lib2.cdb lib3.udn lib4.udn...}</pre>	Specifies cdB libraries and/or user-defined noise (UDN) models to include in the library set. The cdB and/or UDN libraries are required for performing signal integrity analysis. Note: You can use this parameter for CCS-N or ECSM-N models, that is, for cdBs only.
<pre>-socv <i>string</i></pre>	Specifies the SOCV libraries to include in the library set.
<pre>-timing {lib1.lib lib2.lib lib3.lib...}</pre>	Specifies the timing libraries to include in the library set. Note: Libraries intended for use in an ECSM-based multi-library interpolation must be grouped within an additional level of Tcl list structure to identify the intended usage to the library subsystem. Specify three timing libraries that are characterized at different voltages (typically the lower and upper voltage bounds, and a third intermediate point). All voltages required by the software will be interpolated from this set of libraries. The libraries in the set can differ only in the characterization voltage; they must be identical to each other in all other aspects. However, if you want to perform multi-temperature based library interpolation, you can use two library files that are characterized at different temperatures for each voltage value. See the examples for creating an ECSM-based multi-library set and multi-temperature based library interpolation in the Examples section.

The ECSM and CCS libraries can be very large and loading them may take significant time. You can convert the ASCII NLDM (Liberty .lib), ECSM, and CCS libraries to a binary version using the `write_ldb` command. For conversion of libraries to a binary version, refer to `write_ldb`.

Instead of the original ASCII libraries, you can specify these binary libraries in the configuration file:

- **Original MMMC setup file**

Examples

- The following command creates a library set that associates timing libraries and cdB libraries with a nominal voltage of 1 volt with the library name IsCOM-1V:

```
litmus > create_library_set
-name IsCOM-1V -timing [list stdcell_F_1V.lib ram_F.lib pad.lib]
-si [list stdcell_F_2.cdb ram_F.cdb pad.cdb]
```

- The following command creates a library set that can be used to specify the 2-volt version of a core cell library, which could be associated later with a 2-volt MSMV power domain specified in the design:

```
litmus > create_library_set
-name IsCOM-2V
-timing [list stdcell_F_2V.lib]
```

- The following command creates an ECSM-based multi-library set called triLibFast. The libraries are grouped within an additional level of Tcl list structure to identify the intended usage to the library subsystem.

```
litmus > create_library_set
-name triLibFast
-timing [list
    [list stdcell_F_1V.lib stdcell_F_2V.lib stdcell_F_3V.lib]
    ram_F.lib pads.lib
]
```

- The following command performs multi-temperature based library interpolation for libraries characterized at 0.7, 1.0, and 1.1 volts, each of which is also characterized using -40 and -35 temperature values:

```
litmus > create_library_set
-name Trilib
-timing {{ stdcell_0.7_temp-40.lib stdcell_0.7_temp-35.lib stdcell_1.0_temp-40.lib
```

```
stdcell_1.0_temp-35.lib stdcell_1.1_temp-40.lib stdcell_1.1_temp-35.lib } mem.lib io.lib }
```

Related Commands

- [update_library_set](#)

create_opcond

```
create_opcond
[-help]
-name opcondName
[-process processValue]
[-temperature temperatureValue]
[-voltage voltageValue]
```

Creates a set of virtual operating conditions in the specified library without actually modifying the library. These virtual operating conditions can then be referenced by a delay calculation corner ([create_delay_corner](#)) as if they actually exist in the library.

Parameters

-help	Prints out the command usage.
-name <i>opcondName</i>	Specifies the name for the operating condition being created.
-process <i>processValue</i>	Specifies the process value for the operating condition.
-temperature <i>temperatureValue</i>	Specifies the temperature value for the operating condition.
-voltage <i>voltageValue</i>	Specifies the voltage value for the operating condition.

Examples

- The following command creates a virtual operating condition called PVT1:

```
litmus> create_opcond -name PVT1
    -process 1.0
    -voltage 1.2
    -temperature 120
```

- The following command creates a virtual operating condition called PVT2:

```
litmus> create_opcond -name PVT2
    -process 1.0
    -voltage 1.2
    -temperature 20
```

Related Commands

- `create_delay_corner`
- `create_timing_condition`
- `update_timing_condition`

create_property_alias

```
create_property_alias  
[-help]  
-map <alias_property_name_pairs>  
-obj_type {pin port cell net clock lib_cell lib_pin design lib timing_path timing_point timing_arc  
path_group lib_timing_arc si_victim si_attacker lib_pg_pin pg_pin pg_net}
```

Creates a property alias for the selected objects.

Parameters

-help	Prints out the command usage.
-map <alias_property_name_pairs>	Specifies a pair list of aliases and properties to be associated with that alias, such as {alias1 prop1} or {{ialias1 prop1}{alias2 prop2}...}.
-obj_type {pin port cell net clock lib_cell lib_pin design lib timing_path timing_point timing_arc path_group lib_timing_arc si_victim si_attacker lib_pg_pin pg_pin pg_net}	Specifies the object type for which a property alias needs to be created.

create_rc_corner

```
create_rc_corner
[-help]
[-cap_table capTableFile]
-name rcCornerName
[-post_route_cap {capFactor1 capFactor2 capFactor3}]
[-post_route_clock_cap {capFactor1 capFactor2 capFactor3}]
[-post_route_clock_res {resFactor1 resFactor2 resFactor3}]
[-post_route_res {resFactor1 resFactor2 resFactor3}]
[-post_route_cross_cap {xcapFactor1 xcapFactor2 xcapFactor3}]
[-pre_route_cap capFactor]
[-pre_route_clock_cap capFactor]
[-pre_route_clock_res resFactor]
[-pre_route_res resFactor]
[-qrc_tech fileName]
[-temperature temperatureValue]
[-via_variation_file string]
```

Creates a named RC corner object that can be referenced later when creating a delay calculation corner object. An RC corner object provides the software with all of the information necessary to properly extract, annotate, and use the RCs for delay calculation.

Note: You must use the RC corner scaling attributes when running the software in multi-mode multi-corner analysis mode. Use the [extract_rc Category Attributes](#) to set the extraction mode for all RC corner objects in the design.

Note: In multi-mode multi-corner mode, only the default RC corners created by the software inherit the derating factor values defined in the configuration file. You can set different postroute scale factors for the postroute engine variants by specifying them as duplets and triplets. For example,

```
resFactor1 resFactor2 resFactor3
```

Where:

- Single value: If you specify one value, the scale factor applies to effort level `low`. Scale factor value of `1` is used for `medium` and `high` by default.
- Duplet: If you specify two values, the first value is used for effort level `low` and the second value is used for `medium`. Scale factor value of `1` is used for `high` by default.
- Triplet: If you specify three values, the first value is used for effort level `low`, the second value is used for `medium`, and the third value is used for `high`.

Note: If the `read_spef` command is specified without the `-rc_scale` parameter, for example, "`read_spef -rc_corner early_spef_${rc_corner}`", then the software does not apply capacitance scale factor specified in `-post_route_cap` and `-post_route_cross_cap` parameters. If the `read_spef` command is specified with the `-rc_scale` parameter, for example, "`read_spef -rc_corner early_spef_${rc_corner} -rc_scale` ", then the software applies the capacitance scale factor. Here, no factor is specified in the `-rc_scale` option.

Parameters

<code>-help</code>	Outputs a brief description that includes type and default information for each <code>create_rc_corner</code> parameter. For a detailed description of the command and all of its parameters, use the man command: <code>man create_rc_corner</code> .
<code>-cap_table capTableFile</code>	Specifies the capacitance table to use for extraction when using this RC corner. <i>Default:</i> The Litmus extraction system uses internal rules for extracting resistances and capacitances.
<code>-pre_route_cap capFactor</code>	Specifies the capacitance scale factor for RC Extraction in pre-route mode. <i>Default:</i> 1.0
<code>-pre_route_clock_cap capFactor</code>	Specifies the scale factor for capacitance of clock nets in preroute mode. <i>Default:</i> 0 When this parameter is not specified, the value specified with the <code>-pre_route_cap</code> parameter is used for preroute extraction. When specified, this parameter overrides the value specified with the <code>-pre_route_cap</code> parameter for clock nets.
<code>-pre_route_clock_res resFactor</code>	Specifies the scale factor for resistance of clock nets in preroute mode. <i>Default:</i> 0 When this parameter is not specified, the value specified with the <code>-pre_route_res</code> parameter is used for preroute extraction. When specified, this parameter overrides the value specified with the <code>-pre_route_cap</code> parameter for clock nets.
<code>-pre_route_res resFactor</code>	Specifies the resistance scale factor for RC extraction in pre-route mode.

<pre>-post_route_cap {capFactor1 capFactor2 capFactor3}</pre>	Specifies the capacitance scale factor for RC extraction in post-route mode. <i>Default:</i> 1.0
<pre>-post_route_clock_cap {capFactor1 capFactor2 capFactor3}</pre>	Specifies the scale factor for capacitance of clock nets in the postroute extraction mode. <i>Default:</i> 0 When this parameter is not specified, the value specified with the <code>-post_route_cap</code> parameter is used for postroute extraction. When specified, this parameter overrides the value specified with the <code>-post_route_cap</code> parameter.
<pre>-post_route_clock_res {resFactor1 resFactor2 resFactor3}</pre>	Specifies the scale factor for resistance of clock nets in the postroute extraction mode. <i>Default:</i> 0 When this parameter is not specified, the value specified with the <code>-post_route_res</code> parameter is used for postroute extraction. When specified, this parameter overrides the value specified with the <code>-post_route_res</code> parameter for clock nets.
<pre>-post_route_res {resFactor1 resFactor2 resFactor3}</pre>	Specifies the resistance scale factor for RC extraction in post-route mode.
<pre>-post_route_cross_cap {xcapFactor1 xcapFactor2 xcapFactor3}</pre>	Specifies the coupling capacitance scale factor for RC extraction in postroute mode. <i>Default:</i> 1.0
<pre>-name rcCornerName</pre>	Specifies the name for the RC corner object being created. You can choose any name for an RC corner object, but it is recommended that you use a name that can be easily recognized as referring to a specific RC corner (for example, rc-min, rc-max, c-min, c-max, and so on).

<code>-qrc_tech fileName</code>	Specifies the name of the Quantus technology (.tch) file used by Quantus sign-off RC extraction. The technology file is generated by TechGen, a built-in functionality of Quantus. For most technologies you can get this file from technology vendors. For technologies that are not available through the technology vendors, you can enter the fabrication process information into an ASCII-format interconnect technology (ICT) input file. The ICT file is then used as input to IceCaps, which in turn generates the technology file.
<code>-temperature temperatureValue</code>	Specifies the temperature, in units of Celsius, to use to derate resistance values. Use this parameter when you want to override the default temperature specified in the capacitance table or the Quantus technology file. Note: The software does not require that the temperature specified by the PVT operating condition at the delay calculation corner level be the same as the RC nominal or user-specified temperature value. <i>Default:</i> The Litmus extraction system uses the default temperature of 25 degrees Celsius, unless it is overridden by the capacitance table or the Quantus technology file.
<code>-via_variation_file string</code>	Specifies a file for modeling via variation. The VIA variation file lists up all the VIA layer names with the corresponding multiplier. Each RC corner can have its own VIA variation file. For example: <pre>create_rc_corner -name rcCornerName1 \ -via_variation_file viaVariationFileName1 create_rc_corner -name rcCornerName2 \ -via_variation_file viaVariationFileName2</pre>

Examples

- The following command creates an RC corner called `rc-cbest` that uses the capacitance table `myTech_4_bc.CapTbl`, and derates the resistance values based on the temperature of 50 Celsius:

```
create_rc_corner -name rc-cbest -cap_table myTech_4_bc.CapTbl -temperature 50
```

Related Commands

- `update_rc_corner`

create_timing_condition

```
create_timing_condition
[-help]
-library_sets string
-name timingConditionName
[-opcond string]
[-opcond_library string]
```

Creates a set of virtual operating conditions in the specified library.

Parameters

-help	Prints out the command usage. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man create_timing_condition</code> .
-library_sets <i>string</i>	Specifies a list of library sets to associate with the specified timing condition.
-name <i>timingConditionName</i>	Specifies a unique name for the timing condition.
-opcond <i>string</i>	Specifies a single name of an operating condition to associate with this timing condition.
-opcond_library <i>string</i>	Specifies the name of library to search for the named operating condition. Notes: • The opcond library must be a library name, and not a library file path. • In a trilib flow, where multiple library sets of different PVTs are used for opcond interpolation, the opcond library should be close to the operating condition.

Examples

- The following commands create a set of timing conditions for the defined list of library sets:

```
create_library_set -name ls -timing [list typical.lib]
create_timing_condition -name tc1 -library_sets [list ls]
```
- The following commands create a set of virtual operating and timing conditions on the defined library sets:

```
create_opcond -name OC -process 1 -voltage 0.8 -temperature 25
create_timing_condition -name tc1 -library_sets [list ls] -opcond OC
```

Related Commands

- `update_timing_condition`

delete_analysis_view

```
delete_analysis_view  
[-help]  
analysis_view_list
```

Deletes the specified analysis views.

Parameters

<code>-help</code>	Prints out the command usage.
<code>analysis_view_list</code>	Specifies the list of views for deleting.

Related Commands

- `create_analysis_view`
- `update_analysis_view`
- `set_analysis_view`
- `report_analysis_views`

get_capacitance_unit

get_capacitance_unit

Returns the capacitance unit (in picofarads) for the current session, which may be derived from the library file or set with the `set_library_unit -cap` parameter. This unit will be used while generating reports and for constraint settings.

Parameters

-help	Prints out the command usage.
-------	-------------------------------

Examples

The following command returns the session capacitance unit:

```
get_capacitance_unit
```

```
1.000000pf
```

Related Commands

- `set_library_unit`

get_clock_network_objects

```
get_clock_network_objects  
[-help]  
[-clocks clock_list]  
[-type {cell | pin}]
```

Returns the cells or pins that exist in a clock network of the specified clocks.

Parameters

-help	Prints out the command usage.
-clocks <i>clock_list</i>	Returns the cells or pins present in a clock network of the specified clock(s).
-type {cell pin}	Returns the specified cells or pins in the clock network.

Examples

- The following command returns the pins present in the clock network of all the clocks:

```
litmus > get_clock_network_objects
```
- The following command returns the pins that exist in the clock network of CLK and CLK2 clocks:

```
litmus > get_clock_network_objects -clocks {CLK CLK2}
```
- The following command returns the cells that exist in the clock network of CLK clock.

```
litmus > get_clock_network_objects -clocks {CLK} -type cell
```

get_clock_to_clock_association

```
get_clock_to_clock_association  
[-help]  
clock_list  
[-allow_paths]  
[-group_name group_name]  
[-type {logically_exclusive | physically_exclusive | asynchronous}]  
[-view viewName]
```

Reports clock to clock associations.

Parameters

-help	Prints out the command usage.
<i>clock_list</i>	Specifies a list of clocks that must contain two entries.
-allow_paths	Checks whether the two specified clocks are asynchronous and timing analysis is allowed between these clocks.
-group_name <i>group_name</i>	Reports clock to clock association for the specified clocks along with the clock group name.
-type {logically_exclusive physically_exclusive asynchronous}	Checks clock to clock association for the specified type of clocks. You can use one of the following options: • logically_exclusive: Reports clock group with clocks that do not have any functional paths between them, but coexist in the design at the same time. • physically_exclusive: Reports clock group with clocks that cannot exist in the design at the same time. • asynchronous: Reports clock group with asynchronous clocks. Asynchronous clocks have no specified phase relationship.
-view <i>viewName</i>	Generates a report for the specified analysis view.

Related Commands

- `create_clock`
- `create_generated_clock`
- `set_clock_groups`

get_interactive_constraint_modes

`get_interactive_constraint_modes`

Returns a list of the multi-mode multi-corner constraint modes that are in interactive constraint entry mode.

When the software is in interactive constraint entry mode (`set_interactive_constraint_modes`), any timing constraints you specify take effect immediately on all active analysis views that are associated with these constraint modes.

Note: Interactive constraint mode only works when the software is in multi-mode multi-corner timing analysis mode.

Related Commands

- `set_interactive_constraint_modes`

get_multi_cpu_usage

```
get_multi_cpu_usage
[-help]
[-auto_page_fault_monitor]
[-cpu_per_remote_host]
[-keep_license]
[-local_cpu]
[-remote_host]
[-thread_info]
[-verbose]
```

Displays the requested number of threads or hosts.

To request threads or hosts, use `set_multi_cpu_usage`.

Parameters

-help	Prints out the command usage.
- auto_page_fault_monitor	Displays the warning rank set for performance issues caused by major page faults
-cpu_per_remote_host	Displays the number of CPUs available on every remote host for Superthreading.
-help	Outputs the command usage and a brief description about the command parameters.
-keep_license	Displays the current setting for the <code>set_multi_cpu_usage -keep_license</code> parameter.
-local_cpu	Displays the number of available local CPUs for multi-threading.
-remote_host	Displays the number of available remote hosts for distributed processing or Superthreading.
-thread_info	Displays the level of informational messages for each thread.
-verbose	Displays a message with the current multiple-CPU setting.

Examples

- The following command displays the requested number of threads for multi-threading:
`get_multi_cpu_usage -local_cpu`
- The following command displays the requested number of remote hosts for Superthreading or distributed processing:
`get_multi_cpu_usage -remote_host`
- The following command displays the requested number of CPUs per remote hosts for Superthreading:
`get_multi_cpu_usage -cpu_per_remote_host`

Related Commands

- `set_multi_cpu_usage`

get_propagated_clock

```
get_propagated_clock  
[-help]  
[-clock clock_list] [-pin pin_list]  
[> filename]
```

Returns the clock propagation mode, either `ideal` or `propagated`, for the specified clocks or pins.

Parameters

<code>-help</code>	Prints out the command usage.
<code>> <i>filename</i></code>	Specifies the name of a file in which to save the report. You can add the <code>.gz</code> extension to the file name to generate a compressed report. Note: The file name must be the last argument in the list.
<code>-clock <i>clock_list</i></code>	Specifies one or more clock waveforms for which you want to retrieve the clock propagation mode information.
<code>-pin <i>pin_list</i></code>	Specifies one or more pins or ports for which you want to retrieve the clock propagation mode information. You cannot specify hierarchical pins.

Examples

- The following command gets the clock propagation mode information for all clock waveforms:

```
get_propagated_clock -clock {*}
```

Related Commands

- [set_propagated_clock](#)

get_resistance_unit

`get_resistance_unit`

Returns the resistance unit for the current session.

Examples

The following command returns the resistance unit for the current session:

```
get_resistance_unit
```

```
1.000000 kohm
```

get_selectively_expanded_modules

get_selectively_expanded_modules
[-help]

Displays the values set by the [set_selectively_expanded_modules](#) command for abstracting cloned instances of modules with a high number of instantiations, and flattening one or more of them.

Parameters

-help	Prints a brief description that includes the type and default information for each <code>get_selectively_expanded_modules</code> parameter. For a detailed description of the command and all its parameters, use the man command: <pre>man get_selectively_expanded_modules</pre>
-------	--

Related Topics

[set_selectively_expanded_modules](#)

get_time_unit

```
get_time_unit  
[-help]
```

Returns the current time unit (in nanoseconds) for the session derived from the library file or set with the `set_library_unit -time` parameter. This unit will be used during report generation and constraint settings.

Parameters

<code>-help</code>	Prints out the command usage.
--------------------	-------------------------------

Examples

- The following command returns the time unit for the session:

```
get_time_unit
```

```
1.000000ns
```

Related Commands

- `set_library_unit`

list_libraries

```
list_libraries
[-help]
[-power_domain string]
[-used]
[-used_analysis]
[-view string]
```

Lists all the libraries loaded in the session and used in the design.

Parameters

-help	Prints out the command usage.
-power_domain <i>string</i>	Specifies the power domain from which instances are to be selected.
-used	Prints out a list of libraries used in the design and all the other libraries, whose voltages match with those of the used libraries (like vt variant libraries). This list is useful for ECO optimization.
-used_analysis	Prints out a list of used libraries in the design.
-view <i>string</i>	Specifies the view name whose libraries will be listed.

Examples

Here are some examples.

- The following command reports all the libraries loaded in a session:

```
tempus> list_libraries
```

```
Library      Library      Library
Group        Name         Path
-----
max          fast         /cte//data/libs/t18/fast.lib
min          slow         /cte//data/libs/t18/slow.lib
-----
```

Related Commands

- `current_design`

read_boundary_model

```
read_boundary_model  
[-help]  
[-check_contents]  
[-instances string]  
[-report_file string]  
[-skip_version_check]  
[-unconstrained_net_use_infinite_timing_windows]  
[-clock_mapping_file string]  
[-view_mapping_file string]  
{-dir string}
```

Saves the directory information.

This command can be used for each cell boundary model generated by the block-level runs.

Parameters

-help	Prints out the command usage.
-check_contents	Checks for required files in the boundary model directory.

<code>-clock_mapping_file string</code>	Specifies the block to top-level clock mapping file. This command uses the mapping file created by the <code>set_boundary_model_clock_mapping</code> command. The basic syntax of <code>-clock_mapping_file</code> is: <pre>Block_clock_name (view_name) Top_clock_name (view_name) instance_name</pre> However, both <code>view_name</code> and <code>instance_name</code> are optional. If you want to have the same mapping for all instances and views then it is specified as follows: <pre>Block_clock_name Top_clock_name</pre> • If it is instance specific but not view specific then it is specified as follows: <pre>Block_clock_name Top_clock_name instance_name</pre> • If it is view specific but not instance specific then it is specified as follows: <pre>Block_clock_name (view_name) Top_clock_name (view_name)</pre> Note that no spaces are currently allowed between the clock name and the view name.
<code>-dir string</code>	Specifies the directory for boundary model.
<code>-instances string</code>	Specifies the instances for which the boundary model must be read.
<code>-report_file string</code>	Specifies the name of the boundary model check report file. After the boundary model is checked and the results are logged into this file. If this parameter is not specified, then by default the report gets written to <code><cell_name>_bmodel_check.txt</code> file.
<code>-skip_version_check</code>	Does not check for Tempus version consistency.
<code>- unconstrained_net_use_infinite_timing_windows</code>	Forces unconstrained nets to infinite timing windows.
<code>-view_mapping_file string</code>	Uses the mapping file created by the <code>set_boundary_model_view_mapping</code> command.

Examples

- The following command loads the boundary model files from the ./model_dir directory:

```
read_boundary_model -dir ./model_dir
```

- The following command loads the boundary model files from the ./model_dir directory:

```
read_boundary_model -dir ./model_dir -clock_mapping_file clock.data
```

The clock.data file will be read for user-defined updates for clock mappings from the boundary model clock names to the clocks defined in the constraints for the current session.

Related Commands and Variables

- `create_boundary_model`
- `set_boundary_model_view_mapping`
- `set_boundary_model_clock_mapping`

read_ilm

```
read_ilm  
[-help]  
[-no_si]  
[-no_spef]  
[-oa_cellview cell view]
```

Specifies ILM data directory for the specified block. You can run `read_ilm` multiple times in the same session. Each time you specify `read_ilm`, the software overwrites the previous setting for the block. If master/clones exist in the design, the cell name will have the name of the master partition.

Note: You can use this command only if the ILMs are unflattened (`unflattenIlm`) or the design is in black box view. You cannot change ILM settings in flattened or ILM view.

The syntax is order-dependent. You must specify `-cell` before you specify `-directory`.

Note: The ILM is not supported for Signoff in Tempus.

Parameters

<code>-help</code>	Outputs a brief description that includes type and default information for each <code>read_ilm</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man read_ilm</code>
<code>-no_si</code>	Specifies to read the non-SI data from the ilm directory.
<code>-no_spef</code>	Specifies to read the non-Spef data from the ilm directory.
<code>-oa_cellview <i>cell view</i></code>	Specifies the library name, cell name, and view name in the OpenAccess database format.

Related Topic

- [write_ilm](#)

report_analysis_coverage

```
report_analysis_coverage
[-help]
[pin | port]
[-check_type check_type_list]
[-exclude_untested reason_list]
[-include_dynamic_checks]
[-include_parallel_check_arcs]
[-include_when_cond]
[-max_paths {type integer}]
[-nworst integer]
[-retime {aocv | ssta | path_slew_propagation | aocv_path_slew_propagation}]
[-sort {pin | refpin | checktype | slack | reason}]
[-tcl_list]
[-unsorted]
[-verbose check_status_list]
[-view view_name]
[> <filename[.gz]>]
[>> <filename[.gz]>]
```

Provides information about timing checks in the design.

For each type of timing check, the command reports the number of checks that meet constraints, violated constraints, or that are untested in the SUMMARY section of the report. The DETAILS section includes information about the signal pin, reference pin, check type, slack, and the reason for being untested.

When timing_report_group_based_mode is set to true, this command will report a check at end points for all the clocks. In verbose mode, the "clock" column will be displayed.

 Use `check_timing` to look for missing constraints.

Parameters

-help	Prints out the command usage.
-------	-------------------------------


```
{> | >>} file_name
```

When specified with `>`, writes the report to the specified file name. If the file already exists, the software overwrites it.

When specified with `>>`, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file.

The *filename* parameter must be the last argument in the list. The *filename* and `-tcl_list` parameters are mutually exclusive; you cannot specify them together.

Note: To write a compressed report, add the `.gz` extension to the file name.

Default: Report is displayed on standard output without being saved.

`-check_type check_type_list`

Reports checks that as specified in the *check_type_list*.

Valid check types are:

- clock_gating_hold
- clock_gating_pulse_width
- clock_gating_setup
- clock_period
- clock_separation
- data_hold
- data_setup
- external_delay:
- hold
- no_change_hold
- no_change_setup
- path_delay
- pulse_width
- recovery
- removal
- setup
- skew
- time_borrow

Note: If there are no specified timing checks in the design, the report shows "No timing checks found."

Default: All (all types of check(s) are reported)

<code>-exclude_untested {reason_list}</code>	Excludes from the report endpoints that are unconstrained due to a specific reason. Valid reasons are: • <code>const</code>: Constant reaching signal/reference pin of any timing check. • <code>false_path</code>: Path disabled due to <code>set_false_path</code>. • <code>no_endpoint_clock</code>: Missing clock at the reference pin of the timing check. • <code>no_startpoint_clock</code>: Missing clock at the signal pin of the timing check. This may happen if the clock signal does not reach the launching register. • <code>unknown</code>: None of the known reasons are identified for an untested path. • <code>user_disable</code>: Checks arc disable due to <code>set_disable_timing</code>.
<code>-include_dynamic_checks</code>	Reports the dynamic checks (<code>data_to_data</code>, <code>path_delay</code>) on pins that are constrained by <code>set_data_check</code>, <code>set_max_delay</code>, or <code>set_min_delay</code> constraints. By default, these checks will not be reported and the command only reports the pins constrained by all library checks, <code>set_output_delay</code> or inferred clock gating checks. The dynamic checks, such as <code>set_data_check</code> or <code>set_max_delay</code> are not reported by default, when <code>-include_dynamic_checks</code> parameter is not specified.
<code>-include_parallel_check_arcs</code>	Reports all the parallel check arcs that have been merged.
<code>-include_when_cond</code>	Reports the when condition of check arcs.
<code>-max_paths {type integer}</code>	Enumerates the specified number of worst paths in the design, regardless of the endpoint.
<code>-nworst integer</code>	Enumerates the specified number of paths for each endpoint.

<pre>-retime {aocv ssta path_slew_propagation aocv_path_slew_propagation}</pre>	Reanalyzes the set of paths using the specified method.
<pre>pin port</pre>	Limits the report to timing checks on specified pins or ports.
<pre>-sort {pin reffpin checktype slack reason}</pre>	Sorts the checks by the specified option while reporting detailed information using the <code>-verbose</code> option. The <code>-sort</code> parameter does not change the order of the columns. The valid sort options are: <code>pin</code>, <code>reffpin</code>, <code>checktype</code>, <code>slack</code>, <code>reason</code> When sorting by pin name, if the pin names are the same, the software then attempts to sort the checks by slack value. If the slack values are the same, the software attempts to sort by timing check type. When sorting by timing check type, if the check types are the same, the software then attempts to sort by slack value. If the slack values are the same, the software attempts to sort by pin name. When sorting by slack, if the slack values are the same, the software then attempts to sort the checks by pin name. If the pin names are the same, the software attempts to sort by timing check type. When you sort by <code>slack</code>, the slack value is reported in order of least negative to most negative (worst slack last). You can use the following commands for debugging purposes and to improve readability of the report: <pre>report_analysis_coverage -verbose untested - sort reason report_analysis_coverage -verbose violated - sort slack</pre> Default: By default, checks are sorted by slack value (<code>slack</code>).

<code>-tcl_list</code>	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. The <code>-tcl_list</code> and <code>filename</code> parameters are mutually exclusive; you cannot specify them together.
<code>-unsorted</code>	Reports unsorted slacks with untested end points at the end of the report.
<code>-verbose check_status_list</code>	Limits the detailed information to checks as per the specified status. The valid status types are: <code>met</code>, <code>violated</code>, and <code>untested</code>. • <code>met</code>: Shows as a positive value in the Slack column. • <code>violated</code>: Shows as a negative value in the Slack column. • <code>untested</code>: Shows as UNTESTED in the Slack column and the reason is given in the Reason column. The Reason column will be blank when a Slack number is displayed - this indicates the check has been tested. You can reduce the number of lines in the DETAILS section by filtering out the met checks, as shown in the example below: <pre>report_analysis_coverage -verbose {violated untested}</pre> Note: The <code>-verbose</code> parameter supports multi-threading.
<code>-view view_name</code>	Generates a timing check report for the specified analysis view only. You can specify this parameter only when the software is in multi-mode multi-corner analysis mode.

Examples

- The following command shows the report output for pulse width checks for the specified view:

```
litmus > report_analysis_coverage -check_type pulse_width -view func_slow_RCMAx
```

Litmus Text Command Reference
Timing Analysis Commands--report_analysis_coverage

TIMING CHECK COVERAGE SUMMARY (Analysis View: func_slow_RCMAx)

Check Type No. of Met Violated Untested
 Checks

PulseWidth 1541 1096 (71%) 42 (2%) 403 (26%)

- The following command shows the report output for the specified view port:

```
litmus > @report_analysis_coverage port_pad_data_out
```

TIMING CHECK COVERAGE SUMMARY

Check Type No. of Met Violated Untested
 Checks

ExternalDelay (Early) 16 16 (100%) 0 (0%) 0 (0%)
ExternalDelay (Late) 16 0 (0%) 16 (100%) 0 (0%)

- The following command reports checks for graph-based analysis (GBA) paths:

```
litmus > report_analysis_coverage -retime path_slew_propagation
```

TIMING CHECK COVERAGE SUMMARY

Check Type No. of Met Violated Untested
 Checks

ClockPeriod 6 6 (100%) 0 (0%) 0 (0%)
ExternalDelay (Early) 29 26 (89%) 0 (0%) 3 (10%)
ExternalDelay (Late) 29 0 (0%) 26 (89%) 3 (10%)
Hold 1741 1486 (85%) 37 (2%) 218 (12%)
PulseWidth 1541 1096 (71%) 42 (2%) 403 (26%)
Recovery 377 1 (0%) 3 (0%) 373 (98%)
Removal 377 4 (1%) 0 (0%) 373 (98%)
Setup 1741 951 (54%) 574 (32%) 216 (12%)

- The following command shows untested checks falling under the met status:

```
litmus > report_analysis_coverage -exclude_untested false_path -verbose met
```

Litmus Text Command Reference
Timing Analysis Commands--report_analysis_coverage

TIMING CHECK COVERAGE SUMMARY

Check Type	No. of Checks	Met	Violated	Untested
ClockPeriod	6	6 (100%)	0 (0%)	0 (0%)
ExternalDelay (Early)	29	26 (89%)	0 (0%)	3 (10%)
ExternalDelay (Late)	29	0 (0%)	26 (89%)	3 (10%)
Hold	1739	1486 (85%)	37 (2%)	216 (12%)
PulseWidth	1541	1096 (71%)	42 (2%)	403 (26%)
Recovery	138	1 (0%)	3 (2%)	134 (97%)
Removal	138	4 (2%)	0 (0%)	134 (97%)
Setup	1741	950 (54%)	575 (33%)	216 (12%)

TIMING CHECK COVERAGE DETAILS

Pin	Reference Pin	Check Type	Slack	Reason
RTS_INST/c1r_flg/SI ^	RTS_INST/c1r_flg/CK ^	Setup	0.001	-

- The following command generates a timing check report for the specified analysis view:
`litmus > report_analysis_coverage -view scan_slow_RCMAX`

TIMING CHECK COVERAGE SUMMARY (Analysis View: scan_slow_RCMAX)

Check Type	No. of Checks	Met	Violated	Untested
ClockPeriod	6	6 (100%)	0 (0%)	0 (0%)
ExternalDelay (Early)	29	26 (89%)	0 (0%)	3 (10%)
ExternalDelay (Late)	29	26 (89%)	0 (0%)	3 (10%)
Hold	1741	458 (26%)	19 (1%)	1264 (72%)
PulseWidth	1541	1138 (73%)	0 (0%)	403 (26%)
Recovery	377	4 (1%)	0 (0%)	373 (98%)
Removal	377	2 (0%)	0 (0%)	375 (99%)
Setup	1741	1525 (87%)	0 (0%)	216 (12%)

- The following command filters from the report all endpoints that are unconstrained because of the reason `const`:

```
litmus> report_analysis_coverage -exclude_untested const -verbose untested
```

TIMING CHECK COVERAGE SUMMARY

Check Type	No. of Checks	Met	Violated	Untested
Clock Gating Hold	1	0 (0%)	1 (100%)	0 (0%)
Clock Gating Setup	1	1 (100%)	0 (0%)	0 (0%)
ClockPeriod	28	24 (85%)	0 (0%)	4 (14%)
ExternalDelay (Early)	1	0 (0%)	0 (0%)	1 (100%)
ExternalDelay (Late)	1	0 (0%)	0 (0%)	1 (100%)
Hold	13	8 (61%)	1 (7%)	4 (30%)
PulseWidth	28	24 (85%)	0 (0%)	4 (14%)
Setup	13	8 (61%)	1 (7%)	4 (30%)

TIMING CHECK COVERAGE DETAILS

Pin	Reference Pin	Check Type	Slack	Reason
out1	-	ExternalDelay (Early)	UNTESTED	No data signal
out1	-	ExternalDelay (Late)	UNTESTED	No data signal
u99/CK ^	u99/CK ^	ClockPeriod	UNTESTED	No endpoint clock
u99/CK ^	u99/CK v	PulseWidth	UNTESTED	No endpoint clock
u99/CK v	u99/CK ^	PulseWidth	UNTESTED	No endpoint clock
u99/CK v	u99/CK v	ClockPeriod	UNTESTED	No endpoint clock
u99/D	u99/CK ^	Hold	UNTESTED	No endpoint clock
u99/D	u99/CK ^	Setup	UNTESTED	No endpoint clock
u8/CK ^	u8/CK ^	ClockPeriod	UNTESTED	No endpoint clock
u8/CK ^	u8/CK v	PulseWidth	UNTESTED	No endpoint clock
u8/CK v	u8/CK ^	PulseWidth	UNTESTED	No endpoint clock
u8/CK v	u8/CK v	ClockPeriod	UNTESTED	No endpoint clock
seg1/u9/D	seg1/u9/CK ^	Hold	UNTESTED	No data signal
seg1/u9/D	seg1/u9/CK ^	Setup	UNTESTED	No data signal
seg2/u9/D	seg2/u9/CK ^	Hold	UNTESTED	No data signal
seg2/u9/D	seg2/u9/CK ^	Setup	UNTESTED	No data signal

When the software is in multi-mode multi-corner timing analysis mode, You can specify the `-view` parameter to generate a timing check report for the specified analysis view only:

```
litmus 150> report_analysis_coverage -view view1
```

TIMING CHECK COVERAGE SUMMARY (Analysis View: view1)

Check Type	No. of	Met	Violated	Untested
------------	--------	-----	----------	----------

Litmus Text Command Reference
Timing Analysis Commands--report_analysis_coverage

Checks

Clock Gating Hold	1	0 (0%)	1 (100%)	0 (0%)
Clock Gating Setup	1	1 (100%)	0 (0%)	0 (0%)
ClockPeriod	28	24 (85%)	0 (0%)	4 (14%)
ExternalDelay (Early)	1	0 (0%)	0 (0%)	1 (100%)
ExternalDelay (Late)	1	0 (0%)	0 (0%)	1 (100%)
Hold	14	8 (57%)	1 (7%)	5 (35%)
PulseWidth	28	24 (85%)	0 (0%)	4 (14%)
Setup	14	8 (57%)	1 (7%)	5 (35%)

Related Commands

- `check_timing`
- `report_timing`

report_analysis_summary

```
report_analysis_summary
[-help]
[-csv string]
[-merged_views]
[-retime {path_slew_propagation | aocv_path_slew_propagation}]
[-retime_mode {exhaustive}]
[-view viewName]
[> <filename[.gz]>]
[>> <filename[.gz]>]
[-early | -late ]
[-expanded_summary_clocks | -merged_groups]
```

Generates timing slack summary reports for specified analysis view(s). This command categorizes the slack summary in terms of groups - In2reg, Reg2reg, Reg2Out, and In2Out. A column "ALL" is displayed that regroups in2reg, in2out, reg2reg, and reg2out details.

By default, the slack is categorized in terms of worst negative slack (WNS), total negative slack (TNS), and the total number of violations in group-based mode.

The `report_analysis_summary` command is supported in path-based analysis (PBA) mode.

Parameters

<code>-help</code>	Prints out the command usage.
<code>{> >>} filename</code>	When specified with <code>></code>, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with <code>>></code>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. Note: To write a compressed report, add the <code>.gz</code> extension to the file name. <i>Default:</i> Report is displayed on standard output without being saved.
<code>-csv string</code>	Specifies the name of the CSV output file.
<code>-early</code>	Generates report for hold analysis mode for early paths.
<code>-expanded_summary_clocks</code>	Generates report in terms of each possible clock domain pair separately.
<code>-late</code>	Generates report for setup analysis mode for late paths.
<code>-merged_groups</code>	Merges all the clock groups for a consolidated timing summary.
<code>-merged_views</code>	Generates a merged report across all the views.
<code>-retime {path_slew_propagation aocv_path_slew_propagation}</code>	Reanalyzes the defined set of paths using the specified method. By default, the timing report shows a maximum of 2 million paths for retime PBA analysis.
<code>-retime_mode {exhaustive}</code>	Reanalyzes the specified set of paths using the specified retime mode.
<code>-view viewName</code>	Generates report for the specified analysis views only. By default, all the active views are analyzed.

Examples

- The `report_analysis_summary` command shows the following report output:

```
tempus> report_analysis_summary
```

Litmus Text Command Reference

Timing Analysis Commands--report_analysis_summary

```
REPORT ANALYSIS SUMMARY (View:default_emulate_view) (EL:Setup)
```

```
=====
```

```
REPORT ANALYSIS SUMMARY (View:default_emulate_view) (EL:Hold)
```

```
=====
```

```
GroupType:ClockGate
```

```
=====
```

All	IN2REG	IN2OUT	REG2REG	REG2OUT	
WNS:TNS:#VIO	WNS:TNS:#VIO	WNS:TNS:#VIO	WNS:TNS:#VIO	WNS:TNS:#VIO	FromClk->ToClk
-5.614:-5.614:1	0.000:0.000:0	0.000:0.000:0	-5.614:-5.614:1	0.000:0.000:0	*->*

```
GroupType:Standard
```

All	IN2REG	IN2OUT	REG2REG	REG2OUT	
WNS:TNS:#VIO	WNS:TNS:#VIO	WNS:TNS:#VIO	WNS:TNS:#VIO	WNS:TNS:#VIO	FromClk->ToClk
-8.897:-8.897:1	0.000:0.000:0	0.000:0.000:0	-8.897:-8.897:1	0.000:0.000:0	*->CLK_W_1
-0.571:-0.571:1	-0.571:-0.571:1	0.000:0.000:0	0.000:0.000:0	0.000:0.000:0	*->CLK_W_2

- The following command generates a consolidated summary of all the clock groups and for all the views:

```
tempus > report_analysis_summary -merged_groups -merged_views
```

```
REPORT ANALYSIS SUMMARY (EL:Setup)
```

```
=====
```

```
GroupType:*
```

```
=====
```

All	IN2REG	IN2OUT	REG2REG	REG2OUT
WNS:TNS:#VIO	WNS:TNS:#VIO	WNS:TNS:#VIO	WNS:TNS:#VIO	WNS:TNS:#VIO
FromClk->ToClk				
-22.239:-5171.932:604	-2.274:-5.220:4	0.000:0.000:0	-22.239:-5089.035:574	-5.034:-77.678:26
				->

```
REPORT ANALYSIS SUMMARY (EL:Hold)
```

```
=====
```

```
GroupType:*
```

```
=====
```

Litmus Text Command Reference
Timing Analysis Commands--report_analysis_summary

```

All                IN2REG                IN2OUT                REG2REG                REG2OUT
-----
WNS:TNS:#VIO      WNS:TNS:#VIO      WNS:TNS:#VIO      WNS:TNS:#VIO      WNS:TNS:#VIO
  FromClk->ToClk
-----
-2.108:-46.905:37  -1.094:-14.238:19  0.000:0.000:0    -2.108:-32.668:18  0.000:0.000:0    *-
>*
```

- The following command reports retimed early paths for hold analysis mode:

```

tempus 149> report_analysis_summary -retime path_slew_propagation -early

REPORT ANALYSIS SUMMARY (View:func_slow_RCMAX) (EL:Hold)
=====

GroupType:Standard
=====

All                IN2REG                IN2OUT                REG2REG                REG2OUT
-----
WNS:TNS:#VIO      WNS:TNS:#VIO      WNS:TNS:#VIO      WNS:TNS:#VIO      WNS:TNS:#VIO
  FromClk->ToClk
-----
-2.108:-46.804:37  -1.093:-14.151:19  0.000:0.000:0    -2.108:-32.654:18  0.000:0.000:0    *-
>m_clk

INFO: Out of 37 endpoints finally reported to be violating, 0 number of endpoints reported
Graph Based Analysis (GBA) bounded slack. Refer to message TCLCMD-1251 and TCLCMD-1377 in log
file for details regarding the reason for GBA bounding.
```

- The following command reports each possible clock domain pair separately:

```

tempus > report_analysis_summary -expanded_summary_clocks

REPORT ANALYSIS SUMMARY (View:func_slow_RCMAX) (EL:Setup)
=====

GroupType:Async
=====

All                IN2REG                IN2OUT                REG2REG                REG2OUT
-----
--
WNS:TNS:#VIO      WNS:TNS:#VIO      WNS:TNS:#VIO      WNS:TNS:#VIO      WNS:TNS:#VIO      FromClk-
>ToClk
-----
--
-0.618:-1.229:2    -0.618:-1.229:2    0.000:0.000:0    0.000:0.000:0    0.000:0.000:0    refclk->*
-0.618:-1.229:2    -0.618:-1.229:2    0.000:0.000:0    0.000:0.000:0    0.000:0.000:0    m_rcc_clk->*
```

```
-0.618:-1.229:2  -0.618:-1.229:2  0.000:0.000:0  0.000:0.000:0  0.000:0.000:0  m_ram_clk->*
```

Related Commands

- `set_default_view`
- `set_analysis_view`

report_analysis_views

```
report_analysis_views
-help
[-type {all | setup | hold | active}]
```

Generates a hierarchical report of the current multi-mode multi-corner configuration. By default, the `report_analysis_views` command generates a report of all defined views in the design. Use the `-type` parameter to generate a report on a specific type of analysis view.

Use this command after creating analysis views (`create_analysis_view`), and setting active setup and hold views (`set_analysis_view`) for the design.

Parameters

<code>-help</code>	Outputs a brief description that includes type and default information for each <code>report_analysis_views</code> parameter. For a detailed description of the command and all of its parameters, use the man command: <code>man report_analysis_views</code> .
<code>-type {all setup hold active}</code>	Specifies the type of report to generate. • active: Generates a report of all active setup and hold views in the design. • all: Generates a report of all defined views in the design, including those that are currently inactive. • hold: Generates a report of all active hold views in the design. • setup: Generates a report of all active setup views in the design. <i>Default:</i> <code>all</code>

Examples

- The following command reports all the defined views in the design:

```
tempus > report_analysis_views
Multi-Mode Analysis View Report
```

```
+ ALL Views
|
+ Analysis View: func_slow_RCMAx
|
+ Delay Calc Corner: delay_corner_slow_RCMAx
| |
| | + early_timing_condition: default_mapping_tc_2
| | |
| | | + library_sets: libset_slow
| | | |
| | | + timing: ../../libs/liberty/FreePDK45_lib_v1.0_worst.lib
| | | | ../../libs/MRO/LIB/pllclk.lib
| | | | ../../libs/MRO/LIB/ram_256x16A.lib
| | | | ../../libs/MRO/LIB/rom_512x16A.lib
| | |
| | + late_timing_condition: default_mapping_tc_1
| | |
| | | + library_sets: libset_fast
| | | |
| | | + timing: ../../libs/liberty/FreePDK45_lib_v1.0_typical.lib
| | | | ../../libs/MRO/LIB/pllclk.lib
| | | | ../../libs/MRO/LIB/ram_256x16A.lib
| | | | ../../libs/MRO/LIB/rom_512x16A.lib
| | |
| | + early_rc_corner: corner_worst_RCMIN
| | |
| | | + preRoute_res: 0.8
| | | |
| | | + postRoute_res: 0.8 0.8 0.8
| | | |
| | | + preRoute_cap: 0.8
| | | |
| | | + postRoute_cap: 0.8 0.8 0.8
| | | |
| | | + postRoute_xcap: 0.8 0.8 0.8
| | | |
| | | + preRoute_clkcap: 0
| | | |
| | | + postRoute_clkcap: 0 0 0
| | | |
| | | + preRoute_clkres: 0
| | | |
| | | + postRoute_clkres: 0 0 0
```

- The following command generates a hierarchical report for all the active setup views in the design:

```
tempus > report_analysis_views -type setup
Multi-Mode Analysis View Report
```

```
+ SETUP Views
|
+ Analysis View: func_slow_RCMAX
|
+ Delay Calc Corner: delay_corner_slow_RCMAX
| |
| | + early_timing_condition: default_mapping_tc_2
| | |
| | | + library_sets: libset_slow
| | |
| | | + timing: ../../libs/liberty/FreePDK45_lib_v1.0_worst.lib
| | |             ../../libs/MRO/LIB/pllclk.lib
| | |             ../../libs/MRO/LIB/ram_256x16A.lib
| | |             ../../libs/MRO/LIB/rom_512x16A.lib
| |
| | + late_timing_condition: default_mapping_tc_1
| | |
| | | + library_sets: libset_fast
|
|
| | + early_rc_corner: corner_worst_RCMIN
| | |
| | | + preRoute_res: 0.8
| | |
| | | + postRoute_res: 0.8 0.8 0.8
|
+ late_rc_corner: corner_worst_RCMAX
| |
| | + preRoute_res: 1.2
|
+ si_enabled: true
|
+ Constraint Mode: functional_mode
| |
| | + SDC Constraint Files: ../../design/rcvr_core.pr.tcl ../../design/rcvr_core.pr.sdc
```

- The following command generates a hierarchical report of the active setup analysis views in the design:

```
tempus> report_analysis_views -type setup
```

The software reports the following results:

Multi-Mode Analysis View Report

```
+ SETUP Views
|
+ Analysis View: default_emulate_view
|
+ Delay Calc Corner: default_emulate_delay_corner
| |
| |
| |
```

```
| + rc_corner: default_emulate_rc_corner
| | |
| | + preRoute_res: 1
| | |
| | + postRoute_res: 1
| | |
| | + preRoute_cap: 1
| | |
| | + postRoute_cap: 1
| | |
| | + postRoute_xcap: 1
| | |
| | + preRoute_clkcap: 0
| | |
| | + postRoute_clkcap: 0 0 0
| | |
| | + preRoute_clkres: 0
| | |
| | + postRoute_clkres: 0 0 0
| |
| + late_library_set: default_emulate_libset_max
| | |
| | + timing: /icd/ssv_pe_t2b/timing/pawang/unit_test_cases/unit_2/data/cell_w.lib
| | |
| | + si: /icd/ssv_pe_t2b/timing/pawang/unit_test_cases/unit_2/data/celtic-NDC.cdB
| |
| + early_library_set: default_emulate_libset_min
| | |
| | + timing: /icd/ssv_pe_t2b/timing/pawang/unit_test_cases/unit_2/data/cell_w.lib
| | |
| | + si: /icd/ssv_pe_t2b/timing/pawang/unit_test_cases/unit_2/data/celtic-NDC.cdB
| |
| + si_enabled: true
|
+ Constraint Mode: default_emulate_constraint_mode
| |
| + SDC Constraint Files: /dev/null
```

Related Commands

- [create_analysis_view](#)
- [set_analysis_view](#)

report_annotated_assertions

```
report_annotated_assertions
-type {load | transition}
[-list_annotated]
[-list_non_annotated]
[-view viewName]
[-max_lines number]
[-min | -max | -min_max]
obj_list
[{> | >>} file_name]
```

Reports information for annotated assertions, set using the `set_load` or `set_annotated_transition` command, according to the specified assertion type. You can select load or transition reporting.

Note: The `report_annotated_assertions` command is now obsolete. This command will continue to be supported in the current release, but will be removed in the next major release of the software.

Parameters

<code>{> >>} file_name</code>	Writes the report to the specified file name. When specified with <code>></code>, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with <code>>></code>, writes the report to the specified filename. If the file already exists, the software concatenates the report to the end of the file.
<code>-list_annotated</code>	Lists the detailed information for the specified assertion types.
<code>-list_non_annotated</code>	Lists information for non-annotated assertions.
<code>-min -max -min_max</code>	Specifies to report minimum, maximum, or both values of the specified assertion type. When input assertions are applied using both <code>-min</code> and <code>-max</code> parameters, you can specify the <code>-min_max</code> parameter to report both the assertion values. In this case, you do not need to specify this command twice, that is, separately with <code>-min</code> and <code>-max</code> parameters. <i>Default:</i> max

<code>-max_lines number</code>	Controls the number of output lines reported by <code>-list_annotated</code> or <code>-list_non_annotated</code> parameter. By default, the specified net, pin, or port names are reported.
<code>obj_list</code>	Specifies the list of nets, pins, or ports to be reported.
<code>-type {load transition}</code>	Specifies the assertion type, load or transition, to be reported.
<code>-view viewName</code>	Reports the annotations for the specified analysis view. In multi-mode multi-corner timing analysis mode, annotations are reported for the specified mode or corner.

Examples

- The following command reports all the transition annotated assertions for the specified view:

```
litmus > report_annotated_assertions -type transition -list_annotated -view func_slow_RCMAX
```

```
Analysis View: func_slow_RCMAX
```

```
-----  
-----  
Assert type          Total      Annotated    Not Annotated    Percentage Annotated  
-----  
Transition (Max)     29097      0            29097            0.000  
-----
```

- The following command reports details of both the minimum and maximum values of the specified assertion type:

```
litmus > report_annotated_assertions -type load -list_non_annotated -min_max -max_lines 5  
-view scan_slow_RCMAX
```

```
Analysis View: scan_slow_RCMAX
```

```
-----  
Load (Min:Max) Non  
Annotated  
-----  
NetName  
-----  
tdigit[7]  
tdigit[6]  
tdigit[5]  
tdigit[4]  
tdigit[3]  
-----  
-----  
Assert type          Total      Annotated    Not Annotated    Percentage Annotated  
-----  
Load (Min:Max)       8415      0            8415            0.000  
-----
```

Related Commands

- `set_load`
- `set_annotated_transition`

report_annotated_check

```
report_annotated_check
[-help]
[-check_type check_type_list]
[-max_line number]
[instance_list]
[-list_annotated]
[-list_non_annotated]
[-view viewName]
[> | >> filename.gz] | -tcl_list]
```

Reports coverage of annotated timing checks.

Parameters

-help	Prints out the command usage.
-check_type <i>check_type_list</i>	Allows you to specify the check type. The valid check types are <code>setup</code> , <code>hold</code> , <code>pulse_width</code> , <code>clock_period</code> , <code>clock_gating_setup</code> , <code>clock_gating_hold</code> , <code>clock_gating_pulse_width</code> , <code>data_setup</code> , <code>data_hold</code> , <code>recovery</code> , <code>removal</code> , <code>clock_separation</code> , <code>no_change_setup</code> , <code>no_change_hold</code> , and <code>skew</code> .

<code>> filename</code>	When specified with <code>></code>, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with <code>>></code>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The <code>filename</code> parameter must be the last argument in the list. The <code>filename</code> and <code>-tcl_list</code> parameters are mutually exclusive; you cannot specify them together. Note: To write a compressed report, add the <code>.gz</code> extension to the file name. <i>Default:</i> Report is displayed on standard output without being saved.
<code>instance_list</code>	Specifies instance names for which annotations of check arcs are to be reported.
<code>-list_annotated</code>	Lists the detailed annotation information for the following timing checks: • setup • hold • recovery • removal • pulse-width • nochange-setup • clock-period • skew Note: If the software is in multi-mode multi-corner timing analysis mode, you need to specify the <code>-view</code> parameter.

<code>-list_non_annotated</code>	Lists the timing checks that do not have annotated delay information. The software checks for the following timing checks: • setup • hold • recovery • removal • pulse-width • nochange-setup • clock-period • skew Note: If the software is in multi-mode multi-corner timing analysis mode, you need to specify the <code>-view</code> parameter.
<code>-max_line <i>number</i></code>	Controls the number of output lines reported by the <code>-list_annotated</code> or <code>-list_non_annotated</code> parameter.
<code>-tcl_list</code>	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. The <code>-tcl_list</code> and <i>filename</i> parameters are mutually exclusive; you cannot specify them together.
<code>-view <i>viewName</i></code>	Reports coverage of annotated timing checks for the specified analysis view. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode. <i>Default:</i> Reports annotated timing checks for the default analysis view.

Examples

- The following command reports the setup timing checks that do not have annotated delay information:

```
litmus > report_annotated_check -check_type setup -list_non_annotated -max_line 6 -view scan_slow_RCMAX  
Analysis View: scan_slow_RCMAX
```

NonAnnotated-Setup

Checks

SourceSignal	RefSignal	Type	Cond	View
AB_INST/sp_0/D ^	AB_INST/sp_0/CK ^	setup_rising_clk_rise	-	scan_slow_RCMAX
AB_INST/sp_0/D v	AB_INST/sp_0/CK ^	setup_falling_clk_rise	-	scan_slow_RCMAX
AB_INST/sp_0/SE ^	AB_INST/sp_0/CK ^	setup_rising_clk_rise	-	scan_slow_RCMAX
AB_INST/sp_0/SE v	AB_INST/sp_0/CK ^	setup_falling_clk_rise	-	scan_slow_RCMAX
AB_INST/sp_0/SI ^	AB_INST/sp_0/CK ^	setup_rising_clk_rise	-	scan_slow_RCMAX
AB_INST/sp_0/SI v	AB_INST/sp_0/CK ^	setup_falling_clk_rise	-	scan_slow_RCMAX

Check arc type	Total	Annotated	Not Annotated	Percentage Arcs Annotated
Setup arcs	1741	0	1741	0.000000
All arcs	1741	0	1741	0.000000

- The following command reports the detailed annotation information for the setup timing checks of the specified view:

```
litmus > report_annotated_check -check_type setup -list_annotated -max_line 6 -view scan_slow_RCMAX -tcl_list
```

Analysis View: scan_slow_RCMAX

```
{ banner { } } { Check arc Summary { { columns { { arc_type } { total } { total_ann } { total_not_ann } { percentage_ann } } } { row_1 { { Setup arcs } { 1741 } { 0 } { 1741 } { 0.000000 } } } { row_2 { { _ } } } { row_3 { { All arcs } { 1741 } { 0 } { 1741 } { 0.000000 } } } }
```

- The report_annotated_check command shows the following report output in simultaneous setup and hold analysis mode:

```
litmus> report_annotated_check
```

Check arc type	Total	Annotated	Not Annotated	Percentage Arcs Annotated
Setup arcs	1642	2	1640	0.121803
Hold arcs	1642	1	1641	0.060901
Recovery arcs	368	0	368	0.000000
Removal arcs	367	1	366	0.272480
Min pulse width arcs	1403	0	1403	0.000000
Period arcs	6	0	6	0.000000

Litmus Text Command Reference Timing Analysis Commands--report_annotated_check

```

| Clock gating hold arcs | 2      | 0      | 2      | 0.000000 |
+-----+-----+-----+-----+
| All arcs                | 5430   | 4      | 5426   | 0.073665 |
+-----+-----+-----+-----+

```

- The following command lists down all the annotated checks in the `f_slow` view:

```
litmus> report_annotated_check -list_annotated -view f_slow
```

Analysis View: `f_slow`

```

+-----+
-----+
| Setup Checks |
+-----+-----+-----+-----+
-----|
| SourceSignal      | RefSignal          | Delays              | Type                | Cond
|View              |
+-----+-----+-----+-----+
-----
| DTMF_IN/TDSP_CR/PROG| DTMF_IN/TDSP_CR/PROG| 0.500:0.500:0.500|
setup_rising_clk_rise|RN_EQ_1_AN|f_slow |
| BUS_MC_IN/dt_reg_2  | BUS_MC_IN/dt_reg_2  |
|
| D ^                | CK ^                |
|
| DTMF_IN/TDSP_CR/PROG| DTMF_IN/TDSP_CR/PROG|
0.500:0.500:0.500|setup_falling_clk_rise|RN_EQ_1_AN|f_slow |
| BUS_MC_IN/dt_reg_2  | BUS_MC_IN/dt_reg_2  |
|
| D v                | CK ^                |
|
+-----+
-----

```

```

+-----+
| Check arc type      | Total | Annotated | Not      | Percentage|
|                     |       |           | Annotated| Arcs      |
|                     |       |           |           | Annotated |
+-----+-----+-----+-----+
| Setup arcs          | 1642  | 2         | 1640     | 0.121803  |
| Recovery arcs       | 368   | 0         | 368      | 0.000000  |
| Min pulse width arcs| 1403  | 0         | 1403     | 0.000000  |
| Period arcs         | 6      | 0         | 6         | 0.000000  |
+-----+-----+-----+-----+
| All arcs            | 3419  | 2         | 3417     | 0.058497  |
+-----+

```

- The following command lists down all the annotated checks in the `f_fast` view:

Litmus Text Command Reference

Timing Analysis Commands--report_annotated_check

```
litmus > report_annotated_check -list_annotated -view f_fast
```

Analysis View: f_fast

Hold Checks				
SourceSignal	RefSignal	Delays	Type	Cond
DTMF_INST/TEST_INST	DTMF_INST/TEST_INST	0.400:0.400:0.400	hold_rising_clk_rise	
RAM_INST/A[5] ^	RAM_INST/CLK ^			
DTMF_INST/TEST_INST	DTMF_INST/TEST_INST	0.400:0.400:0.400	hold_falling_clk_rise	
RAM_INST/A[5] v	RAM_INST/CLK ^			

Removal Checks				
SourceSignal	RefSignal	Delays	Type	Cond
DTMF_IN/SPI_IN/dflag	DTMF_IN/SPI_IN/dflag	0.300:0.300:0.300	removal_rising_clk_rise	
N ^	K ^			

Check arc type	Total	Annotated	Not Annotated	Percentage
Hold arcs	1642	1	1641	0.060901
Removal arcs	367	1	366	0.272480
Clock gating hold arcs	2	0	2	0.000000

All arcs	2011	2	2009	0.099453	
+-----+					

Related Commands

- `set_annotated_check`
- `reset_annotated_check`

report_annotated_delay

```
report_annotated_delay
[-help]
[-ignore_tied_net_arcs]
[-list_annotated]
[-list_non_annotated]
[-view viewName]
[-max_line number]
[> | >>] filename[.gz] | -tcl_list
[pin_port_list]
```

Reports coverage of annotations on a design. These annotations include SDF delay arc annotations. Besides SDF annotations, the command also reports annotations that are set using the `set_annotated_delay` command.

Parameters

-help	Outputs a brief description that includes type and default information for each <code>report_annotated_delay</code> parameter. For a detailed description of the command and all of its parameters, use the man command: <code>man report_annotated_delay</code> .
> >> filename	When specified with <code>></code>, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with <code>>></code>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The <code>filename</code> parameter must be the last argument in the list. The <code>filename</code> and <code>-tcl_list</code> parameters are mutually exclusive; you cannot specify them together. Note: To write a compressed report, add the <code>.gz</code> extension to the file name. <i>Default:</i> Report is displayed on standard output without being saved.

<code>-ignore_tied_net_arcs</code>	Ignores timing arcs of power and ground nets.
<code>-list_annotated</code>	Lists detailed annotation information for the following annotated arcs: • annotated internal net arcs • annotated input port arcs • annotated output port arcs • annotated cell arcs
<code>-list_non_annotated</code>	Lists the arc information for the non-annotated arcs corresponding to the following types of arcs: • non-annotated internal net arcs • non-annotated input port arcs • non-annotated output port arcs • non-annotated cell arcs
<code>-max_line <i>number</i></code>	Controls the number of output lines reported by <code>-list_annotated</code> OR <code>-list_non_annotated</code> parameter.
<code>pin_port_list</code>	Specifies the pin/port list for which annotated delay is to be reported.
<code>-tcl_list</code>	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. The <code>-tcl_list</code> and <code>filename</code> parameters are mutually exclusive; you cannot specify them together.
<code>-view <i>viewName</i></code>	Generates the report for the specified analysis view only. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode.

Examples

- The following command reports the status of annotation for cell arcs, internal net arcs, net arcs from primary inputs, and net arcs to primary outputs, and all the timing arcs separately.

Litmus Text Command Reference
Timing Analysis Commands--report_annotated_delay

```
litmus > report_annotated_delay -view scan_slow_RCMIN -list_annotated -ignore_tied_net_arcs
```

Analysis View: scan_slow_RCMIN

Delay Arc Type	Total	Annotated	Not Annotated	Percentage Annotated
Cell arcs	38365	0	38365	0.000000
Input net arcs	141	0	141	0.000000
Internal net arcs	20019	0	20019	0.000000
Output net arcs	29	0	29	0.000000
All timing arcs	58554	0	58554	0.000000

- The following command reports the non-annotated input port arcs, internal net arcs, cell delay arcs, and output port arcs for the specified view:

```
litmus > report_annotated_delay -view scan_slow_RCMAX -list_non_annotated -max_line 3
```

Analysis View: scan_slow_RCMAX

Delay Arc Type	Total	Annotated	Not Annotated	Percentage Annotated
Cell arcs	38365	0	38365	0.000000
Input net arcs	141	0	141	0.000000
Internal net arcs	20027	0	20027	0.000000
Output net arcs	29	0	29	0.000000
All timing arcs	58562	0	58562	0.000000

non-annotated input port arcs

From	To	SdfCond	View
reset ^	AB_INST/p25A/A ^		scan_slow_RCMAX

non-annotated internal net

arcs

From	To	SdfCond	View
FE_C6_ar4/Z ^	DT_MX_INST/p3A/B ^		scan_slow_RCMAX

non-annotated cell delay arcs

-----	-----	-----	-----
From	To	SdfCond	View
-----	-----	-----	-----
FE_C6ar4/A ^	FE_C6ar4/Z ^		scan_slow_RCMAX
-----	-----	-----	-----
non-annotated output port arcs			
-----	-----	-----	-----
From	To	SdfCond	View
-----	-----	-----	-----
FE_C63_port_out/Z ^	port_data_out[13] ^		scan_slow_RCMAX
-----	-----	-----	-----

Related Commands

- [set_annotated_delay](#)
- [report_nets](#)

report_case_analysis

```
report_case_analysis
[-help]
[pins_and_ports_list]
[-all]
[-dir]
[-include_ref_name]
[-nosplit]
[-tcl_list]
[-verbose]
[-view view_name]
[> <filename[.gz]>]
[>> <filename[.gz]>]
[-propagated [-include_function]]
```

Generates a User Case Analysis table, which reports ports and pin that have a `set_case_analysis` constraint. These ports and pins are constant only for timing analysis. You can optionally generate a Logic Constant Value table, which reports ports and pins whose constant status is part of the design. Information from this table includes ports or pins with `set_logic` constraints (`set_logic_zero` and `set_logic_one`), netlist explicit constants, and output pins with constant output functions.

Parameters

-help	Prints out the command usage.
-------	-------------------------------

<code>{> >>} filename</code>	When specified with <code>></code>, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with <code>>></code>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The <code>filename</code> parameter must be the last argument in the list. The <code>filename</code> and <code>-tcl_list</code> parameters are mutually exclusive; you cannot specify them together. Note: To write a compressed report, add the <code>.gz</code> extension to the file name. <i>Default:</i> Report is displayed on standard output without being saved.
<code>-all</code>	Specifies that the Logic Constant Value table report pins or ports with <code>set_logic</code> constants, constants in the netlist, and output pins with a constant function. These output pins are from library cells or netlist modules.
<code>-include_function</code>	Displays the pin function of a pin. This must be specified with the <code>-propagated</code> parameter.
<code>-include_ref_name</code>	Displays the reference name of an instance in the report.
<code>-no_split</code>	Specifies that reports with long names will not be split into multiple lines. Note: The <code>-no_split</code> parameter works if <code>set_table_style -no_frame_fix_width</code> command is specified.
<code>-dir</code>	Specifies that the report include the direction of the pin or port, which is reported as input, output, or inout (internal pins may not have constant constraints).
<code>pins_and_ports_list</code>	Generates a report only for the pins and ports in the specified list. <i>Default:</i> Generates a report for all pins and ports in the design.

-propagated	Generates a report that includes all the constant pins, including pins with propagated constants. When specified, the report column header shows the constant values - the column header for constant values is <code>User case analysis value</code>. <i>Default:</i> Report shows pins specified with the <code>set_case_analysis</code> command.
-tcl_list	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. The <code>-tcl_list</code> and <code>filename</code> parameters are mutually exclusive; you cannot specify them together.
-verbose	Generates a report in which each pin is listed on a separate line with a cause. If the pin is a propagated constant, a table is included that reports a chain of constants from an original cause to the reported pin. <i>Default:</i> All pins are reported in a table.
-view view_name	Generates a User Case Analysis table for the specified analysis view only. To specify this parameter, the software must be in multi-mode multi-corner (MMMC) analysis mode. <i>Default:</i> When in MMMC analysis mode, generates a table for all active analysis views

Examples

- The following command reports user case analysis values for all the analysis views:

```
litmus > report_case_analysis
Pin name      User      View Name
              case
              analysis
-----
tst_mode      0          scan_fast_RCMIN
tst_mode      0          scan_slow_RCMIN
tst_mode      0          func_slow_RCMAX
scan_en       0          scan_fast_RCMIN
scan_en       0          scan_slow_RCMIN
```

- The following command reports all the constant pins, including pins with propagated constants - and long

names are not split into multiple lines:

```
litmus > report_case_analysis -propagated -no_split
```

```
-----
```

Pin name	Constant value	View Name

AB_IS/se_0/D	0	func_slow_RCMAX
AB_IS/se_0/RN	1	func_slow_RCMAX
AB_IS/se_0/SE	0	func_slow_RCMAX
AB_IS/se_0/SI	0	func_slow_RCMAX
AB_IS/se_0/SN	1	func_slow_RCMAX
AB_IS/se_1/D	0	func_slow_RCMAX
AB_IS/se_1/RN	1	func_slow_RCMAX
AB_IS/se_1/SE	0	func_slow_RCMAX

- The following command reports the Logic Constant Value table report pins or ports with set_logic constants, constants in the netlist, and output pins with a constant function. These output pins are from library cells or netlist modules.

```
litmus > report_case_analysis -all -no_split
```

```
-----
```

Pin name	Logic constant value

AB_IS/t0/Z	0
AB_IS/t1/Z	1
AB_IS/dma_reg/SE	0
AB_IS/pre_reg[1]/SE	0
AB_IS/ECO1reg[2]/SE	0

```
-----
```

- The following command reports the Logic Constant Value table of pins:

```
litmus > report_case_analysis -all -no_split -verbose
```

```
-----
```

Pin name	Logic constant value

AB_IS/t0/Z	0
AB_IS/t1/Z	1
AB_IS/dma_reg/SE	0

Pin tst_mode 0 view scan_fast_RCMIN is caused by set_case_analysis 0 on pin tst_mode

Pin tst_mode 0 view scan_slow_RCMIN is caused by set_case_analysis 0 on pin tst_mode

Pin tst_mode 0 view scan_fast_RCMAX is caused by set_case_analysis 0 on pin tst_mode

- The following command reports user case analysis values for the specified view:

```
litmus > report_case_analysis -view scan_fast_RCMIN
```

```
-----
Pin name      User      View Name
              case
              analysis
              value
-----
tst_mode      0          scan_fast_RCMIN
scan_en       0          scan_fast_RCMIN
-----
```

- The following command reports pins that include the direction of the pin - as input:

```
litmus> report_case_analysis -dir
```

```
-----
Pin name      Dir      User case
              Analysis value
-----
gscanmode     input     0
gscanenable   input     0
-----
```

- The following command reports all the constant pins, including pins with propagated constants:

```
litmus> report_case_analysis -propagated
```

```
-----
Pin name      Constant
              value
-----
clockandlatch/U8/TE      0
clockandlatch_dup1/U8/TE 1
clockandlatch_dup2/U8/TE 1
clockandlatch_dup3/U8/TE 0
-----
```

- The following command reports details of each pin separately. The pin `crclocklatch/U8/TE` is reported because of a `set_case_analysis 0` assertion on pin `gscan`:

```
litmus> report_case_analysis -propagated -verbose crclocklatch/U8/TE
```

```
-----
Pin name      Constant
              Value
-----
gscan         0
FE_OC36_gscan/I      0
FE_OC36_gscan/ZN     1
-----
```

FE_OC37_gscan/I	1
FE_OC37_gscan/ZN	0
FE_OC35_gscan/I	0
FE_OC35_gscan/Z	0
crclocklatch/U8/TE	0

Related Commands

- `set_case_analysis`

report_cell_instance_timing

```
report_cell_instance_timing
[-help]

```

Reports the following information about the timing context for all the pin and delay arcs of the specified instances:

- Arrival time
- Required time
- Slack
- Delays

Note: Internal pins (pins inside the instance boundary) are also reported.

This command uses the results of earlier timing analysis runs; therefore, you must run the `report_timing` command before using `report_cell_instance_timing`.

To analyze the setup (maximum) paths, you must use the `timing_analysis_check_type setup` attribute and run the `report_timing` command before using the `-late` parameter. Similarly, to analyze the hold (minimum) paths, you must use the `timing_analysis_check_type hold` attribute and run the `report_timing` command before using the `-early` parameter.

The `report_cell_instance_timing` timing report contains the following tables:

- The first table in the report contains information about the pins of the instance.
The propagated slew is the slew that is propagated to the pin, based on the slew propagation mode.
- The second table contains information about the arcs through the instance.
The slew out is the slew calculated at the arc output from the arc input slew, using the timing model.
- The third table, generated for sequential devices, contains information about the timing checks, if present, on the cell.
Only checks that are from the library are shown in this table. Automatic checks, such as clock gating setup and hold, are not reported.

The *Phase* column of each table can contain the following symbols:

- `D` indicates it is a data signal in the clock domain.
- `C` indicates it is a clock signal in the clock domain.

- **P** indicates it is a positive phase.
- **N** indicates it is a negative phase.
- **L** indicates it is a latency phase.
- Asterisk (*) indicates there is a timing exception associated with the signal.

Note: Currently, it is not possible to expand the asterisk to determine which exception is associated with the reported timing phase.

When the software is in multi-mode multi-corner timing analysis mode, the timing context information is reported for each active analysis view, with the view name listed in the *Phase* column.

Parameters

-help	Prints out the command usage.
{> >>} <i>filename</i>	When specified with >, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with >>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The <i>filename</i> parameter must be the last argument in the list. The <i>filename</i> and -tcl_list parameters are mutually exclusive; you cannot specify them together. Note: To write a compressed report, add the .gz extension to the file name. <i>Default:</i> Report is displayed on standard output without being saved.
-clock_timing	Specifies to report slack for clock style checks.
-early -late	Reports the information for early (maximum) path (data hold/clock setup) or late (minimum) path (data setup/clock hold). <i>Default:</i> If you do not specify any of the parameters, the software uses the -late parameter.

<i>hinst_list</i>	Specifies a flat (non-hierarchical) cell instance or list of cell instances. Use curly braces ({}) to enclose the list and separate the instance names with white space.
-tcl_list	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. The -tcl_list and <i>filename</i> parameters are mutually exclusive; you cannot specify them together.
-view <i>view_name</i>	Specifies the analysis view in which to query the object attribute.

Examples

- The following command reports information about the timing context of the specified cell instance:

```
litmus > report_cell_instance_timing FE_63_data_out_13 -view scan_fast_RCMIN
```

```
-----
Instance FE_63_data_out_13 of BUF_X32

-----
Pin      Dir      Propagated   Arrival   Required   Slack      Phase
      Slew
-----
A ^      IN      0.067        5.567     38.295     32.728     m_clk(D) (P) (scan_fast_RCMIN)
A v      IN      0.055        5.528     38.214     32.687     m_clk(D) (P) (scan_fast_RCMIN)
-----
Instance FE_63_data_out_13 of BUF_X32

-----
Arc              Slew
-----
From    To      In      Out      Load   Delay      Phase
-----
A ^      Z ^      0.067   0.248   0.012   0.303     m_clk(D) (P) (scan_fast_RCMIN)
A v      Z v      0.055   0.140   0.012   0.384     m_clk(D) (P) (scan_fast_RCMIN)
-----
```

- The following example reports information about the timing context of instance seg1/u9:

```
litmus > report_cell_instance_timing seg1/u9
```

```
-----
Instance seg3/u14 of DFF
```

Litmus Text Command Reference
Timing Analysis Commands--report_cell_instance_timing

Pin	Dir	Propagated Slew	Arrival	Required	Slack	Phase
CK ^	IN	0.062	0.621			CLK_W_3 (C) (P) *
CK v	IN	0.065				CLK_W_3 (D) (P) *
D v	IN	0.003	1.815	2.465	0.650	CLK_W_3 (D) (P)
CK ^	IN	0.062				CLK_W_3 (D) (N) *

Instance seg3/u14 of DFF

Arc Slew

From	To	In	Out	Load	Delay	Phase
CK ^	QN ^	0.062	0.058	0.000	0.196	CLK_W_3 (C) (P) *
CK ^	QN v	0.062	0.041	0.000	0.236	CLK_W_3 (D) (P) *
CK ^	QN ^	0.062	0.000			CLK_W_3 (D) (N) *

Instance seg3/u14 of DFF

Pins Phase

Check	Sig	Ref	Slack	Delay	Adjust	Sig	Ref
HOLD	D ^	CK ^	1.570	-0.129	0.000	CLK_W_3 (D) (P)	CLK_W_3 (C) (P) *
SETUP	D ^	CK ^	0.372	0.186	-2.000	CLK_W_3 (D) (P)	CLK_W_3 (C) (P) *

- If a constant is asserted or propagated to the instance pin, it is reported in the Phase column of the pin table, as shown in the following example:

litmus > report_cell_instance_timing p5042A

Instance p5042A of MUX2HVTD1

Pin	Dir	Propagated Slew	Arrival	Required	Slack	Phase
I0 ^	IN	0.032	0.127	8.583	8.455	EJ_TCK (D) (P)
I0 v	IN	0.021	0.127	8.568	8.441	EJ_TCK (D) (P)
S	IN					Constant 0
Z ^	OUT	0.020	0.168	8.624	8.455	EJ_TCK (D) (P)

Instance p5042A of MUX2HVTD1

Arc Slew

From	To	In	Out	Load	Delay	Phase
------	----	----	-----	------	-------	-------

```
-----  
I0 ^    Z ^    0.032    0.020    0.002    0.041 EJ_TCK(D) (P)  
I1 ^    Z ^    0.060    0.002                      EJ_TCK(D) (P)  
-----
```

- When the software is in multi-mode multi-corner analysis mode, the timing context information is reported for each active analysis view. The view name is reported in the Phase column, as shown in the following example:

```
litmus > report_cell_instance_timing FE_OFC304_EJ_TDO
```

```
-----  
Instance FE_OFC304_EJ_TDO of BUFFHVTD8  
-----
```

Pin	Dir	Propagated	Arrival	Required	Slack	Phase
Slew						

I ^	IN	0.052	8.812	16.260	7.448	EJ_TCK(D) (N) (setup_view1) *
I v	IN	0.033	8.799	16.319	7.520	EJ_TCK(D) (N) (setup_view1) *

```
-----
```

Related Commands

- [report_nets](#)
- [report_timing](#)

report_clock_gating_check

```
report_clock_gating_check  
[-help]  
[obj_list]  
[-view view_name]  
[{> | >>} file_name]
```

Reports information of all or specific clock gating checks of the current design. If you specify a list of pins, cells, or clocks, the clock gating check information is reported for the specified objects only. If you use the `report_clock_gating_check` command without an object list, all clock gating check information is reported.

The `Type` column of the generated report (see examples below) can contain one of the following abbreviations:

- `I` indicates that the clock gating check is inferred.
- `L` indicates that the clock gating check is from the library (integrated clock gating check).

Parameters

<code>-help</code>	Prints out the command usage.
<code>obj_list</code>	Specifies a list of pins, instances, or clocks. A collection of these can also be specified. • Pin: Any pin of a clock gating cell can be specified and the check information will be reported for that pin. • Instance: Any instance name of a clock gating cell can be specified and the check information will be reported for that instance. • Clock: Only a clock at a given clock (reference) pin of a cell can be specified for reporting the check information. Check information about the clocks at enable (signal) pin cannot be reported. If the <code>obj_list</code> parameter is not specified, all the clock gating check points (inferred and library) of the current design are reported.
<code>-view view_name</code>	Specifies a view for which clock gating check is to be reported. The default is worst of all the views.

{> | >>} *filename*

When specified with >, writes the report to the specified file name. If the file already exists, the software overwrites it.

When specified with >>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file.

Examples

- The following command reports clock gating check details for the specified view:

```
litmus > report_clock_gating_check -view AV_bc_off
```

Clock Gating Check Report

```
-----
Instance      Enable  Clock  Type  Level  Setup  Setup  Hold  Hold  View
              Rise   Fall   Rise   Fall
-----
sub_inst2/LCLK  EN      A      I      H      0.000  0.000  0.000  0.000  AV_bc_off
-----
```

- The following command generates a clock gating check report for the specified collection:

```
litmus > report_clock_gating_check CLK
```

Clock Gating Check Report

```
-----
Instance      Enable  Clock  Type  Level  Setup  Setup  Hold  Hold  View
              Rise   Fall   Rise   Fall
-----
sub_inst2/LCLK  EN      A      I      H      0.000  0.000  0.000  0.000  AV_wc_on
-----
```

- The following command generates a clock gating check report for the current design:

```
litmus > report_clock_gating_check
```

The clock gating check report returned by this command is as follows:

Clock Gating Check Report

```
-----
Instance  Enable  Clock  Type  Level  Setup  Setup  Hold  Hold  View
              Rise   Fall   Rise   Fall
-----
C_and_1   A       B      I      H      0.000  0.000  -     -     view1
-----
```


report_clock_propagation

```
report_clock_propagation
[-help]
-clock clock_list
[-max_paths number of paths]
-to pin_port_list
[-verbose]
[-view viewName]
[> filename[.gz]]
[>> filename[.gz]]
```

This command provides the ability to determine clocks that do not propagate to a particular node in the network, and report the causes for such occurrences.

Parameters

-help	Prints out the command usage
-clock <i>clock_list</i>	List or collection of clocks that are to be reported.
-max_paths <i>number of paths</i>	Reports number of maximum clock paths per clock.
-to <i>pin_port_list</i>	Specifies the pin/port list.
-verbose	Generates a verbose report for clock propagation stop reason.
-view <i>viewName</i>	Generates a report for the specified analysis view.
> <i>filename[.gz]</i> >> <i>filename[.gz]</i>	When specified with >, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with >>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. Note: To write a compressed report, add the .gz extension to the file name.

Examples

- The following command reports the specified clocks and pin details:

```
litmus > report_clock_propagation -clock clk1 -to f0/QN
Clock: clk1
Point: f0/QN
View: view1
Reason 1: The clock 'clk1' propagation stops across the trigger arc:
(f0/CK->f0/QN)
The partial clock path:
```

```
-----
Source      Sink      ArcType      ArcSense
-----
clk1        f0/CK      (net)         (net)
f0/CK       f0/QN      rising_edge    non_unate
-----
```

- The following command shows that clock propagation stops due to an inactive timing arc from b12/A to b12/Y:

```
set_disable_timing -from A -to Y b12
report_clock_propagation -to u99/CK -clock CLK_W_4
Clock: CLK_W_4
Point: u99/CK
Reason 1: The clock 'CLK_W_4' propagation stops due to the inactive timing arc:
(b12/A->b12/Y)
The partial clock path:
```

```
-----
Source      Sink      ArcType      ArcSense
-----
CLK4        b11/A      (net)         (net)
b11/A       b11/Y      combinational positive_unate
b11/Y       b12/A      (net)         (net)
b12/A       b12/Y      combinational positive_unate
-----
```

- The following command shows that clock propagation stops due to no clock source found:

```
report_clock_propagation -to launchff/Q -clock [get_clocks gen_clk]
report_clock_propagation -to seg3/u15/Q -clock CLK_W_5_gen

Clock: CLK_W_5_gen
Point: seg3/u15/Q
Reason 1: The generated clock 'CLK_W_5_gen could not be created on its root pin 'seg3/u15/Q'
```

because no clock source found for it

- The following command shows that clock propagation stops at the register clock pin as all the arcs from the clock pin of the register have been disabled:

```
set_disable_timing -from A -to Y seg3/u17_a
report_clock_propagation -to seg3/u14/CK -clock CLK_W_3
```

Clock: CLK_W_3

Point: seg3/u14/CK

Reason 1: The clock 'CLK_W_3' propagation stops due to the inactive timing arc:
(seg3/u17_a/A->seg3/u17_a/Y)

The partial clock path:

Source	Sink	ArcType	ArcSense
CLK3	seg3/u16/A	(net)	(net)
seg3/u16/A	seg3/u16/Y	combinational	positive_unate
seg3/u16/Y	seg3/u17/A	(net)	(net)
seg3/u17/A	seg3/u17/Y	combinational	positive_unate
seg3/u17/Y	seg3/u17_a/A	(net)	(net)
seg3/u17_a/A	seg3/u17_a/Y	combinational	positive_unate

Related Commands

- [set_disable_timing](#)

report_clock_sense

report_clock_sense
[-help]

Reports all clock sense settings in the design.

Parameters

-help	Prints out the command usage.
-------	-------------------------------

Related Commands

- `set_clock_sense`
- `reset_clock_sense`
- `get_pins`
- `get_ports`

report_clock_timing

```
report_clock_timing
[-help]
[-cprpr_relative]
[-latency_greater_than]
-type [skew | interclock_skew | jitter | summary | latency | cprpr_stage_count
      [ [-launch | -capture]
        [-rise | -fall]
        | [-histogram [-histogram_range interval size]]
        [-logic_level [-source {clock_root | generated_clock}]]]
      ]
[-absolute_compare]
[-early | -late]
[-clock clock_list]
[-from_clock from_clock_list]
[-to_clock to_clock_list]
[-from from_list]
[-to to_list]
[-nworst worst_entries]
[-greater_than lower_limit]
[-view view_name]
[-verbose]
[-format column_list]
[-tcl_list]
[{> | >>} filename]
```

Generates a clock analysis report for the current design. The software generates a separate report for each specified clock or pair of clocks. You specify the clocks using the `-clock` parameter, and clock pairs using the `-from_clock` and `-to_clock` parameters. You can use the `-nworst` and `-greater_than` parameters with the `-from_clock` and `-to_clock` parameters to generate reports with skew to the specified sink pins. You can use the `-early` and `-late` parameters to specify the type of skew to be reported.

Note: To remove common path pessimism from the reported skew, use the `timing timing_analysis_cprpr` both setting before using the `report_clock_timing` command. The report contains a separate column for common path adjustment values.

You can use the `report_clock_timing` command to perform MMMC based SSTA analysis. In MMMC mode, the command reports the latency and clock skew for all individual views. You can use the `report_clock_timing -view` parameter to report clock timing for a user-specified view.

Parameters

<code>-help</code>	Prints out the command usage.
--------------------	-------------------------------

<code>{> >>} filename</code>	When specified with <code>></code>, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with <code>>></code>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The <code>filename</code> parameter must be the last argument in the list. The <code>filename</code> and <code>-tcl_list</code> parameters are mutually exclusive; you cannot specify them together. Note: To write a compressed report, add the <code>.gz</code> extension to the file name. <i>Default:</i> Report is displayed on standard output without being saved.
<code>-absolute_compare</code>	Reports skews with values compared in absolute form.
<code>-clock clock_list</code>	Specifies the list of clocks to be reported. The software generates a separate report for each clock that you specify in the <code>clock_list</code>. <i>Default:</i> Skew report is generated for all the clocks in a design.
<code>-cppr_relative</code>	Reports the latency in skew reports from the CPPR common pin. When specified, the reports are printed with latency measured from branch point to the register's CK pin. The CPPR value is no longer required in reports.
<code>-early</code>	Uses hold skew to generate the report. Hold skew is the difference between minimum latency at the start point flip-flop and the maximum latency at the end point flip-flop. <i>Default:</i> Uses the setup skew.

<code>-format column_list</code>	Specifies which columns to display for path reports, and the order in which they should appear. For example: <pre>-format {instance arc pin cell slew load}</pre> The <code>-format</code> parameter can be used only for latency or skew reports (<code>-type skew</code>, <code>-type interclock_skew</code>, or <code>-type latency</code>), and if the <code>-verbose</code> parameter is specified.
<code>-from from_list</code>	Specifies the list of sequential device clock pins or ports to consider as from-pins in the generated report. If any pin in the <code>from_list</code> is not a sequential device clock pin, it is replaced with all the sequential device clock pins in the transitive fanout of the named pin. If there are no sequential pins in the transitive fanout of the pin, the software drops the pin from the list and generates a warning message.
<code>-from_clock from_clock_list</code>	
	Specifies the list of from-clocks to be used in generating the current interclock skew report. You can use this option only in an interclock skew report. <i>Default:</i> Report is generated considering all clocks as from-clocks.
<code>-greater_than lower_limit</code>	Reports skews with values greater than the <code>lower_limit</code> value.

<code>-histogram</code>	Generates a textual histogram of the latency report. The <code>-histogram</code> parameter can be used only when <code>-type latency</code> is specified. When specified in conjunction with the <code>-clock</code> parameter, the software limits the latency reporting to the specified clocks only. When specified with the <code>-greater_than</code> parameter, the software limits the latency reporting to clocks with a latency value that is greater than the specified value. Note: The <code>-histogram</code> parameter cannot be specified in conjunction with the following parameters: <code>-launch</code>, <code>-capture</code>, <code>-from</code>, <code>-to</code>, <code>-from_clock</code>, <code>-to_clock</code>, <code>-verbose</code>, <code>-nworst</code>.
<code>-histogram_range interval_size</code>	Generates the histogram report at the specified intervals. The interval size specified must be a positive integer or decimal values. For example, if the maximum latency value is 437, and you specify <code>-histogram_range 50</code>, the histogram would be generated with intervals of: 0-50, 50-100, 100-150, ... 400-450. Note: The unit of <code>interval_size</code> will be the same as that used for generating the timing report. This parameter can be used only when the <code>-histogram</code> parameter is specified.
<code>-late</code>	Uses setup skew to generate the report. Setup skew is the difference between the maximum latency at the start point flip-flop and minimum latency at the end point flip-flop.
<code>-latency_greater_than lower_limit</code>	Reports are filtered based on the specified <code>-type</code> option. When specified with: • <code>-type skew</code> or <code>interclock_skew</code>: Reports are filtered for latency greater than the provided value or depth. • <code>-type cpr_stage_count</code>: Reports are filtered for depth greater than the provided value.

<code>-launch -capture</code>	When specified with <code>-type latency</code>, generates a report of either the launch latency values (<code>-launch</code>), or the capture latency values (<code>-capture</code>). Note: You can specify these parameters only with <code>-type latency</code>. <i>Default:</i> <code>-launch</code>
<code>-logic_level</code>	Reports the levels (the stage count of gates) between the clock root, or generated clock root, and the capturing register. The <code>-logic_level</code> parameter can be used only when <code>-type latency</code> is specified. The <code>-logic_level</code> parameter cannot be specified in conjunction with the following parameters: <code>-launch</code>, <code>-capture</code>, <code>-from</code>, <code>-to</code>, <code>-from_clock</code>, <code>-to_clock</code>, <code>-verbose</code>, <code>-nworst</code>.
<code>-nworst worst_entries</code>	Specifies the number of entries to be reported per clock domain. <i>Default:</i> <code>1</code>
<code>-rise -fall</code>	Specifies active transition at sequential devices clock pins. The <code>-rise</code> and <code>-fall</code> parameters are mutually exclusive and cannot be specified together. Note: You can specify these parameters only with the <code>-type latency</code> parameter.
<code>-source {clock_root generated_clock}</code>	Reports the latency from either the master clock, or the generated clock that was derived from it. The <code>-source</code> parameter can be used only when <code>-logic_level</code> is specified. The <code>-source</code> parameter cannot be specified in conjunction with the following parameters: <code>-launch</code>, <code>-capture</code>, <code>-from</code>, <code>-to</code>, <code>-from_clock</code>, <code>-to_clock</code>, <code>-verbose</code>, <code>-nworst</code>. <i>Default:</i> Reports the latency from the master clock (<code>-source clock_root</code>)

<code>-tcl_list</code>	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. The <code>-tcl_list</code> and <code>filename</code> parameters are mutually exclusive; you cannot specify them together.
<code>-to to_list</code>	Specifies the list or collection of sequential device clock pins or ports to be considered for reporting. If any pin in the <code>to_list</code> is not a sequential device clock pin, it is replaced with all the sequential device clock pins in the transitive fanout of the named pin. If there are no sequential pins in the transitive fanout of the pin, the software drops the pin from the list and generates a warning message.
<code>-to_clock to_clock_list</code>	Specifies the list of to-clocks to be used in generating the current interclock skew report. You can use this option only in an interclock skew report. <i>Default:</i> Report is generated considering all clocks as to-clocks.
<code>-type {skew interclock_skew jitter summary latency cpr_stage_count}</code>	Specifies the type of report to generate. The <code>-type</code> options are described below: • <code>skew</code>: Reports the skew for all clocks, or for clocks that you specify using the <code>-clock</code> parameter. • <code>interclock_skew</code>: Reports the skew between all clocks (excluding same clock pair) in the design, or between clocks that you specify using the <code>-from_clock</code> and <code>-to_clock</code> parameters. Note: The <code>skew</code> option can be used to report the previous results that show the missed out same clock skew values (reported using the <code>interclock_skew</code> option). • <code>jitter</code>: Reports the difference in late and early arrival times of a clock phase at a clock end point. When specified, the software generates a

clock jitter report for one clock end point, for each clock in the design.

You can generate a jitter report for all clocks, or clocks specified with the `-clock` parameter. You can limit the report to a specific clock end point, using the `-to` parameter.

Note: The `report_clock_timing -type jitter` option can be used only when the software is in On-Chip Variation (OCV) timing analysis mode. Jitter analysis does not remove any common path pessimism.

- **summary:** Generates a summary report for each analyzed clock that includes the following information:
 - When setup skew is used (`-late`):
 - maximum launch latency
 - minimum capture latency
 - maximum setup skew
 - When hold skew is used (`-early`):
 - minimum launch latency
 - maximum capture latency
 - maximum hold skew
- **latency:** Reports the network and source latency values for all the clock pins in the design, or for clock pins that you specify using the `-clock` parameter.
- **cpr_stage_count:** The generated report is arranged in terms of difference of stages from common pin to CK pin. The reports are generated on the basis of logical CPR points derived between each pair of phases reaching the launch/capture CK pins. Only same clock pairs (similar to type skew) are picked. For example,

```
Branch Point Difference Depth Clock Pin
-----
-
```

	<pre>inv2/Y 5 r ffc1/CK 4 1 f ffc3/CKN</pre>
<code>-verbose</code>	Generates a report that contains the full clock path from the clock source to the sink pin (the clock pin of the flop or latch). You can specify the <code>-verbose</code> parameter only for skew or latency reports (<code>-type skew</code>, <code>-type interclock_skew</code>, or <code>-type latency</code>). When this parameter is specified in SOCV mode, the intermediate arrival times of pins are shown as "-" in the report output.
<code>-view view_name</code>	Generates a report for the specified analysis view only. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode.

Examples

- The following command reports skew details for the specified list of clocks:

```
litmus > report_clock_timing -clock clk1 -type skew
Clock: clk1
Analysis View: view1
```

Skew	Latency	CPPR	Clock Pin	
	0.000		r	f0/CK
0.000	0.000	-0.000	r	f1/CK

- The following command uses hold skew to generate a report and shows details of skew between all the clocks:

```
litmus > report_clock_timing -early -type interclock_skew
Clock: clk2
Analysis View: view1
```

Skew	Latency	CPPR	Clock Pin	
	0.000		r	f0/CK
-0.000	0.000	0.000	r	f1/CK

- The following command reports details of skew between all the clocks for the specified sequential device clock pins:

```
litmus > report_clock_timing -from_clock {refclk m_spi_clk} -type interclock_skew
Clocks: m_spi_clk -> m_clk
```

Analysis View: func_slow_RCMAX

Skew	Latency	CPPR	Clock Pin

			TST_INST/p88/ZN
	0.580		r SPI_INST/spi_sr_reg[7]/CK
			TST_INST/p97/Z
-3.761	4.340	0.000	r TDSP_INST/as_reg/CK

- The following command uses hold skew (greater than 2) to generate a report and shows details of skew between all the clocks:

```
litmus > report_clock_timing -from {refclk m_spi_clk } -type skew -greater_than 2
```

Clock: m_clk

Analysis View: scan_fast_RCMIN

Skew	Latency	CPPR	Clock Pin

	3.960		r TDSP_INST/areg[0]/CK
2.735	1.225	-0.000	r ROM_512x16_INST/CLK

Clock: m_clk

Analysis View: scan_slow_RCMIN

Skew	Latency	CPPR	Clock Pin

	3.960		r TDSP_INST/areg[0]/CK
2.735	1.225	-0.000	r ROM_512x16_INST/CLK

- The following command generates the histogram latency report at the specified interval range:

```
litmus > report_clock_timing -clock m_clk -histogram -type latency -histogram_range 50
[Delay Chart: TST_INST/p21/Z(m_clk)]
```

```
-----
Min Delay :      1.118
Max Delay :      4.332
Count : 406
-----
```

```
-----
Delay Range
Min          Max          Count    1          10          100
-----
0.000        50.000      406    |oooooo    |          |oooo
[Delay Chart: TST_INST/p21/Z(m_clk)]
-----
```

```
-----
Min Delay :      1.052
Max Delay :      3.995
Count : 406
-----
```

Delay Range

Litmus Text Command Reference

Timing Analysis Commands--report_clock_timing

Min	Max	Count	1	10	100
0.000	50.000	406	oooooooo		oooo

- The following command uses setup skew to generate the report and shows skew details for the specified list of clocks:

```
litmus > report_clock_timing -clock m_clk -late -type skew
Clock: m_clk
Analysis View: scan_fast_RCMIN
```

Skew	Latency	CPPR	Clock Pin	
	3.960		r	TDSP_INST/acc_reg[0]/CK
2.735	1.225	-0.000	r	ROM_512x16_0_INST/CLK

```
Clock: m_clk
Analysis View: scan_slow_RCMIN
```

Skew	Latency	CPPR	Clock Pin	
	3.960		r	TDSP_INST/acc_reg[0]/CK
2.735	1.225	-0.000	r	ROM_512x16_0_INST/CLK

- The following command reports hold skew details for the specified list of clocks:

```
litmus > report_clock_timing -clock m_clk -early -type skew
Clock: m_clk
Analysis View: scan_fast_RCMIN
```

Skew	Latency	CPPR	Clock Pin	
	1.225		r	ROM_512x16_0_INST/CLK
2.765	3.990	0.000	r	TSP_CRE/PRG_BS/dt_reg[12]/CK

```
Clock: m_clk
Analysis View: scan_slow_RCMIN
```

Skew	Latency	CPPR	Clock Pin	
	1.225		r	ROM_512x16_0_INST/CLK
2.765	3.990	0.000	r	TSP_CRE/PRG_BS/dt_reg[12]/CK

- The following command reports launch latency values for the specified list of clocks:

```
litmus > report_clock_timing -clock m_clk -capture -type latency
Clock: m_clk
Analysis View: func_slow_RCMAX
```

---Latency---				
Source	Network	Total	Clock Pin	
0.000	1.102	1.102	r	ROM_512x16_0_INST/CLK

```
Clock: m_clk
Analysis View: func_fast_RCMAX
```

---Latency---				
Source	Network	Total	Clock Pin	

0.000	1.310	1.310	r	ROM_512x16_0_INST/CLK

- The following command reports latency details for the specified number of entries per clock domain:

```
litmus > report_clock_timing -clock m_clk -nworst 4 -type latency
Clock: m_clk
Analysis View: func_slow_RCMAX
```

---Latency---				
Source	Network	Total	Clock Pin	

0.000	4.735	4.735	r	DM_IST/write_reg/CK
0.000	4.735	4.735	r	DM_IST/top_flg_reg/CK
0.000	4.735	4.735	r	DM_IST/read_reg/CK
0.000	4.735	4.735	r	DM_IST/state_reg[3]/CK

- The following command reports latency details for all the levels between the generated clock root, and the capturing register.

```
litmus > report_clock_timing -source generated_clock -logic_level -type latency
Start point:refclk(refclk)
```

Delay	Count	End Point
3.697	3	SPI/spr_108/CK r
3.697	3	SPI/spr_107/CK r
3.697	3	SPI/spr_105/CK r
3.697	3	SPI/spr_103/CK r
3.697	3	SPI/spr_101/CK r
3.697	3	SPI/spr_100/CK r
3.697	3	SPI/spr_104/CK r
3.697	3	SPI/spr_102/CK r
3.697	3	SPI/spr_106/CK r
3.697	3	SPI/spr_109/CK r
3.697	3	SPI/spr_110/CK r

- The following command reports worst case skew for all the clocks in a design:

```
litmus > report_clock_timing -type skew -nworst 4
```

```
Clock: CLK_W_4
Skew    Latency    Clock Pin
-----
0.150 r    u99/CK
0.000 0.150 r    u55/CK
```

```
Clock: CLK_W_3
Skew    Latency    Clock Pin
-----
          0.682 r seg3/u9/CK
0.061    0.621 r seg3/u14/CK
```

```
Clock: CLK_W_2
Skew    Latency    Clock Pin
-----
          0.682 r seg2/u9/CK
0.061    0.621 r seg2/u14/CK
```

```
Clock: CLK_W_1
Skew    Latency    Clock Pin
-----
          0.682 r seg1/u9/CK
0.061    0.621 r seg1/u14/CK
```

Here 'r' indicates the transition (r - rise, f - fall) at the reported register clock pin.

- The following command reports interclock skew with -from_clock, -to_clock, -from, and -to parameters defined:

```
litmus > report_clock_timing -type interclock_skew -nworst 10 -from CLK4 -to u5/CK -from_clock
CLK_W_4 -to_clock CLK_W_4_gen2
```

```
Clocks: CLK_W_4 -> CLK_W_4_gen2
Skew      Latency      Clock Pin
-----
                  CLK4
          0.180 r      u99/CK
                  u2/Q
                  u8/Q
-0.205     0.386 ri     u5/CK
```

Here 'r' indicates the transition (r - rise, f - fall) at the reported register clock pin. The 'i' refers to the 'inverted' polarity of the clock signal at the reported register clock pin.

- The following command reports skew for all clocks in the design including common path pessimism removal:

```
litmus > set_db timing_analysis_cpnr both
```



```
litmus > report_clock_timing -type skew -nworst 10
```

Clock: CLK_W_4

Skew	Latency	CPPR	Clock Pin
------	---------	------	-----------

	0.180		r u99/CK
0.000	0.120	-0.060	r u55/CK

Clock: CLK_W_3

Skew	Latency	CPPR	Clock Pin
------	---------	------	-----------

	0.818		r seg3/u9/CK
0.139	0.496	-0.183	r seg3/u14/CK

Clock: CLK_W_2

Skew	Latency	CPPR	Clock Pin
------	---------	------	-----------

	0.818		r seg2/u9/CK
0.139	0.496	-0.183	r seg2/u14/CK

Clock: CLK_W_1

Skew	Latency	CPPR	Clock Pin
------	---------	------	-----------

	0.818		r seg1/u9/CK
0.139	0.496	-0.183	r seg1/u14/CK

- The following command generates a summary report for the clock:

```
litmus > report_clock_timing -type summary -nworst 10 -clock CLK_W_1
```

Clock: CLK_W_1

Skew	Latency	Clock Pin
------	---------	-----------

Maximum launch latency:	0.818 r	seg1/u9/CK
Minimum capture latency:	0.000 r	u2/CK
Maximum skew:	0.818 r	seg1/u9/CK
	0.322 0.496 r	seg1/u14/CK

- The following command reports the Network and source latency values for the clock CLK_W_1:

```
litmus > report_clock_timing -type latency -nworst 10 -clock CLK_W_1
```

Clock: CLK_W_1

---Latency---

Source	Network	Total	Clock Pin
--------	---------	-------	-----------

0.000	0.818	0.818	r seg1/u9/CK
0.000	0.000	0.000	r u2/CK

- The following command reports the Network and source latency values for the clock GCLK1, as well as the full clock paths:

```
litmus > report_clock_timing -type latency -verbose -clock CLK_W_1
```

```
Clock: CLK_W_1
```

```
---Latency---
```

```
Source Network Total Clock Pin
```

```
-----
```

```
0.000 0.818 0.818 r seg1/u9/CK
```

```
Clock latency Path
```

```
Clock Rise Edge 0.000
```

```
= Beginpoint Arrival Time 0.000
```

```
Other End Path:
```

```
-----
```

Instance	Arc	Cell	Delay	Clock Latency

-	CLK1 ^	-	-	0.000
seg1/u16	A ^ -> Y ^	CLKBUF	0.111	0.111
seg1/u17	A ^ -> Y ^	CLKBUF	0.126	0.236
seg1/u17_a	A ^ -> Y ^	CLKBUF	0.107	0.343
seg1/and1	B ^ -> Y ^	AND	0.071	0.818
seg1/u9	CK ^	DFF	0.000	0.818

```
-----
```

```
seg1/u16 A ^ -> Y ^ CLKBUF 0.111 0.111
```

```
seg1/u17 A ^ -> Y ^ CLKBUF 0.126 0.236
```

```
seg1/u17_a A ^ -> Y ^ CLKBUF 0.107 0.343
```

```
seg1/and1 B ^ -> Y ^ AND 0.071 0.818
```

```
seg1/u9 CK ^ DFF 0.000 0.818
```

```
-----
```

- When the software is in multi-mode multi-corner timing analysis mode, the software generates a report for every active analysis view in the design:

```
litmus > report_clock_timing -type latency
```

```
Clock: EJ_TCK
```

```
Analysis View: hold_view1
```

```
---Latency---
```

```
Source Network Total Clock Pin
```

```
-----
```

```
0.000 0.000 0.000 r cregister_q_reg_3/CP
```

```
Clock: EJ_TCK
```

```
Analysis View: hold_view2
```

```
---Latency---
```

```
Source Network Total Clock Pin
```

```
-----
```

```
0.000 0.000 0.000 r cregister_q_reg_3/CP
```

```
Clock: EJ_TCK
```

```
Analysis View: setup_view1
```

```
---Latency---
```

```
Source Network Total Clock Pin
```

```
-----
```

```
0.000 0.000 0.000 r cregister_q_reg_3_/CP
```

```
Clock: EJ_TCK
```

```
Analysis View: setup_view2
```

```
---Latency---
```

```
Source Network Total Clock Pin
```

```
-----  
0.000 0.000 0.000 r cregister_q_reg_3_/CP
```

- Specify the -view parameter to generate a report only for a specific analysis view

```
litmus > report_clock_timing -view setup_view1 -type latency
```

```
Clock: EJ_TCK
```

```
Analysis View: setup_view1
```

```
---Latency---
```

```
Source      Network      Total      Clock      Pin
```

```
-----  
0.000      0.000      0.000      r      cregister_q_reg_3_/CP
```

```
Clock: SI_ClkIn
```

```
Analysis View: setup_view1
```

```
---Latency---
```

```
Source      Network      Total      Clock      Pin
```

```
-----  
0.000      0.000      0.000      r      cregister_q_reg_13_/CP
```

- The following command generates a textual histogram of the latency report:

```
litmus > report_clock_timing -type latency -histogram
```

```
[Delay Chart: CLK4 (CLK_W_4_gen2)]
```

```
*****
```

```
Min Delay : 0.778
```

```
Max Delay : 0.778
```

```
Count : 1
```

```
*****
```

```
-----  
Delay      Range
```

```
Min      Max      Count      1
```

```
-----  
0.000  100.000  1      |o
```

- The following command generates the histogram report at intervals of 0.5:

```
litmus > report_clock_timing -type latency -histogram -histogram_range 0.5
```

```
[Delay Chart: CLK4 (CLK_W_4_gen2)]
```

```
*****
```

```
Min Delay : 0.778
Max Delay : 0.778
Count : 1
*****
-----
Delay Range
Min      Max      Count      1
-----
0.500    1.000    1          |o
0.000    0.500    0
```

- The following command generates a latency report that includes the stage gate counts between the clock root or generated clock root, and the capturing register:

```
litmus > report_clock_timing -type latency -logic_level
```

```
*****
Start point:CLK2 (CLK_W_4_gen1 (CLK_W_3))
-----
      Delay      Count      End Point
      1.018      8        seg2/u9/CK r
      1.018      8        seg2/u3/CK r
      0.945      7        seg2/u14/CK r
*****
```

```
Start point:CLK3 (CLK_W_4_gen2 (CLK_W_4))
-----
      Delay      Count      End Point
      1.345      8        seg3/u6/CK r
      1.345      8        seg3/u1/CK r
      1.218      7        seg3/u12/CK r
```

- The following command generates a latency report starting from the generated clock, and includes the stage gate counts between the generated clock and the capturing register:

```
litmus > report_clock_timing -type latency -logic_level -source generated_clock
```

```
*****
Start point:CLK2 (CLK_W_4_gen1 (CLK_W_3))
-----
      Delay      Count      End Point
```

Litmus Text Command Reference

Timing Analysis Commands--report_clock_timing

```

0.818          8      seg2/u9/CK r
0.818          8      seg2/u3/CK r
0.745          7      seg2/u14/CK r

```

Start point:CLK3 (CLK_W_4_gen2 (CLK_W_4))

Delay	Count	End Point
0.845	8	seg3/u6/CK r
0.845	8	seg3/u1/CK r
0.718	7	seg3/u12/CK r

- The following command generates a latency report starting from the generated clock CLK_W_4_gen2, and includes the stage gate counts between the generated clock and the capturing register:

```
litmus > report_clock_timing -type latency -logic_level -source generated_clock -clock
CLK_W_4_gen2
```

Start point:CLK3 (CLK_W_4_gen2 (CLK_W_4))

Delay	Count	End Point
0.845	8	seg3/u6/CK r
0.845	8	seg3/u1/CK r
0.718	7	seg3/u12/CK r

- The following command uses the -format parameter to customize the generated latency report:

```
litmus > report_clock_timing -type latency -verbose -format {instance arc pin cell slew load user_derate}
```

Clock: CLK_W_4

---Latency---

Source	Network	Total	Clock Pin
0.000	0.152	0.152	r u55/CK

Clock latency Path

Clock Rise Edge 0.000

= Beginpoint Arrival Time 0.000

Other End Path:

Instance	Arc	Pin	Cell	Slew	Load	User Derate

Litmus Text Command Reference

Timing Analysis Commands--report_clock_timing

```

-          CLK4 ^          CLK4      -          0.003  0.025  -
b11        A ^ -> Y ^      Y          BUF      0.131  0.011  1.000
b12        A ^ -> Y ^      Y          BUF      0.117  0.015  1.000
u55        CK ^           CK          DFF      0.117  0.015  1.000
-----

```

- The following command reports the worst case jitter for all clocks in the design:

```
litmus > report_clock_timing -type jitter -nworst 2
```

Clock: CLK_W_2

Jitter	Latency(Late)	Latency(Early)	Clock Pin

0.060	0.180 r	0.120 r	u55/CK
0.060	0.180 r	0.120 r	u99/CK

0.060 0.180 r 0.120 r u55/CK

0.060 0.180 r 0.120 r u99/CK

Clock: CLK_W_1

Jitter	Latency(Late)	Latency(Early)	Clock Pin

0.273	0.818 r	0.545	r seg1/u9/CK
0.273	0.818 r	0.545	r seg1/u3/CK

0.273 0.818 r 0.545 r seg1/u9/CK

0.273 0.818 r 0.545 r seg1/u3/CK

- The following command reports the worst case jitter for the clock CLK_W_1:

```
litmus > report_clock_timing -type jitter -clock CLK_W_1 -nworst 2
```

Clock: CLK_W_1

Jitter	Latency(Late)	Latency(Early)	Clock Pin

0.273	0.818 r	0.545 r	seg1/u9/CK
0.273	0.818 r	0.545 r	seg1/u3/CK

0.273 0.818 r 0.545 r seg1/u9/CK

0.273 0.818 r 0.545 r seg1/u3/CK

- The following command reports the worst case jitter for the specified clock end point:

```
litmus > report_clock_timing -type jitter -to seg1/u3/CK
```

Clock: CLK_W_1

Jitter	Latency(Late)	Latency(Early)	Clock Pin

0.273	0.818 r	0.545 r	seg1/u3/CK

0.273 0.818 r 0.545 r seg1/u3/CK

Related Commands

- [report_timing](#)
- [check_timing](#)

report_clocks

```
report_clocks
[-description]
[-arrival_points]
[-phase_shift_table]
[-total_shift_table]
[-uncertainty_table]
[-adjustment_table]
[-delay_adjustment_table]
[-source_insertion]
[-insertion]
[-hierarchy]
[-groups]
[-clocks clk_signame | clk_signame_list]
[-view view_name]
[<> | >>] filename | -tcl_list]
```

Reports information on clocks created with the [create_clock](#) and [create_generated_clock](#) command. The report includes information about all clock waveforms, clock arrival points, and clock uncertainties.

Use the `-description`, `-arrival_points`, `-uncertainty_table`, and the `*_table` parameters to specify different parts of the report. If one of these options is supplied, only the section that the option corresponds to is reported. If other options are supplied, those sections are also reported.

Based on the current setup or hold analysis mode, the following parameters report only one of the late or early tables:

- `-phase_shift_table`
- `-total_shift_table`
- `-uncertainty_table`
- `-adjustment_table`
- `-delay_adjustment_table`

However, in simultaneous (setup and hold) mode both the tables are reported.



The `report_clocks` command only reports constraints with respect to clock waveforms. Use the `report_ports -type` command for constraints applied to clock pins.

Parameters

<code>{> >>} filename</code>	When specified with <code>></code>, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with <code>>></code>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The <code>filename</code> parameter must be the last argument in the list. The <code>filename</code> and <code>-tcl_list</code> parameters are mutually exclusive; you cannot specify them together. Note: To write a compressed report, add the <code>.gz</code> extension to the file name. <i>Default:</i> Report is displayed on standard output without being saved.
<code>-adjustment_table</code>	Reports two tables that detail the hold and setup cycle adjustments between clock waveforms as set by the <code>set_multicycle_path</code> command. Note: If specific ideal (or generated) clocks are requested with the <code>-clocks clock_name_list</code> option, then only the relationships between those clock signals are reported.
<code>-arrival_points</code>	Reports only the clock arrival points. A clock arrival point is the pin where the create_clock or create_generated_clock command has been asserted. Note: This parameter is obsolete and will be removed in a future release.

<code>-clocks clk_signame clk_signame_list</code>	Requests information on a specific ideal (or generated) clock. If a list is given, only information about those clocks is reported. You can use wildcards in the names. <i>Default:</i> All clocks, except generated clocks, are reported. Use the <code>-generated</code> option to report generated clocks. Note: This parameter is obsolete and will be removed in a future release. Cadence recommends that you specify clock names without the <code>-clocks</code> parameter.
<code>-delay_adjustment_table</code>	Reports two tables that detail the early and late path delay adjustments between clock waveforms as set by the <code>set_max_delay</code> and <code>set_min_delay</code> commands. Note: If specific ideal (or generated) clocks are requested with the <code>-clocks clock_name_list</code> option, then only the relationships between those clock signals are reported.
<code>-description</code>	Reports only the description of the specified clock waveform(s).
<code>-generated</code>	Includes information on generated clocks in the report, such as source and master clock of the generated clock, and other information specified in the <code>create_generated_clock</code> constraints. The report uses the internal clock names created by the system, if you did not specify new clock names while creating these clocks. Note: This parameter is obsolete and will be removed in a future release. The software now reports generated clocks by default.
<code>-groups</code>	Reports the existing clock groups.
<code>-hierarchy</code>	Reports the inter-clock relationship in tree style format.
<code>-insertion</code>	Reports the network insertion delays specified on clock waveforms. The values in the table are in <code>min:typ:max</code> format.

<code>-phase_shift_table</code>	Reports the table that details the phase shift between clock waveforms. Based upon the current analysis mode, the table shows the phase shifts either in late or early mode. In case of simultaneous setup hold mode, both the tables are reported for phase shift in late and early mode. Note: If specific ideal (or generated) clocks are requested with the <code>-clocks clock_name_list</code> option, then only the relationships between those clock signals are reported.
<code>-source_insertion</code>	Reports the source insertion delays specified on clock waveforms.
<code>-tcl_list</code>	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. The <code>-tcl_list</code> and <code>filename</code> parameters are mutually exclusive; you cannot specify them together.
<code>-total_shift_table</code>	Based on the current analysis mode, reports (late or early mode) details and the relationship between clock waveforms. In case of simultaneous setup hold mode all the details are reported. Note: If specific ideal (or generated) clocks are requested with the <code>-clocks clock_name_list</code> option, then only the relationships between those clock signals are reported.
<code>-uncertainty_table</code>	Based on the current analysis mode, reports one of the tables (late mode or early mode) that detail the uncertainty between clock waveforms as specified using the <code>set_clock_uncertainty</code> command. In case of simultaneous setup hold mode both the tables are reported. The tables report only those clock waveforms for which uncertainty has been defined using the <code>set_clock_uncertainty</code> command. Note: If specific ideal (or generated) clocks are requested with the <code>-clocks clock_name_list</code> option, then only the relationships between those clock signals are reported.

`-view view_name`

Generates a clock report for the specified analysis view only. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode.

Examples

- The following command reports the clock arrival points, and details of the clock waveform(s):

```
litmus > report_clocks -arrival_points -description
```

Clock Descriptions							
Attributes							
Clock Name	Source	View	Period	Lead	Trail	Generated	Propagated
m_clk	TST_INST/p21/Z	scan_slow_RCMAX	40.000	0.000	20.000	n	y
m_clk	TST_INST/p21/Z	scan_fast_RCMAX	40.000	0.000	20.000	n	y
m_clk	TST_INST/p21/Z	func_fast_RCMAX	4.000	0.000	2.000	n	y
m_clk	TST_INST/p21/Z	func_slow_RCMAX	4.000	0.000	2.000	n	y
m_clk	TST_INST/p21/Z	scan_slow_RCMIN	40.000	0.000	20.000	n	y
m_clk	TST_INST/p21/Z	func_fast_RCMIN	4.000	0.000	2.000	n	y
m_clk	TST_INST/p21/Z	scan_fast_RCMIN	40.000	0.000	20.000	n	y
m_clk	TST_INST/p21/Z	func_slow_RCMIN	4.000	0.000	2.000	n	y

- The following command reports the clock arrival points, along with the hold and setup cycle adjustments between clock waveforms:

```
litmus > report_clocks -arrival_points -description -clocks m_clk -adjustment_table
```

Clock Descriptions							
Attributes							
Clock Name	Source	View	Period	Lead	Trail	Generated	Propagated
m_clk	TST_INST/p21/Z	scan_slow_RCMAX	40.000	0.000	20.000	n	y
m_clk	TST_INST/p21/Z	func_fast_RCMIN	4.000	0.000	2.000	n	y

Litmus Text Command Reference
Timing Analysis Commands--report_clocks

m_clk	TST_INST/p21/Z	scan_slow_RCMIN	40.000	0.000	20.000	n	y
m_clk	TST_INST/p21/Z	func_slow_RCMIN	4.000	0.000	2.000	n	y
m_clk	TST_INST/p21/Z	scan_fast_RCMIN	40.000	0.000	20.000	n	y
m_clk	TST_INST/p21/Z	func_fast_RCMAX	4.000	0.000	2.000	n	y
m_clk	TST_INST/p21/Z	scan_fast_RCMAX	40.000	0.000	20.000	n	y
m_clk	TST_INST/p21/Z	func_slow_RCMAX	4.000	0.000	2.000	n	y

Clock To Clock Cycle Adjustment (Late)

From	To	View	L->L	L->T	T->L	T->T
m_clk	m_clk	scan_slow_RCMAX	0.000	0.000	0.000	0.000
m_clk	m_clk	func_fast_RCMIN	0.000	0.000	0.000	0.000
m_clk	m_clk	scan_slow_RCMIN	0.000	0.000	0.000	0.000
m_clk	m_clk	func_slow_RCMIN	0.000	0.000	0.000	0.000
m_clk	m_clk	scan_fast_RCMIN	0.000	0.000	0.000	0.000
m_clk	m_clk	func_fast_RCMAX	0.000	0.000	0.000	0.000
m_clk	m_clk	scan_fast_RCMAX	0.000	0.000	0.000	0.000
m_clk	m_clk	func_slow_RCMAX	0.000	0.000	0.000	0.000

- The following command reports details of generated clocks, and includes information about all the clock waveforms and clock arrival points:

```
litmus > report_clocks
```

Clock Descriptions

Attributes

Clock Name	Source	Period	Lead	Trail	Generated	Propagated
CLK1	CLK1	20.000	0.000	10.000	n	y
CLK2	CLK1	10.000	0.000	5.000	n	y

Generated-Clock Descriptions

Name	Generated	Master	Master-clock	Invert	Freq.	Duty-Cycle	Edges	Edge-Shift
------	-----------	--------	--------------	--------	-------	------------	-------	------------

	Source(pin)	Source(pin)			Multiplier			
GCLK	u8/Q	CK1	CLK1	n	1/2	-	-	-

- The following command reports uncertainty information as per the defined assertions:

```
litmus > set_clock_uncertainty 5 CLK_W_1
litmus > set_clock_uncertainty 4 -from CLK_W_1 -to CLK_W_4_gen1
litmus > set_clock_uncertainty 6 -from CLK_W_2 -to CLK_W_4_gen2
litmus > report_clocks -uncertainty_table
```

Clock To Clock Uncertainty (Late)

From	To	L->L	L->T	T->L	T->T
*	CLK_W_1	5.000	5.000	5.000	5.000
CLK_W_1	CLK_W_4_gen1	4.000	4.000	4.000	4.000
CLK_W_2	CLK_W_4_gen2	6.000	6.000	6.000	6.000

Clock To Clock Uncertainty (Early)

From	To	L->L	L->T	T->L	T->T
*	CLK_W_1	-5.000	-5.000	-5.000	-5.000
CLK_W_1	CLK_W_4_gen1	-4.000	-4.000	-4.000	-4.000
CLK_W_2	CLK_W_4_gen2	-6.000	-6.000	-6.000	-6.000

- The following command reports the phase shift between clock waveforms for the defined clock:

```
litmus > create_clock -name "CLK_W_1" -add -period 60 -waveform {0 30}
litmus > report_clocks CLK_W_1 -phase_shift_table
```

Clock To Clock Phase Shift (Late)

From	To	L->L	L->T	T->L	T->T
CLK_W_1	CLK_W_1	60.000	0.000	60.000	60.000

Clock To Clock Phase Shift (Early)

```

From      To      L->L      L->T      T->L      T->T
-----
CLK_W_1 CLK_W_1 0.000 -60.000 0.000 0.000
-----

```

- You can use the following command to define uncertainty for a clock:
Based on the current analysis mode, reports (late or early mode) details and the relationship between clock waveforms.

```
litmus > set_clock_uncertainty 2 CLK_W_1
```

The following command reports (late or early mode) details and displays the changed shift values:

```
litmus > report_clocks -total_shift_table CLK_W_1
```

```

-----
Total Clock To Clock Phase Shift Required (Late)
-----
From      To      L->L      L->T      T->L      T->T
-----
CLK_W_1 CLK_W_1 58.000 -2.000 58.000 58.000
-----

Total Clock To Clock Phase Shift Required (Early)
-----
From      To      L->L      L->T      T->L      T->T
-----
CLK_W_1 CLK_W_1 2.000 -58.000 2.000 2.000
-----

```

You can see that the uncertainty is subtracted from the phase shift.
Consider the following example path:

```

Path 1: MET Setup Check with Pin u2/CK
Endpoint: u2/D (v) checked with leading edge of 'CLK_W_1'
Beginpoint: CLK2 (v) triggered by trailing edge of 'CLK_W_2'
Path Groups: {CLK_W_1}
Other End Arrival Time 0.000
- Setup 0.188
+ Phase Shift 15.000
- Uncertainty 2.000
= Required Time 12.812
- Arrival Time 12.463
= Slack Time 0.349
Clock Fall Edge 12.500
= Beginpoint Arrival Time 12.500
-----
Load  Slew  Delay  Arrival  Cell  Arc  Pin
      Time
-----
0.034 0.003 12.500          CLK2 v  CLK2
0.003 0.003 -0.037 12.463  BUF  A v -> Y v u10999/Y

```

```
0.003  0.003  0.000  12.463  DFF  D v  u2/D
```

Here the begin point CLK_W_2 to end point CLK_W_1 shows a phase shift value of 15.00. This means that if a path starts from the falling edge of CLK_W_2 , and ends at the rising edge of CLK_W_1, the phase shift is 15.00. In other words, when the timing (setup time) is reported, the required time at the end point begins with 15.00 (in addition to adjustments like setup time, time borrowing, uncertainty, etc). However, the arrival time of this path starts at 12.500.

- When the software is in multi-mode multi-corner timing analysis mode, the software reports the clock information for each active analysis view:

```
litmus > report_clocks -total_shift_table -description
```

```
-----
Clock Descriptions
-----
```

```
Attributes
-----
```

Clock Name	Source	View	Period	Lead	Trail	Generated	Propagated
EJ_TCK	EJ_TCK	hold_view2	17.400	0.000	8.700	n	n
EJ_TCK	EJ_TCK	hold_view1	17.400	0.000	8.700	n	n
EJ_TCK	EJ_TCK	setup_view1	17.400	0.000	8.700	n	n

```
-----
Total Clock To Clock Phase Shift Required (Late)
-----
```

From	To	View	L->L	L->T	T->L	T->T
EJ_TCK	EJ_TCK	hold_view2	17.400	0.000	17.400	17.400
EJ_TCK	EJ_TCK	hold_view1	17.400	0.000	17.400	17.400
EJ_TCK	EJ_TCK	setup_view2	17.400	0.000	17.400	17.400

- The following command reports the late or early mode details and displays the changed shift values for the specified view:

```
litmus > report_clocks -total_shift_table -description -view setup_view1 > clk_sv1
```

```
-----
Clock Descriptions
-----
```

```
Attributes
-----
```

Clock Name	Source	View	Period	Lead	Trail	Generated	Propagated
EJ_TCK	EJ_TCK	setup_view1	17.400	0.000	8.700	n	n
virt_in_EJ_TCK	-	setup_view1	17.400	0.000	8.700	n	n

```
-----
Total Clock To Clock Phase Shift Required (Late)
-----
```

From	To	View	L->L	L->T	T->L	T->T
EJ_TCK	EJ_TCK	setup_view1	17.400	0.000	17.400	17.400
virt_in_EJ_TCK	EJ_TCK	setup_view1	17.400	0.000	17.400	17.400

- The `report_clocks` command output displays a column showing active or inactive clocks in the design:

Clock Descriptions							
Attributes							
Clock Name	Source	Period	Lead	Trail	Generated	Propagated	Active
CLK_W_1	CLK1	20.000	0.000	10.000	n	y	y
CLK_W_2	CLK2	25.000	0.000	12.500	n	y	y
CLK_W_4_gen2	u8/Q	20.000	0.000	10.000	y	y	y

Related Commands

- `create_clock`
- `create_generated_clock`
- `set_clock_uncertainty`

report_constraint

```
report_constraint
[-all_violators]
[-late]
[-early]
[-em_type {peak | rms | avg | all}]
[-check_type { pulse_width | clock_period | recovery | removal | clock_gating_setup |
clock_gating_hold | skew | pulse_clock_max_width | pulse_clock_min_width | electromigration |
rail_swing}]
[-clock]
[-connection_class]
[-data]
[-drv_abs_margin float]
[-drv_fields {field_list}]
[-drv_output_format {txt|csv}]
[-drv_ratio_margin float]
[-drv_violation_type { max_capacitance | max_transition | max_fanout | min_capacitance |
min_transition | min_fanout | pulse_clock_max_transition | pulse_clock_min_transition}]
[-max_pins_per_drv <integer>]
[-retime {aocv | path_slew_propagation | aocv_path_slew_propagation}]
[-retime_mode {path | exhaustive}]
[-view viewName ]
[-worst_drv_per_net]
[pin_port_list]
[-verbose]
[ {> | >>} filename[.gz]]
```

Reports constraint information of current design. The constraints information includes information such as whether or not constraint is violated, by how much amount, and the worst violator object.

The following timing checks are reported when you use the `report_constraint` command:

- All setup and hold checks
- DRV
- Clock and asynchronous checks including `pulse_width`, `min_period`, `recovery`, `removal`, `clock_gating_setup`, `clock_gating_hold`, `skew`

You can specify the design rule constraints on pins, ports, cells, or a design using the commands: `set_min_fanout`, `set_max_fanout`, `set_min_transition`, `set_max_transition`, `set_min_capacitance`, and `set_max_capacitance`.

The `report_constraint` command checks against the worst design rule constraint between the library constraint limit and the user-specified constraint value, when both are applied on the violating pin or port.

The `report_constraint` command supports multi-threaded reporting. You can make the following settings to

enable multi-threaded reporting:

```
set_multi_cpu_usage -localCpu number_of_CPU
set_table_style -no_frame_fix_width [-nosplit]
```

By default, the `report_constraint` command reports only the mean slews in transition DRV checks. You can enable DRV checking to use mean as well as the n-sigma slew values (in SOCV mode), with the following setting:

```
set timing_report_max_transition_check_using_nsigma_slew true
```

Configurable DRV Reports

Tempus allows you to perform design rule violation (DRV) checks to ensure that the netlist meets the design rule constraints specified in the design library. To enable configurable DRV reporting, you can use the `timing_report_constraint_format` attribute. For example,

```
set timing_report_constraint_format {list of DRV fields}
report_constraint -drv_violation_type {list of DRV checks} \
    -all_violators | pin_or_port_list
```

You can also use the following command to specify DRV fields:

```
report_constraint -drv_violation_type {list of DRV checks} -drv_fields field_list
```

The field list specified with the `-drv_fields` parameter has higher precedence and will override the field list specified with the `timing_report_constraint_format` global variable. If DRV fields are not specified with this global variable (or with the `report_constraint -drv_fields` command), then the default set of DRV fields will be displayed. For a complete list of DRV fields, refer to the [DRV Fields](#) table.

For more information, refer to the [Design Rule Violation \(DRV\) Reporting](#) section in the *Tempus Stylus User Guide*.

Parameters

<pre>{> >>} <i>filename</i></pre>	When specified with <code>></code>, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with <code>>></code>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The <code><i>filename</i></code> parameter must be the last argument in the list. Note: To write a compressed report, add the <code>.gz</code> extension to the file name. Default: Report is displayed on standard output without being saved.
--	---

<code>-all_violators</code>	Reports the worst negative slack per violating pin against all the timing and design rule constraint checks. For timing checks, the violations are reported at timing endpoints. The DRV is checked and reported on leaf pins and ports. The supported check types are: • setup • hold • clock_gating_setup • clock_gating_hold • recovery • removal • clock_period • pulse_width • max_capacitance • max_transition • max_fanout • min_capacitance • min_transition • min_fanout • skew • pulse_clock_max_width • pulse_clock_min_width • pulse_clock_max_transition • pulse_clock_min_transition You can use the <code>-all_violators</code> parameter for reporting specific check types to generate report for that particular check type only. For example, you can use <code>-all_violators</code> with <code>-late</code> to report setup checks and path delays from <code>set_max_delay</code>.
<code>-check_type</code>	Reports setup or hold checks. The supported check types are: • <code>clock_gating_setup</code>: Reports only the paths that end at the <code>clock_gating_setup</code> timing check. • <code>clock_gating_hold</code>: Reports only the paths that end at the <code>clock_gating_hold</code> timing check.

- `electromigration`: Reports signal EM analysis related violations. When high density current passes through a thin metal wire, the high energy electrons exert forces on atoms causing them to migrate. Electromigration (EM) can drastically reduce the lifetime of a chip by increasing the resistivity of a metal wire or creating a short circuit between adjacent lines. When this option is specified, the software reports standard cell level electromigration violations.

- `recovery`: Reports paths that end at the recovery timing check.

- `removal`: Reports paths that end at the removal timing check.

Note: The check types - `recovery`, `removal`, `clock_gating_setup`, and `clock_gating_hold` are also supported in AOCV mode.

- `pulse_width`: Reports only the paths that end at the `pulse_width` timing check.

If you do not specify the `-all_violators` parameter, then the `pulse_width` option reports only one entry of the most critical `pulse_width` check. The DRV checks also report only the most critical entry.

To report all the violated `pulse_width` checks, you need to specify `report_constraint -check_type pulse_width -all_violators` command (which is consistent with other check types of the `report_constraint` command).

To report both violating and met entries, you can use the `report_min_pulse_width` command.

- `clock_period`: Reports only the paths that end at the `clock_period` timing check.

- `skew`: Reports skew checks.

The `report_constraint` command performs skew checks by default. Skew checks are implemented as setup checks that are reported in the late analysis or simultaneous mode. These checks are not reported in the early mode. Skew checks are performed assuming the signal and reference clocks have the same frequency. If the frequencies of the signal and reference clocks are different, a warning message is displayed.

- `pulse_clock_max_width`: Reports the maximum pulse clock width checks set using the `set_pulse_clock_max_width` command.
- `pulse_clock_min_width`: Reports the minimum pulse clock width checks.
- `rail_swing`: Reports details related to the rail swing checks. For rail swing

	check settings, you can use the following attributes: ◦ <code>timing_rail_swing_checks_low_voltage_threshold</code>
<code>-clock</code>	Specifies that the DRV checks will be restricted to the clock network. If any pin is part of both the clock and data network, it will be reported as a clock.
<code>-connection_class</code>	Reports violations if the connection class attributes of the driving pin and receiving pin of a net are different.
<code>-data</code>	Specifies that the DRV check will be restricted to the data network. If any pin is part of both the clock and data network, it will be reported as a clock network pin. By default, the DRV checks are applied to both clock and data networks.
<code>-drv_abs_margin</code> <i>float</i>	Specifies the absolute DRV margin value applied to obtain the threshold transition.
<code>-drv_fields</code> { <i>field_list</i> }	Allows only the specified fields to be included in the DRV report - in the same order as defined. For a complete list of DRV fields, refer to the DRV Fields table.
<code>-drv_output_format</code> <i>{txt csv}</i>	Specifies that DRV output format – txt or CSV. The default value is “txt”.
<code>-drv_ratio_margin</code> <i>float</i>	Specifies the ratio DRV margin value applied to obtain the threshold transition.

- drv_violation_type	Reports DRV violations. The supported DRV check types are: • max_capacitance: Reports pins having the worst <code>max_capacitance</code> constraint. If you specify the <code>-all_violators</code> parameter, you can see all the pins whose actual capacitance is greater than the <code>max_capacitance</code> value specified in library and timing constraints file. • min_capacitance: Reports pins having the worst <code>min_capacitance</code> constraint. If you specify the <code>-all_violators</code> parameter, you can see all the pins whose actual capacitance is less than the <code>min_capacitance</code> value specified in the library and timing constraints file. • max_transition: Reports pins having the worst <code>max_transition</code> constraint. If you specify the <code>-all_violators</code> parameter, you can see all the pins whose actual transition is greater than the <code>max_transition</code> value specified in the library and timing constraint file. Note: These checks are not reported in hold analysis mode. The <code>set_si_mode -enable_drv_with_delta_slew true</code> setting is required for <code>max_transition</code> DRV violations with SI slew. • min_transition: Reports pins having the worst <code>min_transition</code> constraint. If you specify the <code>-all_violators</code> parameter, you can see all the pins whose actual transition is less than the <code>min_transition</code> value specified in the library and timing constraints file. • max_fanout: Reports pins having the worst <code>max_fanout</code> constraint. If you specify the <code>-all_violators</code> parameter, you can see all the pins whose actual fanout is greater than the <code>max_fanout</code> value specified in the library and timing constraints file. • min_fanout: Reports pins having the worst <code>min_fanout</code> constraint. If you specify the <code>-all_violators</code> parameter, you can see all the pins whose actual fanout is less than the <code>min_fanout</code> value specified in the library and timing constraints file. • pulse_clock_max_transition: Reports maximum pulse clock transition checks. • pulse_clock_min_transition: Reports minimum pulse clock transition checks.
-early	Reports all of the hold checks, including regular, clock_gating_hold, and data-to-data hold checks. Also reports path delays from <code>set_min_delay</code>. Note: This parameter does not report removal checks.

<code>-em_type {peak rms avg all}</code>	Reports DRV violations as per the specified EM type table lookup. When this parameter is specified, the software reads the <code>em_max_toggle_rate</code> table in the Liberty library for the specified EM current type (RMS, peak, or average).
<code>-late</code>	Reports all of the setup checks, including regular, clock_gating_setup, and data-to-data setup checks. Also reports path delays from <code>set_max_delay</code>. Note: This parameter does not report recovery checks.
<code>-max_pins_per_drv</code>	Reports the specified number of top worst violations per DRV category, such as, <code>max_transition</code>, <code>max_capacitance</code>. If this parameter is not specified, then by default 1000 entries per DRV category will be reported.
<code>pin_port_list</code>	Controls the output of DRV constraints. When specified, all DRV violations are reported for the specified pins or ports only.
<code>-retime {aocv path_slew_propagation aocv_path_slew_propagation}</code>	
	Reanalyzes the specified set of paths using the specified method. Note: The parameter supports setup/hold check types only. <code>aocv</code>: Re-evaluates the given set of graph-based analysis (GBA) paths by reverting back graph-based AOCV derate factors and applying path-based AOCV derates instead of less pessimistic derates. In this case the slew (and respective delays) values are worst that are propagated during graph-based analysis. <code>aocv_path_slew_propagation</code>: Re-evaluates the given set of graph-based analysis (GBA) paths by using path-based AOCV derates and delays using actual path-based propagated slew. <code>path_slew_propagation</code>: Re-evaluates the given set of graph-based analysis (GBA) paths by recalculating the delay values based upon actual propagated slew across the path. In this case the AOCV derates are the ones that are reported during graph-based analysis.

<pre>-retime_mode {path exhaustive}</pre>	Allows the <code>report_constraint</code> command to be run in exhaustive path-based analysis (PBA) mode. In this mode, the software will exhaustively examine <code>n</code> worst paths of the selected endpoints and then select the true worst path to be reported - based on retimed slacks of all the examined paths. To enable exhaustive mode, the use model is given below: <pre>report_constraint -all_violators -retime path_slew_propagation - retime_mode exhaustive</pre> When you use this parameter, reporting can be run time intensive as the software needs to do an exhaustive search for true worst paths. Therefore, it is recommended to use this option in the later phases of sign-off stage when there are very few violations left for analysis. <i>Default:</i> <code>path</code>
<pre>-verbose</pre>	Generates a report that displays the full data path with the accompanying required time and slack calculation.
<pre>-view viewName</pre>	Generates a report containing endpoint and other constraint information for the specified analysis view. You can only specify this parameter when the software is in multi-mode multi-corner analysis mode. <i>Default:</i> Generates a report containing the worst endpoint and constraint information based on all of the analysis views.
<pre>- worst_drv_per_net</pre>	Reports only the worst DRV violation per net.

DRV Fields

Field Name	Report Column	Description
capacitance_actual	Actual Capacitance	Specifies the actual minimum or maximum capacitance at a pin or port. This field is used to determine the capacitance slack value.
capacitance_slack	Capacitance Slack	Specifies the <code>max_cap/min_cap</code> slack at a pin or port.

direction	Direction	Specifies the pin or port direction.
driver_max_capacitance	Driver Required Maximum Capacitance	Specifies the max_capacitance limit at the driver pin or port.
driver_max_fanout	Driver Required Maximum Fanout	Specifies the worst max_fanout limit on the driver pin or port.
driver_max_transition	Driver Required Maximum Transition	Specifies the max_transition limit at the driver pin or port.
driver_min_capacitance	Driver Required Minimum Capacitance	Specifies the min_capacitance limit at the driver pin or port.
driver_min_transition	Driver Required Minimum Transition	Specifies the min_transition limit at the driver pin or port.
driver_pin	Driver	Specifies the leaf driver pin or port name.
driver_ref_cell	Driver Libcell	Specifies the leaf driver reference cell or port.
driver_transition	Driver Actual Transition	Specifies the actual transition at driver pin or port.
drv_source	Source of DRV	Specifies the source of the DRV constraint which is used for the DRV check. Reports 'library_constraint' or 'user_constraint' depending on which DRV violation check is used.
edge	Edge	The signal transition direction on the net. Returns "rise" or "fall" if the DRV report corresponds to a rising or falling transition on the net.
fanout_actual	Actual Fanout	Specifies the actual minimum or maximum fanout at a pin or port. This field is used to determine the fanout slack value.
fanout_slack	Fanout Slack	Specifies the max_fanout/min_fanout slack at a pin or port.
hnet	Hierarchical Nets	Reports the hierarchical nets associated with violating pins.
lib_constraint	Library Constraint	Specifies the value of the library constraint on the violating pin. If not defined, reports as "NA".
max_capacitance	Required Capacitance	Specifies the maximum capacitance limit for a pin.

max_clock_frequency	Frequency	Specifies the frequency of the clock reported under the max_frequency_clock field.
max_fanout	Required Fanout	Specifies the maximum fanout limit for a pin.
max_frequency_clock	Maximum Frequency Clock	Specifies the name of the highest frequency clock constraining the pin. For clock network pins, the fastest clock reaching the pin will be reported. For data network pins, the fastest launch clock reaching the pin will be reported.
max_slack	Worst Setup Slack	Specifies the worst setup slack through at a pin or port.
max_transition	Required Transition	Specifies the maximum transition limit for a pin or port.
min_capacitance	Required Capacitance	Specifies the minimum capacitance limit for a pin.
min_fanout	Required Fanout	Specifies the minimum fanout limit for a pin.
min_slack	Worst Hold Slack	Specifies the worst hold slack through at a pin or port.
min_transition	Required Transition	Specifies the minimum transition limit for a pin or port.
net	Net	Specifies the top flat net connected to violating pin (net name at topmost hierarchy) flat net.
hnet	Hierarchical Nets	Reports hierarchical nets. When the <code>timing_enable_hierarchical_get_nets_support</code> attribute is set to <code>true</code> , the software enables support for displaying hierarchical net names.
net_fanout	Net Fanout	Specifies the number of leaf level load pins of the net connected to a pin or port.
net_resistance	NetResistance	Resistance of the net connected to a pin or port.

net_type	Net Type	Specifies one or more of the following identifiers to indicate a special net: • ideal: the net is marked ideal using the set_ideal_network command • clock: the net is part of a clock network • constant: the net is tied to logic 1/0 respectively • multidriven: the net has multiple driver • dont_touch: the net is marked as dont_touch using set_dont_touch command • "-": None of the above
partition	Partition	Specifies the name of the SPEF hierarchy the violating pin appears in.
pin	Pin	Reports the pin or port on which DRV check is performed.
pin_type	Type	Specifies whether the pin/port is a data or a clock pin/port. Any clock network pin will be reported as CLOCK. Any pin that belongs to only data network will be reported as DATA. If a pin/port belongs to both data and clock networks, it will be reported as CLOCK.
receiver_max_transition	Receiver Required Maximum Transition	Specifies the max_transition limit at a receiver pin or port.
receiver_pin	Receiver	Specifies the leaf receiver pin name.
receiver_ref_cell	Receiver Libcell	Specifies the leaf reference cell name of a receiver pin.
receiver_transition	Receiver Actual Transition	Specifies the actual transition at a receiver pin or port.
ref_cell	Violating Libcell	Specifies the reference library cell the violating pin is attached to.
sequential	Sequential	Specifies if the pin belongs to a sequential cell or a combinational cell. For ports, the report will show "false".
total_capacitance	TotalCapacitance	Total capacitance of the net connected to a pin or port.

transition_actual	Actual Transition	Specifies the actual minimum/maximum transition at a pin or port. This can be base transition or base plus crosstalk-induced transition depending on the timing_max_transition_use_si_transition setting to false (default) or true, respectively.
transition_si	SI Transition	Specifies only the delta SI transition time at a pin or port. The total transition time comprises of the base transition time (transition value without SI crosstalk analysis) as well as delta SI transition time.
transition_sigma	Transition N-Sigma	Specifies the N-Sigma multiplier defined for transition with the <code>set_socv_reporting_nsigma_multiplier -transition <nsigma></code> command. This field will be reported for SOCV STA, under the <code>timing_report_max_transition_check_using_nsigma_slew</code> to <code>true</code> setting. This is sensitive to views and -data and -clock specification. Note: This field will impact transition_actual and transition_slack computation for max_transition, min_transition, pulse_clock_max_transition, and pulse_clock_min_transition checks.
transition_slack	Transition Slack	Specifies the max_transition/min_transition slack at a pin/port pin/port.
user_constraint	User Constraint	Specifies user defined tran/cap/fanout limit value set by constraints as command or through SDC on the violating pin. If not defined, reports as "NA".
view	View	Name of the pertaining analysis view.

Examples

- The following command reports the worst negative slack per violating pin under pulse width checks:

```
tempus > report_constraint FE_C9/A -all_violators -check_type pulse_width
```

```
# format : frame 0 : split 0
```

```
**WARN: (TCLCMD-1470): Pins/ports are only supported with '-drv_violation_type' option
```

```
Check type : pulse_width
```

```
-----
```

```
-----
```

```
SEQUENTIAL CLOCK PULSE WIDTH
```

Litmus Text Command Reference
Timing Analysis Commands--report_constraint

Pin	Required Pulse Width	Actual Pulse Width	Slack	View Name
RM_25_TS_INST/RM_25_INST/CLK (high)	0.061	-0.788	-0.849	func_RCMAX
DGINST/out_reg[4]/CK (low)	2.017	1.514	-0.503	func_RCMAX
DGINST/out_reg[3]/CK (low)	2.017	1.514	-0.503	func_RCMAX
DGINST/out_reg[2]/CK (low)	2.017	1.519	-0.499	func_RCMAX
DGINST/out_reg[6]/CK (low)	2.017	1.519	-0.499	func_RCMAX

- The following command reports violations if the connection class attributes of the driving pin and receiving pin of a net are different:

```
tempus > report_constraint -connection_class -early -verbose -view scan_slow_RCMAX
```

```
# format : frame 0 : split 0
```

```
min_delay/hold
```

```
Path 1: VIOLATED (-1.094 ns) Hold Check with Pin TP_CORE/data_out_reg[3]/CK->SI
```

```
View: scan_slow_RCMAX
```

```
Group: m_clk
```

```
Startpoint: (R) scan_in2
```

```
Clock: (R) m_clk
```

```
Endpoint: (R) TP_CORE/data_out_reg[3]/SI
```

```
Clock: (R) m_clk
```

	Capture	Launch
Clock Edge:+	0.000	0.000
Src Latency:+	0.000	0.000
Net Latency:+	4.571 (P)	0.000 (I)
Arrival:=	4.571	0.000
Hold:+	-0.052	
Uncertainty:+	0.600	
Required Time:=	5.118	
Launch Clock:=	0.000	
Input Delay:+	4.000	
Data Path:+	0.024	
Slack:=	-1.094	

```
#-----
```

# Pin	Cell	Delay	Arrival	Edge
#		(ns)	(ns)	
#-----				
scan_in2	(arrival)	-	4.000	R
TP_CORE/data_out_reg[3]/SI	SDFFR_X1	0.024	4.024	R
#-----				

```
Check type : connection_class
```

**INFO: No connection_class information found for given parameters

- Consider a design having 3 sequential endpoints:
 - setup_pin: Having a setup check with slack equal to -9.000
 - recovery_pin: Having a recovery check with slack equal to -8.000
 - both_setup_recovery_pin: Having both setup check (slack = -6.000) and recovery check (slack = -7.000)

In this case, the `report_constraint -late -all_violators` command will display the following output:

```
max_delay/setup
-----
End Point           Slack           Cause
-----
setup_pin f         -9.000         VIOLATED
both_setup_recovery_pin -6.000         VIOLATED
-----
```

- The `report_constraint -check_type recovery -all_violators` command will display the following output:

```
Check type : recovery
-----
End Point           Slack           Cause
-----
recovery_pin r       -8.000         VIOLATED
both_setup_recovery_pin -7.000         VIOLATED
-----
```

- The `report_constraint -late -check_type recovery -all_violators` command displays the following output:

```
max_delay/setup
-----
End Point           Slack           Cause
-----
setup_pin f         -9.000         VIOLATED
recovery_pin r       -8.000         VIOLATED
both_setup_recovery_pin -7.000         VIOLATED
-----
```

- The `report_constraint` command displays the following output:

```
max_delay/setup
```

Litmus Text Command Reference
Timing Analysis Commands--report_constraint

```
-----
End Point          Slack          Cause
-----
out2 r             -4111.834          VIOLATED
-----
```

Check type : clock_period

No paths found

Check type : pulse_width

SEQUENTIAL CLOCK PULSE WIDTH

```
-----
Pin                Required        Actual Pulse      Slack
                  Pulse Width      Width
-----
dff2/CK (low)     0.260          -670.800          -671.060
-----
```

Check type : max_transition

Pin : sub1/dff1/D

max_transition : 4.500

Transition Time: 5114.060

slack: -5109.560 (VIOLATED)

Check type : max_capacitance

Pin : buf133/Y

max_capacitance: 0.000

-Capacitance : 0.423

slack : -0.423 (VIOLATED)

Check type : max_fanout

Pin : specialClkGate/o

max_fanout : 0.557

fanout : 1.000

slack : -0.443 (VIOLATED)

- The `report_constraint -all_violators` command displays the following output:

max_delay/setup

```
-----
```

End Point	Slack	Cause

out2 r	-4111.834	VIOLATED
dff3/D r	-4110.260	VIOLATED
lat2/D r	-3774.174	VIOLATED
dff2/D r	-3770.285	VIOLATED
out r	-3161.115	VIOLATED
and1/B r	-2464.880	VIOLATED
dff1/D r	-2098.868	VIOLATED
sub2/bufH1/dff1/D r	-2098.821	VIOLATED
sub2/bufH2/dff1/D r	-2028.843	VIOLATED
sub1/dff1/D r	-2028.843	VIOLATED
dff5/D r	-1279.155	VIOLATED

```
-----
```

Check type : clock_period

```
-----
```

No paths found

Check type : skew

```
-----
```

No paths found

Check type : pulse_width

```
-----
```

SEQUENTIAL CLOCK PULSE WIDTH

```
-----
```

Pin	Required Pulse Width	Actual Pulse Width	Slack

dff2/CK (low)	0.260	-670.800	-671.060
lat2/GN (low)	0.201	-670.800	-671.001
sub2/bufH1/dff1/CK (high)	0.167	-670.800	-670.968
dff1/CK (high)	0.167	-670.800	-670.968
sub2/bufH2/dff1/CK (high)	0.167	-670.800	-670.968
sub1/dff1/CK (high)	0.167	-670.800	-670.968
dff11/CK (high)	0.167	-670.800	-670.968


```
-----
Check type : max_transition
-----
```

Pin Name	Required	Actual	Slack
and_parallel_1/A	4.500	5114.060	-5109.560
buf4/A	4.500	5114.060	-5109.560
sub1/dff1/D	4.500	5114.060	-5109.560

```
Check type : max_capacitance
-----
```

Pin Name	Required	Actual	Slack
buf133/Y	0.000	0.423	-0.423
buf11/Y	0.000	0.105	-0.105
buf5/Y	0.000	0.105	-0.105
dff11/Q	0.000	0.105	-0.105

```
Check type : max_fanout
-----
```

Pin Name	Required	Actual	Slack
specialClkGate/o	0.557	1.000	-0.443

- The `report_constraint -all_violators -drv_violation_type min_capacitance` command reports the following:

```
-----
Pin Name    Required    Actual    Slack
-----
DIN         0.0400      0.0013    -0.0387
F2/Q        0.0400      0.0013    -0.0387
CLK1        0.0400      0.0051    -0.0349
-----
```

- The `report_constraint -drv_violation_type min_capacitance` command outputs the following:

```
Check type : min_capacitance
-----
```

```
Pin : F2/Q
```

```
Capacitance: 0.0013
```

```
- min_capacitance: 0.0400
-----
```

```
slack: -0.0387 ( VIOLATED )
```

- The `report_constraint -all_violators -drv_violation_type min_transition` command outputs the following:

```
-----
Pin Name      Required      Actual      Slack
-----
I0/A          0.0300        0.0001      -0.0299
I1/A          0.0300        0.0001      -0.0299
CLK0          0.0300        0.0001      -0.0299
CLK1          0.0300        0.0001      -0.0299
DIN           0.0300        0.0001      -0.0299
I4/S0         0.0300        0.0001      -0.0299
F1/DATA       0.0300        0.0001      -0.0299
SEL           0.0300        0.0001      -0.0299
```

- The `report_constraint -drv_violation_type min_transition` command outputs the following:

```
Check type : min_transition
-----
Pin : F1/DATA

Transition Time:0.0001
- min_transition :0.0300
-----
slack : -0.0299 ( VIOLATED )
```

- The `report_constraint -all_violators -drv_violation_type min_fanout` command outputs the following:

```
-----
Pin Name      Required      Actual      Slack
-----
F5/Q          2.0000        0.0000      -2.0000
CLK0          2.0000        1.0000      -1.0000
F3/Q          2.0000        1.0000      -1.0000
F2/Q          2.0000        1.0000      -1.0000
F1/Q          2.0000        1.0000      -1.0000
SEL           2.0000        1.0000      -1.0000
CLK1          2.0000        1.0000      -1.0000
```

- The `report_constraint -drv_violation_type min_fanout` command outputs the following:

```
Check type : min_fanout
-----
```

```
Pin : F5/Q
Fanout: 0.0000
- min_fanout: 2.0000
-----
slack : -2.0000 ( VIOLATED )
```

- The report_constraint -check_type clock_period command reports the following:

```
Check type : clock_period
-----
End Point          Slack      Cause
-----
ffo_ram/mm0/mm1/CLKB f  -0.027    VIOLATED
-----
```

- The following example illustrates how to restrict the number of DRV violations to display for each of the specified DRV checks:

```
report_constraint -drv_violation_type {max_transition max_capacitance} -drv_fields
{max_transition transition_actual transition_slack max_capacitance capacitance_actual
capacitance_slack max_slack net max_frequency_clock max_clock_frequency pin_type ref_cell
driver_pin pin} -all_violators -max_pins_per_drv 6
#####
# format : frame 0 : split 0
# Units: Time (1ns), Capacitance (1pf), Frequency (1MHz).
Check type : max_transition
-----
-----
Required      Actual      Transition   Worst      Net      Maximum      Frequency      Pin
Violating     Driver      Pin
Transition    Transition  Slack        Setup        Frequency      Type
Libcell
Slack        Clock
-----
0.002         0.120       -0.118       NA          clk1      CK1          100.000       CLOCK
DFFRX2        clk1        ff1/CK
0.002         0.120       -0.118       NA          clk2      CK2          200.000       CLOCK
DFFRX2        clk2        ff2/CK
0.002         0.059       -0.057       0.894       n4        CK1          100.000       DATA      BUF1
buf2/Y        buf2/Y
0.002         0.059       -0.057       4.362       n4        CK1          100.000       DATA
DFFRX2        buf2/Y      ff2/D
0.002         0.059       -0.057       0.894       n4        CK1          100.000       DATA      BUF1
buf2/Y        buf3/A
0.002         0.045       -0.043       3.840       n1        CK2          200.000       DATA      BUF1
buf0/Y        buf1/A
```

Litmus Text Command Reference
Timing Analysis Commands--report_constraint

Check type : max_capacitance

Required Violating Capacitance Libcell	Actual Driver Capacitance	Capacitance Pin Slack	Worst Setup Slack	Net	Maximum Frequency Clock	Frequency	Pin Type
---	---------------------------------	-----------------------------	-------------------------	-----	-------------------------------	-----------	-------------

0.002	0.004	-0.002	0.894	n4	CK1	100.000	DATA	BUFX1
buf2/Y	buf2/Y							
0.002	0.002	-0.000	3.840	n1	CK2	200.000	DATA	BUFX1
buf0/Y	buf0/Y							
0.002	0.002	-0.000	3.840	n2	CK2	200.000	DATA	BUFX1
buf1/Y	buf1/Y							
0.002	0.002	-0.000	0.894	n3	CK1	100.000	DATA	
DFFRX2	ff1/Q	ff1/Q						

- The following examples shows the pin-wise DRV report without the worst DRV per net:

```
set timing_report_constraint_format "max_transition transition_actual transition_slack
max_capacitance capacitance_actual capacitance_slack max_slack net max_frequency_clock
max_clock_frequency pin_type ref_cell driver_pin pin"
```

```
report_constraint -drv_violation_type {max_transition max_capacitance} -all_violators
```

```
#####
```

```
# format : frame 0 : split 0
```

```
# Units: Time (1ns), Capacitance (1pf), Frequency (1MHz).
```

```
Check type : max_transition
```

```
-----
```

Required Violating Transition	Actual Driver Transition	Transition Pin Slack	Worst Setup Slack	Net	Maximum Frequency Clock	Frequency	Pin Type	Libcell
-------------------------------------	--------------------------------	----------------------------	-------------------------	-----	-------------------------------	-----------	-------------	---------

0.002	0.120	-0.118	NA	clk1	CK1	100.000	CLOCK	DFFRX2
clk1	ff1/CK							
0.002	0.120	-0.118	NA	clk2	CK2	200.000	CLOCK	DFFRX2
clk2	ff2/CK							
0.002	0.059	-0.057	0.894	n4	CK1	100.000	DATA	BUFX1
buf2/Y	buf2/Y							
0.002	0.059	-0.057	4.362	n4	CK1	100.000	DATA	DFFRX2
buf2/Y	ff2/D							
0.002	0.059	-0.057	0.894	n4	CK1	100.000	DATA	BUFX1

Litmus Text Command Reference
Timing Analysis Commands--report_constraint

```

    buf2/Y    buf3/A
0.002        0.045    -0.043    3.840    n1    CK2    200.000    DATA    BUFX1
    buf0/Y    buf1/A
0.002        0.045    -0.043    3.840    n1    CK2    200.000    DATA    BUFX1
    buf0/Y    buf0/Y
0.002        0.044    -0.042    3.840    n2    CK2    200.000    DATA    BUFX1
    buf1/Y    buf1/Y
0.002        0.044    -0.042    3.840    n2    CK2    200.000    DATA    DFFRX2
    buf1/Y    ff1/D
0.002        0.038    -0.036    0.894    n3    CK1    100.000    DATA    BUFX1
    ff1/Q    buf2/A
0.002        0.038    -0.036    0.894    n3    CK1    100.000    DATA    DFFRX2
    ff1/Q    ff1/Q
0.002        0.031    -0.029    1.208    out2    CK2    200.000    DATA    DFFRX2
    ff2/Q    ff2/Q
0.002        0.030    -0.028    0.894    out1    CK1    100.000    DATA    BUFX1
    buf3/Y    buf3/Y

```

Check type : max_capacitance

```

-----
Required      Actual      Capacitance  Worst   Net   Maximum  Frequency  Pin   Violating
Driver   Pin
Capacitance Capacitance Slack           Setup           Frequency
                                Slack           Clock
-----
0.002        0.004        -0.002        0.894    n4    CK1    100.000    DATA    BUFX1
    buf2/Y    buf2/Y
0.002        0.002        -0.000        3.840    n1    CK2    200.000    DATA    BUFX1
    buf0/Y    buf0/Y
0.002        0.002        -0.000        3.840    n2    CK2    200.000    DATA    BUFX1
    buf1/Y    buf1/Y
0.002        0.002        -0.000        0.894    n3    CK1    100.000    DATA    DFFRX2
    ff1/Q    ff1/Q

```

- The following command example shows pins per net with the worst DRV slack:

```

report_constraint -drv_violation_type {max_transition max_capacitance} -all_violators -
worst_drv_per_net -max_pins_per_drv 5

```

```

#####
# format : frame 0 : split 0
# Units: Time (lns), Capacitance (lpf), Frequency (lMHz).
Check type : max_transition
-----
-----

```

```

Required      Actual      Transition  Worst   Net   Maximum  Frequency  Pin
Violating   Driver   Pin

```

Litmus Text Command Reference
Timing Analysis Commands--report_constraint

Transition	Transition	Slack	Setup Slack		Frequency Clock		Type	Libcell

0.002	0.120	-0.118	NA	clk1	CK1	100.000	CLOCK	DFFRX2
clk1	ff1/CK							
0.002	0.120	-0.118	NA	clk2	CK2	200.000	CLOCK	DFFRX2
clk2	ff2/CK							
0.002	0.059	-0.057	0.894	n4	CK1	100.000	DATA	BUFX1
buf2/Y	buf2/Y							
0.002	0.045	-0.043	3.840	n1	CK2	200.000	DATA	BUFX1
buf0/Y	buf1/A							
0.002	0.044	-0.042	3.840	n2	CK2	200.000	DATA	BUFX1
buf1/Y	buf1/Y							

Check type : max_capacitance

Required Driver	Actual Pin	Capacitance	Worst	Net	Maximum	Frequency	Pin	Violating
Capacitance	Capacitance	Slack	Setup Slack		Frequency Clock		Type	Libcell

0.002	0.004	-0.002	0.894	n4	CK1	100.000	DATA	BUFX1
buf2/Y	buf2/Y							
0.002	0.002	-0.000	3.840	n1	CK2	200.000	DATA	BUFX1
buf0/Y	buf0/Y							
0.002	0.002	-0.000	3.840	n2	CK2	200.000	DATA	BUFX1
buf1/Y	buf1/Y							

- The following command reports the hierarchical nets associated with the violating pins:

```
tempus > report_constraint -drv_violation_type max_transition -drv_fields {pin net hnet}
```

Check type : max_transition

+-----+		
Pin	Net	Hierarchical
		Net
-----+-----+-----		
data_bidi2	data_bidi2	data_bidi2
+-----+		

Related Commands

- `set_min_fanout`
- `set_min_fanout`
- `set_min_transition`
- `set_max_transition`
- `set_min_capacitance`
- `set_max_capacitance`
- `report_timing`

report_design

```
report_design  
[-help]  
[-view viewName]  
[{> | >>} filename | -tcl_list]  
[-early | -late]
```

Reports the minimum and/or maximum operating conditions and active design rules for the current design.

Both minimum and maximum operating conditions are available for on-chip variation analysis mode.

When simultaneous setup hold analysis mode is set to `on`, the minimum or maximum conditions can be reported using the `-early/-late` parameters.

Parameters

<code>-help</code>	Prints out the command usage.
<code>{> >>} filename</code>	When specified with <code>></code>, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with <code>>></code>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The <code>filename</code> parameter must be the last argument in the list. The <code>filename</code> and <code>-tcl_list</code> parameters are mutually exclusive; you cannot specify them together. Note: To write a compressed report, add the <code>.gz</code> extension to the file name. <i>Default:</i> Report is displayed on standard output without being saved.

<code>-early -late</code>	The <code>-early</code> parameter reports minimum operating condition in <code>best_case_worst_case</code> or <code>ocv</code> analysis mode (<code>timing_analysis_type</code> attribute). The <code>-late</code> parameter reports maximum operating condition. The <code>-late</code> and <code>-early</code> parameters are mutually exclusive.
<code>-tcl_list</code>	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. Note: The <code>-tcl_list</code> and <code>filename</code> parameters are mutually exclusive; you cannot specify these parameters together.
<code>-view viewName</code>	Reports view specific operating conditions.

Examples

- The following command reports the operating conditions related to the view, `scan_slow_RCMAX`, along with the design rules:

```
report_design -view scan_slow_RCMAX
```

```
View : scan_slow_RCMAX
```

Design Property	Value	

Design Name	dtmf_recvr_core	

	MAX	MIN
Operating Condition Name	TypTyp_0.8V_25C	TypTyp_0.8V_25C
Process	1.000	1.000
Voltage	0.800	0.800
Temperature	25.000	25.000
Tree Type	balanced	balanced

Design Rule	Value

Max Transition	NA
Min Transition	NA
Max Capacitance	NA
Min Capacitance	NA
Max Fanout	NA
Min Fanout	NA

Related Commands

- [report_timing](#)
- [write_db](#)

report_fanin

```
report_fanin
-to pin_port_net_list
[-trace_through {case_disable | user_disable | all}]
[-pin_levels numOfPinLevels]
[-view viewName]
[-nosplit]
[{> | >> } filename]
```

Allows a cone traversal that is not tied to the timing graph, that is, not blocked by `set_disable_timing` and case analysis. By default, the trace matches the timing view and the reports generated using the `all_fanin` and `all_fanout` commands. The reports indicate whether disabled or case analysis arcs were traversed.

Parameters

<code>> >> filename</code>	When specified with <code>></code>, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with <code>>></code>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file.
<code>-nosplit</code>	Specifies that reports with long names will not be split into multiple lines.
<code>-pin_levels numOfPinLevels</code>	Specifies the desired depth of the output.
<code>-trace_through {case_disable user_disable all}</code>	Allows traversal through disabled arcs. You can specify one of the following options: • <code>case_disable</code>: Arcs which are disabled because of <code>set_case_analysis</code> are traversed but because of <code>set_disable_timing</code> are not traversed • <code>user_disable</code>: Arcs disabled because of <code>set_disable_timing</code> are traversed but because of <code>set_case_analysis</code> are not traversed • <code>all</code>: Trace through all arcs
<code>-to pin_port_net_list</code>	Specifies a list or collection of pins, ports, or nets. A mixed list or heterogeneous collection is also allowed.

-view *viewName*

Specifies disabled arcs in the specified view.

Examples

- The following command generates a report of the specified collection of pins along with the depth of the output for the specified view:

```
litmus > report_fanin -to FE_C674_ts_4 -pin_levels 3 -view scan_fast_RCMAx
```

```
Fanin Network for View: scan_fast_RCMAx
```

```
Fanin Network for Pin: FE_C674_ts_4/A
```

Source Pin	Sink Pin	Cell Type	Timing Sense	Arc	Case
				Disabled	Analysis

RE_INST/FE_O6_t_4/Z	FE_C674_ts_4	(net)	-	-	-

RE_INST/FE_OFC106_4/A	RE_INST/FE_O6_t_4/Z	BUF_X8	positive_unate	-	-

RE_INST/g6253/Z	RE_INST/FE_OFC106_4/A	(net)			

- The `report_fanin -to h0/A` command shows the following output:

```
litmus > report_fanin -to h0/A
```

```
Fanin Network for Pin: h0/A
```

Source Pin	Sink Pin	Cell Type	Timing Sense	Arc	Case
				Disabled	Analysis

u1/Y	h0/A	(net)	-	-	-

u1/A	u1/Y	BUFX2	positive_unate	-	-

in	u1/A	(net)	-	-	-

Related Commands

- `set_disable_timing`
- `set_case_analysis`
- `all_fanin`
- `all_fanout`

report_fanout

```
report_fanout
{-from pin_port_net_list | -clock_tree}
[-trace_through {case_disable | user_disable | all}]
[-pin_levels numOfPinLevels]
[-view viewName]
[-nosplit]
[{> | >> } filename]
```

Allows a cone traversal that is not tied to the timing graph, that is, not blocked by `set_disable_timing` and case analysis. By default, the trace matches the timing view and the reports generated using the `all_fanin` and `all_fanout` commands. The reports indicate whether disabled or case analysis arcs were traversed.

Parameters

<code>> >> filename</code>	When specified with <code>></code>, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with <code>>></code>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file.
<code>-clock_tree</code>	Reports the fanout of the source pin, one after the other for all the clocks in the design.
<code>-nosplit</code>	Specifies that reports with long names will not be split into multiple lines.
<code>-pin_levels numOfPinLevels</code>	Specifies the desired depth of the output.

<code>-trace_through {case_disable user_disable all}</code>	Allows traversal through disabled arcs. You can specify one of the following options: <code>case_disable</code>: Arcs which are disabled because of <code>set_case_analysis</code> are traversed but because of <code>set_disable_timing</code> are not traversed. <code>user_disable</code>: Arcs disabled because of <code>set_disable_timing</code> are traversed but because of <code>set_case_analysis</code> are not traversed. <code>all</code>: Trace through all the arcs.
<code>-from pin_port_net_list</code>	Specifies a list or collection of pins, ports, or nets. A mixed list or heterogeneous collection is also allowed.
<code>-view viewName</code>	Specifies disabled arcs in the specified view.

Examples

- The following command reports details of a collection of pins at the desired depth of pin levels for the specified view:

```
litmus > report_fanout -from FE_C674_ts_4/A -pin_levels 4 -view scan_fast_RCMIN
```

Fanout Network for View: scan_fast_RCMIN

Fanout Network for Pin: FE_C674_ts_4/A

Source Pin	Sink Pin	Cell Type	Timing Sense	Arc Disabled	Case Analysis
FE_C674_ts_4/A	FE_C674_ts_4/Z	BUF_X32	positive_unate	-	-
FE_C674_ts_4/Z	MX_INST/p3A/B	(net)	-	-	-
FE_C674_ts_4/Z	FE_O402_ts_4/A	(net)	-	-	-
FE_C674_ts_4/Z	CS_INST/p21A/A3	(net)	-	-	-
MX_INST/p3A/B	MX_INST/p3A/Z	MUX2_X2	positive_unate	-	-
FE_O402_ts_4/A	FE_O402_ts_4/Z	BUF_X16	positive_unate	-	-
CS_INST/p21A/A3	TSP_INST/p2A/ZN	NOR4_X1	negative_unate	-	-
MX_INST/p3A/Z	FE_OFC105_n_94/A	(net)	-	-	-
FE_O402_ts_4/Z	M_128x16_ST/A[4]	(net)	-	-	-
TSP_INST/p2A/ZN	TST_CL/p3/A	(net)	-	-	-

- The following command reports details of a collection of pins at the desired depth of pin levels for disabled arcs:

```
litmus > report_fanout -from FE_C674_ts_4/A -pin_levels 2 -trace_through case_disable
```

Fanout Network for View: func_slow_RCMAX

Fanout Network for Pin: FE_C674_ts_4/A

Source Pin	Sink Pin	Cell Type	Timing Sense	Arc Disabled	Case Analysis
FE_C674_ts_4/A	FE_C674_ts_4/Z	BUF_X32	positive_unate	-	-
FE_C674_ts_4/Z	MX_INST/p3A/B	(net)	-	-	-
FE_C674_ts_4/Z	FE_O402_ts_4/A	(net)	-	-	-
FE_C674_ts_4/Z	CS_IST/p21A/A3	(net)	-	-	-

- Suppose arc seg1/u1 is disabled due to following constraint:

```
litmus> set_disable_timing -from A -to Y seg1/u1
```

```
litmus> report_fanout -from IN1 (command will show the following output):
```

Fanout Network for Port: IN1

Source Pin	Sink Pin	Cell Type	Timing Sense	Arc Disabled	Case Analysis
IN1	seg1/u1/A	(net)	-	-	-

- The following command will show the output below:

```
litmus> report_fanout -from IN1 -trace_through user_disable
```

Fanout Network for Port: IN1

Source Pin	Sink Pin	Cell Type	Timing Sense	Arc Disabled	Case Analysis
IN1	seg1/u1/A	(net)	-	-	-
seg1/u1/A	seg1/u1/Y	CLKBUF	positive_unate	Y	-
seg1/u1/Y	seg1/u2/A	(net)	-	-	-

Related Commands

- `set_disable_timing`
- `set_case_analysis`
- `all_fanin`
- `all_fanout`

report_instance_library

```
report_instance_library
[-out_file fileName]
[-instance instanceName]
[-view viewName | -delay_corner delayCornerName]
[-power_domain domainName]
[-early | -late]
```

Reports library related information for an instance(s). The report prints the following information about an instance:

- Instance name
- Cell type
- View name or delay corner
- Power domain name
- Library/Libset (early/late)
- Library file (early/late)
- Operating conditions
- Binding Status : Prints out the status for the matched voltage library available. If the user setting is `True`, then the binding status is not displayed.

Parameters

<code>-delay_corner delayCornerName</code>	Specifies the delay corner from which an instance library is to be selected. If no delay corner is specified, then all the instance libraries will be printed.
<code>-early -late</code>	Specifies the library set (early/late) from which a library instance is to be printed. If either of these options are not specified then both the early and late library sets are used.
<code>-out_file fileName</code>	Specifies the name of the output file. If no file is specified, the report will be printed on standard output file.

<code>-instance <i>instanceName</i></code>	Specifies the instance(s) for which the report is to be printed. If no instance is provided the report will print information of all the instances. The value of instance can be a list of regular expressions.
<code>-power_domain <i>domainName</i></code>	Specifies the power domain from which instance(s) are to be selected. Note: This parameter should be specified with <code>-delay_corner</code> OR <code>-view</code> .
<code>-view <i>viewName</i></code>	Specifies the view from which instance library is to be selected.

Examples

- The following command displays the below information about instance `LIN_CONV_INST/F6A` for the specified view:

```
report_instance_library -instance LIN_CONV_INST/F6A -early -view scan_slow_RCMAX
```

```
Instance           : LIN_CONV_INST/F6A
Cell               : INV_X2
Analysis View      : scan_slow_RCMAX
Power Domain       : Default
Library/Libset(early) : F_lib_v1.0/libset_slow
Library File(early)  : /libs/F_lib_v1.0_worst.lib
Binding Status(early) : All PG Pins Voltage Match
Op Cond(early)       : P->1.000000, V->0.800000, T->25.000000 (TypTyp_0.8V_25C)
Cell Type          : gateCell
```

- The following command displays the below information about instance `FE_C674` for the specified delay corner:

```
report_instance_library -delay_corner delay_corner_slow_RCMAX -late
```

```
Instance           : FE_C674
Cell               : BUF_X32
Delay Corner        : delay_corner_slow_RCMAX
PowerDomain         : Default
Library/Libset(late) : F_lib_v1.0/libset_fast
Library File(late)   : /libs/F_lib_v1.0_typic
Binding Status(late) : All PG Pins Voltage Match
Op Cond(late)        : P->1.000000, V->0.800000, T->25.000000 (TypTyp_0.8V_25C)
Cell Type           : gateCell
```

Related Commands

- [check_timing_library_consistency](#)

report_lib_arcs

```
report_lib_arcs
[-help]
-arc <string>
[-fall]
[-nldm]
[-quiet]
[-rise]
[-lvf [-early]]
[-lvf [-late]]
[-moments [-mean_shift]]
[-moments [-std_dev]]
[-moments [-skewness]]
[-input_net_transition <float> | -constrained_pin_transition <float>]
[-input_net_transition <float> | -related_pin_transition <float>]
[-total_output_net_capacitance <float> | -constrained_pin_transition <float>]
[-total_output_net_capacitance <float> | -related_pin_transition <float>]
[-constrained_pin_transition <float> | -delay ] [-constrained_pin_transition <float> | -transition
]
[-related_pin_transition <float> | -delay ]
[-related_pin_transition <float> | -transition ]
```

Reports the library arc data. The command also returns the data in a Tcl object, which can be used in scripts. This command uses a collection of arcs as input, which can be queried using the `get_lib_arcs` command. The units for the input and output data can be obtained using the `get_time_unit` and `get_capacitance_unit` commands.

Parameters

-help	Prints out the command usage.
-arc <string>	Specifies the arc(s) for which the report is to be printed.
-constrained_pin_transition <float>	Specifies the constrained pin slew (valid for constraint arcs) at which the parameters need to be reported. The slew units must be specified using the <code>get_time_unit</code> command.
-delay	Specifies if delay is to be reported.

-early	Specifies if early LVF data is to be reported. This parameter can be used only in conjunction with the <code>-lvf</code> .
-fall	Specifies if details of fall tables are to be reported.
-input_net_transition <float>	Specifies the slew at which the parameters need to be reported. The slew units must be specified using the <code>get_time_unit</code> command.
-late	Specifies if late LVF data is to be reported. This parameter can be used only in conjunction with the <code>-lvf</code> .
-lvf	Specifies if LVF data is to be reported.
-mean_shift	Specifies if the mean shift data is to be reported. This parameter can be used only in conjunction with the <code>-moments</code> .
-moments	Specifies if moments data is to be reported.
-nldm	Specifies if NLDM data is to be reported.
-quiet	Disables output to be displayed on the screen.
-related_pin_transition <float>	Specifies the related pin slew (valid for constraint arcs) at which the parameters need to be reported. The slew units must be specified using the <code>get_time_unit</code> command.
-rise	Specifies if details of rise tables are to be reported.
-skewness	Specifies if skewness data is to be reported. This parameter can be used only in conjunction with the <code>-moments</code> .
-std_dev	Specifies if standard deviation data is to be reported. This parameter can be used only in conjunction with the <code>-moments</code> .
- total_output_net_capacitance <float>	Specifies the load at which the parameters need to be reported. The load must be specified using the <code>get_capacitance_unit</code> command.
-transition	Specifies if the output slew is to be reported.

Examples

- The following command reports the library arc data containing mean shift rise tables:

```
tempus > report_lib_arcs -arc [get_lib_arcs -from cell_w/DFE/CK -to cell_w/DFE/Q] -
input_net_transition 0.08 /
-total_output_net_capacitance 0.055 -mean_shift -rise -delay -moments -nldm
```

Units of Report:- Time: '1000ps', Capacitance: '1000fF'.
input_net_transition:0.080000, total_output_net_capacitance:0.055000

```
-----
| Index | Timing Arc | Value(s) | | | |
|---|---|---|---|---|---|
|      | Cell      | Source/Sink Pin | Timing Sense | Timing Type | cell_rise |
| -----|
| 1     | cell_w/DFE | CK -> Q        | non_unate   | rising_edge | 0.258444  |
| -----|
{{cell_rise 0.258444279432} {ocv_mean_shift_cell_rise NA}}
```

- The following command reports the library arc data containing fall tables for moments data:

```
tempus > report_lib_arcs -arc [get_lib_arcs -from cell_w/BUF/A -to cell_w/BUF/Y] /
-input_net_transition 0.055 -total_output_net_capacitance 0.15 -fall -delay -moments -nldm
```

Units of Report:- Time: '1000ps', Capacitance: '1000fF'.
input_net_transition:0.055000, total_output_net_capacitance:0.150000

```
-----
| Index | Timing Arc | Value(s) | | | |
|---|---|---|---|---|---|
|      | Cell      | Source/Sink Pin | Timing Sense | Timing Type | cell_fall |
| -----|
| 1     | cell_w/BUF | A -> Y        | positive_unate | combinational | 0.129744  |
| -----|
-----
{{cell_fall 0.129} {ocv_std_dev_cell_fall NA} {ocv_mean_shift_cell_fall NA}
{ocv_skewness_cell_fall NA}}
```

- The following command reports the library arc data containing rise tables data with output slew:

```
tempus > report_lib_arcs -arc [get_lib_arcs -from cell_w/DFE/CK -to cell_w/DFE/Q] -
input_net_transition 0.08 /
-total_output_net_capacitance 0.055 -mean_shift -rise -delay -moments -nldm -transition
```

Units of Report:- Time: '1000ps', Capacitance: '1000fF'.
input_net_transition:0.080000, total_output_net_capacitance:0.055000

```
-----
| Index | Timing Arc | Value(s) |
| -----|
```

```

-----
|           | Cell           | Source/Sink Pin | Timing Sense | Timing Type | rise_transition |
cell_rise |
-----
| 1         | cell_w/DFP      | CK -> Q         | non_unate    | rising_edge | 0.21692         |
0.258444   |
-----
{{cell_rise 0.258} {rise_transition 0.216} {ocv_mean_shift_cell_rise NA}
{ocv_mean_shift_rise_transition NA}}

```

This command also returns the data in a Tcl object that can be parsed and used in scripts. A 3-Dimensional Tcl List will return the following information:

```
{{TableName Value} {TableName Value}} {{TableName Value} {TableName Value}}
```

Here, the first list (left column) indicates the Table-Value pair in one arc. The second list indicates the various tables present per arc. And the third list (last column) indicates the list of total arcs in the collection.

- The following command reports the slew and capacitance load:

```

tempus > set items [report_lib_arcs -arc [ get_lib_arcs -from sc_cmoslp/A022X/clk -to
sc_cmoslp/A022X /out1] /
-input_net_transition 0.003 -total_output_net_capacitance 0.021]

```

Units of Report:- Time: '1000ps', Capacitance: '1000fF'.
input_net_transition:0.003, total_output_net_capacitance:0.021

```

-----
| Index | Timing Arc
| Value(s) |
|-----|
|           | Cell           | Source/Sink Pin | Timing Sense | Timing Type | rise_transition |
| fall_transition | cell_rise | cell_fall |
-----
| 1       | sc_cmoslp/A022X | clk -> out1     | positive_unate | combinational | 0.20**
| 0.30**   | 1.00**   | 1.20**   |
-----
| 2       | sc_cmoslp/A022X | clk -> out1     | negative_unate | combinational | 0.20**
| 0.30**   | 1.00**   | 1.20**   |
-----

```



```
** --> Scalar Value{{cell_rise 0.999} {rise_transition 0.199} {cell_fall 1.199}
{fall_transition 0.299}} {{cell_rise 0.999} {rise_transition 0.199} {cell_fall 1.199}
{fall_transition 0.299}}

tempus 5 > puts $items
{{cell_rise 0.999} {rise_transition 0.199} {cell_fall 1.199} {fall_transition 0.299}}
{{cell_rise 0.999} {rise_transition 0.199} {cell_fall 1.199} {fall_transition 0.299}}
```

Related Commands

- `get_lib_arcs`
- `get_time_unit`
- `get_capacitance_unit`

report_mode

```
report_mode
[-help]
inst_list
[-view viewName]
```

Reports the status of library modes associated with a specified instance. The report includes the status of each mode and its condition, for each mode group defined in the timing library.

Parameters

<code>-help</code>	Prints out the command usage.
<code>inst_list</code>	Generates a report for the specified instances.
<code>-view viewName</code>	Reports the status of library modes in the specified active analysis view. Default: Reports the status of library modes in all active analysis views.

Examples

- The following command activates the `tst_mode_disable` library mode for instances of `i_ram`:

```
litmus> set_mode tst_mode_disable [get_cells i_ram]
```

- The following command sets the active setup and hold analysis views:

```
litmus> set_analysis_views -setup {v1 v2} -hold {v3}
```

- The following command reports the status of all modes associated with `m4k_top`:

```
litmus> report_mode m4k_top

-----
Mode Group tst_mode of m4k_top
-----
Mode Name           Status    Condition    View Name
-----
test_mode_enable    ACTIVE    TST_BYPASS    View1
test_mode_disable    INACTIVE  !(TST_BYPASS) View2
-----
```

- The following command reports the status of all modes in the analysis view `View1`:

```
litmus> report_mode m4k_top -view View1

-----
Mode Group tst_mode of m4k_top
```

```
-----  
Mode Name      Status  Condition  View Name  
-----  
test_mode_enable ACTIVE  TST_BYPASS View1  
-----
```

Related Commands

- `set_analysis_view`
- `set_mode`

report_nets

```
report_nets
[-help]
[-min_fanout int]
[-max_fanout int]
{-net list_of_net_name_or_id | -pin list_of_pin_name_or_id}
[-hier]
[-tcl_list]
[> | >>] filename]
```

Reports the net information on the current module. The information includes the net name, the number of source pins, sink pins and bidirectional pins on the net, wire capacitance, and total capacitance.

Electrical information such as pin capacitance, and wire capacitance with which the net is associated are also included.

Use the `-pin` option to report the net that is connected to the specified pin (or port). Both `-net` and `-pin` cannot be specified at the same time. The net (or pin) names can be hierarchical names that are unique relative to the top cell.

This command only searches for the defined nets and does not modify any object in database.

Parameters

<code>-help</code>	Prints out the command usage.
<code>{> >>} filename</code>	When specified with <code>></code>, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with <code>>></code>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The <code>filename</code> parameter must be the last argument in the list. The <code>filename</code> and <code>-tcl_list</code> parameters are mutually exclusive; you cannot specify them together. Note: To write a compressed report, add the <code>.gz</code> extension to the file name. <i>Default:</i> Report is displayed on standard output without being saved.

<code>-hier</code>	Reports all of the nets in the design hierarchy. Without this argument, only the nets in the current level of hierarchy are reported.
<code>-max_fanout int</code>	Reports the net whose number of fanouts is less than or equal to the maximum number specified by <i>integer</i> .
<code>-min_fanout int</code>	Reports the net whose number of fanouts is greater than or equal to the minimum number specified by <i>integer</i> .
<code>-net list_of_net_name_or_id</code>	Reports the set of nets on the list of net names. This argument cannot be specified at the same time as <code>-pin</code> . If neither <code>-net</code> or <code>-pin</code> is specified, all nets are listed by default. Note: This argument also reports the capacitance of the net and the input pins.
<code>-pin list_of_pin_name_or_id</code>	Reports the set of nets to which the pins on the list <i>list_of_pin_name_or_id</i> are connected. This argument cannot be specified at the same time as <code>-net</code> .
<code>-tcl_list</code>	Produces the report in Tcl list format instead of a tabular format. This is useful for extracting information from the report in a Tcl program.

Examples

- The following command reports details of the specified set of nets:

```
tempus > report_nets -net FE_data_out_1
```

```
Net Name           : FE_data_out_1
Number of Sources  : 1
Number of Sinks    : 1
Number of Bidis    : 0
Source of parasitics : Detailed Spef
Net Capacitance    : 0.000
Total Capacitance  : 0.001
```

```
-----
-----
Source
-----
-----
Pin                                     R      Driver Load      Slew
```

Litmus Text Command Reference
Timing Analysis Commands--report_nets

Phase	Name	Dir	Cell	Cap	Model	Model	Ctotal	Rise	Fall
	FE33_data_out_1/Z	OUT	BUF_X16	0.000	Unknown	CTOTAL	1.000	0.072	0.069
	clk(D) (P) (slow_RCMAX)								
	FE33_data_out_1/Z	OUT	BUF_X16	0.000	Unknown	CTOTAL	1.000	0.072	0.069
	clk(D) (P) (fast_RCMAX)								

Sinks for pin FE33_data_out_1/Z

Pin		R	Delay	Slew					
Name		Dir	Cell	Cap	Parameters	Rise	Fall	Rise	Fall
Phase									
FE_C583_data_out_1/A		IN	BUF_X32	0.000	-	0.000	0.000	0.072	0.069
clk(D) (P) (slow_RCMAX)									
FE_C583_data_out_1/A		IN	BUF_X32	0.000	-	0.000	0.000	0.072	0.069
clk(D) (P) (fast_RCMAX)									

- The following command reports nets whose number of fanouts is less than or equal to the specified maximum number 20:

```
tempus > report_nets -pin FE_F5_datain_7/A -max_fanout 20
```

```
Net Name           : ds_datain[7]
Number of Sources   : 1
Number of Sinks     : 1
Number of Bidis     : 0
Source of parasitics : Detailed Spef
Net Capacitance     : 0.000
Total Capacitance   : 0.001
```

Source

Litmus Text Command Reference
Timing Analysis Commands--report_nets

Pin					R	Driver Load		
Slew								

Name	Dir	Cell	Cap	Model	Model	Ctotal	Rise	Fall
Phase								

DATA_MUX_INST/F6A	OUT	INV_X8	0.000	Unknown	CTOTAL	1.000	0.190	0.365
clk(D) (P) (slow_RCMAX)								

Sinks for pin DATA_MUX_INST/F6A								

Pin					R	Delay		Slew

Name	Dir	Cell	Cap	Parameters	Rise	Fall	Rise	Fall
Phase								

FE_F5_datain_7/A	IN	BUF_X16	0.000	-	0.000	0.000	0.190	0.365
clk(D) (P) (slow_RCMAX)								
FE_F5_datain_7/A	IN	BUF_X16	0.000	-	0.000	0.000	0.190	0.365
clk(D) (P) (fast_RCMAX)								

- Control the `report_nets` table fields, such as setting the minimum and maximum column width using the `set_table_style` command. The table names are `report_nets_source_table`, `report_nets_sink_table`, `report_nets_summary_table`, `report_nets`, and `report_nets_header`.
- The following command sets an indentation of six columns for the table and disables printing of the first column:

```
set_table_style -name report_nets_source_table -indent 6 -max_widths {0,}
report_nets
```
- The format of the `report_nets` report is the same for all the examples, as shown in the first example. The following command reports information about net named `in`, as shown in the following example:

```
report_nets -net n5
```

```
Net Name : n5
```

```
Number of Sources : 1
Number of Sinks : 1
Number of Bidis : 0
Source of parasitics : Wireload model(.lib None Generated)
Net Capacitance : 0.000
Total Capacitance : 0.011
```

Source

Pin R Driver Load Slew

Name	Dir	Cell	Cap	Model	Model	Ctotal	Rise	Fall	Phase
u5/Q	OUT	DFF	0.000	Unknown	CTOTAL	1.000	0.003	0.003	CLK_W_4_gen2(D) (N)
u5/Q	OUT	DFF	0.000	Unknown	CTOTAL	1.000	0.003	0.003	CLK_W_4_gen1_inv(D) (P)

Sinks for pin u5/Q

Pin R Delay Slew

Name	Dir	Cell	Cap	Parameters	Rise	Fall	Rise	Fall	Phase
u7/A	IN	BUF	0.011	-	0.000	0.000	0.003	0.003	CLK_W_4_gen2(D) (N)
u7/A	IN	BUF	0.011	-	0.000	0.000	0.003	0.003	CLK_W_4_gen1_inv(D) (P)

- The following command reports the net to which the pin `interesting_pin` is connected:

```
report_nets -pin interesting_pin
```

- The following command reports all the nets in the current module:

```
report_nets -net *
```

- The following command reports all nets with a fanout greater than 16:

```
report_nets -net * -min_fanout 16
```

Related Commands

- [report_path_exceptions](#)
- [report_timing](#)

report_path_exceptions

```
report_path_exceptions  
[-help]  
[-all]  
[-early | -late]  
[-ignored]  
[-view view_name]  
[{> | >>} filename | -tcl_list]
```

Generates a report about path exceptions specified using the `set_false_path`, `set_min_delay`, `set_max_delay` and `set_multicycle_path` commands.

By default, the `report_path_exceptions` command only reports activated path exceptions. To report only the path exceptions that have been ignored by timing analysis, specify the `-ignored` parameter.

If more than one path exception is applied on a timing path, multiple path exceptions that match a given path are prioritized. The adjustment on the path honors the path exception with the highest priority.

When the software is in multi-mode multi-corner timing analysis mode, the report generates path exception information for every active analysis view.

The `report_path_exceptions` command output also includes 2 columns that show the file name and line number. The report output shows the exact file name and line number of the applied/ignored constraints. The compressed constraints will be decompressed first and then shown as individual constraints as written in the SDC file.

Parameters

-help	Prints out command usage.
-all	Reports all the applied and ignored path exceptions.

<pre>{> >>} filename</pre>	When specified with <code>></code>, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with <code>>></code>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The <code>filename</code> parameter must be the last argument in the list. The <code>filename</code> and <code>-tcl_list</code> parameters are mutually exclusive; you cannot specify them together. Note: To write a compressed report, add the <code>.gz</code> extension to the file name. <i>Default:</i> Report is displayed on standard output without being saved.
<pre>-early -late</pre>	Generates the path exception report for either early paths or late paths. <i>Default:</i> Generates a report based on the current timing analysis mode.
<pre>-ignored</pre>	Reports only the path exceptions that have been ignored by timing analysis. This parameter can be used for debugging constraints that may be ineffective or incorrectly entered. A path exception may be ineffective if it is covered by, or overridden by, another path exception. For example: <pre>set_false_path -to OUT set_multicycle_path -to OUT 2</pre> In the above situation, the <code>set_multicycle_path</code> command would be ignored because the false path has a higher priority.
<pre>-tcl_list</pre>	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. The <code>-tcl_list</code> and <code>filename</code> parameters are mutually exclusive; you cannot specify them together.

```
-view view_name
```

Generates a path exception report for the specified analysis view only. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode.

Examples

- The following command generates an exception report of late paths for the specified view:

```
litmus > report_path_exceptions -view scan_slow_RCMAX -late
```

From	To	Late	View Name
scan_se		add 1	scan_slow_RCMAX
tst_mode	refclk	false	scan_slow_RCMAX
SPI/bit_reg[3]/RN	SPI/bit_reg[1]/RN	false	scan_slow_RCMAX

- The following commands shows that a false path has priority over cycle addition on the same path:

```
litmus > set_false_path -from seg1/u3/q -to seg1/u9/d
litmus > set_multicycle_path -from seg1/u3/Q -to seg1/u9/D 2
litmus > report_path_exceptions -ignored
```

From	To	Early	Late
seg1/u3/Q	seg1/u9/D	false ignored	false ignored
seg1/u3/Q	seg1/u9/D	-	add 1 ignored
seg1/u3/Q	seg1/u9/D	add 0 ignored	-

The following command shows how edges are handled:

```
litmus > set_false_path -rise_from {IN1} -through {OUT1}
litmus > set_false_path -rise_from {IN1 IN2} -fall_to {OUT1}
litmus > set_false_path -rise_from {IN1}
litmus > report_path_exceptions -ignored
```

From	Through	To	Early	Late
^ IN1 IN2	-	v OUT1	false ignored	false ignored
^ IN1	OUT1	-	false ignored	false ignored

```
litmus > report_path_exceptions -early
```

From	Through	To	Early
^ IN1	-	-	false

```
litmus > report_path_exceptions -late
-----
From      Through  To      Late
-----
^ IN1     -          -        false
-----
```

When the software is in multi-mode multi-corner timing analysis mode, the software generates path exception information for every active analysis view:

```
litmus > set_analysis_view -setup {setup1 setup2} -hold {hold1 hold2}
litmus > report_path_exceptions
```

```
-----
From          Early   Late      View
              Name
-----
test_mode reset  false           setup2
test_mode reset  false           setup1
test_mode reset           false        hold1
test_mode reset           false        hold2
-----
```

You can specify the `-view` parameter to generate a report only for a specific active analysis view:

```
litmus > report_path_exceptions -view setup1
```

```
-----
From          Early   Late      View
              Name
-----
test_mode reset  false        setup1
-----
```

Related Commands

- [report_timing](#)

report_path_groups

```
report_path_groups  
[-help]  
[-name group_name]  
[{> | >>} filename | -tcl_list]
```

Generates a report that lists the names of all path groups with the corresponding paths.

Parameters

-help	Prints out the command usage.
{> >>} filename	When specified with >, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with >>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The <i>filename</i> parameter must be the last argument in the list. The <i>filename</i> and <i>-tcl_list</i> parameters are mutually exclusive; you cannot specify them together. Note: To write a compressed report, add the <i>.gz</i> extension to the file name. <i>Default:</i> Report is displayed on standard output without being saved.
-name group_name	Limits the report to the specified path group. <i>Default:</i> All path groups are reported.

`-tcl_list`

Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation.

The `-tcl_list` and `filename` parameters are mutually exclusive; you cannot specify them together.

Examples

- The following command generates a report that shows details of the specified path group:

```
litmus > set_path_group -name path1 -through CF_PS_PD_142/NS
litmus > report_path_groups -name path1
```

```
-----
Through          Group Name    View Name
-----
CF_PS_PD_142/NS  path1          setup_gs_mC
-----
```

- The following command shows a sample report:

```
litmus> report_path_groups
-----
From    To          Group Name
-----
CLK1    seg1/u3/D  path1
-----
```

Related Commands

- `set_clock_groups`
- `reset_clock_groups`
- `reset_path_group`
- `get_path_groups`
- `group_path`

report_ports

```
report_ports
[-type {[input] | [source_insertion] | [insertion] | [clock_root]
| [uncertainty] | [arrival] | [required] | [external] | [clk_arrival]
| [port_cap] | [fanout_load] | [fanout_load_limit] | [drive_resistance]
| [drive_cell] | [slew_time] | [slew_limit] | [constant] | external_detail
| drive_resistance_detail}]
[-include_pins]
[-pins port_name_list]
[-view view_name]
[> filename | -tcl_list]
```

Reports timing constraints on ports. By default, the command reports all timing constraints on all ports of the current module.

The direction (*DIR*) column of the generated report can contain one of the following abbreviations:

- `IN` indicates that the pin is an input pin.
- `OUT` indicates that the pin is an output pin.
- `INTL` indicates that the pin is an internal pin (a pin inside the instance boundary).

The *Clock Name* column of the report can contain the following symbols:

- `D` indicates it is a data signal in the clock domain.
- `C` indicates it is a clock signal in the clock domain.
- `P` indicates it is a positive phase.
- `N` indicates it is a negative phase.
- Asterisk (*) indicates there is a timing exception associated with the signal.

Note: Currently, it is not possible to expand the asterisk to determine which exception is associated with the reported timing phase.

Parameters

<code>> filename</code>	Writes the report to the specified file name. If the file already exists, the software overwrites it. The <code>filename</code> parameter must be the last argument in the list. The <code>filename</code> and <code>-tcl_list</code> parameters are mutually exclusive; you cannot specify them together. Note: To write a compressed report, add the <code>.gz</code> extension to the file name. <i>Default:</i> Report is displayed on standard output without being saved.
<code>-include_pins</code>	Reports timing constraints on instance pins, as well as on ports. <i>Default:</i> Reports timing constraints on ports only.
<code>-pins port_name_list</code>	Specifies the pins or ports to be reported. Pins can be specified by name, or by object ID in a Tcl list.
<code>-tcl_list</code>	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. The <code>-tcl_list</code> and <code>filename</code> parameters are mutually exclusive; you cannot specify them together.
<code>-type {assertion_type assertion_type_list}</code>	Generates the report for the specified analysis view only. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode.
<code>-view view_name</code>	Specifies the view name.

Examples

- The following command reports details of the specified list of pins:

```
litmus > report_ports -pins scan_en
```

```
-----
                                Early          Late
-----
Pin Name   Dir   Assertion   Clock Name                                Rise   Fall   Rise   Fall
-----
```


Litmus Text Command Reference

Timing Analysis Commands--report_ports

```
scan_en    IN    input    m_clk (D) (P) (scan_fast_RCMIN)  4.000    4.000    4.000    4.000
scan_en    IN    input    m_clk (D) (P) (scan_slow_RCMAX)  4.000    4.000    4.000    4.000
```

```
-----
Pin Name   Di    Assertion  Value  View
-----
scan_en    IN    constant   0      func_slow_RCMAX
scan_en    IN    constant   0      func_fast_RCMAX
-----
```

- The following command reports details of the specified list of pins: Reports timing constraints on instance pins, as well as on ports.

```
litmus > report_ports -include_pins
```

```
-----
                                Early      Late
-----
Pin Name   Dir    Assertion  Clock Name      Rise    Fall    Rise    Fall
-----
INT/p97/Z  OUT    clock_root  clk(sn_fast_RMIN)  -        -        -        -
INT/p97/Z  OUT    clock_root  clk(sn_slow_RCMIN) -        -        -        -
INT/p97/Z  OUT    clock_root  clk(sn_fast_RCMAX) -        -        -        -
INT/p97/Z  OUT    clock_root  clk(sn_slow_RCMAX) -        -        -        -
```

- The following command shows a port report, as shown in the following example:

```
litmus > report_ports
```

```
-----
Early Late
-----
Pin      Dir      Assertion      Clock Name      Rise    Fall    Rise    Fall
Name
-----
CLK1     IN        clock_root      CK2 (C) (P)
CLK2     IN        clock_root      CLK2 (C) (P)
CLK3     IN        clock_root      CLK3 (C) (P)
IN1      IN        drive_resistance * (D) (P)      2.000    2.000    2.000    2.000
IN2      IN        drive_resistance * (D) (P)      2.000    2.000    2.000    2.000
IN3      IN        drive_cell      * (D) (P)      INV      INV      INV      INV
OUT1     OUT       external        GCLK1 (C) (P)  1.000    1.000    1.000    1.000
OUT2     OUT
OUT3     OUT       derived_clock    GCLK4 (C) (P)
-----
Pin      Dir      Assertion      Value
Name
-----
CLK3     IN        slew_limit      ( 0.900 : )
```

Litmus Text Command Reference Timing Analysis Commands--report_ports

```

CLK3  IN  port_cap_limit      ( 0.020 : 0.900 )
CLK3  IN  fanout_load_limit   5.000
CLK2  IN  slew_limit          ( 0.900 : )
CLK2  IN  port_cap_limit      ( 0.020 : 0.900 )
CLK2  IN  fanout_load_limit   5.000
CLK1  IN  slew_limit          ( 0.900 : )
CLK1  IN  port_cap_limit      ( 0.020 : 0.900 )
CLK1  IN  fanout_load_limit   5.000
IN3    IN  slew_limit          ( 0.900 : )
IN3    IN  port_cap           ( 1.500 : : 2.600 )
IN3    IN  port_wire_cap      ( 1.200 : : 3.000 )
IN3    IN  port_cap_limit      ( 0.020 : 0.900 )
IN3    IN  fanout_load_limit   5.000

IN2    IN  slew_limit          ( 0.900 : )
IN2    IN  port_cap           ( 1.500 : : 2.600 )
IN2    IN  port_wire_cap      ( 1.200 : : 3.000 )
IN2    IN  port_cap_limit      ( 0.020 : 0.900 )
IN2    IN  fanout_load_limit   5.000
IN1    IN  slew_limit          ( 0.900 : )
IN1    IN  port_cap           ( 1.500 : : 2.600 )
IN1    IN  port_wire_cap      ( 1.200 : : 3.000 )
IN1    IN  port_cap_limit      ( 0.020 : 0.900 )
IN1    IN  fanout_load_limit   5.000
OUT3   OUT  slew_limit          ( 0.900 : )
OUT3   OUT  port_cap_limit      ( 0.020 : 0.900 )
OUT2   OUT  slew_limit          ( 0.900 : )
OUT2   OUT  port_cap_limit      ( 0.020 : 0.900 )
OUT1   OUT  slew_limit          ( 0.900 : )
OUT1   OUT  port_cap_limit      ( 0.020 : 0.900 )

```

- The following command shows `source_insertion` and `uncertainty` assertion type options as shown in the following example:

```

litmus > report_ports -type clock_root source_insertion input -pins clk in
litmus > report_ports -type {uncertainty source_insertion} -pins CLK1

```

Early Late

```

-----
Pin    Dir  Assertion      Clock Rise   Fall  Rise   Fall
Name                               Name
-----
CLK1   IN   source_insertion *(C) (P) -0.500 -0.500 -0.500 -0.500
-----

```

```

Pin    Dir  Assertion      Value
Name
-----
CLK1   IN   uncertainty  0.400(T) 0.400(T) 0.400(T) 0.400(T)

```

- The following command shows `clock_root` and `uncertainty` assertion type options as shown in the following example:

```
litmus > report_ports -type {input uncertainty clock_root}
```

```
-----  
Early Late  
-----
```

Pin Name	Dir	Assertion	Clock Name	Rise	Fall	Rise	Fall
CLK1	IN	clock_root	CLK_W_1 (C) (P)				
in1	IN	input	@ (D) (P)	1.000	1.000	1.000	1.000

```
-----
```

Pin Name	Dir	Assertion	Value
CLK1	IN	uncertainty	0.400 (T) 0.400 (T) 0.400 (T) 0.400 (T)

```
-----
```

Related Commands

- [report_clocks](#)
- [report_nets](#)
- [report_timing](#)

report_reference

```
report_reference  
[-help]  
[-flat]  
[-nosplit]  
[-view]  
[> file_name]  
[>> file_name]
```

Reports reference cell usage statistics.

The command reports the unit area, library name, and total count of the reference cells. By default, the report is sorted in the order of number of instances.

Parameters

-help	Prints out the command usage.
-flat	Disables printing of hierarchical instances.
-nosplit	Generates the report in no split mode.
-view	Generates the report for the specified analysis view.
> file_name	Specifies the name of write mode file name.
>> file_name	Specifies the name of the append mode file name.

Examples

- The following command displays the report for the specified view:

```
report_reference -view func_slow_RCMIN
```

```
Design : dtmf_recvr_core
```

```
Attributes :
```

```
c - combinational (standard cell)
```

```
sq - sequential (standard cell)
```

```
m - macro
```

```
h - hierarchical
```

```
b - blackbox
```

Reference	Library	Unit Area	Count	Total Area
-----------	---------	-----------	-------	------------

Litmus Text Command Reference
Timing Analysis Commands--report_reference

Attributes

AND3_X4	FrPK45_lib_v1.0	1.330	8	10.640	c
SDFE_X1	FrPK45_lib_v1.0	6.916	10	69.160	sq
AOI22_X4	FrPK45_lib_v1.0	2.128	9	19.152	c
NAND3_X1	FrPK45_lib_v1.0	1.064	87	92.568	c

report_timing

```
report_timing
[-help]
[-analysis_summary_csv_extended_file string]
[-analysis_summary_csv_file string]
[-analysis_summary_file string]
[-check_clocks]
[-check_type {setup hold clock_gating_setup clock_gating_hold clock_gating_pulse_width data_setup
data_hold recovery removal pulse_width clock_period clock_separation skew no_change_setup
no_change_hold max_delay min_delay}]
[-debug {unconstrained | time_borrow | cpr_point | constraint_arc | context | wire_variation |
vt_skew}]
[-derate_summary]
[-gui]
[-fields column_list]
[-group groupname_list]
[-hpin]
[-max_delay float]
[-max_slack float]
[-min_slack float]
[-nets]
[-output_format {text | csv | gtd | binary}]
[-path_exceptions {applied | ignored | all}]
[-path_type {endpoint | summary | full | full_clock}]
[-retime {aocv | path_slew_propagation | aocv_path_slew_propagation}]
[-retime_delaycal_pins pin_name]
[-retime_format {manual | retime_compare | retime_replace}]
[-retime_mode {path | exhaustive}]
[-skip_io_paths]
[-split_delay]
[-unique_pins]
[-view viewName]
[> <filename[.gz]>]
[>> <filename[.gz]>]
[-late | -early]
[-rise | -fall]
[-begin_end_pair | [[ -max_paths integer] [-nworst integer]]]
[-unconstrained | -point_to_point]
[-collection | -tcl_list ]
[-not_through object_list | -not_rise_through object_list | -not_fall_through object_list]
[ [-from object_list | -from_rise object_list | -from_fall object_list] [-through object_list | -
through_rise object_list | -through_fall object_list] [-to object_list | -to_rise object_list | -
to_fall object_list] ]
```

Generates a timing report that provides information about the various paths in a design.

The reports typically contain data on the delay through the entire path. The start node and the end node of each

path is identified.

The timing report contains the following information:

- The slack times of the arriving signal at the end node
- The start node
- The associated transitions
- The signal required times and the actual signal arrival times
- Summary of propagated vs ideal status of launching and capturing clocks
- Any phase shifts applied when evaluating timing checks
- Any CPPR values applied to timing check evaluation

The `report_timing` command output accommodates both the instance pin names and net names in the same column. Since both instance pin names and net names can be long, to save screen space alternating instance pin and net names (depending on whether the row displays gate delay or net delay) are reported in a single column.

When the `timing_report_timing_header_detail_info` timing attribute is set to `extended`, the software generates timing reports with additional header information, that includes the following details:

- Analysis mode - best-case worst-case (BCWC) or on-chip variation (OCV)
- Active views (displayed only in MMMC mode)
- Clock re-convergence pessimism (CPPR)
- Latch slack mode
- Specified clock propagation value
- Delay calculation
- Specified delay calculation engine

When the `timing_report_group_based_mode` attribute is set to `true`, the paths are grouped by clock domain. In this mode, the `report_timing` command will have the following four types of groups:

- Clock gating group: This is an internal group that reports the clock gating check type paths.
- Asynchronous group: This is also an internal group that reports the asynchronous check type paths.
- Groups for all clocks in a design: These are internal groups that are classified on the basis of each reference clock. The paths belonging to each reference clock are reported separately under each clock name's group. These groups do not honor the clock gating/asynchronous check types. Suppose there are three clocks C1, C2, and C3 in a design, then three groups will be reported for each of these clocks. The endpoint sharing the same reference clock will be part of the same group. In case the endpoint belongs to multiple groups then it will be reported separately in each group.
- Default group: This is an internal group that reports combinational maximum or minimum delay paths.
- User-defined groups: These are user-defined path groups that can be created by users with single or

multiple constraints. These groups honor all the check types and are reported by default.

By default, the `timing_report_group_based_mode` attribute is set to `false`.

Note: This attribute does not honor user-defined path groups, so the paths will still be grouped by different clocks.

The `report_timing` command supports multi-threaded reporting. You can make the following settings to enable multi-threaded reporting:

```
set_multi_cpu_usage -localCpu number_of_CPU
set_table_style -no_frame_fix_width [-nosplit]
```

Note: When the `timing_allow_input_delay_on_clock_source` attribute is enabled and the `set_input_delay` command is applied to a clock input, the specified delay value will be considered as the source clock latency for clock paths and as regular data path arrival time for clock source paths.

You can use a one-step method for generating merged MMMC view reports from DMMMC master shell. After running the `distribute_views` command, you can use the `report_timing` command to generate reports collectively from all the active distributed views (that is, views available for interactive DMMMC analysis). The merged output report is arranged as per the slack/criticality order across active views.

When the `timing_enable_wire_variation` attribute is set to `true`, the timing reports show wire variation details in the below columns:

- `retime_slew_wire_stddev`
- `retime_delay_wire_stddev`
- `arrival_wire_stddev`

For more details on wire variation, refer to the "Wire Variation Analysis" chapter in the *Litmus Stylus User Guide*.

- [Parameters](#)
- [Group Path Reporting for User-Defined Groups](#)
- [Use of Informational Flags](#)
- [Retime Analysis Report Header Flags](#)
- [Report Timing Column List](#)
- [Reporting Timing Checks Between Various Types of Clocks](#)
- [Reports Sections](#)
- [Examples](#)
- [report_timing -from latch/D Settings](#)
- [Related Commands](#)

Parameters

<code>-help</code>	Outputs a brief description that includes type and default information for each <code>report_timing</code> parameter. For a detailed description of the command and all of its parameters, use the man command: <code>man report_timing</code> .
<code>{> >>} filename</code>	When specified with <code>></code>, writes the report to the specified file name. If the file already exists, the software will overwrite it. When specified with <code>>></code>, writes the report to the specified file name. If the file already exists, the software will concatenate the report at the end of the file. The <code>filename</code> parameter must be the last argument in the list. The <code>filename</code> and <code>-tcl_list</code> parameters are mutually exclusive; you cannot specify them together. Note: To write a compressed report, add the <code>.gz</code> extension to the file name. Note: The last argument specified with the <code>report_timing</code> command is not considered a filename. <i>Default:</i> Report is displayed on standard output (not saved).
<code>-analysis_summary_csv_extended_file string</code>	Generates the CSV file to be used for merging. <i>Data_type:</i> string, optional
<code>-analysis_summary_csv_file string</code>	Generates the analysis summary CSV file. <i>Data_type:</i> string, optional
<code>-analysis_summary_file string</code>	Generates the analysis summary file. <i>Data_type:</i> string, optional

<code>-begin_end_pair</code>	Reports all the violating paths between unique source and target register pairs. To report other paths, you can specify this parameter with the <code>-max_slack</code> parameter. The paths are not sorted by slack - paths with the same source pins are reported adjacent to each other. When specified with the <code>-from</code>, <code>-through</code>, or <code>-to</code> parameter, the software only reports the unique register pair paths that match the path description. In merged group-based reporting mode (that is, when <code>timing_report_backward_compatible_max_paths_reporting</code> attribute is set to <code>false</code>), the <code>report_timing -begin_end_pair</code> command <code>report_timing -begin_end_pair</code> will always report the paths grouped by <code>path_groups</code> irrespective of <code>timing_report_group_based_mode</code> attribute setting. You can use the <code>timing_report_begin_end_pair_max_path_limit</code> attribute to specify the maximum paths limit. When the number of maximum paths to be reported by the <code>-begin_end_pair</code> parameter exceeds the specified limit (default is 200000), a warning message will be displayed. Also, some of the paths may not be reported. Note: You cannot specify this parameter with the <code>-max_paths</code>, <code>-nworst</code>, <code>-unique_pins</code>, and <code>-min_slack</code> parameters. <i>Data_type:</i> bool, optional
<code>-check_clocks</code>	Generates reports based on timing paths on the clock network instead of the standard timing to data endpoints. This report includes clock paths that end at the reference end of a check or a clock gating end point. <i>Default:</i> Data paths and all the other clock paths (for example, a clock path ending at a D pin of a register) are reported. <i>Data_type:</i> bool, optional
<code>-check_type check_type</code>	Reports only the paths that end at the specified

timing check. You can specify the following options:

setup, hold, clock_gating_setup, clock_gating_hold, clock_gating_pulse_width, data_setup, data_hold, recovery, removal, pulse_width, clock_period, clock_separation, skew, no_change_setup, no_change_hold, max_delay, and min_delay.

- The clock style checks are: pulse_width, clock_gating_pulse_width, clock_period, skew, and clock_separation.
- In setup analysis mode, the reported check types will be: setup, skew, pulse_width, clock_period, clock_gating_setup, clock_gating_pulse_width, data_setup, recovery, clock_separation, and no_change_setup.
- In hold analysis mode, the reported check types will be: hold, clock_gating_hold, data_hold, removal, and no_change_hold.

If the software is in simultaneous setup and hold analysis mode, all of the check types listed above will be reported.

When the `max_delay` and `min_delay` options are specified, the timing report will display details of timing paths for which the `set_max_delay` or `set_min_delay` constraints are defined.

By default, the following checks are reported:

- setup
- hold
- clock_gating_setup
- clock_gating_hold
- data_setup
- data_hold
- recovery

	• removal • no_change_setup • no_change_hold Do not use this parameter with the <code>-unconstrained</code>, <code>-early</code>, or <code>-late</code> parameters. Note: The <code>report_timing</code> command does not perform skew checks by default. <i>Data_type:</i> enum, optional
<code>-collection</code>	Returns a collection of timing paths, which is useful for performing Tcl queries on selected timing reports. To generate a collection of paths analyzed by Path-Based Analysis (PBA), you can use the <code>-retime</code> parameter. Note: This parameter is not supported in DMMMC mode. <i>Data_type:</i> bool, optional

<pre>-debug {unconstrained time_borrow cprp_point constraint_arc context wire_variation vt_skew}</pre>	You can specify the following options for debugging: • unconstrained: Determines the reasons for a path to be unconstrained. The report output will list all the applicable arcs/constraints that could not constrain the paths, along with the reasons. • time_borrow: Reports time borrowing calculations. Time borrowing is the amount of time borrowed by a previous logic. • cprp_point: Reports the CPRP point information for the reported paths. • constraint_arc: Reports check arc details, such as, constraint arc delay, constraint pins (along with the related pin slews), skewness, user derate, annotation, and when_cond. • context: Reports the launch and capture path interface timing for each instance and indicates the block port that shows latency in the MIM flow. • wire_variation: Reports the corresponding slack shift details at the end of each timing path report. • vt_skew: Reports voltage threshold (VT) skew analysis details. <i>Data_type:</i> string, optional
<pre>-derate_summary</pre>	Generates a summary table for AOCV derating of launch and capture paths. <i>Data_type:</i> bool, optional
<pre>-fields column_list</pre>	Formats the report according to the specified <i>column_list</i>. The <i>column_list</i> specifies the columns to be displayed in the timing report and the order in which they appear. The valid field columns are: adjustment, abs_delay_err, abs_transition_err, annotation, aocv_adj_stages, aocv_derate, aocv_weight, arc, arrival, arrival_mean,

arrival_sigma, cell, delay, delay_mean, delay_sigma, direction, edge, fanin, fanout, flags, hpin, incr_delay, instance, instance_location, latch_window, load, module, net, pct_delay_err, pct_slew_err, phase, phys_info, pin, pin_load, pin_location, power_domain, required, retime_delay, retime_delay_mean, retime_delay_sigma, retime_mis_delta, retime_mis_derate, retime_slew, retime_transition_mean, retime_transition_sigma, stage_count, transition, transition_mean, transition_sigma, socv_derate, spice_arrival, spice_delay, spice_slew, stolen, timing_point, total_derate, user_derate, voltage, vt_skew_derate, when_cond, wire_load, wlmodel

For example:

```
-fields {hpin cell delay required arrival  
required edge}
```

See [Report Timing Column List](#) for details on the above options.

Few notes:

- The default net format (with the `-net` parameter) for the full path will be:

```
{hpin edge net cell delay arrival  
required}
```
- The `required` column will not be displayed, when the `-unconstrained` parameter is specified.
- The `adjustment` column is automatically added to the timing report. To disable this feature you can set the `timing_report_generated_clock_info` attribute to `false`. By default this global variable is set to `true`.
- You can use a combination of the `-fields` and `-tcl_list` parameters to integrate the timing reports into your Tcl scripts. You can use the `-from` parameter to limit the number of paths reported, and to find specific paths in the design.

	• The <code>-format</code> option cannot be used with the <code>-path_type summary</code> option. • You can manage the retiming columns with the <code>timing_report_retime_formatting_mode</code> global variable. • If you specify the <code>report_timing -split_delay</code> parameter, then the timing report format will change to: <pre>{timing_point flags edge cell power_domain fanout transition delay arrival}</pre> • Informational tags are displayed for reporting additional details with the <code>flags</code> option. For details on the 'Flags' column, refer to the section on Use of Informational Flags. <i>Default:</i> {timing_point flags arc edge cell power_domain fanout transition delay arrival} <i>Data_type:</i> optional
<code>-from -from_rise -from_fall</code> <code>object_list</code>	Reports paths starting from the object(s) specified in the <code>object_list</code>. The supported objects are pins, ports, and clocks. Using <code>-from_rise</code> (or <code>-from_fall</code>) parameter specifies that the rising (or falling) edge of the signals on the objects in the <code>object_list</code> are the start of the paths. This parameter also reports on source clock waveform(s). You can use this parameter with the <code>-through</code> and <code>-to</code> parameters for specifying particular paths in the design. <i>Data_type:</i> string, optional
<code>-group groupname_list</code>	Reports paths contained in the groups specified in the <code>groupname_list</code>. <i>Data_type:</i> string, optional
<code>-gui</code>	Displays the timing report in a GUI window. <i>Data_type:</i> bool, optional
<code>-hpin</code>	Prints all the hierarchical crossings of an arc under the <code>hpin</code> column. <i>Data_type:</i> bool, optional

<code>-late -early</code>	Reports for early paths (hold checks) or late paths (setup checks). If you specify a check type, setup or hold, with the <code>timing_analysis_check_type</code> attribute, the <code>report_timing</code> command reports the specified check by default. Note: The <code>-early</code> and <code>-late</code> parameters are not mutually exclusive; you can specify them together. <i>Default:</i> <code>-late</code> <i>Data_type:</i> bool, optional
<code>-max_delay float</code>	Specifies the path delay limit for unconstrained paths. <i>Data_type:</i> string, optional
<code>-max_paths integer</code>	Reports the specified number of worst paths in the design, with maximum of <code>nworst</code> (<code>-nworst</code> parameter) paths to any single endpoint (default 1). When <code>timing_report_group_based_mode</code> (group-based mode) attribute is set to <code>true</code>, the <code>-max_paths</code> parameter specifies the number of worst paths reported per clock group. <i>Default:</i> 1 <i>Data_type:</i> int, optional

`-max_slack float`

Reports only those paths with slack less than the specified maximum slack value.

The `-max_slack` parameter reports paths that fall within the specified range. A positive slack value indicates that timing was met and a negative slack value indicates a timing violation.

When the `-retime_mode exhaustive` option is specified, the `-max_slack` value is set to 0.

The `-max_slack` parameter cannot be used with the `-unconstrained` option.

Data_type: string, optional

`-min_slack float`

Reports only those paths whose slack is greater than the specified minimum slack value.

You can generate a timing report showing paths with slack greater than the specified slack using the `-max_paths` parameter. For example, to report all paths with slack greater than 2ns, you can use the following command:

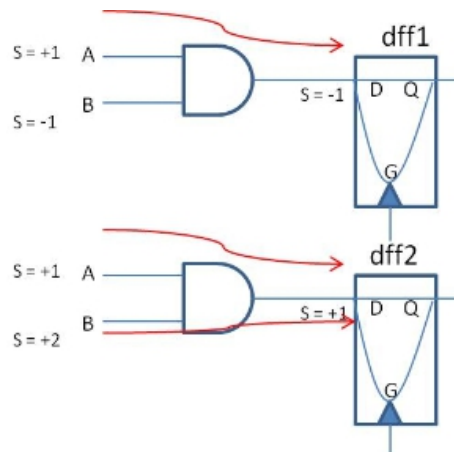
```
min_slack 2.0 -max_paths 1000
```

In this case the maximum number of paths reported will be 1000.

When the `-min_slack` parameter is specified, the total numbers of paths processed will be as determined by the `-max_paths/-nworst` settings and only those paths satisfying the `-min_slack` criteria will be reported. The software will not process paths beyond the specified `max_paths/nworst` limit, even if the `min_slack` criteria is satisfied.

The `-min_slack` cannot be used with the `-unconstrained` parameter.

In the below diagram, if `-min_slack` is 0 and `-max_paths 4 -nworst 2` is specified, then only 3 paths will be reported; path with slack value of -1 will be ignored. Here, even if there is another path with slack that is greater than or equal to 2, then it will not be reported because worst `max_paths` have been processed.



Data_type: string, optional

<code>-nets</code>	Adds a row to the "Timing Point" field column to display the net names. Note: The <code>-split_delay</code> and <code>-nets</code> parameters can be used together. <i>Data_type:</i> bool, optional
<code>{-not_through -not_rise_through -not_fall_through} object_list</code>	You can specify one of the following options: • <code>-not_through</code>: Reports paths that do not traverse through the specified nets, ports, or pins of a cell. By default, this parameter will be ignored if an object has already been specified. In this case the parameter shows the following behavior: ◦ If a collection of all the input ports has been specified (using the <code>all_inputs</code> command), then the report will show paths to the inout part, and paths starting from the internal part of inout will be excluded. ◦ If a collection of all the output ports has been specified (using the <code>all_outputs</code> command), then the report will show paths starting from the internal part of inout, and paths ending at inout part will be excluded. ◦ If any other collection (not specified with <code>all_inputs</code> or <code>all_outputs</code> command) is specified, then paths to both the ports will not be reported. • <code>-not_rise_through</code>: Does not report paths with rise transition. • <code>-not_fall_through</code>: Does not report paths with fall transition. <i>Data_type:</i> string, optional

<code>-nworst integer</code>	Specifies the maximum number of paths which can be reported for a particular endpoint (the total number of paths reported is specified by the <code>-max_paths</code> parameter). When <code>timing_report_group_based_mode</code> attribute is set to <code>true</code> (group-based mode), the <code>-nworst</code> parameter specifies the maximum number of paths for a particular endpoint per clock group. You can use the <code>-nworst</code> parameter to report all the checks at an endpoint (or use <code>-check_type</code>) to report a specific check. Default: 1 <i>Data_type</i>: int, optional
<code>-output_format {text csv gtd binary}</code>	Generates a detailed timing report in the specified format - text, CSV, GTD, or binary. <i>Data_type</i>: enum, optional

```
-path_exceptions {applied | ignored |  
all}
```

Includes information of path exceptions applied and considered for the reported path. You can specify the following options:

- **applied:** Reports the honored exception on the reported path.
- **ignored:** Reports the ignored exceptions on the reported path.
- **all:** Reports both applied and ignored set of exceptions on the reported.

Note: Analyzing all the exceptions for a path will cause increase in runtime.

If multiple from, to, or through pins are specified, the exception report for that path will show only those pins that are part of the current path.

Note: If a path is already unconstrained, then all further path exceptions applied on that particular path will be reported as ignored exceptions.

The `-path_exceptions` parameter also reports -start/-end options with multi-cycle paths. Without the start or end specification, by default, the setup check is relative to the end clock and the hold check is relative to the start clock.

The `report_timing -path_exceptions all` command will report all the non-default -start/-end specifications.

You cannot use this parameter with the following parameters:

- `-point_to_point`
- `-collection`
- `-path_type`
- `-tcl_list`

Also, you cannot use this parameter with block-based SSTA.

Note: Analyzing all the exceptions for a path will increase the runtime.

Data_type: enum, optional

```
-path_type {endpoint | summary | full |  
full_clock}
```

The `-path_type` parameter allows you to choose the report format by path type.

You can specify one of the following options:

- `endpoint`: Generates an endpoint report for each path containing the ID, slack, endpoint, and group view. This option generates a very fast report.
- `summary`: Generates a summary report for each path consisting of a start point, endpoint, cause, slack, arrival time, required time, and phase. The `-fields` option does not have any impact on this report format.
- `full`: Generates a report that displays the full path with accompanying required time and slack calculation. This is the default path type. You can control the report format using the `-fields` option.
- `full_clock`: If the reported path ends at a timing check, the software reports the full clock path (Other End Path), in addition to the full data path (Timing Path).

Data_type: enum, optional

`-point_to_point`

Traces the worst delay path between a pair of from-to pins. The paths reported with this parameter only include the cumulative delay of the path. You must specify either the `-from` or `-to` parameter, to define the begin and end points of the path.

If you do not specify a `-from` pin, the standard definition of the begin point of a path applies. If you specify a `-from` pin, it is treated as the begin point of the path. If you specify a `-from` pin that is not a valid begin point, the arrival time of the first pin starts from 0.

If you do not specify a `-to` pin, the standard definition of the end point applies. If you specify a `-to` pin, it is treated as the end point, and only those paths ending in `-to` pins are reported. If you specify more than one `-to` pin, the worst path to each pin-to-pin is reported.

When from-to pins are specified, the software will trace across the trigger arcs and report a path, if there is a clock reaching the CK pin of the trigger arc. However, if there is a begin or end point present between the `-from` and `-to` pins, the software will not trace beyond it.

If there is no clock reaching the CK pin of the trigger arc, the software cannot trace across the trigger arc, and no path is reported. Additionally, clock path delay is included only when the clock is in propagated mode.

Note: The `-point_to_point` and `-through` parameters are mutually exclusive; you cannot specify them together.

Note: The `-point_to_point` parameter supports a single `-from` and `-to` pin; a list of pins is not supported.

Data_type: bool, optional

<pre>-retime_delaycal_pins pin_name</pre>	Reports the PBA delay calculation details for timing arcs that fanin to the user-specified to-pin list. You can specify multiple pins in the pin list. The delay calculation report will be printed at the end of each timing path for the arcs, which are retimed in that path. The PBA delay calculation report format is similar to the report_delay_calculation output in GBA mode, however, the results may vary because this output is based on path-based analysis. When a cell output pin list is provided, the generated report is for the cell arcs; the cell input pin list generates the net arc report. This parameter is supported for PBA reporting only. <i>Data_type:</i> string, optional
<pre>-retime_format {manual retime_compare retime_replace}</pre>	Allows you to set the format of timing reports for retime related fields. You can specify one of the following options: • manual: Does not change the report_timing format. • retime_compare: If GBA columns are present in the report format, then the corresponding retime options are added. If PBA columns are already present, then there is no change in the format. • retime_replace: If GBA columns are present in the report format, then the GBA columns are replaced by the corresponding PBA columns. If PBA columns are already in the report, then there is no change in the format. Note: The timing report format can be controlled with the timing_report_retime_formatting_mode attribute. The report_timing command setting has precedence over this attribute. <i>Data_type:</i> enum, optional

<pre>-retime_mode {path exhaustive}</pre>	Reanalyzes the specified set of paths using the specified method. You can use one of the following options: • path: Reanalyzes the specified set of paths. By default, the software uses the <code>-retime_mode path</code> setting, when the <code>-retime path_slew_propagation</code> option is specified. • exhaustive: Enables exhaustive path-based analysis mode. In this mode, the software will exhaustively examine <code>n</code> worst paths of the selected endpoints and then select the true worst path to be reported, based on retimed slacks of all the examined paths. When you use this option, reporting can be runtime intensive as the software needs to run an exhaustive search for true worst paths. Therefore, it is recommended to use this option in the later phases of the signoff stage when few violations are left for analysis. <i>Default:</i> <code>path</code> <i>Data_type:</i> enum, optional
<pre>-retime {aocv path_slew_propagation aocv_path_slew_propagation}</pre>	Reanalyzes the specified set of paths using path-based analysis (1 path per endpoint). Note: This feature is not meant to be run on all the paths of a design. You can specify the following options: • aocv: Computes AOCV stage count based on specific paths and applies the AOCV derate values to GBA (graph-based analysis) delays. This option can be used with the following parameters only: ◦ <code>-rise/-fall</code> ◦ <code>-early/-late</code> ◦ <code>-check_type</code> ◦ <code>-max_paths</code> ◦ <code>-begin_end_pair</code>

- -from/-from_rise/-from_fall
- -through/-through_rise/ -through_fall
- -to/-to_rise/-to_fall
- -rise/-fall
- -early/-late
- -check_type
- -max_paths
- -begin_end_pair
- -from/-from_rise/-from_fall
- -through/-through_rise/ -through_fall
- -to/-to_rise/-to_fall
- -path_exceptions
- -path_type
- -max_slack
- -min_slack
- -view
- -derate_summary
- -nworst
- path_slew_propagation: **Re-evaluates the** given set of graph-based analysis (GBA) paths by recalculating the delay values based upon actual propagated slew across the path. In this case the AOCV derates are the ones that are reported during graph-based analysis.

The default timing analysis uses worst-case slew propagation to calculate the delays at each stage. As a result, the timing analysis results can be pessimistic.

Note: This feature should be run only on paths that do not meet the slack margin criteria to help identify the actual failing paths from those that are simply failing due to the conservative delay calculation.

The signal integrity incremental delays can be recomputed for the entire path. You can use the following command to perform path-based signal integrity analysis:

```
report_timing -retime path_slew_propagation
```

In the `report_timing` command output, the `Retime Incr Delay` column reports recomputed signal integrity delays.

The `-retime` parameter also supports retiming a collection of paths. This parameter does not retime an already retimed path collection. This feature works in graph-based analysis (GBA) mode for path collections. You can use the following use model to retime collections:

```
report_timing $collection -retime  
path_slew_propagation
```

A path collection stores context dependent data, that is, a delay property in GBA will reflect delay data of the timing path report generated in GBA mode. The same property in PBA will reflect the retime delay data of the timing path report in PBA mode.

The delay column in timing reports is context dependent, when the

```
timing_report_retime_formatting_mode  
retime_replace
```

 attribute is used.

Note: For memory optimization purposes, both the delays will not be stored in the same path collection. You can use this setting for comparison purposes.

- `aocv_path_slew_propagation`: Re-evaluates the given set of graph-based analysis (GBA) paths by using path-based AOCV derates and delays using actual path-based propagated slew. This option is a combination of `-aocv` and `-path_slew_propagation` options.

For retime report header details, see [Retime Analysis Report Header Flags](#).

Data types are optional.

<code>-skip_io_paths</code>	Specifies whether the input/output paths are to be ignored. <i>Data_type</i>: bool, optional
<code>-rise -fall</code>	Reports the path with the specified edge on the endpoint. If an endpoint is specified using the <code>-to_rise</code> (or <code>-to_fall</code>) parameter, the <code>-rise</code> (or <code>-fall</code>) parameter is ignored and paths with edge specified by <code>-to_rise</code> (or <code>-to_fall</code>) are reported. The <code>-rise</code> option applies to endpoints only. <i>Data_type</i>: bool, optional
<code>-split_delay</code>	Adds a row for the net arc. Note: The <code>-split_delay</code> and <code>-nets</code> parameters can be used together. <i>Data_type</i>: bool, optional
<code>-tcl_list</code>	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions and also for customizing report generation. The <code>-tcl_list</code> and <code>filename</code> parameters are mutually exclusive; you cannot specify them together. <i>Data_type</i>: bool, optional
<code>-through -through_rise -through_fall object_list</code>	Reports paths that pass through pin(s) or net(s) specified by the <i>object_list</i>. Any number of <code>-through</code> pins or nets can be specified. Using the <code>-through_rise</code> (or <code>-through_fall</code>) parameters specifies that the paths go through the rising (or falling) edge of the signals on the pins/nets in the <i>object_list</i>. The <i>object_list</i> is a logical OR function. The resulting path may pass through any of the pins or nets in the <code>-through object_list</code>. To force the report to pass through multiple pins or nets, separate <code>-through</code> statements are needed. You can use these parameters with the <code>-net</code> parameter. <i>Data_type</i>: string, optional

<code>-to -to_rise -to_fall object_list</code>	Reports paths leading to the pin(s) specified by the <i>object_list</i>. The pins in the <i>object_list</i> can be either pins on the design boundary (ports) or pins on an instance. Only one list of -to pins can be specified per report. Using <code>-to_rise</code> (or <code>-to_fall</code>) specifies that the rising (or falling) edge of the signals on the pins in <i>object_list</i> are at the end of the paths. This parameter also reports on target clock waveform(s). The <code>-to_rise</code> option applies to endpoints as well as intermediate pins. <i>Data_type</i>: string, optional
<code>-unconstrained</code>	Reports only the unconstrained paths (paths with no slack). Each signal arriving at the path end node which does not have a matching required time, results in an unconstrained path. The <code>report_timing</code> command without the <code>-unconstrained</code> parameter reports only constrained paths. If no constrained path is found, there may be unconstrained paths or paths may not exist. Note: The <code>-min_slack</code> and <code>-max_slack</code> cannot be specified with the <code>-unconstrained</code> parameter. See Examples for the conditions under which a path is reported as constrained or unconstrained. <i>Data_type</i>: bool, optional
<code>-unique_pins</code>	Reports paths that pass through unique set of pins. When this parameter is specified, in group-based mode for a given begin-end point pair, the software will consider all different launch clock paths for reporting unique paths. If for a given begin-end pair and a given capture clock, there is a possibility of different launch clock paths, then each such path will be reported. Note: Using this parameter may significantly affect runtime and memory usage, if the number of reported paths is high.

`-view viewName`

Generates report for the specified analysis view. By default, all the active views are analyzed.

You must be in multi-mode multi-corner (MMMC) analysis mode in order to use this parameter. To create a view, you use the `create_analysis_view` command. To set an active view for analysis, you use the `set_analysis_view` command.

The `report_timing` command performs block-based analysis in MMMC mode. By default, the command reports the worst endpoint(s) across all views. You can use the `-view` parameter to report for individual views.

Default: Reports the worst path to each endpoint across all the analysis views.

Data_type: bool, optional

Group Path Reporting for User-Defined Groups

- A user-defined group is created with the same name as any of the defined reference clocks and with the same constraint as for the "to" clock. For example, `group_path -name CLK -to CLK`.
In this case only one group named `CLK` will be displayed in the timing report. This will be the same default group with reference clock `CLK`. Note that default groups - clock gating and asynchronous checks - will not be honored for this group.
- A user-defined group is created with the same name as any of the reference clocks but with different constraint as specified for the "to" clock. E.g., `group_path -name CLK -from pin1 -to CLK`.
In this case the two groups named `CLK` will be displayed in the timing report. One group will be the same as the default group with reference clock `CLK` (which will not honor clock gating or asynchronous checks) and the other will be the user-defined path group with name `CLK`, which will honor all the checks.

Use of Informational Flags

Informational tags are used providing additional details. The following informational tags are supported:

Flags	Description
(i)	Net is ideal
(P)	Instance is preserved / don't_touch

(p)	Instance is preserved / don't_touch but may be resized

The output format uses the flag keys on the timing report and adds a description at the end of the report for reference. A sample output is as shown below:

```
#-----
# Timing Point   Delay (ns) Latch   Window   Arrival   Arc   Flags   Adjustment
# (ns)
#-----
f0/CP            -           -       1.000000           CP           -       -
f0/Q            0.254100   -       1.254100           CP->Q        -       -
b32/Z            0.598446   -       1.852546           I->Z         -       -
...
b4/Z            0.086940   -       2.502181           I->Z         -       -
f1/D            0.000000   -       2.502181           D           <<<       -
#-----
```

(P) : Instance is preserved
 (p) : Instance is preserved but may be resized
 (i) : Net is ideal.
 (b) : Timing paths are broken.

Retime Analysis Report Header Flags

AOCV(default)	Indicates that the selected retime analysis includes recalculation of path depths and AOCV derates. The AOCV retime is enabled with the <code>-retime aocv</code> or <code>aocv_path_slew_propagation</code> parameter. The AOCV stage counting mode (specified using the <code>timing_aocv_analysis_mode</code> attribute) is indicated in parenthesis. The stage counting modes are given below: • <code>launch_capture</code> (default) (Default) • <code>combine_launch_capture</code> (Combine) • <code>separate_clock_data</code> (Separate) • <code>clock_only</code> (Clock)
---------------	--

Corr	Indicates that the current SOCV accuracy mode considers correlation of variation between stages. Starting with SOCV accuracy mode <code>medium</code> (specified with the <code>delaycal_socv_accuracy_mode</code> attribute), it is assumed that some portion of the variation is not completely uncorrelated random variation, so the software tracks a correlated and uncorrelated component for each of the applicable moments.
Data Clock	Indicates the portion of the path that has been retimed. By default, only the data path is retimed. If the <code>timing_disable_retime_clock_path_slew_propagation</code> attribute is set to <code>false</code>, then the clock path will also be retimed, and "Clock" will be added to the retime status.
EWM	Indicates that equivalent waveform model is used (<code>delaycal_equivalent_waveform_model</code> attribute).
GBA COLL	Shows a collection of paths for which retime analysis is not performed. This can be controlled with the When the <code>report_timing -path_type exhaustive -collection</code> command. By default, the <code>report_timing -collection</code> parameter does not show GBA and PBA slack values separately.
GP	Indicates that graph-based PBA mode settings are enabled.
GS	Indicates that GBA skewness mode settings are enabled.
PBA COLL	Shows a collection of paths for which retime analysis is performed while creating the collection. This can be controlled with the When the <code>report_timing -path_type exhaustive -collection</code> command. By default, the <code>report_timing -collection</code> parameter does not show GBA and PBA slack values separately.

Path-Slew	Indicates the selected retiming analysis is set to use path correct slew rates to reduce the pessimism from slew merging. This can be controlled with the <code>report_timing -retime</code> parameter. The path slews are recalculated in <code>path_slew_propagation</code> and <code>aocv_path_slew_propagation</code> modes.
PG	Indicates that group-based mode reporting is enabled when N paths are reported for every group.
RC	Indicates that RC variation is enabled with the <code>timing_socv_rc_variation_mode</code> attribute setting, and a SOCV format RC variation side file library is available.
SI	Indicates that SI analysis was enabled during retiming analysis. The path-based analysis will recompute SI effects using path correct victim timing windows.
SOCV(Ultra)	Indicates that SOCV analysis is enabled and has been incorporated into the retiming. The SOCV analysis mode (specified with the <code>delaycal_socv_accuracy_mode</code> attribute) is shown in parenthesis. The accuracy modes are mapped as follows: • low <not shown> • medium Medium • high High • ultra Ultra

Spatial (CS)	Indicates that spatial derating side files have been loaded and an explicit spatial derate factor will be calculated. This is not shown for AOCV 2D derate tables, where the spatial and depth derates are combined. The GBA spatial derate mode (specified with the <code>timing_spatial_derate_distance_mode</code> attribute) is shown in parenthesis with the possible values given below: • <code>chip_size</code> (CS) • <code>bounding_box</code> (BB) The GBA spatial derate mode does not affect the path-based analysis bounding box calculation.
WP	Indicates that waveform propagation is enabled (using the <code>delaycal_equivalent_waveform_model</code> attribute). The path-based retime will also use this delay calculation accuracy setting when recomputing delays and transitions for the path.

Examples of GBA/PBA Collection Reports

- When the `report_timing -path_type exhaustive -collection` command is specified, the following command generates a GBA collection report as shown below:

```
litmus > report_timing $tim_coll ; set tim_coll [report_timing -path_type exhaustive -collection]
```

```
**WARN: (TCLCMD-1248): Since no '-max_slack' has been specified with exhaustive '-retime_mode', tool will assume it's default value in this case which is '0'
```

```
**WARN: (TCLCMD-1251): The PBA exhaustive mode '-nworst' limit has been exhausted for some of the endpoints. Refer to verbose log file of current session for more details.
```

```
INFO: Path Based Analysis (PBA) performed on total '3' paths
```

```
INFO: Max depth 3 reached for end point 'htb/CG_REBAL_HIER_IN21/RC_IC_IN_1/E' during Path Based Analysis
```

```
Path 1: VIOLATED Clock Gating Setup Check with Pin htb/CG_REBAL_HIER_IN21/RC_IC_IN_1/CK
```

```
Endpoint: htb/CG_REBAL_HIER_IN21/RC_IC_IN_1/E (v) checked with leading edge of 'clk'
```

```
Beginpoint: htb/mc_vl_reg_2/Q (v) triggered by leading edge of
```

```
'clk'
```

```
Common point: SPINE_CELL_BBF1/Y
```

```
Path Groups: {clock_gating_default}
```

```
Analysis View: sg_05v_15c_sigmax
```

```
Retime Analysis {GBA COLL}
```

- When the `report_timing -path_type exhaustive -collection` command is specified, the following command shows a PBA collection report:

```
litmus > report_timing $tim_coll ; set tim_coll [report_timing -path_type exhaustive -  
collection]
```

```
**WARN: (TCLCMD-1248): Since no '-max_slack' has been specified with exhaustive '-  
retime_mode', tool will assume it's default value in this case which is '0'
```

```
INFO: Path Based Analysis (PBA) performed on total '530' paths
```

```
INFO: Max depth 131 reached for end point 'htb/CG_REBAL_HIER_IN21/RC_IC_IN_1/E' during Path  
Based Analysis
```

```
Path 1: VIOLATED Clock Gating Setup Check with Pin
```

```
htb/RC_CG_REBALANCE_HIER_INST2989/RC_IC_IN_1/CK
```

```
Endpoint: htb/RC_CG_REBALANCE_HIER_INST2989/RC_IC_IN_1/E (v) checked with leading edge of  
'clk'
```

```
Beginpoint: htb/mc_vl_reg_2/Q (^) triggered by leading edge of  
'clk'
```

```
Common point: SPINE_CELL_BBF1/Y
```

```
Closest CPPR Driver Pin: { Launch: SPINE_CELL_BBF1/Y Capture: SPINE_CELL_BBF1/Y }
```

```
Path Groups: {clock_gating_default}
```

```
Analysis View: sg_05v_15c_sigmax
```

```
Retime Analysis {PBA COLL}
```

VT Skew Reports

The following informational tags are supported for VT skew reports:

Flags	Description	Command Settings
VT-Skew	Shows VT skew details.	set timing_enable_vtskew_derate_mode true

PBA	Shows that the current path has undergone VT skew analysis. However, all the paths might not be considered VT skew candidates - even if VT skew analysis mode is enabled.	<code>set_vt_skew_config -unique_vt_mode {none slow fast nominal}</code>
PU	Indicates unique VT skew for PBA. Displayed when unique VT mode is enabled.	<code>set_vt_skew_config -unique_vt_mode {slow fast nominal}</code>
GU	Indicates unique VT skew for GBA. Displayed when unique VT mode is enabled.	

A sample report is shown below:

```
Path 1: MET Hold Check with Pin CRE_IN/EXE_IN/top_reg[11]/CP
Endpoint:  CRE_IN/EXE_IN/top_reg[11]/D                (v) checked
with leading edge of 'm_clk'
Beginpoint: TDSP_CORE_INST/DATA_BUS_MACH_INST/data_out_reg[11]/Q (v) triggered
by leading edge of 'm_clk'
Path Groups: {m_clk}
Analysis View: view2
Retime Analysis {Data Path-Slew SOCV(High)-Corr SI Vt-Skew (PBA GU PU)}
Retime Vt-Skew Analysis {VT_group_1@fast VT_group_2@fast VT_group_3@fast }
Retime Vt-Skew Stats { Dominant_VT_group: NA Launch_Clock_VTGroups: 2 Capture_
Clock_VTGroups: 1 Total_Clock_VTGroups: 2}
Other End Arrival Time          0.569769 (+3.00S) | 0.531640 | 0.012710
+ Hold                          -0.035400 (+3.00S) | -0.035400 | 0.000000
+ Phase Shift                    0.000000
- CPPR Adjustment               0.051262 (+3.00S) | 0.000100 | 0.017054
= Required Time                 0.462027 (+3.00S) | 0.496140 | 0.011371 (-)
  Arrival Time                  0.769317 (-3.00S) | 0.803430 | 0.011371
  Slack Time                    0.307290 (-3.00S) | 0.307290 | 0.000000
= Slack Time(original)         0.356572 (-3.00S) | 0.356572 | 0.000000
  Clock Rise Edge               0.000000
  = Beginpoint Arrival Time     0.000000 (-3.00S) | 0.000000 | 0.000000
```

Report Timing Column List

Option	Description
--------	-------------

adjustment	Reports generated clock adjustment values. When reporting the detailed path for a generated clock, there are circumstances where the arrival time needs to be adjusted when a <code>create_generated_clock</code> assertion is encountered along the path. The adjustment argument can be used to provide a clearer indication of why a "jump" in the arrival time is seen in the path report. There are two primary reasons for generated clock-related adjustment to occur. • If the <code>timing_enable_generated_clock_edge_based_source_latency</code> attribute is set to <code>false</code>, it is possible that the edge-to-edge requirement specified by the generated clock cannot be satisfied by an existing logic path. This can happen when the source clock to generated clock polarity of the constraint is non-inverting, but the clock path itself constrains an inverting path. In this case, it would be normal to see a half-cycle of adjustment as the software compensates for the path versus constraint mismatch. If the above mentioned attribute is set to true, the software handles the path versus constraint mismatch by zeroing the source latency of the generated clock, and forcing alignment of the source and generated clock. As a result, no further adjustment is required, or reported. • In cases where <code>create_generated_clock - edge_shift</code> is specified, the adjustment column will show the amount of shift specified.
abs_delay_err	Reports the absolute delay difference of Litmus from Spice.
abs_transition_err	Reports the absolute slew difference of Litmus from Spice.

annotation	Reports RC parasitic annotations for net arcs, and delay annotations for either gate or net arcs. The following annotations are reported: • SDF: SDF back-annotation. The annotation status reflects the gate arc, not the interconnect arc. • SPEF: This entry appears if parasitic extraction has been done or annotated using SPEF. • LumpedRC: Lumped RC annotation • DlyAssert: Assertion set with set_annotated_delay command • WLM: Wire load model back-annotation • <none>: All others
aocv_adj_stages	Shows up "Aocv Adjust Stages" column. This column reports the AOCV adjustment stages that indicate the total effective stage count.
aocv_derate	Reports the graph-based AOCV derating factor used for cells/nets in AOCV mode. In retime mode, this column will report graph-based derating for cells/nets before the AOCV branch point and path-based derating.
aocv_weight	Reports AOCV weight values for the path elements.
arc	Reports the arc as described by the from pin, from pin edge, to pin, and to pin edge. For example, the arc from the rising edge of pin A to the falling edge of pin Z is reported as A -> Z.
arrival	Reports the arrival time on the pin.
arrival_mean	Reports mean of the arrival time on a pin.
arrival_sigma	Reports standard deviation of the arrival time on a pin.
cell	Reports the cell name of the given pin's instance.
delay	Reports the arc delay. If the stolen slack at the arc output pin (output of transparent latches) is not zero, the stolen column is also displayed along with this column.

delay_mean	Reports the mean of each arc's delay.
delay_sigma	Reports the standard deviation of each arc's delay. Note: In the absence of correlated and uncorrelated components of variation, the individual delay sigma values will not be summed up.
direction	Reports the pin direction (IN, OUT).
edge	Reports the edge of a pin (^=R, v=F).
fanin	Reports the number of source nodes of a net connected to a timing pin.
fanout	Reports the number of sinks of a net connected to a timing pin.
flags	Indicates the size_only, dont_use, dont_touch, and ideal attributes. When the 'flags' option is specified, a capacitance group entry (if it exists) will be displayed under the "Flags" column. This value indicates the input capacitance group being used for delay calculations – C1C2 (2-piece model) or C1CN (n-piece model).
hpin	Reports the hierarchical name for a given pin. Note: The hpin, pin, and timing_point will print the same values in the report output.
incr_delay	Reports the incremental delay of a pertaining arc. The software reads the incremental delay from an incremental SDF file using the read_sdf command.
instance	Reports the hierarchical name of a given pin's instance.
instance_location	Reports the location (x,y) of an instance. If a design is not placed, this column will be blank. This option displays the instance column automatically.

latch_window	Reports the available window for latch to sample the data.
load	Reports the total capacitance load on a given pin.
module	Reports the module name of the corresponding block hierarchy. This hierarchical module name will be reported only when the <code>-hpin</code> parameter of the <code>report_timing</code> command is specified.
net	Reports the hierarchical name of a net connected to a given pin.
pct_delay_err	Reports the percent delay difference of Litmus from Spice.
pct_slew_err	Reports the percent slew difference of Litmus from Spice.
phase	Reports the phase name of a pin.
phys_info	Shows minimum/maximum assigned layer, and non-default rule name information.
pin	Reports the reference name for the given pin.
pin_load	Reports the pin load data. The pin load refers to the sum of pin capacitance of all the pins connected to the current net.
pin_location	Reports the location (x,y) of a pin. If a design is not placed, this column will be blank.
power_domain	Reports power domain details of the specified path instance.
required	Reports the required time on a pin.
retime_delay	Reports the recalculated arc delay based on the path being reanalyzed with the specified retime method.
retime_delay_mean	Reports the mean of the recalculated arc delay on the path being reanalyzed with the specified retime method.
retime_mis_delta	Reports MIS delta delay in PBA mode for MIS-aware timing arcs in the path.

retime_mis_derate	Reports MIS derate values (in PBA mode), when derate-based advanced multi-input switching (AMIS-Derate) analysis is enabled.
retime_delay_sigma	Reports the standard deviation of each parameter of an arc's retimed delay.
retime_slew	Reports the recalculated slew based on the path being reanalyzed with the specified retime method.
retime_slew_mean	Reports the mean of each parameter of an arc sink pin's retimed slew.
retime_slew_sigma	Reports the standard deviation of each parameter of an arc sink pin's retimed slew.
stage_count	Displays the aggregate stage count of a specific instance in graph-based analysis (GBA) mode, and the actual stage count in path-based analysis (PBA) mode.
transition	Reports the propagated slew at the given pin.
transition_mean	Reports mean of each parameter of an arc sink pin's slew.
transition_sigma	Reports the standard deviation of each parameter of an arc sink pin's slew.
socv_derate	Reports the SOCV or the spatial derating factor applied on the cells or nets in SOCV analysis mode.
spice_arrival	Reports the Spice arrival time at the timing point.
spice_delay	Reports the Spice delay of a timing arc.
spice_slew	Reports the Spice slew of a timing point.
stolen	Reports the slack stolen (or the time given to the previous stage) at the given pin. If the slack is not zero and the <code>delay</code> column is specified, then this column is displayed by default (but only visible on the output of transparent latches).
timing_point	Reports the hierarchical names for the given points.

total_derate	Reports the total derating factor applied on the corresponding cell or nets that reflect the combined impact of all the derates (OCV/AOCV/spatial) applied on each stage of the timing path. Note: By default, the software will take into account the negative delays and keep the SOCV derate column in sync with the user-derate column. The software multiplies the negative delay by (2.0 - derating factor) to make it more pessimistic. However, reporting will always show derates based on the positive delay. The timing report output will show similar values for the SOCV Derate and Total Derate columns, but there will be no changes in the slack values.
user_derate	Reports the derating scale factors set with the <code>set_timing_derate</code> command. Note: The timing system cannot provide an accurate derating factor for the lumped gate and wire delay when the <code>set_timing_derate</code> factors for <code>-cell_delay</code> and <code>-net_delay</code> are not equivalent. In this case, an asterisk (*) is reported in the SSI Derate column.
voltage	Returns the voltage that is used during delay calculations.
vt_skew_derate	Returns voltage threshold (VT) skew analysis details.
when_cond	Reports the "when" condition of the arc specified in the library.
wire_load	Reports the wire load.
wlmodel	Reports the kind of parasitic used by a pin or net, such as SPEF. If the wire load model is derived from a Liberty file, the information is reported in the following format: <pre>Wireload model (.lib model_name library_name)</pre>

Reporting Timing Checks Between Various Types of Clocks

The following table summarizes the timing checks and the min/max delay checks that are performed based on the types of launching and capturing clocks:

- NZ (Non Zero Period Clock - Regular Clock)
- Z (Zero Period Clock - Regular clock whose period is equal to zero)
- @ (Asynchronous Clock - Clock which triggers the default input delay and the input delay without a clock)
- CPD (Combinational Path Delay)

Launching Clock	Capturing Clock	Timing Checks	Path Delay Checks
@	@	NO	CPD
Z	@	NO	CPD
NZ	@	NO	CPD
@	Z	YES	SPD
@	NZ	YES	SPD
NZ	Z	NO	SPD
Z	NZ	YES	SPD
Z	Z	YES	SPD
NZ	NZ	YES	SPD

Reports Sections

Path Summary Report

```
litmus > report_timing -path_type summary
```

A sample path summary report is shown below:

```
Path 1: VIOLATED (-16.913 ns) Setup Check with Pin CRE_IN/EXE_IN/p_reg[30]/CK->D
Group: m_clk
Startpoint: (R) TDSP_CORE_INST/DECODE_INST/ir_reg[9]/CK
Clock: (R) m_clk
Endpoint: (F) CRE_IN/EXE_IN/p_reg[30]/D
Clock: (R) m_clk
```

	Capture	Launch
Clock Edge:+	4.000	0.000
Src Latency:+	0.000	0.000
Net Latency:+	4.206 (P)	4.101 (P)
Arrival:=	8.206	4.101
Setup:-	1.251	
Uncertainty:-	0.400	

```
Cppr Adjust:+      0.000
Required Time:=    6.555
Launch Clock:=     4.101
Data Path:+       19.367
Slack:=           -16.913
```

Advanced Summary Section

The 'advanced summary' section shows specific information, such as, slack analysis, SI pessimism. This section provides cell-by-cell information on the path as well as nets, hierarchical pins, exceptions, and other aspects of the timing path.

A sample advanced summary report is shown below.

```
Path 1: MET (2285 ps) Setup Check with Pin rt/mul_73_17/retime_s5_16_reg/CK->D
. . .
Exceptions/Constraints:
path_adjust      370      scripts/constraints.sdc_line_57
input_delay      1001     scripts/constraints.sdc_line_27

Slack Analysis(3):
Stage(-3): (2123.60 ps) rt/mul_73_17/retime_s2_16_reg/D
Stage(-2): (2210.60 ps) rt/mul_73_17/retime_s3_16_reg/D
Stage(-1): ( 348.60 ps) rt/mul_73_17/retime_s4_16_reg/D
Stage( 0): (2285.30 ps) rt/mul_73_17/retime_s5_16_reg/D
Stage( 1): (-816.00 ps) rt/mul_73_17/retime_s6_9_reg/D
Stage( 2): (1283.90 ps) o6[15]
. . .
```

Note: These reports may contain content that has substantial runtime impact (such as meta-data, etc.).

Path Description Section

The path description section shows the cell-by-cell delay, slew, arc, exception information, and informational flag legends.

A sample path description section is shown below.

```
#-----
# Timing Point          Arc      Edge      Cell      Load      Trans      Delay      Arrival
#                      #              (fF)      (ps)      (ps)      (ps)
#-----
FF_I1_reg[0]/CK        <<< -      R          -          -          150        -          0
FF_I1_reg[0]/Q          CK->Q      R          DFFHQX4     45.4       201        334        334
mul_26_28/Fn0755D13176/Y A->Y      F          INVX2       15.7       103        68         402
mul_26_28/Fn0755D13172/Y A->Y      R          INVX4       53.0       223        130        532
mul_26_28/n0617DTtn0525/Y B->Y      F          NAND2X1     10.9       169        108        640
mul_26_28/n0634Dtn0548/Y A1N->Y    F          AOI2BB2X4   19.3       164        232        872
mul_26_28/Fn0612D/Y     A->Y      R          INVX2       18.5       183        119        992
. . .
mul_26_43/n0014Dtn0331/Y A->Y      F          NAND2X2     17.3       147        94         4168
mul_26_43/Fn0013D16832/Y A->Y      R          INVX1       5.6        145        97         4265
mul_26_43/n0078Dtn0331/Y A1N->Y    R          OAI2BB1X2   18.9       201        203        4468
```

```
mul_26_43/n0064Dtn0331/Y  A->Y    R      XOR2X4      10.6    86    225    4693
mul_26_43/n0025Dtn0331/S  CI->S    F      ADDFHX4      5.2     82    269    4962
FF_O2_reg[0]/D             <<< -    F      DFFHQX2      -       -     0     4962
#-----
```

By default, the timing reports show the 'path summary' and 'path description' section. However, the default can be changed by using the `report_timing -path_type` parameter.

Endpoint Reports

In addition to detailed reports, endpoint reports can also be generated. These reports allow you to include additional endpoint related information that can be useful in debugging or identifying certain issues. For example, an endpoint report can show the number of buffers and inverters on a path. A sample report is as shown:

```
litmus > report_timing -path_type endpoint
#-----
# ID      Slack    Endpoint              Group    View
#          (ns)
#-----
1    -16.913  CRE_IN/p_rg[30]/D    m_clk    def_em_view1
#-----
```

The endpoint reports and detailed reports are mutually exclusive.

Clock Tracing Reports

Optionally, the report includes the clock tracing section that helps in debugging issues pertaining to clocks used by the startpoint and/or endpoint of a path. Clock tracing traces the source of the clock and the detailed path taken to reach the launch and/or capture elements' clock pin. A sample report is shown below.

```
> report_timing -view func1@wc_cworst -path_type full_clock
    -from ff1_reg -to ff3_reg
    -fields {-fanout}
```

```
Path 1: MET (405 ps) Setup Check with Pin ff3_reg/CK->D
    View: func1@wc_cworst
    Group: ck2
    Startpoint: (R) ff1_reg/CK
    Clock: (R) ck1
    Endpoint: (R) ff3_reg/D
    Clock: (R) ck2
```

	Capture	Launch
Clock Edge:+	1000	0
Src Latency:+	0	0
Net Latency:+	0 (I)	0 (I)
Arrival:=	1000	0
Setup:-	61	
Required Time:=	939	
Launch Clock:-	0	
Data Path:-	595	
Slack:=	405	

Litmus Text Command Reference

Timing Analysis Commands--report_timing

```
#-----
```

# Timing Point	Arc	Edge	Cell	Load (fF)	Trans (ps)	Delay (ps)	Arrival (ps)
#							
#-----							
clk1	-	R	(port)	0.9	0	0	0
clk1_buf1/Y	A->Y	R	BUFX4	0.9	24	60	60
clk1_buf2/Y	A->Y	R	BUFX4	0.0	0	67	127
ff1_reg/CK	-	R	DFFX1	-	0	0	127
#-----							
ff1_reg/QN <<<	CK->QN	R	DFFX1	0.6	23	335	462
ff1_i2_and/Y	B->Y	R	AND2X2	1.2	45	133	595
ff3_reg/D <<<	-	R	DFFX4	-	-	0	595
#-----							
clk2	-	R	(port)	0.9	0	0	1000
clk2_buf1/Y	A->Y	R	BUFX4	2.8	31	64	1164
clk2_buf2/Y	A->Y	F	INVX4	2.8	22	24	1188
clk2_buf3/Y	A->Y	R	INVX4	0.0	0	14	1102
ff3_reg/CK	-	R	DFFX4	-	-	0	1102
#-----							

PBA Header

- In PBA, EPBA, and IPBA mode, the timing report shows PBA header for columns (by default):

```
report_timing -retime path_slew_propagation -path_type full_clock
```

```
Path 1: VIOLATED Late External Delay Assertion
```

```
Endpoint: o5 (^) checked with leading edge of 'CLK'
```

```
Beginpoint: ff3/Q (^) triggered by leading edge of 'CLK'
```

```
Analysis View: an_view_setup
```

```
Retime Analysis {Data Clock Path-Slew Spatial(CS) SOCV}
```

```
Retime SOCV Analysis { Distance(um) : 27.653 }
```

	NSigma		Mean		Sigma		Stddev
Other End Arrival Time	0.0000 (-3.00S)		0.0000		0.0000		0.0000
- External Delay	2.5000						
+ Phase Shift	0.5000						
+ CPPR Adjustment	0.0000 (+3.00S)		0.0000		0.0000		0.0000
- Uncertainty	0.5000						
= Required Time	-2.5000 (-3.00S)		-2.5000		0.0000		0.0000
- Arrival Time	18.4496 (+3.00S)		18.4486		0.0004		0.0004
= Slack Time	-20.9496 (-3.00S)		-20.9486		0.0004		0.0004
= Slack Time(original)	-20.9484 (-3.00S)		-20.9484		0.0000		0.0000
Clock Rise Edge	0.0000						
+ Source Insertion Delay	1.0000 (+3.00S)		1.0000		0.0000		0.0000
= Beginpoint Arrival Time	1.0000 (+3.00S)		1.0000		0.0000		0.0000

Litmus Text Command Reference
Timing Analysis Commands--report_timing

Pin		Phase		Edge	Delay	Delay Mean	Delay Stddev	Retime
Delay	Retime	Delay	Retime	Delay	Arrival	Time		
Mean		Stddev		-----				

clk		CLK(C) (P) (an_view_setup)*		^	-	-	-	-
-		-			1.0000			
clk_L1_I0/A		CLK(C) (P) (an_view_setup)*		^	0.0004	0.0004	0.0000	0.0014
0.0005		0.0003			1.0014			
clk_L1_I0/Y		CLK(C) (P) (an_view_setup)*		^	0.0600	0.0600	0.0000	0.0600
0.0600		0.0000			1.0613			
clk_L2_I0/A		CLK(C) (P) (an_view_setup)*		^	0.0001	0.0001	0.0000	0.0004
0.0002		0.0001			1.0613			
clk_L2_I0/Y		CLK(C) (P) (an_view_setup)*		^	0.0604	0.0604	0.0000	0.0604
0.0604		0.0000			1.1217			
ff3/CLK		CLK(C) (P) (an_view_setup)*		^	0.0001	0.0001	0.0000	0.0011
0.0001		0.0003			1.1215			
ff3/Q		CLK(D) (P) (an_view_setup)*		^	17.3092	17.3092	0.0000	
17.3091		17.3091			0.0000			18.4306
o5		CLK(D) (P) (an_view_setup)		^	0.0182	0.0182	0.0000	0.0197
0.0183		0.0005			18.4496			

Examples

- The following command reports paths leading to the pin(s) specified by the *object_list*:

```
litmus > report_timing -to CRE_INST/EXE_INST/p_reg[31]/D

Path 1: VIOLATED Setup Check with Pin CRE_INST/EXE_INST/p_reg[31]/CK
Endpoint: CRE_INST/EXE_IN/p_reg[31]/D (v) checked with leading edge of 'm_clk'
Beginpoint: CRE_INST/DE_INST/ir_reg[10]/Q (^) triggered by leading edge of 'm_clk'
Path Groups: {m_clk}
Other End Arrival Time          5.272
- Setup                        1.298
+ Phase Shift                   4.000
```

Litmus Text Command Reference
Timing Analysis Commands--report_timing

```
- Uncertainty                0.400
= Required Time              7.574
- Arrival Time              38.446
= Slack Time                -30.871
Clock Rise Edge             0.000
+ Clock Network Latency (Prop) 5.348
= Beginpoint Arrival Time    5.348
```

```
-----
Instance                      Arc          Cell          Delay    Arrival Time  Required Time
-----
CRE_INST/DE_INST/ir_reg[10]   CK ^      -            -         5.348         -25.523
CRE_INST/DE_INST/ir_reg[10]   CK ^ -> Q ^ SDFS_X2     0.639        5.987         -24.884
CRE_INST/DE_INST/FE_prt_2     A ^ -> Z ^  BUF_X16     1.477        7.464         -23.407
CRE_IN/GL_IN/lsh_47/F0D26     A ^ -> ZN v INV_X4     1.833        9.297         -21.574
CRE_IN/GL_IN/lsh_47/p0088D    A2 v -> ZN v AND2_X2     1.170       10.467         -20.404
CRE_IN/GL_IN/lsh_47/g9132     A1 v -> ZN v AND2_X2     0.336       10.803         -20.068
CRE_IN/GL_IN/lsh_47/FE_661    A v -> Z v  BUF_X32     0.341       11.144         -19.727
CRE_IN/GL_IN/p0171A          A1 v -> ZN ^ NAND2_X4     0.230       11.375         -19.497
CRE_IN/GL_IN/p0089D          A3 ^ -> ZN ^ AND3_X2     0.385       11.760         -19.111
CRE_IN/GL_IN/p0053D          A1 ^ -> ZN v NAND2_X4     0.417       12.176         -18.695
-----
```

- The following command shows a report similar to the first with the addition of net arc information:

```
litmus > report_timing -to CRE_INST/EXE_INST/p_reg[31]/D -net
```

```
Path 1: VIOLATED Setup Check with Pin CRE_INST/EXE_INST/p_reg[31]/CK
```

```
Endpoint: CRE_INST/EXE_INST/p_reg[31]/D (v) checked with leading edge of 'm_clk'
```

```
Beginpoint: CRE_INST/DE_INST/ir_reg[10]/Q (^) triggered by leading edge of 'm_clk'
```

```
Path Groups: {m_clk}
```

```
Other End Arrival Time      5.272
- Setup                    1.298
+ Phase Shift              4.000
- Uncertainty              0.400
= Required Time            7.574
- Arrival Time            38.446
= Slack Time              -30.871
Clock Rise Edge           0.000
+ Clock Network Latency (Prop) 5.348
= Beginpoint Arrival Time    5.348
```

```
-----
Pin                      Edge  Net                      Cell          Delay    Arrival    Required
                                           Time      Time
-----
```


Litmus Text Command Reference
Timing Analysis Commands--report_timing

CRE_INS/DDE/ir_reg[10]/CK	^	m_clk__L19_N6	-	-	5.348	-25.523
CRE_INS/DDE/ir_reg[10]/Q	^	CRE_INS/DDE/FE_9_ad	SDFS_X2	0.639	5.987	-24.884
CRE_INS/DDE/OFC3_prt_add/A	^	CRE_INS/DDE/FE_9_ad	BUF_X16	0.000	5.987	-24.884
CRE_INS/DDE/OFC3_prt_add/Z	^	port_address[2]	BUF_X16	1.477	7.464	-23.407
CRE_IN/GL/lsh_47/Fn96/A	^	port_address[2]	INV_X4	0.045	7.509	-23.362
CRE_IN/GL/lsh_47/Fn96/ZN	v	CRE_IN/GL/ls_47/n_3	INV_X4	1.789	9.297	-21.574
CRE_IN/GL/lsh_47/p8D/A2	v	CRE_IN/GL/ls_47/n_3	AND2_X2	0.000	9.298	-21.573
CRE_IN/GL/lsh_47/p8D/ZN	v	CRE_IN/GL/ls_47/n_16	AND2_X2	1.170	10.467	-20.404
CRE_IN/GL/lsh_47/g32/A1	v	CRE_IN/GL/ls_47/n_16	AND2_X2	0.000	10.468	-20.404
CRE_IN/GL/lsh_47/g32/ZN	v	CRE_IN/GL/ls_47/FE_61	AND2_X2	0.336	10.803	-20.068
CRE_IN/GL/lsh_47/FE_661/A	v	CRE_IN/GL/ls_47/FE_61	BUF_X32	0.000	10.803	-20.068
CRE_IN/GL/lsh_47/FE_661/Z	v	CRE_IN/GL/n_661	BUF_X32	0.341	11.144	-19.727
CRE_IN/GL/p0171A/A1	v	CRE_IN/GL/n_661	NAND2_X4	0.000	11.144	-19.727
CRE_IN/GL/p0171A/ZN	^	CRE_IN/GL/n_75	NAND2_X4	0.230	11.375	-19.497
CRE_IN/GL/p0089D/A3	^	CRE_IN/GL/n_75	AND3_X2	0.000	11.375	-19.496
CRE_IN/GL/p0089D/ZN	^	CRE_IN/GL_IN/n_223	AND3_X2	0.385	11.760	-19.111

- The `report_timing` command accepts `-collection` as an argument. For example,

```
report_timing [report_timing -collection]
set rpt [report_timing -collection]
report_timing $rpt
```

- The following command displays the worst late path to each violating endpoint that has a slack less than -1.0:

```
report_timing -max_slack -1.0
```

- The following command displays all the late paths that end at port `out[2]` and that have negative slack up to a maximum of 1000 worst paths. If there are more than 1000 paths, only the 1000 worst paths are reported:

```
report_timing -to out[2] -max_paths 1000 -max_slack 0.0
```

- The following command reports the worst late path that starts at `in[0]` and ends at `out[1]`:

```
report_timing -from in[0] -to out[1]
```

- The following command displays the three worst paths that start at `in[1]` and end at `out[3]`. With reconvergent fanout, more than one path may exist:

```
report_timing -from in[1] -to out[3] -max_paths 3
```

- The following command displays the ten worst paths. Only the paths between the specified pins are enumerated. With reconvergent fanout, more than one path may exist:

```
report_timing -from i102/Z -to i123/A -max_paths 10
```

- The following command enumerates the worst ten paths through the given two pins, starting at the beginning points in the design and ending at the endpoints, similar to using the `-from` option:

```
report_timing -through i102/Z -through i123/A -max_paths 10
```

- The following command reports the ten worst paths through blocks A and C, or blocks B and C. The path only has to satisfy one element in a through list, but all through lists must be satisfied. The example can be thought of as ((A or B) and C):

```
report_timing -through {A/*B/*} -through {C/*} -max_paths 10
```

- The following command forces a path through pin A and pin B. Use the syntax shown in the example. Do not use `-through {A B}`:

```
report_timing -through A -through B
```

- The following command displays the clock path through the clock root to Launch flop:

```
report_timing -through aBC_bs/bs_mex1/r2_00_q1_reg_10_/CK -to .... -path_type full_clock
```

- You will not see the path for the launch clock if you specify the following command because the `report_timing` command starts reporting from the CK pin:

```
report_timing -from aBC_bs/bs_mex1/r2_00_q1_reg_10_/CK
```

- The first command displays the path leading to the `CK` pin.

- The following command shows the report generated using the `-unconstrained` parameter:

```
report_timing -unconstrained
```

```
-----
Path 1:Endpoint:   RAM_256x16_IN/g417/A1 (^) (unconstrained output)
Beginpoint: SP_CRE/DE_IN/ir_reg[9]/Q (v) triggered by leading edge of 'm_clk'
Arrival Time                23.760
+ Clock Network Latency (Prop) 5.348
= Beginpoint Arrival Time      5.348
-----
```

Instance	Arc	Cell	Delay	Arrival Time

SP_CRE/DE_IN/ir_reg[9]	CK ^	-	-	5.348
SP_CRE/DE_IN/ir_reg[9]	CK ^ -> Q v	SDFS_X2	0.687	6.035
SP_CRE/DE_IN/OFC186_pt_ad	A v -> Z v	BUF_X16	0.527	6.562
SP_GLE_IN/p8211A	A3 v -> ZN ^	NOR3_X1	0.929	7.492
SP_GLE_IN/p8090A	A2 ^ -> ZN v	NAND2_X1	0.361	7.852
SP_GLE_IN/p6871A	A4 v -> ZN ^	NOR4_X2	1.873	9.725
SP_GLE_IN/p785133	A1 ^ -> ZN v	NAND2_X4	0.528	10.253
SP_GLE_IN/p7333D	A2 v -> ZN ^	NOR2_X4	3.227	13.480
SP_GLE_IN/p7A6115	B1 ^ -> ZN v	AOI222_X1	1.576	15.055

Litmus Text Command Reference
Timing Analysis Commands--report_timing

SP_GLE_IN/p850085	A2 v -> ZN ^	NAND3_X4	1.098	16.154
OFC539_tsp_data_out	A ^ -> Z ^	BUF_X16	1.302	17.456
SM_MUX_IN/p9662A497	B1 ^ -> ZN v	AOI22_X1	0.792	18.248
SM_MUX_IN/OFC17_n_23	A v -> ZN ^	INV_X4	3.115	21.363
SM_MUX_IN/OFC18_n_23	A ^ -> ZN v	INV_X4	0.338	21.700
SM_MUX_IN/C594_OFN18_n_23	A v -> Z v	BUF_X32	0.867	22.567
SM_MUX_IN/Fp9546A496	A v -> ZN ^	INV_X1	0.247	22.815
OFC16_ds_datain_0	A ^ -> Z ^	BUF_X8	0.944	23.759
RAM_256x16_IN/g417	A1 ^	AOI22_X1	0.001	23.760

- The following command sets a false path from CRE_IN/DE_IN/ir_reg[9]/Q to CRE_IN/EXE_IN/p_reg[31]/D:

```
litmus > set_false_path -from CRE_IN/DE_IN/ir_reg[9]/Q -to CRE_IN/EXE_IN/p_reg[31]/D
litmus > set_false_path -through CRE_IN/DE_IN/ir_reg[11]/Q
litmus > report_timing -from CRE_IN/DE_IN/ir_reg[11]/Q -to CRE_IN/EXE_IN/p_reg[25]/D -
unconstrained -path_exceptions all
```

```
Path 1:Endpoint:    CRE_IN/EXE_IN/p_reg[31]/D (^) (unconstrained output)
Beginpoint: CRE_IN/DE_IN/ir_reg[10]/Q (^) triggered by  leading edge of 'm_clk'
Arrival Time                39.024
+ Clock Network Latency (Prop)  5.348
= Beginpoint Arrival Time      5.348
```

Instance	Arc	Cell	Delay	Arrival Time
CRE_IN/DE_IN/ir_reg[10]	CK ^	-	-	5.348
CRE_IN/DE_IN/ir_reg[10]	CK ^ -> Q ^	SDFFS_X2	0.639	5.987
CRE_GLE_INST/p0089D	A3 ^ -> ZN ^	AND3_X2	0.385	11.760
CRE_GLE_INST/p0053D	A1 ^ -> ZN v	NAND2_X4	0.417	12.176
CRE_INST/MPY_32/FE_RC_514_0	A v -> ZN ^	INV_X32	0.271	12.448
CRE_INST/MPY_32/p0258A	A2 ^ -> ZN v	NAND2_X4	0.200	12.648
CRE_INST/MPY_32/Fp0229A	A v -> ZN ^	INV_X8	0.460	13.108
CRE_INST/MPY_32/p0287A	A1 ^ -> ZN v	NAND2_X1	0.261	13.369
CRE_INST/MPY_32/p0073D	A2 v -> ZN ^	NAND2_X4	0.320	13.689
CRE_INST/MPY_32/p0137D	A1 ^ -> ZN v	NAND2_X1	0.215	13.904
CRE_INST/MPY_32/p0041D	A2 v -> ZN ^	NAND2_X4	1.214	15.118
CRE_INST/MPY_32/Fp2263A	A ^ -> ZN v	INV_X4	0.968	34.647
CRE_IN/EXE_IN/n1456D	B1 ^ -> ZN v	AOI22_X1	0.355	38.297
CRE_IN/EXE_IN/n1510D	A v -> ZN ^	INV_X8	0.728	39.024
CRE_IN/EXE_IN/p_reg[31]	D ^	SDFF_X2	0.000	39.024

Applied exceptions:

From	Through	To	Late
------	---------	----	------

```

-----
-
CRE_IN/DE_IN/ir_reg[10]/CK      -                               CRE_IN/EXE_IN/p_reg[31]/D      false
-                               CRE_IN/EXE_IN/n1510D/A      -                               false
-----

```

- For hold paths, if the path has a setup multi-cycle path then those exceptions will also be reported because hold slacks get impacted by inferred path exceptions. For example:

```

litmus > set_multicycle_path 2 -setup -from CRE_IN/DE_IN/ir_reg[9]/CK
litmus > report_timing -from CRE_IN/DE_IN/ir_reg[10]/CK

```

With multi-cycle path:

```

Path 1: VIOLATED (-26.872 ns) Setup Check with Pin CRE_IN/EXE_IN/p_reg[31]/CK->D Group:m_clk
Startpoint: (R) CRE_IN/DE_IN/ir_reg[10]/CK Clock: (R) m_clk
Endpoint: (F) CRE_IN/EXE_IN/p_reg[31]/D
Clock: (R)m_clk
N-Sigma: 3.000

```

	Capture	Launch
Clock Edge:+	8.000	0.000
Src Latency:+	0.000	0.000
Net Latency:+	5.272 (P)	5.348 (P)
Arrival:=	13.272	5.348

```

Setup:- 1.298
Uncertainty:- 0.400
Cpnr Adjust: 0.001
Required Time:= 11.575
Launch Clock:= 5.348
Data Path: 33.099
Slack:= -26.872

```

Without multi-cycle path:

```

Path 1: VIOLATED (-30.872 ns) Setup Check with Pin CRE_IN/EXE_IN/p_reg[31]/CK->D Group:m_clk
Startpoint: (R) CRE_IN/DE_IN/ir_reg[10]/CK Clock: (R) m_clk
Endpoint: (F) CRE_IN/EXE_IN/p_reg[31]/D
Clock: (R)m_clk
N-Sigma: 3.000

```

	Capture	Launch
Clock Edge:+	4.000	0.000
Src Latency:+	0.000	0.000
Net Latency:+	5.272 (P)	5.348 (P)
Arrival:=	9.272	5.348

```

Setup:- 1.298

```

Litmus Text Command Reference
Timing Analysis Commands--report_timing

```
Uncertainty:-      0.400
Cpnr Adjust:+      0.001
Required Time:=    7.575
Launch Clock:=     5.348
Data Path:+        33.099
Slack:=            -30.872
```

- The following command displays the timing report for analysis view `setup_view1`:

```
litmus > report_timing -view setup_view1
```

```
Path 1: VIOLATED (-30.660 ns) Setup Check with Pin CRE_IN/EXE_IN/p_reg[31]/CK->D
```

```
Group: m_clk
```

```
Startpoint: (R) CRE_IN/DE_IN/ir_reg[11]/CK
```

```
Clock: (R) m_clk
```

```
Endpoint: (F) CRE_IN/EXE_IN/p_reg[31]/D
```

```
Clock: (R) m_clk
```

```
N-Sigma: 3.000
```

	Capture	Launch
Clock Edge:+	4.000	0.000
Src Latency:+	0.000	0.000
Net Latency:+	5.272 (P)	5.300 (P)
Arrival:=	9.272	5.300

```
Setup:-      1.298
Uncertainty:- 0.400
Cpnr Adjust:+ 0.001
Required Time:= 7.575
Launch Clock:= 5.300
Data Path:+    32.935
Slack:=        -30.660
```

```
#-----
#
# Timing Point                               Flags  Arc    Edge  Cell        Fanout  Trans  Delay  Arrival
#                                           (ns)   (ns)   (ns)
#-----
CRE_IN/DE_IN/ir_reg[11]/CK      -      CK     R      (arrival)  4        0.139    -      5.300
CRE_IN/DE_IN/ir_reg[11]/Q      -      CK->Q  F      SDFS_X2    1        0.125    0.651  5.951
CRE_IN/MPY_32/p0258A/ZN        -      A2->ZN R      NAND2_X4   3        0.397    0.460  12.676
CRE_IN/MPY_32/Fp0229A/ZN       -      A->ZN  F      INV_X8     5        0.217    0.254  12.929
CRE_IN/MPY_32/Fp2263A/ZN       -      A->ZN  F      INV_X4     6        1.134    0.966  34.437
CRE_IN/MPY_32/inc_ad4/p0A/ZN   -      A1->ZN R      NAND2_X1   2        0.388    0.333  36.458
```

Litmus Text Command Reference
Timing Analysis Commands--report_timing

```

CRE_IN/MPY_32/FE_RC_18_0/ZN      -      A1->ZN  F      NAND2_X4   1      0.231   0.177   37.166
CRE_IN/EXE_IN/n1456D/ZN          -      B1->ZN  R      AOI22_X1   1      1.069   0.734   37.899
CRE_IN/EXE_IN/p_reg[31]/D        -      D        F      SDFX_X2    1      0.413   0.000   38.235

```

```

#-----
-

```

- The following command shows report when -point_to_point parameter is used:

```

litmus > report_timing -to CRE_IN/EXE_IN/p_reg[31]/D -from CRE_IN/DE_IN/ir_reg[11]/Q -
point_to_point

```

```

Path 1:
Startpoint: (F) CRE_IN/DE_IN/ir_reg[11]/Q
Clock: (R) m_clk
Endpoint: (R) CRE_IN/EXE_IN/p_reg[31]/D
Clock:
N-Sigma: 3.000

```

	Capture	Launch
Src Latency:+	0.000	0.000
Net Latency:+	0.000 (I)	0.000 (I)
Arrival:=	0.000	0.000
Phase Shift:+	0.000	
Launch Clock:=	0.000	
Data Path:+	32.862	

```

#-----
-----

```

# Timing Point	Flags	Arc	Edge	Cell	Fanout	Trans	Delay
Arrival							
#						(ns)	(ns)
(ns)							

```

#-----
-----

```

CRE_IN/DE_IN/ir_reg[11]/Q	<<<	Q	F	(arrival)	1	0.125	0.000
0.000							
CRE_IN/DE_IN/FE_OFC192_ir_11_/Z	-	A->Z	F	BUF_X16	25	0.385	0.498
0.498							
CRE_IN/MY_32_IN/p0258A/ZN	-	A2->ZN	R	NAND2_X4	3	0.397	0.460
6.724							
CRE_IN/MY_32_IN/Fp0229A/ZN	-	A->ZN	F	INV_X8	5	0.217	0.254
6.978							
CRE_IN/MY_32_IN/Fp2263A/ZN	-	A->ZN	F	INV_X4	6	1.134	0.966
28.486							
CRE_IN/MY_32_IN/FE_RC_1689_0/ZN	-	A1->ZN	F	NAND2_X4	1	0.705	0.243
31.313							

Litmus Text Command Reference Timing Analysis Commands--report_timing

```
CRE_IN/EXE_IN/n1456D/ZN          -      B1->ZN      F      AOI22_X1      1      1.085      0.355
32.135
CRE_IN/EXE_IN/n1510D/ZN          -      A->ZN      R      INV_X8      1      0.609      0.727
32.862
```

```
#-----
-----
```

- The following command reports pin names and net names in the same column alternatively:

```
litmus > report_timing -fields {load slew delay arrival cell pin_load fanout timing_point} -
net
```

```
Path 1: VIOLATED (-37.450 ns) Setup Check with Pin CRE/EXE_IN/p_reg[31]/CK->D
Group: m_clk
Startpoint: (R) CRE/DE_IN/ir_reg[10]/CK
Clock: (R) m_clk
Endpoint: (F) CRE/EXE_IN/p_reg[31]/D
Clock: (R) m_clk
```

	Capture	Launch
Clock Edge:+	4.000	0.000
Src Latency:+	0.000	0.000
Net Latency:+	6.337 (P)	6.408 (P)
Arrival:=	10.337	6.408
Setup:-	1.298	
Uncertainty:-	0.400	
Cppr Adjust:+	0.001	
Required Time:=	8.640	
Launch Clock:=	6.408	
Data Path:+	39.681	
Slack:=	-37.450	

```
#-----
```

#	Load	Trans	Delay	Arrival	Cell	Pin Load	Fanout	Timing Point
#	(pf)	(ns)	(ns)	(ns)				
#								
0.005	0.145	-	6.408	(arrival)	0.003	4		CRE/DE_IN/ir_reg[10]/CK
0.001	0.145	0.703	7.112	SDFS_X2	0.000	1		CRE/DE_IN/ir_reg[10]/Q
0.001	-	-	-	(net)	0.000	1		CRE/DE_IN/O13_pt_add_2
0.081	0.108	1.835	8.947	BUF_X16	0.021	45		CRE/DE_IN/OF3_prt_add_2/Z
0.081	-	-	-	(net)	0.021	45		port_address[2]
0.022	2.592	2.147	11.094	INV_X4	0.009	16		CRE_GLU_IN/ls_47/Fn26/ZN
0.022	-	-	-	(net)	0.009	16		CRE_GLU_IN/ls_47/n_3
0.005	1.541	1.404	12.497	AND2_X2	0.003	5		CRE_GLU_IN/ls_47/p08D/ZN
0.005	-	-	-	(net)	0.003	5		CRE_GLU_IN/ls_47/n_16
0.001	0.221	0.403	12.900	AND2_X2	0.000	1		CRE_GLU_IN/ls_47/g932/ZN
0.001	-	-	-	(net)	0.000	1		CRE_GLU_IN/ls_47/FE
0.003	1.069	0.403	46.090	INV_X8	-	1		CRE/EXE_IN/n1510D/ZN
0.003	-	-	-	(net)	0.002	1		CRE/EXE_IN/n_680
0.003	0.413	0.000	46.090	SDFS_X2	-	1		CRE/EXE_IN/p_reg[31]/D
#								

- The following command reports paths through unique set of pins:

```
litmus > Report_timing -nworst 2 unique_pins
```

Path 1: VIOLATED (-30.645 ns) Setup Check with Pin CRE_IN/EXE_IN/p_reg[25]/CK->D

Group: m_clk

Startpoint: (R) CRE_IN/DE_IN/ir_reg[11]/CK

Clock: (R) m_clk

Endpoint: (F) CRE_IN/EXE_IN/p_reg[25]/D

Clock: (R) m_clk

	Capture	Launch
Clock Edge:+	4.000	0.000
Src Latency:+	0.000	0.000
Net Latency:+	5.274 (P)	5.300 (P)
Arrival:=	9.274	5.300

Setup:-	1.282
Uncertainty:-	0.400
Cppr Adjust:+	0.001
Required Time:=	7.592
Launch Clock:=	5.300
Data Path:+	32.937
Slack:=	-30.645

#-----									
# Timing Point	Flags	Arc	Edge	Cell	Fanout	Trans	Delay	Arrival	
#						(ns)	(ns)	(ns)	
#-----									
CRE_IN/DE_IN/ir_reg[11]/CK	-	CK	R	(arrival)	4	0.139	-	5.300	
CRE_IN/DE_IN/ir_reg[11]/Q	-	CK->Q	F	SDFFS_X2	1	0.139	0.651	5.952	
CRE_IN/DE_IN/FE_OFCl11_/Z	-	A->Z	F	BUF_X16	25	0.125	0.498	6.450	
CRE_IN/GL_IN/lsh_47/Fn05/ZN	-	A->ZN	R	INV_X4	21	0.386	2.155	8.604	
CRE_IN/Fp0074D/ZN	-	A->ZN	R	INV_X8	1	0.190	0.183	13.635	
CRE_IN/p0052D/ZN	-	A1->ZN	F	NAND2_X1	1	0.151	0.146	13.781	
CRE_IN/EXE_IN/p0136A/ZN	-	B1->ZN	R	AOI22_X1	1	0.525	0.741	37.840	
CRE_IN/EXE_IN/p0059A/ZN	-	A->ZN	F	INV_X4	1	0.792	0.398	38.237	
CRE_IN/EXE_IN/p_reg[25]/D	-	D	F	SDFF_X2	1	0.372	0.000	38.237	
#-----									

Litmus Text Command Reference
Timing Analysis Commands--report_timing

Path 2: VIOLATED (-30.574 ns) Setup Check with Pin CRE_IN/EXE_IN/p_reg[25]/CK->D

Group: m_clk

Startpoint: (R) CRE_IN/DE_IN/ir_reg[11]/CK

Clock: (R) m_clk

Endpoint: (F) CRE_IN/EXE_IN/p_reg[25]/D

Clock: (R) m_clk

Capture	Launch	
Clock Edge:+	4.000	0.000
Src Latency:+	0.000	0.000
Net Latency:+	5.274 (P)	5.300 (P)
Arrival:=	9.274	5.300
Setup:-	1.282	
Uncertainty:-	0.400	
Cppr Adjust:+	0.001	
Required Time:=	7.592	
Launch Clock:=	5.300	
Data Path:+	32.866	
Slack:=	-30.574	

#-----									
# Timing Point	Flags	Arc	Edge	Cell	Fanout	Trans (ns)	Delay (ns)	Arrival (ns)	
#									
#-----									
CRE_IN/DE_IN/ir_reg[11]/CK	-	CK	R	(arrival)	4	0.139	-	5.300	
CRE_IN/DE_IN/ir_reg[11]/Q	-	CK->Q	F	SDFFS_X2	1	0.139	0.651	5.952	
CRE_IN/DE_IN/FE_OFCr_11_/Z	-	A->Z	F	BUF_X16	25	0.125	0.498	6.450	
CRE_IN/GL_IN/lh_47/p08D/ZN	-	A1->ZN	R	AND2_X2	5	3.448	1.144	9.748	
CRE_IN/Fp0161D4462/ZN	-	A->ZN	F	INV_X32	3	0.423	0.132	12.690	
CRE_IN/p0161D/ZN	-	A1->ZN	F	NOR2_X4	4	0.421	0.238	13.277	
CRE_IN/p0278A/ZN	-	A1->ZN	R	NAND2_X1	1	0.208	0.286	13.562	
CRE_IN/M16_IN/ml_14/Fn5D/ZN	-	A->ZN	F	INV_X32	28	1.763	0.760	16.037	
CRE_IN/Fp3A/ZN	-	A->ZN	F	INV_X4	6	2.661	0.966	34.366	
CRE_IN/p3A/ZN	-	B1->ZN	R	AOI22_X1	1	0.407	0.853	36.608	
CRE_IN/EXE_IN/p9A/ZN	-	A->ZN	F	INV_X4	1	0.792	0.398	38.166	
CRE_IN/EXE_IN/p_reg[25]/D	-	D	F	SDFF_X2	1	0.372	0.000	38.166	

- The following command reports all the violating paths (or having slack greater than the value specified with -max_slack) between unique source and target register pairs:

```
litmus > report_timing -to CRE_IN/EXE_IN/p_reg[25]/D -begin_end_pair -max_slack 10
```

Path 1: VIOLATED (-30.645 ns) Setup Check with Pin CRE_IN/EXE_IN/p_reg[25]/CK->D

Group: m_clk

Startpoint: (R) CRE_IN/DE_IN/ir_reg[11]/CK

Clock: (R) m_clk

Endpoint: (F) CRE_IN/EXE_IN/p_reg[25]/D

Clock: (R) m_clk

	Capture	Launch
Clock Edge:+	4.000	0.000
Src Latency:+	0.000	0.000
Net Latency:+	5.274 (P)	5.300 (P)
Arrival:=	9.274	5.300

Setup:- 1.282

Uncertainty:- 0.400

Cpwr Adjust: 0.001

Required Time:= 7.592

Launch Clock:= 5.300

Data Path: 32.937

Slack:= -30.645

# Timing Point	Flags	Arc	Edge	Cell	Fanout	Trans	Delay	Arrival
#						(ns)	(ns)	(ns)
CRE_IN/DE_IN/ir_reg[11]/CK	-	CK	R	(arrival)	4	0.139	-	5.300
CRE_IN/DE_IN/ir_reg[11]/Q	-	CK->Q	F	SDFFS_X2	1	0.139	0.651	5.952
CRE_IN/DE_IN/FE_O92_ir_11_/Z	-	A->Z	F	BUF_X16	25	0.125	0.498	6.450
CRE_IN/MP_32_IN/p0258A/ZN	-	A2->ZN	R	NAND2_X4	3	0.377	0.460	12.676
CRE_IN/MP_32_IN/Fp0229A/ZN	-	A->ZN	F	INV_X8	5	0.397	0.254	12.929
CRE_IN/MP_32_IN/p0287A/ZN	-	A1->ZN	R	NAND2_X1	1	0.217	0.310	13.239
CRE_IN/EXE_IN/p0136A/ZN	-	B1->ZN	R	AOI22_X1	1	0.525	0.741	37.840
CRE_IN/EXE_IN/p0059A/ZN	-	A->ZN	F	INV_X4	1	0.792	0.398	38.237
CRE_IN/EXE_IN/p_reg[25]/D	<<<	D	F	SDFF_X2	1	0.372	0.000	38.237

- The following command shows a path slew propagation report:

```
litmus > report_timing -retime -path_slew_propagation
```

Path 1: VIOLATED (-15.415 ns) Setup Check with Pin CRE_IN/EXE_IN/p_reg[27]/CK->D

Retime Analysis: Data Path-Slew

Group: m_clk

Startpoint: (R) CRE_IN/DE_IN/ir_reg[9]/CK

Litmus Text Command Reference
Timing Analysis Commands--report_timing

Clock: (R) m_clk
Endpoint: (F) CRE_IN/EXE_IN/p_reg[27]/D
Clock: (R) m_clk

	Capture	Launch
Clock Edge:+	4.000	0.000
Src Latency:+	0.000	0.000
Net Latency:+	4.206 (P)	4.101 (P)
Arrival:=	8.206	4.101

Setup:-	1.201
Uncertainty:-	0.400
Cppr Adjust:+	0.000
Required Time:=	6.605
Launch Clock:=	4.101
Data Path:+	17.919
Slack:=	-15.415
Slack(original):=	-16.743

```
#-----
-
# Timing Point                Flags  Arc      Edge  Cell          Fanout Trans   Delay  Arrival
#
#                               Retime  Retime
#-----
-
CRE_IN/DE_IN/ir_reg[9]/CK      -      CK      R      (arrival)     4      0.102      -      4.101
CRE_IN/DE_IN/ir_reg[9]/Q      -      CK->Q   F      SDFS_X2       1      0.117      0.609  4.710
CRE_IN/DE_IN/FE_86_port_add_1/Z -      A->Z    F      BUF_X16       44     0.180      0.362  5.072
CRE_IN/CRE_GLU_IN/lsh_47/F4/ZN -      A->ZN   R      INV_X4        11     1.381      0.835  5.907
CRE_IN/MP_32/M16_IN/m14/p4D/ZN -      A2->ZN  R      NAND2_X4      4      0.369      0.308  11.530
CRE_IN/MP_32/M16_IN/m14/FpD/ZN -      A->ZN   F      INV_X32       14     0.155      0.154  11.684
CRE_IN/EXE_IN/n0385D/ZN       -      B1->ZN  R      AOI22_X1      1      0.440      0.371  21.650
CRE_IN/EXE_IN/FE_P0_n_650/Z   -      A->Z    R      BUF_X16       1      0.078      0.294  21.944
CRE_IN/EXE_IN/n0439D/ZN       -      A->ZN   F      INV_X8        1      0.059      0.077  22.020
CRE_IN/EXE_IN/p_reg[27]/D     -      D       F      SDFS_X2       1      0.059      0.000  22.020
#-----
-
INFO: Path Based Analysis (PBA) performed on total '1' paths
```

- The following command generates the PBA delay calculation report for the net arc terminating at the specified pin:

```
litmus > report_timing -to CRE_IN/EXE_IN/p_reg[31]/D -retime -path_slew_propagation -
retime_delaycal_pins CRE_IN/EXE_IN/p_reg[31]/D
```

Path 1: VIOLATED (-28.273 ns) Setup Check with Pin CRE_IN/EXE_IN/p_reg[25]/CK->D

Retime Analysis: Data Path-Slew SI

Group: m_clk

Startpoint: (R) CRE_IN/DE_IN/ir_reg[11]/CK

Clock: (R) m_clk

Endpoint: (F) CRE_IN/EXE_IN/p_reg[25]/D

Clock: (R) m_clk

	Capture	Launch
Clock Edge:+	4.000	0.000
Src Latency:+	0.000	0.000
Net Latency:+	5.274 (P)	5.300 (P)
Arrival:=	9.274	5.300

Setup:- 1.254

Uncertainty:- 0.400

Cppr Adjust:+ 0.001

Required Time:= 7.620

Launch Clock:= 5.300

Data Path:+ 30.593

Slack:= -28.273

Slack(original):= -30.645

RDC PBA Report

```
-----
From pin   : CRE_IN/EXE_IN/p0059A/ZN
Voltage    : 0.8 V
Cell       : INV_X4
Library    : PK45_lib_v1.0
To pin     : CRE_IN/EXE_IN/p_reg[25]/D
Voltage    : 0.8 V
Cell       : SDFX_X2
Library    : PK45_lib_v1.0
Base or SI: SI delay
Delay type: net
```

	Rise	Rise	Fall	Fall	Slew
Thresholds:	Delay	Slew	Delay	Slew	Derate
From pin :	50	10->90	50	90->10	1
To pin :	50	10->90	50	90->10	1

From pin waveforms

Fall Waveform : { 0 0.8 3.4584e-10 0 }

To pin waveforms

```
-----
Fall Waveform : { 0 0.8 3.4584e-10 0 }
-----
```

```
-----
RC Summary for net CRE_IN/EXE_IN/n_686
-----
```

```
Number of capacitance : 7
Net capacitance       : 0.00051675 pF
Total rise capacitance: 0.00299075 pF
Total fall capacitance: 0.00298075 pF
Number of resistance  : 6
Total resistance      : 10.8929 Ohm
-----
```

```
Underated Delays          Fall
-----
```

```
Net delay                  : 3.25295e-05 ns
-----
```

```
Transitions              Fall
-----
```

```
Input transition          : 0.276672 ns
Output transition         : 0.276672 ns
-----
```

INFO: Path Based Analysis (PBA) performed on total '1' paths

- The following command generates a report that shows timing paths for which `set_max_delay` constraints are defined:

```
report_timing -check_type max_delay -max_paths 10 > 11
```

Path 1: MET (0.721 ns) Path Delay Check with Pin ff2/D

```
Group: CK1
Startpoint: (R) ff1/Q
Clock: (R) CK1
Endpoint: (R) ff2/D
Clock: (R) CK1
```

	Capture	Launch
Path Delay:+	1.000	-
Src Latency:+	0.000	0.000
Net Latency:+	0.010 (P)	0.010 (P)
Arrival:=	1.010	0.010
Setup:-	0.100	
Required Time:=	0.910	
Launch Clock:=	0.010	
Data Path:+	0.179	
Slack:=	0.721	

```
-----
Pin      Edge  Direction  Phase      Arrival  Delay
```

Time					

ff1/CK	^	IN	CK1 (C) (P) *	0.010	-
ff1/Q	^	OUT	CK1 (D) (P) *	0.013	0.003
buf4/Y	^	OUT	CK1 (D) (P) *	0.023	0.010
buf5/Y	^	OUT	CK1 (D) (P) *	0.033	0.010
and1/Y	^	OUT	CK1 (D) (P) *	0.179	0.146
buf6/Y	^	OUT	CK1 (D) (P) *	0.189	0.010
ff2/D	^	IN	CK1 (D) (P) *	0.189	0.000

Path 2: MET (0.987 ns) Path Delay Check with Pin out1

Group: default

Startpoint: (R) ff2/Q

Clock: (R) CK1

Endpoint: (R) out1

Clock:

	Capture	Launch
Path Delay:+	1.000	-
Src Latency:+	0.000	0.000
Net Latency:+	0.000 (I)	0.010 (P)
Arrival:=	1.000	0.010
Output Delay:-	0.000	
Required Time:=	1.000	
Launch Clock:=	0.010	
Data Path:+	0.003	
Slack:=	0.987	

Pin	Edge	Direction	Phase	Arrival Time	Delay

ff2/CK	^	IN	CK1 (C) (P) *	0.010	-
ff2/Q	^	OUT	CK1 (D) (P)	0.013	0.003
out1	^	OUT	CK1 (D) (P) *	0.013	0.000

report_timing -from latch/D Settings

The `report_timing -from [register/latch]/CK` command settings can be defined in line with the path reported in `full_clock` path report (as reported path might be `clock->lat/CK->lat/Q->ff2/D`). So the `report_timing -from CK-` pin can be very intuitive.

However, in latch case scenarios where there are multiple D pins on a latch, Litmus supports `-from latch/D` pin, which is generally used for debug purposes. When propagating across `CK->Q` arc, the software will perform worst casing across all the `D->Q` arcs and report the worst path possible. The `report_timing -from latch/D2` command can be used to propagate arrival only across `D2->Q` arc, so that no worst casing occurs. And path report or slack with respect to a particular D pin of latch can be attained.

For example:

- The following command uses lat/G (clock pin) , lat/D and lat/D2 (2 data pins) , lat/Q (output/begin pin), and shows the below report:

```
report_timing -to ff2/D -path_type full -fields {hpin edge delay arrival arc} -net -from
lat/D2
```

```
-----
Path 1: MET Setup Check with Pin ff2/CK
```

```
Endpoint:   ff2/D (v) checked with trailing edge of 'CK1'
```

```
Beginpoint: lat/Q (v) triggered by trailing edge of 'CK1'
```

```
Path Groups: {CK1}
```

```
Other End Arrival Time          1.400
```

```
- Setup                        0.110
```

```
+ Phase Shift                  2.500
```

```
+ CPPR Adjustment              0.000
```

```
= Required Time                3.790
```

```
- Arrival Time                 1.773
```

```
= Slack Time                   2.017
```

```
    Clock Fall Edge            1.250
```

```
    + Clock Network Latency (Prop) 0.120
```

```
    = Beginpoint Arrival Time      1.370
```

```
-----
Pin      Edge  Delay  Time  Arrival  Arc
                        Given  Time
                        To
                        Start
                        Point
-----
```

```
lat/G    v    -    -    1.370    G v
lat/Q    v    0.150 0.193 1.713    D2 v -> Q v
buf9/A   v    0.000 -    1.713    -
buf9/Y   v    0.030 -    1.743    A v -> Y v
buf10/A  v    0.000 -    1.743    -
buf10/Y  v    0.030 -    1.773    A v -> Y v
ff2/D -> v    0.000 -    1.773    -
-----
```

Related Commands

- `report_nets`
- `report_ports`
- `set_multi_cpu_usage`
- `set_table_style`
- `set_input_delay`
- `set_max_delay`
- `set_min_delay`
- `create_analysis_view`

report_timing_derate

```
report_timing_derate  
[help]  
[-delay_corner delayCornerName]  
[-include_inherited]  
[collection]  
[{> | >>} filename [.gz]]
```

Reports delay scaling or derating factors for early and late clock and data paths in the current design.

By default, the `report_timing_derate` output shows `pulse_width_check` and `clock_period_check` details for instance objects (see `set_timing_derate`).

If you specify this command without any parameters, reports are generated for global derating factors for each instance, net, or cell for which derating is asserted.

The following reports are also generated:

- In best case worst case, maximum and minimum corner
- Early values from minimum corner and late values from maximum corner in on-chip variation mode
- In MMMC mode, specified corners, or for all corners when corner is not specified
- Power domains (if present in the design)
- Derates for pulse width and clock period checks. Also, the report will show mean/sigma values, when the `set_timing_derate -mean/-sigma` parameters are specified.

Parameters

```
> | >> filename
```

When specified with `>`, writes the report to the specified file name. If the file already exists, the software overwrites it.

When specified with `>>`, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file.

Note: To write a compressed report, add the `.gz` extension to the file name.

<code>-delay_corner delayCornerName</code>	Reports derating factors for the specified delay calculation corner only. <i>Default:</i> Reports derating factors for all delay calculation corners.
<code>-include_inherited</code>	Reports the object names from which the derating factor values are inherited. When specified, the report displays the name of the cell for each instance from which derate values are inherited.
<code>collection</code>	Specifies the list of cells, nets, or library cells.

Examples

- The following command reports derating factors for delay corner `dly_cnr_slow_RX`:

```
litmus > report_timing_derate -delay_corner dly_cnr_slow_RX -include_inherited
```

```
---- Report Timing Derate for DC Corner: dly_cnr_slow_RX
```

	-----Clock-----				-----Data-----			
	Rise		Fall		Rise		Fall	
	Early	Late	Early	Late	Early	Late	Early	Late
Cell Delay	0.900	1.100	0.900	1.100	0.820	1.010	0.820	1.010
Net Delay Static	0.900	1.100	0.900	1.100	0.820	1.010	0.820	1.010
Net Delay Dynamic	0.900	1.100	0.900	1.100	0.820	1.010	0.820	1.010

- The following commands display global derating factors:

```
litmus > set_timing_derate 1.2
```

```
litmus > set_timing_derate -clock -early 1.1
```

```
litmus > set_timing_derate -data -late 1.2
```

```
litmus > report_timing_derate
```

```
---- Report Timing Derate for DC Corner: def_emul_delay_corner
```

	-----Clock-----				-----Data-----			
	Rise		Fall		Rise		Fall	
	Early	Late	Early	Late	Early	Late	Early	Late
Cell Delay	1.100	1.200	1.100	1.200	1.200	1.200	1.200	1.200
Net Delay Static	1.100	1.200	1.100	1.200	1.200	1.200	1.200	1.200
Net Delay Dynamic	1.100	1.200	1.100	1.200	1.200	1.200	1.200	1.200
Cell Check	--	--	--	--	--	--	--	--
Retain delay	--	--	--	--	--	--	--	--

Litmus Text Command Reference Timing Analysis Commands--report_timing_derate

```
pulse_width_check    --      --      --      --      --      --      --      --
clock_period          --      --      --      --      --      --      --      --
```

- The following commands display instance derating factors:

```
litmus > set_timing_derate -cell_delay -clock -late 1.3 [get_cells {m_clk_LI89}]
litmus > set_timing_derate -cell_check -late 1.4 [get_cells {m_clk_LI89}]
litmus > report_timing_derate
```

	-----Clock-----				-----Data-----			
	Rise		Fall		Rise		Fall	
	Early	Late	Early	Late	Early	Late	Early	Late
Cell (leaf): m_clk_LI89								
Cell Delay	--	1.300	--	1.300	--	--	--	--
Cell Check	--	1.400	--	1.400	--	1.400	--	1.400
pulse_width_check	--	--	--	--	--	--	--	--
clock_period_check	--	--	--	--	--	--	--	--

- The following commands display hierarchical derating factors:

```
litmus > set_timing_derate 1.1 -late [get_cells {TD_DS_INST}]
litmus > set_timing_derate 1.2 -early [get_cells {TD_DS_INST/TDSP_MUX}]
litmus > set_timing_derate -cell_check 1.2 [get_cells {TD_DS_INST}]
litmus > report_timing_derate -include_inherited
```

---- Report Timing Derate for DC Corner: def_emul_delay_corner

	-----Clock-----				-----Data-----			
	Rise		Fall		Rise		Fall	
	Early	Late	Early	Late	Early	Late	Early	Late
Late								
Cell Delay	1.200	1.200	1.200	1.200	1.200	1.200	1.200	1.200
1.200								
Net Delay Static	1.200	1.200	1.200	1.200	1.200	1.200	1.200	1.200
1.200								
Net Delay Dynamic	1.200	1.200	1.200	1.200	1.200	1.200	1.200	1.200
1.200								
Cell Check	--	--	--	--	--	--	--	--
--								
Retain delay	--	--	--	--	--	--	--	--
--								
pulse_width_check	--	--	--	--	--	--	--	--
--								
clock_period	--	--	--	--	--	--	--	--
--								
Cell (hier): TD_DS_INST								

Litmus Text Command Reference
Timing Analysis Commands--report_timing_derate

Cell Delay	1.200	1.100	1.200	1.100	1.200	1.100	1.200
1.100							
inherited	dt_rvr_cre	--dt_rvr_cre		--dt_rvr_cre		--dt_rvr_cre	
--							
Net Delay Static	1.200	1.100	1.200	1.100	1.200	1.100	1.200
1.100							
inherited	dt_rvr_cre	--dt_rvr_cre		--dt_rvr_cre		--dt_rvr_cre	
--							
Net Delay Dynamic	1.200	1.100	1.200	1.100	1.200	1.100	1.200
1.100							
inherited	dt_rvr_cre	--dt_rvr_cre		--dt_rvr_cre		--dt_rvr_cre	
--							
Cell Check	1.200	1.200	1.200	1.200	1.200	1.200	1.200
1.200							
inherited	--	--	--	--	--	--	--
--							
pulse_width_check	--	--	--	--	--	--	--
--							
clock_period_check	--	--	--	--	--	--	--
--							
inherited	--	--	--	--	--	--	--
--							
pulse_width_check	--	--	--	--	--	--	--
--							
clock_period_check	--	--	--	--	--	--	--
--							

- The following commands display best case worst case derating factors:

```
litmus > set_timing_derate -cell_delay -clock -late 1.2 [get_cells {f2}] -min
litmus > set_timing_derate -cell_delay -late 1.3 [get_cells {f2}] -max
litmus > report_timing_derate
```

```
-----Report Timing Derate for MIN corner-----
               -----Clock-----      -----Data-----
               Early      Late      Early      Late
               -----
Cell Delay      --              --              --              --
Net Delay Static --              --              --              --
Net Delay Dynamic --              --              --              --
Cell Check      --              --              --              --
Retain delay    --              --              --              --

Cell (leaf): f2
Cell Delay      --              1.200000      --              --
```

Litmus Text Command Reference Timing Analysis Commands--report_timing_derate

```

-----Report Timing Derate for MAX corner-----
               -----Clock-----               -----Data-----
               Early                 Late             Early                 Late
               -----
Cell Delay      --                  --                --                  --
Net Delay Static --                  --                --                  --
Net Delay Dynamic --                  --                --                  --
Cell Check      --                  --                --                  --
Retain delay    --                  --                --                  --

Cell (leaf): f2
Cell Delay      --                  1.300000          --                  1.300000
-----

```

- The following example shows a report that is displayed when mean/sigma derates are applied in the SOCV mode:

```

litmus > set_timing_derate 1.11 -mean -delay_corner def_emul_delay_corner [get_lib_cells {INV_X32}]
litmus > set_timing_derate 1.11 -sigma -delay_corner def_emul_delay_corner [get_lib_cells {INV_X32}]
litmus > report_timing_derate [get_lib_cells {INV_X32}]

---- Report Timing Derate for DC Corner: def_emul_delay_corner

               -----Clock-----               -----
               -----Data-----
               Fall                 Rise             Fall                 Rise
               Early                 Late             Early                 Late
               -----
LibraryCell: FPD4_lib_v1.0/INV_X32
Cell_delay      1.110:1.110  1.110:1.110    1.110:1.110  1.110:1.110  1.110:1.110
1.110:1.110  1.110:1.110    1.110:1.110
Input_switching  --                  --                --                  --
--              --              --
Retain_delay     --                  --                --                  --
--              --              --
Input_switching  --                  --                --                  --
--              --              --
Cell Check       --                  --                --                  --
--              --              --
pulse_width_check
clock_period_check
-----
-----

```

Related Commands

- `set_timing_derate`

report_wire_load

```
report_wire_load  
[-help]  
[-instances name_or_collection_of_hierarchical_instances]  
[-ports name_or_collection_of_ports]  
[-late | -early]  
[> | >> filename]
```

Reports the wireload name, library, wire load mode and selection type for all hierarchical instances in the design.

Note: To use the wireload model functionality, you must enable the `enable_wire_load_model_support` attribute using the following command:

```
set_var enable_wire_load_model_support 1
```

Parameters

-help	Prints out the command usage.
> >> <i>filename</i>	When specified with >, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with >>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The <i>filename</i> parameter must be the last argument in the list. Note: To write a compressed report, add the .gz extension to the file name. <i>Default:</i> Report is displayed on standard output without being saved.
-early	Reports wireload model information for the best case condition.

<code>-instances</code> <i>name_or_collection_of_hierarchical_instances</i>	Specifies the name of an instance or collection for which the <code>report_wire_load</code> command reports the wireload name, library, wire load mode, and selection type. The <code>report_wire_load</code> command also supports wildcard names such as U*.
<code>-late</code>	Reports wireload model information in worst-case condition. <i>Data_type</i> : bool, optional
<code>-ports</code> <i>name_or_collection_of_ports</i>	Reports the wireload details for the specified ports.

Examples

- The following command reports the wireload details for the specified ports.

```
report_wire_load -ports clk1 -early
```

```
-----
----
Design Name      top_chip
Wire Load Mode   enclosed

Port      Module      WireLoadModel      Library                               SelctionType
-----
----
clk1      top_chip     MACRO_15             UX8L_wide1_1.1V_TYP_primitive_mvt    automatic-by-area
-----
----
```

Related Commands

- `set_wire_load_model`
- `reset_wire_load_model`

reset_path_adjust

```
reset_path_adjust  
[-help]  
-path_adjust_group path_adjust_group_name  
-view string  
[-setup | -hold]
```

Removes the specified slack adjustment values for timing paths that are set using the `set_path_adjust` command.

Parameters

-help	Prints out the command usage.
-hold	Resets the adjust value for early paths.
-path_adjust_group <i>path_adjust_group_name</i>	Specifies the name of path adjust group that defines the path for which the adjustment will be applied.
-setup	Resets the adjust value for late paths.
-view <i>string</i>	Specifies the view for which adjustment will be reset.

Related Commands

- `set_path_adjust`

reset_path_adjust_group

```
reset_path_adjust_group  
[-help]  
[-name group_name | -all ]
```

Removes all path adjust groups that are set using the `set_path_adjust_group` command for setting path slack adjustment values.

Parameters

-help	Prints out the command usage.
-all	Removes all the path adjust groups.
-name <i>group_name</i>	Removes the named path adjust group completely.

Related Commands

- `set_path_adjust_group`
- `set_path_adjust`

reset_timing_derate

```
reset_timing_derate
[-help]
[-delay_corner delayCornerName]
[-increment]
[-transition]
[obj_list]
```

Resets derate factors specified on a design or a list of instances (cells, net, or library cells). All the derating factors that are set globally or on specific instances are reset to a default value of 1.0.

Derating factors for early or late paths in the design are set using the [set_timing_derate](#) command.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-delay_corner delayCornerName</code>	Specifies the delay corner or a list of delay corners on which reset derating is to be applied.
<code>-increment</code>	Resets the incremental derate values that are set using the set_timing_derate command. The set_timing_derate command saves two different values – the “base” derate and the “incremental” derate. When the <code>-incremental</code> parameter is used, the software populates the incremental derates. The reset_timing_derate command will reset the incremental derate values only.
<code>obj_list</code>	Specifies the list of cells, nets, or library cells to reset. If the <code>obj_list</code> contains hierarchical cells then all cells within the hierarchical cells and across lower hierarchies have reset derate factors.
<code>-transition</code>	Resets the transition derate values.

Examples

- The following example resets all the object-specific and global derating factors:

```
reset_timing_derate *
```

- The following example resets only global derating factors:

```
reset_timing_derate [current_design]
```

- The following example resets derating factor values for the specified delay corner:

```
reset_timing_derate -delay_corner delayCornerName
```

- The following example resets the global derating values for the specified delay corner:

```
reset_timing_derate -delay_corner delayCornerName [current_design]
```

Related Commands

- `set_timing_derate`

reset_wire_load_mode

reset_wire_load_mode

Resets the wire-load mode to the default mode of `top`.

Note: To use the wireload model functionality, you must enable the `enable_wire_load_model_support` attribute using the following command:

```
set_var enable_wire_load_model_support 1
```

Related Commands

- `set_wire_load_mode`

reset_wire_load_model

```
reset_wire_load_model  
[-help]  
[{ obj_list }]
```

Resets wireload models for the specified list of objects. The list of objects can include designs, hierarchical cells, or ports.

When objects are not specified, the software removes the wireload model specified using the `set_wire_load_model` command from the current instance only.

Note: To use the wireload model functionality, you must enable the `enable_wire_load_model_support` attribute using the following command:

```
set_var enable_wire_load_model_support 1
```

Parameters

<code>-help</code>	Outputs a brief description that includes type and default information for each <code>reset_wire_load_model</code> parameter. For a detailed description of the command and all of its parameters, use the man command: <code>man reset_wire_load_model</code> .
<code>obj_list</code>	Specifies the list of instances or ports for which to remove the wireload models.

Examples

- The following command removes the wireload model for `clk1`:

```
reset_wire_load_model clk1
```

Related Commands

- [set_wire_load_model](#)

reset_wire_load_selection_group

```
reset_wire_load_selection_group  
[-help]  
[{ obj_list }]
```

Removes the wireload selection group for the specified list of objects. The list of objects can include designs, hierarchical cells, or ports. When objects are not specified, wireload selection groups are removed for all objects.

Note: To use the wireload model functionality, you must enable the `enable_wire_load_model_support` attribute using the following command:

```
set_var enable_wire_load_model_support 1
```

Parameters

<code>-help</code>	Prints out the command usage.
<code>obj_list</code>	Specifies the list of object names for which to remove the wireload selection groups.

Examples

- The following command removes the wireload selection group for `clk1`:

```
reset_wire_load_selection_group clk1
```

Related Commands

- `set_wire_load_selection_group`

set_analysis_view

```
set_analysis_view
[-help]
[-drv string]
[-dynamic string]
[-ir string]
[-hold string]
[-leakage string]
[-setup string]
[-update_timing]
```

Defines the analysis views to use for setup and hold analysis and optimization. You must define at least one setup and one hold analysis view. The "active" views represent the different design variations that will be analyzed. These views can be changed throughout the flow to utilize different subsets of views. The libraries and data are loaded into the system that are required to support the selected set of active views.

When the `set_analysis_view` command is issued, it will cause a full timing reset. During a timing reset, all the interconnect parasitics, delays, and timing slacks will be recalculated.

The order in which you specify the views with the `-setup` and `-hold` parameters is important. By default, the first views defined in the `-setup` and `-hold` lists are the default views. The default views serve two purposes:

- Provide the library data that is used during the design initialization process.
- Indicate to parts of the application, that are not MMMC-aware, which view's information to use for a particular analysis step (in most cases, the default view is the first setup or hold view). You can use the `set_default_view` command to change the default view to any currently active view without incurring a timing reset. This control is not frequently used since most applications either support MMMC or provide a direct control for selecting the appropriate view to use for that specific analysis.

Using the `set_default_view` command does not affect software performance because it only uses views that are already active in the design. If you use the `set_analysis_view` command to change the default views, the existing timing and delay calculation data is reset.

Use this command after creating multiple analysis views (`create_analysis_view`).

Note: The `set_analysis_view` command does not work in an incremental manner for setting parameters. Each invocation of this command must include the full specification of all the active views required for analysis.

Parameters

-help	Prints out the command usage.
-------	-------------------------------

<code>-drv string</code>	Selects specific views for DRV analysis, e.g., max_transition and max_capacitance checking. When this parameter is specified, additional analysis views are specified for DRV analysis, so the timing system efficiently computes the required data for these checks. This methodology is efficient with respect to memory and performance, because all the views as full Setup views are not included. The following guidelines should be followed when selecting DRV views: • Any setup views that require DRV analysis should not be specified as DRV views. The setup views are automatically considered for DRV. • If a subset of hold-only views require DRV analysis, they should be specified as DRV views as well. The timing system will augment the timing information normally computed for Hold views with additional data required for the DRV analysis. • Additional views that are not active setup or hold views can be specified as DRV only views. The system will efficiently compute the required data for performing accurate DRV checks.
<code>-dynamic string</code>	Sets a view as dynamic view. Delay calculation is required to provide transition times. Delay calculation should not perform SI analysis if the delay corner is only required for the dynamic power. The timer is not required to propagate required times or perform slack computations.
<code>-hold list_of_views</code>	Sets the active views for hold analysis.
<code>-ir string</code>	Specifies the IR view.
<code>-leakage string</code>	Sets a view as leakage view. The timing libraries must be loaded, if not already in use. The Timer constant propagation is required, whereas delay calculation is not required.

<code>-setup list_of_views</code>	Sets the active views for setup analysis.
<code>-update_timing</code>	Reloads the configuration described by the MMMC view and mode objects. When the timing attribute <code>timing_defer_mmmc_obj_updates</code> is set to <code>true</code>, this parameter is required to update the MMMC timing environment. The <code>-update_timing</code> parameter may be specified without any <code>-setup</code> or <code>-hold</code> arguments. In this case, the data associated with the current set of setup and/or hold views will be loaded. If no active setup or hold views exist, then using this parameter alone may display an error. The <code>-update_timing</code> parameter may also be used in conjunction with the <code>-setup</code> and <code>-hold</code> options. In this case, the newly specified setup/hold view configuration will be loaded with the updated MMMC object information.

When you issue the `set_analysis_view` command, the software will perform a timing reset, so that only the new active view information is loaded up. The software requires the `-setup` and `-hold` view parameters to be explicitly specified for each `set_analysis_view` command run. If these parameters are not specified, the software issues an error message. Any optional settings, such as, `-leakage` or `-dynamic` views will be reset if they are not specified with the new `set_analysis_view` command.

Note: It is recommended to use the same power view for both leakage and dynamic, that is, both the `-dynamic` and `-leakage` parameters should have the same user-specified view settings.

Examples

- The following command sets `missionWCCOM` and `mission2WCCOM` as the active views for setup analysis, and `missionBCCOM` and `testBCCOM` as the active views for hold analysis:

```
set_analysis_view -setup {missionWCCOM mission2WCCOM} -hold {missionBCCOM testBCCOM}
```

Related Commands

- `create_analysis_view`
- `set_default_view`

set_clock_exclusivity

```
set_clock_exclusivity  
[-help]  
pin_name  
[-exclude_opposite_polarity]  
[-exclude_related_input_pins]  
[-exclude_same_polarity]  
[-group clock_list]  
[-inputs string]
```

Controls filtering of mutually-exclusive clocks at strategic points in the design, e.g., the output of clock multiplexers.

Parameters

-help	Prints out the command usage.
-exclude_opposite_polarity	Isolates positive and negative phases of the clocks in the same group. By default, the positive and negative phases of the clocks in the same group that arrive at a strategic point will not be isolated from each other.
-exclude_related_input_pins	Allows comparison of clocks going through the same input pins of the specified gate.
-exclude_same_polarity	Allows comparison of clocks going through the same launch or capture paths.
-group <i>clock_list</i>	Defines a set of clocks that are exclusive to the clocks defined in all other groups.
-inputs <i>string</i>	Specifies a list or collection of input pins for which clock exclusivity is to be recognized.
<i>pin_name</i>	Specifies the name of the pin.

Examples

- The following command will cause downstream analysis from the mux to isolate interactions between the groups:

```
tempus > set_clock_exclusivity -group {CLK1 CLK2} -group {CLK3 CLK4} U1/Y
```

This assertion will cause downstream analysis from pin U1/Y to isolate interactions between the groups. The groups {CLK1 CLK2} and {CLK3 CLK4} are allowed, but all other combinations are considered logically exclusive.

Related Commands

- `create_clock`

set_default_view

```
set_default_view  
[-help]  
{[-hold newDefaultHoldView] | [-setup newDefaultSetupView]}
```

Temporarily changes the default active analysis view to a different active view.

Some Litmus applications can function only on a single analysis view at a time. By default, these single-view applications use the default analysis view. If an application or flow step does not provide a native option for specifying which view to use, the `set_default_view` command can be used to temporarily change the default view to a different active view that is better suited to that flow step.

Note: Using the `set_default_view` command does not affect software performance because it only uses views that are already active in the design. If you use the `set_analysis_view` command to change the default views, the existing timing, delay calculation, and RC data is reset.

Use this command after defining active views for the design (`set_analysis_view`).

Parameters

<code>-help</code>	Prints out the command usage.
<code>-hold newDefaultHoldView</code>	Changes the default active hold analysis view to the specified analysis view.
<code>-setup newDefaultSetupView</code>	Changes the default setup analysis view to the specified analysis view.

Examples

- The following command sets `missionWCCOM`, `mission2WCCOM`, and `missionSlow` as the active views for setup analysis, and `missionBCCOM` and `testBCCOM` as the active views for hold analysis. The default active setup view is `missionWCCOM`, and the default active hold view is `missionBCCOM`.

```
set_analysis_view -setup {missionWCCOM mission2WCCOM missionSlow} -hold {missionBCCOM  
testBCCOM}
```

- The following command changes the default active setup view to `missionSlow`:

```
set_default_view -setup missionSlow
```

- The following command changes the default active hold view to `testBCCOM`:

```
set_default_view -hold testBCCOM
```

Related Commands

- `create_analysis_view`

set_interactive_constraint_modes

```
set_interactive_constraint_modes  
[-help]  
{constraint_mode_list}
```

Puts the software into interactive constraint entry mode for the specified multi-mode multi-corner constraint mode objects. Any timing constraints that you specify after this command take effect immediately on all active analysis views that are associated with the specified constraint modes. By default, no constraint modes are considered active.

All new assertions are saved in the SDC files of the specified constraint modes when you save the design ([write_db](#)).

Use the [get_interactive_constraint_modes](#) command to return a list of the constraint mode objects in constraint entry mode.

Note: Interactive constraint mode only works when the software is in multi-mode multi-corner timing analysis mode. In min/max analysis mode, constraints are always accepted interactively.

Parameters

<code>-help</code>	Prints out the command usage.
<code>constraint_mode_list</code>	Specifies the constraint mode objects to update with the new assertions.

Examples

- The following commands put the software into interactive constraint entry mode, and apply the `set_propagated_clock` assertion on all views in the current session that are associated with the constraint mode func1:

```
litmus> set_interactive_constraint_modes func1
litmus> set_propagated_clock [all_clocks]
```

When you specify `write_db` command, the updated constraints file for the func1 constraint mode will include the `set_propagated_clock` constraints.

- The following commands put the software into interactive constraint entry mode, and apply the `set_propagated_clock` assertion on all active analysis views in the current session.

```
litmus> set_interactive_constraint_modes [all_constraint_modes -active]
litmus> set_propagated_clock [all_clocks]
```

When you specify the `write_db` command, the `set_propagated_clock` assertion will be saved persistently as part of a new `.sdc` file.

Related Commands

- `get_interactive_constraint_modes`
- `get_db`
- `write_db`

set_path_adjust

```
set_path_adjust  
[-help]  
adjust_value  
-path_adjust_group path_adjust_group_name  
-view string  
[-setup | -hold]
```

Defines the slack adjustment value for timing paths. The defined adjustment value is added to the slack of a path specified by the path adjust group, referred to by the `-path_adjust_group` parameter. You can specify a path adjust group by using the `set_path_adjust_group` command.

If multiple path adjustment values are applicable on the same pin, which is on the path specified by multiple `set_path_adjust_group` constraints, then the smallest adjustment value will be applied.

Note: The `set_path_adjust` command is not allowed to be used inside the SDC constraints file. It is only allowed in the software run script.

Parameters

<code>-help</code>	Prints out the command usage.
<code>adjust_value</code>	Specifies the value of slack adjustment.
<code>-hold</code>	Specifies to apply the adjust value to early paths.
<code>-path_adjust_group</code> <code>path_adjust_group_name</code>	Specifies the name of path adjust group that defines the path for which the adjustment will be applied.
<code>-setup</code>	Specifies to apply the adjust value to late paths. Note: If you do not specify <code>-setup</code> or <code>-hold</code> parameter, the software sets the adjust value to both setup and hold analysis.
<code>-view string</code>	Specifies a list of analysis views for which adjustment will be applied. This parameter accepts multiple views as argument.

Examples

- The following commands define the slack adjustment values for the specified views:

```
litmus> set_path_adjust 0.44 -path_adjust_group AG1 -view view1 -hold
```

```
litmus> set_path_adjust 0.44 -path_adjust_group AG2 -view view2 -setup
```

```
litmus> set_path_adjust 0.44 -path_adjust_group AG3 -view view3
```

Related Commands

- [set_path_adjust_group](#)

set_path_adjust_group

```
set_path_adjust_group  
[-help]  
[-fall]  
[-fall_through through_list]  
-name path_adjust_group_name  
[-rise]  
[-rise_through through_list]  
[-through through_list]  
[-from from_list | -rise_from from_list | -fall_from from_list]  
[-to to_list | -rise_to to_list | -fall_to to_list]
```

Allows you to specify the path that is used by the `set_path_adjust` command for setting path slack adjustment values.

The `set_path_adjust_group` command is not allowed to be used inside the SDC constraints file. It is only allowed in the tool run script.

If path group definitions are matching/overlapping (when the `-from/-through/-to` parameters are used), then path group resetting is applied. The path group overwriting and prioritization rules are similar to path exceptions.

For more details, refer to [group_path](#).

Parameters

<code>-help</code>	Prints out the command usage.
<code>-fall</code>	Applies <code>set_path_adjust_group</code> at the falling edge of the signal on the path endpoints.
<code>-fall_from from_list</code>	Applies <code>set_path_adjust_group</code> at the falling edge of the signal on the from pins.
<code>-fall_through through_list</code>	Applies <code>set_path_adjust_group</code> at the falling edge of the signal on the through pins.
<code>-fall_to to_list]</code>	Applies <code>set_path_adjust_group</code> at the falling edge of the signal on the to pins.
<code>-from from_list</code>	Specifies the list of pins, instances, or clocks that are the start of the paths.
<code>-name path_adjust_group_name</code>	Specifies the name of the path adjust group.
<code>-rise</code>	Applies <code>set_path_adjust_group</code> at the rising edge of the signal on the path endpoints.
<code>-rise_from from_list</code>	Applies <code>set_path_adjust_group</code> at the rising edge of the signal on the from pins.
<code>-rise_through through_list</code>	Applies <code>set_path_adjust_group</code> at the rising edge of the signal on the through pins.
<code>-rise_to to_list</code>	Applies <code>set_path_adjust_group</code> at the rising edge of the signal on the to pins.
<code>-through through_list</code>	Specifies the list of pins or instances where the paths go through.
<code>-to to_list</code>	Specifies the list of pins, instances, or clocks where the path ends.

Examples

The following commands specify the adjusted path groups:

- `set_path_adjust_group -name AG1 -from in1 -through buf1/A -to out1`
- `set_path_adjust_group -name AG2 -from [get_clocks CK1] -through buf1/A -to [get_clocks CK2]`
- `set_path_adjust_group -name AG3 -from [get_clocks {CK1 CK2}] -to [get_clocks {CK1 CK2}]`

Related Commands

- `reset_path_adjust_group`
- `set_path_adjust`
- `group_path`

set_selectively_expanded_modules

```
set_selectively_expanded_modules  
[-help]
```

While performing Static Timing Analysis (STA) or Engineering Change Orders (ECO), very large designs often contain modules with a high number of instantiations. Creating RTL-level configurations for smaller portions of such designs is often challenging.

The `set_selectively_expanded_modules` command addresses this issue by specifying modules with instantiations that are to be abstracted, or are not timed, based on the specifications set through the `set_expanded_hinsts` command.

This reduces memory usage and runtime significantly during the STA and ECO processes. It helps in handling large designs on smaller machines and facilitates ECO implementation without the need of manually partition the design into smaller segments.

- 
- This command is recommended exclusively for modules with high instantiation counts and is not recommended to use for single-instance modules.
 - All STA views must use consistent flattening specifications. For ECO, the same netlist and flattening specifications must be loaded to ensure alignment.

Parameters

<code>-help</code>	Prints a brief description that includes the type and default information for each <code>set_selectively_expanded_modules</code> parameter. For a detailed description of the command and all its parameters, use the man command: <pre>man set_selectively_expanded_modules</pre>
--------------------	--

Examples

- The following command set presents an example of instantiation of `a` and `c` modules, which are selectively expanded:

```
read_lib my_lib  
read_verilog my_verilog    # instantiations of modules "a" and "c" are to be selectively  
expanded  
set_selectively_expanded_modules "a c"    # expand "u1" and "u1/u2"
```

```
set_expanded_hinsts "u1 u1/u2"; # No order requirement: "u1/u2 u1" is OK, too
set_top_module top
```

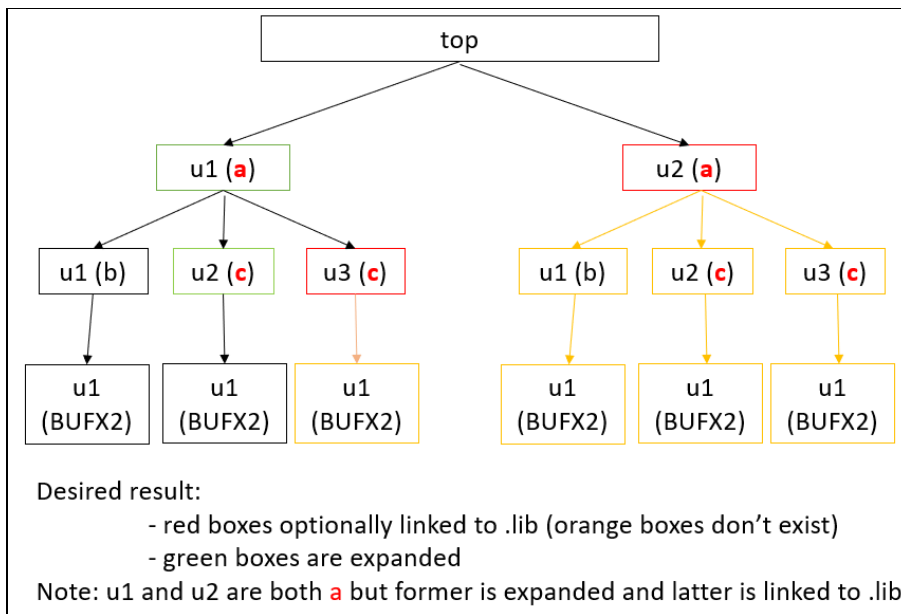
Netlist:

```
module top;
  a u1; a u2;
endmodule

module a;
  b u1; c u2; c u3;
endmodule

module b;
  BUFX2 u1;
endmodule

module c;
  BUFX2 u1;
endmodule
```



Following is the expansion result of the above example:

```
COMPONENTS 4 ;
- u1/u1/u1 BUFX2 ;
- u1/u2/u1 BUFX2 ;
- u1/u3 c ;
- u2 a ;END COMPONENTS
```

- The following command set presents an example of instantiation of c module, which is selectively expanded:

```
read_lib my_lib
```

```
read_verilog my_verilog # instantiations of modules "c" are to be selectively expanded
set_selectively_expanded_modules "c" # expand "u1/u2", "u2/u2" and "u2/u3"
set_expanded_hinsts "u1/u2 u2/u2 u2/u3"
set_top_module top
```

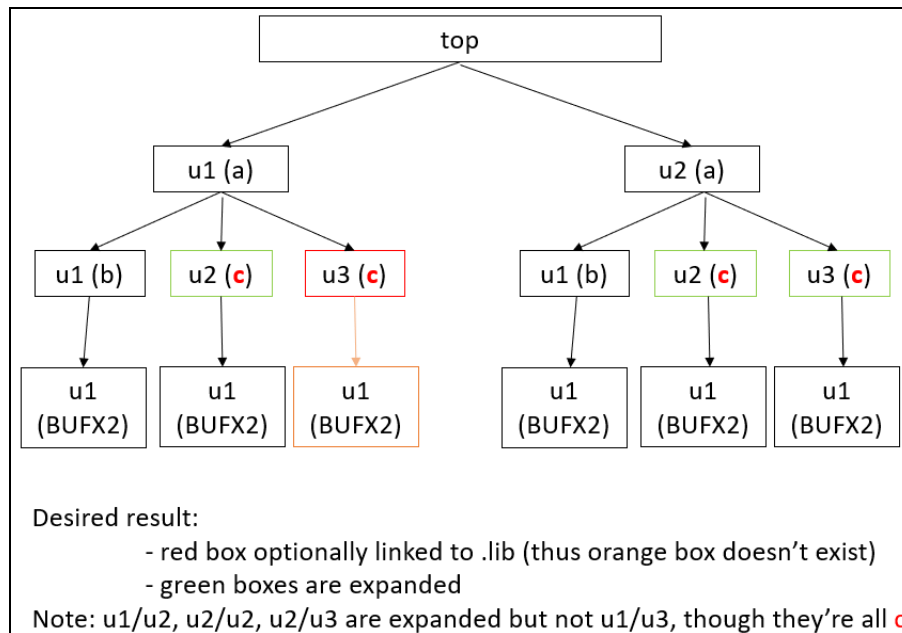
Netlist:

```
module top;
  a u1; a u2;
endmodule

module a;
  b u1; c u2; c u3;
endmodule

module b;
  BUFX2 u1;
endmodule

module c;
  BUFX2 u1;
endmodule
```



Following is the expansion result of the above example:

```
COMPONENTS 6 ;
- u1/u1/u1 BUFX2 ;
- u1/u2/u1 BUFX2 ;
- u1/u3 c ;
- u2/u1/u1 BUFX2 ;
- u2/u2/u1 BUFX2 ;
```



```
- u2/u3/u1 BUFX2 ;
```

```
END COMPONENTS
```

- The following command set presents an example of instantiation of **b** and **c** modules, which are selectively expanded:

```
read_lib my_lib
```

```
read_verilog my_verilog # instantiations of modules "b" and "c" are to be selectively expanded
```

```
set_selectively_expanded_modules "b c" # expand "u2/u1", "u2/u2" and "u2/u3"
```

```
set_expanded_hinsts "u2/u1 u2/u2 u2/u3"
```

```
set_top_module top
```

Netlist:

```
module top;
```

```
  a u1; a u2;
```

```
endmodule
```

```
module a;
```

```
  b u1; c u2; c u3;
```

```
endmodule
```

```
module b;
```

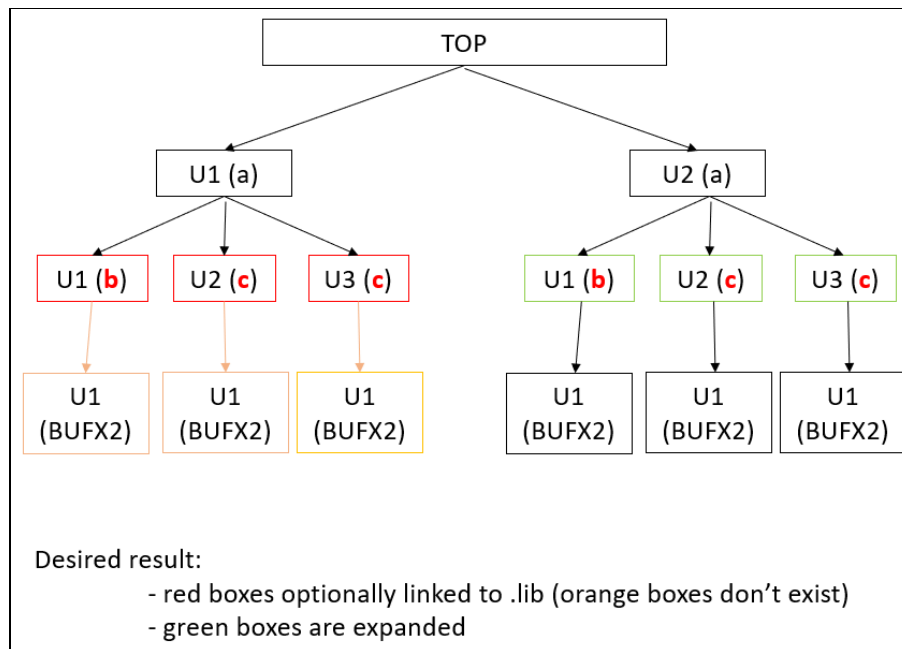
```
  BUFX2 u1;
```

```
endmodule
```

```
module c;
```

```
  BUFX2 u1;
```

```
endmodule
```



Following is the expansion result of the above example:

```
COMPONENTS 6 ;
- u1/u1 b ;
- u1/u2 c ;
- u1/u3 c ;
- u2/u1/u1 BUFX2 ;
- u2/u2/u1 BUFX2 ;
- u2/u3/u1 BUFX2 ;
END COMPONENTS
```

- The following command set presents an example of instantiation the a module, which is selectively expanded:

```
read_lib my_lib
read_verilog my_verilog # instantiations of modules "a" are to be selectively expanded
set_selectively_expanded_modules "a" # do not expand any instantiation of "a"
set_expanded_hinsts ""; # Note: same result if this line is removed
set_top_module top
```

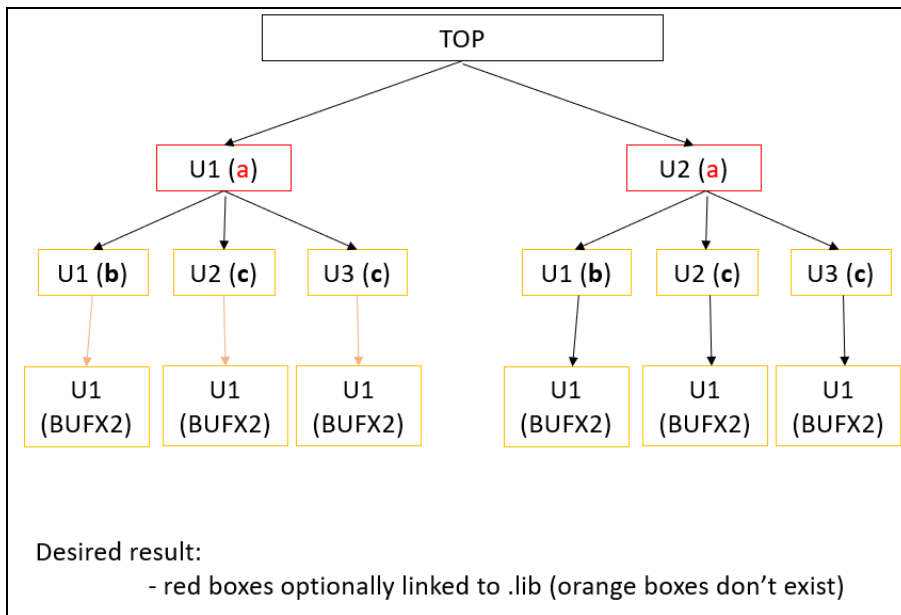
Netlist:

```
module top;
  a u1; a u2;
endmodule

module a;
  b u1; c u2; c u3;
endmodule

module b;
  BUFX2 u1;
endmodule

module c;
  BUFX2 u1;
endmodule
```



Following is the expansion result of the above example:

```
COMPONENTS 2 ;  
- u1 a ;  
- u2 a ;  
END COMPONENTS
```

Related Topics

[get_selectively_expanded_modules](#)

set_table_style

```
set_table_style
-help
-name string
[-major_sort integer]
[-minor_sort integer]
[-max_widths integer]
[-min_widths integer]
[-indent integer]
[-nosplit]
[-reverse_rows] | -no_frame_fix_width
```

Disables printing of specific columns and allows for specifying the minimum and maximum size of each column in a report. It allows for reversing the data, for controlling the left indent, and for controlling the sorting of the columns.

Note: Do *not* include a blank space between commas ", " when specifying widths. If you are specifying a value for the width(s), make sure there is *no* leading or trailing blank space.

The `-frame`, `-no_frame`, and `-no_frame_fix_width` parameters work in a global manner. If you specify one of these parameters, it will affect all reports that can be formatted with this command. You cannot specify them with the `-name` parameter, to change the format of a specific report.

Parameters

<code>-help</code>	Outputs a brief description that includes type and default information for each <code>set_table_style</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man set_table_style</code> .
<code>-frame</code>	Specifies that the table columns have a fixed frame or border, based on the minimum and maximum width settings. Any data that is longer than the maximum width for its column is wrapped within the same column. This is the default format for tables.
<code>-indent <i>integer</i></code>	Specifies the number of spaces to leave on the left for that particular table.
<code>-major_sort <i>integer</i></code>	Specifies the column to be used in the major sort. If the column number is negative, it reverses the sort.

`-max_widths integer`

Specifies the maximum width for each table column in the report specified by the `-name` parameter.

Note: You cannot specify `-max_widths` parameter without `-name`.

You can separate each specified width value with a comma (,). For example, if you specify:

`-max_widths {10,20,30}`

The first column will have a maximum width of 10, the second column will have a maximum width of 20, and the third column will have a maximum width of 30. The fourth and fifth columns will remain the default width spacing.

Commas are also specified as column "placeholders", when you want to change the spacing of a column without changing the spacing of the preceding column. For example, if you specify:

`-max_widths {,20,30}`

The third column will have a maximum width of 20, the fourth column will have a maximum width of 30, and the first, second, and fifth columns will remain the default width spacing.

If you specify:

`-max_widths {20,,30}`

The first column will have a maximum width of 20, the third column will have a maximum width of 30, and the second, fourth and fifth columns will remain the default width spacing.

Each column resizes itself to be between the range of the maximum and minimum widths specified for that column, based on the data in that column. Overruns are wrapped within the same column, unless otherwise specified.

Default: `{50,50,50,50,50}`

<code>-min_widths integer</code>	Specifies the minimum width for each table column in the report specified by the <code>-name</code> parameter. Note: You cannot specify <code>-min_widths</code> parameter without <code>-name</code>. You can separate each specified width value with a comma (,). For example, if you specify: <pre>-min_widths {7,10,15}</pre> The first column will have a minimum width of 7, the second column will have a minimum width of 10, and the third column will have a minimum width of 15. The fourth and fifth columns will remain the default width spacing. Commas are also specified as column "placeholders", when you want to change the spacing of a column without changing the spacing of the preceding column. For example, if you specify: <pre>-min_widths {,,10,15}</pre> The third column will have a minimum width of 10, the fourth column will have a minimum width of 15, and the first, second, and fifth columns will remain the default width spacing. If you specify: <pre>-min_widths {7,,10}</pre> The first column will have a minimum width of 7, the third column will have a width of 10, and the second, fourth and fifth columns will remain the default width spacing. Each column resizes itself to be between the range of the maximum and minimum widths specified for that column, based on the data in that column. Overruns are wrapped within the same column, unless otherwise specified. Default: {5,5,5,5,5}
<code>-minor_sort integer</code>	Specifies the column to be used in the minor sort. It applies to <i>all</i> rows that have the same values in the <code>major_sort</code> column. If the column number is negative, the software reverses the sort.

<code>-name <i>string</i></code>	Changes the formatting for the specified table. See Table for a list of table names you can use, and the commands with which they work.
<code>-no_frame_fix_width</code>	Specifies that the table columns have a fixed minimum width, but no maximum width (the data does not wrap). Column data that fits within the column's minimum width will left-justify and pad its location to fill the specified column space. Column data that is longer than the column's minimum width will take as much space in the table as needed. If a column is overwritten by the data from the preceding column, its data will begin on a new line at the correct column location. This is the default format for tables.
<code>-nosplit</code>	Specifies that each column follows the preceding column without splitting to a new line, if the data in one column overruns the next column. The wrapping behavior for data is controlled by the size of the display window. This format is useful for machine parsing the report. Note: You must specify <code>-no_frame_fix_width</code> in order to use this parameter.
<code>-reverse_rows</code>	Reverses the data rows. This is useful, for example, if you want to trace back from an end point and the default report traces forward from the end point.

The timing report formatting depends on the specified `set_table_style` parameters. These are described below:

- When `set_table_style -no_frame_fix_width` parameter is specified, the timing report displays long inst/hpin names and the rest of the column values are shown on the next line.
- When `set_table_style -no_frame_fix_width -nosplit` is specified, the timing report displays long inst/hpin names and rest of the column values are shown on the same line, because `-nosplit` table style is specified. In this case, other column headers (beyond inst/pin) may not be aligned properly.
- The following settings are specified:

```
set_table_style -no_frame_fix_width -nosplit
set_table_style -name report_timing -max_width {50,50,,,,}
```

The generated report will show properly aligned column headers and end-point/begin-points are not split.

- The following settings generate a report displaying longer (than above reports) inst/pin names:

```
set_table_style -no_frame_fix_width -nosplit
```

```
set_table_style -name report_timing -max_width {120,120,,,,}
```

In this report output, the column headers are aligned properly and are not split.

List of Table Names

Table Name	Formats Reports Generated by
report_cell_instance_delays_arcs	report_cell_instance_timing
report_cell_instance_delays_checks	report_cell_instance_timing
report_cell_instance_delays_pins	report_cell_instance_timing
report_inactive_arcs	report_inactive_arcs
report_nets	report_nets
report_net_header	report_nets
report_net_sink_table	report_nets
report_net_source_table	report_nets
report_net_summary_table	report_nets
report_path_exceptions	report_path_exceptions
report_port	report_ports
report_timing	report_timing
report_timing_summary	report_timing

Examples

- The following command resizes the first column to a maximum range of 70 and specifies a default width of 50 for the second column. The default for the maximum width is 50 and the default for the minimum width is 5:

```
litmus> set_table_style -name report_timing -max_widths {70,}
```

Note: Do *not* include a blank space between commas ", " when specifying widths. If you are specifying a value for the width(s), make sure there is *no* leading or trailing blank space.

- The following command specifies a default width of 50 for the first and second column:

```
litmus> set_table_style -name report_timing -max_widths {,}
```


- The following timing report shows the table style generated with the `-no_frame_fix_width` parameter:

```
litmus> report_timing -format {cell arc delay arrival required instance}
```

```
Path 1: MET Setup Check with Pin seg3/u14/CK
Endpoint: seg3/u14/D (^) checked with leading edge of 'CLK3'
Beginpoint: seg3/u9/Q (^) triggered by leading edge of 'CLK3'
Path Groups: {CLK3}
Other End Arrival Time 0.621
- Setup 0.186
+ Phase Shift 2.000
= Required Time 2.434
- Arrival Time 2.062
= Slack Time 0.372
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.682
= Beginpoint Arrival Time 0.682
```

Cell	Arc	Delay	Arrival Time	Required Time	Instance
	CK ^		0.682	1.054	seg3/u9
DFF	CK ^ -> Q ^	0.185	0.867	1.240	seg3/u9
BUF	A ^ -> Y ^	0.073	0.940	1.312	seg3/u10
INV	A ^ -> Y v	0.317	1.257	1.629	seg3/u11
INV	A v -> Y ^	0.402	1.659	2.031	seg3/u12
BUF	A ^ -> Y ^	0.403	2.062	2.434	seg3/u13
DFF	D ^	0.000	2.062	2.434	seg3/u14

- The following sample timing report shows the table style generated with the `-no_frame_fix_width` and `-nosplit` parameters:

```
litmus> report_timing -format {cell arc delay arrival required instance}
```

```
Path 1: MET Setup Check with Pin seg3/u14/CK
Endpoint: seg3/u14/D (^) checked with leading edge of 'CLK3'
Beginpoint: seg3/u9/Q (^) triggered by leading edge of 'CLK3'
Path Groups: {CLK3}
Other End Arrival Time 0.621
- Setup 0.186
+ Phase Shift 2.000
= Required Time 2.434
- Arrival Time 2.062
= Slack Time 0.372
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.682
= Beginpoint Arrival Time 0.682
```

Litmus Text Command Reference
Timing Analysis Commands--set_table_style

Cell	Arc	Delay	Arrival Time	Required Time	Instance

CK ^			0.682	1.054	seg3/u9
DFF	CK ^ -> Q ^	0.185	0.867	1.240	seg3/u9
BUF	A ^ -> Y ^	0.073	0.940	1.312	seg3/u10
INV	A ^ -> Y v	0.317	1.257	1.629	seg3/u11
INV	A v -> Y ^	0.402	1.659	2.031	seg3/u12
BUF	A ^ -> Y ^	0.403	2.062	2.434	seg3/u13
DFF	D ^	0.000	2.062	2.434	seg3/u14

Related Commands

- [report_cell_instance_timing](#)
- [report_inactive_arcs](#)
- [report_nets](#)
- [report_path_exceptions](#)
- [report_ports](#)
- [report_timing](#)

set_timing_context_clock_mapping

```
set_timing_context_clock_mapping
[-help]
-block_clock block_clock_name
[-instance string]
-top_clock top_clock_name
[-top_view view_name]
[-view view_name]
```

Specifies user-defined clock mapping in context analysis. The auto-inferred clock mappings will be overridden by user-specified mappings. This command should not be specified directly in the Tempus shell.

Note: To ensure correct behavior, this command must be specified in the clock mapping file and subsequently loaded into the Tempus environment with the following command:

```
read_timing_context -clock_mapping_file clock_mapping_file
```

This command should not be run as a standalone command; it must be enclosed in the mapping file.

Parameters

-help	Prints out the command usage.
-block_clock <i>block_clock_name</i>	Specifies the name of the clock from the block-level analysis. The software maps the specified clock with the clock specified using the -top_clock parameter.
-instance <i>string</i>	Specifies the list of block instances for which the clock mapping is to be applied. When this parameter is not specified, the clock mapping is applied to all instances for which context analysis is to be performed.
-top_clock <i>top_clock_name</i>	Specifies the name of top level clock.
-top_view <i>view_name</i>	Specifies the name of the clock from top-level analysis. The software maps the specified clock with the clock specified using the -block_clock parameter.
-view <i>view_name</i>	Specifies the MMMC analysis view for which the top-block clock mapping is to be applied.

Examples

- In the following example, the block clock BCLK is mapped to top clock TCLK in block view B_view:

```
set_timing_context_clock_mapping -top_clock TCLK -block_clock BCLK -view B_view
```
- In the following example, the block clock BCLK is mapped to both top clocks TCLK1 and TCLK2 in block view B_view:

```
set_timing_context_clock_mapping -block_clock BCLK -top_clock TCLK1 -view B_view  
set_timing_context_clock_mapping -block_clock BCLK -top_clock TCLK2 -view B_view -add
```

- In the following example, the block clock BCLK is mapped only to TCLK2 in block view B_view:

```
set_timing_context_clock_mapping -block_clock BCLK -top_clock TCLK1 -view B_view  
set_timing_context_clock_mapping -block_clock BCLK -top_clock TCLK2 -view B_view
```

Related Commands

- [set_timing_context_view_mapping](#)
- [read_timing_context](#)
- [write_timing_context](#)

set_timing_derate

```
set_timing_derate
[-help]
[obj_list]
[value]
[-add]
[-cell_check]
[-cell_delay]
[-clock]
[-clock_period_check]
[-corner]
[-data]
[-dynamic]
[-early float]
[-fall]
[-increment]
[-input_switching]
[-late float]
[-mean]
[-min | -max | -delay_corner delayCornerName]
[-multiply]
[-net_delay]
[-power_domain string]
[-pulse_width_check]
[-retain_delay]
[-rise]
[-sigma]
[-static]
[-statistical]
[-transition]
```

Sets delay scaling or derating factors for early and late paths in the current design. You use this command to perform scaled on-chip variation around best-case and worst-case corners. The timing scaling factors affect the delay values generated in the timing reports. You can set scaling factors for data paths, clock paths, minimum and maximum operating conditions.

If you do not specify the `-clock` or `-data` parameter, the software applies the scaling factors to both clock and data paths. If you specify only one of the two parameters, `-data` or `-clock`, the software uses the last defined value for the other.

Similarly, if you do not specify the `-min` or `-max` parameter, the software applies the scaling factor to both minimum and maximum operating conditions. You should only use the `-min` or `-max` parameter for best-case worst-case analysis. If you specify them in on-chip variation (OCV) mode, the software will use the `-min -early` derating factor for early path scaling, and the `-max -late` derating factor for late path adjustments.

When there are separate max and min libraries specified, only those `set_timing_derate` constraints are honored that are specified with max library. The constraints specified with min library are ignored.

In MMMC mode, for delay corners having different late and early library sets, only those `set_timing_derate` constraints are honored that are specified with late library set of the delay corner. The constraints specified with early library set are ignored.

Note: The `set_timing_derate` command can only be used on the command line. You cannot import it as part of an SDC file.

Note: If the `set_timing_derate` constraint is added incrementally in SI mode, an explicit `update_timing -full` command is required (before `report_timing` or `report_constraint`) for SI full accuracy.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-add</code>	Incrementally adds or subtracts derates. You can also apply derates on specific leaf instances. This parameter also support nets objects.
<code>-cell_check</code>	Specifies scaling on timing checks. Use the <code>-cell_check</code> option with the <code>-early</code> option to specify scaling on hold arcs and use with the <code>-late</code> option to specify scaling on setup arcs. When you specify this parameter, derating factors are also applied to user constraints such as <code>set_output_delay</code> , <code>set_data_check</code> , and <code>set_clock_gating_check</code> . The <code>-cell_check</code> parameter by default does not apply check derate to output external delay (e.g., <code>set_output_delay</code>). To enable this feature, you can set the <code>timing_apply_check_derate_to_external_output_delay</code> timing attribute to <code>true</code> . When set to <code>true</code> , this attribute applies cell derate to the top boundary port.
<code>-cell_delay</code>	Specifies scaling on cell delays. If neither the <code>-cell_delay</code> nor the <code>-net_delay</code> option is specified, the specified scaling factor applies to both types of delays.
<code>-clock</code>	Applies the derating factors to clock paths only.
<code>-clock_period_check</code>	Specifies derate values for clock period checks.

<code>-corner</code>	Applies derating on cells or nets that are modeled as corner data elements. The specified derating factor is applied to both the mean and sensitivities of the delays (corner objects can have propagated sensitivities). A cell with no timing arc data in a library that has any sensitivity information with respect to any parameter defined in SPDF are considered as a corner object. All cells that do not qualify are considered to be statistical objects. A net with parasitic data that has no sensitivity information with respect to any parameter defined in SPDF is considered as a corner object. All nets that do not qualify are considered to be statistical objects.
<code>-data</code>	Applies the derating factors to data paths only.
<code>-delay_corner <i>delayCornerName</i></code>	Applies the derating factor to the specified delay calculation corner or a list of delay corners. Note: The timing derate can be applied to corners associated with active views only.
<code>derate_value</code>	Specifies the derating value to be used.
<code>-dynamic</code>	Applies the specified derating factor to the delta portion of the net and cell delays. The delay typically models SI crosstalk adjustments from detailed RC analysis or annotation. You can use this parameter in conjunction with the <code>-net_delay</code> or <code>-cell_delay</code> parameter to target the derate to desired components of the overall stage delay. The delay calculation controls allow for the SI crosstalk impact to be apportioned to the cell delay and/or net delay. You should coordinate the use of <code>-dynamic derate</code> with your delay calculation configuration. For more information, you can refer to the following si attributes: • <code>si_delay_delta_annotation_mode</code> • <code>si_delay_separate_on_data</code> You cannot specify <code>-dynamic</code> with <code>-mean</code> or <code>-sigma</code> parameters.

<code>-early float</code>	Specifies that the derating factor applies to early paths. Early paths are launching clock during hold analysis, receiving clock during setup analysis and datapath during hold analysis. Note: To use this parameter, you must specify <code>-delay_corner</code>.
<code>-fall</code>	Applies the derating factor to falling delay and check arcs based on the transition direction at the arc's sink pin. This parameter can be used to selectively apply timing derates for the fall arcs (cell, net, or check arcs).
<code>-increment</code>	Incrementally adds the derate value to the total derate value in all the modes - OCV, AOCV, and SOCV mode. The <code>set_timing_derate</code> command saves two different values – the “base” derate and the “incremental” derate. When the <code>-incremental</code> parameter is specified, the software populates the incremental derates. Notes: • This parameter is used to specify incremental derates for leaf cell instances or library cell objects only. • <code>-increment</code> and <code>-multiply</code> are mutually exclusive parameters. • <code>-increment/-multiply</code> are supported with existing parameters, such as, <code>-late/-early</code>, <code>-rise/-fall</code>, <code>-mean/-sigma</code>. • Unlike base transition derate, the derate values with <code>-increment/-multiply</code> can be greater or smaller than 1. • Base transition derate values will be limited to: Late ≥ 1.0 and < 100.0, Early ≤ 1.0 and > 0.0. By default, a limit check on total derate value will be performed during transition for delay calculation. If the total transition derate is out of range, a warning message will be issued.

	By default, the incremental derates are overwritten. To disable this behavior, you can use the following global variables: • <code>timing_derate_incremental_adjust_additive_mode</code>: Incremental derates are added to each other to obtain the final incremental derate value. • <code>timing_derate_incremental_multiply_accumulative_mode</code>: Incremental derates are multiplied with each other to obtain the final incremental derate value. For more information, see timing category attributes. Note: The <code>-increment</code> parameter is mutually exclusive with <code>-pulse_width_check</code>, <code>-clock_period_check</code>, and <code>-input_switching</code> parameters.
<code>-input_switching</code>	Considers the given derate as simultaneous switching input (SSI) derate that is applicable to early paths only.
<code>-late float</code>	Specifies that the derating factor applies to late paths. Late paths are receiving clock during hold analysis, launching clock during setup analysis and data path during setup analysis. Note: To use this parameter, you must specify <code>-delay_corner</code>.
<code>-max</code>	Applies the derating factor to the maximum operating condition during best-case worst-case analysis. This parameter is ignored during on-chip variation analysis. Note: You cannot use the <code>-max</code> parameter when the software is in multi-mode multi-corner (MMMC) mode. You must use the <code>-delay_corner</code> parameter instead.

<code>-mean</code>	Controls nominal derating. You can apply separate derating the mean components of the delay. This parameter can be used in the SOCV analysis flow to specify a derating factor for the mean component of delays. For cell delays and checks, the <code>-mean</code> parameter is supported on library cell and instance objects. For net delays, it is supported as a global level derate factor. The <code>-multiply</code> and <code>-add</code> option can be used with the <code>-mean</code> parameter to provide an incremental adjustment to a previously specified mean base derate.
<code>-min</code>	Applies the derating factor to the minimum operating condition during best-case worst-case analysis. This parameter is ignored during on-chip variation analysis. Note: You cannot use the <code>-min</code> parameter when the software is in multi-mode multi-corner (MMMC) mode. You must use the <code>-delay_corner</code> parameter instead.
<code>-multiply</code>	Incrementally multiplies derates. You can also apply derates on specific leaf instances. This parameter also support nets objects.
<code>-net_delay</code>	Specifies scaling on net delays.
<code>object_list</code>	Specifies the list of instances, library cells, or nets to which the derating will be applied.
<code>-power_domain string</code>	Allows you to specify a specific power domain for the timing derate factors to be applied.
<code>-pulse_width_check</code>	Specifies derate values for pulse width checks. Note: The <code>-pulse_width_check</code> and <code>-clock_period_check</code> parameters are mutually exclusive. To apply check derating to pulse width checks, set the <code>timing_enable_derating_for_pulse_width_checks</code> attribute to <code>true</code>. By default, the attribute is set to <code>false</code>.

<code>-retain_delay</code>	Specifies scaling on retain constructs. If you specify both <code>-retain_delay</code> and <code>-cell_delay</code> parameters, the software uses the <code>retain_delay</code> value for retain construct scaling. If you do not specify the <code>-retain_delay</code> parameter, retain constructs are scaled in the same manner as cell delays. You cannot specify <code>-retain_delay</code> with <code>-mean</code> or <code>-sigma</code> parameters.
<code>-rise</code>	Applies the derating factor to rising delay and check arcs based on the transition direction at the arc's sink pin. This parameter can be used to selectively apply timing derates for the rise arcs (cell, net, or check arcs).
<code>-sigma</code>	Controls derating on the variation component. You can apply separate derating to the sigma components of the delay. This parameter can be used in the SOCV analysis flow to specify a derating factor for the variation component of delays. For cell delays and checks, the <code>-sigma</code> parameter is supported on library cell and instance objects. For net delays, it is supported as a global level derate factor. The <code>-multiply</code> and <code>-add</code> options can be used with the <code>-sigma</code> parameter to provide an incremental adjustment to a previously specified sigma base derate.
<code>-static</code>	Applies the specified derating factor to the non-delta portion of net delays only. You can specify <code>-net_delay</code> or <code>-cell_delay</code> parameter with the <code>-static</code> parameter. You cannot specify <code>-static</code> with <code>-mean</code> or <code>-sigma</code> parameters.

<code>-statistical</code>	Applies derating on cells or nets that are modeled as statistical data elements. The specified derating factor is applied to both the mean and sigma of delays. If both <code>set_timing_derate -corner</code> and <code>-statistical</code> options are not specified, the derating factor is applied on variation and corner objects equally. This is equivalent to issuing to two separate <code>set_timing_derate</code> commands, one with <code>-statistical</code> option and the other with <code>-corner</code> option.
<code>-transition</code>	Specifies scaling on transitions. When specified, the software allows derate transition and stretch waveforms. For incremental derates applied to the base derate (without the <code>-increment</code> parameter), you can use the following parameters with the <code>-transition</code> parameter: • <code>-increment</code> parameter adds to the base derate to obtain the final derate. • <code>-multiply</code> parameter multiplies the incremental derate with the base derate to obtain the final derate value (<code>-increment</code> parameter is not required). • Validity of final derate value (same limits as base transition derate) is applied only during timing analysis, and not at the time of constraint processing. Note: You can use the <code>-transition</code> parameter with the following parameters: • <code>-delay_corner</code> • <code>-early/-late</code> • <code>-rise/-fall</code> • <code>-mean/-sigma</code> • <code>-increment</code> • <code>-multiply</code>

Examples

- The following command (1) can be used when the software is in on-chip variation analysis mode to add pessimism to both the setup and hold analyses by making the early paths 20 percent faster:

```
set_db timing_analysis_type ocv
...
(1) set_timing_derate -early 0.8 -late 1.0
```
- The following example uses two `set_timing_derate` commands to set the scaling factors independently for the min and max corners. When the software is in best-case worst-case analysis mode:

- Command (1) adds pessimism to the setup analysis by making the latching clock path 20 percent faster.
- Command (2) adds pessimism to the hold analysis by making the latching clock path 10 percent slower.

```
(1) set_timing_derate -max -early 0.8 -late 1.0
(2) set_timing_derate -min -early 1.0 -late 1.1
```

- When the software is in on-chip variation analysis mode, there is only one active delay corner; therefore, the `-min` and `-max` parameters should not be used. The following command sequence will generate a warning:

```
set_db timing_analysis_type ocv
...
set_timing_derate -min -early 0.8 -late 0.9
set_timing_derate -max -early 1.1 -late 1.2
```

In this situation, the software uses 0.8 for derating early paths, and 1.2 for derating late paths.

- The following command sets the early path scaling factor of 0.85 for instance `MULT_9` cell delays:

```
set_timing_derate -early -cell_delay 0.85 [get_cells {MULT_9}]
```

- The following command sets the early path scaling factor of 0.75 for the net named `MUX2/Z`:

```
set_timing_derate -net_delay -early 0.75 [get_nets {MUX2/Z}]
```

- The following command specifies power domain `PD1` for derating instances:

```
set_timing_derate -power_domain PD1 -cell_delay 1.1
```

- In the following example, a design having max library as 'slow' and min library as 'fast', only the first two `set_timing_derate` constraints are honored:

```
set_timing_derate -late 1.2 [get_lib_cells {slow/BUFX1}] -cell_delay
set_timing_derate -early 0.8 [get_lib_cells {slow/BUFX1}] -cell_delay
set_timing_derate -late 1.1 [get_lib_cells {fast/BUFX1}] -cell_delay
set_timing_derate -early 0.9 [get_lib_cells {fast/BUFX1}] -cell_delay
```

- Consider the following library sets, in MMMC mode:

```
create_library_set -name libset1 -timing slow.lib
create_library_set -name libset2 -timing fast.lib
create_delay_corner -name corner1 -late_library_set libset1 -early_library_set libset2
```

From the following `set_timing_derate` constraints the first and last are honored:

```
set_timing_derate -late 1.2 -cell_delay [get_lib_cells slow/*] -delay_corner corner1
set_timing_derate -early 2.3 -cell_delay [get_lib_cells slow/*] -delay_corner corner1
set_timing_derate -late 3.4 -cell_delay [get_lib_cells fast/*] -delay_corner corner1
set_timing_derate -early 4.5 -cell_delay [get_lib_cells fast/*] -delay_corner corner1
```

- The following command can be used when the software is in on-chip variation analysis mode to add pessimism to both the setup and hold analyses by making the early paths 20 percent faster:

```
set_db timing_analysis_type ocv
...
set_timing_derate -early 0.8 -late 1.0
```

This command adds pessimism to the setup analysis by making the latching clock path 20 percent faster.

- The following example uses two `set_timing_derate` commands to set the scaling factors independently for the min and max corners. When the software is in best-case worst-case analysis mode:
 - The following command sets the early path scaling factor of 0.85 for instance MULT_9 cell delays:

```
set_timing_derate -early -cell_delay 0.85 [get_cells {MULT_9}]
```
 - The following command sets the early path scaling factor of 0.75 for the net named MUX2/Z:

```
set_timing_derate -net_delay -early 0.75 [get_nets {MUX2/Z}]
```

- The following command specifies power domain PD1 for derating instances:

```
set_timing_derate -power_domain PD1 -cell_delay 1.1
```

- The following command shows library object level support for clock (and pulse width) check options:

```
set_timing_derate -delay_corner -clock_period_check 1.05 [object_list]
```

- The following command sets an increment derate of 0.5 on mean and 0.8 on sigma of the cell:

```
set_timing_derate -increment 0.8 -sigma MP_32_INST/p11A
set_timing_derate -increment 0.5 -mean MP_32_INST/p11A
```

To increment the sigma derate further by 0.9, you can use the following command:

```
set_timing_derate -increment 0.9 -sigma MP_32_INST/p11A
```

The resultant sigma derate can be calculated as $1+0.9=1.9$.

When the `timing_derate_incremental_adjust_additive_mode` global variable is set to `true`, the resultant sigma derate will be $1+0.8+0.9=2.7$.

Related Commands

- `report_timing_derate`
- `reset_timing_derate`

set_wire_load_mode

set_wire_load_mode
[-help]
mode

Controls how the software searches through the design hierarchy to find the appropriate wire load model for a net, or a hierarchical part of a net.

Parameters

<code>-help</code>	Outputs a brief description that includes type and default information for each <code>set_wire_load_mode</code> parameter. For a detailed description of the command and all of its parameters, use the man command: <code>man set_wire_load_mode</code>.
<code>mode</code>	Specifies the wire-load model. You can specify one of the following options. • <code>enclosed</code> • <code>segmented</code> • <code>top</code>

	enclosed: Selects the wire load model using an area lookup table for the module that is at the lowest level in the design hierarchy and contains the entire net. When you set the mode to <code>enclosed</code>, the software uses the area of the smallest block enclosing the net to select the wire load model. In the <code>enclosed</code> mode, the software uses the following order of precedence to search the wire load model in: • The inner-most block enclosing the net on which you set the wire load selection group using the <code>set_wire_load_selection_group</code> command, if the <code>auto_wire_load_selection</code> global variable is enabled. Or: The inner-most block enclosing the net on which you set the wire load model using the <code>set_wire_load_model</code> command. • Each higher level hierarchical block for a <code>set_wire_load_selection_group</code> assertion, all the way up to the top-level block, if the <code>auto_wire_load_selection</code> global variable is enabled. Or: Each higher level hierarchical block for a <code>set_wire_load_model</code> assertion, all the way up to the top-level block. • The <code>default_wire_load_selection</code> field in the first timing library. Or, The <code>default_wire_load</code> field in the first timing library. Note: If even the default values are not present in the timing library, you will not be able to use the wireload model functionality.
	segmented: Selects the wire load model in a way similar to the <code>enclosed</code> mode except that each hierarchical segment of the net is treated separately. In the <code>segmented</code> mode: • A segment is defined by pins (including hierarchical pins) connected to a particular net. • When counting the drivers and receivers, only the pins connected to the segment are considered. This information along with the wire load models determine the parasitic information for the segment. • The parasitics of each segment sum up to form the parasitics for the entire net. Note: Each segment may have different wire load models.

	<code>top</code>: Selects the wire load model in a way similar to <code>enclosed</code> and <code>segmented</code> except that all nets use the wire load selection or wire load models defined by the top-level (largest) block enclosing the net.
--	--

	Note: If the first library read in does not define the default wire load mode, the software uses <code>top</code> as the default mode.
--	--

Examples

- The following command sets the wire load mode to `enclosed`:

```
litmus > set_wire_load_mode enclosed
```

Related Commands

- `reset_wire_load_mode`

set_wire_load_model

```
set_wire_load_model  
[-help]  
[obj_list]  
[-library library_name]  
[-max]  
[-min]  
-name wireload_model
```

Specifies the wire load model to be used from the technology library, and sets the wire load model on the design or on a hierarchical object (instance). If the list of instances is not specified, the wire load model is set on the current instance. Otherwise, the wire load model is set on all the specified instances or ports.

Use wire load models for estimating delays before the actual wire loads are back annotated. A technology library contains different wire load models, previously computed based on the analysis of several designs differing in area.

Parameters

-help	Prints out the command usage.
-library library_name	Specifies the technology library to use when searching for the name of the wire load model. You can specify a library name, or a collection of libraries. <i>Default:</i> All technology libraries are searched.
obj_list	Specifies the list of objects - designs, ports, or hierarchical instances.
-max	Sets the wire load model only for the maximum PVT (process, voltage, temperature) corner. <i>Default:</i> The specified wire load model applies to all three PVT corners if neither the -min nor the -max parameter is specified.
-min	Sets the wire load model only for the minimum PVT (process, voltage, temperature) corner. <i>Default:</i> The specified wire load model applies to all three PVT corners if neither the -min nor the -max parameter is specified.
-name wireload_model	Specifies the name of the wire load model in the library.

Examples

- The following command sets the COM3_LV_WLM_0.0 wire load model from the COM3_LV_WIRE library on the current module for all PVT corners:

```
litmus> set_wire_load_model -library COM3_LV_WIRE -name COM3_LV_WLM_0.0
```

- The following command sets the B1X1 wire load from the lca500kv library on the clk1 port for the minimum PVT corner:

```
litmus> set_wire_load_model -library lca500kv -min -name B1X1 clk1
```

- The following command sets the 3M_TO_4500K wire load from the c13tsmc_slow2s108v125c library on the current module for the minimum PVT corner:

```
litmus> set_wire_load_model -library c13tsmc_slow2s108v125c -min -name 3M_TO_4500K
```

Related Commands

- [report_wire_load](#)
- [reset_wire_load_model](#)

set_wire_load_selection_group

```
set_wire_load_selection_group  
[-help]  
group_name  
[hinst_list]  
[-library library_name]  
[-max]  
[-min]
```

Overrides the wire load selection table in the specified library.

Parameters

<code>-help</code>	Outputs a brief description that includes type and default information for each <code>set_wire_load_selection_group</code> parameter. For a detailed description of the command and all of its parameters, use the man command: <code>man set_wire_load_selection_group</code>
<code>group_name</code>	Specifies the name of the wire load selection group.
<code>-library library_name</code>	Specifies the name of the technology library to search for the wire load selection table. When this parameter is not specified, the software searches all the libraries.
<code>hinst_list</code>	Specifies the list of instances.
<code>-max</code>	Controls the choice of a wire load selection table only for the PVT max corner. <i>Default:</i> Selects the table for all PVT corners if neither the <code>-min</code> nor the <code>-max</code> parameter is specified.

<code>-min</code>	Controls the choice of a wire load selection table only for the minimum PVT corner. <i>Default:</i> Selects the table for all PVT corners if neither the <code>-min</code> nor the <code>-max</code> parameter is specified.
-------------------	--

Examples

- The following command selects `TLM` as the selection table used by the library:

```
set_wire_load_selection_group TLM
```
- The following command selects `ASPEC1` as the selection table used by the `sm333s` library; only the minimum PVT corner is affected:

```
set_wire_load_selection_group -min -library sm333s ASPEC1
```

Related Commands

- `reset_wire_load_selection_group`

update_analysis_view

```
update_analysis_view  
[-help]  
-name existingAnalysisViewName  
{[-constraint_mode newModeName]  
[-delay_corner newDelayCornerObj]  
[-latency_file string]  
[-power_modes powerModesList]}
```

Updates the attribute values of the specified existing analysis view.

You can update an analysis view before the multi-mode multi-corner definition file is loaded into the design, or after. However, after the software processes the file, any change to an existing object when the software is in multi-mode multi-corner mode results in the timing, delay calculation, and RC data being reset.

Parameters

-help	Prints out the command usage.
-constraint_mode <i>newModeName</i>	Specifies the name of the new constraint mode to associate with this analysis view.
-delay_corner <i>newDelayCornerObj</i>	Specifies the name of the new delay calculation corner to associate with this analysis view.
-latency_file <i>string</i>	Specifies the latency file name.
-name <i>existingAnalysisViewName</i>	Specifies the name of the existing analysis view to update.
-power_modes <i>powerModesList</i>	Specifies list of power modes to use with this view. The modes must exist in the power intent file.

Examples

- The following command creates an analysis view called `missionSlow` using the constraint mode `missionSetup` and the delay calculation corner `dcWCCOM`:

```
litmus> create_analysis_view  
-name missionSlow  
-constraint_mode missionSetup  
-delay_corner dcWCCOM
```

The following command changes the delay calculation corner associated with the analysis view `missionSlow` to `dcMax`:

```
litmus> update_analysis_view -name missionSlow -delay_corner dcMax
```

Related Commands

- `create_analysis_view`
- `set_analysis_view`

update_constraint_mode

```
update_constraint_mode
[-help]
[-name modeName]
{[-sdc_files {file1.sdc file2.sdc ...}]
[-ilm_sdc_files {file1.sdc file2.sdc ...}]
[-tcl_variables {{var_name1 value1} {var_name2 value2} {var_name3 value3} ...}]
[-reset_latency_files ]] }
```

Updates the SDC constraint file information for the specified existing constraint mode object.

Note: The `update_constraint_mode` command overwrites the previous constraint mode settings. However, by default, this command updates a constraint mode without affecting other constraint modes, including interactive constraints, such as, `set_dont_use` and `set_dont_touch`.

You can update a constraint mode object before the multi-mode multi-corner definition file is loaded into the design, or after. However, after the software processes the file, any change to an existing object when the software is in multi-mode multi-corner mode results in the timing, delay calculation, and RC data being reset.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-ilm_sdc_files {<i>file1.sdc file2.sdc ...</i>}</code>	Specifies the new SDC constraint files for the ILM flow to associate with this constraint mode.
<code>-name <i>modeName</i></code>	Specifies the name of the existing constraint mode object to update.
<code>-reset_latency_files</code>	Cleans up latency files of all related views.
<code>-sdc_files {<i>file1.sdc file2.sdc ...</i>}</code>	Specifies the new SDC constraint files to associate with this constraint mode.
<code>-tcl_variables {{<i>var_name1 value1</i>} {<i>var_name2 value2</i>} {<i>var_name3 value3</i>} ...}</code>	Specifies a list of TCL variables and values to be used in the constraint mode.

Examples

- The following command groups the SDC files io.sdc, mission1-clks.sdc, and mission1-except.sdc to create a mode object named missionSetup:

```
litmus> create_constraint_mode -name missionSetup  
-sdc_files [list io.sdc mission1-clks.sdc mission1-except.sdc]
```

The following command changes the SDC files associated with the constraint mode missionSetup to io.sdc, mission3-clks.sdc, and mission3-except.sdc:

```
litmus> update_constraint_mode -name missionSetup  
-sdc_files {io.sdc mission3-clks.sdc mission3-except.sdc}
```

Related Commands

- `create_constraint_mode`

update_delay_corner

```
update_delay_corner
[-help]
-name delayCornerName
[-early_estimated_worst_irDrop_factor percent]
[-early_irdrop_data files_or_directories]
[-early_irdrop_files list_of_files]
[-early_rc_corner rcCornerObj]
[-early_temperature_files list_of_files]
[-early_timing_condition conditional_pairs]
[-late_irdrop_data files_or_directories]
[-late_irdrop_files list_of_files]
[-late_rc_corner rcCornerObj]
[-late_temperature_files list_of_files]
[-late_timing_condition conditional_pairs]
[-late_estimated_worst_irDrop_factor percent]
[-irdrop_data files_or_directories]
[-irdrop_files list_of_files]
[-pg_net_voltages voltage_pairs]
[-rc_corner rcCornerObj]
[-si_enabled {true | false}]
[-temperature_files list_of_files]
[-timing_condition conditional_pairs]
```

Modifies the parameters of an existing delay calculation corner object. You can use the `update_delay_corner` command to add or change attribute values for an existing delay calculation corner object, or for an existing power domain definition.

You can update a delay calculation corner object before the multi-mode multi-corner definition file is loaded into the design, or after. However, after the software processes the file, any change to an existing object when the software is in multi-mode multi-corner mode results in the timing, delay calculation, and RC data being reset.

You also can use the `update_delay_corner` command to add a power domain definition to the specified delay calculation corner. Instances contained within an MSMV power domain often require a different library or operating condition that used for the default power domain. Power domain definitions within a delay calculation corner object can be used to assign specific libraries and PVT settings per power domain.

Note: Use separate delay calculation corners to define major PVT corner differences. Use the `-early_*` and `-late_*` parameters within a single delay calculation corner to control on-chip-variation.

The `-early_*` and `-late_*` parameters can be specified separately, to update a delay corner object with early or late information. If you only specify one of the `-early_*` or `-late_*` parameters, the software uses the original information pointer in place of the missing early/late parameter.

For example, if you initially created a delay corner object using:

```
create_delay_corner
-library_set libsNominal
...
```

Use this command after creating at least one delay calculation corner ([create_delay_corner](#)).

Parameters

<code>-help</code>	Prints out the command usage.
<code>-early_estimated_worst_irDrop_factor percent</code>	Specifies the early voltage derating factor. When this parameter is specified, the software performs delay calculations using the voltage = $\text{nomVolt} * (1 - \text{value})$. <i>Default: 0.0</i>
<code>-early_irdrop_data files_or_directories</code>	Specifies the IR drop files or directories to apply when calculating the early arrival times at a single delay corner.
<code>-early_irdrop_files list_of_files</code>	Specifies the list of IR drop files to apply when calculating early arrival times at a single power domain. This parameter is used primarily when configuring the multi-mode multi-corner environment for on-chip-variation (OCV) analysis. Note: This parameter is obsolete and will be removed in a future release. To ensure compatibility with future releases, you should update your scripts.
<code>-early_rc_corner rcCornerObj</code>	Specifies the RC corner object to associate with the early corner object.
<code>-early_temperature_files list_of_files</code>	Specifies the name of the temperature file for temperature aware delay calculation for early corner.
<code>-early_timing_condition conditional_pairs</code>	Specifies pair list of domains/supply sets and timing conditions to associate with this delay corner object.
<code>-irdrop_data files_or_directories</code>	Specifies the IR drop files or directories to apply to both the early and late delay calculation for the specified delay corner object.

<code>-irdrop_files list_of_files</code>	Specifies the list of IR drop files to apply to both early and late delay calculation for this power domain object. This parameter is used primarily to configure single-corner or Best-Case Worst-Case (BC-WC) analysis modes. Note: This parameter is obsolete and will be removed in a future release. To ensure compatibility with future releases, you should update your scripts.
<code>-late_estimated_worst_irDrop_factor percent</code>	Specifies the late voltage derating factor. When this parameter is specified, the software performs delay calculations using the voltage = $\text{nomVolt} * (1 - \text{value})$. <i>Default:</i> 0.0
<code>-late_irdrop_data files_or_directories</code>	Specifies the IR drop files or directories to apply when calculating the late arrival times at a single delay corner.
<code>-late_irdrop_files list_of_files</code>	Specifies the list of IR drop files to apply when calculating late arrival times at a single power domain. This parameter is used primarily when configuring the multi-mode multi-corner environment for on-chip-variation (OCV) analysis. Note: This parameter is obsolete and will be removed in a future release. To ensure compatibility with future releases, you should update your scripts.
<code>-late_temperature_files list_of_files</code>	Specifies the name of the temperature file for temperature aware delay calculation for late corner.
<code>-late_rc_corner rcCornerObj</code>	Specifies the RC corner object to associate with the late corner object.

<code>-late_timing_condition</code> <code>conditional_pairs</code>	Specifies pair list of domains/supply sets and timing conditions to associate with this delay corner object. Note: The <code>-early_timing_condition</code> and <code>-late_timing_condition</code> parameters should be specified together.
<code>-name</code> <i>delayCornerName</i>	Specifies the delay calculation corner to which to add the power domain definition, or to which to update the delay corner or power domain attribute values.
<code>-pg_net_voltages</code> <i>voltage_pairs</i>	Specifies the power/ground net voltages to be used for both early/late timing analysis.
<code>-rc_corner</code> <i>rcCornerObj</i>	Specifies the new RC corner object to associate with this delay corner.
<code>-si_enabled</code> { <code>true</code> <code>false</code> }	Controls signal integrity (SI) analysis. When set to <code>true</code>, the analysis of all MMMC views will include SI analysis by default. You can use this parameter to further determine which analysis views receive SI analysis and which do not. When this parameter is set to <code>false</code> on a delay corner, the software will disable SI analysis for all the views that reference that corner. By default (or when this parameter is set to <code>true</code>), SI analysis is computed for all the views related to that corner - whenever SI is globally enabled. (The global control of SI analysis is determined by the <code>delaycal_enable_si true</code> attribute setting.)
<code>-temperature_files</code> <i>list_of_files</i>	Specifies the name of the temperature file for temperature aware delay calculation for single corner.

`-timing_condition conditional_pairs`

Specifies pair list of domains/supply sets and timing conditions to associate with the defined delay corner object.

Some of the conditional pairs are explained below:

- `timing_condition`: Specifies the default timing condition which applies to all cells not covered by any other timing condition.
- `power_domain_list@timing_condition`: The named timing condition applies to the list of all power domains (defined in a power intent file).
- `supply_set_list@timing_condition`: The named timing condition applies to all blocks tied to the specified supply set.
- `hinst@timing_condition`: The named timing condition applies to the specified hierarchical instance.

The timing conditions must be defined with the `create_timing_condition` command.

Examples

- The following example specifies a list of conditional pairs that are domains/supply sets and the timing conditions associated with the specified delay corner object:

```
update_delay_corner -name ccworst_setup\  
-timing_condition {tc_1 PD1@tc_2 PD2@tc_2 PD3@tc_2 PD4@tc_2}
```

Related Commands

- `create_delay_corner`
- `create_rc_corner`
- `create_timing_condition`

update_library_set

```
update_library_set
[-help]
[-aocv string]
[-library_side_file string]
-name libSetName
[-si string]
[-socv string]
[-timing string]
```

Updates an existing specified library set.

Note: You can edit a library set before (or after) the multi-mode multi-corner view definition file is loaded into the design. However, after the software processes the file, any change to an existing object when the software is in multi-mode multi-corner mode resets the timing, delay calculation, and RC data.

Parameters

-help	Prints out the command usage.
-aocv <i>string</i>	Specifies a list of new AOCV libraries to include in the specified library set.
- library_side_file <i>string</i>	Specifies the library side file.
-name <i>libSetName</i>	Specifies the name of an existing library set to be updated. The library set must have been defined using the <code>create_library_set</code> command.
-si <i>string</i>	Specifies a list of new libraries and user-defined noise (UDN) models to include in the library set.
-socv <i>string</i>	Specifies a list of new SOCV libraries to be included in the library set.
-timing <i>string</i>	Specifies a list of new timing libraries to include in the specified library set. Note: The ECSM and CCS libraries can be very large and loading them may take significant time. You can convert the ASCII NLDM (Liberty .lib), ECSM, and CCS libraries to a binary version using the <code>write_ldb</code> command. For conversion of libraries to a binary version, refer to write_ldb. Instead of the original ASCII libraries, you can specify the following binary libraries in the configuration file: <pre>create_library_set -name libName -timing "AAA.lib BBB.ecsm CCC.ccs"</pre>

Examples

- The following command creates a library set that associates timing libraries with a nominal voltage of 1 volt with the library name IsCOM-1V:

```
litmus> create_library_set -name IsCOM-1V -timing [list stdcell_F_1V.lib ram_F.lib pad.lib]
```

The following command changes the timing libraries associated with the library set IsCOM-1V:

```
litmus> update_library_set -name IsCOM-1V -timing [list stdcell_CVF_1V.lib ram_CVF.lib  
pad.lib]
```

Related Commands

- `create_library_set`

update_opcond

```
update_opcond  
[-help]  
-name opcondName  
[-process processValue]  
[-temperature temperatureValue]  
[-voltage voltageValue]
```

Updates a set of virtual operating conditions in the specified library that were created using the [create_opcond](#) command.

Parameters

-help	Prints out the command usage.
-name <i>opcondName</i>	Specifies the name of the operating condition to be updated.
-process <i>processValue</i>	Specifies the process value for the operating condition.
-temperature <i>temperatureValue</i>	Specifies the temperature value for the operating condition.
-voltage <i>voltageValue</i>	Specifies the voltage value for the operating condition.

Related Commands

- [set_opcond](#)
- [create_opcond](#)

update_rc_corner

```
update_rc_corner
[-help]
[-temperature temperatureValue]
[-cap_table newcapTableFile]
-name existingRcCornerName
[-post_route_res {resFactor1 resFactor2 resFactor3}]
[-post_route_cap {capFactor1 capFactor2 capFactor3}]
[-post_route_cross_cap {xcapFactor1 xcapFactor2 xcapFactor3}]
[-post_route_clock_res {resFactor1 resFactor2 resFactor3}]
[-post_route_clock_cap {capFactor1 capFactor2 capFactor3}]
[-pre_route_clock_res resFactor]
[-pre_route_clock_cap capFactor]
[-pre_route_res resFactor]
[-pre_route_cap capFactor]
[-qrc_tech newFileName]
[-via_variation_file <string>]
```

Adds or changes attribute values for the specified existing RC corner object.

You can edit an RC corner object before the multi-mode multi-corner definition file is loaded into the design, or after. However, after the software processes the file, any change to an existing object when the software is in multi-mode multi-corner mode results in the timing, delay calculation, and RC data being reset.

You can set different postroute scale factors for the postroute engine variants by specifying them as duplets and triplets. For example,

```
resFactor1 resFactor2 resFactor3
```

Where:

- Single value: If you specify one value, the scale factor applies to effort level `low`. Scale factor value of 1 is used for `medium` and `high` by default.
- Duplet: If you specify two values, the first value is used for effort level `low` and the second value is used for `medium`. Scale factor value of 1 is used for `high` by default.
- Triplet: If you specify three values, the first value is used for effort level `low`, the second value is used for `medium`, and the third value is used for `high`.

Note: If the `read_spef` command is specified without the `-rc_scale` parameter, for example, "`read_spef -rc_corner early_spef_${rc_corner}`", then the software does not apply capacitance scale factor specified in `-post_route_cap` and `-post_route_cross_cap` parameters. If the `read_spef` command is specified with the `-rc_scale` parameter, for example, "`read_spef -rc_corner early_spef_${rc_corner} -rc_scale` ", then the software applies the capacitance scale factor. Here, no factor is specified in the `-rc_scale` option.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-cap_table newCapTableFile</code>	Specifies a new capacitance table to use for extraction when using this RC corner.
<code>-pre_route_cap capFactor</code>	Specifies the capacitance scale factor for RC Extraction in pre-route mode.
<code>-pre_route_clock_cap capFactor</code>	Specifies the scale factor for capacitance of clock nets in preroute mode. Default: 0 When this parameter is not specified, the value specified with the <code>-pre_route_cap</code> parameter is used for preroute extraction. When specified, this parameter overrides the value specified with the <code>-pre_route_cap</code> parameter for clock nets.
<code>-pre_route_clock_res resFactor</code>	Specifies the scale factor for resistance of clock nets in preroute mode. Default: 0 When this parameter is not specified, the value specified with the <code>-pre_route_res</code> parameter is used for preroute extraction. When specified, this parameter overrides the value specified with the <code>-pre_route_cap</code> parameter for clock nets.
<code>-pre_route_res resFactor</code>	Specifies the resistance scale factor for RC Extraction in pre-route mode.
<code>-post_route_cap {capFactor1 capFactor2 capFactor3}</code>	Specifies the capacitance scale factor for RC Extraction in post-route mode.

<code>-post_route_clock_cap {capFactor1 capFactor2 capFactor3}</code>	Specifies the scale factor for capacitance of clock nets in the postroute extraction mode. Default: 0 When this parameter is not specified, the value specified with the <code>-post_route_cap</code> parameter is used for postroute extraction. When specified, this parameter overrides the value specified with the <code>-post_route_cap</code> parameter.
<code>-post_route_clock_res {resFactor1 resFactor2 resFactor3}</code>	Specifies the scale factor for resistance of clock nets in the postroute extraction mode. Default: 0 When this parameter is not specified, the value specified with the <code>-post_route_res</code> parameter is used for postroute extraction. When specified, this parameter overrides the value specified with the <code>-post_route_res</code> parameter for clock nets.
<code>-post_route_res {resFactor1 resFactor2 resFactor3}</code>	Specifies the resistance scale factor for RC extraction in post-route mode.
<code>-name existingRcCornerName</code>	Specifies the name of the existing RC corner object to update.
<code>-qrc_tech newFileName</code>	Specifies a new QRC technology (.tch) file which is used to perform sign-off RC extraction.
<code>-temperature newTempValue</code>	Specifies a new temperature, in units of Celsius, to use to derate resistance values.
<code>-post_route_cross_cap {xcapFactor1 xcapFactor2 xcapFactor3}</code>	Specifies the cross-coupling capacitance scale factor for RC extraction in postroute mode.

`-via_variation_file string`

Specifies a file for modeling via variation. The VIA variation file lists up all the VIA layer names with the corresponding multiplier. Each RC corner can have its own VIA variation file.

For example:

```
update_rc_corner
-name rcCornerName1 \
-via_variation_file viaVariationFileName1
update_rc_corner
-name rcCornerName2 \
-via_variation_file viaVariationFileName2
```

Examples

- The following command creates an RC corner called `rc-cbest` that uses the capacitance table `myTech_bc.CapTbl`, and derates the resistance values based on the temperature of 50 Celsius:
`create_rc_corner -name rc-cbest -cap_table myTech.4_bc.CapTbl -temperature 50`
- The following command changes the capacitance table used by RC corner `rc-cbest` to `myCap.2_bc.capTbl`, and the temperature by which resistance values are derated to 25 Celsius:
`update_rc_corner -name rc-cbest -cap_table myCap.2_bc.capTbl -temperature 25`

Related Commands

- `create_rc_corner`

update_timing

update_timing
[-help]
[-full]

Updates timing for the current design.

Parameters

-help	Prints out the command usage.
-full	Runs a full timing update for the current design overriding the default behavior of running incremental timing updates, where only a minimal amount of computation is necessary to update the timing analysis information. Note that for full SI accuracy during reporting, an explicit <code>update_timing -full</code> command is required after incrementally applying constraints such as, <code>set_timing_derate</code>, <code>set_input_delay</code>, <code>set_max_delay</code>, <code>set_clock_groups</code>, or <code>create_clock</code> (these constraints only affect SI delay calculation). Note: It is not required to run the <code>update_timing</code> command without the <code>-full</code> parameter because timing is automatically updated by the <code>report_timing</code> and <code>report_constraint</code> commands - except after incrementally applying constraints that only affect SI delay calculation.

Related Commands

- `check_timing`
- `report_timing`
- `report_constraint`

update_timing_condition

```
update_timing_condition
[-help]
[-library_sets string]
-name timingConditionName
[-opcond string]
[-opcond_library string]
```

Updates the virtual operating conditions (set using [create_timing_condition](#) command) with the new settings.

Parameters

-help	Prints out the command usage
-library_sets <i>string</i>	Specifies list of library sets to associate with the specified timing condition.
-name <i>timingConditionName</i>	Specifies a unique name for the timing condition. This name must exist or an error is issued.
-opcond <i>string</i>	Specifies a single name of an operating condition to associate with the timing condition.
-opcond_library <i>string</i>	Specifies a name of the library to search for the defined operating condition.

Examples

- The following example shows the `update_timing_condition` command performs an update on the specified timing condition:

```
create_timing_condition -name TC1 -library_sets {ls1 ls2} -opcond oc1
update_timing_condition -name TC1 -library_sets {ls3}; # TC1 now only has libset ls3
```

- The following commands result in identical results. However, the first command creates the bin TC2 and the second command updates both the options:

```
create_timing_condition -name TC2 -library_sets {ls1 ls2} -opcond oc1
update_timing_condition -name TC2 -library_sets {ls1 ls2} -opcond oc1
```

Related Commands

- [create_timing_condition](#)

write_sdc

```
write_sdc  
[-help]  
[-clocks <string>]  
[-commands <string>]  
[file_name]  
[-exclude_annotated <string>]  
[-path_adjust_only]  
[-path_group_only]  
[-pins <string>]  
[-strict_sdc]  
[-view <string>]  
[-vt_skew_derate_only]
```

Generates the timing constraints file in a third-party compiler format.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-clocks <string></code>	Specifies the name of the clocks to be included in the SDC file.
<code>-commands <string></code>	Specifies the name of the commands to be included in the SDC file.
<code>file_name</code>	Specifies the name of the constraints file. By default, the constraint file is generated in the current directory. To save to a different directory, you need to specify the complete path.
<code>-exclude_annotated string</code>	Generates the constraints file excluding annotated constraints. You can specify one of the following options: • delays • checks • transitions
<code>-path_adjust_only</code>	Exports <code>set_path_adjust_group</code> and <code>set_path_adjust</code> constraints.
<code>-path_group_only</code>	Exports <code>group_path</code> constraints.
<code>-pins <string></code>	Specifies the name of the pins to be included in the SDC file.
<code>- strict_sdc</code>	Produces a SDC file where functions to manipulate collections are expanded. For example, the <code>remove_from_collection</code> <code>all_inputs</code> , <code>all_outputs</code> , <code>all_clocks</code> , and filter commands are replaced by the resulting list of objects. Note: When you specify the <code>-strict_sdc</code> parameter, a warning is issued stating that using the parameter produces a constraint file containing SDC standard constraints only. This may cause some design constraints to be dropped.
<code>-view viewName</code>	Specifies a view name to obtain the desired SDC.
<code>-vt_skew_derate_only</code>	When specified, the software will check the voltage threshold (VT) skew configuration, and report the VT skew derate details as per the voltage groups defined.

Command Order

Use this command after reading in a timing constraints file.

Examples

- The following command generates constraints file `newc` and saves the generated file in the directory `new`:

```
litmus > write_sdc new/newc
```

- Specifying the following constraints will have new entries in the output constraints file:

```
litmus > set_annotated_delay -from u1/A -to u1/Y -cell 2
litmus > set_annotated_check 2 -setup -from f0/CK -to f0/D
```

The `write_sdc` output will be as follows:

```
set_annotated_delay -cell 2 -from [get_pins {u1/A}] -to [get_pins {u1/Y}]
set_annotated_check -setup -from [get_pins {f0/CK}] -to [get_pins {f0/D}] 2
```

- For the following constraints in the design :

```
litmus > set_annotated_delay -from u1/A -to u1/Y -cell 2
litmus > set_annotated_transition 2 u1/A
litmus > set_annotated_check 2 -setup -from f0/CK -to f0/D
```

The `write_sdc -exclude_annotated delays` output will be as follows:

```
set_annotated_transition 2 [get_pins {u1/A}]
set_annotated_check -setup -from [get_pins {f0/CK}] -to [get_pins {f0/D}] 2
```

The `write_sdc -exclude_annotated {delays transitions}` command outputs the following:

```
set_annotated_check -setup -from [get_pins {f0/CK}] to [get_pins {f0/D}] 2
```

Related Commands

- `read_sdc`
- `write_loop_break_sdc`

Timing Constraint Commands

Path Exception Priorities

The following are the path exception priorities if a path in the design matches more than one path exception:

1. `set_false_path`
2. `set_min_delay`, `set_max_delay`
3. `set_multicycle_path`

If there is more than one exception of a given type, for example the `set_multicycle_path` command, the path exception that is more specific has higher priority. A path exception has higher priority if it specifies more of the `-from/-through/-to` parameters. For example, the `-from -to` options would have priority over the `-from` option.

If the path has the same number of reference points:

- `-from` option has priority over the `-to` option
- `-to` option has priority over the `-through` option
- `-clock_from` option has priority over the `-clock_to` option

Note: To check for ignored path exceptions, use the `report_path_exceptions -ignored` command.

The following table shows the priorities for path exceptions applied to the same path.

Priority	Path Exception
1. (Highest)	<code>set_false_path</code>
2.	<code>set_max_delay -from pin_list</code>
3.	<code>set_max_delay -to pin_list</code>
4.	<code>set_max_delay -through pin_list</code>
5.	<code>set_max_delay -clock_from clkwave_name</code>
6.	<code>set_max_delay -clock_to clkwave_name</code>
7.	<code>set_max_delay</code> (The most constraining adjustment has the higher priority over less constraining adjustments.)
8.	<code>set_multicycle_path -from pin_list</code>

9.	<code>set_multicycle_path -to pin_list</code>
10.	<code>set_multicycle_path -through pin_list</code>
11.	<code>set_multicycle_path -clock_from clkwave_name</code>
12.	<code>set_multicycle_path -clock_to clkwave_name</code>
13. (Lowest)	<code>set_multicycle_path</code> (The most constraining adjustment has the higher priority over less constraining adjustments.)

Examples of Path Exception Priorities

The following are examples of the path exception priorities for a path starting at **A**, going through **B**, and ending with **C**. The pairs of examples are ordered such that the highest priority constraint in one pair has precedence over the highest priority constraint in the next pair.

- In the following pair of constraints, the false path overrides the cycle addition for the path from **A** through **B** to **C**. For all other paths from **A**, the cycle addition applies. `set_false_path -from A -through B -to C`
`set_multicycle_path -from A 2`
- In the following pair, the first constraint is applied (two cycles added) because the `-from` option has priority over the `-to` option: `set_multicycle_path -from A 3` `set_multicycle_path -to C 2`
- In the following pair, the first constraint is applied (two cycles added) because it specifies a `-through` pin as well as a `-to` pin: `set_multicycle_path -through A -to B 3` `set_multicycle_path -to B 2`
- In the following pair, the first constraint is applied (two cycles added) because the `-clock_from` option has precedence over the `-clock_to` option: `set_multicycle_path -clock_from CLKA 3` `set_multicycle_path -clock_to CLKB 2`
- In the following pair, the second constraint is applied because the second constraint overwrites the first one: `set_multicycle_path -from A 2` `set_multicycle_path -from A 3`

Bidirectional Pin Defaults

Bidirectional pins have both an input pin and an output pin type. This section explains the default pin type that the system uses for bidirectional pins.

Defaults for Pin (or Port) Constraints

The following defines the pin and port types that are used in the next table.

Bidirectional Pin (or Port) Type Definitions

Pin Type	Type 1	Type 2
Blackbox pins	out	in
Instance pins	out	in
Netlist (primary) ports	in	out
Hierarchical ports	in	out

Bidirectional Pin Type Defaults

Pin or Port Constraint	Type	Hierarchical Port
<code>set_input_delay</code>	1	ignored
<code>set_output_delay</code>	2	ignored
<code>set_clock_latency</code>	1	ignored
<code>create_generated_clock</code>	1	used
<code>set_input_transition</code>	1	ignored
<code>set_driving_cell</code>	1	ignored
<code>set_drive</code>	1	ignored

Defaults for Port Constraints

The following table defines the port types that are used in the next table.

Bidirectional Port Type Definitions

Pin Type	Type 1	Type 2
Netlist (primary) ports	in	out
Hierarchical ports	in	out

Bidirectional Port Type Defaults

Port Constraint	Type	Hierarchical Port

set_fanout_load	2	ignored
set_load	2	ignored
set_max_transition	2	ignored

List of Commands

- [compare_constraints](#)
- [current_design](#)
- [report_clock_mapping](#)
- [reset_annotated_transition](#)
- [reset_clock_exclusivity](#)
- [reset_clock_latency](#)
- [reset_clock_tree_latency](#)
- [reset_disable_clock_gating_check](#)
- [reset_driving_cell](#)
- [reset_ideal_network](#)
- [reset_load](#)
- [reset_max_time_borrow](#)
- [reset_min_fanout](#)
- [reset_mode](#)
- [reset_path_group](#)
- [reset_property](#)
- [reset_pulse_clock_min_transition](#)
- [reset_sense](#)
- [set_annotated_delay](#)
- [set_clock_gating_check](#)
- [set_clock_sense](#)
- [create_clock](#)
- [current_instance](#)
- [reset_annotated_check](#)
- [reset_case_analysis](#)
- [reset_clock_gating_check](#)
- [reset_clock_sense](#)
- [reset_clock_uncertainty](#)
- [reset_disable_timing](#)
- [reset_generated_clock](#)
- [reset_ideal_transition](#)
- [reset_max_capacitance](#)
- [reset_max_transition](#)
- [reset_min_pulse_width](#)
- [reset_output_delay](#)
- [reset_pll_timing](#)
- [reset_pulse_clock_max_transition](#)
- [reset_pulse_clock_min_width](#)
- [set_active_clocks](#)
- [set_annotated_transition](#)
- [set_clock_groups](#)
- [set_clock_transition](#)
- [create_generated_clock](#)
- [group_path](#)
- [reset_annotated_delay](#)
- [reset_clock](#)
- [reset_clock_groups](#)
- [reset_clock_transition](#)
- [reset_data_check](#)
- [reset_drive](#)
- [reset_ideal_latency](#)
- [reset_input_delay](#)
- [reset_max_fanout](#)
- [reset_min_capacitance](#)
- [reset_min_transition](#)
- [reset_path_exception](#)
- [reset_propagated_clock](#)
- [reset_pulse_clock_max_width](#)
- [reset_resistance](#)
- [set_annotated_check](#)
- [set_case_analysis](#)
- [set_clock_latency](#)
- [set_clock_uncertainty](#)

- [set_data_check](#)
- [set_dont_touch](#)
- [set_drive](#)
- [set_fanout_load](#)
- [set_ideal_transition](#)
- [set_load](#)
- [set_max_capacitance](#)
- [set_max_time_borrow](#)
- [set_min_delay](#)
- [set_min_transition](#)
- [set_output_delay](#)
- [set_property](#)
- [set_pulse_clock_min_transition](#)
- [set_sense](#)
- [set_disable_clock_gating_check](#)
- [set_dont_touch_network](#)
- [set_driving_cell](#)
- [set_ideal_latency](#)
- [set_input_delay](#)
- [set_logic_one](#)
- [set_max_delay](#)
- [set_max_transition](#)
- [set_min_fanout](#)
- [set_mode](#)
- [set_pll_timing](#)
- [set_pulse_clock_max_transition](#)
- [set_pulse_clock_min_width](#)
- [set_target_check_constraints](#)
- [set_disable_timing](#)
- [set_dont_use](#)
- [set_false_path](#)
- [set_ideal_network](#)
- [set_input_transition](#)
- [set_logic_zero](#)
- [set_max_fanout](#)
- [set_min_capacitance](#)
- [set_min_pulse_width](#)
- [set_multicycle_path](#)
- [set_propagated_clock](#)
- [set_pulse_clock_max_width](#)
- [set_resistance](#)

compare_constraints

```
compare_constraints  
[-help]  
[-2d2s_compare]  
[-sdc_compare]
```

Parameters

-help	Prints out the command usage
-2d2s_compare	Enables comparison of 2D2S constraints. (bool, optional)
-sdc_compare	Enables comparison of SDC files. (bool, optional)

create_clock

```
create_clock
-period period_value
[-name clock_name]
[-waveform edge_list]
[-add]
[-comment string]
[source_objects]
```

Creates a clock object and defines its waveform in the current design. If you do not specify any `source_objects`, but you specify the `-name clock_name` argument, a virtual clock is created. The new clock has an ideal clock latency, and the software does not assume any propagation delay through the clock network.

The clock that you create using the `create_clock` command cannot have multiple pulses per period. You can specify multiple constraints on the same pin using the `-add` parameter.

If one of the `source_objects` is already the source of a clock, the source is removed from that clock. If you need to specify multiple clocks on the same source for simultaneous analysis, you must specify the `-add` parameter, to add this clock to the existing clock.

For example, if you specify the following command sequence, the PH2 clock waveform overwrites the PH1 waveform.:

```
create_clock -name PH1 ... [get_ports sysclk]
```

```
create_clock -name PH2 ... [get_ports sysclk]
```

To have both clocks applied, you must specify the following command sequence:

```
create_clock -name PH1 ... [get_ports sysclk]
```

```
create_clock -name PH2 -add ... [get_ports sysclk]
```

You can create a collection of clocks, which is a group of objects referenced by a string identifier using the [get_clocks](#) command.

Note: If the `create_clock` constraint is added incrementally in SI mode, an explicit `update_timing -full` command is required (before [report_timing](#) or [report_constraint](#)) for SI full accuracy.

Parameters

<code>-add</code>	Specifies multiple clocks on the same source for simultaneous analysis with different clock waveforms. You must specify the <code>-name</code> parameter with this parameter.
<code>-comment <i>string</i></code>	Allows you to insert comments. The comments are read but are ignored with a one-time warning message. They are not used during timing analysis reporting nor are they written out during saving of constraints.
<code>-name <i>clock_name</i></code>	Specifies the string name of the ideal clock being created. If you do not specify this option, the clock gets the same name as the first clock source that you specify using the <code>-source_objects</code> option. If you do not specify any <code>source_objects</code> , you must specify the <code>-name <i>clock_name</i></code> option, which creates a virtual clock not associated with a port or pin.
<code>-period <i>period_value</i></code>	Specifies the value of the ideal clock in library time units.
<code>source_objects</code>	Specifies the list of pins or ports associated with an ideal clock waveform of the clock. If you do not specify this option, you must specify the <code>-name <i>clock_name</i></code> option, which creates a virtual clock not associated with a port or pin. If you specify a clock on a pin that already has a clock, the new clock replaces the old one.
<code>-waveform <i>edge_list</i></code>	Specifies the rise and fall edge times of the clock waveforms in library time units over an entire clock period. The first value in the edge list is the time at which the first transition on the clock occurs, which is the first rising transition after time zero, and edges are monotonically increasing. The numbers should represent one full clock period. If you do not specify the <code>-waveform <i>edge_list</i></code> option, a default waveform is assumed, and the leading edge is placed at 0 and the trailing edge is placed at the midpoint of the period, such that a symmetric clock is generated.

Examples

- The following command creates an `ideal_clk1` clock on port `clkA` with a period of 5, a rise at 0, and a fall at 2.5:

```
create_clock [get_ports {clkA}] -name ideal_clk1 -period 5 -waveform {0 2.5}
```
- The following command creates an `ideal_clk2` clock on port `clkB` with a period of 10, a rise at 1, and a fall at 9:

```
create_clock [get_ports {clkB}] -name ideal_clk2 -period 10 -waveform {1 9}
```

Related Commands

- `all_clocks`
- `get_clocks`
- `get_pins`
- `get_ports`
- `set_clock_latency`
- `report_clocks`
- `set_clock_uncertainty`
- `set_input_delay`
- `set_output_delay`

create_generated_clock

```
create_generated_clock
[-help]
target_pin_list
[-add]
[-comment string]
[-duty_cycle percent]
[-edge_shift edge_shift_list]
[-invert]
[-master_clock source_clock_name]
[-name clock_name>]
-source source_pin
{[-multiply_by factor]
[-divide_by factor]
[-edges edge_list]
[-combinational]}
```

Creates a new clock signal from the clock waveform of a given pin in the design and binds it with the pins or hierarchical pins in the *target_pin_list* argument. Whenever the source clock changes, the derived clock(s) change automatically.

Generate the new clock waveform using one of three ways:

- Multiply the frequency of the source clock
- Divide the frequency of the source clock
- Select the edges of the source clock to be mapped to the edges of the new clock

A `create_generated_clock` assertion on a pin overrides any existing `create_generated_clock` assertion if both the constraints have the same source pin. In the following command sequence, the second assertion overrides the first:

```
create_generated_clock -name GCK1 -source CK -divide_by 2 U_FD1/Q
create_generated_clock -name GCK2 -source CK -multiply_by 2 U_FD1/Q
```

To have both assertions applied, you must specify the `-add` parameter. For example:

```
create_generated_clock -name GCK1 -source CK -divide_by 2 U_FD1/Q
create_generated_clock -name GCK2 -add -master_clock CLK -source CK -multiply_by 2 U_FD1/Q
```

Parameters

-help	Prints out the command usage.
-------	-------------------------------

<code>-add</code>	Models multiple generated clocks on the same source when multiple clocks must fan into the source pin. Ideally, one generated clock must be specified for each clock that fans into the master pin. Specify this option with the <code>-name</code> and <code>-master_clock</code> options. By default, the software creates one generated clock at the pin by using the fastest clock present on the source pin as the master clock. However, use the <code>-add</code> option to specify a different clock name for each generated clock when used with the <code>-master_clock</code> option. Subsequently, you can use this clock name for setting other constraints, such as the <code>set_false_path</code> command and the <code>set_input_delay</code> command.
<code>-combinational</code>	Restricts the path search for latency to only paths which do not cross sequential devices, or the source pins of other generated clocks. When you specify <code>-combinational</code>, if there are both sequential and combinational latency paths, the software only considers the combinational paths for the generated clock. If there are no combinational latency paths for the generated clock, the generated clock does not inherit latency from its parent clock hierarchy. By default, when calculating the latency of a generated clock, the software searches for the longest (and shortest) paths from the parent waveform to the generated clock point. All paths are considered, including those which traverse sequential elements such as flip-flops, registers, and latches. The default behavior is intended for cases where there is a cascade of generated clocks, and some of the clocks are not explicitly defined. The <code>-combinational</code> option can be used in conjunction with the <code>-multiply_by</code>, <code>-divide_by</code>, or <code>-edges</code> options.
<code>-comment string</code>	Allows you to insert comments. The comments are read but are ignored with a one-time warning message. They are not used during timing analysis reporting nor are they written out during saving of constraints.
<code>-divide_by integer</code>	Determines the frequency of the new clock by dividing the frequency of the source clock by <i>dfactor</i> . For example, if <i>dfactor</i>=5 , the new frequency is 1/5 of the source frequency, and the new clock period is 5 times the source clock period.
<code>-duty_cycle percent</code>	Sets the duty cycle (high pulse width or clock period). The number (<i>percent</i>) is between 0 and 100. For example, if <i>percent</i>=50 , the high pulse width of the new clock waveform is the same as its low pulse width. Note: The <code>-duty_cycle</code> option can only be used with the <code>-multiply_by</code> option, otherwise it will be ignored with a warning.
<code>-edges edge_list</code>	Selects a list of edges from the source clock that form the edges of the derived clock. See the frequency division example below.
<code>-edge_shift edge_shift_list</code>	Specifies the amount of shift for each edge in the <i>edge_list</i> option.
<code>-invert</code>	Inverts the source clock. Use with the <code>-source</code> option and the <code>-multiply_by</code> or <code>-divide_by</code> options.

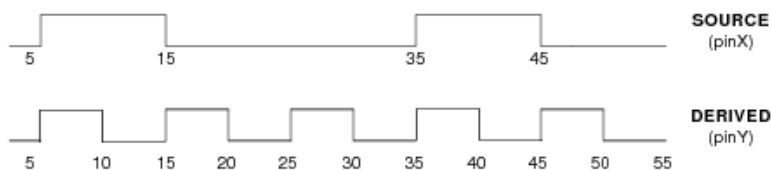
<code>-master_clock</code> <code>source_clock_name</code>	Generates a target clock from the specified clock name. If you specify the <code>-master_clock</code> option and multiple signals arrive at the source pin, the clock specified by this option is used to generate the target clock. Use the <code>-add</code> parameter along with <code>-master_clock</code> parameter to add a clock to the existing clock when you want to specify multiple clocks on the same source for simultaneous analysis.
<code>-multiply_by</code> <code>integer</code>	Determines the frequency of the new clock by multiplying the frequency of the source clock by <i>mfactor</i> . For example, if <i>mfactor</i> =4 , the new frequency is 4 times the source frequency, and the new clock period is 1/4 of the source clock period.
<code>-name</code> <code>clock_signame</code>	Specifies the name for the generated clock. <i>Default:</i> The system creates a name for you. Always specify a name for the generated clock. Some commands issue an error if you use the system-generated name.
<code>-source</code> <code>source_pin</code>	Specifies the name of the source pin from which the new clock is to be derived. This is a mandatory option.
<code>target_pin_list</code>	Specifies the list of pins where the generated clock assertion is applied. The clock signal propagating downstream from each of these pins becomes associated with the generated clock.

Examples

- The following command generates the derived clock waveform, as shown in the figure below:

```
% create_generated_clock -name mult_3 -source pinX -multiply_by 3 -duty_cycle 50 pinY
```

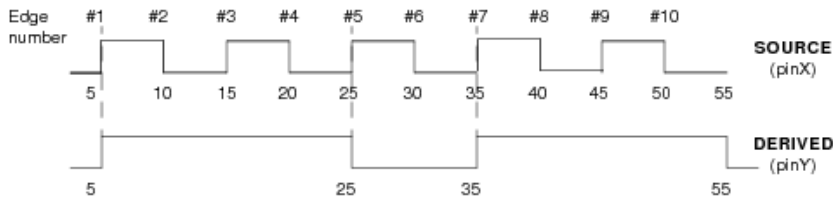
Figure: Derived Clock (Frequency Multiplication)



- The following command shows one way to create a divide-by-3 clock by using edge numbers of the source waveform to specify the new waveform, as shown in the figure below:

```
% create_generated_clock -name genclk -source pinX -edges {1 5 7} pinY
```

Figure: Derived Clock (Frequency Division)



- The following command uses the `-divide_by` option to get the same clock period; the difference is in the duty cycle:

```
% create_generated_clock -name div_3 -source pinX -divide_by 3 pinY
```

With the `-edges` option, the above example creates a clock with a duty cycle of 67 percent (20/30). However, using the `-divide_by 3` option preserves the 50 percent duty cycle of the source clock as shown in the figure below:

Figure: Derived Clock (Frequency Division, 50 Percent Duty-cycle)



- Using the following commands, when there is more than 1 source clock, the tool creates the generated clock based on the source clock specified with the `-master` option:

```
create_clock -name c1 -period 10 [get_ports {ck} ] -add create_clock -name c2 -period 10
[get_ports {ck} ] -add create_generated_clock -name newclk -source ck -divide_by 2 -
master_clock c1 reg/Q -add
```

Using the `report_clocks` command, the output waveform of the newclk clock is a divide-by-2 of the `c1` clock and is reported as shown in the example below.

Example: report_clocks after using the -master option

```
% create_clock -name CK1 -period 20 [get_ports {CLK1} ] -add
% create_clock -name CK2 -period 10 [get_ports {CLK1} ] -add
% create_generated_clock -name GCLK -source CLK1 -divide_by 2 u8/Q -add -master_clock CK2

% report_clocks
```

```
-----
Clock Descriptions
-----
Attributes
-----
```

Clock Name	Source	Period	Lead	Trail	Generated	Propagated
CK1	CLK1	20.000	0.000	10.000	n	n

Litmus Text Command Reference

Timing Constraint Commands--create_generated_clock

```
CK2          CLK1    10.000    0.000    5.000    n          n
-----
```

Generated-Clock Descriptions

Name	Generated Source (pin)	Master Source (pin)	Master-clock	Invert	Freq. Multiplier	Duty-Cycle	Edges	Edge-Shift
GCLK	u8/Q	CLK1	CK2	n	1/2	-	-	-

Related Commands

- `create_clock`
- `get_clocks`
- `report_clocks`

current_design

```
current_design  
[-help]  
[design_name]
```

Changes the current directory in the design hierarchy to the specified design and returns the path to the design.

If no design is specified, the command returns the top-level design.

Parameters

-help	Prints out the command usage
<i>design_name</i>	Specifies the name of the top design.

Related Commands

- `set_max_transition`
- `set_min_capacitance`
- `set_min_transition`
- `list_libraries`

current_instance

current_instance
[-help]
[hinst]

Sets the instance given by *hinst* to be the current instance. Design objects referenced by subsequent commands can be found relative to this instance. All searches for design objects are started in this instance. If design objects are referenced hierarchically, *hinst* is used as the root (top) of the hierarchy.

Parameters

-help	Prints out the command usage.
<i>hinst</i>	Specifies the name of the instance to become the current instance. If <i>hinst</i> is not given, the current instance is set to the top instance of the hierarchy.

Examples

The following command sets the current instance to MAC/MULTI, sets a constant on a register pin inside the hierarchical instance and then later sets the current instance back to top:

```
% current_instance MAC/MULTI
% set_case_analysis 0 [get_pins {reg6/SE}]
...
% current_instance
```

Related Commands

- [set_case_analysis](#)

group_path

```
group_path
[-help]
[-comment string]
-name path_group_name
-from from_list
[-to to_list]
[-through through_list]
```

Groups paths in a design, and identifies them with a path group name. You can use the path group names for reporting.

You cannot use `group_path` statements in the SDC file when the software is in MMMC mode. In this case a warning is issued. However, the software supports the `group_path` command in MMMC mode as a command outside of the SDC file. The path groups are considered to be global in nature and apply to all the views, and are not supported on MMMC mode-specific basis. The path groups are persistent through a `saveDesign` to `restoreDesign` sequence. They are saved in the `<saveDir>/mmmc/pathgroup.sdc` file of a MMMC save directory.

Parameters

-help	Prints out the command usage.
-comment <i>string</i>	Allows you to insert comments. The comments are read but are ignored with a one-time warning message. They are not used during timing analysis reporting nor are they written out during saving of constraints.
-from <i>from_list</i>	Specifies a list of pins, instances, or clocks that are the start of the paths. When you specify a clock object, all paths that are triggered by the clock are included in the path group.
-name <i>path_group_name</i>	Specifies the name of the path group. If the <code>path_group_name</code> already exists, the specified paths are added to the existing path group.
-through <i>through_list</i>	Specifies the pins or instances where the paths go through. When you specify multiple through list, the paths that go through the objects in the specified order are added to the path group.
-to <i>to_list</i>	Specifies a list of pins, instances, or clocks where the path ends. You must specify valid endpoints in the <code>to_list</code> or else the software ignores the constraint. When you specify a clock object, all paths that are checked by the clock are included in the path group. If you specify an instance name in the <code>to_list</code> , all the data input pins of the instance are added to the path group.

Examples

- The following example groups paths that start from specific pins and end at the D pin of any instances whose name start with the string ff. The example also reports the specified path groups:

```
% group_path -name path1 -from {ff1/CLK ff2/CLK} -to {ff*/D}  
% report_timing -path_group path1
```

- The following command groups paths that go through specific points.

```
% group_path -name path2 -through an1/Z -through an2/Z -through an3/Z
```

- If path group definitions are matching/overlapping (when the `-from/-through/-to` parameters are used), then path group resetting is applied. The path group overwriting and prioritization rules are similar to path exceptions. The following examples show how path group resetting is applied.

If the timing path from u1/CK to u2/CK is part of both the path groups, then the path will be reported in the second path group only (that is, the most recent matching path group):

```
group_path -name g2 -from u1/CK,u4/Y -to u2/CK  
group_path -name g1 -from u1/CK -to u2/CK
```

In the following example, the path groups are not overlapping and have different `-from/-thru/-to` and map to the same path, then the path will be reported only in the first path group (that is, the most specific matching path group):

```
group_path -name g2 -from u1/CK -to u2/CK  
group_path -name g1 -to u2/CK
```

In this case, the path will be reported only in the first path group (that is, the most specific matching path group).

Related Commands

- `report_timing`
- `get_path_groups`
- `reset_path_group`
- `report_path_groups`

report_clock_mapping

```
report_clock_mapping
[-help]
[-block_clock <string>]
[-top_clock <string>]
[-unmapped_clocks]
[-scope <string>]
[-clock_path ]
[-data_path ]
```

Reports clock mapping information with optional scoping and clock-based path selection.

Parameters

-help	Prints out the command usage
-block_clock <string>	Specifies the block clock name (string, optional)
-clock_path	Includes clock path information in the clock mapping report (bool, optional)
-data_path	Includes data path information in the clock mapping report (bool, optional)
-scope <string>	Specifies the name of hierarchical instance (string, optional)
-top_clock <string>	Specifies the top clock name (string, optional)
-unmapped_clocks	Includes unmapped clocks in the clock mapping report (bool, optional)

reset_annotated_check

```
reset_annotated_check  
[-help]  
[-clock [rise | fall]]  
[-cond sdf_condtion]  
[-setup | -recovery]  
[-hold | -removal]  
[-nochange_high | -nochange_low]  
[-rise] [-fall]  
{[[-from from_pins][-to to_pins]] | -all}  
[obj_list]
```

Removes annotated timing check values that you had set by using the `set_annotated_check` command or by SDF annotation.

 The `reset_annotated_check` command does not reset the timing check values set by SDF annotations.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-all</code>	Resets all annotated timing check values in the design.
<code>-clock [rise fall]</code>	Specifies the direction of the related pin of the check to reset. <i>Default:</i> Resets checks against both edges of the clock
<code>-cond sdf_condtion</code>	Resets all annotated timing check values for which a condition is specified.
<code>-fall</code>	Restricts the reset to only affect the falling edge of the constrained pins. <i>Default:</i> Checks on both rising and falling signals are reset
<code>-from from_pins</code>	Specifies the beginning instance pin of the timing arcs for which the annotated check is reset. This is usually the reference (for example, clock pin) of the constraint. The selected timing arcs are those that have their source pins specified using this option and the sink pins specified using the <code>-to</code> option. If this option is not specified, only timing arcs whose sink pins are specified using the <code>-to</code> option are selected. You cannot use this parameter with <code>obj_list</code> parameter.
<code>-hold -removal</code>	Resets annotations on the hold or removal check type.
<code>-nochange_high -nochange_low</code>	Resets the annotations on the specified timing check type.
<code>obj_list</code>	Resets the annotated checks on specified list or collection of cell instances.
<code>-rise</code>	Restricts the reset to only affect the rising edge of the constrained pins. <i>Default:</i> Checks on both rising and falling signals are reset
<code>-setup -recovery</code>	Resets annotations on the setup or recovery check type.
<code>-to to_pins</code>	Specifies the terminating instance pin of the timing arcs for which the annotated check is reset. This is usually the constrained pin (data pin) of the check. The selected timing arcs are those that have their sink pins specified using this option and the source pins specified using the <code>-from</code> option. If this option is not specified, only timing arcs whose source pins are specified using the <code>-from</code> option are selected. You cannot use this parameter with <code>obj_list</code> parameter.

Examples

- The following command resets all the timing check annotations in the design:

```
% reset_annotated_check -all
```

- The following command resets all timing check annotations on a particular cell:

```
% reset_annotated_check [get_cells BLK/BR1]
```

- The following command resets specific checks on a specific instance:

```
% reset_annotated_check -setup -hold [get_cells BLK/BR2]
```

- The following command resets checks on a specific timing check arc:

```
% reset_annotated_check -from BLK/BR1/CK -to BLK/BR1/D
```

Related Commands

- [set_annotated_check](#)

reset_annotated_delay

```
reset_annotated_delay  
[-help]  
{[-all] | [[-from from_pins] [-to to_pins]] | [object_list]}
```

Resets annotated delay assertions that have been previously specified using the `set_annotated_delay` command. When you reset an annotated delay, the system uses the delays which are calculated from the original libs for the arc.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-all</code>	Resets all annotated delay assertions in the design.
<code>-from</code> <i>from_pins</i>	Specifies timing arcs for which the delay is reset. The selected timing arcs are those that have their source pins specified using this option and the sink pins specified using the <code>-to</code> option. If this option is not specified, only timing arcs whose sink pins are specified using the <code>-to</code> option are selected.
<i>object_list</i>	Resets the annotated delays on specified list or collection of cell instances.
<code>-to</code> <i>to_pins</i>	Specifies timing arcs for which the delay is reset. The selected timing arcs are those that have their sink pins specified using this option and the source pins specified using the <code>-from</code> option. If this option is not specified, only timing arcs whose source pins are specified using the <code>-from</code> option are selected.

Examples

- The following command resets all the annotated delay assertions in the design:

```
reset_annotated_delay -all
```
- The following command resets all the `set_annotated_delay` assertions applied on pins (either `from` or `to`) of instance `f0`:

```
reset_annotated_delay f0
```
- The following command resets the `set_annotated_delay` assertions with the `from` and `to` pin pair combination:

```
reset_annotated_delay -from u1/Y -to u2/A
```

Related Commands

- [set_annotated_delay](#)

reset_annotated_transition

```
reset_annotated_transition  
[-help]  
{-all | port_or_pin_list}
```

Resets annotated transitions previously asserted by the `set_annotated_transition` command.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-all</code>	Resets all annotated transitions in the design.
<code>port_or_pin_list</code>	Specifies a list of specific pins or primary ports to reset the transition.

Examples

- The following commands reset the `set_annotated_transition` constraint on an internal pin `pad/Y0`.

```
reset_annotated_transition pad/Y0
```

or

```
reset_annotated_transition [get_pins {pad/Y0}]
```

You can also specify a collection as the pin argument. For example:

```
reset_annotated_transition [get_pins pad/*]
```

- The following commands reset the `set_annotated_transition` constraint set on a port `in`:

```
reset_annotated_transition in
```

or

```
reset_annotated_transition [get_ports {in}]
```

Any `set_input_transition` constraint applied on the same port is not removed. This is applicable only for input ports, as `set_input_transition` cannot be applied on output ports.

- The following command resets all the `set_annotated_transition` constraints:

```
reset_annotated_transition -all
```

Related Commands

- `set_annotated_transition`

reset_case_analysis

```
reset_case_analysis  
[-help]  
port_pin_list
```

Removes the assertions set by the set_case_analysis command.

Parameters

<code>-help</code>	Prints out the command usage.
<code>port_pin_list</code>	Specifies the list of ports or pins for which the assertions are to be removed.

Examples

- The following commands sets the assertion to 0 and then resets them:

```
set_case_analysis 0 CM1/S0  
get_property [get_pins CM1/S0] constant_value
```

Returns a value of 0

```
reset_case_analysis [get_pins CM1/S0]  
get_property [get_pins CM1/S0] constant_value
```

Returns a value of NA

Related Commands

- `set_case_analysis`

reset_clock

```
reset_clock  
[-help]  
{-all | clock_list}
```

Removes previously created clock assertions.

Parameters

-help	Prints out the command usage.
-all	Removes all clocks in the design.
clock_list	Specifies the list of clocks for which the clock assertion is removed.

Examples

The following commands reset a list of clocks and all clocks, and display the results using the `report_clocks` command:

```
% report_clocks
```

Generates the following report:

```
-----  
Clock Descriptions  
-----
```

```
Attributes  
-----
```

Clock Name	Source	Period	Lead	Trail	Generated	Propagated
CK1	CLK1	20.000	0.000	10.000	n	n
CK2	CLK1	10.000	0.000	5.000	n	n

```
-----  
Generated-Clock Descriptions  
-----
```

Name	Generated Source (pin)	Master Source (pin)	Master-clock	Invert	Freq. Multiplier	Duty-Cycle	Edges	Edge-Shift
GCLK4	u8/Q	CLK4	CLK_W_4	n	1/2	-	-	-

```
% reset_clock [get_clocks CK1]  
% report_clocks
```

```
-----  
Clock Descriptions  
-----
```

Litmus Text Command Reference

Timing Constraint Commands--reset_clock

Attributes

Clock Name	Source	Period	Lead	Trail	Generated	Propagated
CK2	CLK1	10.000	0.000	5.000	n	n

Generated-Clock Descriptions

Name	Generated Source (pin)	Master Source (pin)	Master-clock	Invert	Freq. Multiplier	Duty-Cycle	Edges	Edge-Shift
GCLK4	u8/Q	CLK4	CLK_W_4	n	1/2	-	-	-

Related Commands

- [create_clock](#)
- [report_clocks](#)

reset_clock_exclusivity

```
reset_clock_exclusivity  
[-help]  
pin_name
```

Allows to reset the `set_clock_exclusivity` constraints applied on pins.

Parameters

<code>-help</code>	Prints out the command usage.
<code><i>pin_name</i></code>	Specifies the name of pins for which clock exclusivity constraints need to be reset.

Related Commands

- `set_clock_exclusivity`

reset_clock_gating_check

```
reset_clock_gating_check  
[-help]  
[-setup]  
[-hold]  
[-rise]  
[-fall]  
[-high | -low]  
[obj_list]
```

Resets the specified setup and hold values for clock gating checks to the default values.

Parameters

-help	Prints out the command usage.
-high -low	Resets the non-controlling value (high or low) of the clock.
-hold	Resets clock gating hold values.
-rise -fall	Resets the specified setup (or hold) value to the rising (or falling) signals on the specified pins.
-setup	Resets clock gating setup values.
obj_list	Specifies a list of clocks, instances, or pins for which clock gating checks are to be reset to default values. If the <i>obj_list</i> parameter is not specified, the clock gating check is reset on the current design.

Related Commands

- [set_clock_gating_check](#)

reset_clock_groups

```
reset_clock_groups  
[-help]  
{-physically_exclusive | -logically_exclusive | -asynchronous}  
{-name name_list | -all}
```

Resets clock groups that have been set using the `set_clock_groups` command.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-all</code>	Removes all physically exclusive, logically exclusive, or asynchronous clock groups in the design. Removes all the physically exclusive, logically exclusive, or asynchronous clock groups in the design. This parameter should be used with one of the parameters: <code>-logically_exclusive</code>, <code>-physically_exclusive</code>, or <code>-asynchronous</code> at one time, and should not be used with the <code>-name</code> parameter.
<code>-asynchronous</code>	Specifies the asynchronous clock groups to be removed. This parameter must be used with either <code>-all</code> or <code>-name</code> parameter.
<code>-logically_exclusive</code>	Specifies the logically exclusive clock groups to be removed. This parameter must be used with either <code>-all</code> or <code>-name</code> parameter.
<code>-name <i>name_list</i></code>	Specifies the previously defined names of clock groups to be removed. This parameter should be used with one of the parameters: <code>-logically_exclusive</code>, <code>-physically_exclusive</code>, or <code>-asynchronous</code> at one time.
<code>-physically_exclusive</code>	Specifies all the physically exclusive clock groups to be removed. This parameter must be used with either <code>-all</code> or <code>-name</code> parameter.

Examples

- The following command resets all physically exclusive clock groups in the design:

```
reset_clock_groups -physically_exclusive -all
```
- The following command resets physically exclusive clock group `CLKGRP0`:

```
reset_clock_groups -physically_exclusive -name CLKGRP0
```

Related Commands

- `set_clock_groups`

reset_clock_latency

```
reset_clock_latency  
[-help]  
[-source]  
[-clock clock_list]  
obj_list
```

Resets assertions made by previous `set_clock_latency` commands.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-clock</code> <i>clock_list</i>	Specifies the list of clocks with which associated <code>set_clock_latency</code> constraints, at the pin level, are to be removed. Note: This parameter cannot be used with clocks specified using the <i>obj_list</i> parameter.
<i>obj_list</i>	Specifies a list of pins, ports, or clock objects for which the assertions are to be removed.
<code>-source</code>	Removes the source latency.

Examples

- The following commands show the use of the `reset_clock_latency` command to remove a leaf-level `set_clock_latency` assertion, after the clock tree has been put into propagated mode:

```
set_clock_latency 3.141 [get_pins BLK/BR2/CK]  
report_timing -to BLK/BR2/D -path_type full_clock
```

The following report is generated:

```
Path 1: MET Setup Check with Pin BLK/BR2/CK  
Endpoint: BLK/BR2/D (v) checked with leading edge of 'WAVE'  
Beginpoint: BLK/BR1/Q (v) triggered by leading edge of 'WAVE'  
Other End Arrival Time 3.141 <---  
- Setup 0.191  
+ Phase Shift 10.000  
= Required Time 12.950  
- Arrival Time 0.249  
= Slack Time 12.701  
...
```

```
set_propagated_clock [get_ports tclk]
report_timing -path_type full_clock -net -to BLK/BR2/D
```

The following report is generated:

```
Path 1: MET Setup Check with Pin BLK/BR2/CK
Endpoint: BLK/BR2/D (v) checked with leading edge of 'WAVE'
Beginpoint: BLK/BR1/Q (v) triggered by leading edge of 'WAVE'
Other End Arrival Time 3.141 <-----
- Setup 0.191
+ Phase Shift 10.000
= Required Time 12.950
- Arrival Time 0.594
= Slack Time 12.356
...

```

In this example, setting the root of the clock tree to propagated mode is not sufficient to override the `set_clock_latency` assertion at the clock endpoint. You must use the `reset_clock_latency` command to remove the assertion, so that the register sees propagated clock timing.

```
reset_clock_latency [get_pins BLK/BR2/CK]
report_timing -path_type full_clock -net -to BLK/BR2/D
```

The following report is generated:

```
Path 1: MET Setup Check with Pin BLK/BR2/CK
Endpoint: BLK/BR2/D (v) checked with leading edge of 'WAVE'
Beginpoint: BLK/BR1/Q (v) triggered by leading edge of 'WAVE'
Other End Arrival Time 0.345 <-----
- Setup 0.186
+ Phase Shift 10.000
= Required Time 10.159
- Arrival Time 0.588
= Slack Time 9.571

```

- The following commands show that `reset_clock_latency` is only effective when it is applied on the same object for which you had set the assertion using the `set_clock_latency` command:

Latency here is specified on a pin object:

```
set_clock_latency 3.141 [get_pins BLK/BR2/CK]
report_timing -path_type full -net -to BLK/BR2/D
```

The following report is generated:

```
Path 1: MET Setup Check with Pin BLK/BR2/CK
Endpoint: BLK/BR2/D (v) checked with leading edge of 'WAVE'
Beginpoint: BLK/BR1/Q (v) triggered by leading edge of 'WAVE'
Other End Arrival Time 3.141 <-
- Setup 0.191
+ Phase Shift 10.000

```

```
= Required Time 12.950
- Arrival Time 0.249
= Slack Time 12.701
```

Resetting of the latency is done on a clock object and has no effect on downstream latencies specified on pin objects.

In this example, a `reset_clock_latency` on the clock waveform that is driving BLK/BR2/CK will have no effect.

```
reset_clock_latency [get_clocks WAVE]
report_timing -path_type full -net -to BLK/BR2/D
```

The following report is generated:

```
Path 1: MET Setup Check with Pin BLK/BR2/CK
Endpoint: BLK/BR2/D (v) checked with leading edge of 'WAVE'
Beginpoint: BLK/BR1/Q (v) triggered by leading edge of 'WAVE'
Other End Arrival Time 3.141 <-
- Setup 0.191
+ Phase Shift 10.000
= Required Time 12.950
- Arrival Time 0.249
= Slack Time 12.701
```

Related Commands

- [set_clock_latency](#)

reset_clock_sense

```
reset_clock_sense  
[-help]  
[-all]  
| [[-clocks clock_list] pin_or_port_list]]
```

Resets clock sense constraint(s) specified using the `set_clock_sense` command.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-all</code>	Resets all the specified <code>clock_sense</code> constraints.
<code>-clocks</code> <i>clock_list</i>	Specifies the list of clocks for which the clock sense constraints are to be removed. The clocks must have previously been defined using the <code>set_clock_sense -clock</code> parameter.
<i>pin_or_port_list</i>	Specifies the list or collection of cell output pins for which the clock sense constraints will be removed.

Examples

The following command resets all the previously defined `clock_sense` constraints from a design:

```
% reset_clock_sense -all
```

Related Commands

- [set_clock_sense](#)

reset_clock_transition

```
reset_clock_transition
[-help]
clock_list
```

Removes the clock transition assertions you had set on the specified list of clock waveforms.

Parameters

<code>-help</code>	Prints out the command usage.
<code>clock_list</code>	Specifies a list of clock waveforms for which the clock transition assertions are removed. The <code>clock_list</code> variable can be a Tcl list or collection of clock waveforms.

Examples

- The following commands set the clock transition assertions, and resets them and displays the results using the `report_timing` command:

```
set_clock_transition 0.123 [get_clocks WAVE]
report_timing -net -to BLK/BR2/D
```

Generates the following report

```
Path 1: MET Setup Check with Pin BLK/BR2/CK
Endpoint: BLK/BR2/D (v) checked with leading edge of 'WAVE'
Beginpoint: BLK/BR1/Q (v) triggered by leading edge of 'WAVE'
Other End Arrival Time 0.000
- Setup 0.192
+ Phase Shift 10.000
= Required Time 9.808
- Arrival Time 0.249
= Slack Time 9.559
Clock Rise Edge 0.000
= Beginpoint Arrival Time 0.000
+-----+
| Instance | Cell   | Arc      | Pin   | Slew  | Delay | Arrival |
|          |        |          |       |       |       | Time    |
+-----+-----+-----+-----+-----+-----+
|          |        | tclk ^   | tclk  | 0.123 |       | 0.000 |
| CG/BC1   | BUFX2  |          | A     | 0.123 | 0.000 | 0.000 |
```

Litmus Text Command Reference

Timing Constraint Commands--reset_clock_transition

```
| CG/BC1      | BUFX2  | A ^ -> Y ^ | Y      | 0.123 | 0.000 | 0.000 |
| TC1        | BUFX2  |             | A      | 0.123 | 0.000 | 0.000 |
| TC1        | BUFX2  | A ^ -> Y ^ | Y      | 0.123 | 0.000 | 0.000 |
| CG1        | AND2X4 |             | B      | 0.123 | 0.000 | 0.000 |
| CG1        | AND2X4 | B ^ -> Y ^ | Y      | 0.123 | 0.000 | 0.000 |
| BLK/blkint | BUFX2  |             | A      | 0.123 | 0.000 | 0.000 |
| BLK/blkint | BUFX2  | A ^ -> Y ^ | Y      | 0.123 | 0.000 | 0.000 |
| BLK/BR1    | DFFHQX1|             | CK     | 0.123 | 0.000 | 0.000 |
```

reset_clock_transition [get_clocks WAVE]

report_timing -net -to BLK/BR2/D

Generates the following report:

Path 1: MET Setup Check with Pin BLK/BR2/CK

Endpoint: BLK/BR2/D (v) checked with leading edge of 'WAVE'

Beginpoint: BLK/BR1/Q (v) triggered by leading edge of 'WAVE'

Other End Arrival Time 0.000

- Setup 0.191

+ Phase Shift 10.000

= Required Time 9.809

- Arrival Time 0.249

= Slack Time 9.560

Clock Rise Edge 0.000

= Beginpoint Arrival Time 0.000

```
+-----+
| Instance | Cell      | Arc          | Pin   | Slew  | Delay | Arrival |
|          |           |              |       |       |       | Time    |
+-----+-----+-----+-----+-----+-----+
|          |           | tclk ^      | tclk  | 0.000 |       | 0.000 |
| CG/BC1   | BUFX2    |             | A     | 0.000 | 0.000 | 0.000 |
| CG/BC1   | BUFX2    | A ^ -> Y ^   | Y     | 0.000 | 0.000 | 0.000 |
| TC1      | BUFX2    |             | A     | 0.000 | 0.000 | 0.000 |
| TC1      | BUFX2    | A ^ -> Y ^   | Y     | 0.000 | 0.000 | 0.000 |
| CG1      | AND2X4   |             | B     | 0.000 | 0.000 | 0.000 |
| CG1      | AND2X4   | B ^ -> Y ^   | Y     | 0.000 | 0.000 | 0.000 |
| BLK/blkint | BUFX2    |             | A     | 0.000 | 0.000 | 0.000 |
| BLK/blkint | BUFX2    | A ^ -> Y ^   | Y     | 0.000 | 0.000 | 0.000 |
| BLK/BR1  | DFFHQX1  |             | CK    | 0.000 | 0.000 | 0.000 |
+-----+-----+-----+-----+-----+-----+-----+
```

Related Commands

- `set_clock_transition`

reset_clock_tree_latency

```
reset_clock_tree_latency  
[-help]  
[obj_list]
```

Resets network clock latency set using the `set_clock_latency` command in the fanout of the specified clock or pins and ports. The `reset_clock_tree_latency` command resets the latency until the root pins of clocks and generated clocks.

Use this command after building the clock tree and setting the clock network from the root to propagated mode using the `set_propagated_clock` command.

Note: You can use the `reset_clock_tree_latency` command only on command line and command script. You cannot specify it in an SDC file.

Parameters

<code>-help</code>	Prints out the command usage.
<code>obj_list</code>	Specifies list of objects - pins, ports, or clocks for which latency will be reset.

Related Commands

- `set_clock_latency`

reset_clock_uncertainty

```
reset_clock_uncertainty
[-help]
[-setup]
[-hold]
[-half_cycle_jitter | -full_cycle_jitter]
{ {-from | -rise_from | -fall_from} clksig_from_list
  {-to | -rise_to | -fall_to} clksig_to_list
  | pin_or_clock_list
}
```

Removes the assertions that were made by previous `set_clock_uncertainty` commands.

Parameters

<code>-help</code>	Prints out the command usage.
<code>{-from -rise_from -fall_from}</code> <code>clksig_from_list</code>	Removes the assertions for uncertainty value for the specified list of launching clocks. Note: You must specify one of these parameters with one of the <code>-to</code> , <code>-rise_to</code> , or <code>-fall_to</code> parameters. You cannot use these parameters when pins are specified.
	<code>-fall_from</code>: Removes the uncertainty value only from the falling edge of the launching clocks.
	<code>-from</code>: Removes the uncertainty value from both the rising and falling edges of the launching clocks.
	<code>-rise_from</code>: Removes the uncertainty value only from the rising edge of the launching clocks.
<code>-half_cycle_jitter</code> <code> -full_cycle_jitter</code>	Removes or resets the clock source jitter assertion(s) of the specified type.
<code>pin_or_clock_list</code>	Removes the target uncertainty assertions from the specified list of clocks or pins. The uncertainty value is removed from the registers clocked by the target waveform, or register pins in the fanout of the specified pin. You can specify a list of clock names, or clock root pin or port names relative to the current module. You also can specify a collection of clocks, or clock root pins or ports. Note: You cannot use this parameter when any of the <code>-from</code> and <code>-to</code> parameter pairs are specified.

<code>-setup/-hold</code>	Specifies whether the clock uncertainty is removed for setup or hold checks. <i>Default:</i> Removes clock uncertainty for both setup and hold check.
<code>{-to -rise_to -fall_to}</code> <code>clkSIG_to_list</code>	Removes the uncertainty value from the specified list of capture clocks. Note: You must specify one of these parameters with one of the <code>-from</code> , <code>-rise_from</code> , or <code>-fall_from</code> parameters. You cannot use these parameters when pins are specified.
	<code>-fall_to</code>: Removes the uncertainty value only from the falling edge of the capture clocks.
	<code>-rise_to</code>: Removes the uncertainty value only from the rising edge of the capture clocks.
	<code>-to</code>: Removes the uncertainty value from both the rising and falling edges of the capture clocks.

Examples

The following commands sets and resets the clock uncertainty assertions and displays the results using the `report_timing` command:

- ```
set_clock_uncertainty 0.345 -from [get_clocks VCLK] -to [get_clocks WAVE]report_timing -net -from tin
Generates the following report:
Path 1: MET Setup Check with Pin TR1/BR1/CK
Endpoint: TR1/BR1/D (v) checked with leading edge of 'WAVE'
Beginpoint: tin (v) triggered by trailing edge of 'VCLK'
Other End Arrival Time 0.000
- Setup 0.192
+ Phase Shift 10.000
- Uncertainty 0.345 <---
= Required Time 9.463
- Arrival Time 5.877
= Slack Time 3.586
```
- ```
reset_clock_uncertainty -from [get_clocks VCLK] -to [get_clocks WAVE]report_timing -net -from tin
# Generates the following report:
Path 1: MET Setup Check with Pin TR1/BR1/CK
Endpoint: TR1/BR1/D (v) checked with leading edge of 'WAVE'
Beginpoint: tin (v) triggered by trailing edge of 'VCLK'
Other End Arrival Time 0.000
```

```
- Setup 0.192
+ Phase Shift 10.000
= Required Time 9.808
- Arrival Time 5.877
= Slack Time 3.931
```

Related Commands

- [set_clock_uncertainty](#)

reset_data_check

```
reset_data_check  
[-help]  
{-from | -rise_from | -fall_from} pin_or_port_list  
{-to | -rise_to | -fall_to} pin_or_port_list  
[-setup]  
[-hold]  
[-clock clock_object]
```

Removes specified data-to-data checks that you have specified using the [set_data_check](#) command.

Parameters

-help	Prints out the command usage.
-clock <i>clock_object</i>	Removes the data check with a related (-from) pin event that is triggered by the specified clock. Use this parameter only if the original set_data_check constraint also used the -clock parameter to specify an explicit clock relationship.
-from -rise_from -fall_from <i>pin_or_port_list</i>	Specifies the related pins of the data-to-data checks to be removed. The <i>pin_or_port_list</i> list can contain either object IDs or hierarchical names relative to the current module. Leaf design cells (instances) can also be specified. Use only one -from, -rise_from, or -fall_from parameter per command. By default, the -from parameter removes checks for both the rising and the falling edges. Using -rise_from parameter removes checks at the rising edge of the signal on the from pins. Using -fall_from parameter removes checks at the falling edge of the signal on the from pins.
-setup/-hold	Specifies whether the data check to be removed is for setup or hold check. If you do not specify either -setup or -hold option, both setup and hold checks are removed.


```
-to | -rise_to | -  
fall_to  
pin_or_port_list
```

Specifies the constrained pins of data-to-data checks to be removed.

The *pin_or_port_list* list can contain either object IDs or hierarchical names relative to the current module. Leaf design cells (instances) can also be specified.

Use only one *-to*, *-rise_to*, or *-fall_to* parameter per command.

By default, the *-to* parameters removes both the rising and the falling edges.

Using *-rise_to* parameter removes the rising edge of the signal on the to pins.

Using *-fall_to* parameter removes the falling edge of the signal on the to pins.

Examples

- The following commands set and reset the data checks from *port30* to *port28*:

```
set_data_check 0.345 -rise_from port30 -to port28  
report_timing -rise_from port30 -to port28
```

The following report is displayed:

```
Path 1: VIOLATED Data To Data Setup Check with Pin port30  
Endpoint: port28 (v) checked with leading edge of 'PH1'  
Beginpoint: U2/Q (v) triggered by leading edge of 'PH1'  
Other End Arrival Time 0.465  
- Data Check Setup 0.345  
+ Phase Shift 0.000  
= Required Time 0.120  
- Arrival Time 0.309  
= Slack Time -0.189
```

```
reset_data_check -rise_from port30 -to port28  
report_timing -rise_from port30 -to port28
```

The following information is displayed:

```
No constrained timing paths with given description found.  
Paths may be unconstrained (try '-unconstrained' option) or may not exist.
```

Related Commands

- [set_data_check](#)

reset_disable_clock_gating_check

```
reset_disable_clock_gating_check  
[-help]  
pin_or_inst_list
```

Enables clock gating checks for the specified instances or pins.

Parameters

<code>-help</code>	Prints out the command usage.
<code>pin_or_inst_list</code>	Specifies the list of disabled pins or instances that are to be enabled.

Related Commands

- `set_disable_clock_gating_check`

reset_disable_timing

```
reset_disable_timing
[-help]
[-from pin_name -to pin_name]
obj_list
```

Restores timing arcs that were disabled using the `set_disable_timing` command.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-from pin_name -to pin_name</code>	Specifies the arc(s) that are to be restored. Specify the <code>-from</code> and <code>-to</code> options together. You can specify the <code>-from</code> and <code>-to</code> parameters when the <code>obj_list</code> contains instance or library cell objects. Do not specify the <code>-from</code> and <code>-to</code> parameters when the <code>obj_list</code> includes instance pin or library cell pin objects. The <code>-from</code> and <code>-to</code> options are optional. If not specified, all cell arcs found in the instance or library cell objects are restored.
<code>obj_list</code>	Specifies the instance, instance pin, library arc, library cell, or library cell pin objects. If the <code>obj_list</code> includes instance pin or library cell pin objects, all cell arcs attached to the specified pin objects are restored. If the <code>obj_list</code> contains instance or library cell objects, and the <code>-from</code> and <code>-to</code> parameters are not specified, all arcs found in the specified instance or library cell objects are restored.

Examples

- The following commands sets and resets the timing arcs and displays the resulting reports:

```
% set_disable_timing -from A -to Y[get_cells b1]
% report_inactive_arcs
```

```
-----
Flags :                const                propagated constant
                snipped                loop snipped arcs
                disable                set_disable_timing
                disable_clock_gating    set_disable_clock_gating
                library_disable        set_disable_cell_timing
                Missing_Phase          No Arc Phase Data
From      To      DisableType  ArcType      Reason
-----
b1/A ^    b1/Y ^    disable      combinational  User Disable
b1/A v    b1/Y v    disable      combinational  User Disable
```

- The following commands disable the timing report header:

```
% set_global timing_disable_report_header_info true
% report_inactive_arcs
```

```
-----
Flags :                const                propagated constant
                snipped                loop snipped arcs
                disable                set_disable_timing
                disable_clock_gating    set_disable_clock_gating
                library_disable        set_disable_cell_timing
                Missing_Phase          No Arc Phase Data

From      To      DisableType  ArcType      Reason
-----
u8/CK ^    u8/D ^    const        hold_rising_clk_rise  u8/D = 1
u8/CK ^    u8/D v    const        hold_falling_clk_rise u8/D = 1
u8/CK ^    u8/D ^    const        setup_rising_clk_rise  u8/D = 1
u8/CK ^    u8/D v    const        setup_falling_clk_rise u8/D = 1
-----
```

- The following commands disable the timing arcs from CK to Q for instance U8, then reset the assertions using the instance reference:

```
% set_disable_timing -from CK -to Q [get_cells u8]
% report_inactive_arcs
```

```
-----
Flags :                const                propagated constant
                snipped                loop snipped arcs
                disable                set_disable_timing
                disable_clock_gating    set_disable_clock_gating
                library_disable        set_disable_cell_timing
                Missing_Phase          No Arc Phase Data

From      To      DisableType  ArcType      Reason
-----
u8/CK ^    u8/Q ^    disable      rising_edge    User Disable
u8/CK ^    u8/Q v    disable      rising_edge    User Disable
-----
```

- The following commands disable the timing arcs from CK to Q for instance U8, then reset the assertions using the instance reference:

```
% reset_disable_timing u8
% report_inactive_arcs
```

```
-----
Flags :                const                propagated constant
                snipped                loop snipped arcs
                disable                set_disable_timing
```

Litmus Text Command Reference
Timing Constraint Commands--reset_disable_timing

			disable_clock_gating	set_disable_clock_gating	
			library_disable	set_disable_cell_timing	
			Missing_Phase	No Arc Phase Data	
From	To	DisableType	ArcType		Reason

u8/CK ^	u8/D ^	const	hold_rising_clk_rise		u8/D = 1
u8/CK ^	u8/D v	const	hold_falling_clk_rise		u8/D = 1
u8/CK ^	u8/D ^	const	setup_rising_clk_rise		u8/D = 1
u8/CK ^	u8/D v	const	setup_falling_clk_rise		u8/D = 1

Related Commands

- [set_disable_timing](#)
- [report_inactive_arcs](#)

reset_drive

```
reset_drive
[-help]
port_list
```

Resets the drive previously asserted with the `set_drive` command on input ports. The `reset_drive` command removes assertions for all rise/fall, minimum/maximum combinations for which the assertion exists.

Parameters

<code>-help</code>	Prints out the command usage.
<code>port_list</code>	Specifies a list of input or bidirectional ports for which the drive resistance is reset.

Related Commands

- `set_drive`

reset_driving_cell

```
reset_driving_cell  
[-help]  
[-rise]  
[-fall]  
[-min]  
[-max]  
port_list
```

Resets the driving cell assertions applied with the `set_driving_cell` command for the given rise/fall, minimum/maximum combination on ports.

If `-rise`, `-fall`, `-min`, or `-max` parameters are not specified, the `reset_driving_cell` command resets the assertion for all rise/fall, minimum/maximum combinations for which the assertion exists.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-rise</code>	Specifies the rising drive assertions.
<code>-fall</code>	Specifies the falling drive assertions.
<code>-min</code>	Specifies the early drive assertion.
<code>-max</code>	Specifies the late drive assertion.
<i>port_list</i>	Specifies a list of ports for which the driving cell assertions are reset.

Related Commands

- `set_driving_cell`

reset_generated_clock

```
reset_generated_clock  
[-help]  
{-all | clock_list }
```

Removes generated clock assertions previously created using the `create_generated_clock` command.

To use the `reset_generated_clock` command you must specify the `set_interactive_constraint_modes` constraints for the applicable modes. You can also include these commands in the SDC constraint file - applicable only for the views that include that SDC.

 The generated clocks may originate from the library files dotlib - you can use `reset_generated_clock` from the command line or add it to all the SDC files in use.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-all</code>	Removes all the generated clocks in the design.
<code>clock_list</code>	Specifies the list of clocks (in collection format) for which the generated clock assertions are to be removed.

Examples

- The following command removes the generated clock assertions on all the generated clocks `RC/Q`:

```
% report_clocks
```

The following report is displayed:

```
-----  
                                Clock Descriptions  
-----  
                                Attributes  
-----  
Clock Name   Source    Period   Lead    Trail    Generated   Propagated  
-----  
CK1          CLK1      20.000   0.000   10.000    n           n  
CK2          CLK1      10.000   0.000   5.000     n           n  
GCLK         u8/Q      20.000   0.000   10.000    y           y  
-----  
  
Generated-Clock Descriptions  
-----
```


Litmus Text Command Reference

Timing Constraint Commands--reset_generated_clock

Name	Generated Source (pin)	Master Source (pin)	Master-clock	Invert	Freq. Multiplier	Duty-Cycle	Edges	Edge-Shift
------	---------------------------	------------------------	--------------	--------	---------------------	------------	-------	------------

GCLK	u8/Q	CK1	CLK1	n	1/2	-	-	-
------	------	-----	------	---	-----	---	---	---

```
% set_interactive_constraint_modes [all_constraint_modes -active]
% reset_generated_clock [get_clocks GCLK]
% report_clocks
```

The following report is displayed:

Clock Descriptions

Attributes

Clock Name	Source	Period	Lead	Trail	Generated	Propagated
CK1	CLK1	20.000	0.000	10.000	n	n
CK2	CLK1	10.000	0.000	5.000	n	n
GCLK	u8/Q	20.000	0.000	10.000	y	y

Generated-Clock Descriptions

Name	Generated Source (pin)	Master Source (pin)	Master-clock	Invert	Freq. Multiplier	Duty-Cycle	Edges	Edge-Shift
------	---------------------------	------------------------	--------------	--------	---------------------	------------	-------	------------

Related Commands

- [create_generated_clock](#)
- [report_clocks](#)
- [set_interactive_constraint_modes](#)

reset_ideal_latency

```
reset_ideal_latency  
[-help]  
[-min]  
[-max]  
[-rise]  
[-fall]  
pin_or_port_list
```

Resets the ideal latency constraints, specified using the `set_ideal_latency` command, on instance pins or ports.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-fall</code>	Resets the ideal latency for the falling edge.
<code>-max</code>	Resets the ideal latency for the maximum corner.
<code>-min</code>	Resets the ideal latency for the minimum corner.
<i>pin_or_port_list</i>	Resets a list of objects - instance pins, ports, or nets.
<code>-rise</code>	Resets the ideal latency for the rising edge.

Related Commands

- `set_ideal_latency`

reset_ideal_network

```
reset_ideal_network  
[-help]  
obj_list
```

Resets the ideal network constraints, specified using the `set_ideal_network` command, on pins, ports, or nets.

Note: The `reset_ideal_network` command applies to all the active views and is not view/mode-based.

Parameters

<code>-help</code>	Prints out the command usage.
<i>obj_list</i>	Resets a list of objects - instance pins, ports, or nets.

Related Commands

- `set_ideal_network`

reset_ideal_transition

```
reset_ideal_transition  
[-help]  
[-min]  
[-max]  
[-rise]  
[-fall]  
pin_or_port_list
```

Resets the ideal transition constraint, specified using the `set_ideal_transition` command, on instance pins or ports.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-fall</code>	Resets the ideal transition for the falling edge.
<code>-max</code>	Resets the ideal transition for the maximum corner.
<code>-min</code>	Resets the ideal transition for the minimum corner.
<i>pin_or_port_list</i>	Resets a list of objects - instance pins, ports, or nets.
<code>-rise</code>	Resets the ideal transition for the rising edge.

Related Commands

- `set_ideal_transition`

reset_input_delay

```
reset_input_delay  
[-help]  
[-clock clk_name]  
[-clock_fall]  
[-rise] [-fall]  
[-max] [-min]  
[-level_sensitive]  
pin_or_port_list
```

Resets previously specified input delay assertions.

Parameters

-help	Prints out the command usage.
-clock <i>clock_name</i>	Specifies the name of the clock. <i>Default:</i> The asynchronous (@) clock. If you specify multiple clocks, an error occurs and the constraint is ignored. When you specify the -clock parameter, the constraints specified for a pin with matching -clock parameter are reset. If you do not specify the -clock parameter, all the constraints applied on the specified pin, with or without the -clock parameter specification, are reset.
-clock_fall	Removes the assertions for the delay that is relative to the falling edge of the clock. You must specify the -clock parameter with this parameter. Default: Removes the delay assertions relative to the falling edge.
-fall	Resets the input delay for the falling edge at the input port. If both the -rise and -fall options are not specified, then the input delay is reset on both the edges.
-level_sensitive	Resets input delay that is launched by a positive level sensitive latch. When you specify the -level_sensitive parameter, the command resets all the set_input_delay constraints specified with the -level_sensitive parameter. If -level_sensitive parameter is not specified, the constraints specified for the same pin, with or without the -level_sensitive parameter, are reset. Note: You must specify the -clock parameter with this parameter.
-max	Removes the delay assertions that refer to the setup analysis. If you do not specify the -max parameter, the input delay assertions for both setup and hold analysis are removed.

<code>-min</code>	Removes the delay assertions that refer to the hold analysis. If you do not specify the <code>-min</code> parameter, the input delay assertions for both setup and hold analysis are removed.
<code>pin_or_port_list</code>	Specifies a single or multiple pins or ports for which the input delay is reset. To specify multiple pins, enclose the list with curly braces (<code>{}</code>) and separate the pin information with white space.
<code>-rise</code>	Resets the input delay for the rising edge at the input port. If both <code>-rise</code> and <code>-fall</code> options are omitted, the input delay is reset on both the edges.

Related Commands

- `set_input_delay`

reset_load

```
reset_load  
[-help]  
port_net_list
```

Resets the existing `set_load` assertions specified on nets or ports.

Parameters

<code>-help</code>	Prints out the command usage.
<code>port_net_list</code>	Specifies a list of nets or ports in the top cell for which assertions are reset.

Related Commands

- `set_load`

reset_max_capacitance

```
reset_max_capacitance  
[-help]  
obj_list
```

Resets maximum capacitance limit specified (using the the `set_max_capacitance` command) on the objects in the object list. The valid objects in the object list are primary ports, clocks, and current design.

Parameters

<code>-help</code>	Prints out the command usage.
<code><i>obj_list</i></code>	Specifies the list of ports, modules, and/or clock waveforms for which assertions are reset.

Related Commands

- [set_max_capacitance](#)

reset_max_fanout

```
reset_max_fanout  
[-help]  
obj_list
```

Resets maximum fanout limit specified (using the `set_max_fanout` command) on the objects in the object list. The valid objects in the object list are input and bidirectional ports and current design.

Parameters

<code>-help</code>	Prints out the command usage.
<i>obj_list</i>	Specifies the list of ports or modules for which assertions are reset.

Related Commands

- `set_max_fanout`

reset_max_time_borrow

```
reset_max_time_borrow  
[-help]  
obj_list
```

Resets the specified maximum time borrow limit of the specified objects.

Parameters

<code>-help</code>	Prints out the command usage.
<code><i>obj_list</i></code>	Specifies the list of pins, instances, or clocks for which the time borrow limit is reset.

Related Commands

- `set_max_time_borrow`

reset_max_transition

```
reset_max_transition  
[-help]  
obj_list
```

Resets the maximum slew (transition) limit specified using the `set_max_transition` command (from the SDC or the Litmus command line) on the objects in the given object list. The valid objects in the object list are primary ports, clocks, and current design.

The maximum transition limit that is set by the user overrides the dotlib value so that sharper slews are implemented on the timing sensitive nets, such as clocks. A violation due to the `set_max_transition` setting does not impact the timing accuracy.

 For command-line usage of this command, you need to use `set_interactive_constraint_modes`. For example
`- set_interactive_constraint_modes [all_constraint_modes -active]`

Parameters

<code>-help</code>	Prints out the command usage.
<code>obj_list</code>	Specifies the list (or collection) of ports, modules, and/or clock waveforms for which assertions are to be reset. For example, <code>reset_max_transition [get_ports *]</code>

Related Commands

- `set_max_transition`

reset_min_capacitance

```
reset_min_capacitance  
[-help]  
obj_list
```

Resets minimum capacitance limit specified (using the the `set_min_capacitance` command) on objects in the object list. The valid objects in the object list are primary ports, clocks, and the current design.

Parameters

<code>-help</code>	Prints out the command usage.
<code><i>obj_list</i></code>	Specifies the list of ports, modules, and/or clock waveforms for which assertions are reset.

Related Commands

- `set_min_capacitance`

reset_min_fanout

```
reset_min_fanout  
[-help]  
obj_list
```

Resets minimum fanout limit specified (using the `set_min_fanout` command) on the objects in the object list. The valid objects in the object list are the inputs, bi-directional ports, and the current design.

Parameters

<code>-help</code>	Prints out the command usage.
<code>obj_list</code>	Specifies the list of ports or modules for which assertions are reset.

Related Commands

- `set_min_fanout`

reset_min_pulse_width

```
reset_min_pulse_width  
[-help]  
obj_list  
[-high]  
[-low]  
[-waveform_aware_type {absolute | source_width_ratio | all}]
```

Resets minimum pulse width constraint on the specified input ports of the top cell and the specified modules.

Parameters

-help	Prints out the command usage.
-low	Resets the minimum pulse width checks that are applied to low signal levels only.
-high	Resets the minimum pulse width checks that are applied to high signal levels only.
obj_list	Specifies a list of pins or clock waveforms for which the minimum pulse width is to be reset. If the list is not specified, the minimum pulse width check applies to all the pins in the clock tree of the current design. If a clock waveform is specified, all the pins driven by the clock are affected.
- waveform_aware_type {absolute source_width_ratio all}	Resets the regular and waveform-aware pulse width assertions.

Related Commands

- [set_min_pulse_width](#)

reset_min_transition

```
reset_min_transition  
[-help]  
obj_list
```

Resets minimum slew (transition) limit specified (using the `set_min_transition` command) on the objects in the object list. The valid objects in the object list are primary ports, clocks, and current design.

Parameters

<code>-help</code>	Prints out the command usage.
<code>obj_list</code>	Specifies the list of ports, modules, and/or clock waveforms for which assertions are reset.

Related Commands

- `set_min_transition`

reset_mode

```
reset_mode  
[-help]  
[-type cell]  
mode_list  
inst_list
```

Resets the Liberty timing library modes for a specified instance.

Library modes are set using the [set_mode](#) command. By default, all library modes in the timing library are active. If you specify one or more library modes using the [set_mode](#) command, only the timing arcs consistent with the specified mode(s) are active for the instance. All other library modes become inactive. When you reset the library mode for the instance, all library modes become active again.

Parameters

-help	Prints out the command usage.
inst_list	Specifies the instances to which to reset the library modes.
mode_list	Specifies the name of the library modes to reset.
-type cell	Resets the library mode at the instance level. Note: Setting the library mode at the instance level is the only form of set_mode that is currently supported. This parameter exists to support constraints coming from external sources.

Examples

- The following command specifies that only the timing arcs consistent with the `test_A` library mode are active for instances of `b1`:

The `test_A` library mode is made inactive.

```
% set_mode test_mode [get_cells BUF]
```

- The following command reports the status of all library modes associated with BUF 'b1'. It shows that `test_A` is active and `test_B` is inactive.

```
% report_mode b1
```

```
-----  
Mode Group test_mode of b1  
-----  
Mode      Status   Condition  
Name  
-----  
test_B    Inactive  !(Y)  
test_A    ACTIVE    Y  
-----
```

- The following command resets the library mode for instances of `b1` :

```
% reset_mode test_A b1
```

If you generate a report of the library mode status for `b1`, it shows that both modes are now active:

```
% report_mode b1
```

```
-----  
Mode Group test_mode of b1  
-----  
Mode      Status   Condition  
Name  
-----  
test_B    ACTIVE    !(Y)  
test_A    ACTIVE    Y  
-----
```

Related Commands

- [set_mode](#)
- [report_mode](#)

reset_output_delay

```
reset_output_delay  
[-help]  
port_or_pin_list  
[-clock clock_name]  
[-clock_fall]  
[-fall]  
[-level_sensitive]  
[-max]  
[-min]  
[-rise]
```

Resets previously specified output delay assertions.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-clock</code> <i>clock_name</i>	Specifies the name of the clock. <i>Default:</i> The asynchronous (@) clock. If you specify multiple clocks, an error occurs and the constraint is ignored. When you specify the <code>-clock</code> parameter, the constraints specified for a pin with matching <code>-clock</code> parameter are reset. If you do not specify the <code>-clock</code> parameter, all the constraints applied on the specified pin, with or without the <code>-clock</code> parameter specification, are reset.
<code>-clock_fall</code>	Removes the assertions for the delay that is relative to the falling edge of the clock. You must specify the <code>-clock</code> parameter with this parameter. <i>Default:</i> Removes the delay assertions relative to the rising edge.
<code>-fall</code>	Resets the output delay for the falling edge at the output port. If both <code>-rise</code> and <code>-fall</code> options are omitted, the output delay is reset on both the edges.
<code>-level_sensitive</code>	Resets output delay that is launched by a positive level sensitive latch. When you specify the <code>-level_sensitive</code> parameter, the command resets all the <code>set_output_delay</code> constraints specified with the <code>-level_sensitive</code> parameter. If <code>-level_sensitive</code> parameter is not specified, the constraints specified for the same pin, with or without the <code>-level_sensitive</code> parameter, are reset. Note: You must specify the <code>-clock</code> parameter with this parameter.
<code>-max</code>	Removes the delay assertions that refer to the setup analysis. If you do not specify the <code>-max</code> parameter, the output delay assertions for both setup and hold analysis are removed.
<code>-min</code>	Removes the delay assertions that refer to the hold analysis. If you do not specify the <code>-min</code> parameter, the output delay assertions for both setup and hold analysis are removed.
<i>pin_or_port_list</i>	Specifies a single or multiple pins or ports for which the output delay is reset. To specify multiple pins, enclose the list with curly braces ({}) and separate the pin information with white space.
<code>-rise</code>	Resets the output delay for the rising edge at the output port. If both <code>-rise</code> and <code>-fall</code> options are omitted, the output delay is reset on both the edges.

Examples

- The following example sets the output delays for the output port Z. The output signal is required at Z at 2 units before the rising edge of clock Clk1, and at 2.5 units before falling edge of clock Clk2:

```
% set_output_delay 2 Z -clock Clk1
% set_output_delay 2.5 Z -clock Clk2 -clock_fall -add_delay
```

- The following command will reset the output delay for the rising edge of clock Clk2:

```
% reset_output_delay Z -clock Clk2
```

```
report_timing -to_rise Z -clock_to Clk2
#####
# Generated by: Cadence Litmus 18.11-s094_1
# OS: Linux x86_64(Host ID noi-rakeshp)
# Generated on: Thu Apr 18 09:50:52 2019
# Design: top
# Command: report_timing -to_rise Z -clock_to Clk2
#####
No constrained timing paths with given description found.
Paths may be unconstrained (try '-unconstrained' option) or may not exist.

report_timing -to_fall Z -clock_to Clk2
#####
Path 1: VIOLATED Late External Delay Assertion
Endpoint: Z (v) checked with trailing edge of 'Clk2'
Beginpoint: dff3/Q (v) triggered by leading edge of 'Clk'
Path Groups: {vclk2}
Other End Arrival Time 2.000
- External Delay 2.500
+ Phase Shift 0.000
= Required Time -0.500
- Arrival Time 0.165
= Slack Time -0.665
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.052
= Beginpoint Arrival Time 0.052
-----
Instance Arc Cell Delay Arrival Required
Time Time
-----
dff3 CK ^ - - 0.052 -0.613
dff3 CK ^ -> Q v DFFHQX1 0.113 0.165 -0.500
- Z v - 0.000 0.165 -0.500
-----
```

- The following command resets the output delay for clock Clk2 with respect to the falling edge:

```
% reset_output_delay Z -clock Clk2 -clock_fall
```

- The following command will reset all the output delays with respect to all the edges for both the clocks - Clk1 and Clk2:

```
% reset_output_delay Z
```

Related Commands

- [set_output_delay](#)

reset_path_exception

```
reset_path_exception
[-help]
[-exact]
[-fall]
[-fall_through <through_list>]
[-ignore_clock_latency] [-rise]
[-rise_through <through_list>]
[-through <through_list>]
[-type <exception_type_list>]
[-all | [ [-from <from_list> | -rise_from <from_list> | -fall_from <from_list>] [-to <to_list> | -
rise_to <to_list> | -fall_to <to_list>] ] ]
[-setup | -hold ]
```

Removes any previously set path exceptions (or path exception of the specified type) for the given paths. To remove all path exceptions, use the `-all` option with the `reset_path_exception` command.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-all</code>	Removes all path exceptions. This option cannot be used with the <code>-from</code> , <code>-through</code> and <code>-to</code> options.
<code>-exact</code>	Forces exact matching of the constraints. When this parameter is specified, the differential list is not matched and the constraint is not reset.
<code>-fall</code>	Applies the assertion at the falling edge of the signal on the path endpoints.
<code>{-from -rise_from -fall_from} from_list</code>	Specifies pins at the start of the path exceptions that are to be reset.
<code>-rise</code>	Applies the assertion at the rising edge of the signal on the path endpoints.
<code>{-through -rise_through -fall_through} through_list</code>	Specifies the pins that the path to be reset goes through.
<code>{-to -rise_to -fall_to} to_list</code>	Specifies pins at the end of the path exceptions that are to be reset.
<code>-setup -hold</code>	Resets only the setup (or hold) paths. The <code>-setup/-hold</code> parameters should be used to reset maximum or minimum delay that is set using the <code>set_max_delay</code> and <code>set_min_delay</code> commands. <i>Default:</i> Both <code>-setup</code> and <code>-hold</code> are reset.
<code>-type {false_path multicycle path_delay}</code>	Removes only path exceptions of the specified type. <i>Default:</i> All path exception assertions are reset.
<code>-ignore_clock_latency</code>	Resets the exceptions specified with <code>-ignore_clock_latency</code> only (bool, optional)

Examples

- In the example below the rising path from R1 to R2 has three different path exceptions asserted on it - they are listed in priority from lowest to highest. As shown in the reports below, the false path is the highest precedence and so no path is reported.

```
% set_multicycle_path -from R1/CK -rise_to R2/D 2
% set_max_delay 3 -from R1/CK -rise_to R2/D
% set_false_path -from R1/CK -rise_to R2/D

% report_timing -from R1/CK -rise_to R2/D
```

The following information is displayed:

```
No constrained timing paths with given description found.
```

Paths may be unconstrained (try '-unconstrained' option) or may not exist.

```
% report_path_exceptions
```

The following report is displayed:

```
-----
From      To      Early    Late
-----
R1/CK     ^ R2/D    false    false
-----
```

To report ignored path exceptions, you can use the following command:

```
% report_path_exception -ignored
```

```
-----
From      To      Early    Late
-----
R1/CK     ^ R2/D                delay 3 ignored
R1/CK     ^ R2/D      add 0 ignored
R1/CK     ^ R2/D                add 1 ignored
-----
```

- In this example, the false path exception is removed from the path. This causes the max path delay assertion to become active - as it is the next highest precedence.

```
% reset_path_exception -from R1/CK -rise_to R2/D -type false_path
% report_path_exceptions
```

The following report is displayed:

```
-----
From      To      Early    Late
-----
R1/CK     ^ R2/D                delay 3
R1/CK     ^ R2/D      add 0
-----
```

```
% report_timing -from R1/CK -rise_to R2/D
```

The following report is displayed:

```
Path 1: MET Setup Check with Pin R2/CK
Endpoint: R2/D (^) checked with leading edge of 'CLK_W_1'
Beginpoint: R1/Q (^) triggered by leading edge of 'CLK_W_1'
Path Groups: {CLK_W_1}
Other End Arrival Time 0.000
- Setup 0.097
+ Path Delay 3.000
= Required Time 2.903
- Arrival Time 0.243
```



```
= Slack Time 2.660
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.000
= Beginpoint Arrival Time 0.000

-----
Load      Slew      Delay      Arrival      Cell      Arc              Pin
          Time
-----
0.026     0.003             0.000             CK ^          R1/CK ->
0.011     0.063     0.187     0.187         DFF         CK ^ -> Q ^    R1/Q
0.003     0.042     0.056     0.243         BUF         A ^ -> Y ^    B1/Y
0.003     0.042     0.000     0.243         DFF         D ^          R2/D ->
-----
```

- In this example, the `set_max_delay` exception is removed, leaving only the `set_multicycle_path` exception as active.

```
% reset_path_exception -from R1/CK -rise_to R2/D -type path_delay
% report_timing -from R1/CK -rise_to R2/D
```

The following report is displayed:

```
Path 1: MET Setup Check with Pin R2/CK
Endpoint: R2/D (^) checked with leading edge of 'CLK_W_1'
Beginpoint: R1/Q (^) triggered by leading edge of 'CLK_W_1'
Path Groups: {CLK_W_1}
Other End Arrival Time 0.000
- Setup 0.097
+ Phase Shift 20.000
+ Cycle Adjustment 20.000 <----- Multi-Cycle timing
= Required Time 39.903
- Arrival Time 0.243
= Slack Time 39.660
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.000
= Beginpoint Arrival Time 0.000

-----
Load      Slew      Delay      Arrival      Cell      Arc              Pin
          Time
-----
0.026     0.003             0.000             CK ^          R1/CK ->
0.011     0.063     0.187     0.187         DFF         CK ^ -> Q ^    R1/Q
0.003     0.042     0.056     0.243         BUF         A ^ -> Y ^    B1/Y
0.003     0.042     0.000     0.243         DFF         D ^          R2/D ->
-----
```

- In this example, the multi-cycle path assertion is removed, and the analysis reverts back to single-cycle timing.

```
% reset_path_exception -rise_to R2/D -type multicycle
% report_path_exceptions
```

None reported

```
% report_timing -from R1 -rise_to R2/D
```

The following report is displayed:

```
Path 1: MET Setup Check with Pin R2/CK
Endpoint: R2/D (^) checked with leading edge of 'CLK_W_1'
Beginpoint: R1/Q (^) triggered by leading edge of 'CLK_W_1'
Path Groups: {CLK_W_1}
Other End Arrival Time 0.000
- Setup 0.097
+ Phase Shift 20.000 <--- Single-cycle timing
= Required Time 19.903
- Arrival Time 0.243
= Slack Time 19.660
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.000
= Beginpoint Arrival Time 0.000
```

Load	Slew	Delay	Arrival Time	Cell	Arc	Pin
0.026	0.003	0.000			CK ^	R1/CK ->
0.011	0.063	0.187	0.187	DFF	CK ^ -> Q ^	R1/Q
0.003	0.042	0.056	0.243	BUF	A ^ -> Y ^	B1/Y
0.003	0.042	0.000	0.243	DFF	D ^	R2/D ->

- The example below shows how the `-hold` and `-setup` options can be used to control the effect of issuing `reset_path_exception`. The example will issue multi-cycle path constraints against both setup and hold modes, and then reset only the setup side.

```
% set_multicycle_path 3 -setup -rise_from [get_clocks PH1] -rise_through U2/A
% set_multicycle_path 2 -hold -rise_from [get_clocks PH1] -rise_through U2/A
% report_timing -late -from R1 -rise_to R2/D
```

The following report is displayed:

```
Path 1: MET Setup Check with Pin R2/CK
Endpoint: R2/D (^) checked with leading edge of 'CLK_W_1'
Beginpoint: R1/Q (^) triggered by leading edge of 'CLK_W_1'
Path Groups: {CLK_W_1}
Other End Arrival Time 0.000
- Setup 0.097
+ Phase Shift 20.000
+ Cycle Adjustment 40.000 <-- 3 cycles
= Required Time 59.903
- Arrival Time 0.243
= Slack Time 59.660
Clock Rise Edge 0.000
```

Litmus Text Command Reference

Timing Constraint Commands--reset_path_exception

```
+ Clock Network Latency (Prop) 0.000
= Beginpoint Arrival Time 0.000
```

Load	Slew	Delay	Arrival Time	Cell	Arc	Pin
0.026	0.003	0.000			CK ^	R1/CK ->
0.011	0.063	0.187	0.187	DFF	CK ^ -> Q ^	R1/Q
0.003	0.042	0.056	0.243	BUF	A ^ -> Y ^	B1/Y
0.003	0.042	0.000	0.243	DFF	D ^	R2/D ->

```
% report_timing -early -from R1 -rise_to R2/D
```

The following report is displayed:

```
Path 1: MET Hold Check with Pin R2/CK
Endpoint: R2/D (^) checked with leading edge of 'CLK_W_1'
Beginpoint: R1/Q (^) triggered by leading edge of 'CLK_W_1'
Path Groups: {CLK_W_1}
Other End Arrival Time 0.000
+ Hold -0.046
+ Phase Shift 0.000
- Cycle Adjustment 0.000 <- Adjusted 2 cycles
= Required Time -0.046
Arrival Time 0.243
Slack Time 0.289
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.000
= Beginpoint Arrival Time 0.000
```

Load	Slew	Delay	Arrival Time	Cell	Arc	Pin
0.026	0.003	0.000			CK ^	R1/CK ->
0.011	0.063	0.187	0.187	DFF	CK ^ -> Q ^	R1/Q
0.003	0.042	0.056	0.243	BUF	A ^ -> Y ^	B1/Y
0.003	0.042	0.000	0.243	DFF	D ^	R2/D ->

```
% reset_path_exception -setup -rise_from [get_clocks CLK_W_1] -rise_through B1/A
% report_timing -late -from R1 -rise_to R2/D
```

The following report is displayed:

```
Path 1: MET Setup Check with Pin R2/CK
Endpoint: R2/D (^) checked with leading edge of 'CLK_W_1'
Beginpoint: R1/Q (^) triggered by leading edge of 'CLK_W_1'
Path Groups: {CLK_W_1}
Other End Arrival Time 0.000
- Setup 0.097
```

Litmus Text Command Reference

Timing Constraint Commands--reset_path_exception

```
+ Phase Shift 20.000 <-- Reset to single-cycle timing
= Required Time 19.903
- Arrival Time 0.243
= Slack Time 19.660
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.000
= Beginpoint Arrival Time 0.000
```

Load	Slew	Delay	Arrival Time	Cell	Arc	Pin
0.026	0.003	0.000			CK ^	R1/CK ->
0.011	0.063	0.187	0.187	DFF	CK ^ -> Q ^	R1/Q
0.003	0.042	0.056	0.243	BUF	A ^ -> Y ^	B1/Y
0.003	0.042	0.000	0.243	DFF	D ^	R2/D ->

```
% report_timing -early -from R1 -rise_to R2/D
```

The following report is displayed:

```
Path 1: MET Hold Check with Pin R2/CK
Endpoint: R2/D (^) checked with leading edge of 'CLK_W_1'
Beginpoint: R1/Q (^) triggered by leading edge of 'CLK_W_1'
Path Groups: {CLK_W_1}
Other End Arrival Time 0.000
+ Hold -0.046
+ Phase Shift 0.000
- Cycle Adjustment 40.000 <-- Still 2 cycle adjustment
= Required Time -40.046 (from new setup edges)
Arrival Time 0.243
Slack Time 40.289
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.000
= Beginpoint Arrival Time 0.000
```

Load	Slew	Delay	Arrival Time	Cell	Arc	Pin
0.026	0.003	0.000			CK ^	R1/CK ->
0.011	0.063	0.187	0.187	DFF	CK ^ -> Q ^	R1/Q
0.003	0.042	0.056	0.243	BUF	A ^ -> Y ^	B1/Y
0.003	0.042	0.000	0.243	DFF	D ^	R2/D ->

- Consider the following `set_false_path` constraint:

```
% set_false_path -from {u1/Y u2/Y} -to {u3/Y}
```

When you specify `reset_path_exception -from {u1/Y}` command without the `-exact` parameter, the command outputs the following:

```
% set_false_path -from {u2/Y} -to {u3/Y}
```

The command `reset_path_exception -from {u1/Y} -exact` outputs the following:

```
% set_false_path -from {u1/Y u2/Y} -to {u3/Y}
```

In this case (when `-exact` parameter is used), the path exception is not reset because differential matching is not allowed.

Related Commands

- [set_false_path](#)

reset_path_group

```
reset_path_group  
[-help]  
[-name group_name]  
[-all]
```

Removes the specified path groups or completely removes all the groups.

Parameters

-help	Prints out the command usage.
-all	Removes all path groups.
-name <i>group_name</i>	Removes the named group completely.

Examples

- The following command removes the group named GRPA:

```
Litmus > reset_path_group -name GRPA
```

Related Commands

- [group_path](#)

reset_pll_timing

```
reset_pll_timing
[-help]
inst
-pll_output_clock clock name or ID
-pll_output_pin output pin name
```

Resets the phase locked loop (PLL) that is used to adjust the delay on the PLL output clock waveform.

Parameters

<code>-help</code>	Prints out the command usage.
<code>inst</code>	Specifies the instance on which we will search reference pin, feedback pin, or output pin.
<code>-pll_output_clock <i>clock name or ID</i></code>	Specifies the PLL output clock.
<code>-pll_output_pin <i>output pin name</i></code>	Specifies the PLL output pin.

Related Commands

- `set_pll_timing`

reset_propagated_clock

```
reset_propagated_clock
[-help]
pin_or_clock_list
```

Removes the propagated clock assertion for the specified clock waveforms or pins.

The ideal or propagated status of clock endpoints in the fanout of the specified pin/clock list is determined by the remaining clock latency propagated clock assertions in the clock network.

When new analysis views are used, the new corners do not have the source latency updated as per the clock propagation (`update_io_latency`) settings, so a reset and re-calculation is required.

 You must specify `set_interactive_constraint_modes` constraint prior to running the `reset_propagated_clock` command.

Parameters

<code>-help</code>	Prints out the command usage.
<code>pin_or_clock_list</code>	Specifies a list of pins or clocks for which you want to reset the clock propagation mode to ideal mode. The <code>pin_or_clock_list</code> parameter can be a collection, however, no hierarchical pins are allowed in the list.

Examples

- The following command sets the CLK1 clock waveform to propagated mode:

```
% set_propagated_clock CLK1
```

- The following command resets the CLK1 clock waveform back to ideal mode:

```
% reset_propagated_clock CLK1
```

- The following commands set the PH1 clocks to propagated mode and then resets them:

```
% set_propagated_clock [get_clocks CLK_W_1]
% report_clocks
```

- The following commands set the CLK_W_1 clocks to propagated mode and then reports them:

```
% set_propagated_clock [get_clocks CLK_W_1]
% report_clocks
```

The following report is displayed:

Clock Descriptions						
					Attributes	
Clock Name	Source	Period	Lead	Trail	Generated	Propagated
CLK1	CLK1	20.000	0.000	10.000	n	y
CLK2	CLK2	25.000	0.000	12.500	n	y
CLK3	CLK3	10.000	0.000	6.000	n	y
GCLK3	u8/Q	20.000	0.000	10.000	y	y
Generated-Clock Descriptions						
Name	Generated Source (pin)	Master Source (pin)	Master-clock	Invert	Freq. Multiplier	Duty-Cycl

Litmus Text Command Reference

Timing Constraint Commands--reset_pll_timing

```
GCLK3  u8/Q      CLK3      CLK3      n      1/2      -      -      -
```

```
% reset_propagated_clock [get_clocks CLK_W_1]
% report_clocks
```

The following report is displayed:

Clock Descriptions								
Attributes								
Clock Name	Source	Period	Lead	Trail	Generated	Propagated		
CLK1	CLK1	20.000	0.000	10.000	n	n		
CLK2	CLK2	25.000	0.000	12.500	n	y		
CLK3	CLK3	10.000	0.000	6.000	n	y		
GCLK3	u8/Q	20.000	0.000	10.000	y	y		
Generated-Clock Descriptions								
--								
Name	Generated	Master	Master-clock	Invert	Freq.	Duty-Cycle	Edges	Edge-
Shift	Source(pin)	Source(pin)			Multiplier			
-								
GCLK3	u8/Q	CLK3	CLK3	n	1/2	-	-	-
-								
-								

Related Commands

- [create_clock](#)
- [get_propagated_clock](#)
- [report_clocks](#)
- [set_clock_latency](#)
- [set_propagated_clock](#)
- [set_interactive_constraint_modes](#)

reset_property

```
reset_property  
[-help]  
[-obj_type {object_type}]  
{obj_list}  
property_name
```

Resets user-defined properties on objects set using the `set_property` command.

Parameters

<code>-help</code>	Prints out the command usage.
<code>obj_list</code>	Specifies a list of objects for which the property is to be reset.
<code>-obj_type {object_type}</code>	Specifies the type of object on which the property is to be reset.
<code>property_name</code>	Specifies the name of the property for which the user-defined property is to be reset.

Examples

The following command resets user-defined property `slk` from pins `C1/A`:

```
% reset_property [get_pins C1/A] slk
```

Related Commands

- `get_property`
- `set_property`
- `define_property`

reset_pulse_clock_max_transition

```
reset_pulse_clock_max_transition  
[-help]  
obj_list  
[-transitive_fanout]  
[-rise]  
[-fall]
```

Resets the maximum pulse clock transition constraint specified (using the `set_pulse_clock_max_transition` command) on the pulse generator cells, pulse generator library cells, clocks, and/or current design.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-fall</code>	Resets transition limit value for falling transition.
<i>obj_list</i>	Specifies a list of objects - pulse generator cells, pulse generator library cells, clocks, or current design - for which the transition value is to be reset.
<code>-rise</code>	Resets transition limit value for rising transition.
<code>- transitive_fanout</code>	Specifies the transition limit value for transitive fanout.

Related Commands

- `set_pulse_clock_max_transition`

reset_pulse_clock_max_width

```
reset_pulse_clock_max_width  
[-help]  
obj_list  
[-transitive_fanout]
```

Resets the maximum pulse clock width constraint specified (using the `set_pulse_clock_max_width` command) on the pulse generator cells, pulse generator library cells, clocks, and/or current design.

Parameters

<code>-help</code>	Prints out the command usage.
<i>obj_list</i>	Specifies a list of objects - pulse generator cells, pulse generator library cells, clocks, or current design - for which the maximum pulse clock width is to be reset.
<code>[-transitive_fanout]</code>	Specifies the pulse width value for transitive fanout.

Related Commands

- `set_pulse_clock_max_width`

reset_pulse_clock_min_transition

```
reset_pulse_clock_min_transition  
[-help]  
obj_list  
[-transitive_fanout]  
[-rise]  
[-fall]
```

Resets the minimum pulse clock transition constraint specified (using the `set_pulse_clock_min_transition` command) on the pulse generator cells, pulse generator library cells, clocks, and/or current design.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-fall</code>	Resets transition limit value for falling transition.
<code>obj_list</code>	Specifies a list of objects - pulse generator cells, pulse generator library cells, clocks, or current design - for which the minimum pulse clock transition value is to be reset.
<code>-rise</code>	Resets transition limit value for rising transition.
<code>-transitive_fanout</code>	Specifies the transition limit value for transitive fanout.

Related Commands

- `set_pulse_clock_min_transition`

reset_pulse_clock_min_width

```
reset_pulse_clock_min_width  
[-help]  
obj_list  
[-transitive_fanout]
```

Resets the minimum pulse clock width constraint specified (using the `set_pulse_clock_min_width` command) on the pulse generator cells, pulse generator library cells, clocks, and/or current design.

Parameters

<code>-help</code>	Prints out the command usage.
<i>obj_list</i>	Specifies a list of objects - pulse generator cells, pulse generator library cells, clocks, or current design - for which the minimum pulse clock width is to be reset.
<code>[-transitive_fanout]</code>	Specifies the pulse width value for transitive fanout.

Related Commands

- `set_pulse_clock_min_width`

reset_resistance

```
reset_resistance
[-help]
net_list
```

Resets the existing `set_resistance` assertions specified on nets.

Parameters

<code>-help</code>	Prints out the command usage.
<code>net_list</code>	Specifies a list of nets for which the wire resistance is reset.

Related Commands

- `set_resistance`

reset_sense

```
reset_sense  
[-help]  
[-all | [[-clocks clock_list] pin_or_port_list]]
```

Resets clock sense constraint(s) specified using the `set_sense` command.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-all</code>	Resets all the specified <code>clock_sense</code> constraints.
<code>-clocks</code> <i>clock_list</i>	Specifies the list of clocks for which the clock sense constraints are to be removed. The clocks must have previously been defined using the <code>set_sense -clocks</code> parameter.
<i>pin_or_port_list</i>	Specifies the list or collection of cell output pins or ports for which the clock sense constraints will be removed.

Examples

- The following command resets all the previously defined `clock_sense` constraints from a design:

```
Litmus > reset_sense -all
```

Related Commands

- `set_sense`

set_active_clocks

```
set_active_clocks
[-help]
active_clock_list | all_clocks
```

Defines a list of active clocks in the design.

When the `set_active_clocks` command is specified, only the specified clocks are made active while others remain inactive. Any clock created after using the `set_active_clocks` command is active by default. The following commands are impacted:

- `report_clocks` command output displays a new column showing active or inactive clocks in the design.
- `get_clocks` command will report all clocks including inactive clocks.
- `write_timing_windows` command will not display windows for inactive clocks in the output report.
- `get_property` command allows you to query on active clocks using the `is_clock_active` property.

Parameters

<code>-help</code>	Prints out the command usage.
<code>active_clock_list all_clocks</code>	Specifies a list of active clocks.

Examples

The following command makes the specified clocks active while others remain inactive (only `Iclk1` is active, while `Iclk2` is inactive):

```
% set_active_clocks Iclk1
```

Related Commands

- `report_clocks`
- `get_clocks`
- `get_property`

set_annotated_check

```
set_annotated_check  
[-help]  
value  
[-clock clock_check]  
[-cond sdf_condtion]  
[-fall]  
-from from_pins  
[-incremental]  
[-max]  
[-min]  
[-rise]  
-to to_pins  
{-setup | -hold | -recovery | -removal | -nochange_high | -nochange_low}
```

Annotates the setup, hold, recovery, or removal timing check value between two or more pins of a cell in the current design.

Parameters

-help	Prints out the command usage.
value	Specifies the timing check value.
-clock <i>clock_check</i>	Specifies clock rising or falling. Default: Sets checks for both clock rise and fall. Use the <i>clock_check</i> argument to specify rise or fall.
-cond <i>sdf_condition</i>	Specifies the condition only if the library has a condition attached to the specified timing check arc. The <i>sdf_expression</i> must match the syntax of the condition specified in the library.
-fall	Specifies the data fall transition for the timing check. Default: Sets both -rise and -fall.
-from <i>from_pins</i>	Specifies start points of the timing arcs for which checks are annotated. Use the <i>from_pins</i> argument to specify a list of leaf cell clock pins.
-incremental	Sets the incremental checks.
-max	Specifies the maximum timing check for both data rise and data fall transitions. Use this option only if the design uses minimum and maximum (<i>min_max</i>) operating conditions.
-min	Specifies the minimum timing check for both data rise and data fall transitions. Use this option only if the design uses minimum and maximum (<i>min-max</i>) operating conditions.
-rise	Specifies the data rise transition for the timing check. Default: Sets both rise and fall.
-setup -hold -recovery -removal -nochange_high - nochange_low	Specifies the timing check type. Make sure there is a corresponding timing arc between the <i>from_pins</i> and the <i>to_pins</i> arguments.
-to <i>to_pins</i>	Specifies endpoints of the timing arcs for which checks are annotated. Use the <i>to_pins</i> argument to specify a list of leaf cell clock pins.

Examples

- The following commands annotate a `nochange` check between data low and clock low. The nochange margin before the clock is 1.3 and the nochange margin after the clock is 2.4:

```
set_annotated_check -nochange_low 1.3 -clock fall -fall -from {I1/CP} -to {I1/EN}  
set_annotated_check -nochange_low 2.4 -clock fall -rise -from {I1/CP} -to {I1/EN}
```

- The following command annotates a 2.0 setup time between the CP clock pin and the A data pin of the `<instance_name>` instance:

```
% set_annotated_check -setup 2.0 -from <instance_name>/CP -to <instance_name>/A
```

Related Commands

- `reset_annotated_check`
- `set_annotated_delay`

set_annotated_delay

```
set_annotated_delay
[-help]
delay_value
[-cond expression]
[-delta_only]
[-fall]
[-increment]
[-max]
[-min]
[-rise]
{-net | -cell }
{[-from from_pins] [-to to_pins]}
```

Annotates delay to timing arcs. If you specify the `set_annotated_delay` command without any parameters, the delay value specified is assumed to include both the base delay and any delta (cross-talk) delay.

Parameters

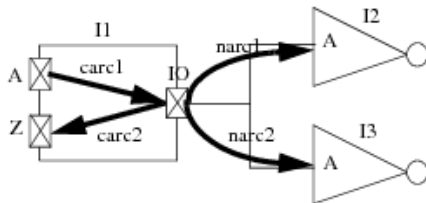
<code>-help</code>	Prints out the command usage.
<code>-cond</code> <code>expression</code>	Specifies the sdf condition of the selected timing arcs. <i>Default:</i> The delay applies to all the timing arcs selected using the <code>-from</code> and the <code>-to</code> options. The expression is ignored for net arcs. Note: You cannot specify the <code>-cond</code> parameter with <code>-net</code>. If specified, an error is displayed.
<code>value</code>	Specifies the delay value to be annotated to the selected timing arcs.
<code>-delta_only</code>	Populates the incremental delay slot with the delay value, so that you can differentiate between the base net delay and a delay offset that is due to crosstalk. The software reports the incremental (or delta) delay value separately in the timing report using <code>report_timing -format incr_delay</code>, and maintains the separate value for CPPR calculations. If the base delay was previously calculated or annotated from the SDF, the <code>-delta_only</code> parameter adds the specified delay to the existing base delay. Note: This parameter only can be used with the <code>-net</code> parameter; you cannot specify it with the <code>-cell</code> parameter.
<code>-from</code> <code>from_pins</code>	Specifies timing arcs to which the delay applies. The selected timing arcs are those that have their source pins specified using this option and the sink pins specified using the <code>-to</code> option. If this option is not specified, only timing arcs whose sink pins are specified using the <code>-to</code> option are selected. Note: You cannot specify hierarchical pins. If specified, an error message is displayed.
<code>-increment</code>	Increments the specified delay value on top of the existing base delay value.

<code>-min -max</code>	Specifies the annotated delay for a particular operating corner. You can apply the <code>-min</code> parameter to the minimum operating corner and the <code>-max</code> parameter to the maximum operating corner.
<code>-net -cell</code>	Specifies whether the delay should be annotated to cell arcs or net arcs.
<code>-rise/-fall</code>	Specifies the transition to which the delay applies. For cell arc, this is the output transition. <i>Default:</i> The delay applies to both transitions.
<code>-to to_pins</code>	Specifies timing arcs to which the delay applies. The selected timing arcs are those that have their sink pins specified using this option and the source pins specified using the <code>-from</code> option. If this option is not specified, only timing arcs whose source pins are specified using the <code>-from</code> option are selected. Note: You cannot specify hierarchical pins. If specified, an error message is displayed.

Examples

- The following example shows the usage of `-cell` and `-net` options:

Example Figure to Show How to Use the -cell and -net Options

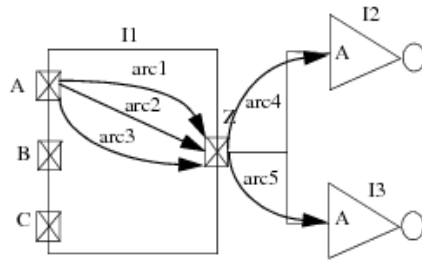


- Use the `-cell` option to select the cell arc `carc2` only:
`set_annotated_delay 3.0 -cell -from {I1/IO}`
- Use the `-net` arc option to select net arcs `narc1` and `narc2` only:
`set_annotated_delay 3.0 -net -from {I1/IO}`

- This TLF description is used for the following examples:

```
CELL (NAND3 . . . . // arc 1 Path( A => Z 10 01 DELAY(ioDelayRiseModel0) SLEW(SlopeRiseModel0) )
Path( A => Z 01 10 DELAY(ioDelayFallModel0) SLEW(SlopeFallModel0) ) // arc2 Path( A => Z 10 01
COND(B & C) SDF_COND(B & C) DELAY(ioDelayRiseModel1) SLEW(SlopeRiseModel1) ) Path( A => Z 01 10
COND(B & C) SDF_COND(B & C) DELAY(ioDelayFallModel1) SLEW(SlopeFallModel1) ) arc3 Path( A => Z 10
01 COND(~B | ~C) SDF_COND(!B | !C) DELAY(ioDelayRiseModel2) SLEW(SlopeRiseModel2) ) Path( A => Z
01 10 COND(~B | ~C) SDF_COND(!B | !C) DELAY(ioDelayFallModel2) SLEW(SlopeFallModel2) ) . . . }
```

Example Figure to Show How to Use the -from and -to Options



- The following command annotates a delay of 3.0 to arcs `arc1`, `arc2`, and `arc3` for the rise and fall transition at the pin `I1/Z`, for the two corners `{min typ max}`: `set_annotated_delay 3.0 -from A -to Z -cell`
- The following command annotates a delay of 2.5 to net `arc4` for both the rising and the falling transition and for the two corners `{min typ max}`: `set_annotated_delay 2.5 -from I1/Z -to I2/A -net`
- The following command annotates a delay of 1.5 to `arc2` for the rising transition and the min corner:
`set_annotated_delay 1.5 -from I1/A -to I1/Z -rise -min -cond "B & C" -cell`
- The following commands annotate a delay of 3.0 to `arc1` and `arc2`, and a delay of 2.4 to the `arc3`:
`set_annotated_delay 3.0 -from I1/A -to I1/Z -cell set_annotated_delay 2.4 -from I1/A -to I1/Z -`
`cond {!B | !C} -cell`
- The following commands first annotate a delay of 3.0 to `arc1`, `arc2` and `arc3`, then overwrite the value set by the first `set_annotated_delay` command: `set_annotated_delay 2.4 -from I1/A -to I1/Z -cond {!B | !C} -cell`
`set_annotated_delay 3.0 -from I1/A -to I1/Z -cell`

Related Commands

- `reset_annotated_delay`
- `set_annotated_check`

set_annotated_transition

```
set_annotated_transition  
[-help]  
[-rise | -fall]  
[-min | -max]  
value  
pin_list
```

Sets the transition time to be annotated on specified ports or pins in the current design. The transition time that you specify using this command overrides the transition time calculated internally within the software. You can use this command to set the transition time of internal cell instance pins.

Parameters

-help	Prints out the command usage.
-min -max	Specifies whether the transition value is set for minimum or maximum transition time. <i>Default:</i> Transition value is set for both transition times.
pin_list	Specifies a list of pins or ports to be annotated with the transition time that you specify using the <i>value</i> parameter.
-rise -fall	Specifies whether the transition value is set for rising or falling transition time. <i>Default:</i> Transition value is set for both transitions.
value	Specifies the transition time to be annotated to the selected pins or ports.

Examples

- The following command annotates a transition time value of 0.6 on port A:

```
% set_annotated_transition 0.6 [get_ports A]
```

- The following command annotates a transition time value of 0.7 on pin I2/A:

```
% set_annotated_transition 0.7 [get_pins I2/A]
```

- The following command annotates a transition value of 0.7 on pin I2/A for rising transition time:

```
% set_annotated_transition -rise 0.7 [get_pins I2/A]
```

- The following command annotates a transition value of 0.7 on pin I2/A for rising transition time and minimum pvt corner:

```
% set_annotated_transition -rise -min 0.7 [get_pins I2/A]
```


Related Commands

- `set_input_transition`
- `get_pins`
- `get_ports`
- `report_ports`
- `reset_annotated_transition`

set_case_analysis

```
set_case_analysis  
[-help]  
{0 | 1 | zero | one | rising | falling | rise | fall}  
pin_or_port_list
```

Sets constant or transitional values to a list of pins or ports for use by the timing engine. The specified constants or transitional values are valid only during timing analysis and do not alter the netlist. If there is a conflict between the specified `set_case_analysis` value and the logical value from the netlist, the value from the `set_case_analysis` is used in timing analysis.

Parameters

<code>-help</code>	Prints out the command usage.
<code>0 1 zero one</code>	Specifies a logic value of 0 or 1, which is propagated through combinational logic to disable or enable parts of logic.
<code>rising falling rise fall</code>	Specifies a valid transition value to be considered by the timing engine. For example, if a rising value is specified at a certain pin or port, only the rising signal transition from the pin or port is considered for timing analysis and the opposite transition type (falling) is ignored.
<code>pin_or_port_list</code>	Specifies a list of ports or pins on which to apply the constant or transitional value. Specify multiple pins by enclosing them with curly braces {} and separating the pin names with white space. The <code>pin_or_port_list</code> argument can be a collection.

Examples

- The following command sets the value of pin i4/A to 1 for use by the timing engine:

```
% set_case_analysis 1 i4/A
```

- The following example shows that the pins P1/P2/B and P1/P3/CI are considered only for a rising transition. The falling transition on these pins is ignored:

```
% set_case_analysis rising {P1/P2/B P1/P3/CI}
```

Related Commands

- `check_timing`
- `report_case_analysis`
- `report_timing`
- `set_disable_timing`
- `set_false_path`

set_clock_gating_check

```
set_clock_gating_check
[-help]
[object_list]
[-fall]
[-hold hold_value]
[-rise]
[-setup setup_value]
[-high | -low]
```

Specifies or overrides the default setup and hold values for clock gating checks.

Parameters

-help	Prints out the command usage.
-high -low	Specifies that the non-controlling value of the clock is high or low. Timing analysis performs the check on the non-controlling value of the clock. By default, the software determines the non-controlling value of the clock using information from the gate's logic. For example, for OR gates the non-controlling value is low. For complex gates, such as XOR and MUX, the clock is never controlling and it does not perform checks unless you specify either the <code>-high</code> option or the <code>-low</code> option. The specified value takes precedence over the default values. This option can only be used with pins or instances. Note: The clock pin value is controlling if it causes the gate output to be a constant (0 or 1).
-hold hold_value	Specifies the hold value. The default is 0.0.
obj_list	Specifies a list of clocks, instances, or pins. If a clock is specified, the check applies to all the clock gating gates driven by this clock. If an instance is specified, the check applies to all input pins of the instance. If the <code>obj_list</code> argument is not specified, the clock gating check is applied to the current design.
-rise - fall	Applies the specified setup (or hold) value to the rising (or falling) signals on the specified pins. If the clock is specified or the <code>obj_list</code> variable is not specified, the value applies to rising or falling, setup or hold. If neither option is specified, the default is both rising and falling.
-setup setup_value	Specifies the setup value. The default is 0.0.

 The clock gating constraint is not propagated along the clock tree.

The setup check is performed with respect to the edge of the clock signal that changes the state of the clock pin from controlling to non-controlling. For example, for AND and NAND gates, the setup check is performed with respect to the rising edge of the clock input. For OR and NOR gates, the setup check is performed with respect to falling edge of the clock input.

The hold check is performed with respect to the edge of the clock signal that changes the state of the clock pin from non-controlling to controlling. For example, for AND and NAND gates, the hold check is performed with respect to the falling edge of the clock input. For OR and NOR gates, the hold check is performed with respect to rising edge of the clock input.

If the gate logic is unknown, setup and hold checks are performed with respect to both the rising edge and the falling edge of the clock signal and the timing analyzer uses the worst-case values.

Priority Rules for Determining Setup or Hold Values

Timing analysis uses the following priority rules to determine the setup (or hold) value to use for a particular clock gating check:

1. Use the clock gating check assertion on the data pin if it exists
2. Otherwise, use the clock gating check assertion on the clock pin if it exists
3. Otherwise, use the clock gating check assertion on the clock waveform for the clock pin if it exists
4. Otherwise, use the attribute clock gating check assertion if it exists
5. If no clock gating check assertion is present, use the default setup (or hold) value of 0.0.

These rules are demonstrated in the examples below.

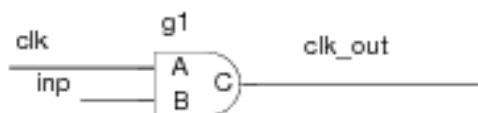
 This command has no effect on a clock gating check defined as a non-sequential setup or hold check in the library and reported as `ClockGatingSetup` or `ClockGatingHold`.

Examples

Here are some examples.

- The following example shows the priority rules for multiple clock gating check constraints for the gated clock shown in figure below:

Gated Clock



Tcl Script for set_clock_gating_check

```
% create_clock -name CLK1 -period 12 -waveform {0 6} clk
```

...

```
# clock-gating timing check assertions:
# On ideal clock waveform
```

```
set_clock_gating_check -setup 0.44 [get_clocks {CLK1}]
```

On clock pin

```
set_clock_gating_check -hold 0.11 [get_pins {g1/A}]
```

On data pin

```
set_clock_gating_check -setup 0.66 -rise [get_pins {g1/B}]  
set_clock_gating_check -hold 0.22 -fall [get_clocks {g1/B}]
```

Given the previous constraints, the following timing margins are used for slack calculation:

- SETUP g1/B rise: 0.66 -- from assertion on data pin g1/B ([rule_1](#))
- SETUP g1/B fall: 0.44 -- from assertion on ideal clock CLK1 ([rule_3](#))
- HOLD g1/B rise: 0.11 -- from assertion on clock pin g1/A ([rule_2](#))
- HOLD g1/B fall: 0.22 -- from assertion on data pin g1/B ([rule_1](#))

Related Commands

- [create_clock](#)
- [reset_clock_gating_check](#)

set_clock_groups

```
set_clock_groups
[-help]
[-name name_list]
[-comment string]
[-logically_exclusive] | [-physically_exclusive] | [[-asynchronous]
[-allow_paths]]
[-group clock_list]
```

Defines clock groups with specified clock definitions. The software assigns all clocks that are defined after you use the `set_clock_groups` command to a default clock group. The software does not apply clock groupings on a master clock to the generated child clocks.

If there is an overlap in the clock group specification, the software uses the following precedence to resolve the behavior:

1. Physically exclusive
2. Asynchronous
3. Logically exclusive

The software drops any explicit false path assertions between clocks that you have set as either physically or logically exclusive or asynchronous using the `set_clock_groups` command.

An alternative to setting the clocks as physically or logically exclusive using the `set_clock_groups` command is to use the multi-mode mechanism. Generally, in an implementation flow, sequential setting of active clocks cannot be used. For concurrent analysis of different modes, you can use a multi-mode-based approach to drive IPO rather than using the `set_clock_groups` command.

 If the `set_clock_groups` constraint is added incrementally in SI mode, an explicit `update_timing - full` command is required (before `report_timing` or `report_constraint`) for SI full accuracy.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-allow_paths</code>	Allows paths between asynchronous clock groups to be traced in timing analysis. These paths are blocked by default. When you use this option, the software still considers the paths to be asynchronous for signal integrity considerations. Signal integrity analysis does not support the <code>-allow_paths</code> parameter. Use the <code>-allow_paths</code> parameter with the <code>-asynchronous</code> parameter.
<code>-asynchronous</code>	Defines clock group with asynchronous clocks. Asynchronous clocks have no specified phase relationship. The software considers asynchronous nets to have infinite timing window overlap for signal integrity considerations and considers these as false paths for timing analysis. Use the <code>-allow_paths</code> parameter to trace these paths in timing analysis.
<code>-comment string</code>	Allows you to insert comments. The comments are read but are ignored with a one-time warning message. They are not used during timing analysis reporting nor are they written out during saving of constraints.
<code>-group clock_list</code>	Defines a set of clocks that are exclusive or asynchronous to the clocks defined in all other groups. If you define only one group, then all clocks not in the group implicitly belong to a default clock group. The software automatically adds any new clocks that you define after the <code>set_clock_groups</code> command to this default clock group.
<code>-logically_exclusive</code>	Defines clock group with clocks that do not have any functional paths between them but coexist in the design at the same time. The software considers aggressor interactions between these clocks for signal integrity considerations but considers these as false paths for timing analysis. Note: The <code>-logically_exclusive</code> parameter does not have any impact on SI analysis. For example, clocks selected via a clock mux select.
<code>-name name_list</code>	Specifies the names of the clock groups to be added.
<code>-physically_exclusive</code>	Defines clock group with clocks that cannot exist in the design at the same time. The software does not consider aggressor interactions between clock groups that are physically exclusive for signal integrity considerations. From the perspective of SI analysis, all attackers that are physically exclusive with the victim net are ignored for both glitch and noise-on-delay analysis. Attacker nets that belong to physically exclusive groups cannot attack together. In this case, all sets of attackers belonging to all the physically exclusive groups are evaluated and the set of nets that has the highest impact on SI analysis is chosen. For timing analysis, the software considers these clock groups as false paths. Example: multiple clocks defined on the same source pin

Examples

Here are some examples.

- The following commands assert two different clocks on the same clock root port, and uses `set_clock_groups` to isolate them from each other:

```
% create_clock -name SYSCLK -period 10 [get_ports sclk]
% create_clock -name SCANCLK -period 100 -add [get_ports sclk]
% set_clock_groups -name PHY_EX_SCLK -physically_exclusive -group {SYSCLK} -group {SCANCLK}
```

- The following command declares a single bus clock to be asynchronous to all of the other on-board clocks in the design (which remain in the default clock group):

```
% set_clock_groups -name ASYNCH_BUS -asynchronous -group {BUSCLK}
```

Assume a design with four clocks CLK1, CLK2, CLK3, and CLK4.

- The following command defines CLK1 physically exclusive to CLK2, CLK3 as well as CLK4:

```
% set_clock_groups -name group1 -group CLK1 -physically_exclusive
```

- The following command defines CLK1 and CLK2 to be physically exclusive to CLK3 and CLK4:

```
% set_clock_groups -name group2 -group {CLK1 CLK2} -physically_exclusive
```

- The following command defines CLK1 and CLK2 as physically exclusive to CLK3:

```
% set_clock_groups -name group3 -group {CLK1 CLK2} -group {CLK3} -physically_exclusive
```

- The following command defines the following:

```
% set_clock_groups -name group4 -group {CLK1} -group {CLK2} -group {CLK3} -physically_exclusive
```

- CLK1 physically exclusive to CLK2 and CLK3.
- CLK2 physically exclusive to CLK1 and CLK3.
- CLK3 physically exclusive to CLK1 and CLK2.

- The following example shows that the clock group {clkA clkB} is asynchronous only to clock group {clkC clkD}:

```
% set_clock_groups -asynchronous -group {clkA clkB} -group {clkC clkD}
```

If an extra clock clkE is defined in the design, then timing check will be allowed in between the first group to clkE and second group to clkE.

- In the following example since only one clock group is specified with the command `set_clock_groups` command, so the specified clock group is considered to be asynchronous to all the other clocks in the design:

```
% set_clock_groups -asynchronous -group {clkA clkB}
% set_clock_groups -asynchronous -group {clkC clkD}
```

Related Commands

- `create_clock`
- `reset_clock_groups`

set_clock_latency

```
set_clock_latency  
[-help]  
[-source [-early | -late]]  
[-rise]  
[-fall]  
[-dynamic jitter_val]  
[-clock clock_list ]  
[-min]  
[-max]  
[-clock_gate]  
value  
obj_list
```

Specifies ideal internal clock value and external clock arrival delay.

Clock value has two components:

1. Source value:

- a. Specified using the `-source` parameter and is the delay from the external clock generator to the clock port of the design.
- b. Applied when the clock is either ideal or propagated.
- c. `set_clock_latency` set on a clock port has higher precedence than `set_clock_latency` set on a clock waveform.
- d. Specified on clock waveforms with both the `set_input_delay` command and the `set_output_delay` command.

2. Network value (clock tree delay):

- a. Default value type and is the delay from the clock port to the register.
- b. Applied only when the clock is ideal. When the clock is propagated, this delay is replaced by the actual clock tree delays.
- c. Specified on a clock waveform or a clock port is ignored if the clock is propagated.
- d. `set_clock_latency` set on a timing pin has higher precedence than `set_clock_latency` set on a waveform.
- e. If there are multiple `set_clock_latency` and `set_propagated_clock` assertions between the clock waveform, clock root and clock endpoint, the closest assertion to the clock endpoint takes precedence.
- f. Specified on internal circuit pins or hierarchical ports and used in the fanout cone of the network value pin, when a clock is ideal.

Use the `set_clock_latency` command in two ways:

1. To specify clock value for a clock waveform:

- a. Use the following syntax for source value associated with a clock signal:

```
set_clock_latency [-source] [-early | -late] [-min ][-max] value obj_list
```

- b. Use the following syntax for network value associated with a clock signal:

```
set_clock_latency [-min] [-max] value obj_list
```

You cannot specify the `-late` and `-early` parameters for network value.

Specifying value on a port or pin overrides any previous value specified on the port pin or waveform. The `obj_list` argument is the list of the clock waveform names as specified by the `create_clock` command.

2. To specify clock value on clock pins:

- a. Use the following syntax for source delay on a clock port:

```
set_clock_latency -source [-early | -late] [-rise] [-fall]  
insertion_delay obj_list
```

- b. Use the following syntax for network delay on a clock pin:

```
set_clock_latency [-min] [-max] [-rise] [-fall]  
insertion_delay -pin obj_list
```

When a clock is ideal, the last specification along a path is used. This is useful when you want to intentionally skew the clock tree and model the physical hierarchical clock tree.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-clock clock_list</code>	Associates clock value constraints, placed at the pin level, with a specific clock. This makes it possible to specify a different value per clock. Note: You cannot use the <code>-clock</code> parameter with clock waveform objects.
<code>-clock_gate</code>	Allows setting the local clock value value on pin or port objects. This setting overwrites the existing value value of clock gating cells and is used when performing timing checks against the specified pins or ports.
<code>-dynamic jitter_val</code>	Specifies the portion of source clock latency value that can have cycle-to-cycle variation. When calculating CPPR adjustment this portion is removed from CPPR adjustment for different clock edge checks (e.g., setup), but it remains as part of cppr adjustment for same clock edge checks (e.g., hold).
<code>-early -late</code>	Specifies clock arrival time with respect to the early or the late time of the clock signal. In setup analysis, launch path is the late path and capture path is the early path. In hold analysis, launch path is the early path and the capture path is the late path. If neither parameter is specified, the default is both early and late. Use these parameters only with the <code>-source</code> parameter. You cannot use these parameters with network value. Specify the clock value at the given operating corner. Note: The <code>-min</code> and <code>-max</code> parameters refer to the operating corner and the <code>-early</code> and <code>-late</code> parameters refer to the clock arrival time at the source. Use the <code>-early</code> and <code>-late</code> parameters with the <code>-source</code> parameter to specify the four different clock latencies.
<code>value</code>	Specifies the value value, as a floating point.
<code>obj_list</code>	Specifies a list of clock waveform names or a list of pins to associate with the clock value. The pin can be a port or an instance pin. Setting source value on a pin is ignored.
<code>-max</code>	Specifies the clock value for a particular operating corner. Apply the <code>-max</code> parameter to the maximum operating corner.
<code>-min</code>	Specifies the clock value for a particular operating corner. Apply the <code>-min</code> parameter to the minimum operating corner.
<code>-rise/ -fall</code>	Specifies the <code>-rise</code> or <code>-fall</code> parameter to the waveform at register clock pins for ideal value and to the waveform at the source for source value. Ideal network value does not change polarity when crossing negative unate arcs.
<code>-source</code>	Specifies a source value. If you do not specify the <code>-source</code> parameter, network value is applied.

Examples

The following set of commands enable you to create a clock using the `create_clock` command and apply latency to the created clock using the `set_clock_latency` command:

```
% create_clock CLK2 -period 20 [get_ports {clkA clkB clkD}]
% set_clock_latency -source 2 {clkB clkD} # Define port settings
% set_clock_latency -rise 5 {clkB clkD} # Define port settings
% set_clock_latency -early -late -source CLK1 # Define waveform settings
% set_clock_latency 3 -early -late -source CLK1 # Define waveform settings
```

Related Commands

- `create_clock`
- `set_clock_transition`
- `set_clock_uncertainty`
- `set_input_delay`
- `set_output_delay`
- `set_propagated_clock`

set_clock_sense

```
set_clock_sense  
[-help]  
[-positive | -negative | -stop_propagation | -logical_stop_propagation | -stoptype_list]  
[-clocks clock_list]  
pin_or_port_list
```

Selects which phase of the clock to filter at the specified point.

Parameters

-help	Prints out the command usage.
-clocks <i>clock_list</i>	Specifies a list of clocks passing through a specified point to which the constraint applies. When this parameter is not specified, the filtering of clock phase will be applicable to all the clocks passing through that specified point. This parameter must be used with a specified pin name and one of the parameters : -positive, -negative, -stop_propagation, -logical_stop_propagation, or -stop.
- logical_stop_propagation	Stops the propagation of specified clocks passing through the specified pins.
-negative	Specifies that positive phase filtered clocks pass through pins in the <i>pin_list</i> .
<i>pin_or_port_list</i>	Specifies the list or collection of cell output pins. The unateness specified by the set_clock_sense command is applied to clocks propagating through the specified point.
-positive	Specifies that negative phase filtered clocks pass through pins in the <i>pin_list</i> .
-stop_propagation	Stops the propagation of specified clocks and clock source data paths passing through the specified pins.

`-stop type_list`

Applies clock sense to object types specified in the *type_list*. The type list options are - all, clock, data, clock_source_data, default_data.

These are explained below:

- **all**: Stops all phases from propagating. This is equivalent to specifying both the **clock** and **data** options.
- **clock**: Stops clock phases launched by clock(s) specified in the *clock_list* from propagating, and is equivalent to existing `-logical_stop_propagation`.
- **data**: Stops all data phases launched by clock(s) specified in *clock_list*, starting from the pins/ports specified in *pin_or_port_list*. The objects in the *pin_or_port_list* can be selected anywhere in the design - clock/data network.
- **clock_source_data**: Stops only clock source data phases from propagating. The combination of **clock_source_data** and **clock** is equivalent to the existing `-stop_propagation` option.
- **default_data**: Stops only default data phases (representing constrained paths) from propagation. This option cannot be specified along with the `-clock clock_list`.

The `-stop` parameter extends the capability of the `set_clocks_sense` command to selectively block data timing phases. This parameter provides a more generalized use model for selecting among all types of paths that should be filtered.

The `-stop` parameter is mutually exclusive with the `-positive`, `-negative`, `-stop_propagation`, and `-logical_stop_propagation` options.

Examples

- The following command stops the propagation of clock CLK1 through the pin CLKBUF1/Z:

```
% set_clock_sense -logical_stop_propagation -clocks CLK1 CLKBUF1/Z
```

- The following command allows the positive phase clock CLK1 to pass through CLKBUF1/Z and filters out the negative phase clock CLK1:

```
% set_clock_sense -positive -clocks CLK1 CLKBUF1/Z
```

- The following command stops clock phases starting from pin1:

```
% set_clock_sense pin1 -stop {clock}
```

or

```
% set_clock_sense pin1 -logical_stop_propagation
```

- The following command stops data phases based only on CK1 and CK2 starting from pin1:

```
% set_clock_sense pin1 -stop {data} -clocks {CK1 CK2}
```

- The following command stops only the clock source data phases starting from pin1:

```
% set_clock_sense pin1 -stop {clock_source_data}
```

- The following command stops the clock and clock source data phases:

```
% set_clock_sense pin1 -stop {clock clock_source_data}
```

or

```
% set_clock_sense pin1 -stop_propagation
```

Related Commands

- [reset_clock_sense](#)

set_clock_transition

```
set_clock_transition
[-help]
[-rise]
[-fall]
[-min]
[-max]
[-min | -max]
transition_time
clock_list
```

Specifies the slew time of register clock pins. This is useful for pre-layout static timing analysis before clock tree synthesis (CTS) is performed since net delays can be inaccurate with large fanout nets.

Use this command only with ideal clocks. In propagated mode, the software calculates slew times on register clock pins.

Parameters

<code>-help</code>	Prints out the command usage.
<code>clock_list</code>	Specifies a list of clocks only with ideal mode in the design, which affects the slew times on all register clock pins in the transitive fanout of the specified clocks. Specified <code>set_clock_transition</code> values on register clock pins in the fanout of a clock with propagated latency are ignored.
<code>-rise -fall</code>	Specifies the rising or falling edge of the register clock pins on which the slew time is asserted.
<code>-min -max</code>	Specifies the clock slew time for minimum or maximum process corner slew time.
<code>transition_time</code>	Specifies the slew time (transition time) of the clock pins.

Examples

The following commands specify a 2.0 rise slew time and a 1.0 fall slew time for register clock pins clocked by `CLKA`:

```
% set_clock_transition -rise 2.0 [get_clocks CLKA]
% set_clock_transition -fall 1.0 [get_clocks CLKA]
```

Related Commands

- `report_clocks`
- `set_clock_latency`
- `set_input_delay`

set_clock_uncertainty

```
set_clock_uncertainty
[-help]
uncertainty_value
[-setup]
[-hold]
[-half_cycle_jitter | -full_cycle_jitter]
[-rise | -fall]
{ {-from | -rise_from | -fall_from}  clksig_from_list
  {-to | -rise_to | -fall_to}  clksig_to_list
  | clock_or_clock_pin
}
```

Specifies the clock uncertainty (skew) on the clock network.

If you set multiple constraints with the `set_clock_uncertainty` command, the constraints set with the `-from*` `clksig_from_list` `/-to*` `clksig_to_list` parameters take precedence over simple constraints set on clock objects.

Clock uncertainty set on pins takes highest precedence and overrides clock-based uncertainty.

The `set_clock_uncertainty` command can also support clock uncertainty for period, pulse width, skew, and separation checks. To enable this feature, you need to set `timing_enable_uncertainty_for_clock_checks` attribute to `true`. By default, the attribute is set to `false`.

Parameters

<code>-help</code>	Prints out the command usage.
<code>uncertainty_value</code>	Specifies the uncertainty value.
<code>-from -rise_from -fall_from</code> <code>clksig_from_list</code>	Applies the uncertainty value to the specified list of launching clocks. Note: You must specify one of these parameters with one of the <code>-to</code> , <code>-rise_to</code> , or <code>-fall_to</code> parameters. You cannot use these parameters when <code>clock_or_clock_pin</code> is specified.
	• <code>-fall_from</code>: Applies the uncertainty value only to the falling edge of the launching clocks.
	• <code>-from</code>: Applies the uncertainty value to both the rising and falling edges of the launching clocks.
	• <code>-rise_from</code>: Applies the uncertainty value only to the rising edge of the launching clocks.

<code>-half_cycle_jitter value - full_cycle_jitter value</code>	Specifies the clock source jitter for half-cycle and for full-cycle paths. These parameters can accept different values, that can be specified in the same constraint. Note : These parameters can be used on clocks only. You cannot use them when any of the <code>-from</code> and <code>-to</code> parameter pairs are specified.
<code>clock_or_clock_pin</code>	Specifies a list of clocks or pins for target uncertainty. The uncertainty value is applied to the registers clocked by the target waveform, or register pins in the fanout of the specified pin. You can specify a list of clock names, or clock root pin or port names relative to the current module. You also can specify a collection of clocks, or clock root pins or ports. Note: You cannot use this parameter when any of the <code>-from</code> and <code>-to</code> parameter pairs are specified.
<code>-rise -fall</code>	Specifies the rising or falling edge at the capture clock pin on which the uncertainty is applied. <i>Default:</i> Applies uncertainty on both edges Note: You must use these parameters with the <code>-to</code> and <code>-from</code> parameters.
<code>-setup / -hold</code>	Specifies whether the clock uncertainty applies to setup or hold checks. <i>Default:</i> Applies to both setup and hold check
<code>-to -rise_to - fall_to clk_sig_to_list</code>	Applies the uncertainty value to the specified list of capture clocks. Note: You must specify one of these parameters with one of the <code>-from</code>, <code>-rise_from</code>, or <code>-fall_from</code> parameters. You cannot use these parameters if <code>clock_or_clock_pin</code> is specified.
	• <code>-fall_to</code>: Applies the uncertainty value only to the falling edge of the capture clocks.
	• <code>-rise_to</code>: Applies the uncertainty value only to the rising edge of the capture clocks.
	• <code>-to</code>: Applies the uncertainty value to both the rising and falling edges of the capture clocks.

Examples

- The following command gives all paths starting from registers clocked by `clk1` to registers clocked by the rising edge of `clk2` a hold uncertainty of 2.0:

```
% set_clock_uncertainty 2.0 -from clk1 -to clk2 -rise -hold
```

In case of multiple constraints, the actual uncertainty applied by the software depends on which constraint matches the specific path in question.

- The following command sets an uncertainty on the `clk_in` clock root pin that is applied to all the register pins in the fanout of `clk_in`:

```
% set_clock_uncertainty 1.2 clk_in
```

- The following command specifies an inter-clock uncertainty of 0.2 between clock waveforms `clk1` and `clk2`:

```
% set_clock_uncertainty 0.2 -from clk1 -to clk2
```

- The following command specifies a target uncertainty of 0.5 for all registers clocked by the clock waveform `ideal_clk1`:

```
% set_clock_uncertainty 0.5 ideal_clk1
```

Related Commands

- [report_clocks](#)
- [set_clock_latency](#)
- [set_clock_transition](#)

set_data_check

```
set_data_check
[-help]
{-from | -rise_from | -fall_from}  pin_or_port_list
{-to | -rise_to | -fall_to}  pin_or_port_list
[-setup]
[-hold]
[-clock clock_object ]
check_value
```

Sets data-to-data checks using the specified values of setup and hold time. Data-to data checks are non-sequential checks that are designed to constrain a data signal with respect to another data signal.

In a normal sequential check, a data signal is constrained with respect to a clock signal. In these cases, single-cycle timing is used - allowing one full clock cycle for the data transfer. In a non-sequential data-to-data check, there is no clock reference. Therefore zero-cycle timing rules are used. You can use the `set_multicycle_path` constraint to adjust the cycle accounting rules.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-clock</code> <i>clock_object</i>	Specifies the name of the clock that triggers the data event for the related (<code>-from</code>) pin. In the case where there are multiple data events triggered by different clock phases arriving at the <code>-from</code> pin, the <code>-clock</code> parameter allows you to specify which data event should be used for the "related pin" side of the timing check.
<code>-from -rise_from</code> <code> -fall_from</code> <i>pin_or_port_list</i>	Specifies the pins or ports that are set as related pins for data-to-data checks. The <i>pin_or_port_list</i> argument can contain either object IDs or hierarchical names relative to the current module. The pins can be on intermediate hierarchical boundaries. Use only one <code>-from</code> , <code>-rise_from</code> , or <code>-fall_from</code> parameter per command. By default, the <code>-from</code> parameter checks both the rising and the falling edges. Using <code>-rise_from</code> parameter checks at the rising edge of the signal on the from pins. Using <code>-fall_from</code> parameter checks at the falling edge of the signal on the from pins.  You cannot use the <code>-from</code> parameter to specify multiple objects.
<code>-setup -hold</code>	Specifies whether the data check value is for setup or hold check. If you do not specify either <code>-setup</code> or <code>-hold</code> option, the value applies to both setup and hold.
<code>-to -rise_to </code> <code>-fall_to</code> <i>pin_or_port_list</i>	Specifies the pins or ports that are set as constrained pins for data-to-data checks. The <i>pin_or_port_list</i> argument can contain either object IDs or hierarchical names relative to the current module. The pins can be on intermediate hierarchical boundaries. Use only one <code>-to</code> , <code>-rise_to</code> , or <code>-fall_to</code> parameter per command. By default, the <code>-to</code> parameter constrains both the rising and the falling edges. Using <code>-rise_to</code> parameter constrains the rising edge of the signal on the to pins. Using <code>-fall_to</code> parameter constrains the falling edge of the signal on the to pins.
<i>check_value</i>	Specifies the value of the setup or hold time for the check.

Examples

The following example demonstrates how the `set_data_check` command can be used to constrain signals: Consider two control signals, say XE and SE, going to a macro block. Both of these are derived from the same state machine, and hence launched from the same clock domain. You want XE to reach the macro earlier than SE. The `set_data_check` constraint can meet this design intent, by applying data checks on non-sequential signals, without setting path specific limits using the following command:

```
% set_data_check -from [get_pins {sub/SE}] -to [get_pins {sub/XE}] -setup 0.5
```

The `set_data_check` command establishes data-to-data checks using similar ideas of setup and hold time as in sequential checks, with the `-from` parameter denoting the reference signal (the clock in a traditional setup/hold timing check), and the `-to` parameter indicating the signal being constrained.

Related Commands

- `reset_data_check`

set_disable_clock_gating_check

```
set_disable_clock_gating_check  
[-help]  
pin_or_inst_list
```

Disables clock gating checks for the specified instances or pins.

The `set_disable_clock_gating_check` command has a character limit for the details that are logged into the SDC file (the `write_sdc` command output). This helps to prevent SDC file errors.

Parameters

<code>-help</code>	Prints out the command usage.
<code>pin_or_inst_list</code>	Specifies the list of pins or instances that are to be disabled.

Examples

The following command disables clock gating checks for the specified instance:

```
% set_disable_clock_gating_check [get_cells clockgate1]
```

Related Commands

- `reset_disable_clock_gating_check`

set_disable_timing

```
set_disable_timing  
[-help]  
[-from pin_name -to pin_name]  
obj_list
```

Disables timing propagation through the specified arcs, or a collection of instance-specific arcs that are created using the `get_arcs` command.

Parameters

-help	Prints out the command usage.
-from <i>pin_name</i> -to <i>pin_name</i>	Specifies the arc that is to be disabled. Specify the <code>-from</code> and <code>-to</code> options together. You can specify the <code>-from</code> and <code>-to</code> parameters when the <i>obj_list</i> contains instance or library cell objects. Do not specify the <code>-from</code> and <code>-to</code> parameters when the <i>obj_list</i> includes instance pin or library cell pin objects. The <code>-from</code> and <code>-to</code> options are optional. If not specified, all cell arcs found in the instance or library cell objects are disabled.
<i>obj_list</i>	Specifies the instance, instance pin, library cell, library cell pin objects, collection of library arcs, or collection of timing arcs. If the <i>obj_list</i> includes instance pin or library cell pin objects, all cell arcs attached to the specified pin objects are disabled. If the <i>obj_list</i> contains instance or library cell objects, and the <code>-from</code> and <code>-to</code> parameters are not specified, all arcs found in the specified instance or library cell objects are disabled.

Examples

Here are some examples.

- The following command disables the arcs that lead to the output pins of the `i1`, `i2`, and `i3` instances. Note that timing checks at the inputs of `i1`, `i2`, and `i3` are still active.

```
Litmus > set_disable_timing [get_cells {i1 i2 i3}]
```
- The following command disables the arc from pin `B` to pin `Z` in the `i1` and `i2` instances, and the arc from pin `A` to pin `Z` in the `i3` instance:

```
Litmus > set_disable_timing -from B -to Z [get_cells {i1 i2}]  
Litmus > set_disable_timing -from A -to Z [get_cells {i3}]
```
- The following example disables timing check arcs from pin `CLK` to pin `D` of the `ff1` instance:

```
Litmus > set_disable_timing -from CLK -to D [get_cells ff1]
```

- The following example disables the outgoing cell arcs from instance input pin `i1/B`:

```
Litmus > set_disable_timing [get_pins i1/B]
```

- The following example disables all arcs found in any instances whose reference cell is `AN2`:

```
Litmus > set_disable_timing [get_lib_cells AN2]
```

- The following example disables a collection of timing arcs:

```
Litmus > set_arcs [get_lib_arcs -from typical/MX2X1/S0 -to typical/MX2X1/Y]
```

```
Litmus > set_disable_timing [index_collection $arcs 3]
```

- The following example disables arcs created using `filter_collection` on library timing arcs:

```
Litmus > set_arc [filter_collection $arcs {when == "(A)&(!(B))"}]
```

```
Litmus > set_disable_timing $arc
```

Related Commands

- `report_timing`
- `reset_disable_timing`

set_dont_touch

```
set_dont_touch  
[-help]  
obj_list  
[true | false]
```

Prevents the specified object from being modified during optimization. However, the specified object can be moved during optimization.

When you specify a net, it cannot be buffered or become a part of the logic cone restructuring. However, you can resize the instances connected to this net because the `dont_touch` attribute set of the net does not propagate on the connected instances.

Parameters

<code>-help</code>	Prints out the command usage.
<code>true false</code>	Sets or removes the <code>set_dont_touch</code> attribute for the specified <code>obj_list</code> . <i>Default:</i> <code>true</code>
<code>obj_list</code>	Specifies the list of instances, nets, library cells, or submodules.

Examples

The following command prevents the specified instance from being modified:

```
% set_dont_touch [get_cells {TWA/FF1}]
```

Related Commands

- `get_cells`

set_dont_touch_network

```
set_dont_touch_network  
[-help]  
obj_list  
[-no_propagate]
```

Prevents the combinational path connected to the given clocks, pins, or ports from being modified.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-no_propagate</code>	Preserves all the nets driven by the specified pin up to the first leaf-cell input (combinational or sequential). If this parameter is omitted, then preserving stops at the first sequential cell in the network.
<i>obj_list</i>	Specifies the objects - clocks, pins, or ports - to be preserved.

Examples

- The following command prevents the combinational path connected to the specified pin from being modified:

```
set_dont_touch_network [get_pins and1/z]
```

Related Commands

- [get_pins](#)

set_dont_use

```
set_dont_use
[-help]
lib_cell_list
[true | false]
```

Prevents the specified design, hierarchical module, or library cells from being used during optimization. However, optimization is allowed to size out don't use cells if instances are already there in the initial netlist.

Parameters

-help	Prints out the command usage.
lib_cell_list	Specifies the list of library cells.
true false	Sets or removes the set_dont_use attribute for the specified object list. Default: true

Examples

- The following command resets all the previously defined clock_sense constraints from a design:
Litmus > reset_clock_sense -all

Related Commands

- set_clock_sense

set_drive

```
set_drive
[-help]
[-rise] [-fall]
[-max] [-min]
resistance_value
port_list
```

Sets the drive resistance on the specified input and/or bidirectional ports of the top cell.

The drive resistance models the cell driving the port. This resistance is used to compute the load dependent delay and the slew at the specified ports. The load-dependent delay is added to the arrival times.

The load-dependent delay and the slew are computed by multiplying the drive resistance by the wire capacitance of the net connected to the port.

Use the [report_ports](#) command to view drive resistances asserted on ports.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-fall</code>	Specifies that the drive resistance is to be used for the falling transition at the specified ports. By default, if neither the <code>-rise</code> nor the <code>-fall</code> option are specified, the drive resistance applies to both the rising transition and the falling transition.
<code>-min</code>	Specifies that the drive resistance is to be used to compute the early path delay. By default, if neither the <code>-min</code> option nor the <code>-max</code> option is specified, the drive resistance applies to both the early path delay and the late path delay.
<code>-max</code>	Specifies that the drive resistance is to be used to compute the late path delay. By default, if neither the <code>-min</code> option nor the <code>-max</code> option is specified, the drive resistance applies to both the early path delay and the late path delay.
<code>port_list</code>	Specifies a list of input or bidirectional ports on which the drive resistance is set.
<code>resistance_value</code>	Specifies the value of the drive resistance. This value must be greater than or equal to zero. The drive resistance is in units found in the target technology library.
<code>-rise</code>	Specifies that the drive resistance is to be used for the rising transitions at the specified ports. By default, if neither the <code>-rise</code> option nor the <code>-fall</code> option are specified, the drive resistance applies to both the rising transition and the falling transition.

Examples

The following command sets a drive resistance of 2.0 for the early path, the late path, the rising and the falling transitions at the in1 and in2 ports and shows how drive resistances are reported using the [report_ports](#) command as shown in the example below:

```
% set_drive 2.0 [get_ports {in1 in2}]
% report_ports -type drive_resistance -pin [get_ports {in1 in2}]
```

The following report is displayed:

Early Late

Pin	Dir	Assertion	Clock	Rise	Fall	Rise	Fall
Name			Name				

in1	IN	drive_resistance	*(D) (P)	2.000	2.000	2.000	2.000
in2	IN	drive_resistance	*(D) (P)	2.000	2.000	2.000	2.000

Related Commands

- `all_inputs`
- `get_ports`
- `report_ports`
- `reset_drive`
- `set_driving_cell`
- `set_input_transition`
- `set_load`

set_driving_cell

```
set_driving_cell
[-help]
[-library library_name]
-lib_cell cell_name
[-pin pin_name]
[-from_pin from_pin_name]
[-rise]
[-fall]
[-min]
[-max]
[-multiply_by factor]
[-no_design_rule]
[-input_transition_rise rise_slew_value]
[-input_transition_fall fall_slew_value]
port_list
```

Models the drive capability of an external driver connected to the input port. Use this command only for timing analysis. The command does not affect the electrical properties of the design, which means that the capacitance of the output pin of the driving cell does not add to the total capacitance of the input port.

To model the drive capability of the external driver, the timing analyzer computes an offset to the arrival time of an input, and changes the slew time used to compute the delay for the next cell. The timing analyzer computes the offset by subtracting the zero load delay from the total delay of the cell. The `set_driving_cell` command uses the offset value to identify a cell from the library to drive the specified input port(s).

This command overrides the `set_drive` command per port or per net, if it is the last command applied. Also, if the `set_driving_cell` command is set for a port, the `set_input_transition` command is ignored for that port. If you do not define drive using the `set_driving_cell` command, the software uses the default value defined in the configuration file. The default value is 0.1ps.

Parameters

-help	Prints out the command usage.
-fall	Annotates only the falling drive assertion. The default is to annotate both rise and fall delays.
-from_pin <i>from_pin_name</i>	Specifies the name of the input port of the driver cell that has an arc to the output port of the driver cell, which is connected to the input port of the design. This option is required only if there are multiple timing arcs in the driver cell from its input ports to its output ports, and the arcs from those pins have different drive characteristics. If you do not specify the <code>-from_pin</code> option, the default is to use the first timing arc found on the library cell.

- input_transition_fall fall_slew_value	Specifies the slew value for the falling signal at the input of the drive cell.
- input_transition_rise rise_slew_value	Specifies the slew value for the rising signal at the input of the drive cell.
-lib_cell cell_name	Specifies the name of the driver cell. This parameter supports wildcard patterns and collections.
-library library_name	Specifies the name of the library that contains the driving cell.
-max	Annotates only the late drive assertion. The default is to annotate both early and late.
-min	Annotates only the early drive assertion. The default is to annotate both early and late.
-multiply_by factor	Specifies a multiplier for the drive.
-no_design_rule	Does not allow transfer of design rules from the driving cell to the port. If you do not specify this option, the design rules (such as max_capacitance) of the library pin are applied to the port.
-pin pin_name	Specifies the name of the output port of the driver cell that drives the signal at the input port of the design. This option is required if there are multiple output ports on the driver. If there is only one output port on the driver cell, then using this option is not necessary.
port_list	Specifies the list of ports driven by the driver cell.
-rise	Specifies only the rising drive assertion is annotated. The default is to annotate both rise and fall delays.

Examples

The following command sets 'AND' cell (arc B -> Y) as the driving cell for the input port IN1:

```
% set_driving_cell -lib_cell AND -from_pin B -pin Y IN1
```

Related Commands

- `reset_driving_cell`
- `all_inputs`
- `report_ports`
- `set_drive`
- `set_input_transition`

set_false_path

```
set_false_path
[-help]
[-hold | -setup]
[-rise]
[-fall]
[-comment string ]
[{-from | -rise_from | -fall_from} from_list ]
[{-through | -rise_through | -fall_through} through_list ]
[{-to | -rise_to | -fall_to} to_list ]
```

Identifies false paths in a design, and breaks or disables specific instance timing arcs in a design.

To break combinational loops, use the `set_disable_timing` command instead of the `set_false_path` command.

The `set_false_path` command differs from the `set_disable_timing` command as follows:

- `set_false_path` command selectively disables the arrival time information depending on the precedence rules.
- `set_false_path` command does not affect constant values.
- `set_disable_timing` command physically snips a timing arc such that neither arrival times nor constant values propagate through it.
- `set_disable_timing` command is applied in all cases to valid timing arcs.

The `set_false_path -from fpin` command disables all timing paths whose source pin is an `fpin`. Similarly, the `set_false_path -to tpin` command disables all timing paths whose sink pin is a `tpin`. Using the `set_false_path -from fpin -to tpin` command disables all timing paths whose source is `fpin` and sink is `tpin`.

Specify a group of pins by enclosing the group with curly braces (`{}`) within one `set_false_path` command.

✔ To set a false path for any path beginning at `p1` or `p2` and terminating at `t1` or `t2`, group the pins as follows:

```
set_false_path -from {p1 p2} -to {t1 t2}
```

⚠ This method of pin grouping reduces the number of exceptions. Having large numbers of exceptions adversely impacts the run time of the timing analysis commands.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-comment</code> <i>string</i>	Allows you to insert comments. The comments are read but are ignored with a one-time warning message. They are not used during timing analysis reporting nor are they written out during saving of constraints.

<pre>{-from - rise_from - fall_from} from_list</pre>	Specifies a list of timing paths start points. The <code>from_list</code> argument can contain either object IDs or hierarchical names relative to the current module. The pins can be on intermediate hierarchical boundaries.. The <code>from_list</code> argument can be a collection. The valid start points are: • Input port • Bidirectional port • Clock pin of sequential cell Clock pin of sequential cell is one of the following: • Structural clock pin that is part of latch or flip-flop. • Inferred clock pin in cells like macro cells. A pin is an inferred clock pin if it is associated with one of the following: • Reference end of setup check • Reference end of hold check • Reference end of removal check • Reference end of recovery check • Minimum period check • Minimum pulse width check • Pin associated with <code>set_input_delay</code> Specify one of the following:
	<code>-from:</code> Applies the false path constraint to all paths originating from the <code>from_list</code> list. If a clock is specified as the <code>-from</code> object, all primary inputs and registers clocked by that clock are used as start points. By default, the <code>-from</code> option applies the constraint to both the rising and the falling edges.
	• <code>-rise_from:</code> Applies the false path constraint at the rising edge of the signal on the from pins.
	• <code>-fall_from:</code> Applies the false path constraint at the falling edge of the signal on the from pins.
<pre>-hold/-setup</pre>	Sets false paths for either setup or hold analysis.  If you do not specify <code>-setup</code> or <code>-hold</code>, the software sets false paths for both setup and hold analysis.

<code>-rise -fall</code>	Applies the assertion at the rising edge of the signal on the path endpoints. Applies the assertion at the falling edge of the signal on the path endpoints.
<code>{-through -rise_through -fall_through} through_list</code>	Specifies a list of timing paths through points. The <code>through_list</code> argument can contain either object IDs or hierarchical names relative to the current module. The pins can be on intermediate hierarchical boundaries. The <code>through_list</code> argument can be a collection. Specify one of the following:
	• -through: When you use this parameter without using the <code>-from</code> or <code>-to</code> options, the false path constraint is set for all paths that go through the specified pins. When you use it with both the <code>-from</code> and <code>-to</code> options, any path starting at any pin on the <code>-from</code> list and going through any point on the <code>-through</code> pin list and going to any point on the <code>-to</code> list is affected by the constraint. By default, the <code>-through</code> option applies the constraint to both the rising and the falling edges.
	• -rise_through: Applies the assertion at the rising edge of the signal on the through pins.
	• -fall_through: Applies the assertion at the falling edge of the signal on the through pins.

<pre>{-to - rise_to - fall_to} to_list</pre>	Specifies a list of timing paths end points. The <code>to_list</code> argument can contain either object IDs or hierarchical names relative to the current module. The pins can be on intermediate hierarchical boundaries. The <code>to_list</code> argument can be a collection. The valid end points are: • Output port • Bidirectional port • Data pin of sequential cell Data pin of sequential cell is one of the following: • Structural data pin that is part of latch or flip-flop. • Inferred data pin in cells like macro cells. A pin is an inferred data pin if it is associated with one of the following: • Signal end of setup check • Signal end of hold check • Signal end of non-sequential setup check • Signal end of non-sequential hold check • Signal end of removal check • Signal end of recovery check • Pin associated with <code>set_external_delay</code> • Data pin of clock gating cell Select from one of the following:
	<code>-to:</code> Applies the false path constraint to all paths ending in the <code>-to</code> pin list. If a clock is specified as the <code>-to</code> object, all primary outputs and registers clocked by that clock are used as endpoint. By default, the <code>-to</code> option applies the constraint to both the rising and the falling edges.
	<code>-rise_to:</code> Applies the false path constraint at the rising edge of the signal on the to pins.
	<code>-fall_to:</code> Applies the false path constraint at the falling edge of the signal on the to pins.

Examples

- The following command disables hold checking from `ff1` to `ff2`:

```
% set_false_path -hold -from ff1 -to ff2
```

- The following command sets all timing paths from `ff1/CK` to `D` input of `ff2` that pass through `u1/Y` and one or more of `u2/Y` and `u3/Y` as false paths:

```
% set_false_path -from ff1/CK -through {u1/Y} -through {u2/Y u3/Y} -to ff2/D
```

- The following command sets all timing paths from the rise of `ff1/CK` to `ff2/D` through one or more of `u1/Y` and `u4/Y` as false paths:

```
% set_false_path -rise_from ff1/CK -through {u1/Y u4/Y} -to ff2/D
```

Related Commands

- `set_disable_timing`
- `set_max_delay`
- `set_min_delay`
- `set_multicycle_path`

set_fanout_load

```
set_fanout_load  
[-help]  
value  
port_list
```

Specifies the fanout load on a set of ports.

The library instance pins can have fanout loads defined in the technology library. The fanout does not have specific units so it is assumed that a fanout as used by the constraints is consistent with the fanout defined in the technology libraries. If a pin or port has no fanout attribute, either from the constraints or from the technology library, then the fanout load will be zero. The fanout load is used only for design rule checking and does not impact the timing results.

Parameters

<code>-help</code>	Prints out the command usage.
<code>value</code>	Specifies the fanout load value for the port or ports.
<code>port_list</code>	Specifies the list of ports for which the fanout load is applied.

Examples

- The following command sets the fanout load of port `out` to 2:

```
Litmus > set_fanout_load 2 out
```
- The following command sets the fanout load of ports `in1` and `in2` to 1:

```
Litmus > set_fanout_load 1 {in1 in2}
```
- The following command sets the fanout load of element 3 of port `out` to 12:

```
Litmus > set_fanout_load 12 { out[3] }
```
- The following command sets the fanout load of all the ports to 2:

```
Litmus > set_fanout_load 2 [get_ports *]
```

Related Commands

- [report_nets](#)
- [report_ports](#)
- [set_max_fanout](#)
- [report_constraint](#)

set_ideal_latency

```
set_ideal_latency  
[-help]  
[-min]  
[-max]  
[-rise]  
[-fall]  
value  
pin_or_port_list
```

Specifies the ideal latency to use for input or output leaf-cell pins and top-level ports that are part of an ideal network. These set a point delay on the specified object.

 You should specify `set_ideal_latency` with `set_ideal_network` to view the ideal latency result.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-fall</code>	Specifies the ideal latency for the falling edge.
<code>value</code>	Specifies the ideal latency value.
<code>-max</code>	Specifies the ideal latency for the maximum corner.
<code>-min</code>	Specifies the ideal latency for the minimum corner.
<code>pin_or_port_list</code>	Specifies a list of objects - instance pins or ports.
<code>-rise</code>	Specifies the ideal latency for the rising edge.

Examples

The following command adds an ideal latency of 1.5ns to the input pin A of cell FE_OFC413_scan_se:

```
% set_ideal_latency 1.5 FE_OFC413_scan_se/A
```

Please note that the above command will only be valid if the pin is part of an ideal network.

Now, this ideal latency value can be observed in the output of the following `report_timing` command:

```
% report_timing -though FE_OFC413_scan_se/A
```

```
Path 1: MET Setup Check with Pin RESULTS_CONV_INST/r697_reg[14]/CK  
Endpoint: RESULTS_CONV_INST/r697_reg[14]/SE (v) checked with trailing edge of 'm_rcc_clk'  
Beginpoint: scan_se (v) triggered by leading edge of 'm_ram_clk'  
Path Groups: {m_rcc_clk}
```

Litmus Text Command Reference

Timing Constraint Commands--set_ideal_latency

Analysis View: scan_fast_RCMIN

Other End Arrival Time	20.679
- Setup	1.123
+ Phase Shift	0.000
+ Cycle Adjustment	40.000
- Uncertainty	0.400
= Required Time	59.156
- Arrival Time	8.000
= Slack Time	51.156
Clock Rise Edge	0.000
+ Input Delay	4.000
+ Ideal Latency	1.500
= Beginpoint Arrival Time	5.500

Pin	Cell	Load	Slew	Delay	Arc	Annotation
scan_se		0.000	0.000			Ideal
FE_OFC413_scan_se/Z	BUF_X32	0.000	0.000	0.000		Ideal
FE_OFC247_scan_se/A	BUF_X16	0.000	0.000	0.000		Ideal
FE_OFC247_scan_se/Z	BUF_X16	0.000	0.000	0.000		Ideal
FE_OFC248_scan_se/A	BUF_X16	0.000	0.000	0.000		Ideal
FE_OFC248_scan_se/Z	BUF_X16	0.000	0.000	0.000		Ideal
FE_OFC249_scan_se/A	BUF_X8	0.000	0.000	0.000		Ideal

Related Commands

- [reset_ideal_latency](#)
- [report_timing](#)
- [set_ideal_network](#)

set_ideal_network

```
set_ideal_network  
[-help]  
obj_list  
[-no_propagate]
```

Identifies driver pins or ports as sources of an ideal network. By default, all pins and nets in the transitive fanout of the specified port or driver pin are marked as ideal. Ideal network marking normally propagates through combinational cells and stops at sequential cells.

This command is not view/mode-based - it applies to all the active views.

The `set_ideal_network` command sets 0 slew for bi-directional pins. This ensures that unnecessary DRV violations do not occur.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-no_propagate</code>	Prevents propagation through transitive fanout.
<code>obj_list</code>	Specifies a list of objects - instance pins, ports, or nets.

Examples

The following command marks the fanout of the input port `scan_se` as an ideal network:

```
% set_ideal_network scan_se
```

This can be observed in the output of below mentioned `report_timing` command where the whole fanout of '`scan_se`' is considered as ideal:

```
% report_timing -from scan_se
```

```
Path 1: MET Setup Check with Pin RESULTS_CONV_INST/r697_reg[12]/CK  
Endpoint: RESULTS_CONV_INST/r697_reg[12]/SE (v) checked with trailing edge of 'm_rcc_clk'  
Beginpoint: scan_se (v) triggered by leading edge of 'm_ram_clk'  
Path Groups: {m_rcc_clk}  
Analysis View: scan_fast_RCMIN  
Other End Arrival Time 20.700  
- Setup 1.122  
+ Phase Shift 0.000  
+ Cycle Adjustment 40.000  
- Uncertainty 0.400  
= Required Time 59.178  
- Arrival Time 4.000  
= Slack Time 55.178  
Clock Rise Edge 0.000
```

Litmus Text Command Reference

Timing Constraint Commands--set_ideal_network

```
+ Input Delay 4.000
= Beginpoint Arrival Time 4.000
```

Pin	Cell	Load	Slew	Delay	Arc Annotation
scan_se ->		0.000	0.000		Ideal
FE_OFC413_scan_se/Z	BUF_X32	0.000	0.000	0.000	Ideal
FE_OFC247_scan_se/Z	BUF_X16	0.000	0.000	0.000	Ideal
FE_OFC248_scan_se/Z	BUF_X16	0.000	0.000	0.000	Ideal
FE_OFC249_scan_se/Z	BUF_X8	0.000	0.000	0.000	Ideal
FE_OFC250_scan_se/Z	BUF_X16	0.000	0.000	0.000	Ideal
FE_OFC251_scan_se/Z	BUF_X4	0.000	0.000	0.000	Ideal
RES_CONV_IN/r697/SE	SDFF_X2	0.000	0.000	0.000	Ideal

Related Commands

- [reset_ideal_network](#)

set_ideal_transition

```
set_ideal_transition  
[-help]  
[-min]  
[-max]  
[-rise]  
[-fall]  
value  
pin_or_port_list
```

Specifies the ideal transition to use for input or output leaf-cell pins and top-level ports that are part of an ideal network. These set a point delay on the specified object.

 You should specify `set_ideal_transition` with `set_ideal_network` to view ideal transition result.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-fall</code>	Specifies the ideal transition for the falling edge.
<code>-max</code>	Specifies the ideal transition for the maximum corner.
<code>-min</code>	Specifies the ideal transition for the minimum corner.
<code>pin_or_port_list</code>	Specifies a list of objects - instance, pins or ports.
<code>-rise</code>	Specifies the ideal transition for the rising edge.
<code>value</code>	Specifies the ideal transition time.

Examples

Here are some examples.

The following command specifies the ideal transition of 0.5ns on pin `scan_se`:

```
% set_ideal_transition 0.5 scan_se
```

Please note that this ideal transition would only be applicable if the pin is part of an ideal network, otherwise this value would be ignored.

This transition will be propagated through the ideal network as shown in the output of the following `report_timing` command:

```
% report_timing -from scan_se
```

Litmus Text Command Reference

Timing Constraint Commands--set_ideal_transition

Path 1: MET Setup Check with Pin RESULTS_CONV_INST/r697_reg[14]/CK
Endpoint: RESULTS_CONV_INST/r697_reg[14]/SE (v) checked with trailing edge of 'm_rcc_clk'
Beginpoint: scan_se (v) triggered by leading edge of 'm_ram_clk'
Path Groups: {m_rcc_clk}
Analysis View: scan_fast_RCMIN
Other End Arrival Time 20.699
- Setup 1.277
+ Phase Shift 0.000
+ Cycle Adjustment 40.000
- Uncertainty 0.400
= Required Time 59.022
- Arrival Time 4.000
= Slack Time 55.022
Clock Rise Edge 0.000
+ Input Delay 4.000
= Beginpoint Arrival Time 4.000

Pin	Cell	Load	Slew	Delay	Arc	Annotation
scan_se ->		0.000	0.500			Ideal
FE_OFC413_scan_se/Z	BUF_X32	0.000	0.500	0.000		Ideal
FE_OFC247_scan_se/Z	BUF_X16	0.000	0.500	0.000		Ideal
FE_OFC248_scan_se/Z	BUF_X16	0.000	0.500	0.000		Ideal
FE_OFC249_scan_se/Z	BUF_X8	0.000	0.500	0.000		Ideal
FE_OFC250_scan_se/Z	BUF_X16	0.000	0.500	0.000		Ideal
FE_OFC251_scan_se/Z	BUF_X4	0.000	0.500	0.000		Ideal
RESULT_CONV_INST/SE	SDFF_X2	0.000	0.500	0.000		Ideal

Related Commands

- [reset_ideal_transition](#)

set_input_delay

```
set_input_delay
[-help]
[-clock clock_name]
[-clock_fall]
[-rise] [-fall]
[-max] [-min]
[-add_delay]
[-network_latency_included]
[-source_latency_included]
[-reference_pin pin_name]
[-level_sensitive]
value
pin_or_port_list
```

Defines the arrival time relative to a clock edge on input ports or internal input pins. This input path delay models the delay from an external register to an input port of the module.

If the input delay is not specified on an input port, it is assumed to be zero.

 If the `set_input_delay` constraint is added incrementally in SI mode, an explicit `update_timing -full` command is required (before `report_timing` or `report_constraint`) for SI full accuracy.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-add_delay</code>	Adds delay information to the existing input delay. The additional delay information can be for multiple paths relative to different clocks or clock edges leading to an input port or internal input pins. If you do not use the <code>-add_delay</code> option, the software uses the last <code>set_input_delay</code> related to a pin when there are multiple <code>set_input_delay</code> statements for the same pin.
<code>-clock <i>clock_name</i></code>	Specifies that the delay is relative to the rising edge of the given clock. If you do not specify the <code>-clock</code> parameter, the delay is relative to time zero. To specify a delay relative to the falling edge of the clock, use the <code>-clock_fall</code> option with the <code>-clock</code> option. When you specify multiple clocks, an error occurs and the constraint is ignored.
<code>-clock_fall</code>	Specifies that the delay is relative to the falling edge of the clock. You must specify the <code>-clock</code> parameter with this parameter. <i>Default:</i> Delay is relative to the rising edge. You cannot specify the <code>-clock_fall</code> parameter without the <code>-clock</code> parameter.
<code><i>value</i></code>	Specifies the value for the path delay.

<code>-fall</code>	Specifies that the input delay refers to the falling edge of the signal at specified ports or pins. If you do not specify the <code>-fall</code> option, the input delay applies to both the edges.
<code>-level_sensitive</code>	Models input delay that is launched by a positive level sensitive latch. Use the <code>-clock</code> parameter to model a positive level sensitive latch, or the <code>-clock_fall</code> parameter to model a negative level sensitive latch. You cannot specify the <code>-level_sensitive</code> parameter without the <code>-clock</code> parameter.
<code>-max</code>	Specifies that the delay refers to the setup analysis. If you do not specify the <code>-max</code> parameter, the input delay applies to both setup and hold analysis. Note: If for a specific clock the maximum delay is less than the minimum delay, then during analysis the software makes the maximum delay equal to the minimum delay.
<code>-min</code>	Specifies that the delay refers to the hold analysis. If you do not specify the <code>-min</code> parameter, the input delay applies to both setup and hold analysis.
<code>-network_latency_included</code>	Specifies that the clock network latency should not be added to the input delay value. If you do not specify this option, the clock network latency of the related clock is added to the input delay value. This parameter has no effect if the specified clock is in propagated mode.
<code>pin_or_port_list</code>	Specifies a list of input port or pin names. The <code>pin_or_port_list</code> argument can be a collection.
<code>-reference_pin pin_name</code>	Adds source and network latency of the specified reference pin to the input delay value in propagated mode. A reference pin is a leaf pin in the fanout cone of the clock source of the clock that you specify using the <code>-clock</code> parameter. You cannot use the <code>-reference_pin</code> parameter to specify multiple pins. If specified, the command issues a warning and accepts the first pin. You cannot use the <code>-reference_pin</code> parameter with the <code>-network_latency_included</code> or <code>-source_latency_included</code> parameters. You cannot use the <code>-reference_pin</code> parameter to specify hierarchical pins.
<code>-rise</code>	Specifies that input delay refers to the rising edge of the signal at the specified ports or pins. If you do not specify the <code>-rise</code> option, the input delay applies to the both edges.

<code>-source_latency_included</code>	Specifies that the clock source latency should not be added to the input delay value. With different delay values for <code>-rise/-fall</code> and <code>-min/-max</code>, only the last specified combination of <code>-network_latency_included</code> and <code>-source_latency_included</code> parameters is honored. The specified combination can be one of the following: a) Either <code>-network_latency_included</code> or <code>-source_latency_included</code> b) Both of these c) None of these
---------------------------------------	---

Examples

- The following command sets an input delay of 1.0 for all inputs in the current design relative to the rising edge of the CLK1 clock:

```
% set_input_delay -clock CLK1 1.0 [all_inputs]
```

- The following command sets an input delay of 2.0 for the input IN1 port relative to the falling edge of the CLK1 clock:

```
% set_input_delay -clock CLK1 2.0 -clock_fall { IN1 }
```

- The following example shows an input IN1 port that is constrained by both the rising edge and the falling edge of the same CLK1 clock. The `-add_delay` option prevents the second `set_input_delay` statement from overwriting the first `set_input_delay`:

```
% set_input_delay -clock CLK1 1.0 { IN1 }  
% set_input_delay -clock CLK1 -clock_fall -add_delay 2.0 { IN1 }
```

- When multiple `-min` and `-max` values are specified relative to the same clock using the `-add_delay` option at the same port or pin, the worst case value for each case is used. For the `-min` case the smallest value is retained, and for the `-max` case the largest value is retained. For example, the input delay value of 1.0 is stored for the shortest path and the value of 4.0 is stored for the longest path in the following example:

```
% set_input_delay -min -clock CLK1 1.0 { IN1 }  
% set_input_delay -min -clock CLK1 2.0 -add_delay { IN1 }  
% set_input_delay -max -clock CLK1 3.0 -add_delay { IN1 }  
% set_input_delay -max -clock CLK1 4.0 -add_delay { IN1 }
```

Related Commands

- `all_inputs`
- `create_clock`
- `report_ports`
- `set_load`
- `set_output_delay`

set_input_transition

```
set_input_transition  
[-help]  
[-min]  
[-max]  
[-rise]  
[-fall]  
value  
port_list
```

Specifies the slew time for a port on the top level module. This command is useful only in specifying a hard ramp signal as if the port were driven by a very powerful driver. The command does not change the arrival time of an input, but changes the slew time used to compute the delay of the cell on the sink of a net. If you do not define transition using the `set_input_transition` command, the software uses the default input port transition defined in the configuration file. The default value is 0.1ps. You can also set the `delaycal_input_transition_delay` attribute to override the default value.

The `set_input_transition` command specification for clock signals does not impact setup/hold timing calculations.

Note: The `set_input_transition` command does not support internal pins. For specifying transition on internal pins use the `set_annotated_transition` command.

Note: The most recent command settings (of `set_driving_cell`, `set_input_transition`, or `set_drive`) that are applied to a port will be considered - the software will override the previous commands.

The `set_input_threshold` command should be issued after the `set_lib_binding_by_voltage` command.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-fall</code> <code>-rise</code>	Specifies that the slew time should be applied to the rising edge or the falling edge of the signal (data or clock). If neither time is specified, the default is to use both.
<code>-min</code> <code>-max</code>	Indicates whether the slew times are with respect to the early (hold) or the late (setup) time checks of the data signal. If neither option is specified, the default is to use both.
<code>port_list</code>	Specifies the list of ports for which the slew time is specified. Note: You cannot use the <code>set_input_transition</code> command to specify transition on output ports. Note: When you use both <code>set_input_transition</code> and <code>set_annotated_transition</code> commands for specifying transition on input ports, the worst value is used as the transition value.
<code>value</code>	Specifies the transition time.

Examples

- The following command specifies the slew time of 0.5 on port `input3`.

```
set_input_transition 0.5 input3
```

Related Commands

- `report_ports`
- `set_driving_cell`
- `set_drive`
- `set_clock_transition`

set_load

```
set_load  
[-help]  
[-max] [-min]  
[-pin_load]  
[-wire_load]  
[-subtract_pin_load]  
capacitance_value  
port_or_net_list
```

Sets the specified capacitance on the defined ports (or nets) of the top cell.

The specified capacitance overrides the capacitance from the wire load models and SPEF.

You can use the [report_ports](#) command to view capacitance of asserted ports. The pin load appears in the report as `port_cap` in the assertion column and the wire load appears as `port_wire_cap` in the assertion column.

In the timing report (generated using the [report_timing](#) command), the load value for ports is displayed as a sum of the `set_load` value on a port and the existing wire capacitance obtained from the SPEF file, and pin capacitance obtained from the library files.

Parameters

-help	Prints out the command usage.
<i>capacitance_value</i>	Specifies the value for the capacitance. The capacitance is in units from the target technology library.
-min	Specifies the minimum capacitance. The minimum capacitance is used to compute best-case delays in BcWc analysis mode and early path delays in OCV analysis mode.
-max	Specifies the maximum capacitance. The maximum capacitance is used to compute worst-case delays in BcWc analysis mode and late path delays in OCV mode. It is also used to check the maximum capacitance design rule.
<i>port_or_net_list</i>	Specifies a list of ports or nets in the top cell.
-pin_load	Specifies the external pin capacitance for a port. You can use this parameter for ports only. For ports, the asserted capacitance is either the external pin capacitance when the <code>-pin_load</code> parameter is specified, or the external wire capacitance when the <code>-wire_load</code> parameter is specified, or the total external capacitance when neither option is specified.
- subtract_pin_load	This parameter can be used only if nets are specified in the object list. When this parameter is specified, the total net capacitance value will be the same as set by the <code>set_load capacitance_value object_list</code> command. When this parameter is not specified, the asserted load value will only change the wire capacitance value from the SPEF. The total net capacitance value will be the <code>set_load</code> capacitance value plus the pin capacitance from the library file.
-wire_load	Interprets the specified capacitance as the external wire capacitance of the port. Use this option only for ports. Use this option with the <i>port_net_list</i> argument if the list contains only ports.

Examples

Here are some examples.

- The following command sets an external minimum pin capacitance of 1.5 units on the *in1 in2* ports:

```
% set_load 1.5 -min -pin_load [get_ports {in1 in2}]
```

- The following command sets an external minimum wire capacitance of 1.2 units on the *in1 in2* ports:

```
% set_load 1.2 -min -wire_load [get_ports {in1 in2}]
```

- The following command sets an external maximum pin capacitance of 2.6 units on the *in1 in2* ports:

```
% set_load 2.6 -max -pin_load [get_ports {in1 in2}]
```

- The following command sets an external maximum wire capacitance of 3.0 units on the `in1 in2` ports:

```
% set_load 3.0 -max -wire_load [get_ports {in1 in2}]
```

- The following command reports the capacitances set on the ports, as shown in the example below:

```
% report_ports -pin [get_ports {in1}]
```

```
-----
Early Late
-----
Pin   Dir Assertion      Clock   Rise   Fall   Rise   Fall
Name                                Name
-----
in1   IN drive_resistance *(D) (P) 2.000  2.000  2.000  2.000
-----

-----
Pin   Dir   Assertion      Value
Name
-----
in1   IN    port_cap           ( 1.500 : : 2.600 )
in1   IN    port_wire_cap      ( 1.200 : : 3.000 )
-----
```

Related Commands

- [all_outputs](#)
- [report_ports](#)
- [report_timing](#)
- [set_drive](#)
- [set_resistance](#)

set_logic_one

```
set_logic_one
-help
pin_or_port_list
```

Sets ports or pins to logic 1 in the design.

Parameters

<code>-help</code>	Outputs a brief description that includes type and default information for each <code>set_logic_one</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man set_logic_one</code> .
<code>pin_or_port_list</code>	Specifies the list of the ports or pins that are to be set to logic 1.

Examples

The following command sets the specified port to logic 1:

```
% set_logic_one [get_ports {din[1]}]
```

Related Commands

- `get_ports`

set_logic_zero

set_logic_zero
-help
pin_or_port_list

Sets ports or pins to logic 0 in the design.

Parameters

-help	Outputs a brief description that includes type and default information for each <code>set_logic_zero</code> parameter. For a detailed description of the command and all of its parameters, use the man command: <code>man set_logic_zero</code> .
pin_or_port_list	Specifies the list of the ports or pins that are to be set to logic 0.

Examples

The following command sets the specified port to logic 0:

```
% set_logic_zero [get_ports {din[0]}]
```

Related Commands

- `get_ports`

set_max_capacitance

```
set_max_capacitance
[-help]
limit
obj_list
[-clock_path]
[-data_path]
[-fall]
[-override]
[-rise]
```

Sets the maximum capacitance limit on the specified ports of the top cell, the specified designs, or clock waveforms.

When set on a port, the total capacitance of the net connected to that port must be less than or equal to the specified limit. Maximum capacitance limits also come from library objects, which is the default maximum capacitance limit of libraries and the maximum capacitance limit on library cell pins. Pins in the design inherit the limits from libraries.

Note: The limit set on the top cell applies to its ports. Thus, if maximum capacitance limits are set on the top cell and its ports, the most constraining limit will apply to the ports.

When limits are set on a clock waveform, you can use `-clock_path`, `-data_path`, `-rise`, and `-fall` parameters to provide more specifications.

The `set_max_capacitance` constraint on clock objects will be honored only by the `report_constraint` command for DRV violations. The constraint will not be used in optimizations.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-clock_path</code>	Applies the max capacitance constraint only to pins in the clock network of the clocks defined in the object list.
<code>-data_path</code>	Applies the max capacitance constraint only to data path pins, where the path is launched by the clocks defined in the object list.
<code>-fall</code>	Applies the max capacitance constraint only to falling capacitance.
<code>-rise</code>	Applies the max capacitance constraint only to rising capacitance.
<code>limit</code>	Specifies the value for the minimum capacitance limit. The limit must be greater than or equal to zero. The limit is in capacitance units from the target technology library.
<code>obj_list</code>	Specifies a list of objects - ports, instance pins, library cell pins, designs, and/or clock waveform.
<code>-override</code>	Overrides the capacitance limit from other sources, such as, library or constraints. This parameter can be used to force a specific constraint value for a specified list of instance pins/port/library cell pins - this overrides the maximum/minimum values set prior to this setting.

Examples

Here are some examples.

- The following example sets a maximum capacitance limit of 1.2 units on all the ports of the top cell and a maximum capacitance limit of 0.5 units on the port named d_out:

```
Litmus > set_max_capacitance 1.2 [get_ports *]  
Litmus > set_max_capacitance 0.5 d_out
```
- The following example uses the `report_ports` command to show the maximum capacitance limit set on ports:

```
Litmus > set_max_capacitance 0.9 [get_ports *]  
Litmus > report_ports -type port_cap_limit -pin [get_ports *]
```

The following report is displayed:

```
-----  
Pin      Dir  Assertion      Value  
Name  
-----  
CLK2     IN   port_cap_limit  ( : 0.900 )  
CLK1     IN   port_cap_limit  ( : 0.900 )  
in2      IN   port_cap_limit  ( : 0.900 )  
in1      IN   port_cap_limit  ( : 0.900 )  
out2     OUT   port_cap_limit  ( : 0.900 )  
out1     OUT   port_cap_limit  ( : 0.900 )  
-----
```

Related Commands

- `reset_max_capacitance`
- `get_ports`
- `report_ports`
- `set_max_transition`
- `set_max_fanout`
- `report_constraint`

set_max_delay

```
set_max_delay
[-help]
[-combinational_from_to]
[{-from | -rise_from | -fall_from} from_list]
[{-through | -rise_through | -fall_through} through_list]
[{-to | -rise_to | -fall_to} to_list]
[-comment string]
[-rise]
[-fall]
value
```

Specifies a maximum delay for a timing path. The `set_max_delay` command takes precedence over other path exceptions specified for the same path, except for the `set_false_path` command.

Use either the `-from`, `-to`, or `-through` option with the `set_max_delay` command to identify the paths. Specifying both the `-from` and `-to` options identifies a path with specific starting and ending points.

When you specify a sequential cell name for starting point of the timing path, the valid path starts from the clk pin. This means the latency and delay from clk to output Q is also included in the delay. For path segmentation, you can specify the path to start from Q output of the register by adding `/Q` after the cell name. In this case, path from clk to Q is not considered in the calculation.

Use only one `-from`, `-rise_from`, or `-fall_from`, `-to`, `-rise_to`, or `-fall_to` parameter per command. You can use multiple `-through` parameters in the same command.

Note: If the `set_max_delay` constraint is added incrementally in SI mode, an explicit `update_timing -full` command is required (before `report_timing` or `report_constraint`) for SI full accuracy.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-combinational_from_to</code>	Defines a path defined by <code>-from/-to</code> pins as combinational.
<code>-comment string</code>	Allows you to insert comments. The comments are read but are ignored with a one-time warning message. They are not used during timing analysis reporting nor are they written out during saving of constraints.
<code>-fall</code>	Apply the assertion at the falling edge of the signal on the path endpoints.

<pre>-from -rise_from - fall_from from_list</pre>	Specifies a list of timing paths start points. The <i>from_list</i> argument can contain either object IDs or hierarchical names relative to the current module or clock names. The pins can be on intermediate hierarchical boundaries. The <i>from_list</i> argument can be a collection. The valid start points are: • Input port • Bidirectional port • Clock pin of sequential cell Clock pin of sequential cell is one of the following: • Structural clock pin that is part of latch or flip-flop. • Inferred clock pin in cells like macro cells. A pin is an inferred clock pin if it is associated with one of the following: • Reference end of setup check • Reference end of hold check • Reference end of removal check • Reference end of recovery check • Minimum period check • Minimum pulse width check • Pin associated with <code>set_input_delay</code> Specify one of the following:
	-from: Applies the maximum delay constraint to all paths originating from the <i>from_list</i> list. If a clock is specified as the <i>-from</i> object, all primary inputs and registers clocked by that clock are used as start points. By default, the <i>-from</i> option applies the constraint to both the rising and the falling edges.
	-rise_from: Applies the maximum delay constraint at the rising edge of the signal on the from pins.
	-fall_from: Applies the maximum delay constraint at the falling edge of the signal on the from pins.
<pre>-rise</pre>	Applies the assertion at the rising edge of the signal on the path endpoints.

<code>-through - rise_through - fall_through through_list</code>	Specifies a list of timing paths through points. The <i>through_list</i> argument can contain either object IDs or hierarchical names relative to the current module or clock names. The pins can be on intermediate hierarchical boundaries. The <i>through_list</i> argument can be a collection. Specify one of the following:
	<code>-through:</code> When you use this parameter without using the <code>-from</code> or <code>-to</code> options, all paths that go through the specified pins have the specified delay. When you use it with both the <code>-from</code> and <code>-to</code> options, any path starting at any pin on the <code>-from</code> list <i>and</i> going through any point on the <code>-through</code> pin list <i>and</i> going to any point on the <code>-to</code> list is affected by the constraint. By default, the <code>-through</code> option applies the constraint to both the rising and the falling edges.
	<code>-rise_through:</code> Applies the assertion at the rising edge of the signal on the through pins.
	<code>-fall_through:</code> Applies the assertion at the falling edge of the signal on the through pins.

<pre>-to -rise_to - fall_to to_list</pre>	Specifies a list of timing paths end points. The <i>to_list</i> argument can contain either object IDs or hierarchical names relative to the current module or clock names. The pins can be on intermediate hierarchical boundaries. The <i>to_list</i> argument can be a collection. The valid end points are: • Output port • Bidirectional port • Data pin of sequential cell Data pin of sequential cell is one of the following: • Structural data pin that is part of latch or flip-flop. • Inferred data pin in cells like macro cells. A pin is an inferred data pin if it is associated with one of the following: • Signal end of setup check • Signal end of hold check • Signal end of non-sequential setup check • Signal end of non-sequential hold check • Signal end of removal check • Signal end of recovery check • Pin associated with <i>set_external_delay</i> • Data pin of clock gating cell Select from one of the following:
	-to: Applies the maximum delay constraint to all paths ending in the <i>-to</i> pin list. If a clock is specified as the <i>-to</i> object, all primary outputs and registers clocked by that clock are used as endpoint. By default, the <i>-to</i> option applies the constraint to both the rising and the falling edges.
	-rise_to: Applies the maximum delay constraint at the rising edge of the signal on the <i>to</i> pins.
	-fall_to: Applies the maximum delay constraint at the falling edge of the signal on the <i>to</i> pins.
<pre>value</pre>	Specifies the maximum delay value.

Examples

The following command specifies that all paths from *in1* to *out1* must have delays less than 10 units:


```
% set_max_delay 10 -from in1 -to out1
```

Related Commands

- `create_clock`
- `set_false_path`
- `set_input_delay`
- `set_min_delay`
- `set_multicycle_path`
- `set_output_delay`

set_max_fanout

```
set_max_fanout
[-help]
limit
obj_list
[-override]
```

Sets the maximum fanout load limit constraint on the specified instance pins, library cell pins, ports of the top cell, clock waveforms, and/or specified modules. When pins of cell instances have their own maximum constraint defined in the library, the software will pick the more aggressive value of the two.

When set on a port, the fanout load of the net connected to the port must be less than or equal to the specified limit value to meet the constraint. The fanout load of a net is the sum of the fanout loads of the pins driven by the net. When set on a module, the limit applies to all the nets of that module. Libraries and library cell pins may also have the maximum fanout load limit. In this case the most constraining limit is applied to the nets.

Use the `report_ports -type fanout_load_limit` command to view the maximum fanout load limits set on ports of the top cell.

Fanout load limit can be asserted on input ports, output pins, bidirectional pins, or ports and library pins. You cannot apply the constraint on input pins and output ports.

Parameters

<code>-help</code>	Prints out the command usage.
<code>limit</code>	Specifies the value of the maximum fanout load limit constraint. This is a float number and it must be greater than or equal to zero.
<code>obj_list</code>	Specifies a list of objects - input ports, instance output pins, designs, or library cell output pins.
<code>-override</code>	Overrides the maximum fanout limit from other sources, such as, library or constraints. This parameter can be used to force a specific constraint value for a specified list of instance pins/port/library cell pins - this overrides the maximum/minimum values set prior to this setting.

Examples

- The following example sets a maximum fanout load limit of 5.0 units on all the ports of the top cell:

```
Litmus > set_max_fanout 5.0 [get_ports *]
```

- The following example shows a report of the maximum fanout load limit set on ports using the `report_ports` command:

```
Litmus > set_max_fanout 5.0 [get_ports *]
```

```
Litmus > report_ports -type fanout_load_limit -pin [get_ports *]
```

The following report is displayed:

Pin Name	Dir	Assertion	Value
CLK2	IN	fanout_load_limit	5.000
CLK1	IN	fanout_load_limit	5.000
in2	IN	fanout_load_limit	5.000
in1	IN	fanout_load_limit	5.000

- The following example sets a maximum fanout load limit of 5.0 units on all the ports of the top cell:

```
Litmus > set_max_fanout 5.0 [get_ports *]
```

- The following example shows a report of the maximum fanout load limit set on ports using the `report_ports` command, as shown in the example below:

```
Litmus > set_max_fanout 5.0 [get_ports *] report_ports -type fanout_load_limit -pin [get_ports *]
```

Related Commands

- `reset_max_fanout`
- `get_ports`
- `report_ports`
- `set_fanout_load`
- `set_max_capacitance`
- `set_max_transition`
- `report_constraint`

set_max_time_borrow

```
set_max_time_borrow  
[-help]  
value  
object_list
```

Specifies the maximum time that can be borrowed by one stage from the next logic stage following a latch to meet timing constraints. Time borrowing is sometimes called cycle stealing. Time borrowing is performed automatically when performing timing analysis.

Use the `set_max_time_borrow` assertion on pins or waveforms. If the assertion is placed on a clock waveform, all latches triggered by this clock signal get the specified maximum time borrow limit. If both the clock waveform arriving at the clock pin of a latch and the clock pin itself have time borrow limit constraints, the maximum borrow limit on the latch is decided by the assertion on the latch clock pin.

Parameters

<code>-help</code>	Prints out the command usage.
<code>value</code>	Specifies the maximum amount of time that can be borrowed. The minimum value allowed is 0, which disables time borrowing. The maximum allowed is equal to the pulse width minus the setup time: $\text{Max} = \text{pulse width} - \text{setup}$
<code>object_list</code>	Specifies the list of instance pins, instances, or clocks on which the borrow value is applied. The <code>object_list</code> argument can be a collection.

Examples

Here are some examples.

```
% set_max_time_borrow 2 [get_cells {i1, i2, i3}]
```

```
% set_max_time_borrow 1.7 I4/g
```

```
% set_max_time_borrow 2.5 [get_clocks CLK1]
```

Related Commands

- `get_clocks`
- `get_cells`
- `get_pins`
- `reset_max_time_borrow`

set_max_transition

```
set_max_transition  
[-help]  
[-clock_path]  
[-data_path]  
[-rise]  
[-fall]  
[-override]  
transition_limit  
object_list
```

Sets the maximum slew time limit (transition) on the specified ports of the top cell, and/or clock waveforms.

When set on a port, the slew time at the port must be less than or equal to the specified limit. Maximum transition limits on pins can come from library objects, which are the default maximum transition of the libraries or maximum transition on library cell pins. In this case, the most constraining limit, which is the smallest, will apply on pins and ports.

Use the `report_ports -type slew_limit` command to view the maximum transition limits set on ports of the top cell.

When limits are set on a clock waveform, you can use `-clock_path`, `-data_path`, `-rise`, and `-fall` parameters to provide more specifications.

Parameters

<code>-help</code>	Outputs a brief description that includes type and default information for each <code>set_max_transition</code> parameter. For a detailed description of the command and all of its parameters, use the man command: <code>man set_max_transition</code> .
<code>-clock_path</code>	Applies the max transition constraint only to pins in the clock network of the clocks defined in the object list.
<code>-data_path</code>	Applies the max transition constraint only to data path pins, where the path is launched by the clocks defined in the object list.
<code>-fall</code>	Applies the max transition constraint only to falling transitions.
<code>object_list</code>	Specifies the list of objects - ports, instance pins, library cell pins, clock waveforms, or current design.
<code>-rise</code>	Applies the max transition constraint only to rising transitions.
<code>transition_limit</code>	Specifies the value of the maximum slew time limit (maximum transition), which must be greater than or equal to zero. The maximum slew time limit is in time units in the current session.

<code>-override</code>	Overrides the transition limit from other sources, such as, library or constraints. This parameter can be used to force a specific constraint value for a specified list of instance pins/port/library cell pins - this overrides the maximum/minimum values set prior to this setting.
------------------------	---

Examples

Here are some examples.

- The following command sets a maximum transition limit of 0.9 units on all the ports of the top cell:

```
Litmus > set_max_transition 0.9 [get_ports *]
```

- The following example shows a report of the maximum transition limit set on ports using the `report_ports` command, as shown in the example below:

```
Litmus > set_max_transition 0.9 [get_ports *]
```

```
Litmus > report_ports -type slew_limit -pin [get_ports *]
```

The following report is displayed:

Pin Name	Dir	Assertion	Value
in1	IN	slew_limit	(: 0.9000)
in2	IN	slew_limit	(: 0.9000)
out1	OUT	slew_limit	(: 0.9000)
out2	OUT	slew_limit	(: 0.9000)
out3	OUT	slew_limit	(: 0.9000)
out4	OUT	slew_limit	(: 0.9000)

- The following command sets the maximum transition limit of 0.001 units on pins in the `all_clocks` collection:

```
Litmus > set_max_transition 0.001 -clock_path [all_clocks]
```

- The following command sets the transitions to the specified value for your entire design:

```
Litmus > set_max_transition value [current_design]
```

- The following command impacts `report_constraint` output as shown below:

```
set_max_transition 0.01 TDP_COE_INST/TDP_COE_GL_INST/p82
```

The following command shows output as shown below:

```
report_constraint -all_violators -drv_violation_type max_transition
```

```
#####
```

```
# format : frame 0 : split 0
```

```
Check type : max_transition
```

```
-----
Pin Name                               Required Actual Slack View
```

```
-----  
TDP_COE_INST/TDP_COE_GL_INST/p82  0.010      1.138      -1.128      func_slow_RCMAX  
-----
```

Related Commands

- `get_ports`
- `report_ports`
- `set_max_capacitance`
- `report_constraint`

set_min_capacitance

```
set_min_capacitance  
[-help]  
[-clock_path]  
[-data_path]  
[-rise]  
[-fall]  
[-override]  
capacitance_limit  
obj_list
```

Sets the minimum capacitance limit on the specified ports of the top timing module and/or clock waveforms.

When set on a port, the total capacitance of the net connected to that port must be greater than or equal to the specified limit. When set on a module, the limit applies to all the nets of that module. The total capacitance of each net of the module must be greater than or equal the specified limit. The minimum capacitance limit also comes from library objects, which is the default minimum capacitance limit of libraries and minimum capacitance limit on library cell pins. Pins in the design inherit the limits from libraries. When limits are set on a module and the limit is inherited from libraries, the most constraining limit, which is the largest limit, is used.

Note: The limit set on the top timing module applies to its ports. Thus, if minimum capacitance limits are set on the top timing module and its ports, the most constraining limit will apply to the ports.

Use the `report_ports -type port_cap_limit` command to view the minimum capacitance limits set on ports.

When limits are set on a clock waveform, you can use `-clock_path`, `-data_path`, `-rise`, and `-fall` parameters to provide more specifications.

The `set_min_capacitance` constraint on clock objects will be honored only by the `report_constraint` command for DRV violations. The constraint will not be used in optimizations.

Note: This command syntax is applicable only when the flow-compatible mode is on.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-clock_path</code>	Specifies all pins that are in the network of the specific clocks in the object list.
<code>-data_path</code>	Specifies all pins that are in the data paths launched by the specific clocks in the object list.
<code>-fall</code>	Specifies falling capacitance.
<code>-rise</code>	Specifies rising capacitance for all selected pins.
<code>capacitance_limit</code>	Specifies the value for the minimum capacitance limit. The limit must be greater than or equal to zero. The limit is in capacitance units from the target technology library.
<code>obj_list</code>	Specifies the list of objects - ports, instance pins, library cell pins, modules, and/or clock wave forms.
<code>-override</code>	Overrides the minimum capacitance limit from other sources, such as, library or constraints. This parameter can be used to force a specific constraint value for a specified list of instance pins/port/library cell pins - this overrides the maximum/minimum values set prior to this setting.

Examples

- The following example sets a minimum capacitance limit of 0.02 units on all the ports of the top timing module and a minimum capacitance limit of 0.05 units on the top timing module:

```
Litmus > set_min_capacitance 0.02 [get_ports *]
Litmus > set_min_capacitance 0.05 [current_design]
```

The limit used for ports is 0.05, which is the most restrictive limit set on the top timing module (0.05) and the limit set on ports (0.02).

- The following example uses the `report_port` command to show the minimum capacitance limit set on ports:

```
Litmus > set_min_capacitance 0.02 [get_ports *]
Litmus > report_ports -type port_cap_limit -pin [get_ports *]
```

- The following command sets the minimum capacitance limit of 0.001 units on pins in the `all_clocks` collection (see example below):

```
Litmus > set_min_capacitance 0.001 -data_path [all_clocks]
Litmus > report_ports -type port_cap_limit -pin [get_ports *]
```

The following report is displayed:

```
-----
Pin      Dir      Assertion      Value
Name
-----
CLK2     IN       port_cap_limit  ( 0.020 : 0.900 )
CLK1     IN       port_cap_limit  ( 0.020 : 0.900 )
```

```
in2      IN      port_cap_limit  ( 0.020 : 0.900 )
in1      IN      port_cap_limit  ( 0.020 : 0.900 )
out2     OUT     port_cap_limit  ( 0.020 : 0.900 )
out1     OUT     port_cap_limit  ( 0.020 : 0.900 )
-----
```

Related Commands

- [report_ports](#)
- [reset_min_capacitance](#)
- [set_max_capacitance](#)
- [set_max_transition](#)
- [set_min_transition](#)
- [report_constraint](#)

set_min_delay

```
set_min_delay
[-help]
[-combinational_from_to]
[{-from | -rise_from | -fall_from} from_list]
[{-through | -rise_through | -fall_through} through_list]
[{-to | -rise_to | -fall_to} to_list]
[-rise]
[-fall]
[-comment string]
value
```

Specifies a minimum delay for a timing path. The `set_min_delay` command takes precedence over other path exceptions specified for the same path, except for `set_false_path` command specifications.

Use either the `-from`, `-to`, or `-through` option with the `set_min_delay` command to identify the paths. Specifying both the `-from` and `-to` options identifies a path with specific starting and ending points. If a clock is specified as the `-to` object, all primary outputs and registers clocked by that clock are used as an endpoint.

When you specify a sequential cell name for starting point of the timing path, the valid path starts from the clk pin. This means the latency and delay from clk to output Q is also included in the delay. For path segmentation, you can specify the path to start from Q output of the register by adding `/Q` after the cell name. In this case, path from clk to Q is not considered in the calculation. For example, in the following set of commands, the valid path starts from different points:

```
set_min_delay 0.66 -from TR1/BR1 -to BLK/BR1/D
set_min_delay 0.66 -from TR1/BR1/Q -to BLK/BR1/D
```

For the first command, the valid path starts from clk, and for the second command the valid path starts from the Q output. In the second case, the path from clk to Q is not considered. For an example of the results generated for paths with and without path segmentation, see [Examples](#).

Use only one `-from`, `-rise_from`, or `-fall_from`, `-to`, `-rise_to`, or `-fall_to` parameter per command. You can use multiple `-through` parameters in the same command.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-combinational_from_to</code>	Defines a path defined by <code>-from/-to</code> pins as combinational.
<code>-comment string</code>	Allows you to insert comments. The comments are read but are ignored with a one-time warning message. They are not used during timing analysis reporting nor are they written out during saving of constraints.
<code>-fall</code>	Applies the assertion at the falling edge of the signal on the path endpoints.

<pre>-from -rise_from -fall_from from_list</pre>	Specifies a list of timing paths start points. The <i>from_list</i> argument can contain either object IDs or hierarchical names relative to the current module or clock names. The <i>from_list</i> argument can be a collection. The valid start points are: • Input port • Bidirectional port • Clock pin of sequential cell Clock pin of sequential cell is one of the following: • Structural clock pin that is part of latch or flip-flop. • Inferred clock pin in cells like macro cells. A pin is an inferred clock pin if it is associated with one of the following: • Reference end of setup check • Reference end of hold check • Reference end of removal check • Reference end of recovery check • Minimum period check • Minimum pulse width check • Pin associated with set_input_delay Specify one of the following:
	-from: Applies the minimum delay constraint to all paths originating from the <i>from_list</i> list. If a clock is specified as the <i>-from</i> object, all primary inputs and registers clocked by that clock are used as start points. By default, the <i>-from</i> option applies the constraints to both the rising and the falling edges.
	-rise_from: Applies the minimum delay constraint at the rising edge of the signal on the from pins.
	-fall_from: Applies the minimum delay constraint at the falling edge of the signal on the from pins.
<pre>-rise</pre>	Applies the assertion at the rising edge of the signal on the path endpoints.

<code>-through - rise_through - fall_through through_list</code>	Specifies a list of timing paths through points. The <i>through_list</i> argument can contain either object IDs or hierarchical names relative to the current module or clock names. The pins can be on intermediate hierarchical boundaries. The <i>through_list</i> argument can be a collection. Specify one of the following:
	<code>-through</code>: When you use this parameter without using the <code>-from</code> or <code>-to</code> options, all paths that go through the specified pins have the specified delay. When you use it with both the <code>-from</code> and <code>-to</code> options, any path starting at any pin on the <code>-from</code> list <i>and</i> going through any point on the <code>-through</code> pin list <i>and</i> going to any point on the <code>-to</code> list is affected by the constraint. By default, the <code>-through</code> option applies the constraint to both the rising and the falling edges.
	<code>-rise_through</code>: Applies the assertion at the rising edge of the signal on the through pins.
	<code>-fall_through</code>: Applies the assertion at the falling edge of the signal on the through pins.

<pre>-to -rise_to -fall_to to_list</pre>	Specifies a list of timing paths end points. The <i>to_list</i> argument can contain either object IDs or hierarchical names relative to the current module or clock names. The pins can be on intermediate hierarchical boundaries. The <i>pin_or_clock_list</i> argument can be a collection. The valid end points are: • Output port • Bidirectional port • Data pin of sequential cell Data pin of sequential cell is one of the following: • Structural data pin that is part of latch or flip-flop. • Inferred data pin in cells like macro cells. A pin is an inferred data pin if it is associated with one of the following: • Signal end of setup check • Signal end of hold check • Signal end of non-sequential setup check • Signal end of non-sequential hold check • Signal end of removal check • Signal end of recovery check • Pin associated with <i>set_external_delay</i> • Data pin of clock gating cell Select from one of the following:
	<i>-to</i>: Applies the minimum delay constraint to all paths ending in the <i>-to</i> pin list. If a clock is specified as the <i>-to</i> object, all primary outputs and registers clocked by that clock are used as endpoint. By default, the <i>-to</i> option applies the constraint to both the rising and the falling edges.
	<i>-rise_to</i>: Applies the minimum delay constraint at the rising edge of the signal on the <i>to</i> pins.
	<i>-fall_to</i>: Applies the minimum delay constraint at the falling edge of the signal on the <i>to</i> pins.
<pre>value</pre>	Specifies the minimum delay value.

Examples

- The following command specifies a minimum delay of 2.5 for paths from `in1` to `out1`:

```
% set_min_delay 2.5 -from in1 -to out1
```

- The following commands show the difference in `report_timing` results when you specify a minimum delay with and without path segmentation:

```
% set_min_delay 9 -from R1 -to R2/D
% set_min_delay 9 -from R1/Q -to R2/D
```

The [report_timing](#) results for the first command with path segmentation are:

```
% report_timing -early -from R1
```

```
Path 1: VIOLATED Hold Check with Pin R2/CK
Endpoint: R2/D (v) checked with leading edge of 'CLK_W_1'
Beginpoint: R1/Q (v) triggered by leading edge of 'CLK_W_1'
Path Groups: {CLK_W_1}
Other End Arrival Time 0.000
+ Hold -0.042
+ Path Delay 9.000
= Required Time 8.958
Arrival Time 0.212
Slack Time -8.745
Clock Rise Edge 0.000
+ Clock Network Latency (Ideal) 0.000
= Beginpoint Arrival Time 0.000
```

Pin	Cell	Delay	Arrival Time	Required Time	Instance
R1/CK ->	-	-	0.000	8.745	R1
R1/Q	DFF	0.144	0.144	8.889	R1
B1/Y	BUF	0.068	0.212	8.958	B1
R2/D	DFF	0.000	0.212	8.958	R2

- The `report_timing` results for the second command without path segmentation are:

```
% report_timing -early -from R1/Q
```

```
Path 1: VIOLATED Hold Check with Pin R2/CK
Endpoint: R2/D (^) checked with leading edge of 'CLK_W_1'
Beginpoint: R1/Q (^) triggered by leading edge of 'CLK_W_1'
Path Groups: {CLK_W_1}
Other End Arrival Time 0.000
+ Hold -0.047
+ Path Delay 9.000
```

```
= Required Time 8.953
Arrival Time 0.056
Slack Time -8.897
Clock Rise Edge 0.000
= Beginpoint Arrival Time 0.000
```

Pin	Cell	Delay	Arrival Time	Required Time	Instance

R1/Q ->	-	-	0.000	8.897	R1
B1/Y	BUF	0.056	0.056	8.953	B1
R2/D	DFF	0.000	0.056	8.953	R2

Related Commands

- `set_input_delay`
- `set_output_delay`
- `set_false_path`
- `set_max_delay`
- `set_multicycle_path`

set_min_fanout

```
set_min_fanout
[-help]
limit
obj_list
[-override]
```

Sets the minimum fanout load limit constraint on the specified input ports of the top cell.

When set on a port, the fanout load of the net connected to the port must be greater than or equal to the specified limit value to meet the constraint. The fanout load of a net is the sum of the fanout loads of the pins driven by the net. When set on a module, the limit applies to all the nets of that module. The libraries and library cell pins may also have the minimum fanout load limit. In this case the most constraining limit is applied to the nets.

The constraint will not be used in optimizations.

Fanout load limit can be asserted on input ports, output pins, bidirectional pins, or ports and library pins. The constraint cannot be applied on input pins and output ports.

Parameters

<code>-help</code>	Prints out the command usage.
<code>limit</code>	Specifies the value of the minimum fanout load limit constraint. The specified value must be greater than or equal to zero.
<code>obj_list</code>	Specifies a list of objects - ports, instance pins, modules, or library cell pins.
<code>- override</code>	Overrides the minimum fanout limit from other sources, such as, library or constraints. This parameter can be used to force a specific constraint value for a specified list of instance pins/port/library cell pins - this overrides the maximum/minimum values set prior to this setting.

Examples

- The following example sets a minimum fanout load limit of 0.001 units on all the ports of the top cell:

```
Litmus > set_min_fanout 0.001 [get_ports *]
```

- The following example shows a report of the minimum fanout load limit set on ports using the report_ports command, as shown in the following example:

```
Litmus > set_min_fanout 5.0 [get_ports *]
```

```
Litmus > report_ports -type fanout_load_limit -pin [get_ports *]
```

The following report is displayed:

```
-----
Pin      Dir  Assertion      Value
```

Name

CLK2	IN	fanout_load_limit	5.000
CLK1	IN	fanout_load_limit	5.000
in2	IN	fanout_load_limit	5.000
in1	IN	fanout_load_limit	5.000

Related Commands

- `get_ports`
- `report_ports`
- `reset_min_fanout`
- `set_fanout_load`
- `set_max_capacitance`
- `set_max_transition`
- `report_constraint`

set_min_pulse_width

```
set_min_pulse_width  
[-help]  
[-low]  
[-high]  
[-waveform_aware_type {absolute | source_width_ratio}]  
value  
object_list
```

Selects the most restrictive value when multiple minimum pulse width checks are set.

Note: The `timing_disable_lib_pulse_width_checks` attribute has no effect on pulse width checks created by the `set_min_pulse_width` command.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-high</code>	Applies the minimum pulse width check to high signal levels only. If neither the <code>-low</code> or <code>-high</code> option is specified, the value is applied to both low and high signal levels.
<code>list</code>	Specifies a list of pins or clock waveforms to which the minimum pulse width check is to be applied. If the list is not specified, the minimum pulse width check applies to all the pins in the clock tree of the current design. If a clock waveform is specified, all the pins driven by the clock are affected.
<code>-low</code>	Applies the minimum pulse width check to low signal levels only. If neither the <code>-low</code> or <code>-high</code> option is specified, the value is applied to both low and high signal levels.
<code>object_list</code>	Specifies list of objects: pins, ports, or clocks.
<code>value</code>	Specifies a non-negative floating minimum pulse width check value. No default value is assumed for minimum pulse width checks.
<code>- waveform_aware_type {absolute source_width_ratio}</code>	Specifies the type of waveform-aware pulse width check to apply as <code>absolute</code> or <code>source_width_ratio</code> .

Examples

- The following command sets a minimum pulse width, both low and high, check of 1.0 to the clock network of the current design:

```
% set_min_pulse_width 1.0
```

- The following command sets a minimum pulse width low check of 2.0 to all the pins clocked by the CLK1 clock:

```
% set_min_pulse_width -low 2.0 [get_clocks CLK1]
```

- The following command sets a minimum pulse width -high check of 1.0 to the CP clock pin of register ff1, as shown in the following example:

```
% set_min_pulse_width 0.5 u55/CK -low
% set_min_pulse_width 1.0 u55/CK -high
% report_min_pulse_width -pin u55/CK
```

SEQUENTIAL CLOCK PULSE WIDTH

Pin	Required Pulse Width	Actual Pulse Width	Slack
u55/CK (low)	0.500	4.200	3.700
u55/CK (high)	1.000	5.800	4.800

```
% report_timing -check_type pulse_width -to u55/CK
```

```
Path 1: MET User PulseWidth Check with Pin u55/CK
Ending Clock Edge: u55/CK (^) checked with leading edge of 'CLK4'
Beginning Clock Edge: u55/CK (v) triggered by trailing edge of 'CLK4'
Path Groups: {CLK4}
Other End Arrival Time 0.152
- PulseWidth 0.500
+ Phase Shift 10.000
= Required Time 9.651
- Arrival Time 5.951
= Slack Time 3.700
Clock Fall Edge 6.000
+ Clock Network Latency (Prop) -0.049
= Beginpoint Arrival Time 5.951
```

Load	Slew	Delay	Arrival Time	Cell	Arc	Pin
0.015	0.019	5.951	DFF	CK	v	u55/CK ->

Related Commands

- `report_analysis_coverage`
- `report_timing`
- `reset_min_pulse_width`

set_min_transition

```
set_min_transition  
[-help]  
[-clock_path]  
[-data_path]  
[-rise]  
[-fall]  
[-override]  
limit  
obj_list
```

Sets the minimum slew time limit (transition) on the specified ports of the top timing module and/or clock waveforms.

When set on a port, the slew time at the port must be greater than or equal to the specified limit. When set on a module, the limit applies to all the pins in the module and its ports if the module is the top timing module. The minimum transition limits on pins can come from library objects, which are the default minimum transition of the libraries or minimum transition on library cell pins. In this case, the most constraining limit, which is the largest, will apply on pins and ports.

Use the `report_ports -type slew_limit` command to view the minimum transition limits set on ports of the top timing module.

When limits are set on a clock waveform, you can use `-clock_path`, `-data_path`, `-rise`, and `-fall` parameters to provide more specifications.

The `set_min_transition` constraint on clock objects will be honored only by the `report_constraint` command for DRV violations. The constraint will not be used in optimizations.

Note: This command syntax is applicable only when the flow-compatible mode is on.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-clock_path</code>	Specifies all pins that are in the network of the specific clocks in the object list.
<code>-data_path</code>	Specifies all pins that are in the data paths launched by the specific clocks in the object list.
<code>-fall</code>	Specifies falling capacitance.
<code>obj_list</code>	Specifies the list of objects - ports, instance pins, library cell pins, modules, and/or clock waveforms.
<code>-override</code>	Overrides the minimum transition limit from other sources, such as, library or constraints. This parameter can be used to force a specific constraint value for a specified list of instance pins/port/library cell pins - this overrides the maximum/minimum values set prior to this setting.
<code>-rise</code>	Specifies rising capacitance for all selected pins.
<code>limit</code>	Specifies the value of the minimum slew time limit (minimum transition). The minimum slew time limit is in time units from the target technology library. Transition limit value (float, required) Min=-1000000 Max=1000000

Examples

- The following example sets a minimum transition limit of 0.9 units on all the ports of the top timing module, and a minimum transition limit of 0.7 units on the top timing module:

```
> set_min_transition 0.9 [get_ports *]
> set_min_transition 0.7 [current_design]
```

The minimum limit that applies to ports is 0.9, which is the largest constraints value.

- The following example shows a report of the minimum transition limit set on ports using the `report_ports` command, as shown in the example below:

```
> set_min_transition 0.9 [get_ports *]
> report_ports -type slew_limit -pin [get_ports *]
```

The following report is displayed:

```
-----
Pin      Dir  Assertion  Value
Name
-----
```

```
CLK2    IN    slew_limit  ( 0.900 : )
CLK1    IN    slew_limit  ( 0.900 : )
in2     IN    slew_limit  ( 0.900 : )
in1     IN    slew_limit  ( 0.900 : )
out2    OUT   slew_limit  ( 0.900 : )
out1    OUT   slew_limit  ( 0.900 : )
-----

> report_constraint -drv_violation_type min_transition

Check type : min_transition
-----
Pin : CLK4

Transition Time: 0.003
- min_transition: 0.900
-----

slack : -0.897 ( VIOLATED )
```

Related Commands

- [set_max_capacitance](#)
- [reset_min_transition](#)
- [report_ports](#)
- [report_constraint](#)

set_mode

```
set_mode
[-help]
[-type cell]
mode_list
inst_list
```

Specifies the Liberty timing library modes to be active for a specific instance.

A library mode is a subset of active timing arcs that is defined in the timing library. Library modes are similar to conditional timing arcs, but do not require the application of constants to set. Instead of changing a cell's mode by applying constants to its input pins, you can use the `set_mode` command to refer to sets of active arcs by name (such as `READ` or `WRITE`, for a memory cell).

By default, all library modes in the timing library are active. If you specify one or more library modes using the `set_mode` command, only the timing arcs consistent with the specified mode(s) are active for the instance. All other library modes become inactive.

To reset the library modes for an instance, use the `reset_mode` command.

Parameters

<code>-help</code>	Prints out the command usage.
<code>inst_list</code>	Specifies the instances to which to apply the library mode.
<code>mode_list</code>	Specifies the name of the library mode value defined in the timing library. You can specify one or more mode values, but only one mode value per defined mode group in the Liberty library.
<code>-type cell</code>	Sets the library mode at the instance level. Note: Setting the library mode at the instance level is the only form of <code>set_mode</code> that is currently supported. This parameter exists to support constraints coming from external sources.

Examples

The following command specifies that only the timing arcs consistent with the `test_A` library mode are active for instances of `b1` (The `test_A` library mode is made inactive.):

```
% set_mode test_mode [get_cells b1]
```

The following command reports the status of all library modes associated with `b1`. It shows that `test_A` is active and `test_B` is inactive.

```
% report_mode b1
```

```
-----
Mode Group test_mode of b1
```

```

-----
Mode      Status      Condition
Name
-----
test_B    Inactive    !(Y)
test_A    ACTIVE      Y
-----

```

If you specify the `report_inactive_arcs` command, the report shows the disabled arcs for the b1 instance are from the `test_mode` library mode:

```

-----
Flags :  const propagated constant
         snipped loop snipped arcs
         disable set_disable_timing
         disable_clock_gating set_disable_clock_gating
         library_disable set_disable_cell_timing
         Missing_Phase No Arc Phase Data

```

From	To	DisableType	ArcType	Reason
u8/CK ^	u8/D ^	const	hold_rising_clk_rise	u8/D = 1
u8/CK ^	u8/D v	const	hold_falling_clk_rise	u8/D = 1
u8/CK ^	u8/D ^	const	setup_rising_clk_rise	u8/D = 1
u8/CK ^	u8/D v	const	setup_falling_clk_rise	u8/D = 1

```

-----

```

Related Commands

- `reset_mode`
- `report_mode`
- `report_inactive_arcs`
- `get_cells`

set_multicycle_path

```
set_multicycle_path
[-help]
number_of_cycles
[-comment string]
[-fall]
[-fall_through through_list]
[-hold]
[-rise]
[-rise_through through_list]
[-setup]
[-through through_list]
[-start | -end]
[-from from_list | -rise_from from_list | -fall_from from_list]
[-to to_list] | -rise_to to_list | -fall_to to_list]]
```

Specifies multicycle paths between specific timing paths in the design or between clock domains. Multicycle paths are timing paths that require more than one clock period for execution. By default, all paths are considered as one cycle paths for setup and hold check evaluation. The `set_multicycle_path` command lets you specify additional cycles for the paths.

Parameters

<code>-help</code>	Prints out the command usage.
<code>number_of_cycles</code>	Specifies the number of cycles allowed for the path.
<code>-comment <i>string</i></code>	Allows you to insert comments. The comments are read but are ignored with a one-time warning message. They are not used during timing analysis reporting nor are they written out during saving of constraints.
<code>-fall</code>	Applies the assertion at the falling edge of the signal on the path endpoints.
<code>-fall_through <i>through_list</i></code>	Applies the assertion at the falling edge of the signal on the through pins.

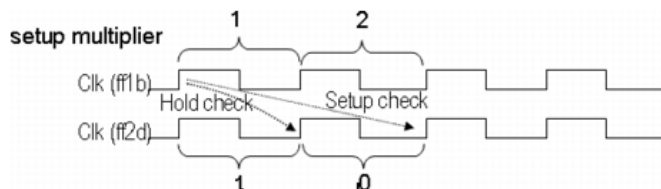
<pre>-from from_list -rise_from from_list - fall_from from_list</pre>	Specifies the clocks, ports, pins, or sequential instances that are at the start of the multicycle paths. The port direction can be either input or inout. Pins must be valid path startpoints such as sequential instance clock pins, latch data pins or pins on which input delays have been specified. Pins can be on intermediate hierarchical boundaries. The <i>from_list</i> variable can contain either collection IDs or object names. If the <i>-from</i> option is used without the <i>-to</i> option, all paths originating from the <i>from_list</i> are multicycle paths. Use either the <i>-from</i>, <i>-to</i>, or <i>-through</i> option with the <i>set_multicycle_path</i> command to identify the paths. Specifying both the <i>-from</i> and <i>-to</i> options identifies a path with specific starting and ending points. Use only one <i>-from</i>, <i>-rise_from</i>, or <i>-fall_from</i> option per command.
<pre>-hold -setup</pre>	Specifies whether the multicycle adjustment applies to setup/hold check. The <i>-setup</i> parameter applies multicycles to setup check. The hold check is affected but the hold multiplier does not change. The <i>-hold</i> parameter applies multicycles to hold check only. <i>Default:</i> If you do not specify <i>-setup</i> or <i>-hold</i> parameter, the software considers the specified multicycle value for setup check and uses 0 for the hold check.
<pre>-rise</pre>	Applies the assertion at the rising edge of the signal on the path endpoints.
<pre>-rise_through through_list</pre>	Applies the assertion at the rising edge of the signal on the through pins.
<pre>-start -end</pre>	Specifies whether the multicycle adjustment is to be applied relative to the starting clock edge for the path or relative to the ending clock edge for the path. Use these options for multiple clocks designs. By default, the setup check is relative to the end clock and the hold check is relative to the start clock. Note: The <i>-start</i> and <i>-end</i> parameters are mutually exclusive; you cannot specify them together. You can set the <i>timing_enable_multicycle_data_check_compatibility</i> attribute to <i>true</i> to enable checking for SDC compatible data checks. When the <i>-start</i> parameter is specified, the parameter reverses the reference clock by one clock cycle.

<pre>-through through_list</pre>	Specifies the ports, pins, instances, or nets that the path goes through. The pins can be on intermediate hierarchical boundaries. The <i>through_list</i> variable can contain either collection IDs or object names. When used without the <i>-from</i> or <i>-to</i> option, all paths that go through the <i>-through</i> pins are multicycle paths. When used with both the <i>-from</i> and <i>-to</i> options, any path starting at any pin on the <i>-from</i> list and going through any point on the <i>-through</i> pin list and going to any point on the <i>-to</i> list is affected by the assertion. You can use only one <i>-from</i> and <i>-to</i> option, but you can specify several <i>-through</i> options in the same command. By default, the <i>-through</i> option applies the assertion to both the rising and the falling edges. Using the <i>-rise_through</i> option applies the assertion at the rising edge of the signal on the through pins. Using the <i>-fall_through</i> option applies the assertion at the falling edge of the signal on the through pins.
<pre>-to to_list] - rise_to to_list -fall_to to_list</pre>	Specifies the clocks, ports, pins, or sequential instances that are at the end of the multicycle paths. The port direction can be either output or inout. Pins must be valid path end-points such as sequential instance data pins or pins on which output delays have been specified. Pins can be on intermediate hierarchical boundaries. The <i>to_list</i> variable can contain either collection IDs or object names. If the <i>-to</i> option is used without the <i>-from</i> option, all paths ending in the <i>-to</i> pin list are multicycle paths. Use only one <i>-to</i> option per command. Use only one <i>-to</i>, <i>-rise_to</i>, or <i>-fall_to</i> option per command.

Examples

- The following command sets all paths between ff1b + ff2d to two cycles for setup:

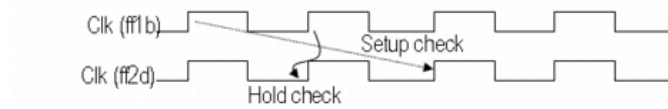
```
set_multicycle_path -setup 2 -from {ff1b} -to {ff2d}
```



- The following command moves hold check to start clock preceding edge:

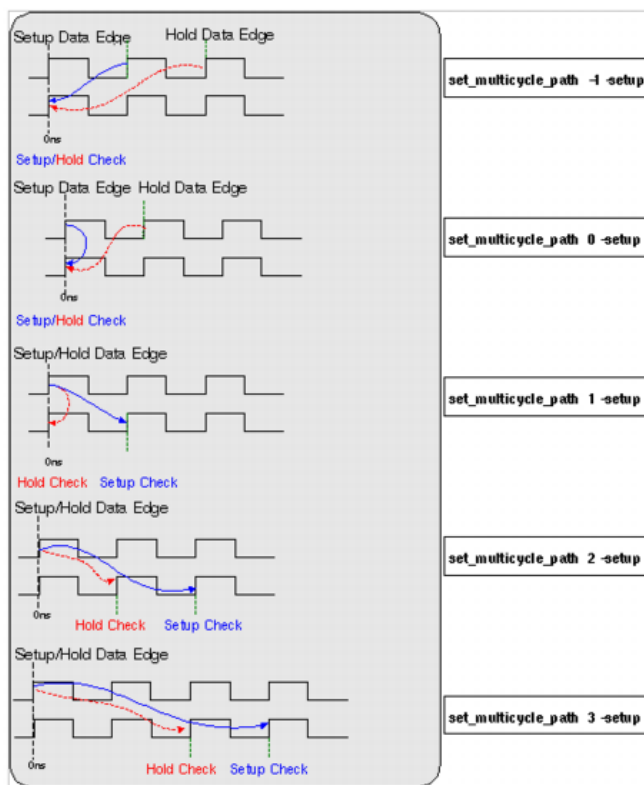
```
set_multicycle_path -hold 1 -from {ff1b} -to {ff2d}
```

hold multiplier



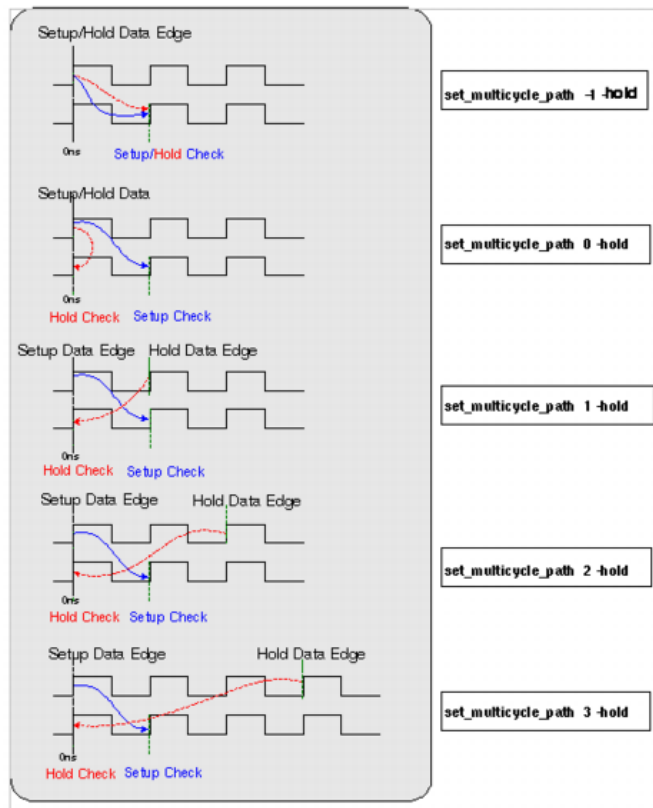
- The following commands show setup multiplier values from -1 to 3:

```
set_multicycle_path -1 -setup
set_multicycle_path 0 -setup
set_multicycle_path 1 -setup
set_multicycle_path 2 -setup
set_multicycle_path 3 -setup
```



- The following commands show hold multiplier values from -1 to 3:

```
set_multicycle_path -1 -hold
set_multicycle_path 0 -hold
set_multicycle_path 1 -hold
set_multicycle_path 2 -hold
set_multicycle_path 3 -hold
```



Related Commands

- `set_false_path`
- `set_max_delay`
- `set_min_delay`

set_output_delay

```
set_output_delay
[-help]
-clock clock_name
[-clock_fall]
[-rise]
[-fall]
[-max]
[-min]
[-add_delay]
[-network_latency_included]
[-source_latency_included]
[-reference_pin pin_name]
[-level_sensitive]
[-group_path group_name]
value
pin_or_port_list
```

Specifies the data required time on output ports, hierarchical ports, or internal output pins.

Parameters

-help	Prints out the command usage.
-add_delay	Adds delay information to the existing output delay. The additional delay information can be for multiple paths relative to different clocks or clock edges leading to an input port or internal input pins. If you do not use the <code>-add_delay</code> option, the software uses the last <code>set_output_delay</code> related to a pin when there are multiple <code>set_output_delay</code> statements for the same pin.
-clock <i>clock_name</i>	Specifies that the delay is relative to the specified clock. You cannot use the <code>set_output_delay</code> command without the <code>-clock</code> parameter. Such constraints are accepted but are ignored during timing analysis. When you specify multiple clocks, an error occurs and the constraint is ignored.
-clock_fall	Specifies that the delays are with respect to the falling edge of the clock waveform. If you do not specify the <code>-clock_fall</code> option, the delay is with respect to the rising edge of the clock waveform. You cannot specify the <code>-clock_fall</code> parameter without the <code>-clock</code> parameter.
<i>value</i>	Specifies the value for the path delay.
-fall	Specifies that the output delay refers to the falling edge of the signal at specified ports or pins. If you do not specify the <code>-fall</code> option, the output delay applies to both the edges.

<code>-group_path group_name</code>	Specifies the group name where paths ending at the specified ports or pins are to be added. The <code>-group_path</code> parameter is not honored in multi-mode multi-corner (MMMC) designs. If specified, a warning is issued and the constraints are applied as if the <code>-group_path</code> parameter is not specified.
<code>-level_sensitive</code>	Models output delay that is captured by a positive level sensitive latch. For setup checks, the timing system checks the arrival time at the output port with the opening edge of the latch. The timing system treats any arrival time exceeding the opening edge as negative slack. For hold checks, the timing system performs the checks with respect to the closing edge of the latch. Use the <code>-clock</code> parameter to model a positive level sensitive latch, or the <code>-clock_fall</code> parameter to model a negative level sensitive latch. You cannot use the <code>-level_sensitive</code> parameter without the <code>-clock</code> parameter.
<code>-max</code>	Specifies that the delay refers to the setup analysis. If you do not specify the <code>-max</code> parameter, the output delay applies to both setup and hold analysis.
<code>-min</code>	Specifies that the delay refers to the hold analysis. If you do not specify the <code>-min</code> parameter, the output delay applies to both setup and hold analysis.
<code>-network_latency_included</code>	Specifies that the clock network latency should not be added to the output delay value. If you do not specify this option, the clock network latency of the related clock is added to the output delay value. This parameter has no effect if the specified clock is in propagated mode.
<code>pin_or_port_list</code>	Specifies a list of output port or pin names. The <code>pin_or_port_list</code> argument can be a collection.
<code>-reference_pin pin_name</code>	Adds source and network latency of the specified reference pin to the output delay value in propagated mode. A reference pin is a leaf pin in the fanout cone of the clock source of the clock that you specify using the <code>-clock</code> parameter. You cannot specify multiple pins for <code>-reference_pin</code> parameter. The command issues a warning and accepts the first pin. You cannot use the <code>-reference_pin</code> parameter with the <code>-network_latency_included</code> or <code>-source_latency_included</code> parameters. You cannot use the <code>-reference_pin</code> parameter to specify hierarchical pins.
<code>-rise</code>	Specifies that output delay refers to the rising edge of the signal at the specified ports or pins. If you do not specify the <code>-rise</code> option, the output delay applies to the both edges.

<code>-source_latency_included</code>	Specifies that the clock source latency should not be added to the output delay value. With different delay values for <code>-rise/-fall</code> and <code>-min/-max</code>, only the last specified combination of <code>-network_latency_included</code> and <code>-source_latency_included</code> parameters is honored. The specified combination can be one of the following: a) Either <code>-network_latency_included</code> or <code>-source_latency_included</code> b) Both the parameters c) None of these
---------------------------------------	---

Examples

- The following example sets the input and output delays for the bidirectional INOUT2 port. The input signal arrives at INOUT2 at 4.5 units after the falling edge of CLKB. The output signal is required at INOUT2 at 2.5 units before the rising edge of CLKD:

```
% set_input_delay 4.5 -clock CLKB -clock_fall { INOUT2 }
% set_output_delay 2.5 -clock CLKD { INOUT1 }
```

- The following example shows how to use the `-group_path` option to add ports into a named group:

```
% set_output_delay 2.5 -max -clock CLK -group_path busC {busC[*]}
```

Related Commands

- `all_outputs`
- `create_clock`
- `report_ports`
- `reset_output_delay`
- `set_input_delay`
- `set_load`

set_pll_timing

```
set_pll_timing
[-help]
inst
-pll_feedback_pin <feedback pin name>
-pll_output_clock <clock name or ID>
-pll_output_pin <output pin name>
-pll_reference_pin <reference pin name>
```

This command identifies phase locked loop (PLL) insertion delays and the associated generated clock(s) that require adjustment. To remove the effect of clock tree latency, PLLs can be used to adjust the delay on the PLL output clock waveform so that the effective edges of the clock at the clock tree endpoints are aligned with the reference edges of the input clock to the PLL.

When you use the `set_pll_timing` command, the software uses the settings to automatically calculate and apply a delay offset to a PLL output clock to compensate for internal clock tree latency.

The `set_pll_timing` command works in conjunction with a generated clock definition to provide the correct clock timing at the output of the PLL. To use this command, you must first assert a generated clock on the `set_pll_timing -pll_output_pin`. The `-source` option of the `create_generated_clock` command must also match the `-pll_reference_pin` specified by the `set_pll_timing` command.

Parameters

<code>-help</code>	Prints out the command usage.
<code>inst</code>	Specifies the instance name or the object ID of a PLL cell, on the basis of which the reference pins, feedback pins, or the output pins will be searched.
<code>-pll_feedback_pin <feedback pin name></code>	Specifies the name of the PLL feedback pin.
<code>-pll_output_clock <clock name or ID></code>	Specifies the generated clock on the PLL output.
<code>-pll_output_pin <output pin name></code>	Specifies the PLL clock output pin name.
<code>-pll_reference_pin <reference pin name></code>	Specifies the PLL reference pin name.

Examples

Here are some examples.

- The following example shows the defined clocks in a sample design:

```
% create_clock [get_ports {clk_in}] -name CLK -period 6 -waveform {0 3}
% create_generated_clock -name GCLK1 -source [get_pins {mypll/CLK}] -multiply_by 1 -
master_clock [get_clocks {CLK}] -add [get_pins {mypll/GCLK}]
```

The following `report_timing` command shows the insertion delay values at various points in the timing report:

```
% report_timing -from_rise mypll/GCLK -to_rise mypll/CEN -point_to_point
```

```
Path 1:Endpoint: mypll/CEN (^) (unconstrained output)
Beginpoint: mypll/GCLK (^) triggered by leading edge of 'GCLK1'
Arrival Time 0.496
Analysis View: default_analysis_view_setup
-----
Pin           Cell      Delay   Arrival   Generated Clock
              Time      Time      Adjustment
-----
mypll/CLK           0.112
mypll/GCLK ->  PLL      0.133  0.246      GCLK1 Adj. = 0.000
clkTreeL0/Y    CLKBUF1  0.139  0.385
clkTreeL1_1/Y  CLKBUF1  0.118  0.503
mypll/CEN ->  PLL      -0.007  0.496
-----
```

From the above table, it can be seen that the rise insertion delay of clock CLK at pin mypll/CLK is 0.112ns.

Now, the following command will adjust the source latency of clock GCLK1 such that the edges at mypll/CLK and myPLL/CEN are aligned i.e. the clock's rising edge should reach mypll/CEN at 0.112ns only:

```
% set_pll_timing [get_cells mypll] -pll_reference_pin CLK -pll_feedback_pin CEN -pll_output_pin
GCLK -pll_output_clock [get_clocks GCLK1]
```

- The following `report_timing` command shows the source latency of GCLK clock as -0.138ns, and the clock reaches mypll/CEN pin at 0.112ns (same as on mypll/CLK):

```
% report_timing -from_rise mypll/GCLK -to_rise mypll/CEN -point_to_point
```

```
Path 1:Endpoint: mypll/CEN (^) (unconstrained output)
Beginpoint: mypll/GCLK (^) triggered by leading edge of 'GCLK1'
Arrival Time 0.112
Analysis View: default_analysis_view_setup
+ Source Insertion Delay 0.112
- PLL Feedback Delay 0.250
= Beginpoint Arrival Time -0.138
-----
Pin           Cell      Delay   Arrival   Generated Clock
              Time      Time      Adjustment
-----
mypll/GCLK ->           -0.138      GCLK1 Pll Adj. = 0.000
-----
```

Litmus Text Command Reference

Timing Constraint Commands--set_pll_timing

clkTreeL0/Y	CLKBUF1	0.139	0.001
clkTreeL1_1/Y	CLKBUF1	0.118	0.120
mypll/CEN ->	PLL	-0.007	0.112

Related Commands

- `create_generated_clock`
- `create_clock`
- `reset_pll_timing`

set_propagated_clock

```
set_propagated_clock  
[-help]  
obj_list
```

Puts the `propagated_clock` assertion on the specified pin, port, or clock object. This will cause all clock endpoints in the fanout of the specified object to receive propagated clock timing unless there is a `set_clock_latency` with higher precedence. If there are multiple `set_clock_latency` and `set_propagated_clock` assertions between the clock waveform, clock root and clock endpoint, the closest assertion to the clock endpoint takes precedence.

There are two methods of clock propagation: depending on whether the clock is ideal or propagated.

- When a clock is ideal, the delay from a clock port to a register clock pin (network insertion delay) comes from design constraints using the `set_clock_latency` command.
- When a clock is propagated, the network insertion delay is computed from the actual gates and interconnects in the clock network.

To set the `propagated_clock` property on all clock waveform objects, use the following command:

```
set_propagated_clock [all_clocks]
```

When `set_propagated_clocks [all_clocks]` command is asserted, it will apply to all the currently defined clock waveform objects. This setting will not apply to clocks created later in the script. Whether all the clock endpoints receive propagated or ideal timing depends on the precedence of `set_clock_latency` in the fanout.

The `set_propagated_clock [all_clocks]` command puts the `propagated_clock` assertion on all clocks except virtual clocks. When you put the assertion on a real clock that is used to define I/O timing (with `set_input_delay -clock` or `set_output_delay -clock`), the software does not take the specified network latency into account. Instead, the software calculates the network latency, which changes the I/O timing. You can change this behavior by using the following attribute:

```
set_db timing_io_use_clock_network_latency always
```

You can use this command after creating the clock with the `create_clock` command.

Parameters

<code>-help</code>	Prints out the command usage.
<code>obj_list</code>	Specifies the objects on which you want to put the <code>propagated_clock</code> assertion. The <code>pin_or_clock_list</code> argument can be a collection. When a pin is specified, it affects the propagation mode for all the registers in the transitive fanout (TFO) of the pin.

Examples

- The following command puts the propagated clock assertion on the clock waveform CLK1:

```
% set_propagated_clock CLK1
```

- The following command puts the propagated clock assertion on all clock waveforms:

```
% set_propagated_clock [all_clocks]
```

Related Commands

- `all_clocks`
- `get_propagated_clock`
- `report_clocks`
- `reset_propagated_clock`
- `set_clock_latency`
- `set_clock_uncertainty`

set_property

```
set_property
[-help]
[-obj_type {object_type}]
{obj_list}
property_name
property_value
[-quiet]
```

Sets user-defined properties on objects.

The `set_property` command needs a property to be defined using the `define_property` command.

Parameters

<code>-help</code>	Prints out the command usage.
<code>obj_list</code>	Specifies a list of objects on which the property is to be applied.
<code>-obj_type {object_type}</code>	Specifies the type of object on which the property is to be defined.
<code>property_name</code>	Specifies the name of the property.
<code>property_value</code>	Specifies the value of the property.
<code>-quiet</code>	Specifies to exit quietly without issuing an error.

Examples

- The following command applies a user-defined property `slk` to pins C1/A:

```
% set_property [get_pins C1/A] slk 2
```

- The following command applies `aocv_weight` property to library arcs:

```
% set_property [get_lib_arcs -of_objects slow/INVX1] aocv_weight 8
```

Related Commands

- `get_property`
- `report_property`
- `define_property`
- `reset_property`

set_pulse_clock_max_transition

```
set_pulse_clock_max_transition  
[-help]  
limit  
obj_list  
[-transitive_fanout]  
[-rise]  
[-fall]
```

Sets the maximum pulse clock transition constraint on the specified pulse generator cells, pulse generator library cells, clocks, and/or current design.

Parameters

-help	Prints out the command usage.
-fall	Sets transition limit value for falling transition.
obj_list	Specifies a list of objects - pulse generator cells, pulse generator library cells, clocks, or current design.
-rise	Sets transition limit value for rising transition.
limit	Specifies the transition limit value.
- transitive_fanout	Specifies the transition limit value for transitive fanout.

Examples

- The following example sets the max pulse clock transition constraint of 0.1ns on the input pins of all the instances of the lib cell `PULSE_GEN_00`:

```
% set_pulse_clock_max_transition 0.1 PULSE_GEN_00
```

- This constraint can be observed on the input pin 'A' of an instance 'pg_inst' of this lib cell using the following command:

```
% report_constraint pg_inst/A -drv_violation_type pulse_clock_max_transition
```

Fragmented output:

```
Check type : pulse_clock_max_transition_rise
```

```
-----
```

```
-----
```

```
Pin Name Required Actual Slack
```

```
-----
```

```
pg_inst/A 0.100 0.030 0.070
```

```
Check type : pulse_clock_max_transition_fall
```

```
-----
```

```
-----
```

```
Pin Name Required Actual Slack
```

```
-----
```

```
pg_inst/A 0.100 0.030 0.070
```

Related Commands

- `get_clocks`
- `get_cells`
- `reset_pulse_clock_max_transition`

set_pulse_clock_max_width

```
set_pulse_clock_max_width  
[-help]  
width  
obj_list  
[-transitive_fanout]
```

Sets the maximum pulse clock width constraint on the specified pulse generator cells, pulse generator library cells, clocks, and/or current design.

Parameters

<code>-help</code>	Prints out the command usage.
<code>obj_list</code>	Specifies a list of objects - pulse generator cells, pulse generator library cells, clocks, or current design.
<code>width</code>	Specifies the pulse width value.
<code>[-transitive_fanout]</code>	Specifies the pulse width value for transitive fanout.

Examples

The following example sets the max pulse clock width constraint of 1 ns on the pulse generator instance 'pg_inst':

```
% set_pulse_clock_max_width 1 pg_inst
```

This constraint can be observed using the following command:

```
% report_constraint pg_inst/Y -check_type pulse_clock_max_width
```

Fragmented output:

```
-----  
Pin Required Actual Pulse Slack Clock Type  
Pulse Width Width  
-----  
pg_inst/Y (high) 1.000 0.366 0.634 PG  
-----
```

Related Commands

- `get_clocks`
- `get_cells`
- `reset_pulse_clock_max_transition`
- `reset_pulse_clock_max_width`

set_pulse_clock_min_transition

```
set_pulse_clock_min_transition
[-help]
limit
obj_list
[-transitive_fanout]
[-rise]
[-fall]
```

Sets the minimum pulse clock transition constraint on the specified pulse generator cells, pulse generator library cells, clocks, and/or current design.

Parameters

-help	Prints out the command usage.
-fall	Sets transition limit value for falling transition.
obj_list	Specifies a list of objects - pulse generator cells, pulse generator library cells, clocks, or current design.
-rise	Sets transition limit value for rising transition.
limit	Specifies the transition limit value.
-transitive_fanout	Specifies the transition limit value for transitive fanout.

Examples

The following example sets the min pulse clock transition constraint of 0.05ns on the input pins of all the instances of the lib cell PULSE_GEN_00:

```
% set_pulse_clock_min_transition 0.05 PULSE_GEN_00
```

This constraint can be observed on the input pin 'A' of an instance 'pg_inst' of this lib cell using the following command:

```
% report_constraint pg_inst/A -drv_violation_type pulse_clock_min_transition
```

Fragmented output:

```
Check type : pulse_clock_min_transition_rise
-----
-----
Pin Name Required Actual Slack
-----
pg_inst/A 0.050 0.030 -0.020
```

Check type : pulse_clock_min_transition_fall

Pin Name	Required	Actual	Slack
----------	----------	--------	-------

pg_inst/A	0.050	0.030	-0.020
-----------	-------	-------	--------

Related Commands

- [reset_pulse_clock_min_transition](#)
- [get_clocks](#)
- [get_cells](#)

set_pulse_clock_min_width

```
set_pulse_clock_min_width  
[-help]  
pulse_width  
object_list  
[-transitive_fanout]
```

Sets the minimum pulse clock width constraint on the specified pulse generator cells, pulse generator library cells, clocks and/or current design.

When you set this constraint on a clock, the settings are applied to all the pulse generators that are driven by the clock. When this constraint is used for a design, the software applies the constraint to all the clock networks in the design.

The specified minimum pulse width is expected to be larger than the pulse width value. The width is defined when a clock object is created, and its waveform is defined in the current design (using the `create_clock - waveform` parameter).

In case of conflicting minimum width constraints, the more restrictive constraint is used. When applied on a non-pulse generator object, or a non-clock target, this constraint has no effect.

 You must specify `set_interactive_constraint_modes` constraint prior to running the `set_pulse_clock_min_width` command.

Parameters

<code>-help</code>	Prints out the command usage.
<code>object_list</code>	Specifies a list of objects - pulse generator cells, pulse generator library cells, clocks, or current design.
<code>pulse_width</code>	Specifies the minimum pulse width value.
<code>- transitive_fanout</code>	Specifies the pulse width value for transitive fanout.

Examples

The following example sets the min pulse clock width constraint of 0.3 ns on the pulse generator instance `pg_inst`:

```
% set_pulse_clock_min_width 0.3 pg_inst
```

This constraint can be observed using the following command:

```
% report_constraint pg_inst/Y -check_type pulse_clock_min_width
```

Fragmented output:

```
-----  
Pin Required Actual Pulse Slack Clock Type  
Pulse Width Width  
-----  
pg_inst/Y (high)    0.300 0.366 0.066 PG  
-----
```

Related Commands

- `reset_pulse_clock_min_width`
- `set_pulse_clock_max_width`
- `set_interactive_constraint_modes`

set_resistance

```
set_resistance  
-help  
[-max]  
[-min]  
value  
net_list
```

Sets the wire resistance value on the specified nets of the top cell, which is used to compute the net delay of the specified nets. The resistance that you specify using this command overrides the resistance from wire load models and SPEF.

 In the best-case worst-case (BC-WC) analysis mode, the minimum resistance is used to compute the early path delay and the late path delay for the minimum corner. The maximum resistance is used to compute the early path delay and the late path delay for the maximum corner.

Parameters

<code>-help</code>	Outputs a brief description that includes type and default information for each <code>set_resistance</code> parameter. For a detailed description of the command and all of its parameters, use the <code>man</code> command: <code>man set_resistance</code> .
<code>-max</code>	Specifies that the asserted resistance is the maximum resistance. <i>Default:</i> If the <code>-max</code> (or <code>-min</code>) parameter is not specified, the resistance value is used as both minimum and maximum resistance.
<code>-min</code>	Specifies that the asserted resistance is the minimum resistance. <i>Default:</i> If the <code>-min</code> (or <code>-max</code>) parameter is not specified, the resistance value is used as both minimum and maximum resistance.
<code>net_list</code>	Specifies a list of nets on which the wire resistance is set.
<code>resistance_value</code>	Specifies the value for the wire resistance. This value (in units) is found in the target technology library.

Examples

- The following example set the maximum wire resistance of 2.0 units on the net n1:

```
% set_resistance -max 2.0 [get_nets n1]
```

- The following example set the minimum wire resistance of 1.2 units and a maximum wire resistance of 2.0 units on the net n1:

```
% set_resistance 1.2 -min [get_nets n1]  
% set_resistance 2.0 -max [get_nets n1]
```

Related Commands

- [reset_resistance](#)
- [report_timing](#)
- [set_drive](#)
- [set_load](#)

set_sense

```
set_sense
[-help]
pin_or_port_list
[-clocks clock_list]
{[-positive | -negative | -stop_propagation | -clock_leaf]
[-type type_list]}
```

Selects which phase of the clock to filter at the specified point.

This command accepts timing arcs collection as a valid argument.

Parameters

-help	Prints out the command usage.
-clocks clock_list	Specifies a list of clocks passing through a specified point to which the constraint applies. When this parameter is not specified, the filtering of clock phase will be applicable to all the clocks passing through that specified point. This parameter must be used with a specified pin name and one of the parameters : -positive, -negative, -stop_propagation, Or -clock_leaf.
-clock_leaf	When specified, the software propagates clock phases to the specified pin.
-negative	Specifies that positive phase filtered clocks pass through pins in the <i>pin_list</i>. Note: The -positive, -negative and -stop_propagation parameters are mutually exclusive.
pin_or_port_list	Specifies the list or collection of cell output pins. The unateness specified by the set_sense command is applied to clocks propagating through the specified point.
-positive	Specifies that negative phase filtered clocks pass through pins in the <i>pin_list</i>. Note: The -positive, -negative and -stop_propagation parameters are mutually exclusive.
- stop_propagation	Stops the propagation of the specified clocks passing through the specified pins, or any phases that are specified by the -type parameter. Note: The -positive, -negative and -stop_propagation parameters are mutually exclusive.

`-type type_list`

Applies clock sense to object types specified in the `type_list`. The type list options are `-all`, `clock`, `data`, and `clock_source_data`.

These are explained below:

- `all`: Stops all phases from propagating. This is equivalent to specifying both the `clock` and `data` options.
- `clock`: Stops clock phases launched by clock(s) specified in the `clock_list` from propagating.
- `data`: Stops all data phases launched by clock(s) specified in `clock_list`, starting from the pins/ports specified in `pin_or_port_list`. The objects in the `pin_or_port_list` can be selected anywhere in the design - clock/data network.
- `clock_source_data`: Stops only clock source data phases from propagating.

Notes:

- The `-type clock` option should be used with the `-stop_propagation` parameter. When the `-type` parameter is not specified, then `set_sense -stop_propagation` will stop only the clock phases.
- The `-type data` option should be used with the `-stop_propagation` and `-clock` parameters.
- The `-type data` option is not supported with both `-positive` and `-negative` parameters.
- The `write_sdc` command will automatically add `-type clock` when `set_sense` is specified with the `-stop_propagation` parameter (without `-type`).

Examples

- The following command allows the positive phase clock CLK1 to pass through CLKBUF1/Z and filters out the negative phase clock CLK1:

```
Litmus > set_sense -positive -clocks CLK1 CLKBUF1/Z
```

- The following command stops clock phases starting from pin1:

```
Litmus > set_sense pin1 -type {clock}
```

- The following command stops data phases based only on CK1 and CK2 starting from pin1:

```
Litmus > set_sense pin1 -type {data} -clocks {CK1 CK2}
```

- The following command stops only the clock source data phases starting from pin1:

```
Litmus > set_sense pin1 -type {clock_source_data}
```

- The following command stops the clock and clock source data phases:

```
Litmus > set_sense pin1 -type {clock clock_source_data}
```

or

```
Litmus > set_sense pin1 -stop_propagation
```

Related Commands

- [reset_sense](#)

set_target_check_constraints

```
set_target_check_constraints  
[-help]  
[-commit]  
[-files <string>]  
[-instances <string>]  
[-module <string>]  
[-ref_view <string>]
```

Parameters

-help	Prints out the command usage
-commit	Indicates that all target constraints have been specified and should be read. (Boolean flag, optional)
-files <string>	Specifies the constraint files to be checked. (String, optional)
-instances <string>	Names the instances into which the constraints should be read. Enables hierarchical constraint checking.
-module <string>	Specifies the module name for applying constraints. If the module is not the top cell, hierarchical constraint checking is performed. (String, optional)
-ref_view <string>	Specifies the view used for reference constraints. (<i>String, optional</i>)

Timing Constraint Hierarchical Signoff Commands

- [report_hierarchical_clock_mapping](#)
- [delete_hierarchical_clock_mapping](#)
- [check_target_constraints](#)
- [read_hierarchical_constraints](#)

report_hierarchical_clock_mapping

```
report_hierarchical_clock_mapping  
[-help]  
  
[-unmapped_clocks]  
  
[-top_clock]  
  
[-block_clock]  
  
[-clock_path]  
  
[-data_path]
```

Reports clock mapping for the top clocks and the block clocks in the hierarchical signoff flow.

Parameters

-help	Prints the command usage.
-unmapped_clocks	Reports the clock mapping for the unmapped top clocks and block clocks.
-top_clock	Specifies the name of top-level clock.
-block_clock	Specifies the name of block-level clock
-clock_path	Specifies the clock paths.
-data_path	Specifies the data paths.
-scope	Specifies the hierarchical instance name.

Command Order

Use this command after design initialization ([init_design](#)).

Examples

The following command reports clock mapping for all the top clocks and block clocks:

```
% report_hierarchical_clock_mapping
```

Related Commands

- [delete_hierarchical_clock_mapping](#)

delete_hierarchical_clock_mapping

delete_hierarchical_clock_mapping

[-help]

[-top_clock]

[-block_clock]

[-instance]

[-reset]

Deletes clock mapping for the top clocks and the block clocks in the hierarchical signoff flow.

Parameters

-help	Prints the command usage.
-top_clock	Specifies the name of top-level clock.
-block_clock	Specifies the name of block-level clock.
-reset	Resets the clock mapping.
-instance	Specifies the hierarchical instance name.

Command Order

Use this command after design initialization ([init_design](#)).

Examples

The following command deletes the clock mapping for all the top clocks and block clocks:

```
% delete_hierarchical_clock_mapping
```

Related Commands

- [report_hierarchical_clock_mapping](#)

check_target_constraints

```
check_target_constraints
[-help]
[-instances <string>]
[-modules <string>]
[-types <string>]
[> <file>]
[>> <file>]
```

Performs timing constraint hierarchical signoff analysis, which checks the rules in the category of Constraints Hierarchical Signoff.

Parameters

-help	Prints out the command usage
-instances <string>	Specifies the instances to be checked. (Optional)
-modules <string>	Specifies the modules to be checked. (Optional)
-types <string>	Specifies the constraint type list to be checked. (Optional)
> <file>	Specifies the output file to which the results should be written. (Optional)
>> <file>	Specifies the output file to which the results should be appended. (Optional)

Command Order

Use this command after design initialization ([init_design](#)).

Related Commands

- [init_design](#)

read_hierarchical_constraints

```
read_hierarchical_constraints  
[-help]  
-top_sdc <string>  
| { -block_sdc <string> { -block <string> | -instances <string> } }
```

Loads top-level and block-level constraint files, which associates block or instance-specific SDC files with the SoC context.

Parameters

-help	Prints usage information.
-top_sdc <string>	Specifies top-level SDCs. These must be loaded separately from block-level SDCs.
-block <string>	Specifies the module name when applying the block-level SDC to all instances of that module.
-block_sdc <string>	Specifies the block-level SDC file(s) to register for promotion.
-instances <string>	Associates the block-level SDC with one or more explicit instances.

Example

Load block SDC for a specific instance

```
read_hierarchical_constraints -block_sdc module_a.sdc -instance instance_a
```

Load block SDC for all instances of a module

```
read_hierarchical_constraints -block_sdc module_a.sdc -block module_a
```

Load multiple block constraints (mixed module-based and instance-based)

```
read_hierarchical_constraints -block_sdc module_a.sdc -block module_a
read_hierarchical_constraints -block_sdc module_b.sdc -instance instance_b
read_hierarchical_constraints -block_sdc module_c.sdc -instance instance_c
```

Note:

- Top-level and block-level SDCs cannot be loaded in a **single command call**. Use multiple commands and load the SoC constraints first using `-top_sdc`, then load block constraints via `-block_sdc`.
- When a module has **multiple instances**, the `-block` option allows the command to automatically associate the same block SDC with all corresponding instances.
- The `read_hierarchical_constraints` command does not load or apply constraints into the timing database immediately. Instead, it registers and validates the referenced SDC files, and performs checks such as file existence and syntax, so that they can be applied later during the `promote_hierarchical_constraints` step.

Related Commands

- [init_design](#)

Timing Constraint Signoff Commands

- [verify_timing_intent](#)

verify_timing_intent

```
verify_timing_intent  
[-help]  
[-force]
```

Performs timing constraint signoff analysis, which checks the rules in the category of `CONSTRAINT_SIGNOFF`.

Parameters

<code>-help</code>	Prints out the command usage.
<code>-force</code>	Forces timing constraint signoff analysis to perform.

Command Order

Use this command after design initialization ([init_design](#)).

Examples

- The following command performs timing constraint signoff analysis:

```
% read_libs cell.lib  
% read_netlist -top top top.v  
% read_sdc top.sdc  
% init_design  
% verify_timing_intent  
% report_rule_check -category CONSTRAINT_SIGNOFF
```

Related Commands

- `init_design`

